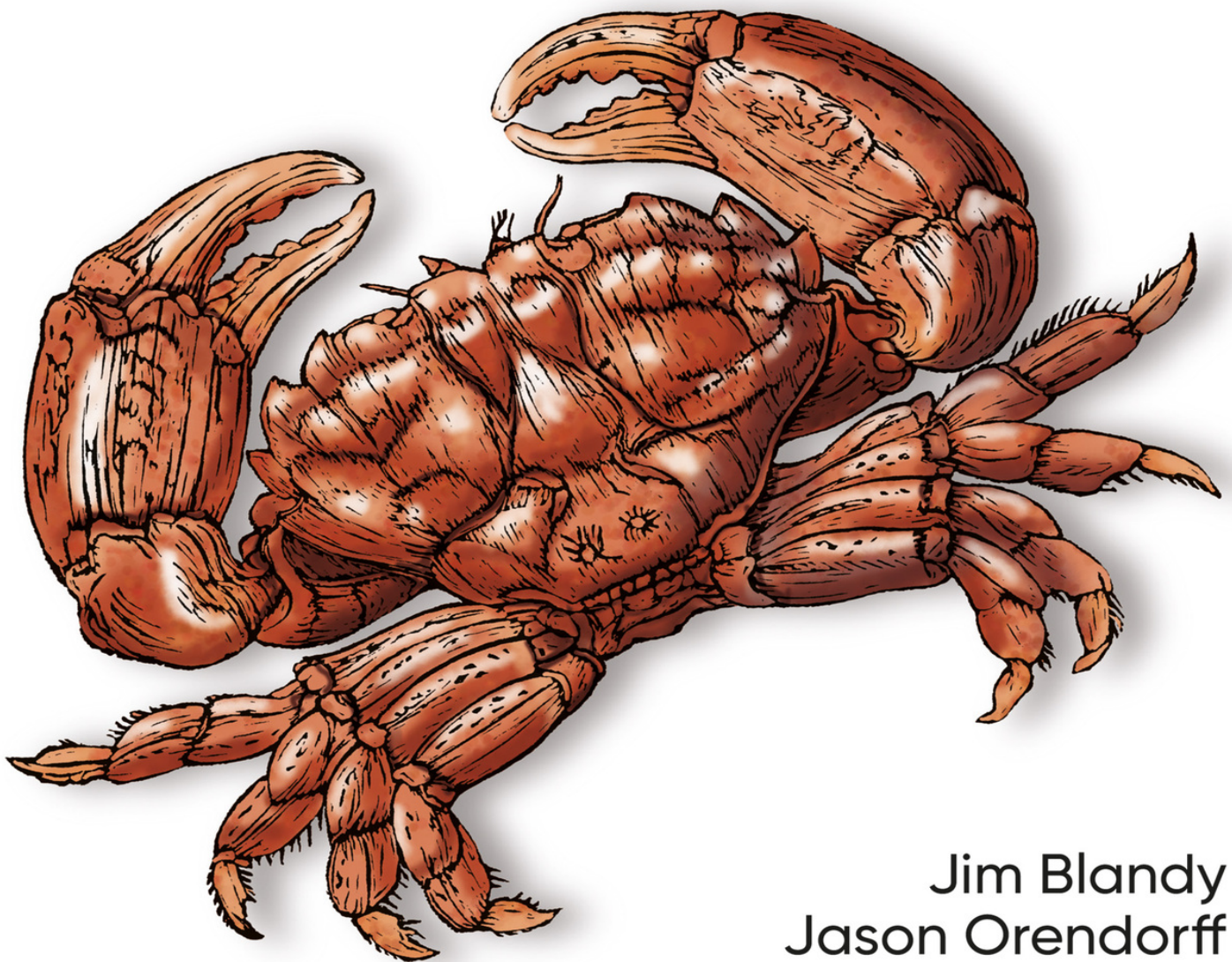


O'REILLY®

2ª Edição
Aborda Rust 2021

Programação em Rust

Desenvolvimento de sistemas rápidos e seguros



novatec

Jim Blandy
Jason Orendorff
Leonora F. S. Tindall

Programação em Rust

Desenvolvimento de sistemas rápidos e seguros

SEGUNDA EDIÇÃO REVISADA — ABORDA RUST EDIÇÃO
2021

Jim Blandy
Jason Orendorff
Leonora F. S. Tindall

O'REILLY®
Novatec

Authorized Portuguese translation of the English edition of Programming Rust, 2E ISBN 9781492052593 © 2021 Jim Blandy, Leonora F. S. Tindall, Jason Orendorff. This translation is published and sold by permission of O'Reilly Media, Inc., the owner of all rights to publish and sell the same.

Tradução em português autorizada da edição em inglês da obra Programming Rust, 2E ISBN 9781492052593 © 2021 Jim Blandy, Leonora F. S. Tindall, Jason Orendorff. Esta tradução é publicada e vendida com a permissão da O'Reilly Media, Inc., detentora de todos os direitos para publicação e venda desta obra.

© Novatec Editora Ltda. [2023].

Todos os direitos reservados e protegidos pela Lei 9.610 de 19/02/1998. É proibida a reprodução desta obra, mesmo parcial, por qualquer processo, sem prévia autorização, por escrito, do autor e da Editora.

Editor: Rubens Prates BAR20230720

Tradução: Edson Furmankiewicz

Revisão da tradução: Nilo Ney Coutinho Menezes

Revisão gramatical: Tássia Carvalho

ISBN do impresso: 978-85-7522-860-9

ISBN do ebook: 978-85-7522-861-6

Histórico de impressões:

Agosto/2023 Primeira edição

Novatec Editora Ltda.

Rua Luís Antônio dos Santos 110

02460-000 – São Paulo, SP – Brasil

Tel.: +55 11 2959-6529

Email: novatec@novatec.com.br

Site: <https://novatec.com.br>

Twitter: twitter.com/novateceditora

Facebook: facebook.com/novatec

LinkedIn: linkedin.com/company/novatec-editora/
BAR20230720

Sumário

Prefácio

Capítulo 1 Os programadores de sistemas podem ter coisas legais

O Rust carrega o peso para você
A programação paralela foi domada
E, mesmo assim, o Rust ainda é rápido
O Rust facilita a colaboração

Capítulo 2 Um passeio pelo Rust

rustup e Cargo
Funções no Rust
Escrevendo e executando testes unitários
Manipulando argumentos de linha de comando
Servindo páginas para a web
Concorrência
O que é realmente o conjunto de Mandelbrot
Analizando argumentos de linha de comando em pares
Mapeando pixels para números complexos
Plotando o conjunto
Gravando arquivos de imagem
Um programa de Mandelbrot concorrente
Executando o plotter de Mandelbrot
A segurança é invisível
Sistemas de arquivos e ferramentas de linha de comando
Interface de linha de comando
Lendo e gravando arquivos
Localizar e substituir

Capítulo 3 Tipos fundamentais

Tipos numéricos de tamanho fixo
Tipos inteiros
Aritmética: verificação, enquadramento, saturação e estouro

- [Tipos de ponto flutuante](#)
- [Tipo bool](#)
- [Caracteres](#)
- [Tuplas](#)
- [Tipos de ponteiro](#)
 - [Referências](#)
 - [Boxes](#)
 - [Ponteiros brutos](#)
- [Arrays, vetores e fatias](#)
 - [Arrays](#)
 - [Vetores](#)
 - [Fatias](#)
- [Tipos de string](#)
 - [Literais de string](#)
 - [Strings de bytes](#)
 - [Strings na memória](#)
 - [String](#)
 - [Usando strings](#)
 - [Outros tipos semelhantes a strings](#)
- [Aliases de tipo](#)
- [Além do básico](#)

Capítulo 4 Posse e movimentos

- [Posse \(ownership\)](#)
- [Movimentos](#)
 - [Mais operações que movem](#)
 - [Movimentos e fluxo de controle](#)
 - [Movimentos e conteúdo indexado](#)
- [Tipos de cópia: A exceção aos movimentos](#)
- [Rc e Arc: Posse compartilhada](#)

Capítulo 5 Referências

- [Referências a valores](#)
- [Trabalhando com referências](#)
 - [Referências Rust versus referências C++](#)
 - [Atribuição de referências](#)
 - [Referências a referências](#)

[Comparando referências](#)
[Referências nunca são nulas](#)
[Emprestando referências a expressões arbitrárias](#)
[Referências a fatias e objetos trait](#)
[Segurança de referência](#)
[Emprestando uma variável local](#)
[Recebendo referências como argumentos de função](#)
[Passando referências para funções](#)
[Retornando referências](#)
[Structs contendo referências](#)
[Parâmetros de tempo de vida distintos](#)
[Omitindo parâmetros de tempo de vida](#)
[Compartilhamento versus mutação](#)
[Pegando em armas contra um mar de objetos](#)

Capítulo 6 Expressões

[Uma linguagem de expressão](#)
[Precedência e associatividade](#)
[Blocos e pontos e vírgulas](#)
[Declarações](#)
[if e match](#)
[if let](#)
[Loops](#)
[Fluxo de controle em loops](#)
[Expressões return](#)
[Por que o Rust tem loops](#)
[Chamadas de função e método](#)
[Campos e elementos](#)
[Operadores de referência](#)
[Operadores aritméticos, de bit a bit, de comparação e lógicos](#)
[Atribuição](#)
[Coerções de tipo](#)
[Closures](#)
[A seguir](#)

Capítulo 7 Tratamento de erros

[Pânico](#)

[Desempilhamento](#)

[Abortando](#)

[Result](#)

[Capturando erros](#)

[Aliases do tipo Result](#)

[Imprimindo erros](#)

[Propagando erros](#)

[Trabalhando com vários tipos de erro](#)

[Lidando com erros que “não podem acontecer”](#)

[Ignorando erros](#)

[Tratando erros em main\(\)](#)

[Declarando um tipo de erro personalizado](#)

[Por que Results?](#)

Capítulo 8 Crates e módulos

[Crates](#)

[Edições](#)

[Perfis de compilação](#)

[Módulos](#)

[Módulos aninhados](#)

[Módulos em arquivos separados](#)

[Caminhos e importações](#)

[Prelúdio padrão](#)

[Criando declarações use públicas](#)

[Criando campos struct públicos](#)

[Variáveis estáticas e constantes](#)

[Transformando um programa em uma biblioteca](#)

[Diretório src/bin](#)

[Atributos](#)

[Testes e documentação](#)

[Testes de integração](#)

[Documentação](#)

[Doc-Tests](#)

[Especificando dependências](#)

[Versões](#)

[Cargo.lock](#)

[Publicando crates em crates.io](#)

[Espaços de trabalho](#)

[Mais coisas legais](#)

[Capítulo 9 Structs](#)

[Structs de campo nomeado](#)

[Structs do tipo tupla](#)

[Structs do tipo unidade](#)

[Disposição de um struct na memória](#)

[Definindo métodos com impl](#)

[Passando Self como um Box, Rc ou Arc](#)

[Funções associadas a tipos](#)

[Consts associadas](#)

[Structs genéricos](#)

[Structs genéricos com parâmetros de tempo de vida](#)

[Structs genéricos com parâmetros constantes](#)

[Derivando traits comuns para tipos de struct](#)

[Mutabilidade interior](#)

[Capítulo 10 Enums e padrões](#)

[Enums](#)

[Enums com dados](#)

[Enums na memória](#)

[Estruturas de dados ricas usando enums](#)

[Enums genéricos](#)

[Padrões](#)

[Literais, variáveis e curingas em padrões](#)

[Padrões com tupla e struct](#)

[Padrões de array e fatia](#)

[Padrões de referência](#)

[Guardas de correspondência](#)

[Combinando múltiplas possibilidades](#)

[Vinculando com padrões @](#)

[Onde os padrões são permitidos](#)

[Preenchendo uma árvore binária](#)

[Visão geral](#)

Capítulo 11 Traits e genéricos

Utilizando traits

Objetos trait

Funções genéricas e parâmetros de tipo

Qual utilizar

Definindo e implementando traits

Métodos padrão

Traits e tipos de outras pessoas

Self em traits

Subtraits

Funções associadas a tipos

Chamadas de método totalmente qualificadas

Traits que definem relacionamentos entre tipos

Tipos associados (ou como funcionam os iteradores)

Traits genéricos (ou como funciona a sobrecarga do operador)

impl Trait

Consts associadas

Engenharia reversa de limites

Traits como uma fundação

Capítulo 12 Sobrecarga de operador

Operadores aritméticos e de bit a bit

Operadores unários

Operadores binários

Operadores de atribuição composta

Comparações de equivalência

Comparações ordenadas

Index e IndexMut

Outros operadores

Capítulo 13 Traits utilitários

Drop

Sized

Clone

Copy

Deref e DerefMut

[Default](#)

[AsRef e AsMut](#)

[Borrow e BorrowMut](#)

[From e Into](#)

[TryFrom e TryInto](#)

[ToOwned](#)

[Borrow e ToOwned em funcionamento: Cow \("clone on write"\)](#)

Capítulo 14 Closures

[Capturando variáveis](#)

[Closures que pedem emprestado](#)

[Closures que roubam](#)

[Função e tipos de closure](#)

[Desempenho de closure](#)

[Closures e segurança](#)

[Closures que matam](#)

[FnOnce](#)

[FnMut](#)

[Copy e Clone para closures](#)

[Callbacks](#)

[Usando closures de forma eficaz](#)

Capítulo 15 Iteradores

[Traits Iterator e IntoIterator](#)

[Criando iteradores](#)

[Métodos iter e iter_mut](#)

[Implementações de IntoIterator](#)

[from fn e sucessores](#)

[Métodos drain](#)

[Outras fontes do iterador](#)

[Adaptadores iteradores](#)

[map e filter](#)

[filter_map e flat_map](#)

[flatten](#)

[take e take_while](#)

[skip e skip_while](#)

[peekable](#)

[fuse](#)

[Iteradores reversíveis e rev](#)

[inspect](#)

[chain](#)

[enumerate](#)

[zip](#)

[by_ref](#)

[cloned, copied](#)

[cycle](#)

[Consumindo iteradores](#)

[Acumulação simples: contagem, soma, produto](#)

[max, min](#)

[max_by, min_by](#)

[max_by_key, min_by_key](#)

[Comparando sequências de itens](#)

[any e all](#)

[position, rposition e ExactSizeIterator](#)

[fold e rfold](#)

[try_fold e try_rfold](#)

[nth, nth_back](#)

[last](#)

[find, rfind e find_map](#)

[Construindo coleções: collect e FromIterator](#)

[Trait Extend](#)

[partition](#)

[for_each e try_for_each](#)

[Implementando seus próprios iteradores](#)

[Capítulo 16 Coleções](#)

[Visão geral](#)

[Vec<T>](#)

[Acessando elementos](#)

[Iteração](#)

[Crescendo e encolhendo vetores](#)

[Unindo](#)

[Dividindo](#)

- [Permutando](#)
- [Preenchimento](#)
- [Ordenando e pesquisando](#)
- [Comparando fatias](#)
- [Elementos aleatórios](#)
- [O Rust descarta erros de invalidação](#)
- [VecDeque<T>](#)
- [BinaryHeap<T>](#)
- [HashMap<K, V> e BTreeMap<K, V>](#)
- [Entradas](#)
- [Iteração em mapas](#)
- [HashSet<T> e BTreeSet<T>](#)
- [Iteração em conjuntos](#)
- [Quando valores iguais são diferentes](#)
- [Operações sobre o conjunto inteiro](#)
- [Hash](#)
- [Usando um algoritmo de hash personalizado](#)
- [Além das coleções padrão](#)

Capítulo 17 Strings e texto

- [Princípios básicos de Unicode](#)
- [ASCII, Latin-1 e Unicode](#)
- [UTF-8](#)
- [Direcionalidade do texto](#)
- [Caracteres \(char\)](#)
- [Classificando caracteres](#)
- [Manipulando dígitos](#)
- [Conversão de maiúsculas e minúsculas para caracteres](#)
- [Conversões de e para inteiros](#)
- [String e str](#)
- [Criando valores string](#)
- [Inspeção simples](#)
- [Acrescentando e inserindo texto](#)
- [Removendo e substituindo texto](#)
- [Convenções para pesquisa e iteração](#)
- [Padrões para pesquisar texto](#)

[Pesquisando e substituindo](#)

[Iterando por texto](#)

[Recortando uma string](#)

[Conversão de maiúsculas e minúsculas para strings](#)

[Convertendo outros tipos a partir de strings](#)

[Convertendo outros tipos em strings](#)

[Emprestando como outros tipos semelhantes a texto](#)

[Acessando texto como UTF-8](#)

[Produzindo texto a partir de dados UTF-8](#)

[Adiando a alocação](#)

[Strings como coleções genéricas](#)

[Formatando valores](#)

[Formatando valores de texto](#)

[Formatando números](#)

[Formatando outros tipos](#)

[Valores de formatação para depuração](#)

[Ponteiros de formatação para depuração](#)

[Referindo-se a argumentos por índice ou nome](#)

[Larguras dinâmicas e precisões](#)

[Formatando os próprios tipos](#)

[Usando a linguagem de formatação em seu próprio código](#)

[Expressões regulares](#)

[Uso básico de Regex](#)

[Construindo valores regex preguiçosamente \(lazily\)](#)

[Normalização](#)

[Formas de normalização](#)

[Crate de normalização Unicode](#)

Capítulo 18 Entrada e saída

[Leitores e escritores](#)

[Leitores](#)

[Leitores bufferizados](#)

[Lendo linhas](#)

[Coletando linhas](#)

[Escritores](#)

[Arquivos](#)

[Posicionando](#)

[Outros tipos de leitores e escritores](#)

[Dados binários, compactação e serialização](#)

[Arquivos e diretórios](#)

[OsStr e Path](#)

[Métodos Path e PathBuf](#)

[Funções de acesso ao sistema de arquivos](#)

[Lendo diretórios](#)

[Recursos específicos da plataforma](#)

[Redes](#)

Capítulo 19 Concorrência

[Paralelismo fork-join](#)

[spawn e join](#)

[Tratamento de erros entre threads](#)

[Compartilhamento de dados imutáveis entre threads](#)

[Rayon](#)

[Revisitando o conjunto de Mandelbrot](#)

[Canais](#)

[Enviando valores](#)

[Recebendo valores](#)

[Executando o pipeline](#)

[Recursos e desempenho do canal](#)

[Segurança de thread: Send e Sync](#)

[Utilizando pipes em quase qualquer iterador para um canal](#)

[Além de pipelines](#)

[Estado mutável compartilhado](#)

[O que é um mutex?](#)

[Mutex<T>](#)

[mut e Mutex](#)

[Por que mutexes nem sempre são uma boa ideia](#)

[Deadlock](#)

[Mutexes envenenados](#)

[Canais multiconsumidores usando mutexes](#)

[Bloqueios de leitura/escrita \(RwLock<T>\)](#)

[Variáveis de condição \(Condvar\)](#)

[Atômicos](#)

[Variáveis globais](#)

[Como é hackear código concorrente em Rust](#)

[Capítulo 20 Programação assíncrona](#)

[De síncrono a assíncrono](#)

[Futuros](#)

[Funções assíncronas e expressões await](#)

[Chamando funções assíncronas a partir do código síncrono:](#)

[block on](#)

[Gerando tarefas assíncronas](#)

[Blocos assíncronos](#)

[Construindo funções assíncronas a partir de blocos assíncronos](#)

[Gerando tarefas assíncronas em um pool de threads](#)

[Mas seu futuro implementa Send?](#)

[Cálculos de longa duração: yield now e spawn blocking](#)

[Comparando projetos assíncronos](#)

[Um cliente HTTP assíncrono real](#)

[Um cliente e servidor assíncronos](#)

[Tipos de erro e resultado](#)

[O protocolo](#)

[Recebendo a entrada do usuário: Fluxos assíncronos](#)

[Enviando pacotes](#)

[Recebendo pacotes: mais fluxos assíncronos](#)

[A função main do cliente](#)

[A função main do servidor](#)

[Lidando com conexões de bate-papo: Mutexes assíncronos](#)

[A tabela de grupo: Mutexes síncronos](#)

[Grupos de bate-papo: canais de transmissão do tokio](#)

[Primitivas de futuro e executores: Quando vale a pena verificar um futuro novamente?](#)

[Invocando wakers: spawn blocking](#)

[Implementando block on](#)

[Fixando](#)

[As duas fases da vida de um futuro](#)

[Ponteiros fixados](#)

[Trait Unpin](#)

[Quando o código assíncrono é útil?](#)

[Capítulo 21 Macros](#)

[Noções básicas de macro](#)

[Noções básicas de expansão de macro](#)

[Consequências não intencionais](#)

[Repetição](#)

[Macros internas](#)

[Depurando macros](#)

[Construindo a macro json!](#)

[Tipos de fragmento](#)

[Recursão em macros](#)

[Usando traits com macros](#)

[Escopo e higiene](#)

[Importando e exportando macros](#)

[Evitando erros de sintaxe durante a correspondência](#)

[Para além de macro rules!](#)

[Capítulo 22 Código inseguro](#)

[Inseguro de quê?](#)

[Blocos inseguros](#)

[Exemplo: Um tipo de string ASCII eficiente](#)

[Funções inseguras](#)

[Bloco inseguro ou função insegura?](#)

[Comportamento indefinido](#)

[Traits inseguros](#)

[Ponteiros brutos](#)

[Desreferenciando ponteiros brutos com segurança](#)

[Exemplo: RefWithFlag](#)

[Ponteiros que podem ser nulos](#)

[Tamanhos e alinhamentos de tipos](#)

[Aritmética de ponteiro](#)

[Movendo dados para dentro e para fora da memória](#)

[Exemplo: GapBufferGenericName](#)

[Segurança contra pânico em código inseguro](#)

[Reinterpretando a memória com uniões](#)

[Correspondendo uniões](#)

[Pegando emprestadas uniões](#)

[Capítulo 23 Funções externas](#)

[Encontrando representações de dados comuns](#)

[Declarando funções e variáveis externas](#)

[Usando funções de bibliotecas](#)

[Uma interface bruta para libgit2](#)

[Uma interface segura para libgit2](#)

[Conclusão](#)

Prefácio

Rust é uma linguagem para programação de sistemas.

Isso tem alguma explicação hoje em dia, já que a programação de sistemas não é familiar para a maioria dos programadores profissionais. Mas isso está subjacente a tudo o que fazemos.

Você fecha o laptop. O sistema operacional detecta isso, suspende todos os programas em execução, desliga a tela e coloca o computador em hibernação. Mais tarde, você abre o laptop: a tela e outros componentes são ligados novamente e cada programa pode continuar de onde parou. Nós tomamos isso como certo. Mas os programadores de sistemas escreveram muito código para fazer isso acontecer.

A programação de sistemas é para:

- Sistemas operacionais
- Drivers de dispositivos de todos os tipos
- Sistemas de arquivos
- Bancos de dados
- Código que roda em dispositivos muito baratos, ou dispositivos que devem ser extremamente confiáveis
- Criptografia
- Codecs de mídia (software para ler e gravar arquivos de áudio, vídeo e imagem)
- Processamento de mídia (por exemplo, reconhecimento de fala ou software de edição de fotos)
- Gerenciamento de memória (por exemplo, implementando um coletor de lixo)
- Renderização de texto (a conversão de texto e fontes em pixels)
- Implementação de linguagens de programação de alto nível (como JavaScript e Python)
- Redes

- Contêineres de virtualização e software
- Simulações científicas
- Jogos

Em suma, a programação de sistemas é um tipo de programação *com restrição de recursos*. É programação quando cada byte e cada ciclo da CPU importam.

A quantidade de código de sistema envolvida no suporte a um aplicativo básico é impressionante.

Este livro não ensinará programação de sistemas. Na verdade, ele aborda muitos detalhes do gerenciamento de memória que podem parecer desnecessariamente obscuros no início, caso você ainda não tenha feito alguma programação de sistemas por conta própria. Mas, se for um programador de sistemas experiente, descobrirá que o Rust é algo excepcional: uma nova ferramenta que elimina problemas importantes e bem compreendidos que atormentam toda uma indústria há décadas.

A quem se destina este livro

Se você já é um programador de sistemas e está pronto para uma alternativa ao C++, este livro é para você. Se é um desenvolvedor experiente em qualquer linguagem de programação, seja C#, Java, Python, JavaScript ou qualquer outra, este livro também é para você.

Contudo, não precisa apenas aprender Rust. Para tirar o máximo proveito da linguagem, você também precisa ganhar alguma experiência com programação de sistemas. Recomendamos a leitura deste livro ao mesmo tempo que implementa alguns projetos paralelos de programação de sistemas em Rust. Construa algo que você nunca construiu antes, algo que aproveite a velocidade, concorrência e segurança do Rust. A lista de tópicos no início deste prefácio deve lhe dar algumas ideias.

Por que escrevemos este livro

Decidimos escrever o livro que gostaríamos de ter quando começamos a aprender Rust. Nosso objetivo era apresentar os grandes e novos conceitos do Rust de frente, de forma clara e profunda, para minimizar o aprendizado por tentativa e erro.

Navegando por este livro

Os dois primeiros capítulos deste livro apresentam o Rust e fornecem um breve tour antes de passarmos para os tipos de dados fundamentais no Capítulo 3. Os capítulos 4 e 5 abordam os conceitos básicos de posse e referências. Recomendamos a leitura dos primeiros cinco capítulos na ordem.

Os capítulos de 6 a 10 abordam os fundamentos da linguagem: expressões (Capítulo 6), tratamento de erros (Capítulo 7), crates e módulos (Capítulo 8), structs (Capítulo 9) e enums e padrões (Capítulo 10). Não há problema em pular algo aqui e ali, mas não pule o capítulo sobre tratamento de erros. Confie em nós.

O Capítulo 11 aborda traits e genéricos, os dois últimos grandes conceitos que você precisa conhecer. Traits são como interfaces em Java ou C#. Também são a principal maneira como o Rust suporta a integração de seus tipos na própria linguagem. O Capítulo 12 mostra como os traits suportam a sobrecarga de operadores, e o Capítulo 13 aborda muitos outros traits utilitários.

Compreender traits e genéricos desbloqueia o restante do livro. Closures e iteradores, duas ferramentas importantes que você não vai querer perder, são abordados nos capítulos 14 e 15, respectivamente. Você pode ler os capítulos restantes em qualquer ordem ou simplesmente os consultar conforme necessário. Eles abordam outros recursos da linguagem: coleções (Capítulo 16), strings e texto (Capítulo 17), entrada e saída (Capítulo 18), concorrência (Capítulo 19), programação assíncrona (Capítulo 20), macros (Capítulo 21), código inseguro (Capítulo 22) e funções de chamada em outras linguagens (Capítulo 23).

Convenções utilizadas neste livro

As seguintes convenções tipográficas são utilizadas neste livro:

Itálico

Indica novos termos, URLs, endereços de e-mail, nomes de arquivo e extensões de arquivo.

Fonte monoespçada

Usado para listagens de programas, bem como dentro de parágrafos para se referir a elementos do programa, como nomes de variáveis ou funções, bancos de dados, tipos de dados, variáveis de ambiente, instruções e palavras-chave.

Fonte monoespçada em negrito

Indica comandos ou outro texto que devem ser digitados literalmente pelo usuário.

Fonte monoespçada em itálico

Indica que o texto deve ser substituído por valores fornecidos pelo usuário ou por valores determinados pelo contexto.



Este ícone significa uma observação geral.

Usando exemplos de código

Material suplementar (exemplos de código, exercícios etc.) está disponível para download em <https://github.com/ProgrammingRust>.

Este livro está aqui para ajudá-lo a realizar seu trabalho. De maneira geral, se um código de exemplo for oferecido com este livro, você poderá usá-lo em seus programas e documentação. Não precisa entrar em contato conosco para obter permissão, a menos que esteja reproduzindo uma parte significativa do código. Por exemplo, escrever um programa que usa vários blocos de código deste livro não requer permissão. Vender ou distribuir exemplos de livros da O'Reilly requer permissão. Responder a uma pergunta citando este livro e citando um código de exemplo não requer permissão. Incorporar uma quantidade significativa de código de exemplo deste livro na documentação do seu produto requer permissão.

Agradecemos, mas não exigimos, os créditos. Um crédito geralmente inclui o título, autor, editora e ISBN. Por exemplo: “*Programação em Rust, segunda edição*, por Jim Blandy, Jason Orendorff e Leonora FS Tindall (O’Reilly). Copyright 2021 Jim Blandy, Leonora FS Tindall e Jason Orendorff, 978-1-492-05259-3”.

Se você acha que seu uso de exemplos de código está fora do uso justo ou da permissão dada acima, sinta-se à vontade para nos contatar em permissions@oreilly.com.

Como entrar em contato conosco

Envie comentários e dúvidas sobre este livro para: novatec@novatec.com.br.

Temos uma página da web para este livro, na qual incluímos a lista de erratas, exemplos e qualquer outra informação adicional.

- Página da edição em português
<http://novatec.com.br/livros/programacao-em-rust-2ed/>
- Página da edição original, em inglês
<https://oreil.ly/programming-rust-2e>
- Material complementar no GitHub
<https://github.com/ProgrammingRust>

Para obter mais informações sobre livros da Novatec, acesse nosso site em:

<https://novatec.com.br>

Agradecimentos

O livro que você tem em mãos se beneficiou muito da atenção de nossos revisores técnicos oficiais: Brian Anderson, Matt Brubeck, J. David Eisenberg, Ryan Levick, Jack Moffitt, Carol Nichols e Erik Nordin; e de nossos tradutores: Hidemoto Nakada (中田 秀基) (japonês), Sr. Songfeng Li (chinês simplificado) e Adam Bochenek e Krzysztof Sawka (polonês).

Muitos outros revisores não oficiais leram os primeiros rascunhos e forneceram feedback inestimável. Gostaríamos de agradecer a Eddy

Bruel, Nick Fitzgerald, Graydon Hoare, Michael Kelly, Jeffrey Lim, Jakob Olesen, Gian-Carlo Pascutto, Larry Rabinowitz, Jaroslav Šnajdr, Joe Walker e Yoshua Wuyts pelos comentários atenciosos. Jeff Walden e Nicolas Pierron foram especialmente generosos com seu tempo, revisando quase todo o livro. Como qualquer empreendimento de programação, um livro de programação se destaca quando conta com relatórios de bugs de qualidade. Obrigado.

A Mozilla foi extremamente compreensiva com o trabalho de Jim e Jason neste projeto, embora estivesse fora de nossas responsabilidades oficiais e competisse com eles por nossa atenção. Agradecemos aos gerentes de Jim e Jason: Dave Camp, Naveed Ihsanullah, Tom Tromey e Joe Walker, pelo apoio. Eles têm uma visão ampla do que é a Mozilla; esperamos que esses resultados justifiquem a confiança que depositaram em nós.

Também gostaríamos de expressar nossa gratidão a todos na O'Reilly que ajudaram a concretizar este projeto, especialmente nossos surpreendentemente pacientes editores Jeff Bleiel e Brian MacDonald, e nosso editor de aquisições Zan McQuade.

Acima de tudo, nossos sinceros agradecimentos a nossas famílias pelo amor inabalável, entusiasmo e paciência.

CAPÍTULO 1

Os programadores de sistemas podem ter coisas legais

Em certos contextos – por exemplo, o contexto a que o Rust visa –, ser 10x ou mesmo 2x mais rápido que a concorrência é uma questão de tudo ou nada. Isso decide o destino de um sistema no mercado, tanto quanto faria no mercado de hardware.

–Graydon Hoare (<https://oreil.ly/Akgzc>)

Todos os computadores agora são paralelos... Programação paralela é programação.

– Michael McCool e outros, *Structured Parallel Programming*

Falha do analisador de fontes TrueType explorada por um invasor de outro estado-nação visando à espionagem; todo software é sensível à segurança.

– Andy Wingo (<https://oreil.ly/7dnHr>)

Escolhemos abrir nosso livro com as três citações acima por um motivo. Mas vamos começar com um mistério. O que o seguinte programa em C faz?

```
int main(int argc, char **argv) {  
    unsigned long a[1];  
    a[3] = 0x7fff7b36cebUL;  
    return 0;  
}
```

Esta manhã, esse programa imprimiu no laptop de Jim:

```
undef: Error: .netrc file is readable by others.  
undef: Remove password or make file unreadable by others.
```

Então ele travou. Se você o experimentar em sua máquina, pode acontecer outra coisa. O que está acontecendo aqui?

O programa é falho. O array `a` tem apenas um elemento, então utilizar `a[3]` é, de acordo com o padrão da linguagem de programação C, um *comportamento indefinido*:

Comportamento, mediante uso de uma construção de programa não portátil ou errônea, ou de dados errôneos, para os quais essa Norma (standard) Internacional não impõe requisitos.

O comportamento indefinido não tem apenas um resultado imprevisível: o padrão permite explicitamente que o programa faça absolutamente *qualquer coisa*. Em nosso caso, armazenar esse valor específico no quarto elemento desse array específico corrompe o call stack (pilha de chamadas) de função, de modo que o retorno da função `main`, em vez de sair do programa normalmente como deveria, salta para o meio do código da biblioteca C padrão para recuperar uma senha de um arquivo no diretório pessoal do usuário. Isso não parece nada bom.

C e C++ têm centenas de regras para evitar comportamento indefinido. Em geral, elas seguem o bom senso: não acesse a memória que não deveria, não deixe as operações aritméticas estourarem, não divida por zero e assim por diante. Mas o compilador não impõe essas regras; ele não tem obrigação de detectar nem mesmo violações flagrantes. De fato, o programa anterior compila sem erros ou avisos. A responsabilidade de evitar comportamentos indefinidos recai inteiramente sobre você, o programador.

Empiricamente falando, nós programadores não temos um grande histórico nesse quesito. Enquanto estudante na Universidade de Utah, o pesquisador Peng Li modificou compiladores C e C++ para fazer com que os programas traduzidos por eles informassem se executavam de maneira que pudesse ser considerada comportamento indefinido. Ele descobriu que quase todos os programas o fazem, incluindo aqueles de projetos respeitados que mantêm seu código em padrões elevados. Supor que você pode evitar comportamentos indefinidos em C e C++ é como supor que você pode ganhar uma partida de xadrez simplesmente porque conhece as regras.

A mensagem ocasional estranha ou falha pode ser um problema de qualidade, mas o comportamento indefinido inadvertido também tem sido uma das principais causas de falhas de segurança desde que o vírus (worm) Morris de 1988 usou uma variação da técnica mostrada anteriormente para se propagar de um computador para outro no início da internet.

Portanto, C e C++ colocam os programadores em uma posição difícil: essas linguagens são os padrões da indústria para programação de sistemas, mas as demandas que impõem aos programadores praticamente garantem um fluxo constante de travamentos e problemas de segurança. Responder ao nosso mistério apenas levanta uma questão maior: não podemos fazer melhor?

O Rust carrega o peso para você

Nossa resposta é ilustrada por nossas três citações de abertura. A terceira citação refere-se a relatos de que o Stuxnet, um worm de computador encontrado invadindo equipamentos de controle industrial em 2010, obteve o controle dos computadores das vítimas utilizando, entre muitas outras técnicas, um comportamento indefinido em um código que analisava fontes TrueType incorporadas em documentos de processamento de texto. É seguro apostar que os autores desse código não esperavam que ele fosse utilizado dessa maneira, ilustrando que não são apenas sistemas operacionais e servidores que precisam se preocupar com a segurança: qualquer software que lide com dados de uma fonte não confiável pode ser o alvo de um exploit.

A linguagem Rust faz uma promessa simples: se o seu programa passar nas verificações do compilador, ele estará livre de comportamento indefinido. Ponteiros perdidos (*dangling pointers*), liberações duplas (*double free*) e de referências de ponteiro nulo são capturados em tempo de compilação. As referências de array são protegidas com uma mistura de verificações em tempo de compilação e em tempo de execução, portanto não há estouros de

buffer: o equivalente no Rust do nosso desafortunado programa em C é finalizado de maneira segura e elegante com uma mensagem de erro.

Além disso, o Rust pretende ser tanto *seguro* como *agradável de usar*. Para obter garantias mais fortes sobre o comportamento do seu programa, o Rust impõe mais restrições ao seu código do que C e C++, e essas restrições exigem prática e experiência para se acostumar. Mas, no geral, a linguagem é flexível e expressiva. Isso é atestado pela amplitude dos códigos escritos no Rust e pela variedade de áreas em que está sendo aplicado.

Com base na nossa experiência, ser capaz de confiar na linguagem para detectar mais erros nos encoraja a tentar projetos mais ambiciosos. A modificação de programas grandes e complexos é menos arriscada quando você sabe que os problemas de gerenciamento de memória e validade de ponteiros estão resolvidos. E a depuração é muito mais simples quando as possíveis consequências de um bug não incluem a corrupção de partes não relacionadas do seu programa.

Claro, ainda existem muitos bugs que o Rust não consegue detectar. Mas, na prática, retirar o comportamento indefinido da mesa muda substancialmente o caráter do desenvolvimento para melhor.

A programação paralela foi domada

A concorrência é notoriamente difícil de utilizar corretamente em C e C++. Os desenvolvedores em geral recorrem à concorrência apenas quando o código de thread único provou ser incapaz de atingir o desempenho de que precisam. Mas a segunda citação de abertura argumenta que o paralelismo é muito importante para as máquinas modernas serem tratadas como um método de último recurso.

Acontece que as mesmas restrições que garantem a segurança da memória no Rust também garantem que os programas Rust estejam livres de corridas de dados (*data races*). Você pode compartilhar dados livremente entre threads, contanto que não mudem. Os dados que mudam só podem ser acessados utilizando primitivas de

sincronização. Todas as ferramentas tradicionais de concorrência estão disponíveis: mutexes, variáveis de condição, canais, atômicos e assim por diante. O Rust simplesmente verifica se você as está utilizando corretamente.

Isso torna o Rust uma excelente linguagem para explorar as habilidades das modernas máquinas de múltiplos núcleos (cores). O ecossistema Rust oferece bibliotecas que vão além das primitivas de concorrência usuais e ajudam você a distribuir cargas complexas uniformemente entre pools de processadores, utilizar mecanismos de sincronização sem bloqueio, como Read-Copy-Update e muito mais.

E, mesmo assim, o Rust ainda é rápido

Essa, finalmente, é a nossa primeira citação de abertura. O Rust compartilha as ambições que Bjarne Stroustrup articulou para o C++ em seu artigo “Abstraction and the C++ Machine Model”:

Em geral, as implementações do C++ obedecem ao princípio da sobrecarga zero: Você não paga por aquilo que não utiliza. E ainda: Você não poderia codificar manualmente melhor aquilo que não utiliza.

A programação de sistemas geralmente se preocupa em levar a máquina aos seus limites. Para videogames, toda a máquina deve ser dedicada a criar a melhor experiência para o jogador. Para navegadores da web, a eficiência do navegador define o limite do que os autores de conteúdo podem fazer. Dentro das limitações inerentes da máquina, o máximo possível de atenção à memória e ao processador deve ser deixado para o próprio conteúdo. O mesmo princípio se aplica aos sistemas operacionais: o kernel deve disponibilizar os recursos da máquina para os programas do usuário, e não os consumir ele mesmo.

Mas quando dizemos que Rust é “rápido”, o que isso realmente significa? Pode-se escrever código lento em qualquer linguagem de uso geral. Seria mais preciso dizer que, se você estiver pronto para fazer o investimento a fim de projetar seu programa para fazer o

melhor uso dos recursos da máquina subjacente, o Rust o apoiará nesse esforço. A linguagem foi projetada com padrões eficientes e oferece a capacidade de controlar como a memória é utilizada e como a atenção do processador é gasta.

O Rust facilita a colaboração

Nós ocultamos uma quarta citação no título deste capítulo: “Os programadores de sistemas podem ter coisas boas”. Isso se refere ao suporte do Rust para compartilhamento e reutilização de código.

O gerenciador de pacotes e ferramenta de criação do Rust, o Cargo, facilita o uso de bibliotecas publicadas por outras pessoas no repositório público de pacotes do Rust, o site crates.io. Você simplesmente adiciona o nome da biblioteca e o número da versão desejada a um arquivo e o Cargo se encarrega de baixar a biblioteca, com quaisquer outras bibliotecas que ele possa utilizar, vinculando todo o conteúdo. Você pode pensar no Cargo como a resposta do Rust ao NPM ou ao RubyGems, com ênfase no gerenciamento de versão correto e compilações reproduzíveis. Existem bibliotecas Rust populares que fornecem de tudo, desde serialização pronta para uso de clientes e servidores HTTP e APIs gráficas modernas.

Indo além, a própria linguagem também foi projetada para oferecer suporte à colaboração: os traits e os genéricos do Rust permitem criar bibliotecas com interfaces flexíveis para que possam servir em muitos contextos diferentes. E a biblioteca padrão do Rust fornece um conjunto básico de tipos fundamentais que estabelecem convenções compartilhadas para casos comuns, facilitando o uso conjunto de diferentes bibliotecas.

O próximo capítulo visa provar as alegações gerais que fizemos neste capítulo, com um passeio por vários pequenos programas Rust que mostram os pontos fortes da linguagem.

CAPÍTULO 2

Um passeio pelo Rust

O Rust apresenta um desafio aos autores de um livro como este: o que confere à linguagem sua reputação não é uma característica específica e surpreendente, que podemos mostrar na primeira página, mas sim a maneira como todas as suas partes são projetadas para funcionarem juntas, sem problemas, a serviço dos objetivos que estabelecemos no capítulo anterior: programação de sistemas segura e de alto desempenho. Cada parte da linguagem tem uma razão de ser.

Portanto, em vez de abordar um recurso da linguagem de cada vez, preparamos um passeio por alguns programas pequenos, mas completos, cada um dos quais apresenta alguns recursos da linguagem, no contexto:

- Como aquecimento, temos um programa que faz um cálculo simples em seus argumentos de linha de comando, com testes unitários (*unit tests*). Isso mostra os tipos principais do Rust e introduz *traits*.
- Em seguida, construímos um servidor web. Usaremos uma biblioteca de terceiros para lidar com os detalhes do HTTP e introduzir a manipulação de strings, closures e tratamento de erros.
- Nosso terceiro programa plota um belo fractal, distribuindo a computação por vários threads para ganhar velocidade. Isso inclui um exemplo de uma função genérica, ilustra como lidar com algo como um buffer de pixels e mostra o suporte do Rust para concorrência.
- Por fim, mostramos uma ferramenta robusta de linha de comando que processa arquivos utilizando expressões regulares. Isso apresenta os recursos da biblioteca padrão do Rust para trabalhar

com arquivos, e a biblioteca de expressões regulares de terceiros mais comumente utilizada.

A promessa do Rust de evitar comportamento indefinido com impacto mínimo no desempenho influencia o design de todas as partes do sistema, desde as estruturas de dados padrão, como vetores e strings, até a maneira como os programas Rust usam bibliotecas de terceiros. Os detalhes de como isso é gerenciado são abordados ao longo do livro. Mas, por enquanto, queremos mostrar a você que o Rust é uma linguagem poderosa e agradável de utilizar. Primeiro, é claro, você precisa instalar o Rust no seu computador.

rustup e Cargo

A melhor maneira de instalar o Rust é utilizar `rustup`. Visite <https://rustup.rs> e siga as instruções.

Você pode, alternativamente, ir para o site do Rust (<https://oreil.ly/4Q2FB>) a fim de obter pacotes pré-compilados para Linux, macOS e Windows. O Rust também está incluído em algumas distribuições de sistemas operacionais. Preferimos `rustup` porque é uma ferramenta para gerenciar instalações Rust, como RVM para Ruby ou NVM para Node. Por exemplo, quando uma nova versão do Rust for lançada, você poderá atualizar apenas digitando `rustup update`.

De qualquer forma, depois de concluir a instalação, você deverá ter três novos comandos disponíveis em sua linha de comando:

```
$ cargo --version
cargo 1.56.0 (4ed5d137b 2021-10-04)
$ rustc --version
rustc 1.56.0 (09c42c458 2021-10-18)
$ rustdoc --version
rustdoc 1.56.0 (09c42c458 2021-10-18)
```

Aqui o caractere `$` é o prompt de comando; no Windows, isso seria `c:\>` ou algo semelhante. Executamos os três comandos que instalamos, pedindo a cada um que informe sua versão. Examinando cada um dos comandos, temos:

- `cargo` é o gerenciador de compilação, gerenciador de pacotes e ferramenta de uso geral do Rust. Você pode utilizar o Cargo para

iniciar um novo projeto, criar e executar seu programa e gerenciar quaisquer bibliotecas externas das quais seu código dependa.

- `rustc` é o compilador do Rust. Normalmente, deixamos o Cargo invocar o compilador para nós, mas às vezes é útil executá-lo diretamente.
- `rustdoc` é a ferramenta de documentação do Rust. Se você escrever documentação na forma apropriada, em comentários no código-fonte do seu programa, `rustdoc` pode construir HTML bem formatado a partir deles. Como ocorre com `rustc`, geralmente deixamos o Cargo executar `rustdoc` para nós.

Por conveniência, o Cargo pode criar um novo pacote Rust para nós, com alguns metadados padrão organizados adequadamente:

```
$ cargo new hello
Created binary (application) `hello` package
```

Esse comando cria um novo diretório de pacotes chamado *hello*, pronto para criar um executável de linha de comando.

Examinando o diretório de nível superior do pacote:

```
$ cd hello
$ ls -la
total 24
drwxrwxr-x. 4 jimb jimb 4096 Sep 22 21:09 .
drwx----- 62 jimb jimb 4096 Sep 22 21:09 ..
drwxrwxr-x. 6 jimb jimb 4096 Sep 22 21:09 .git
-rw-rw-r--. 1 jimb jimb  7 Sep 22 21:09 .gitignore
-rw-rw-r--. 1 jimb jimb 88 Sep 22 21:09 Cargo.toml
drwxrwxr-x. 2 jimb jimb 4096 Sep 22 21:09 src
```

Podemos ver que o Cargo criou um arquivo *Cargo.toml* a fim de armazenar metadados para o pacote. No momento, esse arquivo tem poucos dados:

```
[package]
name = "hello"
version = "0.1.0"
edition = "2021"
# Veja mais chaves e suas definições em
# https://doc.rust-lang.org/cargo/reference/manifest.html

[dependencies]
```

Se nosso programa tiver dependências de outras bibliotecas, podemos registrá-las nesse arquivo e o Cargo se encarregará de

baixar, criar e atualizar essas bibliotecas para nós. Abordaremos o arquivo *Cargo.toml* em detalhes no Capítulo 8.

O Cargo configurou nosso pacote para uso com o sistema de controle de versão `git`, criando um subdiretório *.git* de metadados e um arquivo *.gitignore*. Você pode configurar o Cargo para pular essa etapa passando `--vcs none` para `cargo new` na linha de comando.

O subdiretório *src* contém o código-fonte Rust real:

```
$ cd src
$ ls -l
total 4
-rw-rw-r--. 1 jimb jimb 45 Sep 22 21:09 main.rs
```

Parece que o Cargo começou a escrever o programa para nós. O arquivo *main.rs* contém o texto:

```
fn main() {
    println!("Hello, world!");
}
```

No Rust, você nem precisa escrever o próprio programa “Hello, World!”. É esta a extensão do código boilerplate¹ para um novo programa Rust: dois arquivos, totalizando treze linhas.

Podemos invocar o comando `cargo run` de qualquer diretório no pacote para construir e executar nosso programa:

```
$ cargo run
Compiling hello v0.1.0 (/home/jimb/rust/hello)
Finished dev [unoptimized + debuginfo] target(s) in 0.28s
Running `/home/jimb/rust/hello/target/debug/hello`
Hello, world!
```

Aqui, o Cargo invocou o compilador do Rust, `rustc` e, em seguida, rodou o executável que foi gerado. O Cargo coloca o executável no subdiretório *target* no topo do pacote:

```
$ ls -l ../target/debug
total 580
drwxrwxr-x. 2 jimb jimb 4096 Sep 22 21:37 build
drwxrwxr-x. 2 jimb jimb 4096 Sep 22 21:37 deps
drwxrwxr-x. 2 jimb jimb 4096 Sep 22 21:37 examples
-rwxrwxr-x. 1 jimb jimb 576632 Sep 22 21:37 hello
-rw-rw-r--. 1 jimb jimb 198 Sep 22 21:37 hello.d
drwxrwxr-x. 2 jimb jimb 68 Sep 22 21:37 incremental
$ ../target/debug/hello
Hello, world!
```


Quando terminarmos, o Cargo pode apagar os arquivos gerados para nós:

```
$ cargo clean
$ ../target/debug/hello
bash: ../target/debug/hello: No such file or directory
```

Funções no Rust

A sintaxe do Rust é deliberadamente trivial. Se você estiver familiarizado com C, C++, Java ou JavaScript, provavelmente poderá se localizar na estrutura geral de um programa Rust. Eis uma função que calcula o máximo divisor comum de dois números inteiros, utilizando o algoritmo de Euclides (<https://oreil.ly/DFpyb>). Você pode adicionar este código ao final de *src/main.rs*:

```
fn gcd(mut n: u64, mut m: u64) -> u64 {
    assert!(n != 0 && m != 0);
    while m != 0 {
        if m < n {
            let t = m;
            m = n;
            n = t;
        }
        m = m % n;
    }
    n
}
```

A palavra-chave `fn` (pronuncia-se “fun”) introduz uma função. Aqui, estamos definindo uma função chamada `gcd`, que recebe dois parâmetros, `n` e `m`, cada um dos quais é do tipo `u64`, um inteiro de 64 bits sem sinal. O token `->` precede o tipo de retorno: nossa função retorna um valor `u64`. O recuo de quatro espaços é o estilo padrão do Rust.

Os nomes do tipo inteiro da linguagem Rust refletem seu tamanho e sinal (com ou sem): `i32` é um inteiro de 32 bits com sinal; `u8` é um inteiro de 8 bits sem sinal (usado para valores que cabem num “byte”) e assim por diante. Os tipos `isize` e `usize` armazenam inteiros com e sem sinal de um ponteiro, com tamanho de 32 bits nas plataformas de 32 bits, e 64 bits nas plataformas de 64 bits. O Rust também tem dois tipos de ponto flutuante, `f32` e `f64`, que são os tipos

de ponto flutuante de precisão simples e dupla IEEE, como `float` e `double` em C e C++.

Por padrão, uma vez que uma variável é inicializada, seu valor não pode ser alterado, mas colocar a palavra-chave `mut` (pronuncia-se “mute”, abreviação de *mutable*) antes dos parâmetros `n` e `m` permite que o corpo da nossa função os modifique. Na prática, a maioria das variáveis não é modificada; mas, naquelas que são, a palavra-chave `mut` pode ser uma dica útil ao se ler o código.

O corpo da função começa com uma chamada à macro `assert!`, verificando se nenhum dos argumentos é zero. O caractere `!` marca isso como uma invocação de macro, e não uma chamada de função. Como ocorre com a macro `assert` em C e C++, `assert!` do Rust verifica se o seu argumento é verdadeiro e, se não for, encerra o programa com uma mensagem útil, incluindo o local de origem da falha verificada; esse tipo de término abrupto é chamado de *pânico*. Ao contrário de C e C++, nos quais as asserções podem ser ignoradas, o Rust sempre verifica as asserções, independentemente de como o programa foi compilado. Também há uma macro `debug_assert!`, cujas asserções são ignoradas quando o programa é compilado para ter maior velocidade.

O coração da nossa função é um loop `while` contendo uma instrução `if` e uma atribuição. Ao contrário de C e C++, o Rust não requer parênteses em torno das expressões condicionais, mas requer chaves em torno das instruções que elas controlam.

Uma instrução `let` declara uma variável local, como `t` em nossa função. Não precisamos escrever o tipo de `t`, desde que o Rust possa inferi-lo a partir de como a variável é utilizada. Em nossa função, o único tipo que funciona para `t` é `u64`, correspondendo a `m` e `n`. O Rust apenas infere tipos dentro do corpo de uma função: você deve escrever os tipos de parâmetro da função e seus valores de retorno, como fizemos antes. Se quiséssemos explicitar o tipo de `t`, poderíamos escrever:

```
let t: u64 = m;
```

O Rust tem uma instrução `return`, mas a função `gcd` não precisa de

uma. Se o corpo de uma função terminar com uma expressão que *não* é seguida por um ponto e vírgula, esse é o valor de retorno da função. Na verdade, qualquer bloco entre chaves pode funcionar como uma expressão. Por exemplo, esta é uma expressão que imprime uma mensagem e, em seguida, produz `x.cos()` como seu valor:

```
{  
    println!("evaluating cos x");  
    x.cos()  
}
```

É típico no Rust empregar essa forma para estabelecer o valor da função quando esta não tiver valor de retorno, e usar instruções `return` somente para retornos antecipados explícitos do meio de uma função.

Escrevendo e executando testes unitários

O Rust tem suporte simples para testes integrados na linguagem. Para testar nossa função `gcd`, podemos adicionar este código no final de *src/main.rs*:

```
#[test]  
fn test_gcd() {  
    assert_eq!(gcd(14, 15), 1);  
  
    assert_eq!(gcd(2 * 3 * 5 * 11 * 17,  
                 3 * 7 * 11 * 13 * 19),  
               3 * 11);  
}
```

Aqui definimos uma função chamada `test_gcd`, que chama `gcd` e verifica se ela retorna os valores corretos. O marcador `#[test]` na primeira linha da definição indica que `test_gcd` é uma função de teste, a ser ignorada em compilações normais, mas incluída e chamada automaticamente se executarmos nosso programa com o comando `cargo test`. Podemos ter funções de teste espalhadas por toda a árvore do nosso código-fonte, colocadas próximas do código que elas afetam, e `cargo test` vai reuni-las automaticamente e executar todas.

O marcador `#[test]` é um exemplo de *atributo*. Os atributos são um sistema aberto para marcar funções e outras declarações com

informações extras, como atributos em C++ e C# ou anotações em Java. Eles são utilizados para controlar avisos do compilador e verificações de estilo de código, incluir código condicionalmente (como `#ifdef` em C e C++), instruir o Rust a interagir com o código escrito em outras linguagens e assim por diante. Veremos mais exemplos de atributos conforme avançamos.

Com nossas definições `gcd` e `test_gcd` adicionadas ao pacote *hello* criado no início do capítulo e o nosso diretório atual em algum lugar dentro da subárvore do pacote, podemos executar os testes da seguinte maneira:

```
$ cargo test
  Compiling hello v0.1.0 (/home/jimb/rust/hello)
  Finished test [unoptimized + debuginfo] target(s) in 0.35s
  Running unittests (/home/jimb/rust/hello/target/debug/deps/hello-2375...)

running 1 test
test test_gcd ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

Manipulando argumentos de linha de comando

Para que nosso programa receba uma série de números como argumentos de linha de comando e imprima seu máximo divisor comum, podemos substituir a função `main` em *src/main.rs* pelo seguinte:

```
use std::str::FromStr;
use std::env;

fn main() {
    let mut numbers = Vec::new();

    for arg in env::args().skip(1) {
        numbers.push(u64::from_str(&arg)
            .expect("error parsing argument"));
    }

    if numbers.len() == 0 {
        eprintln!("Usage: gcd NUMBER ...");
        std::process::exit(1);
    }
}
```

```

    let mut d = numbers[0];
    for m in &numbers[1..] {
        d = gcd(d, *m);
    }

    println!("The greatest common divisor of {:?} is {}",
            numbers, d);
}

```

Esse é um grande bloco de código, então vamos analisar parte por parte:

```

use std::str::FromStr;
use std::env;

```

A primeira declaração `use` traz o `trait FromStr` da biblioteca padrão para dentro do escopo. Um `trait` é uma coleção de métodos que os tipos podem implementar. Qualquer tipo que implemente o `trait FromStr` tem um método `from_str` que tenta analisar um valor desse tipo em uma `string`. O tipo `u64` implementa `FromStr`, e vamos chamar `u64::from_str` para analisar nossos argumentos de linha de comando. Embora nunca usemos o nome `FromStr` em outra parte do programa, um `trait` deve estar no escopo para que seus métodos sejam utilizados. Abordaremos `traits` em detalhes no Capítulo 11.

A segunda declaração `use` traz o módulo `std::env`, que fornece várias funções e tipos úteis para interagir com o ambiente de execução, incluindo a função `args`, que nos dá acesso aos argumentos de linha de comando do programa.

Passando para a função `main` do programa:

```

fn main() {

```

Nossa função `main` não retorna um valor, então podemos simplesmente omitir o `->` e o tipo de retorno que normalmente seguiria a lista de parâmetros.

```

    let mut numbers = Vec::new();

```

Declaramos uma variável local mutável `numbers` e a inicializamos com um vetor vazio. `Vec` é o tipo de vetor expansível do Rust, análogo ao `std::vector` do C++, uma lista no Python ou um `array` em JavaScript. Mesmo que os vetores sejam projetados para serem aumentados e reduzidos dinamicamente, ainda devemos marcar a variável com `mut` para o Rust nos deixar inserir números no final dele.

O tipo de `numbers` é `Vec<u64>`, um vetor de valores `u64`, mas, como antes, não precisamos escrever isso. O Rust vai inferir isso para nós, em parte porque o que armazenamos no vetor são valores `u64`, e, também, porque passamos os elementos do vetor para `gcd`, que aceita apenas valores `u64`.

```
for arg in env::args().skip(1) {
```

Aqui utilizamos um loop `for` para processar nossos argumentos de linha de comando, definindo a variável `arg` para cada argumento a cada vez e avaliando o corpo do loop.

A função `args` do módulo `std::env` retorna um *iterador*, um valor que produz cada argumento sob demanda e indica quando terminamos. Os iteradores são onipresentes no Rust; a biblioteca padrão inclui outros iteradores que produzem os elementos de um vetor, as linhas de um arquivo, as mensagens recebidas em um canal de comunicação e quase qualquer outra coisa pela qual faça sentido iterar. Os iteradores do Rust são muito eficientes: o compilador geralmente é capaz de traduzi-los no mesmo código de um loop escrito manualmente. Mostraremos como isso funciona e daremos exemplos no Capítulo 15.

Além de seu uso com loops `for`, os iteradores incluem uma ampla seleção de métodos que você pode utilizar diretamente. Por exemplo, o primeiro valor gerado pelo iterador retornado por `args` é sempre o nome do programa que está sendo executado. Queremos pular isso, então chamamos o iterador `skip` para produzir um novo iterador que omite esse primeiro valor.

```
    numbers.push(u64::from_str(&arg)
        .expect("error parsing argument"));
```

Aqui chamamos `u64::from_str` para tentar analisar nosso argumento de linha de comando `arg` como um inteiro sem sinal de 64 bits. Em vez de um método que estamos invocando em algum valor `u64` que temos em mãos, `u64::from_str` é uma função associada ao tipo `u64`, parecido com um método estático em C++ ou Java. A função `from_str` não retorna um `u64` diretamente, mas sim um valor `Result` que indica se a análise foi bem-sucedida ou falhou. Um valor `Result` é uma das duas variantes:

- Um valor escrito `Ok(v)`, indicando que a análise foi bem-sucedida e `v` é o valor gerado.
- Um valor escrito `Err(e)`, indicando que a análise falhou e `e` é um valor de erro explicando o motivo.

As funções que fazem qualquer coisa que possa falhar, como fazer entrada ou saída ou interagir de outra forma com o sistema operacional, podem retornar tipos `Result` cujas variantes `Ok` carregam resultados bem-sucedidos – a contagem de bytes transferidos, o arquivo aberto e assim por diante – e cujas variantes `Err` carregam um código de erro indicando o que está errado. Ao contrário da maioria das linguagens modernas, o Rust não tem exceções: todos os erros são tratados utilizando `Result` ou pânico, conforme descrito no Capítulo 7.

Utilizamos o método `expect` de `Result` para verificar o sucesso de nossa análise. Se o resultado for um `Err(e)`, `expect` imprime uma mensagem que inclui uma descrição de `e` e sai do programa imediatamente. Mas, se o resultado for `Ok(v)`, `expect` simplesmente retorna o próprio `v`, que finalmente somos capazes de armazenar no final do nosso vetor de números.

```
if numbers.len() == 0 {
    eprintln!("Usage: gcd NUMBER ...");
    std::process::exit(1);
}
```

Não há máximo divisor comum de um conjunto vazio de números, então verificamos se nosso vetor tem pelo menos um elemento e finalizamos o programa com um erro caso não tenha. Utilizamos a macro `eprintln!` para mostrar nossa mensagem de erro na saída de erro padrão.

```
let mut d = numbers[0];
for m in &numbers[1..] {
    d = gcd(d, *m);
}
```

Este loop usa `d` como seu valor atual, atualizando-o para permanecer o máximo divisor comum de todos os números que processamos até agora. Como antes, devemos marcar `d` como mutável para que possamos modificá-lo no loop.

O loop `for` tem dois bits surpreendentes. Primeiro, escrevemos `for m in &numbers[1..];` para que serve o operador `&`? Em segundo lugar, escrevemos `gcd(d, *m);` para que serve o `*` em `*m`? Esses dois detalhes são complementares entre si.

Até este ponto, nosso código operou apenas em valores simples como inteiros que cabem em blocos de memória de tamanho fixo. Mas agora estamos prestes a iterar sobre um vetor, que pode ser de qualquer tamanho – possivelmente muito grande. O Rust é cauteloso ao lidar com tais valores: ele quer deixar o programador no controle sobre o consumo de memória, deixando claro quanto tempo cada valor dura, enquanto ainda garante que a memória seja liberada imediatamente quando não for mais necessária.

Então, quando iteramos, queremos dizer ao Rust que a *posse* do vetor deve permanecer com `numbers`; estamos meramente *emprestando* seus elementos para o loop. O operador `&` em `&numbers[1..]` pede emprestada uma *referência* aos elementos do vetor do segundo em diante. O loop `for` itera por meio dos elementos referenciados, deixando `m` emprestar cada elemento sucessivamente. O operador `*` em `*m` *desreferencia* `m`, produzindo o valor que ele referencia; este é o próximo `u64` que queremos passar para `gcd`. Por fim, como `numbers` possui o vetor, o Rust automaticamente o libera quando `numbers` sai do escopo no final de `main`.

As regras do Rust para posse (propriedade) e referências são essenciais para o gerenciamento de memória do Rust e concorrência segura; nós os discutimos em detalhes nos capítulos 4 e 5. Você precisará se familiarizar com essas regras para se sentir confortável no Rust, mas, para este passeio introdutório, tudo o que você precisa saber é que `&x` empresta uma referência a `x`, e esse `*r` é o valor a que a referência `r` se refere.

Continuando nosso passeio pelo programa:

```
println!("The greatest common divisor of {:?} is {:?}",
        numbers, d);
```

Tendo iterado sobre os elementos de `numbers`, o programa mostra os resultados na saída padrão. A macro `println!` pega uma string de template, substitui versões formatadas dos argumentos restantes

pelas formas {...} conforme aparecem na string do template e mostra o resultado na saída padrão.

Ao contrário de C e C++, que requerem que `main` retorne zero se o programa for concluído com sucesso, ou um status de saída diferente de zero se algo der errado, o Rust assume que, se `main` retornar, o programa foi concluído com sucesso. Apenas chamando explicitamente funções como `expect` ou `std::process::exit` podemos fazer com que o programa termine com um código de status de erro.

O comando `cargo run` nos permite passar argumentos para nosso programa, a fim de que possamos experimentar nosso processamento de linha de comando:

```
$ cargo run 42 56
Compiling hello v0.1.0 (/home/jimb/rust/hello)
Finished dev [unoptimized + debuginfo] target(s) in 0.22s
Running `/home/jimb/rust/hello/target/debug/hello 42 56`
The greatest common divisor of [42, 56] is 14
$ cargo run 799459 28823 27347
Finished dev [unoptimized + debuginfo] target(s) in 0.02s
Running `/home/jimb/rust/hello/target/debug/hello 799459 28823 27347`
The greatest common divisor of [799459, 28823, 27347] is 41
$ cargo run 83
Finished dev [unoptimized + debuginfo] target(s) in 0.02s
Running `/home/jimb/rust/hello/target/debug/hello 83`
The greatest common divisor of [83] is 83
$ cargo run
Finished dev [unoptimized + debuginfo] target(s) in 0.02s
Running `/home/jimb/rust/hello/target/debug/hello`
Usage: gcd NUMBER ...
```

Utilizamos alguns recursos da biblioteca padrão do Rust nesta seção. Se você estiver curioso sobre o que mais está disponível, recomendamos fortemente que visite a documentação online do Rust. Ela tem um recurso de pesquisa que facilita a exploração e inclui até links para o código-fonte. O comando `rustup` instala automaticamente uma cópia da documentação em seu computador quando você instala o próprio Rust. Você pode visualizar a documentação da biblioteca padrão no site Rust (<https://oreil.ly/CGsB5>), ou no seu navegador com o comando:

```
$ rustup doc --std
```

Servindo páginas para a web

Um dos pontos fortes do Rust é a coleção de pacotes de bibliotecas disponíveis gratuitamente publicados no site crates.io (<https://crates.io>). O comando `cargo` torna mais fácil para seu código utilizar um pacote `crates.io`: ele baixa a versão correta do pacote, a compila e atualiza conforme solicitado. Um pacote Rust, seja uma biblioteca ou um executável, é chamado de *crate*; Cargo e `crates.io` derivam seus nomes desse termo.

Para mostrar como isso funciona, vamos montar um servidor web simples utilizando o crate do framework web `actix-web`, o crate de serialização `serde` e vários outros crates dos quais esse servidor depende. Conforme mostrado na Figura 2.1, nosso site solicitará ao usuário dois números e calculará seu máximo divisor comum.

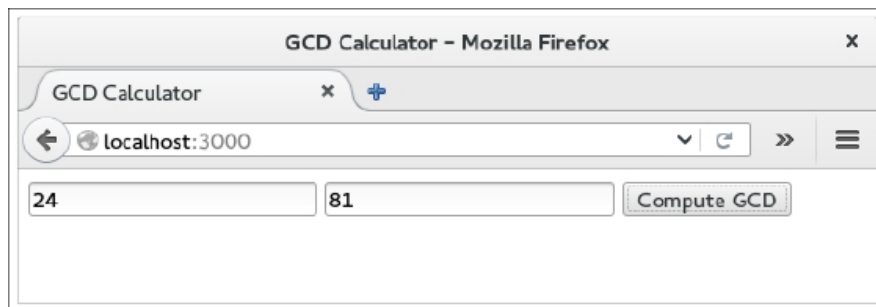


Figura 2.1: Página da web que permite calcular o MDC.

Primeiro, faremos com que o Cargo crie um novo pacote, chamado `actix-gcd`:

```
$ cargo new actix-gcd
   Created binary (application) `actix-gcd` package
$ cd actix-gcd
```

Então, vamos editar o arquivo *Cargo.toml* do nosso novo projeto para listar os pacotes que queremos utilizar; seu conteúdo deve ser o seguinte:

```
[package]
name = "actix-gcd"
version = "0.1.0"
edition = "2021"
```

```
# Veja mais chaves e suas definições em
# https://doc.rust-lang.org/cargo/reference/manifest.html
```

```
[dependencies]
actix-web = "1.0.8"
serde = { version = "1.0", features = ["derive"] }
```

Cada linha na seção `[dependencies]` de *Cargo.toml* fornece o nome de um crate em crates.io e a versão desse crate que gostaríamos de utilizar. Neste caso, queremos a versão 1.0.8 do crate `actix-web` e a versão 1.0 do crate `serde`. Pode haver versões desses crates em crates.io mais recentes do que as mostradas aqui, mas, ao nomear as versões específicas com as quais testamos esse código, podemos garantir que o código continuará a compilar mesmo quando novas versões dos pacotes forem publicadas. Discutiremos o gerenciamento de versões com mais detalhes no Capítulo 8.

Os crates podem ter recursos opcionais: partes da interface ou implementação de que nem todos os usuários necessitam, mas que faz sentido incluir nesse crate. O crate `serde` oferece uma maneira maravilhosamente concisa de lidar com os dados de formulários web, mas, de acordo com a documentação do `serde`, só está disponível se selecionarmos o crate do recurso `derive`, por isso o solicitamos em nosso arquivo *Cargo.toml* como mostrado.

Observe que precisamos apenas nomear os crates que utilizaremos diretamente; `cargo` cuida de trazer quaisquer outros crates necessários.

Para nossa primeira iteração, manteremos o servidor da web simples: ele servirá apenas a página que solicita ao usuário os números utilizados para fazer os cálculos. Em *actix-gcd/src/main.rs*, colocaremos o seguinte texto:

```
use actix_web::{web, App, HttpResponse, HttpServer};

fn main() {
    let server = HttpServer::new(|| {
        App::new()
            .route("/", web::get().to(get_index))
    });

    println!("Serving on http://localhost:3000...");
    server
        .bind("127.0.0.1:3000").expect("error binding server to address")
        .run().expect("error running server");
}
```

```

fn get_index() -> HttpResponse {
  HttpResponse::Ok()
    .content_type("text/html")
    .body(
      r#"
        <title>GCD Calculator</title>
        <form action="/gcd" method="post">
        <input type="text" name="n"/>
        <input type="text" name="m"/>
        <button type="submit">Compute GCD</button>
        </form>
      "#,
    )
}

```

Começamos com uma instrução `use` para facilitar a obtenção de algumas das definições do crate `actix-web`. Quando escrevemos `use actix_web::{...}`, cada um dos nomes listados dentro das chaves torna-se diretamente utilizável em nosso código; em vez de ter de digitar o nome completo `actix_web::HttpResponse` cada vez que o utilizamos, podemos simplesmente o referenciar como `HttpResponse`. (Vamos chegar ao crate `serde` daqui a pouco.)

Nossa função `main` é simples: chama `HttpServer::new` para criar um servidor que responda às solicitações de um único caminho, `"/"`; imprime uma mensagem lembrando-nos de como nos conectar a ele; e, em seguida, define-o como escuta na porta TCP 3000 na máquina local.

O argumento que passamos para `HttpServer::new` é a expressão *closure* `|| { App::new() ... }`. Uma closure é um valor que pode ser chamado como se fosse uma função. Essa closure não requer argumentos, mas, se precisasse, seus nomes apareceriam entre barras verticais `||`. O `{ ... }` é o corpo da closure. Quando iniciamos nosso servidor, o Actix inicia um pool de threads para lidar com as solicitações recebidas. Cada thread chama nossa closure para obter uma nova cópia do valor `App` que informa como rotear e tratar solicitações.

O closure chama `App::new` para criar um novo `App` vazio e, então, chama seu método `route` para acrescentar uma única rota ao caminho `"/"`. O manipulador fornecido para essa rota,

`web::get().to(get_index)`, trata requisições HTTP GET chamando a função `get_index`. O método `route` retorna o mesmo `App` que foi invocado, agora aprimorado com a nova rota. Como não há ponto e vírgula no final do corpo do closure, o `App` é o valor de retorno do closure, pronto para o thread `HttpServer` utilizar.

A função `get_index` constrói um valor `HttpResponse` que representa a resposta a uma solicitação HTTP GET /. `HttpResponse::Ok()` representa um status HTTP 200 OK, indicando que a solicitação foi bem-sucedida. Chamamos seus métodos `content_type` e `body` para preencher os detalhes da resposta; cada chamada retorna o `HttpResponse` que recebeu, com as modificações feitas. Por fim, o valor de retorno de `body` serve como o valor de retorno de `get_index`.

Como o texto da resposta contém muitas aspas duplas, escrevemos utilizando a sintaxe "raw string" ("string bruta") do Rust: a letra `r`, zero ou mais cerquilhas (ou seja, o caractere `#`), uma aspa dupla e, em seguida, o conteúdo da string, finalizado por outra aspa dupla seguida pelo mesmo número de cerquilhas. Qualquer caractere pode ocorrer em uma string bruta sem escape, incluindo aspas duplas; na verdade, nenhuma sequência de escape como `\` é reconhecida. Sempre podemos garantir que a string termine onde pretendemos, utilizando mais cerquilhas em torno das aspas que sempre aparecem no texto.

Tendo escrito *main.rs*, podemos utilizar o comando `cargo run` para fazer tudo o que é necessário para executá-lo: buscar os crates necessários, compilá-los, construir nosso próprio programa, vincular tudo e iniciá-lo:

```
$ cargo run
  Updating crates.io index
Downloading crates ...
  Downloaded serde v1.0.100
  Downloaded actix-web v1.0.8
  Downloaded serde_derive v1.0.100
...
  Compiling serde_json v1.0.40
  Compiling actix-router v0.1.5
  Compiling actix-http v0.2.10
  Compiling awc v0.2.7
```

```
Compiling actix-web v1.0.8
Compiling gcd v0.1.0 (/home/jimb/rust/actix-gcd)
Finished dev [unoptimized + debuginfo] target(s) in 1m 24s
Running `/home/jimb/rust/actix-gcd/target/debug/actix-gcd`
Serving on http://localhost:3000...
```

Neste ponto, podemos visitar a URL fornecida em nosso navegador e ver a página mostrada anteriormente na Figura 2.1.

Infelizmente, clicar em Compute GCD (Calcular MDC) não faz nada além de levar nosso navegador para uma página em branco. Vamos corrigir isso a seguir, adicionando outra rota ao nosso App para lidar com a solicitação POST em nosso formulário.

Finalmente chegou a hora de utilizar o crate `serde` que listamos em nosso arquivo *Cargo.toml*: ele fornece uma ferramenta útil que nos ajudará a processar os dados do formulário. Primeiro, precisamos adicionar a seguinte diretiva `use` na parte superior de *src/main.rs*:

```
use serde::Deserialize;
```

Os programadores Rust geralmente reúnem todas as suas declarações `use` na parte superior do arquivo, mas isso não é estritamente necessário: o Rust permite que as declarações ocorram em qualquer ordem, desde que apareçam no nível apropriado de indentação.

Em seguida, vamos definir um tipo de estrutura Rust que representa os valores que esperamos de nosso formulário:

```
#[derive(Deserialize)]
struct GcdParameters {
    n: u64,
    m: u64,
}
```

Isso define um novo tipo chamado `GcdParameters` que tem dois campos, `n` e `m`, cada um dos quais é um `u64` – o tipo de argumento que nossa função `gcd` espera.

A anotação acima da definição `struct` é um atributo, como o atributo `#[test]` que utilizamos anteriormente para marcar as funções de teste. Colocar um atributo `#[derive(Deserialize)]` acima de uma definição de tipo instrui o crate `serde` a examinar o tipo quando o programa for compilado e a gerar automaticamente um código para analisar um

valor desse tipo a partir de dados no formato que os formulários HTML utilizam para solicitações POST. Na verdade, esse atributo é suficiente para permitir que você analise um valor `GcdParameters` de quase qualquer tipo de dados estruturados: JSON, YAML, TOML ou qualquer um de vários outros formatos textuais e binários. O crate `serde` também fornece um atributo `Serialize` que gera código para fazer o inverso, pegar valores Rust e escrevê-los em um formato estruturado.

Com essa definição, podemos escrever nossa função de tratamento com bastante facilidade:

```
fn post_gcd(form: web::Form<GcdParameters>) -> HttpResponse {
    if form.n == 0 || form.m == 0 {
        return HttpResponse::BadRequest()
            .content_type("text/html")
            .body("Computing the GCD with zero is boring.");
    }

    let response =
        format!("The greatest common divisor of the numbers {} and {} \
            is <b>{}</b>\n",
            form.n, form.m, gcd(form.n, form.m));

    HttpResponse::Ok()
        .content_type("text/html")
        .body(response)
}
```

Para que uma função sirva como um manipulador de solicitações do Actix, todos os seus argumentos devem ter tipos que o Actix saiba como extrair de uma solicitação HTTP. Nossa função `post_gcd` recebe um argumento, `form`, cujo tipo é `web::Form<GcdParameters>`. O Actix sabe como extrair um valor de qualquer tipo `web::Form<T>` de uma solicitação HTTP se, e somente se, `T` puder ser desserializado a partir dos dados POST do formulário HTML. Como colocamos o atributo `#[derive(Deserialize)]` em nossa definição de tipo `GcdParameters`, o Actix pode desserializá-lo a partir dos dados do formulário, portanto as funções manipuladoras de solicitação podem esperar um valor `web::Form<GcdParameters>` como parâmetro. Essas relações entre tipos e funções são todas trabalhadas em tempo de compilação; se escrevermos uma função manipuladora com um tipo de argumento

que o Actix não saiba manipular, o compilador Rust nos informará sobre nosso erro imediatamente.

Examinando `post_gcd`, a função primeiro retorna um erro HTTP 400 BAD REQUEST se um dos parâmetros for zero, pois nossa função `gcd` gera um pânico caso sejam. Em seguida, a função constrói uma resposta para a solicitação utilizando a macro `format!`. A macro `format!` é como a macro `println!`, exceto que, em vez de escrever o texto na saída padrão, ela o retorna como uma string. Uma vez obtido o texto da resposta, `post_gcd` empacota-o em uma resposta HTTP 200 OK, define seu tipo de conteúdo e o retorna para ser entregue ao remetente.

Também temos de registrar `post_gcd` como o manipulador do formulário. Agora vamos substituir nossa função `main` por esta versão:

```
fn main() {
    let server = HttpServer::new(|| {
        App::new()
            .route("/", web::get().to(get_index))
            .route("/gcd", web::post().to(post_gcd))
    });

    println!("Serving on http://localhost:3000...");
    server
        .bind("127.0.0.1:3000").expect("error binding server to address")
        .run().expect("error running server");
}
```

A única mudança aqui é que adicionamos outra chamada para `route`, estabelecendo `web::post().to(post_gcd)` como o manipulador para o caminho `/gcd`.

A última peça restante é a função `gcd` que escrevemos anteriormente, e que fica no arquivo `actix-gcd/src/main.rs`. Com isso no lugar, você pode interromper qualquer servidor que tenha deixado em execução, recompilar e reiniciar o programa:

```
$ cargo run
Compiling actix-gcd v0.1.0 (/home/jimb/rust/actix-gcd)
Finished dev [unoptimized + debuginfo] target(s) in 0.0 secs
Running `target/debug/actix-gcd`
Serving on http://localhost:3000...
```

Desta vez, visitando `http://localhost:3000`, inserindo alguns números

e clicando no botão Compute GCD, você deve ver alguns resultados (Figura 2.2).



Figura 2.2: Página da web mostrando os resultados da computação do máximo divisor comum.

Concorrência

Um dos grandes pontos fortes do Rust é o suporte à programação concorrente. As mesmas regras que garantem que os programas Rust estejam livres de erros de memória também garantem que os threads possam compartilhar memória apenas de maneiras que evitem corridas de dados (data races). Por exemplo:

- Se você utilizar um mutex para coordenar threads que fazem alterações em uma estrutura de dados compartilhada, o Rust garante que você não possa acessar os dados, exceto quando estiver segurando o bloqueio, e libera o bloqueio automaticamente quando você terminar. Em C e C++, o relacionamento entre um mutex e os dados que ele protege é deixado para os comentários.
- Se você deseja compartilhar dados somente leitura entre vários threads, o Rust garante que você não possa modificar os dados acidentalmente. Em C e C++, o sistema de tipos pode ajudar com isso, mas é fácil errar.
- Se você transferir a posse de uma estrutura de dados de um thread para outro, o Rust garante que você realmente cedeu todo o acesso a ele. Em C e C++, cabe a você verificar se nada no thread de envio terá acesso aos dados novamente. Se você não acertar, os efeitos podem depender do que está no cache do processador e de quantas gravações na memória você fez recentemente. Não que sejamos pessimistas.

Nesta seção, vamos guiá-lo pelo processo de escrever seu segundo programa multithread.

Você já escreveu o primeiro: o framework web Actix utilizado para implementar o servidor do máximo divisor comum usa um pool de threads para executar funções manipuladoras de solicitação. Se o servidor receber requisições simultâneas, ele pode executar as funções `get_index` e `post_gcd` em vários segmentos ao mesmo tempo. Isso pode ser um pouco chocante, já que certamente não tínhamos a concorrência em mente quando escrevemos essas funções. Mas Rust garante que isso é seguro, não importa o quão elaborado seu servidor seja: se o seu programa compilar, ele estará livre de corridas de dados (data races). Todas as funções do Rust são thread-safe.

O programa desta seção plota o conjunto de Mandelbrot, um fractal gerado pela iteração de uma função simples em números complexos. Plotar o conjunto de Mandelbrot é frequentemente chamado de algoritmo *embarçosamente paralelo*, porque o padrão de comunicação entre os threads é muito simples; abordaremos padrões mais complexos no Capítulo 19, mas essa tarefa demonstra alguns dos fundamentos.

Para começar, criaremos um novo projeto Rust:

```
$ cargo new mandelbrot
    Created binary (application) `mandelbrot` package
$ cd mandelbrot
```

Todo o código entrará *mandelbrot/src/main.rs*, e adicionaremos algumas dependências a *mandelbrot/Cargo.toml*.

Antes de entrarmos na implementação concorrente de Mandelbrot, precisamos descrever a computação que vamos realizar.

O que é realmente o conjunto de Mandelbrot

Ao lermos o código, é útil ter uma ideia concreta do que ele está tentando fazer, então vamos fazer uma pequena excursão em matemática pura. Começaremos com um caso simples e depois

adicionaremos detalhes complicadores até chegarmos ao cálculo central do conjunto de Mandelbrot.

Eis um loop infinito, escrito utilizando a sintaxe dedicada do Rust para isso, uma declaração `loop`:

```
fn square_loop(mut x: f64) {  
    loop {  
        x = x * x;  
    }  
}
```

Na vida real, o Rust pode ver que `x` nunca é utilizado para nada e, portanto, pode não se preocupar em calcular seu valor. Mas, por enquanto, suponha que o código seja executado conforme escrito. O que acontece com o valor de `x`? Elevar ao quadrado qualquer número menor que 1 o torna menor, de modo que se aproxima de zero; o quadrado de 1 resulta em 1; elevar ao quadrado um número maior que 1 o torna maior, de modo que se aproxima do infinito; e o quadrado de um número negativo o torna positivo, após o que ele se comporta como um dos casos anteriores (Figura 2.3).

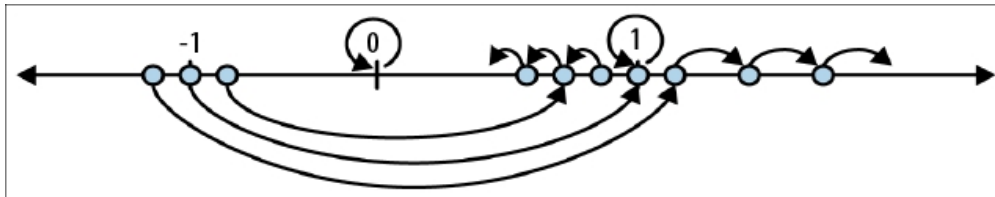


Figura 2.3: Efeitos de elevar ao quadrado um número repetidamente.

Então, dependendo do valor que você passa para `square_loop`, `x` permanece em zero ou um, aproxima-se de zero ou aproxima-se do infinito.

Agora considere um loop ligeiramente diferente:

```
fn square_add_loop(c: f64) {  
    let mut x = 0.;  
    loop {  
        x = x * x + c;  
    }  
}
```

Dessa vez, `x` começa em zero e ajustamos seu progresso a cada

iteração somando c depois de elevá-lo ao quadrado. Isso torna mais difícil ver como x se comporta, mas algumas experiências mostram que, se c for maior que 0,25 ou menor que -2,0, então x eventualmente se torna infinitamente grande; caso contrário, ele fica em algum lugar na vizinhança de zero.

A próxima questão: em vez de utilizar valores `f64`, considere o mesmo loop utilizando números complexos. O crate `num` em `crates.io` fornece um tipo de número complexo que podemos utilizar, então devemos adicionar uma linha para `num` na seção `[dependencies]` no arquivo *Cargo.toml* do nosso programa. Eis o arquivo inteiro, até este ponto (vamos ampliá-lo mais tarde):

```
[package]
name = "mandelbrot"
version = "0.1.0"
edition = "2021"

# Veja mais chaves e suas definições em
# https://doc.rust-lang.org/cargo/reference/manifest.html

[dependencies]
num = "0.4"
```

Agora podemos escrever a penúltima versão do nosso loop:

```
use num::Complex;

fn complex_square_add_loop(c: Complex<f64>) {
    let mut z = Complex { re: 0.0, im: 0.0 };
    loop {
        z = z * z + c;
    }
}
```

É costume utilizar z para números complexos, então renomeamos nossa variável de loop. A expressão `Complex { re: 0.0, im: 0.0 }` é a maneira como escrevemos o zero complexo utilizando o tipo `Complex` do crate `num`. `Complex` é um tipo de estrutura do Rust (ou *struct*), assim definido:

```
struct Complex<T> {
    /// Parte real do número complexo
    re: T,

    /// Parte imaginária do número complexo
    im: T,
}
```

O código anterior define uma estrutura chamada `Complex`, com dois campos, `re` e `im`. `Complex` é uma estrutura *genérica*: você pode ler o `<T>` após o nome do tipo como “para qualquer tipo T ”. Por exemplo, `Complex<f64>` é um número complexo cujos campos `re` e `im` são valores `f64`, `Complex<f32>` utilizaria floats de 32 bits e assim por diante. Dada essa definição, uma expressão como `Complex { re: 0.24, im: 0.3 }` produz um valor `Complex` com seu campo `re` inicializado em 0.24, e seu campo `im` inicializado em 0.3.

O crate `num` arranja para `*`, `+` e outros operadores aritméticos para trabalhar valores `Complex`, então o restante da função funciona exatamente como a versão anterior, exceto que opera em pontos no plano complexo, não apenas em pontos ao longo da reta numérica real. Explicaremos no Capítulo 12 como você pode fazer os operadores do Rust funcionarem com seus próprios tipos.

Finalmente, chegamos ao destino de nossa excursão em matemática pura. O conjunto de Mandelbrot é definido como o conjunto dos números complexos c para o qual z não tende ao infinito. Nosso loop quadrado simples original era previsível o suficiente: qualquer número maior que 1 ou menor que -1 desaparece. Adicionar um `+` c a cada iteração torna o comportamento um pouco mais difícil de antecipar: como dissemos anteriormente, os valores de c maiores que 0,25 ou menores que -2 fazem z tender ao infinito. Mas expandir o jogo para números complexos produz padrões verdadeiramente bizarros e elegantes, que é o que queremos plotar.

Como um número complexo c tem componentes reais e imaginários `c.re` e `c.im`, vamos tratá-los como as coordenadas x e y de um ponto no plano cartesiano e pintar o ponto de preto se c estiver no conjunto de Mandelbrot, ou uma cor mais clara, caso contrário. Portanto, para cada pixel em nossa imagem, devemos executar o loop anterior no ponto correspondente no plano complexo, ver se ele escapa para o infinito ou orbita em torno da origem para sempre, e colori-lo de acordo.

O loop infinito demora um pouco para rodar, mas há dois truques para os impacientes. Primeiro, se desistirmos de executar o loop

indefinidamente e apenas tentarmos um número limitado de iterações, veremos que ainda obteremos uma aproximação decente do conjunto. Saber de quantas iterações precisamos vai depender de quão precisamente queremos plotar o limite. Em segundo lugar, foi demonstrado que, se z alguma vez deixar o círculo de raio 2 centrado na origem, ele definitivamente sumirá no infinito em algum momento, longe da origem. Então, eis a versão final do nosso loop e o coração do nosso programa:

```
use num::Complex;

/// Tenta determinar se `c` está no conjunto de Mandelbrot, utilizando
/// no máximo `limit` iterações para decidir.
/// Se `c` não for um membro, retorna `Some(i)`, onde `i` é o número de
/// iterações necessárias para `c` deixar o círculo de raio 2 centrado no
/// origem. Se `c` parece ser um membro (mais precisamente, se chegamos ao
/// limite de iterações sem ser capaz de provar que `c` não é um membro),
/// retorna `None`.
fn escape_time(c: Complex<f64>, limit: usize) -> Option<usize> {
    let mut z = Complex { re: 0.0, im: 0.0 };
    for i in 0..limit {
        if z.norm_sqr() > 4.0 {
            return Some(i);
        }
        z = z * z + c;
    }

    None
}
```

Essa função pega o número complexo c que queremos testar para saber se pertence ao conjunto de Mandelbrot e um limite para o número de iterações a serem tentadas antes de desistir (*limit*) e declarar que c provavelmente é um membro.

O valor de retorno da função é um `Option<usize>`. A biblioteca padrão do Rust define o tipo `Option` da seguinte forma:

```
enum Option<T> {
    None,
    Some(T),
}
```

`Option` é um *tipo enumerado*, muitas vezes chamado de `enum`, porque sua definição enumera diversas variantes que um valor desse tipo poderia assumir: para qualquer tipo T , um valor do tipo `Option<T>` ou

é `Some(v)`, em que `v` é um valor do tipo `T`, ou é `None`, indicando que nenhum valor do tipo `T` está disponível. Como o tipo `Complex` que discutimos anteriormente, `Option` é um tipo genérico: você pode utilizar `Option<T>` para representar um valor opcional de qualquer tipo `T` que quiser.

No nosso caso, `escape_time` retorna um `Option<usize>` para indicar se `c` está no conjunto de Mandelbrot – e, se não estiver, quantas vezes tivemos de iterar para descobrir isso. Se `c` não está no conjunto, `escape_time` retorna `Some(i)`, em que `i` é o número da iteração em que `z` deixou o círculo de raio 2. Caso contrário, `c` está aparentemente no conjunto, e `escape_time` retorna `None`.

```
for i in 0..limit {
```

Os exemplos anteriores mostraram loops `for` iterando sobre argumentos de linha de comando e elementos vetoriais; esse loop `for` simplesmente itera sobre o intervalo de inteiros começando com 0 e indo até (mas não incluindo) `limit`.

A chamada de método `z.norm_sqr()` retorna o quadrado da distância de `z` em relação à origem. Para decidir se `z` saiu do círculo de raio 2, em vez de calcular uma raiz quadrada, apenas comparamos a distância ao quadrado com 4,0, que é mais rápido.

Você deve ter notado que utilizamos `///` para marcar as linhas de comentário acima da definição da função; os comentários acima dos membros da estrutura `Complex` também começam com `///`. Esses são *comentários da documentação*; a `rustdoc` utilitário sabe como analisá-los, com o código que eles descrevem, e produzir documentação online. A documentação da biblioteca padrão do Rust está escrita dessa forma. Descrevemos os comentários da documentação em detalhes no Capítulo 8.

O restante do programa se preocupa em decidir qual parte do conjunto plotar e em qual resolução, e então em distribuir o trabalho por vários threads para acelerar o cálculo.

Analizando argumentos de linha de comando em pares

O programa usa vários argumentos de linha de comando que controlam a resolução da imagem que gravaremos e a parte do conjunto de Mandelbrot que a imagem mostra. Como todos esses argumentos de linha de comando seguem um formato comum, eis uma função para analisá-los:

```
use std::str::FromStr;

/// Analise a string `s` como um par de coordenadas, como `"400x600"` ou `"1.0,0.5"`.
///
/// Especificamente, `s` deve ter a forma <left><sep><right>, onde <sep> é
/// o caractere dado pelo argumento `separator`, e <left> e <right> são
/// strings que podem ser analisadas por `T::from_str`. `separator`
/// deve ser um caractere ASCII.
///
/// Se `s` tiver a forma adequada, retorna `Some<(x, y)>`.
/// Se não analisar corretamente, retorna `None`.
fn parse_pair<T: FromStr>(s: &str, separator: char) -> Option<(T, T)> {
    match s.find(separator) {
        None => None,
        Some(index) => {
            match (T::from_str(&s[..index]), T::from_str(&s[index + 1..])) {
                (Ok(l), Ok(r)) => Some((l, r)),
                _ => None
            }
        }
    }
}

#[test]
fn test_parse_pair() {
    assert_eq!(parse_pair::<i32>("", ','), None);
    assert_eq!(parse_pair::<i32>("10,", ','), None);
    assert_eq!(parse_pair::<i32>(",10", ','), None);
    assert_eq!(parse_pair::<i32>("10,20", ','), Some((10, 20)));
    assert_eq!(parse_pair::<i32>("10,20xy", ','), None);
    assert_eq!(parse_pair::<f64>("0.5x", 'x'), None);
    assert_eq!(parse_pair::<f64>("0.5x1.5", 'x'), Some((0.5, 1.5)));
}
```

A definição de `parse_pair` é uma *função genérica*:

```
fn parse_pair<T: FromStr>(s: &str, separator: char) -> Option<(T, T)> {
```

Você pode ler a cláusula `<T: FromStr>` em voz alta, como “Para qualquer tipo `T` que implementa o `trait FromStr...`”. Isso efetivamente nos permite definir uma família inteira de funções de uma só vez:

`parse_pair::<i32>` é uma função que analisa pares de `i32` valores, `parse_pair::<f64>` analisa pares de valores de ponto flutuante e assim por diante. Isso é muito parecido com um modelo de função em C++. Um programador Rust chamaria τ um *parâmetro de tipo* de `parse_pair`. Quando você usa uma função genérica, o Rust geralmente consegue inferir parâmetros de tipo para você, e não é necessário escrevê-los como fizemos no código de teste.

Nosso tipo de retorno é `Option<(T, T)>`: ou `None` ou um valor `Some((v1, v2))`, em que `(v1, v2)` é uma tupla de dois valores, ambos do tipo τ . A função `parse_pair` não usa uma instrução de retorno explícita, então seu valor de retorno é o valor da última (e única) expressão em seu corpo:

```
match s.find(separator) {
    None => None,
    Some(index) => {
        ...
    }
}
```

O método `find` do tipo `String` procura na string um caractere que corresponda a `separator`. Se `find` retornar `None`, o que significa que o caractere separador não ocorre na string, toda a expressão `match` é avaliada como `None`, indicando que a análise falhou. Caso contrário, assumimos que `index` seja a posição do separador na string.

```
match (T::from_str(&s[..index]), T::from_str(&s[index + 1..])) {
    (Ok(l), Ok(r)) => Some((l, r)),
    _ => None
}
```

Isso começa a mostrar o poder da expressão `match`. O argumento para a correspondência é esta expressão de tupla:

```
(T::from_str(&s[..index]), T::from_str(&s[index + 1..]))
```

As expressões `&s[..index]` e `&s[index + 1..]` são fatias da string, precedendo e sucedendo o separador. A função associada `from_str` do parâmetro de tipo τ pega cada um deles e tenta analisá-los como um valor do tipo τ , produzindo uma tupla de resultados. Isto é o que nós comparamos:

```
(Ok(l), Ok(r)) => Some((l, r)),
```

Esse padrão corresponde apenas se ambos os elementos da tupla

forem variantes `Ok` do tipo `Result`, indicando que ambas as análises foram bem-sucedidas. Se sim, `Some((l, r))` é o valor da expressão de correspondência e, portanto, o valor de retorno da função.

```
_ => None
```

O padrão curinga `_` corresponde a qualquer coisa e ignora seu valor. Se chegamos a este ponto, então `parse_pair` falhou e, portanto, avaliamos como `None`, novamente fornecendo o valor de retorno da função.

Agora que temos `parse_pair`, é fácil escrever uma função para analisar um par de coordenadas de ponto flutuante e retorná-las como um valor `Complex<f64>`:

```
/// Analisa um par de números de ponto flutuante separados
/// por uma vírgula como um número complexo
fn parse_complex(s: &str) -> Option<Complex<f64>> {
    match parse_pair(s, ',') {
        Some((re, im)) => Some(Complex { re, im }),
        None => None
    }
}

#[test]
fn test_parse_complex() {
    assert_eq!(parse_complex("1.25,-0.0625"),
        Some(Complex { re: 1.25, im: -0.0625 }));
    assert_eq!(parse_complex("-0.0625"), None);
}
```

A função `parse_complex` chama `parse_pair`, constrói um valor `Complex` se as coordenadas foram analisadas com sucesso e passa as falhas para seu chamador.

Se você leu atentamente, deve ter notado que utilizamos uma notação abreviada para construir o valor `Complex`. É comum inicializar os campos de uma struct com variáveis de mesmo nome; então, em vez de forçá-lo a escrever `Complex { re: re, im: im }`, o Rust permite que você simplesmente escreva `Complex { re, im }`. Isso é modelado em notações semelhantes em JavaScript e Haskell.

Mapeando pixels para números complexos

O programa precisa trabalhar em dois espaços de coordenadas

relacionados: cada pixel na imagem de saída corresponde a um ponto no plano complexo. A relação entre esses dois espaços depende de qual parte do conjunto de Mandelbrot vamos plotar e a resolução da imagem solicitada, conforme determinado pelos argumentos de linha de comando. A função a seguir converte um *espaço de imagem* em um *espaço de número complexo*:

```
/// Dada a linha e a coluna de um pixel na imagem de saída,  
/// retorna o ponto correspondente no plano complexo  
///  
/// `bounds` é um par que dá a largura e a altura da imagem em pixels.  
/// `pixel` é um par (coluna, linha) que indica um pixel específico nessa imagem.  
/// Os parâmetros `upper_left` e `lower_right` são pontos no plano  
/// complexo que designam a área que nossa imagem cobre.  
fn pixel_to_point(bounds: (usize, usize),  
                  pixel: (usize, usize),  
                  upper_left: Complex<f64>,  
                  lower_right: Complex<f64>)  
-> Complex<f64>  
{  
    let (width, height) = (lower_right.re - upper_left.re,  
                           upper_left.im - lower_right.im);  
    Complex {  
        re: upper_left.re + pixel.0 as f64 * width / bounds.0 as f64,  
        im: upper_left.im - pixel.1 as f64 * height / bounds.1 as f64  
        // Por que subtração aqui? pixel.1 aumenta à medida que descemos,  
        // mas o componente imaginário aumenta à medida que subimos  
    }  
}
```

```
#[test]  
fn test_pixel_to_point() {  
    assert_eq!(pixel_to_point((100, 200), (25, 175),  
                             Complex { re: -1.0, im: 1.0 },  
                             Complex { re: 1.0, im: -1.0 }),  
               Complex { re: -0.5, im: -0.75 });  
}
```

A Figura 2.4 ilustra o cálculo que `pixel_to_point` realiza.

O código de `pixel_to_point` é simplesmente cálculo, então não vamos explicá-lo em detalhes. Contudo, há alguns aspectos a apontar. Expressões com essa forma referenciam elementos de tupla:

`pixel.0`

Isso referencia o primeiro elemento da tupla `pixel`.

pixel.0 as f64

Essa é a sintaxe do Rust para uma conversão de tipo: isso converte pixel.0 em um valor f64. Ao contrário de C e C++, Rust geralmente se recusa a converter entre tipos numéricos implicitamente; você deve escrever as conversões necessárias. Isso pode ser tedioso, mas ser explícito sobre quais conversões ocorrem e quando ocorrem é surpreendentemente útil. As conversões implícitas de inteiros parecem bastante inocentes, mas historicamente elas têm sido uma fonte frequente de bugs e falhas de segurança em código C e C++ do mundo real.

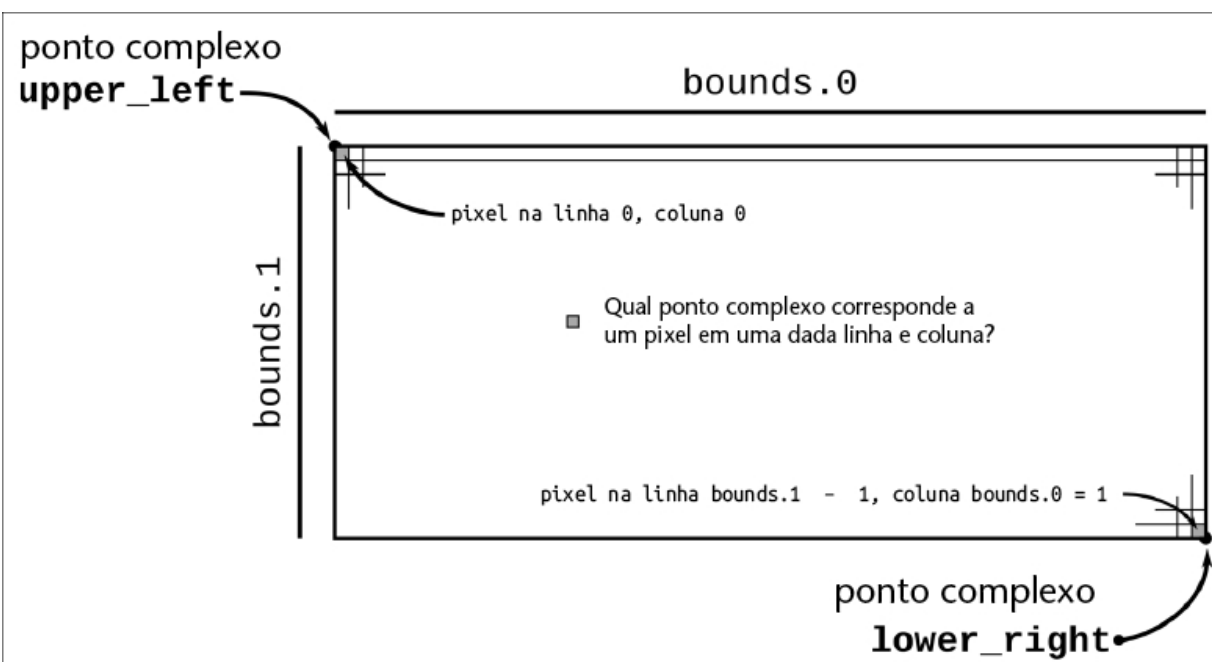


Figura 2.4: A relação entre o plano complexo e os pixels da imagem.

Plotando o conjunto

Para plotar o conjunto de Mandelbrot, para cada pixel da imagem, simplesmente aplicamos `escape_time` para o ponto correspondente no plano complexo e pintamos o pixel dependendo do resultado:

```
/// Renderiza um retângulo do conjunto de Mandelbrot em um buffer de pixels.  
/// O argumento `bounds` dá a largura e a altura dos `pixels` do buffer,  
/// que contém um pixel em tons de cinza por byte.  
/// Os argumentos `upper_left` e `lower_right` especificam  
/// pontos no plano complexo correspondente aos cantos  
/// superior esquerdo e inferior direito do buffer de pixels.
```

```

fn render(pixels: &mut [u8],
          bounds: (usize, usize),
          upper_left: Complex<f64>,
          lower_right: Complex<f64>)
{
    assert!(pixels.len() == bounds.0 * bounds.1);

    for row in 0..bounds.1 {
        for column in 0..bounds.0 {
            let point = pixel_to_point(bounds, (column, row),
                                       upper_left, lower_right);
            pixels[row * bounds.0 + column] =
                match escape_time(point, 255) {
                    None => 0,
                    Some(count) => 255 - count as u8
                };
        }
    }
}

```

Tudo isso deve parecer bastante familiar neste momento.

```

pixels[row * bounds.0 + column] =
    match escape_time(point, 255) {
        None => 0,
        Some(count) => 255 - count as u8
    };

```

Se `escape_time` diz que `point` pertence ao conjunto, `render` pinta o pixel correspondente de preto (0). Por outro lado, `render` atribui cores mais escuras aos números que demoraram mais para sair do círculo.

Gravando arquivos de imagem

O crate `image` fornece funções para ler e gravar uma ampla variedade de formatos de imagem, com algumas funções básicas de tratamento de imagem. Em particular, ele inclui um codificador para o formato de arquivo de imagem PNG, que esse programa usa para salvar os resultados finais do cálculo. Para utilizar `image`, adicione a seguinte linha à seção `[dependencies]` de *Cargo.toml*:

```
image = "0.13.0"
```

Feito isso, podemos escrever:

```

use image::ColorType;
use image::png::PNGEncoder;
use std::fs::File;

```

```

/// Escreva o buffer `pixels`, cujas dimensões são dadas por `bounds`,
/// para o arquivo chamado `filename`
fn write_image(filename: &str, pixels: &[u8], bounds: (usize, usize))
    -> Result<(), std::io::Error>
{
    let output = File::create(filename)?;
    let encoder = PNGEncoder::new(output);
    encoder.encode(pixels,
                   bounds.0 as u32, bounds.1 as u32,
                   ColorType::Gray(8))?;

    Ok(())
}

```

A operação dessa função é bastante direta: ela abre um arquivo e tenta gravar a imagem nele. Passamos ao codificador os dados de pixel reais de `pixels`, e sua largura e altura de `bounds`, e então um argumento final que diz como interpretar os bytes em `pixels`: o valor de `ColorType::Gray(8)` indica que cada byte é um valor de escala de cinza de oito bits.

Isso é simples e direto. O que é interessante sobre essa função é como ela lida quando algo dá errado. Se encontrarmos um erro, precisamos reportá-lo ao nosso chamador. Como mencionamos antes, funções que podem falhar no Rust devem retornar um valor `Result`, que é `Ok(s)` em caso de sucesso, em que `s` é o valor de sucesso, ou `Err(e)` em caso de falha, em que `e` é um código de erro. Então, quais são os tipos de sucesso e erro de `write_image`?

Quando tudo corre bem, nossa função `write_image` não tem valor útil para retornar; ela gravou tudo interessante no arquivo. Portanto, seu tipo de sucesso é o tipo `unit()`, assim chamado porque tem apenas um valor, também escrito `()`. O tipo `unit` é semelhante a `void` em C e C++.

Quando ocorre um erro, é porque ou `File::create` não foi capaz de criar o arquivo ou `encoder.encode` não foi capaz de gravar a imagem nele; a operação de E/S retornou um código de erro. O tipo de retorno de `File::create` é `Result<std::fs::File, std::io::Error>`, enquanto o de `encoder.encode` é `Result<(), std::io::Error>`, então ambos compartilham o mesmo tipo de erro, `std::io::Error`. Faz sentido nossa função `write_image` fazer o mesmo. Em ambos os casos, a falha deve resultar em um retorno imediato,

repassando o valor `std::io::Error` descrevendo o que está errado.

Então, para lidar corretamente com o resultado `File::create`, precisamos fazer o valor de retorno corresponder com a instrução `match`, da seguinte forma:

```
let output = match File::create(filename) {  
    Ok(f) => f,  
    Err(e) => {  
        return Err(e);  
    }  
};
```

Em caso de sucesso, faça com que `output` seja o `File` dentro no valor `Ok`. Em caso de falha, repasse o erro ao nosso próprio chamador.²

Esse tipo de instrução `match` é um padrão tão comum no Rust que a linguagem fornece o operador `?` como uma abreviação para a coisa toda. Portanto, em vez de escrever essa lógica explicitamente toda vez que tentamos algo que pode falhar, você pode utilizar a seguinte instrução equivalente e muito mais legível:

```
let output = File::create(filename)?;
```

Se `File::create` falhar, o operador `?` retorna de `write_image`, passando o erro. Por outro lado, `output` mantém o `File` aberto com sucesso.



É um erro comum de iniciante tentar utilizar `?` na função `main`. Entretanto, como `main` em si não retorna um valor, isso não funcionará; em vez disso, você precisa utilizar uma declaração `match`, ou um dos métodos de abreviação como `unwrap` e `expect`. Há também a opção de simplesmente mudar `main` para retornar um `Result`, que abordaremos mais adiante.

Um programa de Mandelbrot concorrente

Todas as peças estão no lugar, e podemos mostrar-lhe a função `main`, na qual podemos colocar a concorrência para trabalhar para nós. Primeiro, uma versão não concorrente para simplificar:

```
use std::env;
```

```

fn main() {
    let args: Vec<String> = env::args().collect();

    if args.len() != 5 {
        eprintln!("Usage: {} FILE PIXELS UPPERLEFT LOWERRIGHT",
            args[0]);
        eprintln!("Example: {} mandel.png 1000x750 -1.20,0.35 -1,0.20",
            args[0]);
        std::process::exit(1);
    }

    let bounds = parse_pair(&args[2], 'x')
        .expect("error parsing image dimensions");
    let upper_left = parse_complex(&args[3])
        .expect("error parsing upper left corner point");
    let lower_right = parse_complex(&args[4])
        .expect("error parsing lower right corner point");

    let mut pixels = vec![0; bounds.0 * bounds.1];

    render(&mut pixels, bounds, upper_left, lower_right);

    write_image(&args[1], &pixels, bounds)
        .expect("error writing PNG file");
}

```

Depois de coletar os argumentos da linha de comando em um vetor de Strings, analisamos cada um deles e começamos os cálculos.

```
let mut pixels = vec![0; bounds.0 * bounds.1];
```

Uma chamada à macro `vec![v; n]` cria um vetor de n elementos de comprimento que são inicializados como v , então o código anterior cria um vetor de zeros cujo comprimento é $\text{bounds.0} * \text{bounds.1}$, em que `bounds` é a resolução da imagem analisada na linha de comando. Usaremos esse vetor como uma array retangular de valores de pixel em tons de cinza de um byte, conforme mostrado na Figura 2.5.

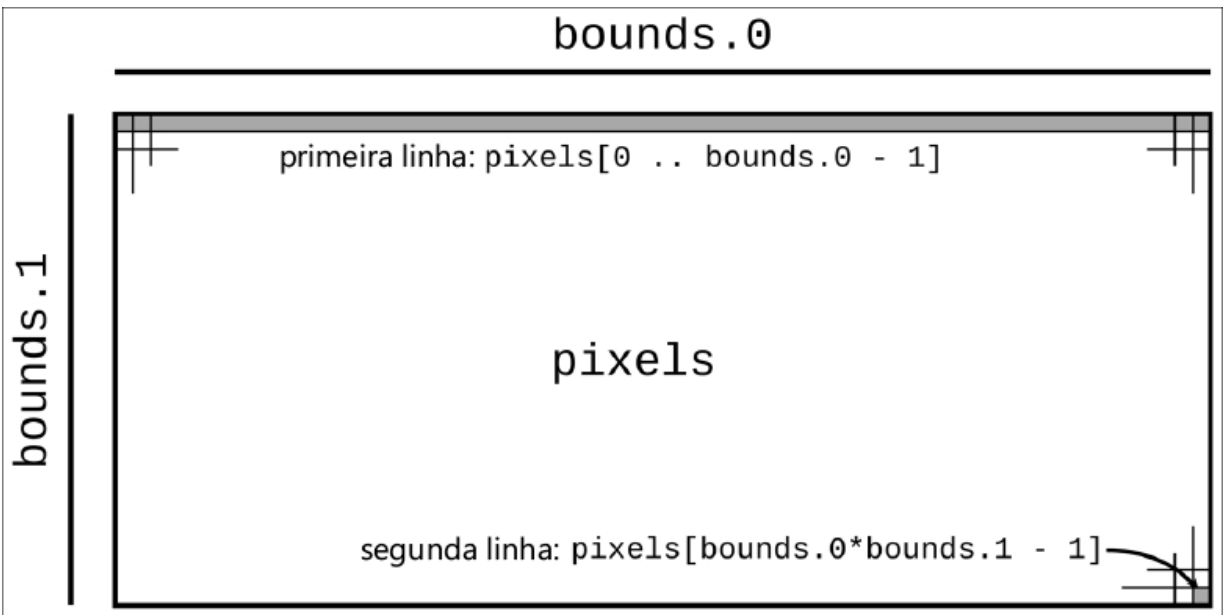


Figura 2.5: Utilizando um vetor como um array retangular de pixels.

A próxima linha de interesse é esta:

```
render(&mut pixels, bounds, upper_left, lower_right);
```

Isso chama a função `render` para realmente calcular a imagem. A expressão `&mut pixels` empresta uma referência mutável ao nosso buffer de pixels, permitindo que `render` o preencha com valores de tons de cinza calculados, mesmo enquanto `pixels` permanece como o proprietário do vetor. Os argumentos restantes passam as dimensões da imagem e o retângulo do plano complexo que escolhemos plotar.

```
write_image(&args[1], &pixels, bounds)
    .expect("error writing PNG file");
```

Por fim, escrevemos o buffer de pixels no disco como um arquivo PNG. Nesse caso, passamos uma referência compartilhada (não mutável) ao buffer, pois `write_image` não deve ter necessidade de modificar o conteúdo do buffer.

Neste ponto, podemos construir e executar o programa no modo `release`, o que permite muitas otimizações poderosas do compilador e, após alguns segundos, ele gravará uma bela imagem no arquivo *mandel.png*:

```
$ cargo build --release
    Updating crates.io index
```

```
Compiling autocfg v1.0.1
...
Compiling image v0.13.0
Compiling mandelbrot v0.1.0 ($RUSTBOOK/mandelbrot)
Finished release [optimized] target(s) in 25.36s
$ time target/release/mandelbrot mandel.png 4000x3000 -1.20,0.35 -1,0.20
real    0m4.678s
user    0m4.661s
sys     0m0.008s
```

Esse comando deve criar um arquivo chamado *mandel.png*, que você pode visualizar com o programa de visualização de imagens do seu sistema ou em um navegador da web. Se tudo correu bem, deve ficar parecido com a Figura 2.6.

Na transcrição anterior, utilizamos o programa Unix `time` para analisar o tempo de execução do programa: levou cerca de cinco segundos no total para executar o cálculo de Mandelbrot em cada pixel da imagem. Mas quase todas as máquinas modernas têm processadores com vários núcleos, e esse programa usava apenas um. Se pudéssemos distribuir o trabalho por todos os recursos de computação que a máquina tem a oferecer, poderíamos concluir a imagem muito mais rapidamente.

Para isso, vamos dividir a imagem em seções, uma por processador, e deixar que cada processador cora os pixels atribuídos a ele. Para simplificar, vamos dividi-lo em faixas horizontais, conforme mostrado na Figura 2.7. Quando todos os processadores terminarem, podemos gravar os pixels no disco.

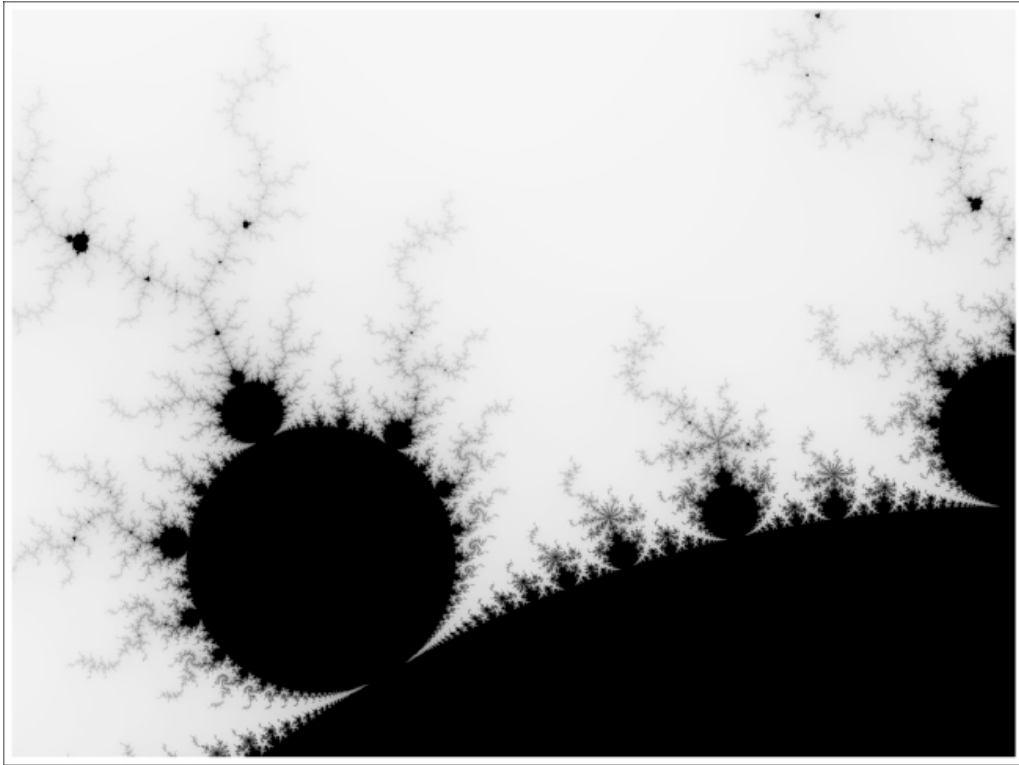


Figura 2.6: Resultados do programa paralelo de Mandelbrot.

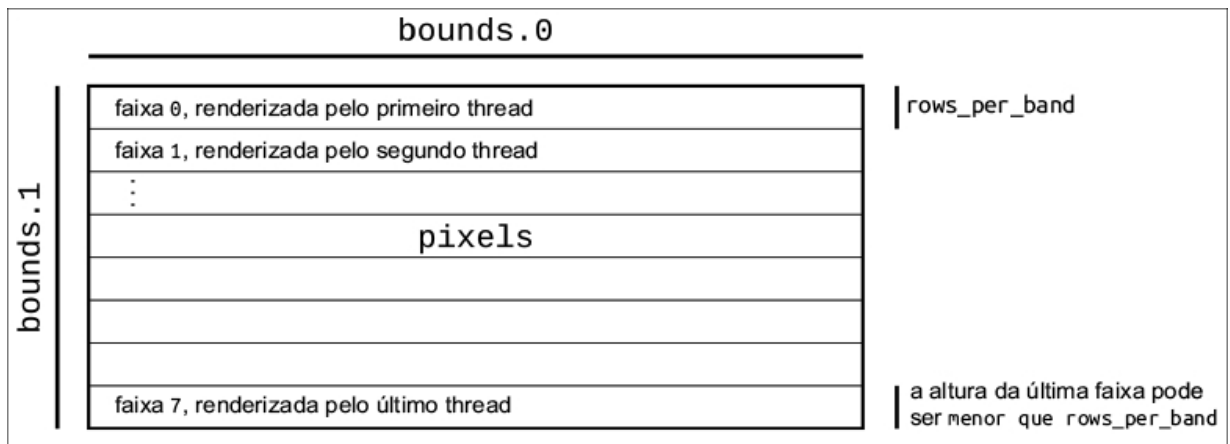


Figura 2.7: Dividindo o buffer de pixels em faixas para renderização paralela.

O crate `crossbeam` fornece vários recursos de concorrência valiosos, incluindo um recurso de thread com escopo que faz exatamente o que precisamos aqui. Para utilizá-lo, devemos adicionar a seguinte linha ao nosso arquivo *Cargo.toml*:

```
crossbeam = "0.8"
```

Então, precisamos pegar a chamada de linha única `render` e substituí-la pelo seguinte:

```
let threads = 8;
let rows_per_band = bounds.1 / threads + 1;

{
  let bands: Vec<&mut [u8]> =
    pixels.chunks_mut(rows_per_band * bounds.0).collect();
  crossbeam::scope(|spawner| {
    for (i, band) in bands.into_iter().enumerate() {
      let top = rows_per_band * i;
      let height = band.len() / bounds.0;
      let band_bounds = (bounds.0, height);
      let band_upper_left =
        pixel_to_point(bounds, (0, top), upper_left, lower_right);
      let band_lower_right =
        pixel_to_point(bounds, (bounds.0, top + height),
          upper_left, lower_right);
      spawner.spawn(move |_| {
        render(band, band_bounds, band_upper_left, band_lower_right);
      });
    }
  }).unwrap();
}
```

Dividindo isso da maneira usual:

```
let threads = 8;
let rows_per_band = bounds.1 / threads + 1;
```

Aqui decidimos utilizar oito threads.³ Em seguida, calculamos quantas linhas de pixels cada banda deve ter. Arredondamos a contagem de linhas para cima para garantir que as faixas cubram toda a imagem, mesmo que a altura não seja um múltiplo de `threads`.

```
let bands: Vec<&mut [u8]> =
  pixels.chunks_mut(rows_per_band * bounds.0).collect();
```

Aqui, dividimos o buffer de pixels em bandas. O método `chunks_mut` do buffer retorna um iterador produzindo fatias mutáveis e não sobrepostas do buffer, cada uma das quais inclui `rows_per_band * bounds.0` pixels – em outras palavras, `rows_per_band` linhas completas de pixels. A última fatia que `chunks_mut` produz pode conter menos linhas, mas cada linha conterá o mesmo número de pixels. Por fim, o método `collect` do iterador constrói um vetor contendo essas fatias mutáveis e não sobrepostas.

Agora podemos colocar a biblioteca `crossbeam` para trabalhar:

```
crossbeam::scope(|spawner| {  
    ...  
}).unwrap();
```

O argumento `|spawner| { ... }` é uma closure do Rust que espera um único argumento, `spawner`. Observe que, ao contrário das funções declaradas com `fn`, não precisamos declarar os tipos de argumentos de uma closure; o Rust vai inferi-los, com seu tipo de retorno. Nesse caso, `crossbeam::scope` chama a closure, passando como argumento um valor `spawner` que a closure pode utilizar para criar novos threads. A função `crossbeam::scope` espera que todos esses threads terminem a execução antes de retornar. Esse comportamento permite que o Rust tenha certeza de que tais threads não acessarão suas partes de pixels depois de ter saído do escopo, e nos permite ter certeza de que, quando `crossbeam::scope` retornar, a computação da imagem estará completa. Se tudo correr bem, `crossbeam::scope` retorna `Ok(())`, mas, se algum dos tópicos retornados gerar um pânico, ele retornará um `Err`. Chamamos `unwrap` sobre aquele `Result` para que, nesse caso, também geremos um pânico e o usuário receba um informe.

```
for (i, band) in bands.into_iter().enumerate() {
```

Aqui, iteramos sobre as faixas do buffer de pixels. O iterador `into_iter()` dá a cada iteração do corpo do loop a posse exclusiva de uma banda, garantindo que apenas um thread possa gravar nele por vez. Explicamos como isso funciona em detalhes no Capítulo 5. Então, o adaptador `enumerate` produz tuplas pareando cada elemento do vetor com seu índice.

```
let top = rows_per_band * i;  
let height = band.len() / bounds.0;  
let band_bounds = (bounds.0, height);  
let band_upper_left =  
    pixel_to_point(bounds, (0, top), upper_left, lower_right);  
let band_lower_right =  
    pixel_to_point(bounds, (bounds.0, top + height),  
                    upper_left, lower_right);
```

Fornecidos o índice e o tamanho real da banda (lembre-se de que a última pode ser mais curta que as outras), podemos produzir uma

caixa delimitadora do tipo que `render` requer, mas que referencia apenas essa banda do buffer, não a imagem inteira. Da mesma forma, reaproveitamos a função `pixel_to_point` do renderizador para descobrir onde os cantos superior esquerdo e inferior direito da faixa caem no plano complexo.

```
spawner.spawn(move |_| {  
    render(band, band_bounds, band_upper_left, band_lower_right);  
});
```

Por fim, criamos um thread, executando a closure `move |_| { ... }`. A palavra-chave `move` na frente indica que essa closure toma posse das variáveis que usa; em particular, apenas a closure pode utilizar a fatia mutável `band`. A lista de argumentos `|_|` significa que a closure recebe um argumento, o qual ela não usa (outro gerador para criar threads aninhadas).

Como mencionamos anteriormente, a chamada a `crossbeam::scope` garante que todos os threads tenham sido concluídos antes de retornar, o que significa que é seguro salvar a imagem em um arquivo, que é nossa próxima ação.

Executando o plotter de Mandelbrot

Utilizamos vários crates externos neste programa: `num` para aritmética de números complexos, `image` para gravar arquivos PNG e `crossbeam` para as primitivas de criação de thread com escopo. Eis aqui o arquivo *Cargo.toml* final, incluindo todas essas dependências:

```
[package]  
name = "mandelbrot"  
version = "0.1.0"  
edition = "2021"
```

```
[dependencies]  
num = "0.4"  
image = "0.13"  
crossbeam = "0.8"
```

Com isso pronto, podemos construir e executar o programa:

```
$ cargo build --release  
    Updating crates.io index  
    Compiling crossbeam-queue v0.3.2  
    Compiling crossbeam v0.8.1
```

```
Compiling mandelbrot v0.1.0 ($RUSTBOOK/mandelbrot)
Finished release [optimized] target(s) in ### secs
$ time target/release/mandelbrot mandel.png 4000x3000 -1.20,0.35 -1,0.20
real    0m1.436s
user    0m4.922s
sys     0m0.011s
```

Aqui, utilizamos `time` novamente para ver quanto tempo o programa demorou para ser executado; observe que, embora ainda gastássemos quase cinco segundos de tempo do processador, o tempo real decorrido foi de apenas cerca de 1,5 segundo. Você pode verificar se uma parte desse tempo é gasta gravando o arquivo de imagem comentando o código que faz isso e medindo novamente. No laptop em que esse código foi testado, a versão concorrente reduz o tempo de cálculo de Mandelbrot propriamente dito por um fator de quase quatro. Mostraremos como melhorar substancialmente isso no Capítulo 19.

Como antes, esse programa terá criado um arquivo chamado *mandel.png*. Com essa versão mais rápida, você pode explorar mais facilmente o conjunto de Mandelbrot alterando os argumentos da linha de comando ao seu gosto.

A segurança é invisível

No final, o programa paralelo com o qual terminamos não é substancialmente diferente do que poderíamos escrever em qualquer outra linguagem: distribuímos partes do buffer de pixels entre os processadores, deixamos cada um trabalhar em sua parte separadamente e, quando terminamos tudo, apresentamos o resultado. Então, o que há de tão especial no suporte à concorrência do Rust?

O que não mostramos aqui são todos os programas Rust que *não* podemos escrever. O código que examinamos neste capítulo particiona o buffer entre os threads corretamente, mas há muitas pequenas variações nesse código que não (e, portanto, introduzem corridas de dados); nenhuma dessas variações passará nas verificações estáticas do compilador Rust. Um compilador C ou C++ o ajudará alegremente a explorar o vasto espaço de programas com

corridas de dados sutis; o Rust avisa, desde o início, quando algo pode dar errado.

Nos capítulos 4 e 5, descreveremos as regras do Rust para segurança de memória. O Capítulo 19 explica como essas regras também garantem a higiene de concorrência adequada.

Sistemas de arquivos e ferramentas de linha de comando

O Rust encontrou um nicho significativo no mundo das ferramentas de linha de comando. Como uma linguagem de programação de sistemas moderna, segura e rápida, oferece aos programadores uma caixa de ferramentas que eles podem utilizar para montar interfaces de linha de comando que replicam ou estendem as funcionalidades das ferramentas existentes. Por exemplo, o comando `bat` fornece um `cat` alternativo com destaque de sintaxe e suporte integrado para ferramentas de paginação e `hyperfine` pode comparar automaticamente qualquer coisa que possa ser executada com um comando ou pipeline.

Embora algo tão complexo esteja fora do escopo deste livro, o Rust facilita a imersão no mundo dos aplicativos ergonômicos de linha de comando. Nesta seção, mostraremos como criar sua própria ferramenta de pesquisa e substituição completa, com saída colorida e mensagens de erro amigáveis.

Para começar, criaremos um novo projeto Rust:

```
$ cargo new quickreplace
   Created binary (application) `quickreplace` package
$ cd quickreplace
```

Para o nosso programa, precisaremos de mais dois crates: `text-colorizer` para criar uma saída colorida no terminal e `regex` para a funcionalidade real de pesquisa e substituição. Como antes, colocamos esses crates em *Cargo.toml* para instruir `cargo` que precisamos deles:

```
[package]
name = "quickreplace"
version = "0.1.0"
```



```
edition = "2021"
```

```
# Veja mais chaves e suas definições em
```

```
# https://doc.rust-lang.org/cargo/reference/manifest.html
```

```
[dependencies]
```

```
text-colorizer = "1"
```

```
regex = "1"
```

Os crates do Rust que atingiram a versão 1.0, como estes, seguem as regras de “versionamento semântico”: até que o número da versão principal 1 mude, as versões mais recentes devem sempre ser extensões compatíveis de seus predecessores. Portanto, se testarmos nosso programa em relação à versão 1.2 de algum crate, ainda deve funcionar com versões 1.3, 1.4, e assim por diante; mas a versão 2.0 poderia introduzir mudanças incompatíveis. Quando simplesmente solicitamos a versão “1” de uma crate em um arquivo *Cargo.toml*, o Cargo utilizará a versão mais recente disponível do crate antes de 2.0.

Interface de linha de comando

A interface do programa é bastante simples. São necessários quatro argumentos: uma string (ou expressão regular) para pesquisar, uma string (ou expressão regular) para substituí-la, o nome de um arquivo de entrada e o nome de um arquivo de saída. Vamos começar nosso arquivo *main.rs* com uma estrutura contendo estes argumentos:

```
#[derive(Debug)]
struct Arguments {
    target: String,
    replacement: String,
    filename: String,
    output: String,
}
```

O atributo `#[derive(Debug)]` diz ao compilador que gere um código extra que nos permite formatar o `struct Arguments` com `{:?}` em `println!`.

Caso o usuário insira o número errado de argumentos, é comum imprimir uma explicação concisa de como utilizar o programa. Vamos fazer isso com uma função simples chamada `print_usage` e importar

tudo de `text-colorizer` para que possamos adicionar um pouco de cor:

```
use text_colorizer::*;

fn print_usage() {
    eprintln!("{}", - change occurrences of one string into another",
        "quickreplace".green());
    eprintln!("Usage: quickreplace <target> <replacement> <INPUT> <OUTPUT>");
}
```

Apenas adicionar `.green()` ao final de uma string literal produz uma string envolvida pelos códigos de escape ANSI apropriados para exibir como verde em um emulador de terminal. Essa string é então interpolada no restante da mensagem antes de ser impressa.

Agora podemos coletar e processar os argumentos do programa:

```
use std::env;

fn parse_args() -> Arguments {
    let args: Vec<String> = env::args().skip(1).collect();

    if args.len() != 4 {
        print_usage();
        eprintln!("{}", wrong number of arguments: expected 4, got {}. ",
            "Error:".red().bold(), args.len());
        std::process::exit(1);
    }

    Arguments {
        target: args[0].clone(),
        replacement: args[1].clone(),
        filename: args[2].clone(),
        output: args[3].clone()
    }
}
```

Para obter os argumentos que o usuário inseriu, utilizamos o mesmo iterador `args` utilizado nos exemplos anteriores. `.skip(1)` ignora o primeiro valor do iterador (o nome do programa que está sendo executado) para que o resultado tenha apenas os argumentos da linha de comando.

O método `collect()` produz um `Vec` de argumentos. Em seguida, verificamos se o número correto está presente e, caso contrário, imprimimos uma mensagem e saímos com um código de erro. Além disso, colorimos novamente parte da mensagem e utilizamos `.bold()`

para colocar o texto em negrito. Se o número certo de argumentos estiver presente, nós os colocamos em um struct `Arguments` e o retornamos.

Então vamos acrescentar uma função `main` que apenas chama `parse_args` e imprime a saída:

```
fn main() {  
    let args = parse_args();  
    println!("{}", args);  
}
```

Neste ponto, podemos executar o programa e ver que ele apresenta a mensagem de erro correta:

```
$ cargo run  
Updating crates.io index  
Compiling libc v0.2.82  
Compiling lazy_static v1.4.0  
Compiling memchr v2.3.4  
Compiling regex-syntax v0.6.22  
Compiling thread_local v1.1.0  
Compiling aho-corasick v0.7.15  
Compiling atty v0.2.14  
Compiling text-colorizer v1.0.0  
Compiling regex v1.4.3  
Compiling quickreplace v0.1.0 (/home/jimb/quickreplace)  
Finished dev [unoptimized + debuginfo] target(s) in 6.98s  
Running `target/debug/quickreplace`  
quickreplace - change occurrences of one string into another  
Usage: quickreplace <target> <replacement> <INPUT> <OUTPUT>  
Error: wrong number of arguments: expected 4, got 0
```

Se você der alguns argumentos ao programa, ele imprimirá uma representação do struct `Arguments`:

```
$ cargo run "find" "replace" file output  
Finished dev [unoptimized + debuginfo] target(s) in 0.01s  
Running `target/debug/quickreplace find replace file output`  
Arguments { target: "find", replacement: "replace", filename: "file", output: "output" }
```

Este é um bom começo! Os argumentos são escolhidos corretamente e colocados nas partes corretas do struct `Arguments`.

Lendo e gravando arquivos

Em seguida, precisamos de alguma maneira de obter dados do sistema de arquivos para que possamos processá-los e gravá-los

quando terminarmos. O Rust tem um conjunto robusto de ferramentas para entrada e saída, mas os projetistas da biblioteca padrão sabem que a leitura e gravação de arquivos é muito comum e eles facilitaram isso de propósito. Tudo o que precisamos fazer é importar um módulo, `std::fs`, e temos acesso às funções `read_to_string` e `write`:

```
use std::fs;
```

`std::fs::read_to_string` retorna um `Result<String, std::io::Error>`. Se a função for bem-sucedida, ela produzirá uma `String`. Se falhar, produz um `std::io::Error`, o tipo da biblioteca padrão para representar problemas de E/S. De forma similar, `std::fs::write` retorna um `Result<(), std::io::Error>`: nada no caso de sucesso ou os mesmos detalhes de erro se algo der errado.

```
fn main() {
    let args = parse_args();

    let data = match fs::read_to_string(&args.filename) {
        Ok(v) => v,
        Err(e) => {
            eprintln!("{}", "failed to read from file '{}': {:?}",
                          "Error:".red().bold(), args.filename, e);
            std::process::exit(1);
        }
    };

    match fs::write(&args.output, &data) {
        Ok(_) => {},
        Err(e) => {
            eprintln!("{}", "failed to write to file '{}': {:?}",
                          "Error:".red().bold(), args.filename, e);
            std::process::exit(1);
        }
    };
}
```

Aqui, estamos utilizando a função `parse_args()` que escrevemos anteriormente e passando os nomes de arquivo resultantes para `read_to_string` e `write`. As instruções `match` nas saídas dessas funções lidam com erros normalmente, imprimindo o nome do arquivo, o motivo fornecido para o erro e um pequeno toque de cor para chamar a atenção do usuário.

Com essa função `main` atualizada, podemos executar o programa e ver que, naturalmente, o conteúdo dos arquivos novos e antigos é exatamente o mesmo:

```
$ cargo run "find" "replace" Cargo.toml Copy.toml
Compiling quickreplace v0.1.0 (/home/jimb/rust/quickreplace)
Finished dev [unoptimized + debuginfo] target(s) in 0.01s
Running `target/debug/quickreplace find replace Cargo.toml Copy.toml`
```

O programa lê o arquivo de entrada *Cargo.toml*, e grava no arquivo de saída *Copy.toml*, mas, como não escrevemos nenhum código para realmente localizar e substituir, nada na saída foi alterado. Podemos verificar facilmente executando o comando `diff`, que não detecta diferenças:

```
$ diff Cargo.toml Copy.toml
```

Localizar e substituir

O toque final para este programa é implementar sua funcionalidade real: localizar e substituir. Para isso, utilizaremos o crate `regex`, que compila e executa expressões regulares. Ele fornece uma estrutura chamada `Regex`, que representa uma expressão regular compilada. `Regex` tem um método `replace_all`, que faz exatamente o que diz: procura em uma string todas as correspondências da expressão regular e substitui cada uma por uma determinada string substituta. Podemos extrair essa lógica em uma função:

```
use regex::Regex;
fn replace(target: &str, replacement: &str, text: &str)
    -> Result<String, regex::Error>
{
    let regex = Regex::new(target)?;
    Ok(regex.replace_all(text, replacement).to_string())
}
```

Observe o tipo de retorno dessa função. Assim como as funções de biblioteca padrão que utilizamos anteriormente, `replace` retorna um `Result`, desta vez com um tipo de erro fornecido pelo crate `regex`.

`Regex::new` compila o `regex` fornecido pelo usuário e pode falhar se receber uma string inválida. Como no programa de Mandelbrot, utilizamos `?` para encurtar o processo caso `Regex::new` falhe, mas nesse caso a função retorna um tipo de erro específico do crate `regex`.

Depois que o regex é compilado, seu método `replace_all` substitui quaisquer correspondências em `text` pela string `replacement` fornecida.

Se `replace_all` encontrar correspondências, ele retornará uma nova `String` com essas correspondências substituídas pelo texto que lhe fornecemos. Por outro lado, `replace_all` retorna um ponteiro para o texto original, evitando alocação de memória e cópia desnecessárias. Nesse caso, porém, sempre queremos uma cópia independente, então utilizamos o método `to_string` para obter uma `String` em ambos os casos e retornar essa string envolvida em `Result::Ok`, como nas outras funções.

Agora, é hora de incorporar a nova função em nosso código `main`:

```
fn main() {
    let args = parse_args();

    let data = match fs::read_to_string(&args.filename) {
        Ok(v) => v,
        Err(e) => {
            eprintln!("{}", "failed to read from file '{}': {:?}",
                "Error:".red().bold(), args.filename, e);
            std::process::exit(1);
        }
    };

    let replaced_data = match replace(&args.target, &args.replacement, &data) {
        Ok(v) => v,
        Err(e) => {
            eprintln!("{}", "failed to replace text: {:?}",
                "Error:".red().bold(), e);
            std::process::exit(1);
        }
    };

    match fs::write(&args.output, &replaced_data) {
        Ok(v) => v,
        Err(e) => {
            eprintln!("{}", "failed to write to file '{}': {:?}",
                "Error:".red().bold(), args.filename, e);
            std::process::exit(1);
        }
    };
}
```

Com este toque final, o programa está pronto e você poderá testá-

lo:

```
$ echo "Hello, world" > test.txt
$ cargo run "world" "Rust" test.txt test-modified.txt
  Compiling quickreplace v0.1.0 (/home/jimb/rust/quickreplace)
  Finished dev [unoptimized + debuginfo] target(s) in 0.88s
  Running `target/debug/quickreplace world Rust test.txt test-modified.txt`

$ cat test-modified.txt
Hello, Rust
```

E, naturalmente, o tratamento de erros também está em vigor, relatando os erros de maneira elegante ao usuário:

```
$ cargo run "[[a-z]" "0" test.txt test-modified.txt
  Finished dev [unoptimized + debuginfo] target(s) in 0.01s
  Running `target/debug/quickreplace '[a-z]' 0 test.txt test-modified.txt`
Error: failed to replace text: Syntax(
~~~~~
~~~~~
regex parse error:
  [[a-z]
  ^
error: unclosed character class
~~~~~
~~~~~
)
```

É claro, há muitos recursos faltando nessa demonstração simples, mas os fundamentos foram apresentados. Você viu como ler e gravar arquivos, propagar e exibir erros e colorir a saída para melhorar a experiência do usuário no terminal.

Nos próximos capítulos serão exploradas técnicas mais avançadas para desenvolvimento de aplicativos, desde coleções de dados e programação funcional com iteradores até técnicas de programação assíncrona para concorrência extremamente eficiente, mas primeiro você precisará da base sólida do próximo capítulo sobre os tipos de dados fundamentais do Rust.

¹ N.R.: Boilerplates são seções de código que fornecem uma estrutura básica (esqueleto), que pode ser utilizada repetidamente com pouca ou nenhuma alteração, sendo um ponto de partida, a partir do qual você faz o resto.

² N.R.: Em caso de sucesso, o valor da variável output será igual a f passado no valor retornado por File::create, ou seja, Ok(f) quando o arquivo é aberto corretamente. Em caso de falha, (File::create retorna Err(e)) retorna o erro para a função que chamou

`write_file.`

[3](#) A crate `num_cpus` fornece uma função que retorna o número de CPUs disponíveis no sistema atual.

CAPÍTULO 3

Tipos fundamentais

Existem muitos, muitos tipos de livros no mundo, o que faz muito sentido, porque existem muitos, muitos tipos de pessoas e todo mundo quer ler algo diferente.

– Lemony Snicket

Em grande medida, a linguagem Rust é projetada em torno de seus tipos. Seu suporte para código de alto desempenho surge ao permitir que os desenvolvedores escolham a representação de dados que melhor se adapta à situação, com o equilíbrio certo entre simplicidade e custo. As garantias de segurança de memória e thread do Rust também se baseiam na solidez de seu sistema de tipos, e a flexibilidade do Rust decorre de seus tipos e traits genéricos.

Este capítulo aborda os tipos fundamentais do Rust para representar valores. Esses tipos no nível de código-fonte têm contrapartes concretas no nível de máquina, com custos e desempenho previsíveis. Embora o Rust não prometa que representará as coisas exatamente como você solicitou, ele toma o cuidado de se desviar de suas solicitações apenas quando é uma melhoria confiável.

Comparado a uma linguagem de tipagem dinâmica como JavaScript ou Python, o Rust requer mais planejamento de sua parte. Você deve especificar os tipos de argumento de função e valores de retorno, campos struct e algumas outras construções (constructs). No entanto, dois recursos do Rust tornam isso menos problemático do que você poderia esperar:

- A partir dos tipos que você explicita, a *inferência de tipo* do Rust vai descobrir a maior parte do restante para você. Na prática, geralmente há apenas um tipo que funcionará para uma determinada variável ou expressão; quando esse é o caso, o Rust

permite omitir, ou *elidir*, o tipo. Por exemplo, você pode explicitar cada tipo em uma função, assim:

```
fn build_vector() -> Vec<i16> {  
    let mut v: Vec<i16> = Vec::<i16>::new();  
    v.push(10i16);  
    v.push(20i16);  
    v  
}
```

Mas isso é confuso e repetitivo. Fornecido o tipo de retorno da função, é óbvio que `v` deve ser um `Vec<i16>`, um vetor de inteiros de 16 bits com sinal: nenhum outro tipo funcionaria. E disso segue que cada elemento do vetor deve ser um `i16`. Esse é exatamente o tipo de raciocínio que a inferência de tipo do Rust aplica, permitindo que, em vez disso, você escreva:

```
fn build_vector() -> Vec<i16> {  
    let mut v = Vec::new();  
    v.push(10);  
    v.push(20);  
    v  
}
```

Essas duas definições são exatamente equivalentes, e o Rust vai gerar de qualquer maneira o mesmo código de máquina. A inferência de tipo devolve grande parte da legibilidade de linguagens tipadas dinamicamente, ao mesmo tempo que detecta erros de tipo em tempo de compilação.

- As funções podem ser *genéricas*: uma única função pode operar em valores de muitos tipos diferentes.

No Python e JavaScript, todas as funções operam dessa maneira naturalmente: uma função pode operar em qualquer valor que tenha as propriedades e os métodos necessários à função. (Essa é a característica frequentemente chamada *tipagem pato* (*duck typing*): se grasna como um pato, é um pato.) Mas é exatamente essa flexibilidade que torna tão difícil que essas linguagens detectem antecipadamente erros de tipo; testes geralmente são a única maneira de detectar esses erros. As funções genéricas do Rust dão à linguagem um grau da mesma flexibilidade e, ao mesmo tempo, ainda detectam todos os erros de tipo em tempo

de compilação.

Apesar de sua flexibilidade, as funções genéricas são tão eficientes quanto suas contrapartes não genéricas. Não há nenhuma vantagem de desempenho inerente ao escrever, digamos, uma função `sum` para cada número inteiro em relação a escrever uma genérica que lida com todos os números inteiros. Discutiremos funções genéricas em detalhes no Capítulo 11.

O restante deste capítulo aborda os tipos do Rust de baixo para cima, começando com tipos numéricos simples, como inteiros e valores de ponto flutuante, passando para tipos que armazenam mais dados: boxes, tuplas, arrays e strings.

Eis um resumo dos tipos que você verá no Rust. A Tabela 3.1 mostra os tipos primitivos do Rust, alguns tipos muito comuns da biblioteca padrão e alguns exemplos de tipos definidos pelo usuário.

Tabela 3.1: Exemplos de tipos no Rust

Tipo	Descrição	Valores
i8, i16, i32, i64, i128 u8, u16, u32, u64, u128	Inteiros com sinal e sem sinal, de determinada largura ou tamanho em bits	42, -5i8, 0x400u16, 0o100i16, 20_922_789_888_000u64, b'*' (literal de byte u8)
isize, usize	Inteiros com sinal e sem sinal, do mesmo tamanho de um endereço na máquina (32 ou 64 bits)	137, -0b0101_0010isize, 0xffff_fc00usize
f32, f64	Números de ponto flutuante IEEE, precisão simples e dupla	1.61803, 3.14f32, 6.0221e23f64
bool	Boolean	true, false
char	Caractere Unicode, 32 bits de largura	'*', '\n', '字', '\x7f', '\u{CA0}'
(char, u8, i32)	Tupla: tipos mistos permitidos	('%', 0x7f, -1)
()	"Unidade" (tupla vazia)	()
struct S { x: f32, y: f32 }	Struct de campo nomeado	S { x: 120.0, y: 209.0 }
struct T (i32, char);	Struct tipo tupla	T(120, 'X')
struct E;	Struct tipo unidade; não tem campos	E
enum Attend { OnTime, Late(u32) }	Enumeração, tipo de dados algébricos	Attend::Late(5), Attend::OnTime
Box<Attend>	Box: ponteiro proprietário para valor no heap	Box::new(Late(15))

Tipo	Descrição	Valores
<code>&i32, &mut i32</code>	Referências compartilhadas e mutáveis: ponteiros não proprietários que não devem durar mais que seu referente	<code>&s.y, &mut v</code>
<code>String</code>	<code>String</code> UTF-8, dimensionada dinamicamente	<code>"ラーメン: ramen".to_string()</code>
<code>&str</code>	Referência a <code>str</code> : ponteiro não proprietário para texto UTF-8	<code>"そば: soba", &s[0..12]</code>
<code>[f64; 4], [u8; 256]</code>	Array, comprimento fixo; todos os elementos do mesmo tipo	<code>[1.0, 0.0, 0.0, 1.0], [b' '; 256]</code>
<code>Vec<f64></code>	Vetor, comprimento variável; todos os elementos do mesmo tipo	<code>vec![0.367, 2.718, 7.389]</code>
<code>&[u8], &mut [u8]</code>	Referência a fatia: referência a uma parte de um array ou vetor, compreendendo ponteiro e comprimento	<code>&v[10..20], &mut a[..]</code>
<code>Option<&str></code>	Valor opcional: ou <code>None</code> (ausente) ou <code>Some(v)</code> (presente, com valor <code>v</code>)	<code>Some("Dr."), None</code>
<code>Result<u64, Error></code>	Resultado da operação que pode falhar: um valor de sucesso <code>Ok(v)</code> , ou um erro <code>Err(e)</code>	<code>Ok(4096), Err(Error::last_os_error())</code>
<code>&dyn Any, &mut dyn Read</code>	Objeto <code>Trait</code> : referência a qualquer valor que implementa um determinado conjunto de métodos	<code>value as &dyn Any, &mut file as &mut dyn Read</code>
<code>fn(&str) -> bool</code>	Ponteiro para função	<code>str::is_empty</code>
(Os tipos <code>closure</code> não têm forma escrita)	Closure	<code> a, b { a*a + b*b }</code>

A maioria desses tipos é abordada neste capítulo, exceto os seguintes:

- Dedicamos a tipos `struct` um capítulo próprio, o Capítulo 9.
- Dedicamos a tipos enumerados um capítulo próprio, o Capítulo 10.
- Descrevemos os objetos `trait` no Capítulo 11.
- Descrevemos os fundamentos de `String` e `&str` aqui, mas fornecemos mais detalhes no Capítulo 17.
- Abordamos os tipos de função e `closure` no Capítulo 14.

Tipos numéricos de tamanho fixo

A base do sistema de tipos do Rust é uma coleção de tipos numéricos de tamanho fixo, escolhidos para corresponder aos tipos que quase todos os processadores modernos implementam diretamente no hardware.

Tipos numéricos de tamanho fixo podem estourar (overflow) ou perder precisão, mas são adequados para a maioria dos aplicativos e podem ser milhares de vezes mais rápidos do que representações como números inteiros de precisão arbitrária e números racionais exatos. Se precisar desses tipos de representações numéricas, eles são suportados no crate `num`.

Os nomes dos tipos numéricos do Rust seguem um padrão regular, explicando sua largura em bits e a representação que utilizam (Tabela 3.2).

Tabela 3.2: Tipos numéricos do Rust

Tamanho (bits)	Número inteiro sem sinal	Número inteiro com sinal	Ponto flutuante
8	u8	i8	
16	u16	i16	
32	u32	i32	f32
64	u64	i64	f64
128	u128	i128	
Palavra da máquina	usize	isize	

Aqui uma palavra (word) de máquina é um valor do tamanho de um endereço na máquina em que o código é executado, 32 ou 64 bits.

Tipos inteiros

Os tipos inteiros sem sinal do Rust utilizam um intervalo completo para representar valores positivos e zero (Tabela 3.3).

Tabela 3.3: Tipos inteiros sem sinal do Rust

Tipo	Intervalo
u8	0 a 2^8-1 (0 a 255)

Tipo	Intervalo
u16	0 a $2^{16}-1$ (0 a 65.535)
u32	0 a $2^{32}-1$ (0 a 4.294.967.295)
u64	0 a $2^{64}-1$ (0 a 18.446.744.073.709.551.615 ou 18 quintilhões)
u128	0 a $2^{128}-1$ (0 a aproximadamente de $3,4 \times 10^{38}$)
usize	0 a $2^{32}-1$ ou $2^{64}-1$

Os tipos inteiros com sinal do Rust utilizam a representação de complemento de dois, usando os mesmos padrões de bits que o tipo sem sinal correspondente, para abranger um intervalo de valores positivos e negativos (Tabela 3.4).

Tabela 3.4: Os tipos inteiros com sinal do Rust

Tip o	Intervalo
i8	-2^7 para 2^7-1 (-128 a 127)
i16	-2^{15} para $2^{15}-1$ (-32,768 a 32,767)
i32	-2^{31} para $2^{31}-1$ (-2.147.483.648 a 2.147.483.647)
i64	-2^{63} para $2^{63}-1$ (-9.223.372.036.854.775.808 a 9.223.372.036.854.775.807)
i128	-2^{127} para $2^{127}-1$ (aproximadamente $-1,7 \times 10^{38}$ para $1,7 \times 10^{38}$)
isize	Ou -2^{31} para $2^{31}-1$, ou -2^{63} para $2^{63}-1$

O Rust utiliza o tipo `u8` para valores de byte. Por exemplo, ler dados de um arquivo binário ou soquete gera um fluxo de valores `u8`.

Ao contrário do C e C++, o Rust trata caracteres como distintos dos tipos numéricos: um `char` não é um `u8`, nem é um `u32` (embora tenha 32 bits de comprimento). Descrevemos o tipo `char` do Rust em “Caracteres”, na página [84](#).

Os tipos `usize` e `isize` são análogos a `size_t` e `ptrdiff_t` em C e C++. Sua precisão corresponde ao tamanho do espaço de endereço na máquina de destino: eles têm 32 bits em arquiteturas de 32 bits e 64 bits em arquiteturas de 64 bits. O Rust requer que os índices do

array sejam valores `usize`. Valores que representam os tamanhos de arrays, vetores ou contagens do número de elementos em alguma estrutura de dados geralmente também têm o tipo `usize`.

Literais de inteiro no Rust podem receber um sufixo indicando seu tipo: `42u8` é um valor `u8` e `1729isize` é um `isize`. Se um literal de inteiro não possui um sufixo de tipo, o Rust adia a determinação do tipo até encontrar o valor que está sendo utilizado de uma maneira que o resolva: armazenado em uma variável de um tipo específico, passado para uma função que espera um tipo específico, comparado com outro valor de um tipo específico, ou algo assim. No final, se múltiplos tipos puderem funcionar, o Rust assume o padrão de `i32` se isso estiver entre as possibilidades. Caso contrário, o Rust relata a ambiguidade como um erro.

Os prefixos `0x`, `0o` e `0b` designam literais hexadecimais, octais e binários.

Para tornar os números longos mais legíveis, você pode inserir sublinhados entre os dígitos. Por exemplo, você pode escrever o maior valor `u32` como `4_294_967_295`. A colocação exata dos sublinhados não é significativa, portanto você pode dividir números hexadecimais ou binários em grupos de quatro dígitos em vez de três, como em `0xffff_ffff`, ou definir o sufixo de tipo dos dígitos, como em `127_u8`. Alguns exemplos de literais de inteiro são ilustrados na Tabela 3.5.

Tabela 3.5: Exemplos de literais de inteiro

Literal	Tipo	Valor decimal
<code>116i8</code>	<code>i8</code>	116
<code>0xcafeu32</code>	<code>u32</code>	51966
<code>0b0010_1010</code>	Inferido	42
<code>0o106</code>	Inferido	70

Embora os tipos numéricos e o tipo `char` sejam distintos, o Rust fornece literais de `byte`, literais semelhantes a caracteres para valores `u8`: `b'X'` representa o código ASCII para o caractere `x`, como

um valor `u8`. Por exemplo, como o código ASCII para `A` é 65, os literais `b'A` e `65u8` são exatamente equivalentes. Somente caracteres ASCII podem aparecer em literais de byte.

Existem alguns caracteres que você não pode simplesmente inserir após as aspas simples, porque isso seria sintaticamente ambíguo ou difícil de ler. Os caracteres na Tabela 3.6 só podem ser escritos utilizando uma notação substituta, introduzida por uma barra invertida.

Tabela 3.6: Caracteres que requerem uma notação substituta

Caractere	Literal de byte	Equivalente numérico
Aspas simples, <code>'</code>	<code>b'\"</code>	<code>39u8</code>
Barra invertida, <code>\</code>	<code>b'\"'</code>	<code>92u8</code>
Nova linha	<code>b'\\n'</code>	<code>10u8</code>
Retorno de carro	<code>b'\\r'</code>	<code>13u8</code>
Tab	<code>b'\\t'</code>	<code>9u8</code>

Para caracteres difíceis de escrever ou ler, você pode escrever o código em hexadecimal. Um literal de byte da forma `b'\\xHH'`, em que `HH` é qualquer número hexadecimal de dois dígitos, representa o byte cujo valor é `HH`. Por exemplo, você pode escrever um literal de byte para o caractere de controle de “escape” ASCII como `b'\\x1b'`, já que o código ASCII para “escape” é 27, ou 1B em hexadecimal. Como os literais de byte são apenas outra notação para valores `u8`, considere se um literal numérico simples pode ser mais legível: provavelmente faz sentido utilizar `b'\\x1b'` em vez de simplesmente 27 somente quando você deseja enfatizar que o valor representa um código ASCII.

Você pode converter de um tipo inteiro em outro utilizando o operador `as`. Explicamos como funcionam as conversões em “Coerções de tipo”, na página [185](#), mas eis alguns exemplos:

```
assert_eq!( 10_i8 as u16, 10_u16); // no intervalo
assert_eq!( 2525_u16 as i16, 2525_i16); // no intervalo

assert_eq!( -1_i16 as i32, -1_i32); // sinal estendido
```



```
assert_eq!(65535_u16 as i32, 65535_i32); // zero estendido

// Conversões que estão fora do intervalo para o destino produzem valores
// que são equivalentes ao módulo do original com 2^N, onde N é a largura
// do destino em bits. Isso às vezes é chamado "truncamento"
assert_eq!(1000_i16 as u8, 232_u8);
assert_eq!(65535_u32 as i16, -1_i16);

assert_eq!(-1_i8 as u8, 255_u8);
assert_eq!(255_u8 as i8, -1_i8);
```

A biblioteca padrão fornece algumas operações como métodos em números inteiros. Por exemplo:

```
assert_eq!(2_u16.pow(4), 16);           // exponenciação
assert_eq!((-4_i32).abs(), 4);          // valor absoluto
assert_eq!(0b101101_u8.count_ones(), 4); // contagem de população
```

Você pode encontrá-los na documentação on-line. Observe, porém, que a documentação contém páginas separadas para o próprio tipo em `"i32 (tipo primitivo)"` e para o módulo dedicado a esse tipo (procure `"std::i32"`).

No código real, você geralmente não precisará escrever explicitamente os sufixos de tipo como fizemos aqui, porque o contexto determinará o tipo. Quando isso não acontece, porém, as mensagens de erro podem ser surpreendentes. Por exemplo, o seguinte não compila:

```
println!("{}", (-4).abs());
```

O Rust reclama:

```
error: can't call method `abs` on ambiguous numeric type `{integer}`
```

Isso pode ser um pouco confuso: todos os tipos inteiros com sinal têm um método `abs`, então qual é o problema? Por razões técnicas, o Rust quer saber exatamente qual tipo inteiro um valor tem antes de chamar os próprios métodos do tipo. O padrão de `i32` é aplicado apenas se o tipo ainda for ambíguo depois que todas as chamadas de método foram resolvidas, então é tarde demais para ajudar aqui. A solução é especificar qual tipo você pretende, seja com um sufixo ou utilizando a função de um tipo específico:

```
println!("{}", (-4_i32).abs());
println!("{}", i32::abs(-4));
```

Observe que as chamadas de método têm precedência maior do que

os operadores de prefixo unário, portanto tenha cuidado ao aplicar métodos a valores invertidos (negativos). Sem os parênteses ao redor de `-4_i32` na primeira declaração, `-4_i32.abs()` aplicaria o método `abs` ao valor positivo 4, produzindo 4 positivo e então negaria isso, produzindo -4.

Aritmética: verificação, enquadramento, saturação e estouro

Quando uma operação aritmética de inteiros estoura, o Rust gera um pânico, em uma compilação de depuração (debug). Em uma versão de release, a operação é enquadrada¹: produz o valor equivalente ao módulo do resultado matematicamente correto com o intervalo do valor. (Em nenhum dos casos o estouro é comportamento indefinido, como em C e C++.)

Por exemplo, o código a seguir gera um pânico em uma compilação de depuração:

```
let mut i = 1;
loop {
    i *= 10; // pânico: tentativa de multiplicar com estouro
             // (mas apenas em compilações de depuração!)
}
```

Em uma versão de release (build), essa multiplicação é enquadrada em um número negativo e o loop é executado indefinidamente.

Quando esse comportamento padrão não é o que você precisa, os tipos inteiros fornecem métodos que permitem especificar exatamente o que você deseja. Por exemplo, o seguinte gera um pânico em qualquer compilação:

```
let mut i: i32 = 1;
loop {
    // pânico: multiplicação estourou (em qualquer compilação)
    i = i.checked_mul(10).expect("multiplication overflowed");
}
```

Esses métodos aritméticos de inteiros enquadram-se em quatro categorias gerais:

- Operações *verificadas* (*checked*) retornam uma `Option` do resultado: `Some(v)` se o resultado matematicamente correto puder

ser representado como um valor desse tipo, ou `None` se não puder.
Por exemplo:

```
// A soma de 10 e 20 pode ser representada como um u8
assert_eq!(10_u8.checked_add(20), Some(30));

// Infelizmente, a soma de 100 e 200 não pode
assert_eq!(100_u8.checked_add(200), None);

// Efetua a soma; gera um pânico se estourar
let sum = x.checked_add(y).unwrap();

// Estranhamente, a divisão com sinal também pode estourar, em um caso particular
// Um tipo de n bits com sinal pode representar  $-2^{n-1}$ , mas não  $2^{n-1}$ 
assert_eq!((-128_i8).checked_div(-1), None);
```

- Operações de *enquadramento* retornam o valor equivalente ao módulo do resultado matematicamente correto com o intervalo do valor:

```
// O primeiro produto pode ser representado como um u16;
// o segundo não pode, então obtemos 250000 módulo  $2^{16}$ 
assert_eq!(100_u16.wrapping_mul(200), 20000);
assert_eq!(500_u16.wrapping_mul(500), 53392);

// As operações em tipos com sinal podem encapsular valores negativos
assert_eq!(500_i16.wrapping_mul(500), -12144);

// Em operações de deslocamento bit a bit, a distância de deslocamento
// é enquadrada para cair dentro do tamanho do valor
// Portanto, um deslocamento de 17 bits em um tipo de 16 bits é um
// deslocamento de 1 ( $17 \% 16$ )
assert_eq!(5_i16.wrapping_shl(17), 10);
```

Como explicado, é assim que os operadores aritméticos comuns se comportam em versões de release. A vantagem desses métodos é que eles se comportam da mesma maneira em todas as compilações.

- Operações *saturating* (saturante) retornam o valor representável que está mais próximo do resultado matematicamente correto. Em outras palavras, o resultado é limitado aos valores máximos e mínimos que o tipo pode representar:

```
assert_eq!(32760_i16.saturating_add(10), 32767);
assert_eq!((-32760_i16).saturating_sub(10), -32768);
```

Não há métodos de saturação de divisão, resto ou deslocamento bit a bit.

- Operações que resultam em um *estouro* (*overflow/underflow*) retornam uma tupla (*result, overflowed*), em que *result* é o que a versão de encapsulamento da função retornaria e *overflowed* é um *bool* indicando se ocorreu um estouro:

```
assert_eq!(255_u8.overflowing_sub(2), (253, false));
assert_eq!(255_u8.overflowing_add(2), (1, true));
```

overflowing_shl e *overflowing_shr* desviam um pouco do padrão: eles retornam *true* em *overflowed* somente se a distância de deslocamento for tão grande ou maior que o tamanho de bits do próprio tipo. O deslocamento real aplicado é o módulo do deslocamento solicitado com a largura de bits do tipo:

```
// Um deslocamento de 17 bits é muito grande para `u16` e 17 módulo 16 é 1
assert_eq!(5_u16.overflowing_shl(17), (10, true));
```

Os nomes das operações que seguem o prefixo *checked_*, *wrapping_*, *saturating_* OU *overflowing_* são mostrados na Tabela 3.7.

Tabela 3.7: Nomes de operação

Operação	Sufixo de nome	Exemplo
Adição	<i>add</i>	<code>100_i8.checked_add(27) == Some(127)</code>
Subtração	<i>sub</i>	<code>10_u8.checked_sub(11) == None</code>
Multiplicação	<i>mul</i>	<code>128_u8.saturating_mul(3) == 255</code>
Divisão	<i>div</i>	<code>64_u16.wrapping_div(8) == 8</code>
Resto	<i>rem</i>	<code>(-32768_i16).wrapping_rem(-1) == 0</code>
Negação	<i>neg</i>	<code>(-128_i8).checked_neg() == None</code>
Valor absoluto	<i>abs</i>	<code>(-32768_i16).wrapping_abs() == -32768</code>
Exponenciação	<i>pow</i>	<code>3_u8.checked_pow(4) == Some(81)</code>
Deslocamento à esquerda bit a bit	<i>shl</i>	<code>10_u32.wrapping_shl(34) == 40</code>
Deslocamento à direita bit a bit	<i>shr</i>	<code>40_u64.wrapping_shr(66) == 10</code>

Tipos de ponto flutuante

O Rust fornece tipos de ponto flutuante de precisão simples e dupla do IEEE. Esses tipos incluem infinitos positivo e negativo, valores zero positivos e negativos distintos e um valor *não-é-um-número*

(*not-a-number*) (Tabela 3.8).

Tabela 3.8: Tipos de ponto flutuante de precisão simples e dupla do IEEE

Tip o	Precisão	Intervalo
f32	Precisão única IEEE (pelo menos 6 dígitos decimais)	Aproximadamente $-3,4 \times 10^{38}$ a $+3,4 \times 10^{38}$
f64	Precisão dupla IEEE (pelo menos 15 dígitos decimais)	Aproximadamente $-1,8 \times 10^{308}$ a $1,8 \times 10^{308}$

Os tipos `f32` e `f64` do Rust correspondem aos tipos `float` e `double` em C e C++ (em implementações que suportam ponto flutuante IEEE), bem como Java (que sempre utiliza ponto flutuante IEEE).

Literais de ponto flutuante têm a forma geral diagramada na Figura 3.1.

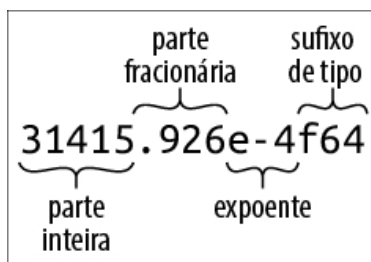


Figura 3.1: Um literal de ponto flutuante.

Cada parte de um número de ponto flutuante após a parte inteira é opcional, mas pelo menos uma parte fracionária, expoente ou sufixo de tipo deve estar presente, para distingui-la de um literal de inteiro. A parte fracionária pode consistir em um único ponto decimal, então `5.` é uma constante de ponto flutuante válida.

Se um literal de ponto flutuante não tiver um sufixo de tipo, o Rust verificará o contexto para ver como os valores são utilizados, da mesma forma que ocorre com os literais de inteiro. Se, por fim, ele descobrir que qualquer um dos tipos de ponto flutuante pode servir, ele escolhe `f64` por padrão.

Para fins de inferência de tipo, o Rust trata literais de inteiro e literais de ponto flutuante como classes distintas: ele nunca inferirá

um tipo de ponto flutuante para um literal de inteiro ou vice-versa. A Tabela 3.9 mostra alguns exemplos de literais de ponto flutuante.

Tabela 3.9: Exemplos de literais de ponto flutuante

Literal	Tipo	Valor matemático
-1.5625	Inferido	$-(1^{9/116})$
2.	Inferido	2
0.25	Inferido	1/4
1e4	Inferido	10.000
40f32	f32	40
9.109_383_56e-31f64	f64	Aproximadamente $9,10938356 \times 10^{-31}$

Os tipos f32 e f64 têm constantes associadas para os valores especiais exigidos pelo IEEE, como INFINITY, NEG_INFINITY (infinito negativo), NAN (o valor não-é-um-número), e MIN e MAX (os maiores e menores valores finitos):

```
assert!((-1. / f32::INFINITY).is_sign_negative());  
assert_eq!(-f32::MIN, f32::MAX);
```

Os tipos f32 e f64 fornecem um conjunto completo de métodos para cálculos matemáticos; por exemplo, `2f64.sqrt()` é a raiz quadrada de precisão dupla de dois. Alguns exemplos:

```
assert_eq!(5f32.sqrt() * 5f32.sqrt(), 5.); // exatamente 5,0, pelo IEEE  
assert_eq!((-1.01f64).floor(), -2.0);
```

Novamente, as chamadas de método têm precedência maior do que os operadores de prefixo; portanto, certifique-se de colocar corretamente entre parênteses as chamadas de método em valores negativos.

Os módulos `std::f32::consts` e `std::f64::consts` fornecem várias constantes matemáticas comumente utilizadas como `E`, `PI` e a raiz quadrada de dois.

Ao pesquisar a documentação, lembre-se de que existem páginas para ambos os tipos, denominados “f32 (tipo primitivo)” e “f64 (tipo primitivo)” e os módulos para cada tipo, `std::f32` e `std::f64`.

Como acontece com números inteiros, você geralmente não precisará escrever explicitamente sufixos de tipo em literais de ponto flutuante no código real, mas, quando precisar, colocar um tipo no literal ou na função será suficiente:

```
println!("{}", (2.0_f64).sqrt());  
println!("{}", f64::sqrt(2.0));
```

Ao contrário do C e C++, o Rust quase não realiza conversões numéricas implicitamente. Se uma função espera um argumento `f64`, é um erro passar um valor `i32` como argumento. Na verdade, o Rust nem mesmo converte implicitamente um valor `i16` em um valor `i32`, embora cada valor `i16` também seja um valor `i32`. Mas você sempre pode escrever *conversões explícitas* utilizando o operador `as`: `i as f64` OU `x as i32`.

A falta de conversões implícitas às vezes torna uma expressão Rust mais prolixa do que seria um código em C ou C++ análogo. Contudo, as conversões implícitas de inteiros têm um histórico bem estabelecido de causar bugs e falhas de segurança, especialmente quando os inteiros em questão representam o tamanho de algo na memória e ocorre um estouro imprevisto. Em nossa experiência, o ato de escrever explicitamente conversões numéricas no Rust nos alertou para problemas que, de outra forma, não teríamos percebido.

Explicamos exatamente como as conversões se comportam em “Coerções de tipo”, na página [185](#).

Tipo bool

Tipo booleano do Rust, `bool`, tem os dois valores usuais para esses tipos, `true` e `false`. Operadores de comparação como `==` e `<` produzem resultados `bool`: o valor de `2 < 5` é `true`.

Muitas linguagens são indulgentes com o uso de valores de outros tipos em contextos que exigem um valor booleano: C e C++ convertem implicitamente caracteres, inteiros, números de ponto flutuante e ponteiros em valores booleanos, para que possam ser utilizados diretamente como a condição em uma declaração `if` ou

`while`. O Python permite strings, listas, dicionários e até conjuntos em contextos booleanos, tratando esses valores como verdadeiros se não estiverem vazios. O Rust, porém, é muito rígido: estruturas de controle como `if` e `while` exigem que suas condições sejam expressões `bool`, assim como os operadores lógicos que curto-circuitam² `&&` e `||`. Você deve escrever `if x != 0 { ... }`, não simplesmente `if x { ... }`.

O operador `as` do Rust pode converter valores `bool` em tipos inteiros:

```
assert_eq!(false as i32, 0);
assert_eq!(true as i32, 1);
```

No entanto, `as` não converterá na outra direção, de tipos numéricos em `bool`. Em vez disso, você deve escrever uma comparação explícita como `x != 0`.

Embora um `bool` precise de apenas um único bit para representá-lo, o Rust utiliza todo um byte para um valor `bool` na memória, de modo que você possa criar um ponteiro para ele.

Caracteres

O tipo de caractere `char` do Rust representa um único caractere Unicode, como um valor de 32 bits.

O Rust utiliza o tipo `char` para caracteres únicos isoladamente, mas utiliza a codificação UTF-8 para strings e fluxos (streams) de texto. Então, uma `String` representa o texto como uma sequência de bytes UTF-8, não como um array de caracteres.

Literais de caractere são caracteres entre aspas simples, como `'8'` ou `'!'`. Você pode utilizar toda a extensão do Unicode: `'錆'` é um `char` literal representando o kanji japonês para *sabi* (rust).

Assim como os literais de byte, as barras invertidas são necessárias para alguns caracteres (Tabela 3.10).

Tabela 3.10: Caracteres que exigem escapes de barra invertida

Caractere	Literal de caractere Rust
Aspas simples, <code>'</code>	<code>'\"'</code>
Barra invertida, <code>\</code>	<code>'\\'</code>

Caractere	Literal de caractere Rust
Nova linha	<code>'\n'</code>
Retorno de carro	<code>'\r'</code>
Tab	<code>'\t'</code>

Se preferir, você pode escrever explicitamente o ponto de código Unicode de um caractere em hexadecimal:

- Se o ponto de código do caractere estiver no intervalo U+0000 a U+ 007F (ou seja, se ele for extraído do conjunto de caracteres ASCII), você poderá escrever o caractere como `'\xHH'`, em que HH é um número hexadecimal de dois dígitos. Por exemplo, os literais de caractere `'*'` e `'\x2A'` são equivalentes, porque o ponto de código do caractere `*` é 42, ou 2A em hexadecimal.
- Você pode escrever qualquer caractere Unicode como `'\u{HHHHHH}'`, em que HHHHHH é um número hexadecimal de até seis dígitos, com sublinhados permitidos para agrupamento como de costume. Por exemplo, o literal de caractere `'\u{CA0}'` representa o caractere “ᐐ”, um caractere canarês³ utilizado como de emoji de olhar de desaprovação em um Unicode, “ᐐ_ᐐ”. O mesmo literal também poderia ser simplesmente escrito como `'ᐐ'`.

Um `char` sempre contém um ponto de código Unicode no intervalo de 0x0000 a 0xD7FF ou 0xE000 a 0x10FFFF. Um `char` nunca é uma metade de par substituto (surrogate), ou seja, um ponto de código no intervalo de 0xD800 a 0xDFFF) ou um valor fora do espaço de código Unicode (ou seja, maior que 0x10FFFF). O Rust utiliza o sistema de tipos e verificações dinâmicas para assegurar que os valores `char` sempre estejam dentro do intervalo permitido.

O Rust nunca converte implicitamente entre `char` e qualquer outro tipo. Você pode utilizar o operador de conversão `as` para converter um `char` em um tipo de inteiro; para tipos menores que 32 bits, os bits superiores do valor do caractere são truncados:

```
assert_eq!('*' as i32, 42);
assert_eq!('ᐐ' as u16, 0xca0);
assert_eq!('ᐐ' as i8, -0x60); // U+0CA0 truncado para oito bits, com sinal
```

Na outra direção, `u8` é o único tipo que o operador `as` vai converter em `char`: o Rust pretende que o operador `as` execute apenas conversões baratas e infalíveis, mas todo tipo de inteiro além de `u8` inclui valores que não são permitidos em pontos de código Unicode, portanto essas conversões exigiriam verificações em tempo de execução. Em vez disso, a função de biblioteca padrão `std::char::from_u32` aceita qualquer valor `u32` e retorna um `Option<char>`: se o `u32` não for um ponto de código Unicode permitido, então `from_u32` retorna `None`; caso contrário, retorna `Some(c)`, onde `c` é o resultado `char`.

A biblioteca padrão fornece alguns métodos úteis sobre caracteres, que você pode procurar na documentação on-line em “`char` (tipo primitivo)” e o módulo “`std::char`.” Por exemplo:

```
assert_eq!('*'.is_alphabetic(), false);
assert_eq!('β'.is_alphabetic(), true);
assert_eq!('8'.to_digit(10), Some(8));
assert_eq!('፬'.len_utf8(), 3);
assert_eq!(std::char::from_digit(2, 10), Some('2'));
```

Naturalmente, caracteres únicos isolados não são tão interessantes quanto strings e fluxos de texto. Descreveremos o tipo padrão `String` do Rust e manipulação de texto em geral em “Tipos de String”, na página 97.

Tuplas

Uma *tupla* é um par, ou tripla, quádrupla, quártupla etc. (portanto, *n-tupla*, *enupla*, ou *tupla*), dos valores de tipos variados. Você pode escrever uma tupla como uma sequência de elementos, separados por vírgulas e entre parênteses. Por exemplo, `("Brazil", 1985)` é uma tupla cujo primeiro elemento é uma string alocada estaticamente e cujo segundo é um inteiro; seu tipo é `(&str, i32)`. Dado um valor de tupla `t`, você pode acessar seus elementos como `t.0`, `t.1` e assim por diante.

Até certo ponto, as tuplas se parecem com arrays: ambos os tipos representam uma sequência ordenada de valores. Muitas linguagens de programação fundem ou combinam os dois conceitos, mas, no

Rust, eles são completamente distintos. Por um lado, cada elemento de uma tupla pode ter um tipo diferente, enquanto os elementos de um array devem ser todos do mesmo tipo. Além disso, tuplas permitem apenas constantes como índices, como `t.4`. Você não pode escrever `t.i` ou `t[i]` para obter o *i*-ésimo elemento.

O código Rust geralmente utiliza tipos de tupla para retornar múltiplos valores de uma função. Por exemplo, o método `split_at` em fatias de string, que divide uma string em duas metades e retorna ambas, é declarado assim:

```
fn split_at(&self, mid: usize) -> (&str, &str);
```

O tipo de retorno `(&str, &str)` é uma tupla de duas fatias de string. Você pode utilizar a sintaxe de correspondência de padrões para atribuir cada elemento do valor de retorno a uma variável diferente:

```
let text = "I see the eigenvalue in thine eye";
let (head, tail) = text.split_at(21);
assert_eq!(head, "I see the eigenvalue ");
assert_eq!(tail, "in thine eye");
```

Isso é mais legível do que o equivalente:

```
let text = "I see the eigenvalue in thine eye";
let temp = text.split_at(21);
let head = temp.0;
let tail = temp.1;
assert_eq!(head, "I see the eigenvalue ");
assert_eq!(tail, "in thine eye");
```

Também veremos tuplas utilizadas como uma espécie de struct sem drama. Por exemplo, no programa Mandelbrot no Capítulo 2, precisamos passar a largura e a altura da imagem para as funções que plotam e a escrevem em disco. Poderíamos declarar um struct com os membros `width` e `height`, mas essa é uma notação bastante trabalhosa para algo tão óbvio, assim simplesmente utilizamos uma tupla:

```
/// Grava o buffer `pixels`, cujas dimensões são dadas por `bounds`,
/// no arquivo chamado `filename`.
fn write_image(filename: &str, pixels: &[u8], bounds: (usize, usize))
    -> Result<(), std::io::Error>
{ ... }
```

O tipo do parâmetro `bounds` é `(usize, usize)`, uma tupla de dois valores `usize`. Poderíamos muito bem escrever explicitamente os parâmetros

`width` e `height` separados, e, de qualquer maneira, o código de máquina seria quase o mesmo. É uma questão de clareza. Pensamos no tamanho como um valor, não dois, e utilizar uma tupla permite escrever o que temos em mente.

O outro tipo de tupla comumente utilizado é a tupla zero `()`. Isso é tradicionalmente chamado de *tipo unidade* porque tem apenas um valor, também escrito `()`. O Rust utiliza o tipo unidade onde não há valor significativo a executar, mas mesmo assim o contexto requer alguma espécie de tipo.

Por exemplo, uma função que não retorna nenhum valor tem um tipo de retorno de `()`. A função `std::mem::swap` da biblioteca padrão não tem um valor de retorno significativo; apenas troca os valores de seus dois argumentos. A declaração para `std::mem::swap` é lida como:

```
fn swap<T>(x: &mut T, y: &mut T);
```

O `<T>` significa que `swap` é *genérico*: você pode usá-lo em referências a valores de qualquer tipo `T`. Mas a assinatura omite completamente o tipo de retorno de `swap`, que é uma abreviação para retornar o tipo de unidade:

```
fn swap<T>(x: &mut T, y: &mut T) -> ();
```

Da mesma forma, o `write_image` de exemplo que mencionamos antes tem um tipo de retorno de `Result<(), std::io::Error>`, o que significa que a função retorna um valor `std::io::Error` se algo der errado, mas não retorna nenhum valor em caso de sucesso.

Se quiser, você pode incluir uma vírgula após o último elemento de uma tupla: os tipos `(&str, i32,)` e `(&str, i32)` são equivalentes, assim como as expressões `("Brazil", 1985,)` e `("Brazil", 1985)`. O Rust permite consistentemente uma vírgula extra em todos os lugares onde vírgulas são utilizadas: argumentos de função, arrays, definições de `struct` e `enum` etc. Isso pode parecer estranho para os leitores humanos, mas pode facilitar a leitura das diferenças quando as entradas são adicionadas e removidas no final de uma lista.

Para fins de consistência, existem até tuplas que contêm um único valor. O literal `("lonely hearts",)` é uma tupla contendo uma única string; seu tipo é `(&str,)`. Aqui, a vírgula após o valor é necessária para

distinguir a tupla singleton de uma expressão entre parênteses simples.

Tipos de ponteiro

O Rust tem vários tipos que representam endereços de memória.

Essa é uma grande diferença entre o Rust e a maioria das linguagens com coleta de lixo (garbage collection). No Java, se `class Rectangle` contém um campo `Vector2D upperLeft`;, então `upperLeft` é uma referência a outro objeto `Vector2D` criado separadamente. Objetos nunca contêm fisicamente outros objetos em Java.

O Rust é diferente. A linguagem é projetada para ajudar a manter o mínimo de alocações. Valores agrupados por padrão. O valor `((0, 0), (1440, 900))` é armazenado como quatro inteiros adjacentes. Se você o armazenar em uma variável local, você terá uma variável local com quatro inteiros de largura. Nada é alocado no heap.

Isso é ótimo para eficiência de memória, mas, como consequência, quando um programa Rust precisa de valores para apontar para outros valores, ele deve utilizar tipos de ponteiro explicitamente. A boa notícia é que os tipos de ponteiro utilizados no Rust seguro⁴ (safe) são restritos para eliminar o comportamento indefinido, portanto os ponteiros são muito mais fáceis de utilizar corretamente no Rust do que no C++.

Discutiremos três tipos de ponteiro aqui: referências, boxes e ponteiros não seguros (unsafe).

Referências

Um valor do tipo `&String` (pronuncia-se “ref String”) é uma referência a um valor `String`, um `&i32` é uma referência a um `i32` e assim por diante.

É mais fácil começar pensando em referências como o tipo de ponteiro básico do Rust. Em tempo de execução, uma referência a um `i32` é uma única palavra de máquina contendo o endereço do `i32`, que pode estar no stack (pilha) ou no heap. A expressão `&x` produz

uma referência a x ; na terminologia do Rust, dizemos que *empresta uma referência a x* . Dada uma referência r , a expressão $*r$ referencia o valor para o qual r aponta. Estes são muito parecidos com os operadores $\&$ e $*$ em C e C++. E como um ponteiro C, uma referência não libera automaticamente nenhum recurso quando sai do escopo.

Ao contrário dos ponteiros C, porém, as referências do Rust nunca são nulas: simplesmente não há como produzir uma referência nula em Rust seguro. E, diferentemente do C, o Rust rastreia a posse e o tempo de vida dos valores, portanto erros como ponteiros perdidos, liberações duplas e invalidação de ponteiro são descartados em tempo de compilação.

As referências do Rust têm duas versões:

$\&T$

Uma referência imutável e compartilhada. Você pode ter muitas referências compartilhadas a um determinado valor em um dado momento, mas elas são somente leitura: modificar o valor para o qual elas apontam é proibido, como acontece com $\text{const } T^*$ em C.

$\&\text{mut } T$

Uma referência mutável e exclusiva. Você pode ler e modificar o valor para o qual aponta, como com um T^* em C. Mas, enquanto a referência existir, você não pode ter nenhuma outra referência de qualquer tipo a esse valor. Na verdade, a única maneira de acessar o valor é por meio da referência mutável.

O Rust utiliza essa dicotomia entre referências compartilhadas e mutáveis para impor uma regra de “único escritor ou vários leitores”: você pode ler e escrever/gravar o valor ou pode ser compartilhado por qualquer número de leitores, mas nunca os dois ao mesmo tempo. Essa separação, imposta por verificações em tempo de compilação, é fundamental para as garantias de segurança do Rust. O Capítulo 5 explica as regras do Rust para uso seguro de referências.

Boxes

A maneira mais simples de alocar um valor no heap é utilizar `Box::new`:

```
let t = (12, "eggs");  
let b = Box::new(t); // aloca uma tupla no heap
```

O tipo de `t` é `(i32, &str)`, então o tipo de `b` é `Box<(i32, &str)>`. A chamada para `Box::new` aloca memória suficiente para conter a tupla no heap. Quando `b` sai do escopo, a memória é liberada imediatamente, a menos que `b` tenha sido *movido* – sendo retornado, por exemplo. Os movimentos são essenciais para a maneira como o Rust lida com os valores alocados em heap; explicamos tudo isso em detalhes no Capítulo 4.

Ponteiros brutos

O Rust também tem os tipos de ponteiro bruto `*mut T` e `*const T`. Na verdade, ponteiros brutos são como ponteiros em C++. Utilizar um ponteiro bruto não é seguro, porque o Rust não faz nenhum esforço para rastrear aquilo para o que ele aponta. Por exemplo, os ponteiros brutos podem ser nulos ou podem apontar para a memória que foi liberada ou que agora contém um valor de um tipo diferente. Todos os erros clássicos de ponteiro do C++ são oferecidos para sua diversão.

Entretanto, você só pode desreferenciar ponteiros brutos dentro de um bloco `unsafe` (inseguro). Um bloco `unsafe` é o mecanismo de inclusão do Rust para recursos avançados de linguagem cuja segurança depende de você. Se seu código não tiver blocos `unsafe` (ou se aqueles que eles têm estão escritos corretamente), então as garantias de segurança que enfatizamos ao longo deste livro continuam válidas. Para mais detalhes, ver o Capítulo 22.

Arrays, vetores e fatias

O Rust tem três tipos para representar uma sequência de valores na memória:

- O tipo `[T; N]` representa um array de valores `N`, cada um do tipo `T`. O tamanho de um array é uma constante determinada em tempo

de compilação e é parte do tipo; você não pode acrescentar novos elementos ou reduzir um array.

- O tipo `Vec<T>`, chamado *vetor de* T s, é uma sequência alocada dinamicamente e expansível dos valores de tipo T . Os elementos de um vetor residem no heap, então você pode redimensionar os vetores à vontade: inserir novos elementos neles, acrescentar outros vetores a eles, excluir elementos e assim por diante.
- Os tipos `&[T]` e `&mut [T]`, chamados *fatia compartilhada de* T s e *fatia mutável de* T s, são referências a uma série de elementos que são parte de algum outro valor, como um array ou vetor. Você pode pensar em uma fatia como um ponteiro para o primeiro elemento, com uma contagem do número de elementos que você pode acessar a partir desse ponto. Uma fatia mutável `&mut [T]` permite ler e modificar elementos, mas não pode ser compartilhado; uma fatia compartilhada `&[T]` permite compartilhar o acesso entre vários leitores, mas não permite modificar elementos.

Dado um valor v de qualquer um desses três tipos, a expressão `v.len()` dá o número de elementos em v e `v[i]` referencia i -ésimo elemento de v . O primeiro elemento é `v[0]`, e o último elemento é `v[v.len() - 1]`. O Rust verifica que i sempre está dentro desse intervalo; se não estiver, a expressão gera um pânico. O comprimento de v pode ser zero, caso em que qualquer tentativa de indexá-lo gera um pânico. i deve ser um valor `usize`; você não pode utilizar nenhum outro tipo de inteiro como um índice.

Arrays

Existem várias maneiras de escrever valores de array. A mais simples é escrever uma série de valores entre colchetes:

```
let lazy_caterer: [u32; 6] = [1, 2, 4, 7, 11, 16];
let taxonomy = ["Animalia", "Arthropoda", "Insecta"];

assert_eq!(lazy_caterer[3], 7);
assert_eq!(taxonomy.len(), 3);
```

Para o caso comum de um longo array preenchido com algum valor, você pode escrever `[ $v$ ;  $n$ ]`, em que v é o valor que cada elemento deve ter e n é o comprimento. Por exemplo, `[true; 10000]` é um array

de 10.000 elementos `bool`, todos definidos como `true`:

```
let mut sieve = [true; 10000];
for i in 2..100 {
    if sieve[i] {
        let mut j = i * i;
        while j < 10000 {
            sieve[j] = false;
            j += i;
        }
    }
}
```

```
assert!(sieve[211]);
assert!(!sieve[9876]);
```

Você verá essa sintaxe utilizada para buffers de tamanho fixo: `[0u8; 1024]` pode ser um buffer de um Kbyte, preenchido com zeros. O Rust não possui notação para um array não inicializado. (Em geral, o Rust assegura que o código nunca possa acessar qualquer tipo de valor não inicializado.)

O comprimento de um array é parte de seu tipo e é fixo em tempo de compilação. Se `n` é uma variável, você não pode escrever `[true; n]` para obter um array de `n` elementos. Quando você precisa de um array cujo comprimento varia em tempo de execução (e você geralmente precisa), utilize um vetor.

Os métodos úteis que você quer ver em arrays – iterar por elementos, pesquisar, ordenar, preencher, filtrar e assim por diante – são todos fornecidos como métodos em fatias, não em arrays. Mas o Rust converte implicitamente uma referência a um array em uma fatia ao pesquisar métodos, para que você possa chamar qualquer método de fatia sobre um array diretamente:

```
let mut chaos = [3, 5, 4, 1, 2];
chaos.sort();
assert_eq!(chaos, [1, 2, 3, 4, 5]);
```

Aqui o método `sort` é realmente definido em fatias, mas como ele utiliza o operando por referência, o Rust produz implicitamente uma fatia `&mut [i32]` que referencia todo o array e passa isso para `sort` trabalhar. Na verdade, o método `len` que mencionamos anteriormente também é um método de fatia. Discutimos fatias em mais detalhes

em “Fatias”, na página 96.

Vetores

Um vetor `Vec<T>` é um array redimensionável de elementos do tipo `T`, alocado no heap.

Existem várias maneiras de criar vetores. A mais simples é utilizar a macro `vec!`, que fornece uma sintaxe para vetores que se parece muito com um literal de array:

```
let mut primes = vec![2, 3, 5, 7];
assert_eq!(primes.iter().product::<i32>(), 210);
```

Mas é claro, isso é um vetor, não um array, assim podemos adicionar elementos a ele dinamicamente:

```
primes.push(11);
primes.push(13);
assert_eq!(primes.iter().product::<i32>(), 30030);
```

Você também pode construir um vetor repetindo um determinado valor um certo número de vezes, novamente utilizando uma sintaxe que imita literais de array:

```
fn new_pixel_buffer(rows: usize, cols: usize) -> Vec<u8> {
    vec![0; rows * cols]
}
```

A macro `vec!` é equivalente a chamar `Vec::new` para criar um novo vetor vazio e, em seguida, inserir os elementos nele, o que é outro idioma:

```
let mut pal = Vec::new();
pal.push("step");
pal.push("on");
pal.push("no");
pal.push("pets");
assert_eq!(pal, vec!["step", "on", "no", "pets"]);
```

Outra possibilidade é construir um vetor a partir dos valores gerados por um iterador:

```
let v: Vec<i32> = (0..5).collect();
assert_eq!(v, [0, 1, 2, 3, 4]);
```

Muitas vezes, você precisará fornecer o tipo ao utilizar `collect` (como fizemos aqui), porque pode construir muitos tipos diferentes de coleções, não apenas vetores. Especificando o tipo de `v`, tornamos inequívoco o tipo de coleção que queremos.

Como com arrays, você pode utilizar métodos de fatia em vetores:

```
// Um palíndromo!  
let mut palindrome = vec!["a man", "a plan", "a canal", "panama"];  
palindrome.reverse();  
// Razoável, mas decepcionante:  
assert_eq!(palindrome, vec!["panama", "a canal", "a plan", "a man"]);
```

Aqui o método `reverse` é na verdade definido em fatias, mas a chamada implicitamente empresta uma fatia `&mut [&str]` do vetor e invoca `reverse` nisso.

`Vec` é um tipo essencial para o Rust – é utilizado em quase todos os lugares onde uma lista de tamanho dinâmico é necessária –, assim existem muitos outros métodos que constroem novos vetores ou estendem os existentes. Vamos abordá-los no Capítulo 16.

Um `Vec<T>` consiste em três valores: um ponteiro para o buffer alocado no heap para os elementos, que é criado e possuído `Vec<T>`; o número de elementos que o buffer tem capacidade de armazenar; e o número que realmente contém agora (em outras palavras, seu comprimento). Quando o buffer alcançou sua capacidade, adicionar outro elemento ao vetor envolve alocar um buffer maior, copiar o conteúdo atual para ele, atualizar o ponteiro do vetor e a capacidade para descrever o novo buffer e, finalmente, liberar o antigo.

Se você souber antecipadamente o número de elementos de que um vetor precisará, em vez de `Vec::new` você pode chamar `Vec::with_capacity` para criar um vetor com um buffer grande o suficiente para conter todos eles, desde o início; então, você pode adicionar os elementos ao vetor, um de cada vez, sem causar nenhuma realocação. A macro `vec!` utiliza um truque como esse, pois sabe quantos elementos o vetor final terá. Observe que isso apenas estabelece o tamanho inicial do vetor; se você exceder a estimativa, o vetor simplesmente aumentará o armazenamento como de costume.

Muitas funções de biblioteca buscam a oportunidade de utilizar `Vec::with_capacity` em vez de `Vec::new`. Por exemplo, no exemplo de `collect`, o iterador `0..5` sabe de antemão que vai gerar cinco valores e a função `collect` aproveita isso para pré-alocar o vetor que retorna com a capacidade correta. Veremos como isso funciona no Capítulo 15.

Assim como o método `len` de um vetor retorna o número de elementos que ele contém agora, o método `capacity` retorna o número de elementos que poderia conter sem realocação:

```
let mut v = Vec::with_capacity(2);
assert_eq!(v.len(), 0);
assert_eq!(v.capacity(), 2);

v.push(1);
v.push(2);
assert_eq!(v.len(), 2);
assert_eq!(v.capacity(), 2);

v.push(3);
assert_eq!(v.len(), 3);
// Normalmente imprime "capacity is now 4":
println!("capacity is now {}", v.capacity());
```

Não é garantido que a capacidade impressa no final seja exatamente 4, mas será pelo menos 3, pois o vetor contém três valores.

Você pode inserir e remover elementos onde quiser em um vetor, embora essas operações desbloqueiem todos os elementos após a posição afetada para a frente ou para trás; portanto, podem ser lentas se o vetor for longo:

```
let mut v = vec![10, 20, 30, 40, 50];

// Faça o elemento no índice 3 ser 35
v.insert(3, 35);
assert_eq!(v, [10, 20, 30, 35, 40, 50]);

// Remova o elemento no índice 1
v.remove(1);
assert_eq!(v, [10, 30, 35, 40, 50]);
```

Você pode utilizar o método `pop` para remover o último elemento e retorná-lo. Mais precisamente, extrair um valor de um `Vec<T>` retorna um `Option<T>`: `None` se o vetor já estava vazio, ou `Some(v)` se o último elemento era `v`:

```
let mut v = vec!["Snow Puff", "Glass Gem"];
assert_eq!(v.pop(), Some("Glass Gem"));
assert_eq!(v.pop(), Some("Snow Puff"));
assert_eq!(v.pop(), None);
```

Você pode utilizar um loop `for` para iterar por um vetor:

```
// Obtenha nossos argumentos de linha de comando como um vetor de Strings
```

```

let languages: Vec<String> = std::env::args().skip(1).collect();
for l in languages {
    println!("{}", l,
        if l.len() % 2 == 0 {
            "functional"
        } else {
            "imperative"
        });
}

```

Executar esse programa com uma lista de linguagens de programação é esclarecedor:

```

$ cargo run Lisp Scheme C C++ Fortran
Compiling proglangs v0.1.0 (/home/jimb/rust/proglangs)
Finished dev [unoptimized + debuginfo] target(s) in 0.36s
Running `target/debug/proglangs Lisp Scheme C C++ Fortran`
Lisp: functional
Scheme: functional
C: imperative
C++: imperative
Fortran: imperative
$

```

Por fim, uma definição satisfatória para o termo linguagem funcional. Apesar de seu papel fundamental, `Vec` é um tipo comum definido no Rust, não integrado na linguagem. Abordaremos as técnicas necessárias para implementar esses tipos no Capítulo 22.

Fatias

Uma fatia (slice), escrita `[T]` sem especificar o comprimento, é uma região de um array ou vetor. Como uma fatia pode ter qualquer comprimento, as fatias não podem ser armazenadas diretamente em variáveis ou passadas como argumentos de função. As fatias são sempre passadas por referência.

Uma referência a uma fatia é uma *ponteiro gordo* (*fat pointer*): um valor de duas palavras que compreende um ponteiro para o primeiro elemento da fatia e o número de elementos na fatia.

Suponha que você execute o seguinte código:

```

let v: Vec<f64> = vec![0.0, 0.707, 1.0, 0.707];
let a: [f64; 4] = [0.0, -0.707, -1.0, -0.707];
let sv: &[f64] = &v;

```

```
let sa: &[f64] = &a;
```

Nas duas últimas linhas, o Rust converte automaticamente a referência `&Vec<f64>` e a referência `&[f64; 4]` em referências de fatia que apontam diretamente para os dados.

No final, a memória se parece com a Figura 3.2.

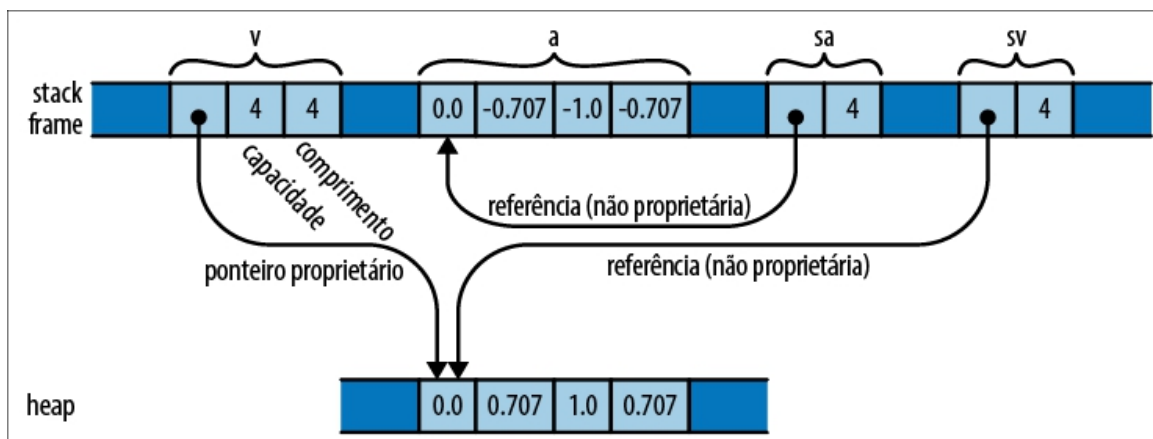


Figura 3.2: Um vetor `v` e um array `a` na memória, com fatias `sv` e `sa` referenciando-os respectivamente.

Enquanto uma referência comum é um ponteiro não proprietário (sem posse) para um único valor, uma referência a uma fatia é um ponteiro não proprietário para um intervalo de valores consecutivos na memória. Isso torna as referências a fatias uma boa escolha quando você deseja escrever uma função que opera em um array ou em um vetor. Por exemplo, eis uma função que imprime uma fatia dos números, um por linha:

```
fn print(n: &[f64]) {  
    for elt in n {  
        println!("{}", elt);  
    }  
}  
  
print(&a); // funciona em arrays  
print(&v); // funciona em vetores
```

Como essa função utiliza uma referência de fatia como argumento, você pode aplicá-la a um vetor ou a um array, conforme mostrado. Na verdade, muitos métodos que você pode imaginar como pertencentes a vetores ou arrays são métodos definidos em fatias: por exemplo, os métodos `sort` e `reverse`, que classificam ou invertem

uma sequência de elementos no lugar, são na verdade métodos no tipo de fatia `[T]`.

Você pode obter uma referência a uma fatia de um array ou vetor, ou uma fatia de uma fatia existente, indexando-a com um intervalo:

```
print(&v[0..2]); // imprime os dois primeiros elementos de v
print(&a[2..]); // imprime elementos de a começando com a[2]
print(&sv[1..3]); // imprime v[1] e v[2]
```

Assim como nos acessos comuns a arrays, o Rust verifica se os índices são válidos. Tentar pegar emprestado uma fatia que se estende para além do final dos dados gera um pânico.

Como as fatias quase sempre aparecem atrás de referências, geralmente nos referimos apenas a tipos como `&[T]` ou `&str` como “fatias”, utilizando o nome mais curto para o conceito mais comum.

Tipos de string

Os programadores familiarizados com C++ lembrarão que existem dois tipos de string na linguagem. Os literais de string têm o tipo de ponteiro `const char *`. A biblioteca padrão também oferece uma classe, `std::string`, para criar strings dinamicamente em tempo de execução.

O Rust tem um design semelhante. Nesta seção, mostraremos todas as maneiras de escrever literais de strings e, em seguida, apresentaremos os dois tipos string do Rust. Fornecemos mais detalhes sobre strings e manipulação de texto no Capítulo 17.

Literais de string

Literais de string são colocados entre aspas duplas. Eles utilizam as mesmas sequências de escape de barra invertida como literais `char`:

```
let speech = "\"Ouch!\" said the well.\n";
```

Em literais de string, ao contrário de literais `char`, aspas simples não precisam de escape de barra invertida, e aspas duplas precisam.

Uma string pode abranger várias linhas:

```
println!("In the room the women come and go,\n    Singing of Mount Abora");
```

O caractere de nova linha nesse literal de string é incluído na string

e, portanto, na saída. Assim como os espaços no início da segunda linha.

Se uma linha de uma string terminar com uma barra invertida, o caractere de nova linha e o espaço em branco inicial na próxima linha serão dropados:

```
println!("It was a bright, cold day in April, and \
there were four of us—\
more or less.");
```

Isso imprime uma única linha de texto. A string contém um espaço único entre "and" e "there" porque há um espaço antes da barra invertida no programa, e nenhum espaço entre o travessão e "more".

Em alguns casos, a necessidade de dobrar cada barra invertida em uma string é um incômodo. (Os exemplos clássicos são expressões regulares e caminhos (paths) do Windows.) Para esses casos, o Rust oferece *strings brutas*. Uma string bruta é marcada com a letra minúscula *r*. Todas as barras invertidas e caracteres de espaço em branco dentro de uma string bruta são incluídos textualmente na string. Nenhuma sequência de escape é reconhecida:

```
let default_win_install_path = r"C:\Program Files\Gorillas";

let pattern = Regex::new(r"\d+(\.\d+)*");
```

Você não pode incluir um caractere de aspas duplas em uma string bruta simplesmente colocando uma barra invertida na frente dela – lembre-se, dissemos *nenhuma* sequência de escape é reconhecida. Contudo, há uma solução para isso também. O início e o fim de uma string bruta podem ser marcados com cerquilhas (#):

```
println!(r###"
This raw string started with 'r####'.
Therefore it does not end until we reach a quote mark (""")
followed immediately by three pound signs ('###'):
"###);
```

Você pode adicionar quantas cerquilhas (#) forem necessárias para deixar claro onde a string bruta termina.

Strings de bytes

Um literal de string com o prefixo `b` é uma *string de bytes*. Essa string é uma fatia dos valores `u8` – ou seja, bytes – em vez de texto Unicode:

```
let method = b"GET";  
assert_eq!(method, &[b'G', b'E', b'T']);
```

O tipo de `method` é `&[u8; 3]`: é uma referência a um array de três bytes. Ele não possui nenhum dos métodos de string que discutiremos a seguir. A coisa mais parecida com uma string é a sintaxe que utilizamos para escrevê-la.

Strings de byte podem utilizar todas as outras sintaxes de string que mostramos: elas podem abranger várias linhas, utilizar sequências de escape e usar barras invertidas para unir linhas. Strings de bytes brutas começam com `br`.

Strings de bytes não podem conter caracteres Unicode arbitrários. Elas devem se contentar com sequências de escape ASCII e `\xHH`.

Strings na memória

Strings do Rust são sequências de caracteres Unicode, mas não são armazenadas na memória como arrays de `char`s. Em vez disso, são armazenadas utilizando UTF-8, uma codificação de tamanho variável. Cada caractere ASCII em uma string é armazenado em um byte. Outros caracteres ocupam vários bytes.

A Figura 3.3 mostra os valores `String` e `&str` criados pelo seguinte código:

```
let noodles = "noodles".to_string();  
let oodles = &noodles[1..];  
let poodles = "ð_ð";
```

Um `String` tem um buffer redimensionável contendo texto UTF-8. O buffer é alocado no heap, portanto ele pode redimensionar seu buffer conforme necessário ou solicitado. No exemplo, `noodles` é uma `String` que possui um buffer de oito bytes, dos quais sete estão em uso. Você pode pensar em uma `String` como um `Vec<u8>` que garantidamente contém UTF-8 bem formado; na verdade, é assim que `String` é implementada.

Um `&str` (pronuncia-se “stir” ou “string slice”) é uma referência a uma

string de texto UTF-8 possuída por outra pessoa: ele “pega emprestado” o texto. No exemplo, `oodles` é um `&str` referindo-se aos últimos seis bytes do texto pertencente a `noodles`, portanto representa o texto “oodles”. Como outras referências a fatias, um `&str` é um ponteiro gordo, contendo o endereço dos dados reais e seu comprimento. Você pode pensar em um `&str` como sendo nada mais do que um `&[u8]` que é garantido conter UTF-8 bem formado.

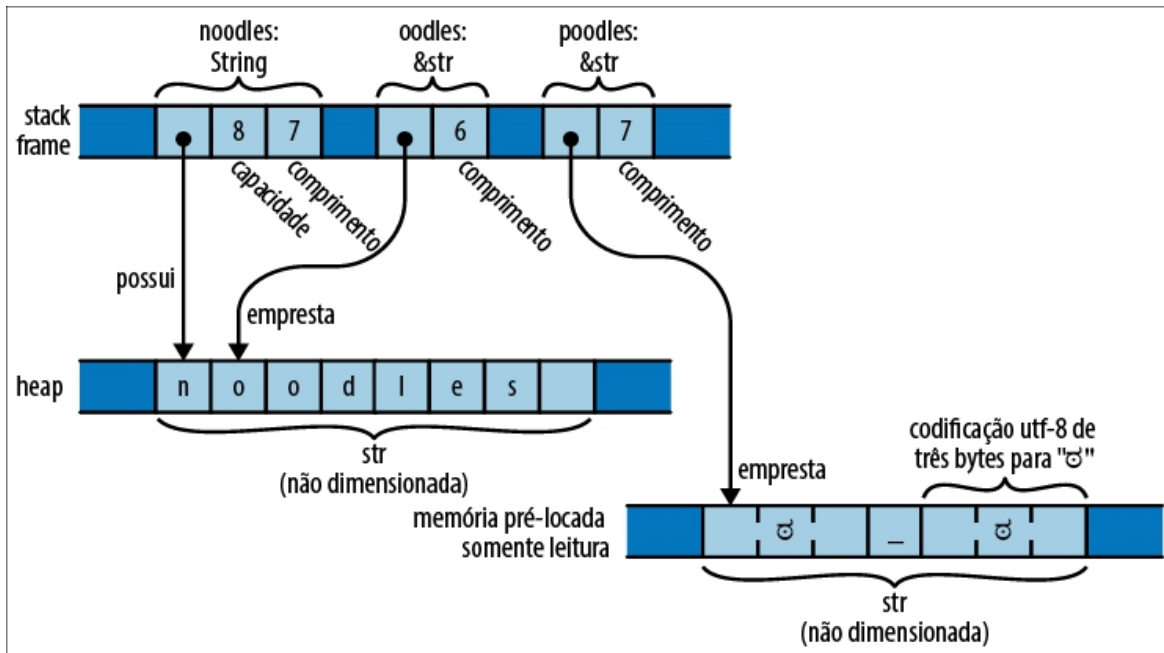


Figura 3.3: String, &str e str.

Um literal de string é um `&str` que referencia texto pré-alocado, normalmente armazenado na memória somente leitura com o código de máquina do programa. No exemplo anterior, `poodles` é um literal de string, apontando para sete bytes que são criados quando o programa inicia a execução e que duram até ele terminar.

Um método `.len()` de `String` ou `&str` retorna seu comprimento. O comprimento é medido em bytes, não em caracteres:

```
assert_eq!("ó_ó".len(), 7);
assert_eq!("ó_ó".chars().count(), 3);
```

É impossível modificar um `&str`:

```
let mut s = "hello";
s[0] = 'c'; // erro: `&str` não pode ser modificado, e outros motivos
s.push('\n'); // erro: nenhum método chamado `push` encontrado para a referência
```

``&str``

Para criar novas strings em tempo de execução, utilize `String`.

O tipo `&mut str` existe, mas não é muito útil, já que quase qualquer operação em UTF-8 pode alterar o comprimento geral de bytes e uma fatia não pode realocar seu referente. De fato, as únicas operações disponíveis em `&mut str` são `make_ascii_uppercase` e `make_ascii_lowercase`, que modificam o texto no local e afetam apenas caracteres de byte único, por definição.

String

`&str` se parece muito com `&[T]`: um ponteiro gordo para alguns dados. `String` é análogo a `Vec<T>`, conforme descrito na Tabela 3.11.

Tabela 3.11: `Vec<T>` e `String` comparação

	<code>Vec<T></code>	<code>String</code>
Libera buffers automaticamente	Sim	Sim
Expansível	Sim	Sim
funções <code>::new()</code> e <code>::with_capacity()</code> associadas a tipo	Sim	Sim
métodos <code>.reserve()</code> e <code>.capacity()</code>	Sim	Sim
métodos <code>.push()</code> e <code>.pop()</code>	Sim	Sim
Sintaxe de intervalo <code>v[start..stop]</code>	Sim, retorna <code>&[T]</code>	Sim, retorna <code>&str</code>
Conversão automática	<code>&Vec<T></code> em <code>&[T]</code>	<code>&String</code> em <code>&str</code>
Herda métodos	De <code>&[T]</code>	De <code>&str</code>

Como um `Vec`, cada `String` tem seu próprio buffer alocado no heap que não é compartilhado com nenhuma outra `String`. Quando uma variável `String` sai do escopo, o buffer é liberado automaticamente, a menos que a `String` tenha sido movida.

Existem várias maneiras de criar `Strings`:

- O método `.to_string()` converte um `&str` em uma `String`. Isso copia a string:

```
let error_message = "too many pets".to_string();
```

O método `.to_owned()` faz a mesma coisa, e você pode vê-lo utilizado da mesma maneira. Também funciona para alguns outros

tipos, como discutiremos no Capítulo 13.

- A macro `format!()` funciona como `println!()`, exceto que retorna uma nova `String` em vez de escrever texto para `stdout` e não adiciona automaticamente uma nova linha no final:

```
assert_eq!(format!("{0}{:02}'{:02}'N", 24, 5, 23),
"2405'23"N".to_string());
```

- Arrays, fatias e vetores de strings têm dois métodos, `.concat()` e `.join(sep)`, que formam uma nova `String` de muitas strings:

```
let bits = vec!["veni", "vidi", "vici"];
assert_eq!(bits.concat(), "venividivici");
assert_eq!(bits.join(", "), "veni, vidi, vici");
```

Às vezes, há a opção de qual tipo utilizar: `&str` ou `String`. O Capítulo 5 aborda essa questão em detalhes. Por ora, bastará ressaltar que uma `&str` pode referenciar qualquer fatia de qualquer string, seja um literal de string (armazenado no executável) ou uma `String` (alocada e liberada em tempo de execução). Isso significa que `&str` é mais apropriado para argumentos de função quando o chamador deve ter permissão para passar qualquer tipo de string.

Usando strings

Strings suportam os operadores `==` e `!=`. Duas strings são iguais se contiverem os mesmos caracteres na mesma ordem (independentemente de apontarem para a mesma posição na memória):

```
assert!("ONE".to_lowercase() == "one");
```

Strings também suportam os operadores de comparação `<`, `<=`, `>` e `>=`, bem como muitos métodos e funções úteis que você pode encontrar na documentação on-line em `"str (tipo primitivo)"` ou no módulo `"std::str"` (ou apenas passe para o Capítulo 17). Eis alguns exemplos:

```
assert!("peanut".contains("nut"));
assert_eq!("ø_ø".replace("ø", "■"), "■_■");
assert_eq!("  clean\n".trim(), "clean");
```

```
for word in "veni, vidi, vici".split(", ") {
    assert!(word.starts_with("v"));
}
```

Tenha em mente que, dada a natureza do Unicode, a comparação simples `char-por-char` *nem* sempre dá as respostas esperadas. Por exemplo, as strings Rust `"th\u{e9}"` e `"the\u{301}"` são representações Unicode válidas para *thé*, a palavra francesa para chá. O Unicode diz que ambas devem ser exibidas e processadas da mesma maneira, mas o Rust as trata como duas strings completamente distintas. Da mesma forma, os operadores de ordenação do Rust como `<` utilizam uma ordem lexicográfica simples baseada em valores de ponto de código de caractere. Essa ordenação às vezes lembra a ordenação utilizada para texto no idioma e cultura do usuário. Discutimos essas questões em mais detalhes no Capítulo 17.

Outros tipos semelhantes a strings

O Rust garante que as strings são UTF-8 válidas. Às vezes, um programa realmente precisa ser capaz de lidar com strings que *não* são Unicode válidas. Isso geralmente acontece quando um programa Rust precisa interoperar com algum outro sistema que não impõe nenhuma dessas regras. Por exemplo, na maioria dos sistemas operacionais, é fácil criar um arquivo com um nome de arquivo que não é Unicode válido. O que deve acontecer quando um programa Rust se depara com esse tipo de nome de arquivo?

A solução do Rust é oferecer alguns tipos semelhantes a strings para essas situações:

- Atenha-se a `String` e `&str` para texto Unicode.
- Ao trabalhar com nomes de arquivos, em vez disso, utilize `std::path::PathBuf` e `&Path`.
- Ao trabalhar com dados binários que não são codificados em UTF-8, utilize `Vec<u8>` e `&[u8]`.
- Ao trabalhar com nomes de variável de ambiente e argumentos de linha de comando no formato nativo apresentado pelo sistema operacional, utilize `OsString` e `&OsStr`.
- Ao interoperar com bibliotecas C que utilizam strings terminadas em nulo, utilize `std::ffi::CString` e `&CStr`.

Aliases de tipo

A palavra-chave `type` pode ser utilizada como `typedef` em C++ para declarar um novo nome para um tipo existente (apelido, alias):

```
type Bytes = Vec<u8>;
```

O tipo `Bytes` que estamos declarando aqui é uma abreviação para esse tipo particular de `Vec`:

```
fn decode(data: &Bytes) {  
    ...  
}
```

Além do básico

Tipos são uma parte central do Rust. Continuaremos a discutir tipos e apresentar novos ao longo do livro. Em particular, os tipos do Rust definidos pelo usuário fornecem à linguagem boa parte de seu poder, porque é onde os métodos são definidos. Existem três tipos de tipos definidos pelo usuário e vamos abordá-los em três capítulos sucessivos: structs no Capítulo 9, enums no Capítulo 10 e traits no Capítulo 11.

Funções e closures têm seus próprios tipos, abordados no Capítulo 14. E os tipos que compõem a biblioteca padrão são abordados ao longo do livro. Por exemplo, o Capítulo 16 apresenta os tipos de coleção padrão.

Mas tudo isso terá de esperar. Antes de prosseguirmos, é hora de abordar os conceitos que estão no centro das regras de segurança do Rust.

¹ N.R.: Aqui enquadrar está sendo usado no sentido de “wraps around”, pois quando um valor é maior do que o tipo pode representar, apenas uma parte do valor é mantida, perdendo-se o restante, como um limite de tamanho, daí enquadramento. Em Rust, quando isso acontece, o valor resultante é o valor do módulo do valor correto com o máximo que pode ser apresentado pelo tipo. Por exemplo, 1000 enquadrado em `u8` seria representado como $1000 \% 256 (2^8) = 232$.

² N.R.: Um curto-circuito na avaliação de um operador booleano é uma otimização que não executa o restante da expressão caso o resultado da primeira parte seja determinante. Por exemplo, com `&&`, ambos os operandos têm de ser verdadeiros. Se o primeiro for falso, o segundo nem é avaliado, pois o resultado do `&&` será falso independentemente de seu valor. No `||` o mesmo acontece quando um dos operandos é verdadeiro.

[3](https://pt.wikipedia.org/wiki/L%C3%ADngua_canaresa) N.R.: Idioma falado na Índia: https://pt.wikipedia.org/wiki/L%C3%ADngua_canaresa

[4](#) N.R.: Rust tem operações seguras (safe) e inseguras (unsafe). O programador pode escolher ter partes inseguras (unsafe) no código, ou seja, cujo comportamento não pode ser verificado pelo compilador Rust.

CAPÍTULO 4

Posse e movimentos

Quando se trata de gerenciamento de memória, há duas características que gostaríamos de encontrar em nossas linguagens de programação:

- Gostaríamos que a memória fosse liberada prontamente, no momento de nossa escolha. Isso nos dá controle sobre o consumo de memória do programa.
- Nunca queremos utilizar um ponteiro para um objeto depois que ele foi liberado. Isso seria um comportamento indefinido, levando a travamentos e falhas de segurança.

Mas essas características parecem ser mutuamente exclusivas: liberar um valor enquanto existem ponteiros para ele deixa, necessariamente, esses ponteiros perdidos (inválidos). Quase todas as principais linguagens de programação se enquadram em um dos dois campos, dependendo de qual das duas qualidades elas não mantêm:

- O campo “Segurança em primeiro lugar” utiliza coleta de lixo para gerenciar a memória, liberando objetos automaticamente quando todos os ponteiros que podem acessá-los forem removidos. Isso elimina os ponteiros perdidos, simplesmente mantendo os objetos ao redor até que não haja mais ponteiros que os utilizam. Quase todas as linguagens modernas se enquadram nesse campo, de Python, JavaScript e Ruby a Java, C# e Haskell.

Mas confiar na coleta de lixo significa abrir mão do controle sobre quando exatamente os objetos são liberados para o coletor. Em geral, os coletores de lixo são feras surpreendentes e entender por que a memória não foi liberada quando você esperava pode ser um desafio.

- O campo “Control First” deixa você encarregado de liberar

memória. O consumo de memória do seu programa está inteiramente em suas mãos, mas evitar ponteiros perdidos também se torna inteiramente uma preocupação sua. C e C++ são as únicas linguagens dominantes nesse campo.

Isso é ótimo se você nunca cometer erros, mas as evidências sugerem que, eventualmente, você cometerá. O uso indevido do ponteiro tem sido um culpado comum em problemas de segurança relatados desde que os dados começaram a ser coletados.

O Rust visa ser seguro e eficiente, então nenhum desses compromissos é aceitável. Mas, se a reconciliação fosse fácil, alguém a teria feito muito antes. Algo fundamental precisa mudar.

O Rust quebra o deadlock (impasse) de uma forma surpreendente: restringindo como seus programas podem utilizar ponteiros. Este capítulo e o próximo são dedicados a explicar exatamente o que são essas restrições e por que funcionam. Por enquanto, basta dizer que algumas estruturas comuns que você está acostumado a utilizar podem não se encaixar nas regras e você precisará procurar alternativas. Mas o efeito fundamental dessas restrições é trazer ordem suficiente ao caos para permitir que as checagens de tempo de compilação do Rust verifiquem se seu programa está livre de erros de segurança de memória: ponteiros perdidos, liberações duplas, uso de memória não inicializada e assim por diante. Em tempo de execução, seus ponteiros são endereços simples na memória, assim como seriam em C e C++. A diferença é que seu código provou usá-los com segurança.

Essas mesmas regras também formam a base do suporte do Rust para segurança na programação concorrente. Utilizando as primitivas de encadeamento cuidadosamente projetadas do Rust, as regras que garantem que seu código utilize a memória corretamente também servem para provar que ele está livre de problemas de concorrência (data races). Um bug em um programa Rust não pode fazer com que um thread corrompa os dados de outro, introduzindo falhas difíceis de reproduzir em partes não relacionadas do sistema. O comportamento não determinístico inerente ao código multithread é isolado, para os recursos projetados para lidar com ele – mutexes,

canais de mensagem, valores atômicos e assim por diante – em vez de aparecer em referências de memória comuns. O código multithread em C e C++ ganhou sua má reputação, mas o Rust o reabilita muito bem.

A aposta radical do Rust, a defesa em que se baseia seu sucesso e que forma a raiz da linguagem, é que, mesmo com essas restrições em vigor, você achará a linguagem mais flexível do que o suficiente para quase todas as tarefas, e que os benefícios – a eliminação de amplas classes de erros de gerenciamento de memória e concorrência – justificarão as adaptações que você precisará fazer em seu estilo. Os autores deste livro estão otimistas com o Rust, exatamente por causa de nossa ampla experiência com C e C++. Para nós, a proposta do Rust é irrecusável.

As regras do Rust provavelmente são diferentes do que você viu em outras linguagens de programação. Aprender a trabalhar com elas e aproveitá-las é, em nossa opinião, o desafio central de aprender Rust. Neste capítulo, primeiro forneceremos informações sobre a lógica e a intenção por trás das regras do Rust, mostrando como os mesmos problemas subjacentes funcionam em outras linguagens. Em seguida, explicaremos as regras do Rust em detalhes, observando o que significa posse em um nível conceitual e mecânico, como as mudanças na posse são rastreadas em vários cenários e os tipos que distorcem ou quebram algumas dessas regras para fornecer mais flexibilidade.

Posse (ownership)

Se você leu muito código C ou C++, provavelmente já se deparou com um comentário dizendo que uma instância de alguma classe possui algum outro objeto para o qual ela aponta. Isso geralmente significa que o objeto proprietário decide quando liberar o objeto possuído: quando o proprietário é destruído, ele destrói suas posses com ele.

Por exemplo, suponha que você escreva o seguinte código em C++:

```
std::string s = "frayed knot";
```

A string `s` geralmente é representada na memória como mostrado na Figura 4.1.

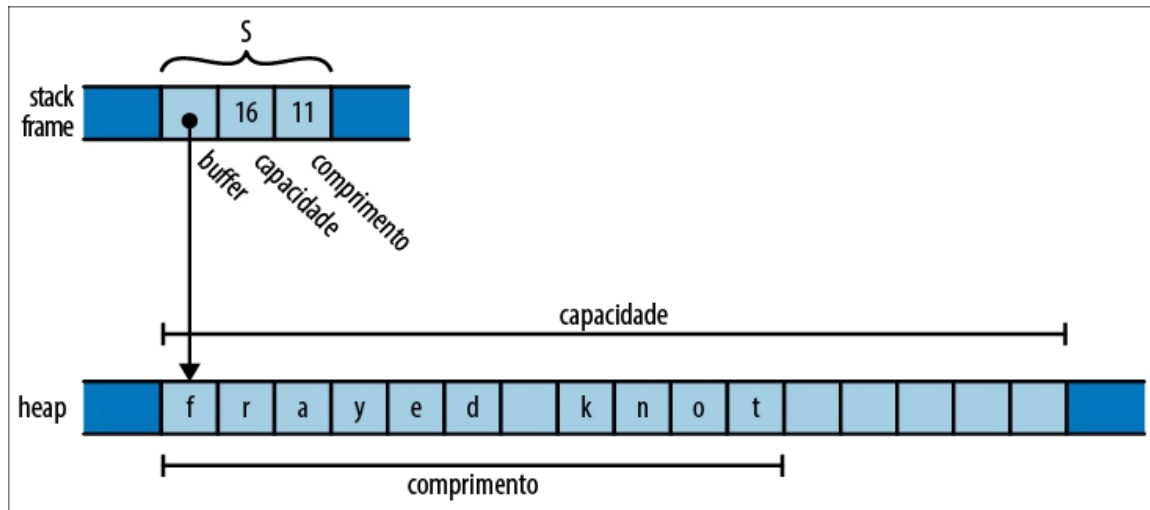


Figura 4.1: O valor `std::string` C++ no stack, apontando para seu buffer alocado no heap.

Aqui, o objeto real `std::string` em si tem sempre exatamente três palavras, compreendendo um ponteiro para um buffer alocado no heap, a capacidade geral do buffer (ou seja, o tamanho que o texto pode crescer antes que a string precise alocar um buffer maior para mantê-lo) e o comprimento do texto que ele contém agora. Estes são campos privados para a classe `std::string`, não acessível aos usuários da string.

Uma `std::string` possui seu buffer: quando o programa destrói a string, o destruidor da string libera o buffer. No passado, algumas bibliotecas C++ compartilhavam um único buffer entre vários valores `std::string`, utilizando uma contagem de referência para decidir quando o buffer deveria ser liberado. Versões mais recentes da especificação C++ efetivamente excluem essa representação; todas as bibliotecas C++ modernas utilizam a abordagem mostrada aqui.

Nessas situações, geralmente é entendido que, embora seja benéfico para outro código criar ponteiros temporários para a memória já possuída, é responsabilidade desse código garantir que seus ponteiros tenham desaparecido antes que o proprietário decida destruir o objeto possuído. Você pode criar um ponteiro para um

caractere que está em uma `std::string`, mas, quando a string é destruída, seu ponteiro se torna inválido e cabe a você garantir que seu código não o utilize mais. O proprietário determina o tempo de vida do objeto possuído e todos os demais devem respeitar suas decisões.

Nós utilizamos `std::string` aqui como um exemplo de como é a posse em C++: é apenas uma convenção que a biblioteca padrão geralmente segue e, embora a linguagem encoraje a seguir práticas semelhantes, a maneira como você projeta seus próprios tipos depende, em última análise, de você mesmo.

No Rust, porém, o conceito de posse é incorporado à própria linguagem e reforçado por verificações em tempo de compilação. Cada valor tem um único proprietário que determina seu tempo de vida. Quando o proprietário é liberado – *dropped* (*dropado*), na terminologia Rust –, o valor possuído também é liberado. Essas regras destinam-se a facilitar a verificação do tempo de vida de qualquer valor simplesmente inspecionando o código, dando a você o controle sobre o tempo de vida de qualquer valor que uma linguagem de sistema deve fornecer.

Uma variável possui seu valor. Quando o controle sai do bloco no qual a variável é declarada, a variável é dropada, portanto seu valor é dropado com ela. Por exemplo:

```
fn print_padovan() {  
    let mut padovan = vec![1,1,1]; // alocado aqui  
    for i in 3..10 {  
        let next = padovan[i-3] + padovan[i-2];  
        padovan.push(next);  
    }  
    println!("P(1..10) = {:?}", padovan);  
} // dropado aqui
```

O tipo da variável `padovan` é `Vec<i32>`, um vetor de inteiros de 32 bits. Na memória, o valor final de `padovan` será algo como a Figura 4.2.

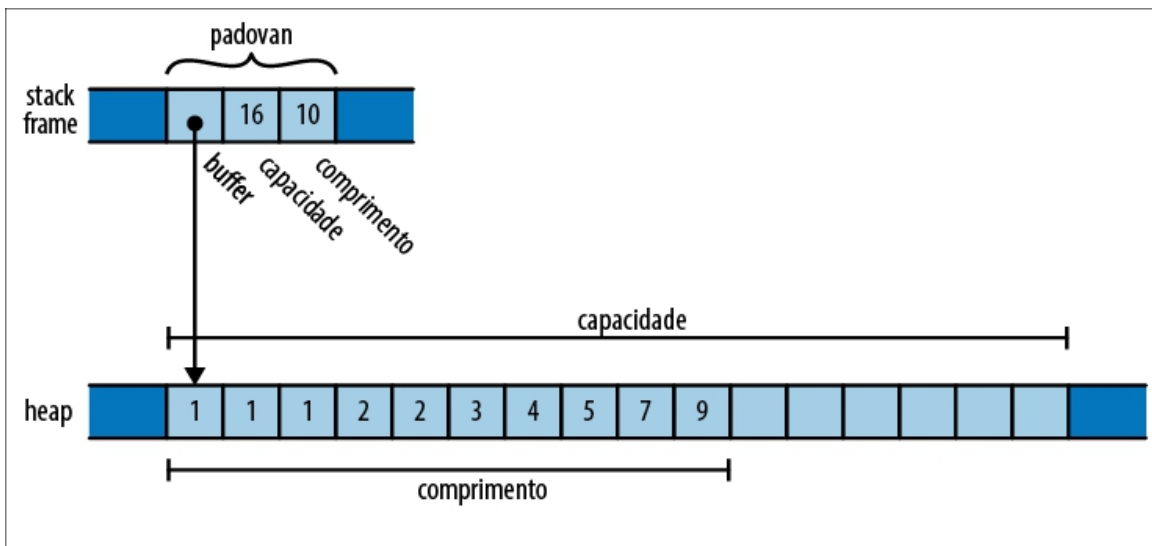


Figura 4.2: Um `Vec<i32>` no stack, apontando para seu buffer no heap.

Isso é muito parecido com a `std::string` do C++ que mostramos anteriormente, exceto que os elementos no buffer são valores de 32 bits, não caracteres. Note que as palavras contendo o ponteiro `padovan`, a capacidade e o comprimento permanecem diretamente no stack frame função `print_padovan`; apenas o buffer do vetor é alocado no heap.

Assim como a string `s` anterior, o vetor possui o buffer que contém seus elementos. Quando a variável `padovan` sai do escopo no final da função, o programa dropa o vetor. E como o vetor possui seu buffer, o buffer o acompanha.

O tipo `Box` do Rust serve como outro exemplo de posse. Um `Box<T>` é um ponteiro para um valor do tipo `T` armazenado no heap. Chamar `Box::new(v)` aloca algum espaço no heap, move o valor `v` para ele e retorna um `Box` apontando para o espaço do heap. Como uma `Box` possui o espaço para o qual aponta, quando o `Box` é dropado, ele libera o espaço também.

Por exemplo, você pode alocar uma tupla no heap da seguinte forma:

```
{
    let point = Box::new((0.625, 0.5)); // ponto alocado aqui
    let label = format!("{:?}", point); // rótulo alocado aqui
    assert_eq!(label, "(0.625, 0.5)");
```

```
} // ambos dropados aqui
```

Quando o programa chama `Box::new`, ele aloca espaço para uma tupla de dois valores `f64` no heap, move seu argumento `(0.625, 0.5)` para esse espaço e retorna um ponteiro para ele. No momento em que o controle atinge a chamada para `assert_eq!`, o stack frame se parece com a Figura 4.3.

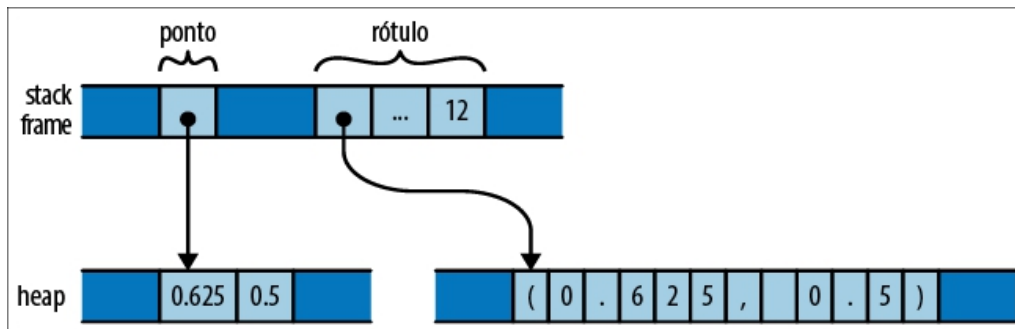


Figura 4.3: Duas variáveis locais, cada uma possuindo memória no heap.

O próprio stack frame contém as variáveis `point` e `label`, cada uma das quais referencia uma alocação de heap que ela possui. Quando eles são dropados, as alocações que possuem são liberadas com eles.

Assim como variáveis possuem seus valores, structs possuem seus campos e tuplas, arrays e vetores possuem seus elementos:

```
struct Person { name: String, birth: i32 }

let mut composers = Vec::new();
composers.push(Person { name: "Palestrina".to_string(),
                        birth: 1525 });
composers.push(Person { name: "Dowland".to_string(),
                        birth: 1563 });
composers.push(Person { name: "Lully".to_string(),
                        birth: 1632 });
for composer in &composers {
    println!("{}", born {}, composer.name, composer.birth);
}
```

Aqui, `composers` é um `Vec<Person>`, um vetor de structs, cada um contendo uma string e um número. Na memória, o valor final de `composers` se parece com a Figura 4.4.

Existem muitos relacionamentos de posse aqui, mas cada um é bastante direto: `composers` possui um vetor; o vetor possui seus

elementos, cada um dos quais é uma estrutura `Person`; cada estrutura possui seus campos; e o campo `string` possui seu texto. Quando o controle sai do escopo em que `composers` é declarado, o programa dropa seu valor e leva consigo todo o arranjo. Se houvesse outros tipos de coleções na imagem – um `HashMap`, talvez, ou um `BTreeSet` –, a história seria a mesma.

Neste ponto, dê um passo para trás e considere as consequências das relações de posse que apresentamos até agora. Cada valor tem um único proprietário, facilitando a decisão de quando dropá-lo. Mas um único valor pode possuir muitos outros valores: por exemplo, o vetor `composers` possui todos os seus elementos. E esses valores podem possuir outros valores por sua vez: cada elemento de `composers` possui uma `string`, que possui seu texto.

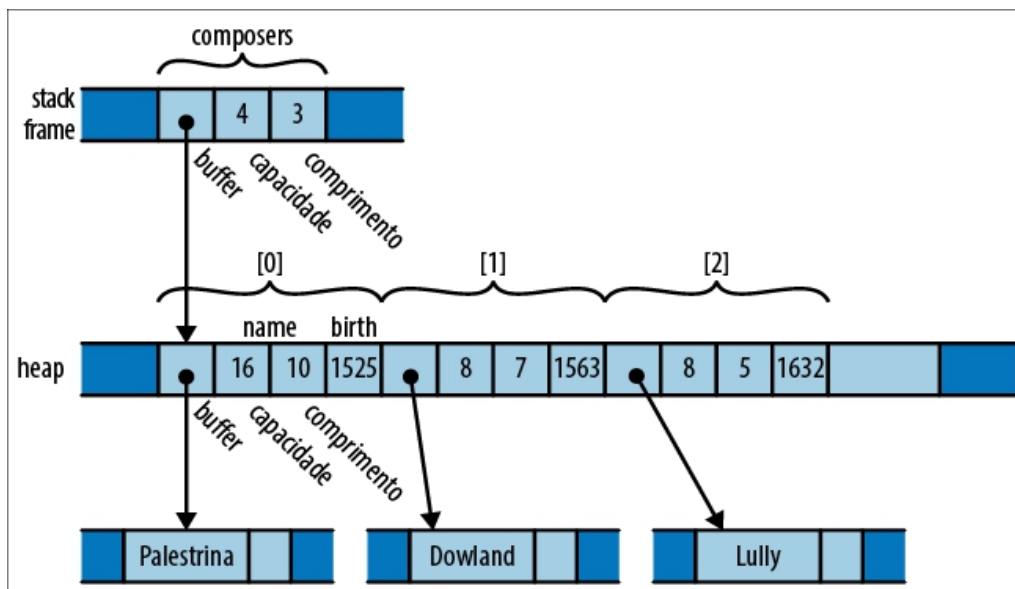


Figura 4.4: Uma árvore de posse mais complexa.

Segue-se que os proprietários e seus valores próprios formam árvores: seu proprietário é seu pai e os valores que você possui são seus filhos. E na raiz final de cada árvore está uma variável; quando essa variável sai do escopo, toda a árvore vai com ela. Podemos ver essa árvore de posse no diagrama para `composers`: não é uma “árvore” no sentido de uma estrutura de dados de árvore de busca, ou um documento HTML feito de elementos DOM. Em vez disso, temos uma árvore construída a partir de uma mistura de tipos, com a regra

de proprietário único do Rust proibindo qualquer reunião de estrutura que possa tornar o arranjo mais complexo do que uma árvore. Todo valor em um programa Rust é membro de alguma árvore, enraizado em alguma variável.

Os programas Rust geralmente não dropam valores explicitamente, da maneira como os programas C e C++ utilizariam `free` e `delete`. A maneira de dropar um valor no Rust é removê-lo da árvore de posse de alguma forma: deixando o escopo de uma variável ou excluindo um elemento de um vetor ou algo desse tipo. Nesse ponto, o Rust garante que o valor seja dropado adequadamente, com tudo o que possui.

Em certo sentido, o Rust é menos poderoso do que outras linguagens: todas as outras linguagens de programação práticas permitem que você construa grafos arbitrários de objetos que apontam uns para os outros da maneira que você achar melhor. Mas é exatamente pelo fato de o Rust ser menos poderoso que as análises que a linguagem pode fazer em seus programas podem ser mais poderosas. As garantias de segurança do Rust são possíveis exatamente porque os relacionamentos que ele pode encontrar em seu código são mais tratáveis. Isso faz parte da “aposta radical” do Rust que mencionamos anteriormente: na prática, o Rust defende, e geralmente há mais flexibilidade do que suficiente no modo como alguém resolve um problema para garantir que pelo menos algumas soluções perfeitamente boas caiam dentro das restrições que a linguagem impõe.

Dito isso, o conceito de posse como explicamos até agora ainda é muito rígido para ser útil. O Rust estende essa ideia simples de várias maneiras:

- Você pode mover (transferir) valores de um proprietário para outro. Isso permite construir, reorganizar e derrubar a árvore.
- Tipos muito simples como inteiros, números de ponto flutuante e caracteres são dispensados das regras de posse. Estes são chamados tipos `Copy`.
- A biblioteca padrão fornece os tipos de ponteiro contados por

referência `Rc` e `Arc`, que permitem que os valores tenham vários proprietários, sob algumas restrições.

- Você pode “emprestar uma referência” a um valor; as referências são ponteiros não proprietários, com tempo de vida limitado.

Cada uma dessas estratégias contribui com flexibilidade para o modelo de posse, mantendo as promessas do Rust. Explicaremos cada um por sua vez, com as referências abordadas no próximo capítulo.

Movimentos

No Rust, para a maioria dos tipos, operações como atribuir um valor a uma variável, passá-lo para uma função ou retorná-lo de uma função não copiam o valor: elas o *movem*. A origem cede a posse do valor para o destino e torna-se não inicializada; o destino agora controla o tempo de vida do valor. Os programas Rust constroem e destroem estruturas complexas, um valor por vez, um movimento por vez.

Você pode se surpreender com o fato de o Rust mudar o significado de tais operações fundamentais; certamente a atribuição é algo que deveria ser muito bem definido neste ponto da história. No entanto, se você observar atentamente como as diferentes linguagens escolheram lidar com as tarefas, verá que há uma variação significativa de uma escola para outra. A comparação também facilita a compreensão do significado e das consequências da escolha do Rust.

Considere o seguinte código Python:

```
s = ['udon', 'ramen', 'soba']  
t = s  
u = s
```

Cada objeto Python carrega uma contagem de referência, rastreando o número de valores que o estão referenciando atualmente. Assim, após a atribuição de `s`, o estado do programa se parece com a Figura 4.5 (observe que alguns campos foram deixados de fora).

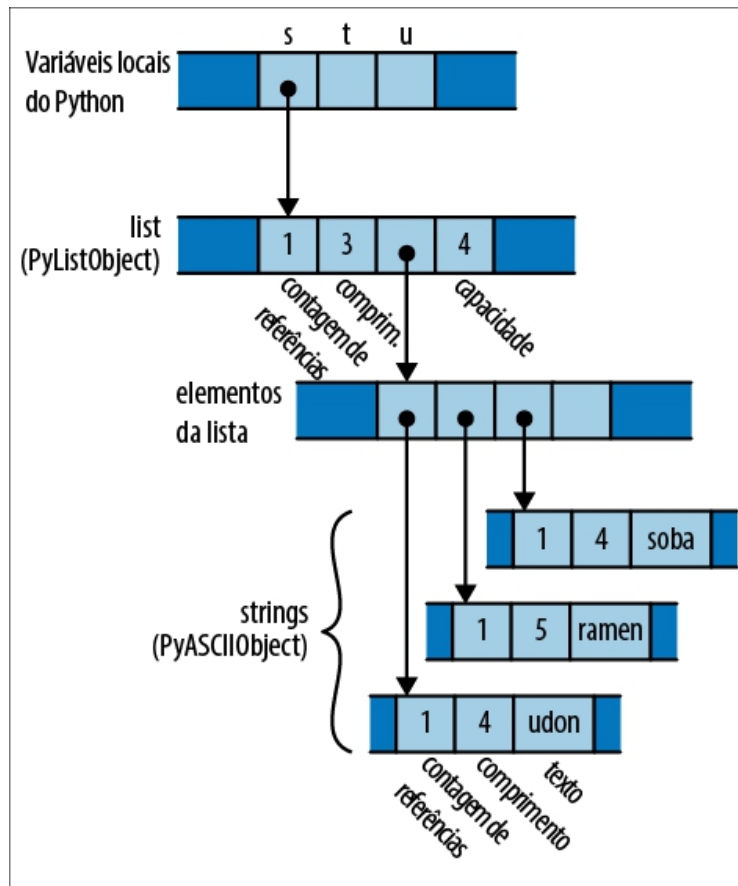


Figura 4.5: Como o Python representa uma lista de strings na memória.

Desde que apenas *s* está apontando para a lista, a contagem de referência da lista é 1; e como a lista é o único objeto apontando para as strings, cada uma de suas contagens de referência também é 1.

O que acontece quando o programa executa as atribuições para *t* e *u*? O Python implementa a atribuição simplesmente fazendo com que o destino aponte para o mesmo objeto que a origem e incrementando a contagem de referência do objeto. Portanto, o estado final do programa é algo como a Figura 4.6.

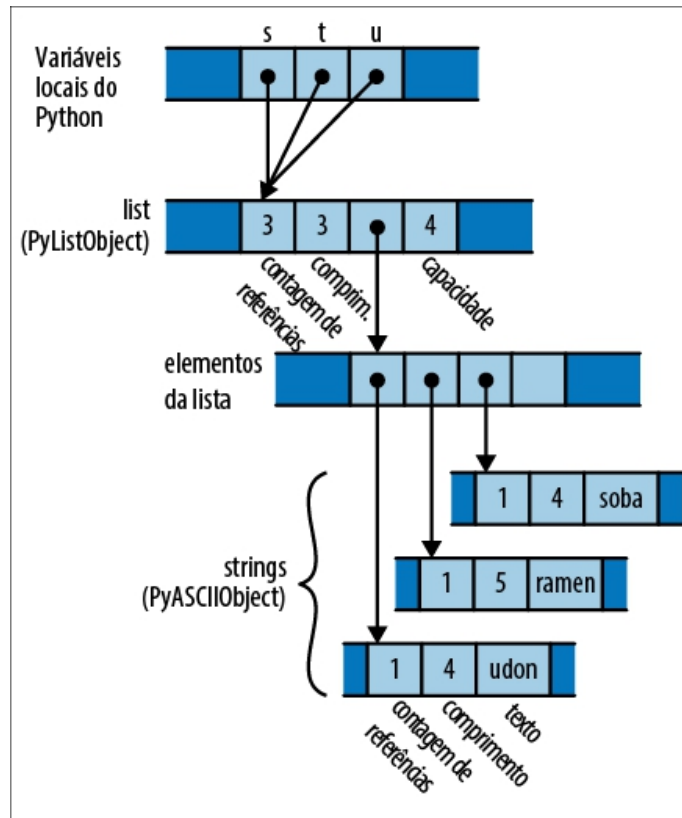


Figura 4.6: O resultado da atribuição de s tanto para t como para u no Python.

O Python copiou o ponteiro de s para t e u e atualizou a contagem de referência da lista para 3. A atribuição no Python é barata, mas, como cria uma nova referência ao objeto, devemos manter contagens de referência para saber quando podemos liberar o valor.

Agora considere o código C++ análogo:

```
using namespace std;
vector<string> s = { "udon", "ramen", "soba" };
vector<string> t = s;
vector<string> u = s;
```

O valor inicial de s se parece com a Figura 4.7 na memória.

O que acontece quando o programa atribui s para t e u ? Atribuir um `std::vector` produz uma cópia do vetor em C++; `std::string` se comporta de forma semelhante. Portanto, quando o programa chega ao final desse código, ele já alocou três vetores e nove strings (Figura 4.8).

Dependendo dos valores envolvidos, a atribuição em C++ pode consumir quantidades sem controle de memória e tempo do

processador. A vantagem, porém, é que é fácil para o programa decidir quando liberar toda essa memória: quando as variáveis saem do escopo, tudo alocado aqui é liberado automaticamente.

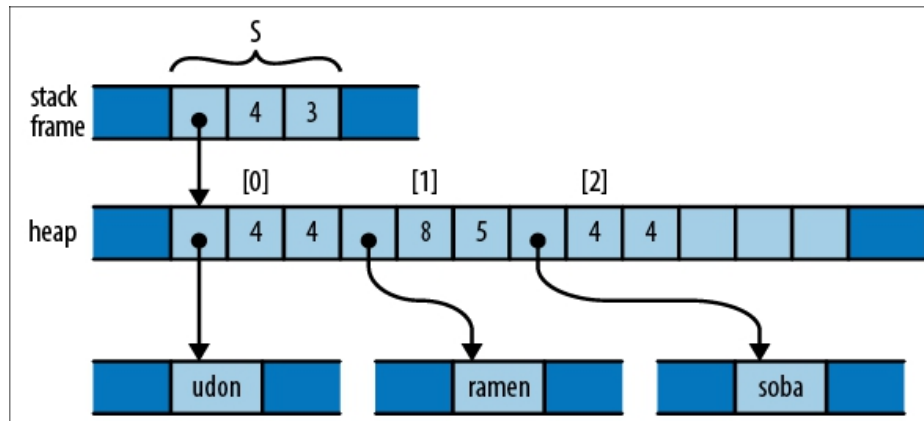


Figura 4.7: Como o C++ representa um vetor de strings na memória.

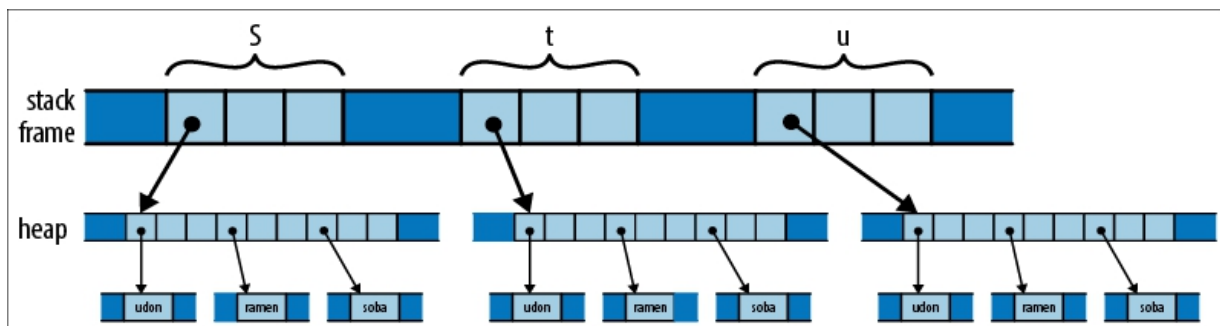


Figura 4.8: O resultado da atribuição s tanto para t como para u em C++.

De certa forma, C++ e Python escolheram compensações opostas: o Python torna a atribuição barata, exigindo contagem de referência (e, no caso geral, coleta de lixo). O C++ mantém a posse de toda a memória clara, à custa de fazer a atribuição realizar uma cópia profunda do objeto. Os programadores C++ geralmente não se entusiasmam com essa escolha: cópias profundas podem ser caras e geralmente há alternativas mais práticas.

Então, o que o programa análogo faria no Rust? Eis o código:

```
let s = vec!["udon".to_string(), "ramen".to_string(), "soba".to_string()];  
let t = s;  
let u = s;
```

Como C e C++, o Rust coloca literais de string simples como "udon" na memória somente leitura; portanto, para uma comparação mais clara com os exemplos C e Python, chamamos `to_string` aqui para obter valores `String` alocados no heap.

Após realizar a inicialização do `s`, uma vez que o Rust e C++ utilizam representações semelhantes para vetores e strings, a situação parece exatamente como em C++ (Figura 4.9).

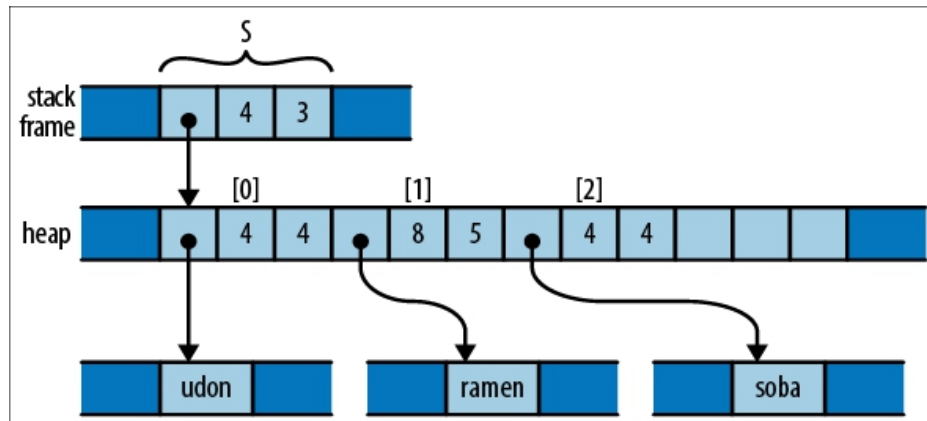


Figura 4.9: Como o Rust representa um vetor de strings na memória.

Mas lembre-se de que, no Rust, as atribuições da maioria dos tipos movem o valor da origem para o destino, deixando a origem não inicializada. Então, depois de inicializar `t`, a memória do programa se parece com a Figura 4.10.

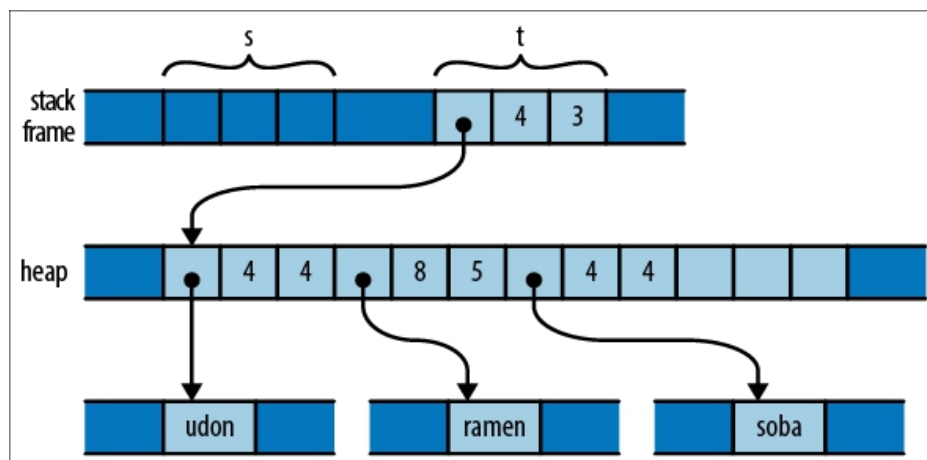


Figura 4.10: O resultado da atribuição de `s` para `t` no Rust.

O que aconteceu aqui? A inicialização `let t = s;` moveu os três campos de cabeçalho do vetor de `s` para `t`; agora `t` possui o vetor. Os elementos do vetor permaneceram exatamente onde estavam e nada aconteceu com as strings também. Cada valor ainda tem um único proprietário, embora um tenha mudado de mãos. Não havia contagens de referência a serem ajustadas. E o compilador agora considera `s` não inicializado.

Então, o que acontece quando atingimos a inicialização `let u = s;`? Isso atribuiria o valor não inicializado de `s` para `u`. O Rust proíbe, prudentemente, o uso de valores não inicializados, então o compilador rejeita esse código com o seguinte erro:

```
error: use of moved value: `s`
|
7 | let s = vec!["udon".to_string(), "ramen".to_string(), "soba".to_string()];
|   - move occurs because `s` has type `Vec<String>`,
|     which does not implement the `Copy` trait
8 | let t = s;
|   - value moved here
9 | let u = s;
|   ^ value used here after move
```

Considere as consequências do uso de um movimento pelo Rust aqui. Como o Python, a atribuição é barata: o programa simplesmente move o cabeçalho de três palavras do vetor de um local para outro. Mas, assim como ocorre no C++, a posse é sempre clara: o programa não precisa de contagem de referência ou coleta de lixo para saber quando liberar os elementos do vetor e o conteúdo da string.

O preço que você paga é que deve solicitar cópias explicitamente quando precisar delas. Se quiser terminar no mesmo estado do programa C++, com cada variável contendo uma cópia independente da estrutura, você deve chamar o método `clone` do vetor, que executa uma cópia profunda do vetor e seus elementos:

```
let s = vec!["udon".to_string(), "ramen".to_string(), "soba".to_string()];
let t = s.clone();
let u = s.clone();
```

Você também pode recriar o comportamento do Python utilizando os tipos de ponteiro contados por referência do Rust; discutiremos isso

em breve em “Rc e Arc: Posse compartilhada”, na página 125.

Mais operações que movem

Nos exemplos até agora, mostramos inicializações, fornecendo valores para variáveis à medida que entram no escopo em uma declaração `let`. Atribuir um valor a uma variável é um pouco diferente, pois, se você mover um valor para uma variável que já foi inicializada, o Rust dropa o valor anterior da variável. Por exemplo:

```
let mut s = "Govinda".to_string();  
s = "Siddhartha".to_string(); // valor "Govinda" dropado aqui
```

Neste código, quando o programa atribui a string "Siddhartha" a `s`, seu valor anterior "Govinda" é dropado primeiro. Mas considere o seguinte:

```
let mut s = "Govinda".to_string();  
let t = s;  
s = "Siddhartha".to_string(); // nada é dropado aqui
```

Desta vez, `t` tomou posse da string original de `s`, de modo que, no momento em que atribuímos um valor a `s`, essa variável não está inicializada. Nesse cenário, nenhuma string é dropada.

Utilizamos inicializações e atribuições nos exemplos aqui porque são simples, mas o Rust aplica a semântica de movimento a quase qualquer uso de um valor. Passar argumentos para funções transfere a posse para os parâmetros da função; retornar um valor de uma função transfere a posse para o chamador. A construção de uma tupla move os valores para a tupla. E assim por diante.

Agora você pode ter uma visão melhor do que realmente está acontecendo nos exemplos que oferecemos na seção anterior. Por exemplo, quando estávamos construindo nosso vetor de compositores, escrevemos:

```
struct Person { name: String, birth: i32 }  
  
let mut composers = Vec::new();  
composers.push(Person { name: "Palestrina".to_string(),  
                        birth: 1525 });
```

Esse código mostra vários locais em que os movimentos ocorrem, além da inicialização e atribuição:

Retornando valores de uma função

A chamada `Vec::new()` constrói um novo vetor e retorna, não um ponteiro para o vetor, mas o próprio vetor: sua posse se move de `Vec::new` para a variável `composers`. Da mesma forma, a chamada a `to_string` retorna uma nova instância de `String`.

Construindo novos valores

O campo `name` da nova estrutura `Person` é inicializado com o valor de retorno de `to_string`. A estrutura toma posse da string.

Passando valores para uma função

A estrutura `Person` inteira, e não um ponteiro para ela, é passada para método `push` do vetor, que o move para o final da estrutura. O vetor toma posse de `Person` e assim se torna o proprietário indireto do nome `String` também.

Mover valores como esse pode parecer ineficiente, mas há duas coisas a serem lembradas. Primeiro, os movimentos sempre se aplicam ao valor adequado, não ao armazenamento no heap que eles possuem. Para vetores e strings, o *valor adequado* é o cabeçalho de três palavras sozinho; os arrays de elementos potencialmente grandes e os buffers de texto ficam onde estão no heap. Em segundo lugar, a geração de código do compilador Rust é boa em “ver através” todos esses movimentos; na prática, o código de máquina geralmente armazena o valor diretamente aonde ele pertence.

Movimentos e fluxo de controle

Todos os exemplos anteriores têm um fluxo de controle muito simples; como os movimentos interagem com códigos mais complicados? O princípio geral é que, se for possível que uma variável tenha seu valor removido e não tenha recebido um novo valor desde então, ela é considerada não inicializada. Por exemplo, se uma variável ainda tiver um valor após a avaliação da condição de uma expressão `if`, então podemos usá-la em ambas as ramificações:

```
let x = vec![10, 20, 30];
if c {
    f(x); // ... ok mover de x aqui
} else {
```



```
    g(x); // ... e ok para também passar de x aqui
}
h(x); // ruim: x não está inicializado aqui se qualquer caminho o utilizar
```

Por razões semelhantes, mover um valor de uma variável em um loop é proibido:

```
let x = vec![10, 20, 30];
while f() {
    g(x); // ruim: x seria movido na primeira iteração,
          // não inicializada na segunda
}
```

Isto é, a menos que tenhamos definitivamente dado um novo valor na próxima iteração:

```
let mut x = vec![10, 20, 30];
while f() {
    g(x); // move de x
    x = h(); // atribui a x um novo valor
}
e(x);
```

Movimentos e conteúdo indexado

Mencionamos que um movimento deixa sua origem não inicializada, pois o destino assume a posse do valor. Mas nem todo tipo de proprietário de valor está preparado para se tornar não inicializado. Por exemplo, considere o seguinte código:

```
// Construa um vetor das strings "101", "102", ... "105"
let mut v = Vec::new();
for i in 101 .. 106 {
    v.push(i.to_string());
}
// Retire elementos aleatórios do vetor
let third = v[2]; // erro: Não é possível sair do índice de Vec
let fifth = v[4]; // aqui também
```

Para que isso funcione, o Rust precisaria, de alguma forma, lembrar que o terceiro e o quinto elementos do vetor não foram inicializados e rastrear essas informações até que o vetor seja dropado. No caso mais geral, os vetores precisariam carregar informações extras com eles para indicar quais elementos estão ativos e quais não foram inicializados. Esse claramente não é o comportamento correto para uma linguagem de programação de sistemas; um vetor não deve ser

nada além de um vetor. Na verdade, o Rust rejeita o código anterior com o seguinte erro:

```
error: cannot move out of index of `Vec<String>`
|
14 |     let third = v[2];
|                   ^^^^
|                   |
|                   move occurs because value has type `String`,
|                   which does not implement the `Copy` trait
|                   help: consider borrowing here: `&v[2]`
```

Ele também faz uma reclamação semelhante sobre a mudança para `fifth`. Na mensagem de erro, o Rust sugere o uso de uma referência, caso você queira acessar o elemento sem movê-lo. Isso é muitas vezes o que você quer. Mas e se você realmente quiser mover um elemento para fora de um vetor? Você precisa encontrar um método que o faça respeitando as limitações do tipo. Aqui estão três possibilidades:

```
// Construa um vetor das strings "101", "102", ... "105"
let mut v = Vec::new();
for i in 101 .. 106 {
    v.push(i.to_string());
}

// 1. Retire um valor do final do vetor:
let fifth = v.pop().expect("vector empty!");
assert_eq!(fifth, "105");

// 2. Mova um valor para fora de um determinado índice no vetor,
// e mova o último elemento para o seu lugar:
let second = v.swap_remove(1);
assert_eq!(second, "102");

// 3. Troque outro valor pelo que estamos tirando:
let third = std::mem::replace(&mut v[2], "substitute".to_string());
assert_eq!(third, "103");

// Vamos ver o que resta do nosso vetor
assert_eq!(v, vec!["101", "104", "substitute"]);
```

Cada um desses métodos move um elemento para fora do vetor, mas o faz de maneira a deixar o vetor em um estado totalmente preenchido, embora talvez menor.

Tipos de coleção como `Vec` geralmente também oferecem métodos para consumir todos os seus elementos em um loop:

```

let v = vec!["liberté".to_string(),
             "égalité".to_string(),
             "fraternité".to_string()];

for mut s in v {
    s.push('!');
    println!("{}", s);
}

```

Quando passamos o vetor para o loop diretamente, como em `for ... in v`, isso move o vetor para fora de `v`, deixando `v` não inicializado. O mecanismo interno do loop `for` toma posse do vetor e o dissectiona em seus elementos. A cada iteração, o loop move outro elemento para a variável `s`. Como agora `s` possui a string, podemos modificá-la no corpo do loop antes de imprimi-la. E, como o próprio vetor não é mais acessível para o código, nada pode observá-lo durante o loop em algum estado parcialmente vazio.

Se você precisar mover um valor de um proprietário que o compilador não pode rastrear, considere alterar o tipo do proprietário para algo que possa rastrear dinamicamente se ele tem um valor ou não. Por exemplo, eis uma variante do exemplo anterior:

```

struct Person { name: Option<String>, birth: i32 }

let mut composers = Vec::new();
composers.push(Person { name: Some("Palestrina".to_string()),
                        birth: 1525 });

```

Você não pode fazer isto:

```
let first_name = composers[0].name;
```

Isso apenas provocará o mesmo erro “não é possível sair do índice” mostrado anteriormente. Mas como você mudou o tipo de campo `name` de `String` para `Option<String>`, isso significa que `None` é um valor legítimo para o campo armazenar, então isso funciona:

```

let first_name = std::mem::replace(&mut composers[0].name, None);
assert_eq!(first_name, Some("Palestrina".to_string()));
assert_eq!(composers[0].name, None);

```

A chamada `replace` move para fora o valor de `composers[0].name`, deixando `None` em seu lugar, e passa a posse do valor original para seu chamador. Na verdade, utilizando `Option` desta forma é bastante comum que o tipo forneça um método `take` para esse fim. Você poderia escrever a manipulação anterior de forma mais legível da

seguinte forma:

```
let first_name = composers[0].name.take();
```

Essa chamada para `take` tem o mesmo efeito que a chamada anterior para `replace`.

Tipos de cópia: A exceção aos movimentos

Os exemplos que mostramos até agora, de valores sendo movidos, envolvem vetores, strings e outros tipos que podem utilizar muita memória e ser caros para copiar. Os movimentos mantêm clara a posse de tais tipos e barata a atribuição. Mas para tipos mais simples, como números inteiros ou caracteres, esse tipo de tratamento cuidadoso realmente não é necessário.

Compare o que acontece na memória quando atribuímos uma `String` com o que acontece quando atribuímos um valor `i32`:

```
let string1 = "somnambulance".to_string();  
let string2 = string1;
```

```
let num1: i32 = 36;  
let num2 = num1;
```

Depois de executar esse código, a memória se parece com a Figura 4.11.

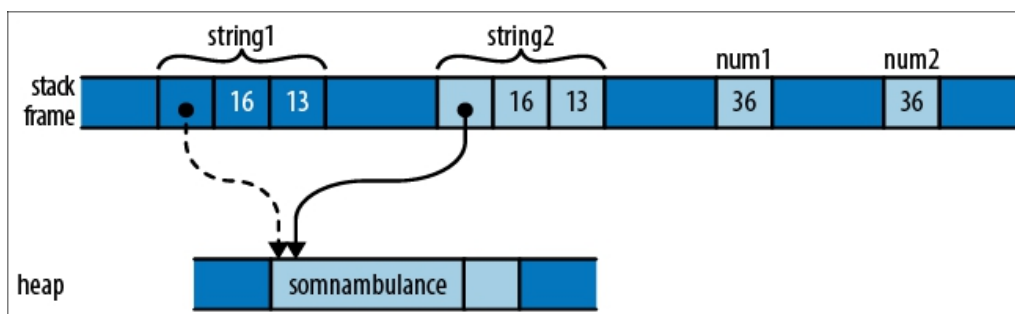


Figura 4.11: Atribuir uma `String` move o valor, enquanto atribuir um `i32` o copia.

Como com os vetores anteriores, a atribuição move `string1` para `string2` para que não acabemos com duas strings responsáveis por liberar o mesmo buffer. Mas a situação com `num1` e `num2` é diferente. Um `i32` é simplesmente um padrão de bits na memória; ele não possui nenhum recurso no heap ou realmente não depende de nada além

dos bytes que ele contém. No momento em que movemos seus bits para `num2`, fizemos uma cópia totalmente independente de `num1`.

Mover um valor deixa a origem do movimento não inicializada. Mas considerando que serve a um propósito essencial tratar `string1` como sem valor, tratar `num1` dessa forma é sem sentido; não haveria nenhum dano em continuar a usá-lo. As vantagens de um movimento não se aplicam aqui e são inconvenientes.

Anteriormente, tivemos o cuidado de dizer que a maioria dos tipos são movidos; agora chegamos às exceções, os tipos que o Rust designa como *tipos copiados* (*Copy types*). Atribuir um valor de um tipo `Copy` copia o valor, em vez de movê-lo. A fonte da atribuição permanece inicializada e utilizável, com o mesmo valor que tinha antes. A passagem de tipos `Copy` para funções e construtores se comporta de forma semelhante.

Os tipos `Copy` padrão incluem todos os tipos numéricos inteiros e de ponto flutuante da máquina, os tipos `char` e `bool`, e alguns outros. Uma tupla ou array de tamanho fixo de tipos `Copy` é ela própria um tipo `Copy`.

Somente os tipos para os quais uma simples cópia de bit a bit é suficiente podem ser `Copy`. Como já explicamos, `String` não é um tipo `Copy`, porque ele possui um buffer alocado no heap. Por razões semelhantes, `Box<T>` não é `Copy`; ele possui seu referente alocado no heap. O tipo `File`, representando um identificador de arquivo do sistema operacional, não é `Copy`; duplicar tal valor implicaria solicitar ao sistema operacional outro identificador de arquivo. Da mesma forma, o tipo `MutexGuard`, representando um mutex bloqueado, não é `Copy`: esse tipo não faz nem sentido copiar, pois apenas um thread pode conter um mutex por vez.

Como regra geral, qualquer tipo que precise fazer algo especial quando um valor é dropado não pode ser `Copy`: um `Vec` precisa liberar seus elementos, um `File` precisa fechar seu identificador de arquivo, um `MutexGuard` precisa desbloquear seu mutex e assim por diante. A duplicação bit a bit desses tipos não deixaria claro qual valor seria responsável agora pelos recursos do original.

E os tipos que você mesmo define? Por padrão, os tipos `struct` e `enum` não são `Copy`:

```
struct Label { number: u32 }

fn print(l: Label) { println!("STAMP: {}", l.number); }

let l = Label { number: 3 };
print(l);
println!("My label number is: {}", l.number);
```

Isso não vai compilar; o Rust reclama:

```
error: borrow of moved value: `l`
  |
10 |   let l = Label { number: 3 };
   |       - move occurs because `l` has type `main::Label`,
   |       which does not implement the `Copy` trait
11 |   print(l);
   |       - value moved here
12 |   println!("My label number is: {}", l.number);
   |                                     ^^^^^^^^^
   |                                     value borrowed here after move
```

Como `Label` não é `Copy`, passá-lo para `print` transferiu a posse do valor para a função `print`, que a dropou antes de retornar. Mas isso é bobagem; um `Label` nada mais é do que um `u32` com pretensões. Não há razão para a passagem de `l` para `print` mover o valor.

Mas os tipos definidos pelo usuário sendo não-`Copy` são apenas o padrão. Se todos os campos de seu `struct` forem `Copy`, então você também pode fazer o tipo `Copy` colocando o atributo `#[derive(Copy, Clone)]` acima da definição, assim:

```
#[derive(Copy, Clone)]
struct Label { number: u32 }
```

Com essa alteração, o código anterior compila sem reclamar. Contudo, se tentarmos isso em um tipo cujos campos não são todos `Copy`, não funcionará. Suponha que compilamos o seguinte código:

```
#[derive(Copy, Clone)]
struct StringLabel { name: String }
```

Ele provoca este erro:

```
error: the trait `Copy` may not be implemented for this type
  |
7 | #[derive(Copy, Clone)]
  |     ^^^^
```

```
8 | struct StringLabel { name: String }  
  | ----- this field does not implement `Copy`
```

Por que os tipos definidos pelo usuário não são automaticamente `Copy`, assumindo que eles são elegíveis? Se um tipo é `Copy` ou não tem um grande efeito sobre como o código pode usá-lo: os tipos `Copy` são mais flexíveis, pois a atribuição e as operações relacionadas não deixam o original não inicializado. Mas, para o implementador de um tipo, o oposto é verdadeiro: tipos `Copy` são muito limitados em relação a quais tipos eles podem conter, enquanto os tipos não-`Copy` podem utilizar alocação de heap e possuir outros tipos de recursos. Então, tornar um tipo `Copy` representa um sério compromisso por parte do implementador: se for necessário alterá-lo para não-`Copy` mais tarde, muito do código que o utiliza provavelmente precisará ser adaptado.

Enquanto o C++ permite sobrecarregar operadores de atribuição e definir construtores especializados para copiar e mover, o Rust não permite esse tipo de personalização. No Rust, cada movimento é uma cópia superficial byte por byte que deixa a origem não inicializada. As cópias são as mesmas, exceto que a fonte permanece inicializada. Isso significa que as classes C++ podem fornecer interfaces convenientes que os tipos Rust não podem, onde o código de aparência comum ajusta implicitamente as contagens de referência, adia cópias caras para mais tarde ou utiliza outros truques de implementação sofisticados.

Mas o efeito dessa flexibilidade em C++ como linguagem é tornar as operações básicas como atribuição, passagem de parâmetros e retorno de valores de funções menos previsíveis. Por exemplo, anteriormente neste capítulo, mostramos como atribuir uma variável a outra em C++ pode exigir quantidades arbitrárias de memória e tempo de processador. Um dos princípios do Rust é que os custos devem ser aparentes para o programador. As operações básicas devem permanecer simples. Operações potencialmente caras devem ser explícitas, como as chamadas para `clone` no exemplo anterior, que fazem cópias profundas de vetores e das strings que eles contêm.

Nesta seção, falamos sobre `Copy` e `Clone` em termos vagos, como

características que um tipo pode ter. De fato, eles são exemplos de traits, a facilidade aberta do Rust para categorizar tipos com base no que você pode fazer com eles. Descrevemos os traits em geral no Capítulo 11 e Copy e Clone em particular no Capítulo 13.

Rc e Arc: Posse compartilhada

Embora a maioria dos valores tenha proprietários únicos no código Rust típico, em alguns casos é difícil encontrar para cada valor um único proprietário que tenha o tempo de vida necessário; você gostaria que o valor simplesmente existisse até que todos terminassem de usá-lo. Para esses casos, o Rust fornece os tipos de ponteiro contados por referência Rc e Arc. Como seria de se esperar do Rust, eles são totalmente seguros de utilizar: você não pode esquecer de ajustar a contagem de referência, de criar outros ponteiros para o referente que o Rust não perceba, ou tropeçar em qualquer um dos outros tipos de problemas que acompanham tipos de ponteiros com contagem de referência em C++.

Os tipos Rc e Arc são muito semelhantes; a única diferença entre eles é que um Arc é seguro compartilhar entre threads diretamente – o nome Arc é a abreviação de *atomic reference count* (contagem de referência atômica) – enquanto um simples Rc utiliza um código que não é thread-safe mais rápido para atualizar sua contagem de referência. Se você não precisa compartilhar os ponteiros entre threads, não há razão para pagar a penalidade de desempenho de um Arc, então você deve utilizar Rc; o Rust impedirá que você passe acidentalmente por um limite de thread. Os dois tipos são equivalentes; portanto, no restante desta seção, falaremos apenas sobre Rc.

Anteriormente, mostramos como o Python utiliza contagens de referência para gerenciar o tempo de vida de seus valores. Você pode utilizar Rc para obter um efeito semelhante no Rust. Considere o seguinte código:

```
use std::rc::Rc;
```

```
// O Rust pode inferir todos esses tipos; escritos aqui para maior clareza
```



```
let s: Rc<String> = Rc::new("shirataki".to_string());
let t: Rc<String> = s.clone();
let u: Rc<String> = s.clone();
```

Para qualquer tipo T , um valor $Rc<T>$ é um ponteiro para um T alocado no heap que teve uma contagem de referência afixada nele. Clonar um valor $Rc<T>$ não copia o T ; em vez disso, simplesmente cria outro ponteiro para ele e incrementa a contagem de referência. Portanto, o código anterior produz a situação ilustrada na Figura 4.12 na memória.

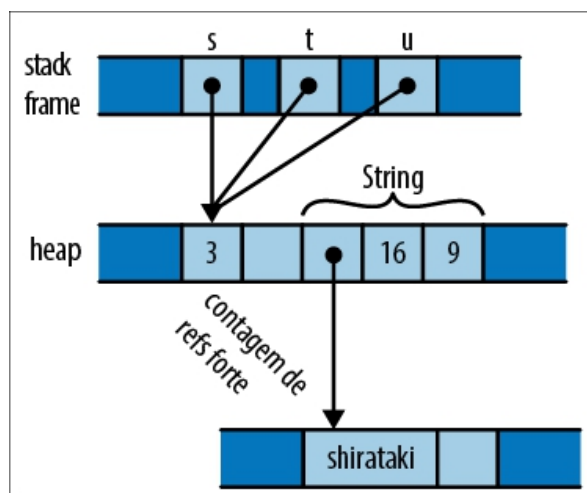


Figura 4.12: Uma string com contagem de referência com três referências.

Cada um dos três $Rc<String>$ ponteiros está referenciando o mesmo bloco de memória, que contém uma contagem de referências e espaço para a `String`. As regras usuais de posse se aplicam aos próprios ponteiros Rc e, quando o último Rc existente é dropado, o Rust dropa a `String` também.

Você pode utilizar qualquer um dos métodos usuais de `String` diretamente em um $Rc<String>$:

```
assert!(s.contains("shira"));
assert_eq!(t.find("taki"), Some(5));
println!("{}", u);
```

Um valor pertencente a um Rc ponteiro é imutável. Suponha que você tente adicionar algum texto ao final da string:

```
s.push_str(" noodles");
```

O Rust vai recusar:

```
error: cannot borrow data in an `Rc` as mutable
```

```
13 | s.push_str(" noodles");  
    | ^ cannot borrow as mutable
```

As garantias de memória e thread-safe do Rust dependem da garantia de que nenhum valor seja simultaneamente compartilhado e mutável. O Rust assume que o referente de um ponteiro `Rc` pode, em geral, ser compartilhado, então não deve ser mutável. Explicamos por que essa restrição é importante no Capítulo 5.

Um problema bem conhecido com o uso de contagens de referência para gerenciar a memória é que, se houver dois valores com contagem de referência que apontem um para o outro, cada um manterá a contagem de referência do outro acima de zero, de modo que os valores nunca serão liberados (Figura 4.13).

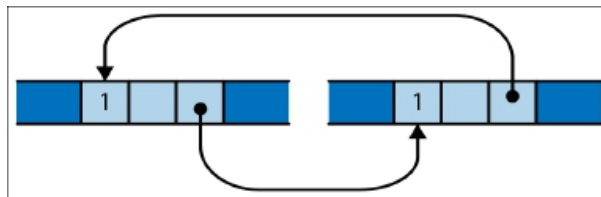


Figura 4.13: Um loop de contagem de referência; esses objetos não serão liberados.

É possível que valores vazem no Rust dessa maneira, mas essas situações são raras. Você não pode criar um ciclo sem, em algum momento, fazer um valor mais antigo apontar para um valor mais novo. Isso obviamente requer que o valor mais antigo seja mutável. Como os ponteiros `Rc` mantêm seus referentes imutáveis, normalmente não é possível criar um ciclo. Mas o Rust fornece maneiras de criar partes mutáveis de valores imutáveis; isso se chama *mutabilidade interior* e o abordamos em “Mutabilidade interior”, na página 265. Se combinar essas técnicas com ponteiros `Rc`, você pode criar um ciclo e vazarmos memória.

Às vezes, você pode evitar criar ciclos de ponteiros `Rc` utilizando ponteiros fracos, `std::rc::Weak`, para alguns dos links. Entretanto, não

abordaremos isso neste livro; consulte a documentação da biblioteca padrão para obter detalhes.

Movimentos e ponteiros contados por referência são duas maneiras de relaxar a rigidez da árvore de posse. No próximo capítulo, veremos uma terceira maneira: emprestar referências a valores. Depois de se sentir confortável com a posse e o empréstimo, você terá escalado a parte mais íngreme da curva de aprendizagem do Rust e estará pronto para aproveitar os pontos fortes exclusivos dele.

CAPÍTULO 5

Referências

As bibliotecas não podem fornecer novas incapacidades.
– Mark Miller

Todos os tipos de ponteiro que vimos até agora – o simples ponteiro de heap `Box<T>` e os ponteiros internos para valores `String` e `Vec` – são ponteiros proprietários: quando o proprietário é dropado, o referente vai com ele. O Rust também possui tipos de ponteiro não proprietários chamados *referências*, que não têm efeito sobre o tempo de vida de seus referentes.

Na verdade, é exatamente o oposto: as referências nunca devem sobreviver a seus referentes. Você deve deixar claro em seu código que nenhuma referência pode sobreviver ao valor para o qual aponta. Para enfatizar isso, o Rust refere-se a criar uma referência a algum valor como *emprestar* o valor: o que você pegou emprestado, deve devolver ao seu proprietário ao final.

Se você sentiu um momento de ceticismo ao ler a frase “Você deve deixar claro em seu código”, está em excelente companhia. As referências em si não são nada de especial – sob o capô, são apenas endereços. Mas as regras que as mantêm seguras são novas para Rust; fora das linguagens de pesquisa, você nunca viu nada parecido antes. E, embora essas regras sejam a parte do Rust que requer mais esforço para dominar, a amplitude de erros clássicos e absolutamente cotidianos que elas evitam é surpreendente e seu efeito na programação multithread é libertador. Essa é a aposta radical do Rust, novamente.

Neste capítulo, vamos ver como as referências funcionam no Rust; mostrar como referências, funções e tipos definidos pelo usuário incorporam informações de tempo de vida (uso) para garantir que sejam utilizados com segurança; e ilustrar algumas categorias

comuns de bugs que esses esforços evitam, em tempo de compilação e sem penalidades de desempenho em tempo de execução.

Referências a valores

Como exemplo, suponha que vamos construir uma tabela de artistas da Renascença que pintaram assassinatos e as obras pelas quais são conhecidos. A biblioteca padrão do Rust inclui um tipo de tabela hash, ou tabela de dispersão, então podemos definir nosso tipo assim:

```
use std::collections::HashMap;
```

```
type Table = HashMap<String, Vec<String>>;
```

Em outras palavras, essa é uma tabela hash que mapeia valores `String` para valores `Vec<String>`, levando o nome de um artista a uma lista de nomes de suas obras. Você pode iterar pelas entradas de um `HashMap` com um loop `for`, e assim podemos escrever uma função para imprimir uma `Table`:

```
fn show(table: Table) {  
    for (artist, works) in table {  
        println!("works by {}: ", artist);  
        for work in works {  
            println!("  {}", work);  
        }  
    }  
}
```

Construir e imprimir a tabela é simples:

```
fn main() {  
    let mut table = Table::new();  
    table.insert("Gesualdo".to_string(),  
                vec!["many madrigals".to_string(),  
                    "Tenebrae Responsoria".to_string()]);  
    table.insert("Caravaggio".to_string(),  
                vec!["The Musicians".to_string(),  
                    "The Calling of St. Matthew".to_string()]);  
    table.insert("Cellini".to_string(),  
                vec!["Perseus with the head of Medusa".to_string(),  
                    "a salt cellar".to_string()]);  
  
    show(table);  
}
```

E tudo funciona bem:

```
$ cargo run
  Running `/home/jimb/rust/book/fragments/target/debug/fragments`
works by Gesualdo:
  many madrigals
  Tenebrae Responsoria
works by Cellini:
  Perseus with the head of Medusa
  a salt cellar
works by Caravaggio:
  The Musicians
  The Calling of St. Matthew
$
```

Mas, se você leu a seção do capítulo anterior sobre movimentos, essa definição para `show` deve levantar algumas questões. Em particular, `HashMap` não é `Copy` – nem pode ser, pois possui uma tabela alocada dinamicamente. Então, quando o programa chama `show(table)`, toda a estrutura é movida para a função, deixando a variável `table` não inicializada. (O programa também itera pelo seu conteúdo sem nenhuma ordem específica; portanto, se você obteve uma ordem diferente, não se preocupe.) Se o código de chamada tentar utilizar `table` agora, ele terá problemas:

```
...
show(table);
assert_eq!(table["Gesualdo"][0], "many madrigals");
```

O Rust reclama que `table` não está mais disponível:

```
error: borrow of moved value: `table`
|
20 |   let mut table = Table::new();
|       ^^^^^^^^^ move occurs because `table` has type
|                   `HashMap<String, Vec<String>>`,
|                   which does not implement the `Copy` trait
...
31 |   show(table);
|       ^^^^^ value moved here
32 |   assert_eq!(table["Gesualdo"][0], "many madrigals");
|       ^^^^^^ value borrowed here after move
```

De fato, se olharmos para a definição de `show`, o loop `for` exterior toma posse da tabela `hash` e a consome inteiramente; e o loop `for` interior faz o mesmo com cada um dos vetores. (Vimos esse

comportamento anteriormente, no exemplo “liberté, égalité, fraternité”.) Por causa da semântica do movimento, destruimos completamente toda a estrutura simplesmente tentando imprimi-la. Obrigado, Rust!

A maneira correta de lidar com isso é utilizar referências. Uma referência permite acessar um valor sem afetar sua posse. As referências são de dois tipos:

- Uma *referência compartilhada* permite que você leia, mas não modifique seu referente. Contudo, você pode ter tantas referências compartilhadas quanto desejar para um determinado valor. A expressão `&e` produz uma referência compartilhada para valor de `e`; se `e` tem o tipo τ , então `&e` tem o tipo $\&\tau$, que se pronuncia como “ref τ ”. As referências compartilhadas são *Copy*.
- Se você tem uma *referência mutável* a um valor, pode ler e modificar o valor. No entanto, não pode ter nenhuma outra referência ativa de qualquer tipo para esse valor ao mesmo tempo. A expressão `&mut e` produz uma referência mutável para valor de `e`; você escreve seu tipo como `&mut T`, que se pronuncia como “ref mut τ ”. Referências mutáveis não são *Copy*.

Você pode pensar na distinção entre referências compartilhadas e mutáveis como uma forma de impor uma regra *vários leitores ou único escritor* em tempo de compilação. Na verdade, essa regra não se aplica apenas a referências; abrange também o proprietário do valor emprestado. Enquanto houver referências compartilhadas a um valor, nem mesmo seu proprietário poderá modificá-lo; o valor está bloqueado. Ninguém pode modificar `table` enquanto `show` estiver trabalhando com ela. Da mesma forma, se houver uma referência mutável a um valor, ela terá acesso exclusivo ao valor; você não pode utilizar o proprietário de forma alguma, até que a referência mutável desapareça. Manter o compartilhamento e a mutação totalmente separados acaba sendo essencial para a segurança da memória, por motivos que abordaremos mais adiante neste capítulo.

A função `print` do nosso exemplo não precisa modificar a tabela, basta ler seu conteúdo. Portanto, o chamador deve ser capaz de passar uma referência compartilhada para a tabela, como segue:

```
show(&table);
```

Referências são ponteiros não proprietários, então a variável `table` permanece como proprietária de toda a estrutura; `show` apenas a emprestou por um tempo. Naturalmente, precisaremos ajustar a definição de `show` para combinar, mas você terá de olhar de perto para ver a diferença:

```
fn show(table: &Table) {  
    for (artist, works) in table {  
        println!("works by {}: ", artist);  
        for work in works {  
            println!("  {}", work);  
        }  
    }  
}
```

O tipo de parâmetro `table` de `show` mudou de `Table` para `&Table`: em vez de passar a tabela por valor (e, portanto, mover a posse para a função), agora estamos passando uma referência compartilhada. Essa é a única mudança textual. Mas como isso funciona enquanto trabalhamos o corpo?

Considerando que nosso loop `for` externo original tomou posse do `HashMap` e o consumiu, em nossa nova versão ele recebe uma referência compartilhada ao `HashMap`. Iterar por uma referência compartilhada para um `HashMap` é definido para produzir referências compartilhadas à chave e ao valor de cada entrada: `artist` mudou de uma `String` para uma `&String`, e `works` de uma `Vec<String>` para uma `&Vec<String>`.

O loop interno é alterado de forma semelhante. A iteração por uma referência compartilhada para um vetor é definida para produzir referências compartilhadas para seus elementos, então `work` agora é um `&String`. Nenhuma posse muda de mãos em qualquer lugar nesta função; trata-se apenas de passar referências não proprietárias.

Agora, se quisermos escrever uma função para colocar em ordem alfabética as obras de cada artista, uma referência compartilhada não é suficiente, pois as referências compartilhadas não permitem modificações. Em vez disso, a função de ordenação precisa ter uma referência mutável à tabela:


```
fn sort_works(table: &mut Table) {  
    for (_artist, works) in table {  
        works.sort();  
    }  
}
```

E precisamos passar-lhe uma:

```
sort_works(&mut table);
```

Esse empréstimo mutável concede a `sort_works` a capacidade de ler e modificar nossa estrutura, conforme exigido pelo método `sort` dos vetores.

Quando passamos um valor para uma função de uma maneira que transfira a posse do valor para a função, dizemos que o passamos *por valor*. Se, em vez disso, passarmos à função uma referência ao valor, dizemos que passamos o valor *por referência*. Por exemplo, corrigimos nossa função `show` alterando-a para aceitar a tabela por referência, em vez de por valor. Muitas linguagens fazem essa distinção, mas isso é especialmente importante no Rust, porque explica como a posse é afetada.

Trabalhando com referências

O exemplo anterior mostra um uso bastante típico para referências: permitir que funções acessem ou manipulem uma estrutura sem tomar posse dela. Mas as referências são mais flexíveis do que isso, então vamos ver alguns exemplos para obter uma visão mais detalhada do que está acontecendo.

Referências Rust versus referências C++

Se você estiver familiarizado com as referências em C++, saberá que elas têm algo em comum com as referências do Rust. Mais importante, ambas são apenas endereços no nível de máquina. Mas, na prática, as referências do Rust têm uma aparência muito diferente.

Em C++, as referências são criadas implicitamente por conversão e também implicitamente desreferenciadas:

```
// Código C++!
```

```
int x = 10;
int &r = x;          // a inicialização cria a referência implicitamente
assert(r == 10);     // desreferencia r implicitamente para ver o valor de x
r = 20;              // armazena 20 em x, o r em si ainda aponta para x
```

No Rust, as referências são criadas explicitamente com o operador `&` e desreferenciadas explicitamente com o operador `*`:

```
// De volta ao código Rust deste ponto em diante
let x = 10;
let r = &x;          // &x é uma referência compartilhada para x
assert!(*r == 10);   // desreferencia r explicitamente
```

Para criar uma referência mutável, utilize o operador `&mut`:

```
let mut y = 32;
let m = &mut y;      // &mut y é uma referência mutável para y
*m += 32;            // desreferencia m explicitamente para definir
assert!(*m == 64);   // o valor de y e para ver o novo valor de y
```

Mas você deve se lembrar de que, quando corrigimos a função `show` de pegar a tabela de artistas por referência e não por valor, nunca tivemos de utilizar o operador `*`. Qual é a razão disso?

Como as referências são tão amplamente utilizadas no Rust, o operador `.` (ponto) desreferencia implicitamente seu operando esquerdo, se necessário:

```
struct Anime { name: &'static str, bechdel_pass: bool }
let aria = Anime { name: "Aria: The Animation", bechdel_pass: true };
let anime_ref = &aria;
assert_eq!(anime_ref.name, "Aria: The Animation");

// Equivalente ao acima, mas com a desreferência explícita:
assert_eq!((*anime_ref).name, "Aria: The Animation");
```

A macro `println!` utilizada na função `show` se expande para o código que utiliza o operador `.`; portanto, também aproveita essa desreferência implícita.

O operador `.` também pode emprestar implicitamente uma referência ao seu operando esquerdo, se for necessário para uma chamada de método. Por exemplo, o método `sort` de `Vec` utiliza uma referência mutável para o vetor, então essas duas chamadas são equivalentes:

```
let mut v = vec![1973, 1968];
v.sort();          // empresta implicitamente uma referência mutável
(&mut v).sort();   // para v equivalente, mas mais prolixo
```

Resumindo, enquanto C++ converte implicitamente entre

referências e lvalues (ou seja, expressões que referenciam posições na memória), com essas conversões aparecendo em qualquer lugar em que sejam necessárias, no Rust você utiliza os operadores `&` e `*` para criar e seguir referências, com exceção do operador `.`, que empresta e desreferencia implicitamente.

Atribuição de referências

Atribuir uma referência a uma variável faz com que essa variável aponte para algum lugar novo:

```
let x = 10;  
let y = 20;  
let mut r = &x;  
  
if b { r = &y; }  
  
assert!(*r == 10 || *r == 20);
```

A referência `r` inicialmente aponta para `x`. Mas, se `b` é verdade, o código aponta para `y` em vez disso, conforme ilustrado na Figura 5.1.

Esse comportamento pode parecer óbvio demais para valer a pena mencionar: obviamente `r` agora aponta para `y`, já que armazenamos `&y` nela. Mas ressaltamos isso porque as referências em C++ se comportam de maneira muito diferente: como mostrado anteriormente, atribuir um valor a uma referência em C++ armazena o valor em seu referente. Uma vez que uma referência em C++ foi inicializada, não há como fazê-la apontar para qualquer outra coisa.

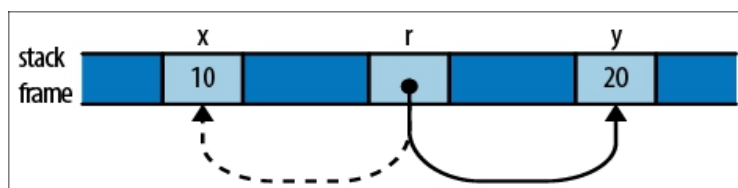


Figura 5.1: A referência `r`, agora apontando para `y` em vez de `x`.

Referências a referências

O Rust permite referências a referências:

```
struct Point { x: i32, y: i32 }  
let point = Point { x: 1000, y: 729 };
```

```
let r: &Point = &point;  
let rr: &&Point = &r;  
let rrr: &&&Point = &rr;
```

(Escrevemos os tipos de referência para maior clareza, mas você pode omiti-los; não há nada aqui que o Rust não possa inferir.) O operador `.` (ponto) segue quantas referências forem necessárias para encontrar seu destino:

```
assert_eq!(rrr.y, 729);
```

Na memória, as referências são dispostas conforme a Figura 5.2.

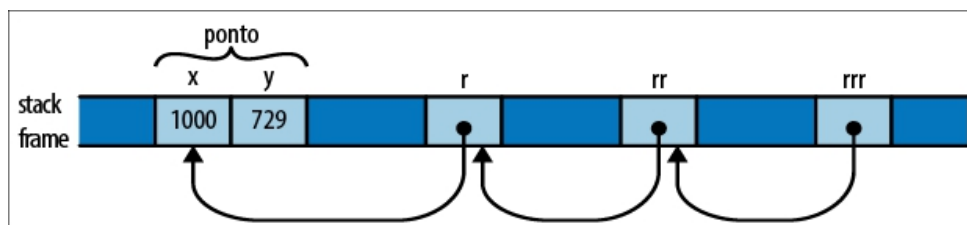


Figura 5.2: Uma cadeia de referências para referências.

Então, a expressão `rrr.y`, orientada pelo tipo de `rrr`, na verdade percorre três referências para chegar ao `Point` antes de buscar seu campo `y`.

Comparando referências

Como o operador `.` (ponto), os operadores de comparação do Rust “veem através” de qualquer número de referências:

```
let x = 10;  
let y = 10;  
  
let rx = &x;  
let ry = &y;  
  
let rrx = &rx;  
let rry = &ry;  
  
assert!(rrx <= rry);  
assert!(rrx == rry);
```

A asserção final aqui é bem-sucedida, embora `rrx` e `rry` apontem para valores diferentes (ou seja, `rx` e `ry`), porque o operador `==` segue todas as referências e realiza a comparação em seus alvos finais, `x` e `y`. Esse é quase sempre o comportamento que você deseja, especialmente ao escrever funções genéricas. Se você realmente

deseja saber se duas referências apontam para a mesma memória, pode utilizar `std::ptr::eq`, que os compara como endereços:

```
assert!(rx == ry);           // seus referentes são iguais
assert!(!std::ptr::eq(rx, ry)); // mas ocupam endereços diferentes
```

Observe que os operandos de uma comparação devem ser exatamente do mesmo tipo, incluindo as referências:

```
assert!(rx == rrx); // erro: tipo incompatível: `&i32` vs `&&i32`
assert!(rx == *rrx); // isso está certo
```

Referências nunca são nulas

Referências do Rust nunca são nulas. Não há análogo ao `NULL` do C ou ao `nullptr` do C++. Não há valor inicial padrão para uma referência (você não pode utilizar nenhuma variável até que ela seja inicializada, independentemente de seu tipo) e o Rust não converterá números inteiros em referências (fora de código `unsafe`), então você não pode converter zero em uma referência.

Os códigos em C e C++ geralmente utilizam um ponteiro nulo para indicar a ausência de um valor: por exemplo, a função `malloc` retorna um ponteiro para um novo bloco de memória ou `nullptr` se não houver memória suficiente disponível para atender à solicitação. No Rust, se precisamos de um valor que seja uma referência a algo ou não, utilizamos o tipo `Option<&T>`. No nível de máquina, o Rust representa `None` como um ponteiro nulo e `Some(r)`, em que `r` é um valor `&T`, como o endereço diferente de zero, então `Option<&T>` é tão eficiente quanto um ponteiro nulo em C ou C++ embora seja mais seguro: seu tipo exige que você verifique se é `None` antes de usá-lo.

Emprestando referências a expressões arbitrárias

Considerando que C e C++ apenas permitem que você aplique o operador `&` a certos tipos de expressões, o Rust permite que você empreste uma referência ao valor de qualquer tipo de expressão:

```
fn factorial(n: usize) -> usize {
    (1..n+1).product()
}
```

```
let r = &factorial(6);  
// Os operadores aritméticos podem ver através de um nível de referências  
assert_eq!(r + &1009, 1729);
```

Em situações como essa, o Rust simplesmente cria uma variável anônima para manter o valor da expressão e faz a referência apontar para essa. O tempo de vida dessa variável anônima depende do que você faz com a referência:

- Se você atribuir imediatamente a referência a uma variável em uma declaração `let` (ou torná-la parte de algum struct ou array que está sendo imediatamente atribuído), o Rust, então, torna a variável anônima ativa enquanto a variável `let` inicializa. No exemplo anterior, o Rust faria isso para o referente de `r`.
- Caso contrário, a variável anônima existirá até o final da instrução anexa. Em nosso exemplo, a variável anônima criada para armazenar 1009 dura apenas até o final da declaração `assert_eq!`.

Se você está acostumado com C ou C++, isso pode parecer propenso a erros. Mas lembre-se de que o Rust nunca permitirá que você escreva um código que produza uma referência perdida. Se a referência puder ser utilizada além do tempo de vida da variável anônima, o Rust sempre relatará o problema para você em tempo de compilação. Você pode corrigir seu código para manter o referente em uma variável nomeada com um tempo de vida apropriado.

Referências a fatias e objetos trait

As referências que mostramos até agora são todas endereços simples. No entanto, o Rust também inclui dois tipos de *ponteiros gordos* (*fat pointers*), valores de duas palavras contendo o endereço de algum valor, com algumas informações adicionais necessárias para colocar o valor em uso.

Uma referência a uma fatia é um ponteiro gordo, carregando o endereço inicial da fatia e seu comprimento. Descrevemos fatias em detalhes no Capítulo 3.

O outro tipo de ponteiro gordo do Rust é um *objeto trait*, uma referência a um valor que implementa um determinado trait. Um objeto trait carrega um endereço de valor e um ponteiro para a

implementação do trait apropriado para esse valor, para invocar os métodos do trait. Abordaremos objetos trait em detalhes em “Objetos trait”, na página 301.

Além de carregar esses dados extras, as referências a objetos trait e fatias se comportam exatamente como os outros tipos de referências que mostramos até agora neste capítulo: elas não possuem seus referentes, não podem sobreviver a seus referentes, podem ser mutáveis ou compartilhadas e assim por diante.

Segurança de referência

Como as apresentamos até agora, as referências se parecem muito com ponteiros comuns em C ou C++. Mas estes não são seguros; como o Rust mantém suas referências sob controle? Talvez a melhor maneira de ver as regras em ação seja tentando quebrá-las.

Para explicar as ideias fundamentais, começaremos com os casos mais simples, mostrando como o Rust garante que as referências sejam utilizadas adequadamente em um único corpo de função. Em seguida, veremos como passar referências entre funções e armazená-las em estruturas de dados. Isso implica dar as ditas funções e tipos de dados *parâmetros de tempo de vida*, que vamos explicar. Por fim, apresentaremos alguns atalhos que o Rust fornece para simplificar os padrões de uso comuns. Ao longo do caminho, mostraremos como o Rust aponta códigos errados e geralmente sugere soluções.

Emprestando uma variável local

Eis um caso bastante óbvio. Você não pode emprestar uma referência a uma variável local e retirá-la do escopo da variável:

```
{
    let r;
    {
        let x = 1;
        r = &x;
    }
    assert_eq!(*r, 1); // ruim: lê a memória que `x` ocupava
}
```

O compilador Rust rejeita esse programa, com uma mensagem de erro detalhada:

```
error: `x` does not live long enough
  |
7 |     r = &x;
  |         ^^ borrowed value does not live long enough
8 | }
  | - `x` dropped here while still borrowed
9 | assert_eq!(*r, 1); // ruim: lê a memória que `x` ocupava
10| }
```

A reclamação do Rust é que `x` existe apenas até o final do bloco interno, enquanto a referência permanece ativa até o final do bloco externo, tornando-se um ponteiro perdido, o que é proibido.

Embora seja óbvio para um leitor humano que esse programa está quebrado, vale a pena ver como o próprio Rust chegou a essa conclusão. Mesmo esse exemplo simples mostra as ferramentas lógicas que o Rust utiliza para verificar códigos muito mais complexos.

O Rust tenta atribuir a cada tipo de referência em seu programa um *tempo de vida* que atenda às restrições impostas pela forma como é utilizado. Um tempo de vida é algum trecho do seu programa durante o qual pode ser seguro utilizar uma referência: uma instrução, uma expressão, o escopo de alguma variável ou algo semelhante. Tempos de vida são inteiramente determinados em tempo de compilação do Rust. Em tempo de execução, uma referência nada mais é do que um endereço; seu tempo de vida faz parte de seu tipo e não tem representação em tempo de execução.

Neste exemplo, há três tempos de vida cujas relações precisamos resolver. Tanto a variável `r` como a variável `x` têm um tempo de vida, estendendo-se desde o ponto em que são inicializadas até o ponto em que o compilador pode provar que não estão mais em uso. O terceiro tempo de vida é o de um tipo de referência: o tipo de referência que emprestamos a `x` e armazenamos em `r`.

Eis uma restrição que deve parecer bastante óbvia: se você tiver uma variável `x`, então uma referência a `x` não deve sobreviver ao próprio `x`, como mostra a Figura 5.3.

Além do ponto onde `x` sai do escopo, a referência seria um ponteiro perdido. Dizemos que o tempo de vida da variável deve *conter* ou *incluir* o da referência emprestada dele.

Eis outro tipo de restrição: se você armazenar uma referência em uma variável `r`, o tipo da referência deve ser válido durante todo o tempo de vida da variável, desde sua inicialização até seu último uso, conforme mostra a Figura 5.4.

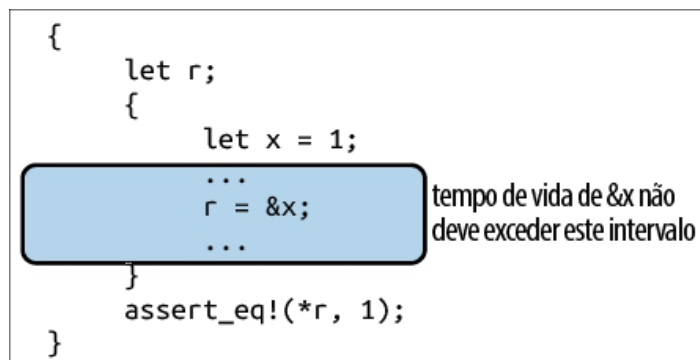


Figura 5.3: Tempos de vida permitidos para caracter em estilo -> &.

Se a referência não puder existir pelo menos tanto quanto a variável, então em algum ponto `r` será um ponteiro perdido. Dizemos que o tempo de vida da referência deve conter ou incluir o da variável.

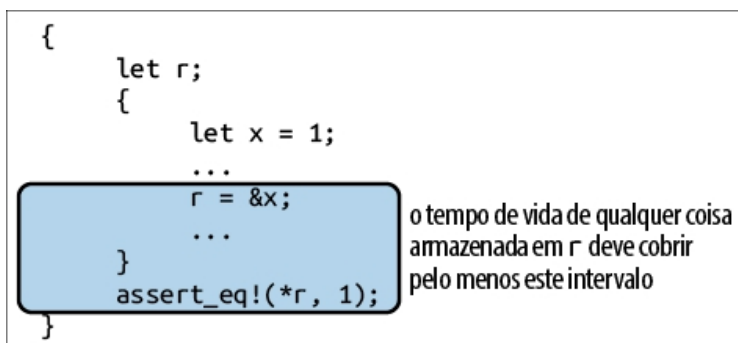


Figura 5.4: Tempos de vida permitidos para referência armazenados em r.

O primeiro tipo de restrição limita o quão grande pode ser o tempo de vida de uma referência, enquanto o segundo tipo limita o quão pequeno ele pode ser. O Rust simplesmente tenta encontrar um tempo de vida para cada referência que satisfaça todas essas

restrições. Em nosso exemplo, porém, não existe esse tempo de vida, conforme mostrado na Figura 5.5.

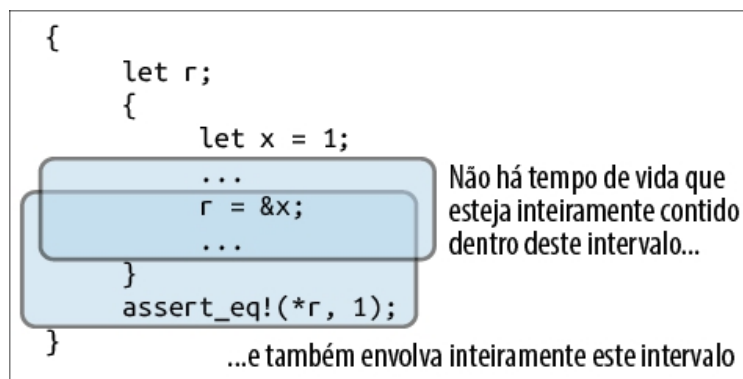


Figura 5.5: Uma referência com restrições contraditórias em seu tempo de vida.

Vamos agora considerar um exemplo diferente em que as coisas funcionam. Temos os mesmos tipos de restrição: o tempo de vida da referência deve estar contido no de `x`, mas totalmente envolver o de `r`. Mas como o tempo de vida de `r` é menor agora, há um tempo de vida que atende às restrições, conforme mostrado na Figura 5.6.

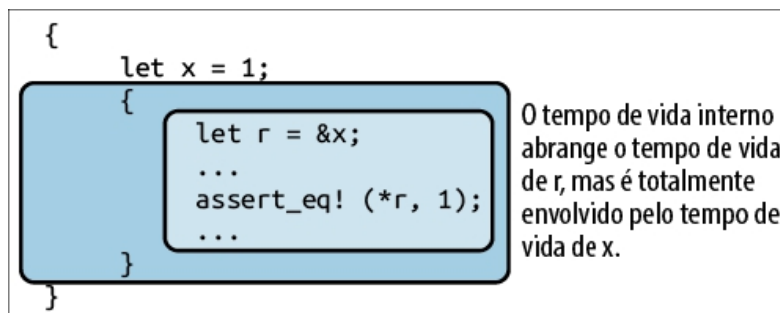


Figura 5.6: Uma referência com um período de tempo de vida envolvendo o escopo de `r`, mas dentro do escopo de `x`.

Essas regras se aplicam de maneira natural quando você empresta uma referência a alguma parte de alguma estrutura de dados maior, como um elemento de um vetor:

```
let v = vec![1, 2, 3];  
let r = &v[1];
```

Como `v` possui o vetor, que possui seus elementos, o tempo de vida de `v` deve incluir o do tipo de referência de `&v[1]`. Da mesma forma,

se você armazenar uma referência em alguma estrutura de dados, seu tempo de vida deve incluir o da estrutura de dados. Por exemplo, se você construir um vetor de referências, todas elas devem ter tempos de vida abrangendo o da variável que possui o vetor.

Essa é a essência do processo que o Rust utiliza para todo o código. Trazer mais recursos de linguagem para o quadro geral – por exemplo, estruturas de dados e chamadas de função – introduz novos tipos de restrições, mas o princípio permanece o mesmo: primeiro, entenda as restrições decorrentes da maneira como o programa utiliza as referências; então, encontre tempos de vida que os satisfaçam. Isso não é tão diferente do processo que os programadores de C e C++ impõem a si mesmos; a diferença é que o Rust conhece as regras e as aplica.

Recebendo referências como argumentos de função

Quando passamos uma referência a uma função, como o Rust garante que a função a utilize com segurança? Suponha que temos uma função `f` que pega uma referência e a armazena em uma variável global. Precisaremos fazer algumas revisões, mas aqui está um primeiro corte:

```
// Este código tem vários problemas e não compila
static mut STASH: &i32;
fn f(p: &i32) { STASH = p; }
```

O equivalente do Rust a uma variável global é chamado de variável estática, ou `static`: é um valor criado quando o programa inicia e dura até este terminar. (Como qualquer outra declaração, o sistema de módulos do Rust controla onde as estáticas são visíveis, então elas são apenas “globais” em seu tempo de vida, não em sua visibilidade.) Abordamos variáveis estáticas no Capítulo 8, mas por enquanto vamos apenas citar algumas regras que o código mostrado não segue:

- Cada variável estática deve ser inicializada.
- As variáveis estáticas mutáveis não são inerentemente seguras

para threads (afinal, qualquer thread pode acessar uma variável estática a qualquer momento) e, mesmo em programas de thread única, elas podem ser vítimas de outros tipos de problemas de reentrada. Por esses motivos, você pode acessar uma variável estática mutável apenas dentro de um bloco `unsafe`. Neste exemplo, não estamos preocupados com esses problemas específicos, então vamos apenas usar um bloco `unsafe` e seguir em frente.

Com essas revisões feitas, agora temos o seguinte:

```
static mut STASH: &i32 = &128;
fn f(p: &i32) { // ainda não é bom o suficiente
    unsafe {
        STASH = p;
    }
}
```

Estamos quase terminando. Para ver o problema restante, precisamos escrever algumas coisas que o Rust está gentilmente nos deixando omitir. A assinatura de `f` conforme escrito aqui é, na verdade, uma abreviação para o seguinte:

```
fn f<'a>(p: &'a i32) { ... }
```

Aqui, o tempo de vida `'a` (pronuncia-se “tique A”) é um *parâmetro de tempo de vida* de `f`. Você pode ler `<'a>` como “para qualquer tempo de vida `'a`”, então quando escrevemos `fn f<'a>(p: &'a i32)`, estamos definindo uma função que recebe uma referência a um `i32` com qualquer tempo de vida `'a`.

Já que devemos permitir `'a` ser qualquer tempo de vida, é melhor que as coisas funcionem no menor tempo de vida possível: um apenas envolvendo a chamada a `f`. Essa atribuição torna-se então um ponto de contenção:

```
STASH = p;
```

Como `STASH` existe durante toda a execução do programa, o tipo de referência que contém deve ter um tempo de vida da mesma extensão; o Rust chama isso de *static lifetime* (*tempo de vida estático*). Mas o tempo de vida da referência de `p` é algum `'a`, que pode ser qualquer coisa, desde que envolva a chamada a `f`. Então, o Rust rejeita nosso código:

```
error: explicit lifetime required in the type of `p`
```

```

5 | STASH = p;
   | ^ lifetime `static` required

```

Neste ponto, está claro que nossa função não pode aceitar qualquer referência como argumento. Mas, como aponta o Rust, a função deve ser capaz de aceitar uma referência que tenha um tempo de vida 'static: armazenar uma referência assim em STASH não pode criar um ponteiro perdido. E, de fato, o código a seguir compila perfeitamente:

```

static mut STASH: &i32 = &10;

fn f(p: &'static i32) {
    unsafe {
        STASH = p;
    }
}

```

Desta vez, a assinatura de `f` diz que `p` deve ser uma referência com tempo de vida 'static, então não há mais nenhum problema em armazená-la em STASH. Só podemos aplicar `f` a referências a outras variáveis estáticas, mas essa é a única coisa que certamente não deixará STASH pendente de nenhuma maneira. Então podemos escrever:

```

static WORTH_POINTING_AT: i32 = 1000;
f(&WORTH_POINTING_AT);

```

Como a `WORTH_POINTING_AT` é uma variável estática, o tipo de `&WORTH_POINTING_AT` é `&'static i32`, que é seguro passar para `f`.

Porém, dê um passo para trás e observe o que aconteceu com a assinatura de `f` quando corrigimos nosso código corretamente: o original `f(p: &i32)` acabou como `f(p: &'static i32)`. Em outras palavras, não conseguimos escrever uma função que escondesse uma referência em uma variável global sem refletir essa intenção na assinatura da função. No Rust, a assinatura de uma função sempre expõe o comportamento do corpo.

Por outro lado, se virmos uma função com uma assinatura como `g(p: &i32)` (ou com tempos de vida explicitamente escritos, `g<'a>(p: &'a i32)`), podemos dizer que ela não esconde seu argumento `p` em qualquer lugar que sobreviva à chamada. Não há necessidade de olhar para a

definição de `g`; a assinatura por si só nos diz o que `g` pode e não pode fazer com seu argumento. Esse fato acaba sendo muito útil quando você está tentando estabelecer a segurança de uma chamada à função.

Passando referências para funções

Agora que mostramos como a assinatura de uma função se relaciona com seu corpo, vamos examinar como ela se relaciona com os chamadores da função. Suponha que você tenha o seguinte código:

```
// Isso poderia ser escrito mais sucintamente: fn g(p: &i32),  
// mas vamos escrever os tempos de vida por enquanto.  
fn g<'a>(p: &'a i32) { ... }
```

```
let x = 10;  
g(&x);
```

A partir apenas da assinatura de `g`, o Rust sabe que não vai salvar `p` em qualquer lugar que possa sobreviver à chamada: qualquer tempo de vida que inclua a chamada deve funcionar para `'a`. Então, o Rust escolhe o menor tempo de vida possível para `&x`: o da chamada a `g`. Isso atende a todas as restrições: não sobrevive a `x` e inclui toda a chamada para a função `g`. Portanto, esse código passou no teste.

Note que, embora `g` aceite um parâmetro de tempo de vida `'a`, não precisamos mencioná-lo ao chamar `g`. Você só precisa se preocupar com parâmetros de tempo de vida ao definir funções e tipos; ao usá-los, o Rust infere os tempos de vida para você.

E se tentássemos passar `&x` para nossa função `f` antes de ela armazenar seu argumento em uma variável estática?

```
fn f(p: &'static i32) { ... }
```

```
let x = 10;  
f(&x);
```

Isso falha ao compilar: a referência `&x` não deve sobreviver a `x`, mas passando-a para `f` nós a forçamos a existir pelo menos tanto quanto `'static`. Não há como satisfazer a todos aqui, então o Rust rejeita o código.

Retornando referências

É comum que uma função pegue uma referência a alguma estrutura de dados e então retorne uma referência para alguma parte dessa estrutura. Por exemplo, eis uma função que retorna uma referência ao menor elemento de uma fatia:

```
// v deve ter pelo menos um elemento
fn smallest(v: &[i32]) -> &i32 {
    let mut s = &v[0];
    for r in &v[1..] {
        if *r < *s { s = r; }
    }
    s
}
```

Omitimos os tempos de vida da assinatura dessa função da maneira usual. Quando uma função recebe uma única referência como argumento e retorna uma única referência, o Rust assume que as duas devem ter o mesmo tempo de vida. Escrever isso explicitamente nos daria:

```
fn smallest<'a>(v: &'a [i32]) -> &'a i32 { ... }
```

Suponha que chamemos `smallest` assim:

```
let s;
{
    let parabola = [9, 4, 1, 0, 1, 4, 9];
    s = smallest(&parabola);
}
assert_eq!(*s, 0); // ruim: aponta para o elemento do array dropado
```

A partir da assinatura de `smallest`, podemos ver que seu argumento e valor de retorno devem ter o mesmo tempo de vida, `'a`. Em nossa chamada, o argumento `¶bola` não deve sobreviver à própria `parabola`, ainda que o valor de retorno de `smallest` deva durar pelo menos tanto quanto `s`. Não há tempo de vida possível `'a` que possa satisfazer ambas as restrições, então o Rust rejeita o código:

```
error: `parabola` does not live long enough
|
11 |     s = smallest(&parabola);
|                   ----- borrow occurs here
12 | }
|   ^ `parabola` dropped here while still borrowed
13 | assert_eq!(*s, 0); // ruim: aponta para o elemento do array dropado
```

```
| - borrowed value needs to live until here
14 | }
```

Mover `s` de modo que seu tempo de vida esteja claramente contido dentro de `parabola` corrige o problema:

```
{
    let parabola = [9, 4, 1, 0, 1, 4, 9];
    let s = smallest(&parabola);
    assert_eq!(*s, 0); // ótimo: parabola ainda está viva
}
```

Os tempos de vida em assinaturas de função permitem que o Rust avalie as relações entre as referências que você passa para a função e aquelas que a função retorna, e eles garantem que estão sendo utilizadas com segurança.

Structs contendo referências

Como o Rust lida com referências armazenadas em estruturas de dados? Eis o mesmo programa errado que vimos anteriormente, exceto que colocamos a referência dentro de uma estrutura:

```
// Isso não compila
struct S {
    r: &i32
}

let s;
{
    let x = 10;
    s = S { r: &x };
}

assert_eq!(*s.r, 10); // ruim: lê de `x` dropado
```

As restrições de segurança que o Rust coloca nas referências não podem desaparecer magicamente só porque escondemos a referência dentro de um struct. De alguma forma, essas restrições devem acabar se aplicando a `s` também. De fato, o Rust é cético:

```
error: missing lifetime specifier
|
7 |     r: &i32
|       ^ expected lifetime parameter
```

Sempre que um tipo de referência aparecer dentro da definição de outro tipo, você deve escrever explicitamente seu tempo de vida.

Você pode escrever isto:

```
struct S {  
    r: &'static i32  
}
```

Isso diz que `r` só pode referenciar valores `i32` que durarão pelo tempo de vida do programa, o que é bastante limitador. A alternativa é dar ao tipo um parâmetro de tempo de vida `'a` e utilizar isso para `r`:

```
struct S<'a> {  
    r: &'a i32  
}
```

Agora o tipo `s` tem um tempo de vida, assim como os tipos de referência. Cada valor do tipo `s` que você cria ganha um novo tempo de vida `'a`, que se torna limitado pela maneira como você utiliza o valor. O tempo de vida de qualquer referência que você armazena em `r` deve incluir `'a`, e `'a` deve durar mais do que o tempo de vida de onde quer que você armazene o `s`.

Voltando ao código anterior, a expressão `s { r: &x }` cria um novo valor `s` com algum tempo de vida `'a`. Quando você armazena `&x` no campo `r`, restringe `'a` a estar inteiramente dentro do tempo de vida de `x`.

A atribuição `s = s { ... }` armazena esse `s` em uma variável cujo tempo de vida se estende até o final do exemplo, restringindo `'a` a superar o tempo de vida de `s`. E agora o Rust chegou às mesmas restrições contraditórias de antes: `'a` não deve sobreviver a `x`, mas deve existir pelo menos tanto quanto `s`. Não existe tempo de vida satisfatório e o Rust rejeita o código. Desastre evitado!

Como um tipo com um parâmetro de tempo de vida se comporta quando colocado dentro de algum outro tipo?

```
struct D {  
    s: S // não adequado  
}
```

O Rust é cético, assim como quando tentamos colocar uma referência em `s` sem especificar seu tempo de vida:

```
error: missing lifetime specifier  
|  
8 |     s: S // não adequado  
|       ^ expected named lifetime parameter  
|
```

Não podemos deixar o parâmetro de tempo de vida de `s` aqui: o Rust precisa saber como o tempo de vida de `D` relaciona-se com o da referência em seu `s` a fim de aplicar as mesmas verificações em `D` que faz em `s` e referências simples.

Nós poderíamos dar a `s` o tempo de vida `'static`. Isto funciona:

```
struct D {  
    s: S<'static>  
}
```

Com essa definição, o campo `s` só pode emprestar valores que vivem durante toda a execução do programa. Isso é um pouco restritivo, mas significa que `D` possivelmente não pode emprestar uma variável local; não há restrições especiais ao tempo de vida de `D`.

A mensagem de erro do Rust na verdade sugere outra abordagem, que é mais geral:

```
help: consider introducing a named lifetime parameter  
|  
7 | struct D<'a> {  
8 |     s: S<'a>  
|
```

Aqui, damos a `D` seu próprio parâmetro de tempo de vida e passamos isso para `s`:

```
struct D<'a> {  
    s: S<'a>  
}
```

Tomando um parâmetro de tempo de vida `'a` e utilizando-o no tipo de `s`, permitimos que o Rust relacionasse o tempo de vida do valor de `D` àquele da referência que `s` armazena.

Mostramos anteriormente como a assinatura de uma função expõe o que ela faz com as referências que passamos. Agora, mostramos algo semelhante sobre tipos: os parâmetros de tempo de vida de um tipo sempre revelam se ele contém referências com tempos de vida interessantes (ou seja, não-`'static`) e quais podem ser esses tempos de vida.

Por exemplo, suponha que temos uma função de análise que pega uma fatia de bytes e retorna uma estrutura contendo os resultados da análise:

```
fn parse_record<'i>(input: &'i [u8]) -> Record<'i> { ... }
```

Sem olhar para a definição do tipo `Record`, podemos dizer que, se recebermos um `Record` a partir de `parse_record`, quaisquer referências que ele contenha devem apontar para o buffer de entrada que passamos e para nenhum outro lugar (exceto talvez em valores `'static`).

Na verdade, essa exposição do comportamento interno é a razão pela qual o Rust exige que tipos que contenham referências tenham parâmetros de tempo de vida explícitos. Não há razão para o Rust simplesmente não criar um tempo de vida distinto para cada referência no struct e poupar o trabalho de escrevê-los. As primeiras versões do Rust realmente se comportavam dessa maneira, mas os desenvolvedores acharam confuso: é útil saber quando um valor empresta algo de outro valor, especialmente ao trabalhar com erros.

Não são apenas referências e tipos como `s` que têm tempos de vida. Cada tipo no Rust tem um tempo de vida, incluindo `i32` e `String`. A maioria é simplesmente `'static`, o que significa que os valores desses tipos podem durar o quanto você quiser; por exemplo, um `Vec<i32>` é independente e não precisa ser dropado antes que qualquer variável específica saia do escopo. Mas um tipo como `Vec<&'a i32>` tem um tempo de vida que deve ser delimitado por `'a`: deve ser dropado enquanto seus referentes ainda estiverem vivos.

Parâmetros de tempo de vida distintos

Suponha que você tenha definido uma estrutura contendo duas referências como esta:

```
struct S<'a> {  
    x: &'a i32,  
    y: &'a i32  
}
```

Ambas as referências utilizam o mesmo tempo de vida `'a`. Isso pode ser um problema se o seu código quiser fazer algo assim:

```
let x = 10;  
let r;  
{  
    let y = 20;
```

```

    {
        let s = S { x: &x, y: &y };
        r = s.x;
    }
}
println!("{}", r);

```

Esse código não cria ponteiros perdidos. A referência a *y* fica em *s*, que sai do escopo antes de *y*. A referência a *x* acaba em *r*, que não sobrevive a *x*.

Se você tentar compilar isso, porém, o Rust reclamará que *y* não existe por tempo suficiente, embora claramente exista. Por que o Rust está preocupado? Se você trabalhar com o código com cuidado, poderá seguir seu raciocínio:

- Ambos os campos de *s* são referências com o mesmo tempo de vida 'a, então o Rust deve encontrar um único tempo de vida que funcione tanto para *s.x* como para *s.y*.
- Atribuímos *r = s.x*, exigindo que 'a inclua o tempo de vida de *r*.
- Inicializamos *s.y* com *&y*, exigindo que 'a não tenha mais que o tempo de vida de *y*.

Essas restrições são impossíveis de satisfazer: nenhum tempo de vida é menor que o escopo de *y*, mas maior que o de *r*. O Rust empaca.

O problema surge porque ambas as referências em *s* têm o mesmo tempo de vida 'a. Mudar a definição de *s* para deixar cada referência ter um tempo de vida distinto corrige tudo:

```

struct S<'a, 'b> {
    x: &'a i32,
    y: &'b i32
}

```

Com essa definição, *s.x* e *s.y* têm tempos de vida independentes. O que fazemos com *s.x* não tem efeito sobre o que armazenamos em *s.y*, então é fácil satisfazer as restrições agora: 'a pode simplesmente ter o tempo de vida de *r*, e 'b pode ter o de *s*. (O tempo de vida de *y* também funcionaria para 'b, mas o Rust tenta escolher o menor tempo de vida que funciona.) Tudo acaba bem.

As assinaturas de função podem ter efeitos semelhantes. Suponha

que temos uma função como esta:

```
fn f<'a>(r: &'a i32, s: &'a i32) -> &'a i32 { r } // talvez muito restrito
```

Aqui, ambos os parâmetros de referência utilizam o mesmo tempo de vida 'a, que pode restringir desnecessariamente o chamador da mesma forma que mostramos anteriormente. Se isso for um problema, você pode deixar o tempo de vida dos parâmetros variar independentemente:

```
fn f<'a, 'b>(r: &'a i32, s: &'b i32) -> &'a i32 { r } // mais livre
```

A desvantagem disso é que adicionar tempos de vida pode dificultar a leitura de tipos e assinaturas de função. Seus autores tendem a tentar primeiro a definição mais simples possível e depois afrouxam as restrições até que o código seja compilado. Como o Rust não permitirá que o código seja executado a menos que seja seguro, simplesmente esperar para ser avisado quando houver um problema é uma tática perfeitamente aceitável.

Omitindo parâmetros de tempo de vida

Até agora, mostramos várias funções neste livro que retornam referências ou as tomam como parâmetros, mas geralmente não precisamos especificar qual tempo de vida é qual. Os tempos de vida estão lá; o Rust está apenas nos deixando omiti-los quando é razoavelmente óbvio o que eles deveriam ser.

Nos casos mais simples, talvez você nunca precise escrever tempos de vida para seus parâmetros. O Rust apenas atribui um tempo de vida distinto a cada ponto que precisa de um. Por exemplo:

```
struct S<'a, 'b> {  
    x: &'a i32,  
    y: &'b i32  
}  
  
fn sum_r_xy(r: &i32, s: S) -> i32 {  
    r + s.x + s.y  
}
```

A assinatura dessa função é uma abreviação para:

```
fn sum_r_xy<'a, 'b, 'c>(r: &'a i32, s: S<'b, 'c>) -> i32
```

Se você retornar referências ou outros tipos com parâmetros de

tempo de vida, o Rust ainda tentará facilitar os casos inequívocos. Se houver apenas um único tempo de vida que aparece entre os parâmetros de sua função, o Rust assume que qualquer tempo de vida em seu valor de retorno deve ser aquele:

```
fn first_third(point: &[i32; 3]) -> (&i32, &i32) {  
    (&point[0], &point[2])  
}
```

Com todos os tempos de vida escritos explicitamente, o equivalente seria:

```
fn first_third<'a>(point: &'a [i32; 3]) -> (&'a i32, &'a i32)
```

Se houver vários tempos de vida entre seus parâmetros, não há razão natural para preferir um a outro para o valor de retorno, e o Rust faz com que você explique o que está acontecendo.

Se sua função é um método em algum tipo e recebe seu parâmetro `self` por referência, isso desfaz o deadlock (impasse): o Rust assume que o tempo de vida de `self` é aquele ao qual ele deve dar tudo em seu valor de retorno. (Um parâmetro `self` referencia o valor sobre o qual o método está sendo chamado no equivalente do Rust de `this` em C++, Java ou JavaScript, ou `self` no Python. Abordaremos os métodos em “Definindo métodos com `impl`”, na página 252.)

Por exemplo, você pode escrever o seguinte:

```
struct StringTable {  
    elements: Vec<String>,  
}  
  
impl StringTable {  
    fn find_by_prefix(&self, prefix: &str) -> Option<&String> {  
        for i in 0 .. self.elements.len() {  
            if self.elements[i].starts_with(prefix) {  
                return Some(&self.elements[i]);  
            }  
        }  
        None  
    }  
}
```

A assinatura do método do `find_by_prefix` é uma abreviação para:

```
fn find_by_prefix<'a, 'b>(&'a self, prefix: &'b str) -> Option<&'a String>
```

O Rust assume que qualquer coisa que você esteja pegando emprestado, você está pegando de `self`.

Novamente, essas são apenas abreviações destinadas a serem úteis sem introduzir surpresas. Quando eles não são o que você deseja, você sempre pode escrever os tempos de vida explicitamente.

Compartilhamento versus mutação

Até agora, discutimos como o Rust garante que nenhuma referência aponte para uma variável que saiu do escopo. Mas há outras maneiras de introduzir ponteiros perdidos. Eis um caso fácil:

```
let v = vec![4, 8, 19, 27, 34, 10];
let r = &v;
let aside = v; // mover o vetor para o lado
r[0];          // ruim: utiliza `v`, que agora não foi inicializado
```

A atribuição a `aside` move o vetor, deixando `v` não inicializado, e torna `r` um ponteiro perdido, conforme mostrado na Figura 5.7.

Embora `v` permaneça no escopo por todo o tempo de vida de `r`, o problema aqui é que o valor de `v` é movido para outro lugar, deixando `v` não inicializado enquanto `r` ainda o referencia. Naturalmente, o Rust detecta o erro:

```
error: cannot move out of `v` because it is borrowed
|
9 |   let r = &v;
|       - borrow of `v` occurs here
10 |   let aside = v; // move o vetor para o lado
|       ^^^^^ move out of `v` occurs here
```

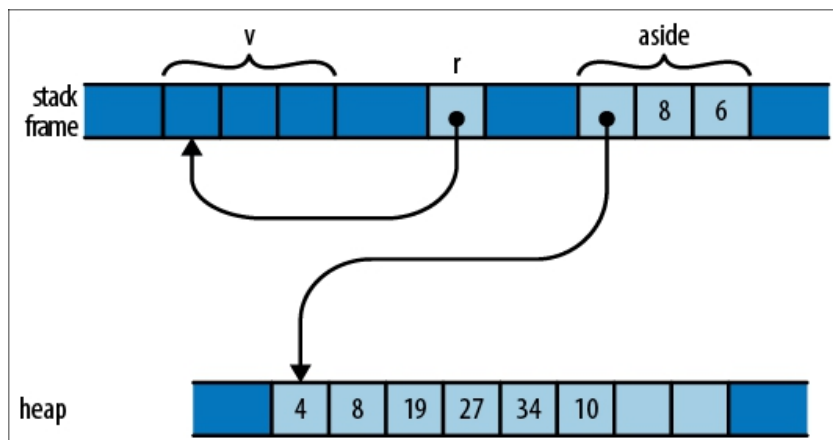


Figura 5.7: Uma referência a um vetor que foi movido para fora.
Ao longo de seu tempo de vida, uma referência compartilhada torna

seu referente somente leitura: você não pode atribuir algo ao referente ou mover seu valor para outro lugar. Neste código, o tempo de vida de `r` contém a tentativa de mover o vetor, então o Rust rejeita o programa. Se você alterar o programa conforme mostrado aqui, não há problema:

```
let v = vec![4, 8, 19, 27, 34, 10];
{
    let r = &v;
    r[0];    // ok: o vetor ainda está lá
}
let aside = v;
```

Nesta versão, `r` sai do escopo mais cedo, o tempo de vida da referência termina antes que `v` seja movido para o lado, e tudo está ok.

Eis uma maneira diferente de causar estragos. Suponha que temos uma função útil para estender um vetor com os elementos de uma fatia:

```
fn extend(vec: &mut Vec<f64>, slice: &[f64]) {
    for elt in slice {
        vec.push(*elt);
    }
}
```

Essa é uma versão menos flexível (e muito menos otimizada) do método `extend_from_slice` da biblioteca padrão em vetores. Podemos usá-lo para construir um vetor a partir de fatias de outros vetores ou arrays:

```
let mut wave = Vec::new();
let head = vec![0.0, 1.0];
let tail = [0.0, -1.0];

extend(&mut wave, &head); // estende wave com outro vetor
extend(&mut wave, &tail); // estende wave com um array

assert_eq!(wave, vec![0.0, 1.0, 0.0, -1.0]);
```

Construímos um período de uma onda senoidal aqui. Se quisermos adicionar outra ondulação, podemos acrescentar o vetor a ele mesmo?

```
extend(&mut wave, &wave);
assert_eq!(wave, vec![0.0, 1.0, 0.0, -1.0,
                     0.0, 1.0, 0.0, -1.0]);
```


Isso pode parecer bom em uma inspeção casual. Mas lembre-se de que, quando adicionamos um elemento a um vetor, se seu buffer estiver cheio, ele deve alocar um novo buffer com mais espaço. Suponha que `wave` começa com espaço para quatro elementos e, portanto, deve alocar um buffer maior quando `extend` tenta adicionar um quinto. A memória acaba se parecendo com a Figura 5.8.

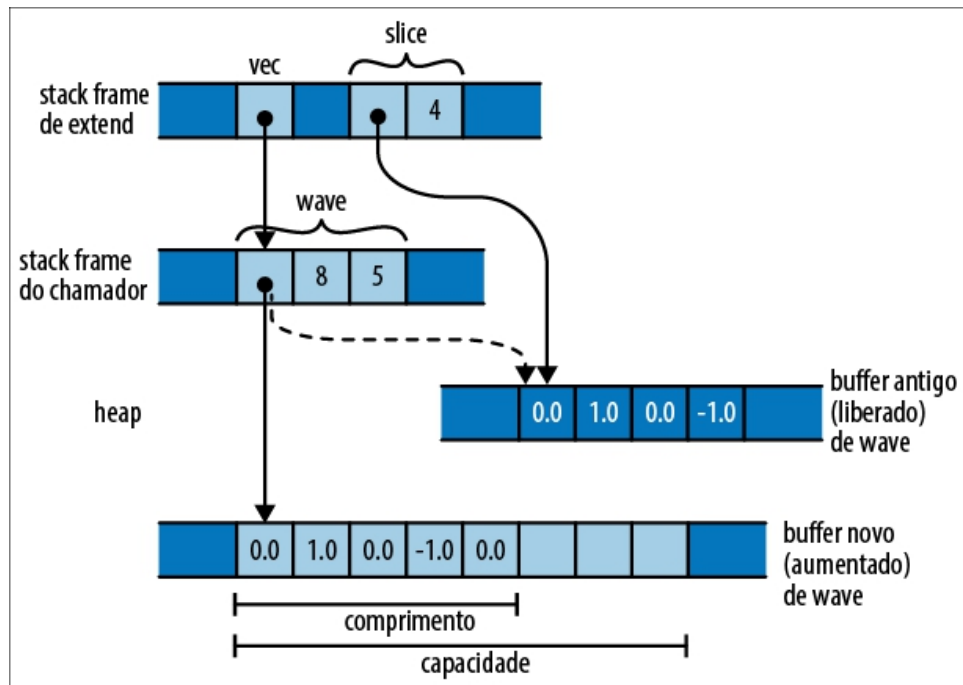


Figura 5.8: Uma fatia se transformou em um ponteiro perdido por uma realocação de vetor.

O argumento `vec` da função `extend` empresta `wave` (possuído pelo chamador), que alocou para si um novo buffer com espaço para oito elementos. Mas `slice` continua a apontar para o antigo buffer de quatro elementos, que foi dropado.

Esse tipo de problema não é exclusivo do Rust: modificar coleções enquanto aponta para elas é um território delicado em muitas linguagens. Em C++, a especificação `std::vector` adverte que “a realocação [do buffer do vetor] invalida todas as referências, ponteiros e iteradores referentes aos elementos na sequência”. Da mesma forma, diz o Java, sobre modificar um objeto `java.util.Hashtable`:

Se a `Hashtable` for estruturalmente modificada a qualquer

momento após a criação do iterador, de qualquer forma, exceto por meio do próprio método `remove` do iterador, o iterador gerará uma `ConcurrentModificationException`.

O que é especialmente difícil sobre esse tipo de bug é que ele não acontece o tempo todo. Nos testes, seu vetor pode sempre ter espaço suficiente, o buffer pode nunca ser realocado e o problema pode nunca vir à tona.

O Rust, porém, relata o problema com nossa chamada a `extend` em tempo de compilação:

```
error: cannot borrow `wave` as immutable because it is also
       borrowed as mutable
   |
9 |   extend(&mut wave, &wave);
   |           ^^^^^- mutable borrow ends here
   |           |
   |           immutable borrow occurs here
   | mutable borrow occurs here
```

Em outras palavras, podemos emprestar uma referência mutável ao vetor e podemos emprestar uma referência compartilhada a seus elementos, mas o tempo de vida dessas duas referências não deve se sobrepor. Em nosso caso, os tempos de vida de ambas as referências contêm a chamada para `extend`, então o Rust rejeita o código.

Esses erros decorrem de violações das regras do Rust para mutação e compartilhamento:

Acesso compartilhado é acesso somente leitura

Valores emprestados por referências compartilhadas são somente leitura. Durante o tempo de vida de uma referência compartilhada, nem seu referente, nem qualquer coisa alcançável a partir desse referente, pode ser alterado *por qualquer coisa*. Não existem referências mutáveis ativas para nada nessa estrutura, seu proprietário é considerado somente leitura, e assim por diante. Está realmente congelado.

Acesso mutável é acesso exclusivo

Um valor emprestado por uma referência mutável é alcançável exclusivamente por meio dessa referência. Durante o tempo de

vida de uma referência mutável, não há outro caminho utilizável para seu referente ou para qualquer valor alcançável a partir dela. As únicas referências cujos tempos de vida podem se sobrepor a uma referência mutável são aquelas que você empresta da própria referência mutável.

O Rust relatou o exemplo `extend` como uma violação da segunda regra: já que pegamos emprestado uma referência mutável a `wave`, essa referência mutável deve ser a única maneira de alcançar o vetor ou seus elementos. A referência compartilhada à fatia é em si outra forma de alcançar os elementos, violando a segunda regra.

Mas o Rust também pode ter tratado nosso bug como uma violação da primeira regra: já que pegamos emprestado uma referência compartilhada aos elementos de `wave`, os elementos e o próprio `Vec` são todos somente leitura. Você não pode pegar emprestado uma referência mutável a um valor somente leitura.

Cada tipo de referência afeta o que podemos fazer com os valores ao longo do caminho de posse para o referente e os valores alcançáveis a partir do referente (Figura 5.9).

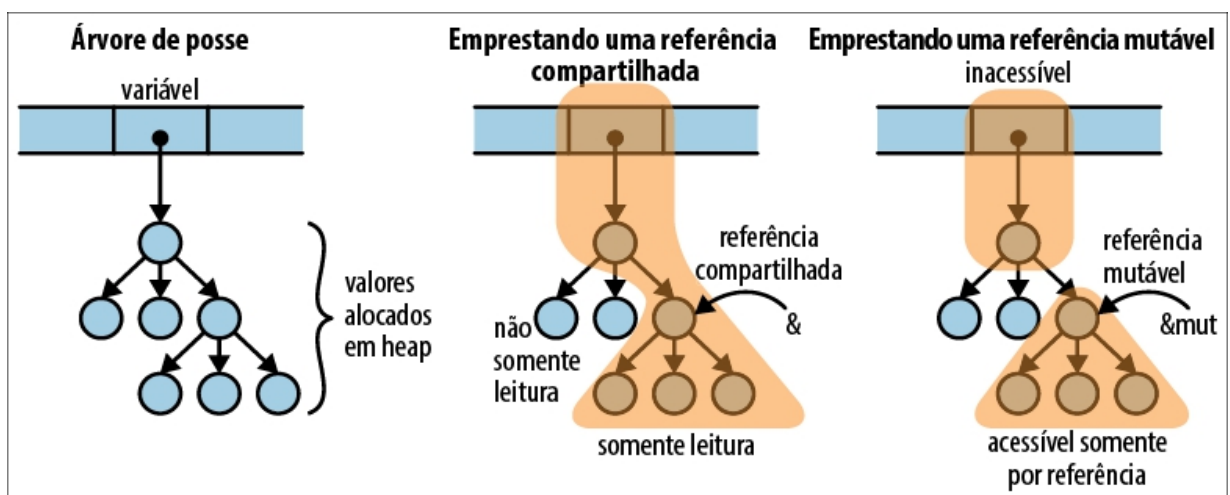


Figura 5.9: O empréstimo de uma referência afeta o que você pode fazer com outros valores na mesma árvore de posse.

Observe que, em ambos os casos, o caminho de posse que leva ao referente não pode ser alterado durante o tempo de vida da referência. Para um empréstimo compartilhado, o caminho é

somente leitura; para um empréstimo mutável, é completamente inacessível. Portanto, não há como o programa fazer algo que invalide a referência.

Reduzindo esses princípios aos exemplos mais simples possíveis:

```
let mut x = 10;
let r1 = &x;
let r2 = &x;    // ok: vários empréstimos compartilhados permitidos
x += 10;        // erro: não é possível atribuir a `x` porque ele foi emprestado
let m = &mut x; // erro: não é possível emprestar `x` como mutável porque
                // ele também foi emprestado como imutável
println!("{}", {}, {}, r1, r2, m); // as referências são utilizadas aqui,
                                   // então seus tempos de vida devem durar
                                   // pelo menos por esse tempo

let mut y = 20;
let m1 = &mut y;
let m2 = &mut y; // erro: não é possível emprestar como mutável mais de uma vez
let z = y;       // erro: não é possível utilizar `y` porque foi emprestado como mutável
println!("{}", {}, {}, m1, m2, z); // referências são utilizadas aqui
```

Não há problema em emprestar novamente uma referência compartilhada de uma referência compartilhada:

```
let mut w = (107, 109);
let r = &w;
let r0 = &r.0;    // ok: reemprestar compartilhado como compartilhado
let m1 = &mut r.1; // erro: não é possível reemprestar compartilhado como mutável
println!("{}", r0); // r0 é utilizado aqui
```

Você pode emprestar novamente de uma referência mutável:

```
let mut v = (136, 139);
let m = &mut v;
let m0 = &mut m.0; // ok: reemprestando mutável de mutável
*m0 = 137;
let r1 = &m.1;      // ok: reemprestar compartilhado de mutável,
                    // e não se sobrepõe a m0
v.1;                // erro: acesso por outros caminhos ainda proibido
println!("{}", r1); // r1 é utilizado aqui
```

Essas restrições são bastante rígidas. Voltando à nossa tentativa de chamada `extend(&mut wave, &wave)`, não há uma maneira rápida e fácil de corrigir o código para funcionar da maneira como gostaríamos. E o Rust aplica essas regras em todos os lugares: se pegarmos emprestado, digamos, uma referência compartilhada a uma chave em um `HashMap`, não podemos pegar emprestado uma referência mutável ao `HashMap` até que o tempo de vida da referência

compartilhada termine.

Mas há uma boa justificativa para isso: projetar coleções para suportar iteração e modificação irrestritas e simultâneas é difícil e muitas vezes impede implementações mais simples e eficientes. O `Hashtable` do Java e o `vector` do C++ não se preocupam, e nem os dicionários Python nem os objetos JavaScript definem exatamente como esse acesso se comporta. Outros tipos de coleção em JavaScript se preocupam, mas requerem implementações mais pesadas como resultado. O `std::map` do C++ promete que a inserção de novas entradas não invalida os ponteiros para outras entradas no mapa, mas, ao fazer essa promessa, o padrão impede designs mais eficientes em cache, como o `BTreeMap` do Rust, que armazena várias entradas em cada nó da árvore.

Eis outro exemplo do tipo de bug que essas regras pegam. Considere o seguinte código C++, destinado a gerenciar um descritor de arquivo. Para simplificar, mostraremos apenas um construtor e um operador de atribuição de cópia e omitiremos o tratamento de erros:

```
struct File {  
    int descriptor;  
  
    File(int d) : descriptor(d) { }  
  
    File& operator=(const File &rhs) {  
        close(descriptor);  
        descriptor = dup(rhs.descriptor);  
        return *this;  
    }  
};
```

O operador de atribuição é bastante simples, mas falha muito em uma situação como esta:

```
File f(open("foo.txt", ...));  
...  
f = f;
```

Se atribuirmos um `File` para ele próprio, tanto `rhs` como `*this` são o mesmo objeto, então `operator=` fecha o próprio descritor de arquivo para o qual está prestes a passar `dup`. Destruímos exatamente o recurso que deveríamos copiar.

No Rust, o código análogo seria:

```
struct File {  
    descriptor: i32  
}  
  
fn new_file(d: i32) -> File {  
    File { descriptor: d }  
}  
  
fn clone_from(this: &mut File, rhs: &File) {  
    close(this.descriptor);  
    this.descriptor = dup(rhs.descriptor);  
}
```

(Isso não é Rust idiomático. Existem maneiras excelentes de dar aos tipos do Rust suas próprias funções e métodos construtores, que descrevemos no Capítulo 9, mas as definições anteriores funcionam para esse exemplo.)

Se escrevermos o código Rust correspondente ao uso de `File`, obtemos:

```
let mut f = new_file(open("foo.txt", ...));  
...  
clone_from(&mut f, &f);
```

O Rust, naturalmente, se recusa a compilar esse código:

```
error: cannot borrow `f` as immutable because it is also  
       borrowed as mutable  
18 |     clone_from(&mut f, &f);  
   |           - ^- mutable borrow ends here  
   |           | |  
   |           | immutable borrow occurs here  
   |           mutable borrow occurs here
```

Isso deve parecer familiar. Acontece que dois bugs clássicos do C++ – falha em lidar com autoatribuição e uso de iteradores invalidados – são o mesmo tipo de bug subjacente! Em ambos os casos, o código assume que está modificando um valor enquanto consulta outro, quando na verdade ambos são o mesmo valor. Se você já acidentalmente deixou a origem e o destino de uma chamada a `memcpy` ou `strcpy` se sobrepor em C ou C++, essa é outra forma que o bug pode assumir. Ao exigir que o acesso mutável seja exclusivo, o Rust evitou uma ampla classe de erros cotidianos.

A impossibilidade de misturar referências compartilhadas e mutáveis realmente demonstra seu valor ao escrever código concorrente. Uma condição de concorrência só é possível quando algum valor é mutável e compartilhado entre threads – o que é exatamente o que as regras de referência do Rust eliminam. Um programa Rust concorrente que evita código `unsafe` está livre de condições de concorrência *por construção*. Abordaremos esse aspecto com mais detalhes quando falarmos sobre concorrência no Capítulo 19, mas, em resumo, concorrência é muito mais fácil de utilizar no Rust do que na maioria das outras linguagens.

Referências compartilhadas do Rust versus ponteiros de C para `const`

Na primeira inspeção, as referências compartilhadas do Rust parecem muito semelhantes aos ponteiros de C e C++ para valores `const`. Contudo, as regras do Rust para referências compartilhadas são muito mais rígidas. Por exemplo, considere o seguinte código C:

```
int x = 42;           // variável int, não const
const int *p = &x;    // ponteiro para const int
assert(*p == 42);
x++;                 // muda a variável diretamente
assert(*p == 43);    // o valor do referente "constante" mudou
```

O fato de que `p` é um `const int *` significa que você não pode modificar seu referente via o próprio `p`: `(*p)++` é proibido. Mas você também pode chegar ao referente diretamente como `x`, que não é `const`, e alterar seu valor dessa maneira. A palavra-chave `const` da família C tem seus usos, mas constante ela não é.

No Rust, uma referência compartilhada proíbe todas as modificações em seu referente, até que seu tempo de vida termine:

```
let mut x = 42;       // variável i32 não constante
let p = &x;           // referência compartilhada para i32
assert_eq!(*p, 42);
x += 1;               // erro: não é possível atribuir a x porque ele foi emprestado
assert_eq!(*p, 42);   // se você aceitar a atribuição, isto é verdade
```

Para garantir que um valor seja constante, precisamos

acompanhar todos os caminhos possíveis para esse valor e garantir que eles não permitam modificação ou não possam ser utilizados. Os ponteiros C e C++ são muito irrestritos para o compilador verificar isso. As referências do Rust estão sempre vinculadas a um tempo de vida específico, tornando possível verificá-las em tempo de compilação.

Pegando em armas contra um mar de objetos

Desde o surgimento do gerenciamento automático de memória na década de 1990, a arquitetura padrão de todos os programas têm sido o mar de objetos, mostrado na Figura 5.10.

Isso é o que acontece se você tiver coleta de lixo e começar a escrever um programa sem projetar nada. Todos nós construímos sistemas parecidos com esse.

Essa arquitetura tem muitas vantagens que não aparecem no diagrama: o progresso inicial é rápido, é fácil hackear coisas e, alguns anos depois, você não terá dificuldade em justificar em reescrever tudo do zero. (Já está ouvindo “Highway to Hell” do AC/DC?)

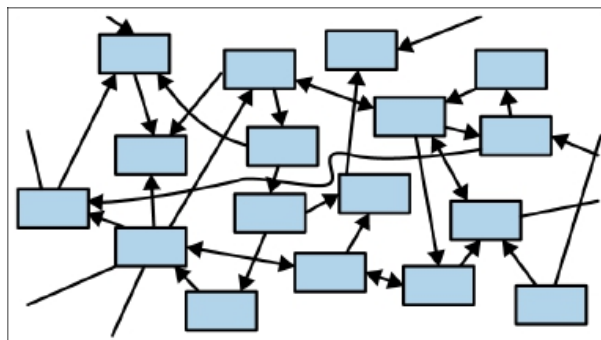


Figura 5.10: Um mar de objetos.

Naturalmente, também existem desvantagens. Quando tudo depende de todo o resto como neste caso, fica difícil testar, evoluir ou mesmo pensar em algum componente isoladamente.

Uma coisa fascinante sobre Rust é que o modelo de posse coloca uma lombada na estrada para o inferno. Dá um certo trabalho fazer

um ciclo no Rust – dois valores de forma que cada um contenha uma referência apontando para a outra. Você precisa utilizar um tipo de ponteiro inteligente, como `Rc`, e mutabilidade interior – um tópico que ainda nem abordamos. O Rust prefere que ponteiros, posse e fluxo de dados passem pelo sistema em uma direção, conforme mostrado na Figura 5.11.

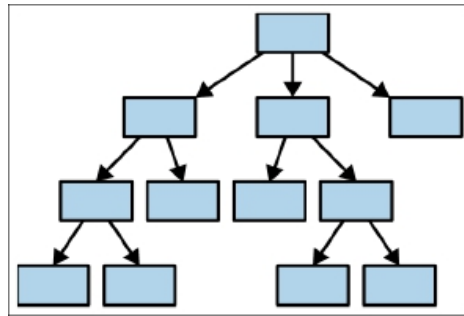


Figura 5.11: Uma árvore de valores.

A razão pela qual mencionamos isso agora é que seria natural, depois de ler este capítulo, querer sair correndo e criar um “mar de structs”, todas amarradas com ponteiros `Rc` inteligentes e recriar todos os antipadrões orientados a objetos com os quais você está familiarizado. Isso não vai funcionar para você imediatamente. O modelo de posse do Rust causará alguns problemas. A solução é fazer algum projeto inicial e construir um programa melhor.

O Rust tem tudo a ver com transferir a dor de entender seu programa do futuro para o presente. Ele funciona muito bem: o Rust não só pode forçá-lo a entender por que seu programa é thread-safe, como também pode exigir uma certa quantidade de projeto arquitetônico de alto nível.

CAPÍTULO 6

Expressões

Os programadores LISP sabem o valor de tudo, mas o custo de nada.

– Alan Perlis, epigrama #55

Neste capítulo, abordaremos *expressões* do Rust, os blocos de construção que compõem o corpo das funções Rust e, portanto, a maior parte do código Rust. A maioria das coisas no Rust são expressões. Neste capítulo, exploraremos o poder que isso traz e como trabalhar com suas limitações. Abordaremos o fluxo de controle, que no Rust é totalmente orientado a expressões, e como os operadores fundamentais do Rust funcionam isoladamente e combinados.

Alguns conceitos que se enquadram tecnicamente nessa categoria, como closures e iteradores, são profundos o suficiente para que possamos dedicar um capítulo inteiro a eles posteriormente. Por enquanto, pretendemos cobrir o máximo possível de sintaxe em poucas páginas.

Uma linguagem de expressão

O Rust se parece visualmente com a família C de linguagens, mas isso é meio capcioso. Em C, há uma distinção nítida entre *expressões*, fragmentos de código que se parecem com isto:

```
5 * (fahr-32) / 9
```

e *instruções*, que se parecem mais com isto:

```
for (; begin != end; ++begin) {  
    if (*begin == target)  
        break;  
}
```

Expressões têm valores. Instruções não.

O Rust é o que se chama de *linguagem de expressão*. Isso significa que segue uma tradição mais antiga, que remonta ao Lisp, em que as expressões fazem todo o trabalho.

Em C, `if` e `switch` são instruções. Elas não produzem um valor e não podem ser utilizadas no meio de uma expressão. No Rust, `if` e `match` *podem* produzir valores. Já vimos uma expressão `match` que produz um valor numérico no Capítulo 2:

```
pixels[r * bounds.0 + c] =  
    match escapes(Complex { re: point.0, im: point.1 }, 255) {  
        None => 0,  
        Some(count) => 255 - count as u8  
    };
```

Uma expressão `if` pode ser utilizada para inicializar uma variável:

```
let status =  
    if cpu.temperature <= MAX_TEMP {  
        HttpStatus::Ok  
    } else {  
        HttpStatus::ServerError // o servidor derreteu  
    };
```

Uma expressão `match` pode ser passada como um argumento para uma função ou macro:

```
println!("Inside the vat, you see {}. ",  
    match vat.contents {  
        Some(brain) => brain.desc(),  
        None => "nothing of interest"  
    });
```

Isso explica por que o Rust não possui o operador ternário do C (`expr1 ? expr2 : expr3`). Em C, esse recurso é um análogo útil no nível de expressão à instrução `if`. Seria redundante no Rust: a expressão `if` lida com ambos os casos.

A maioria das ferramentas de fluxo de controle em C são instruções. No Rust, todas são expressões.

Precedência e associatividade

A Tabela 6.1 resume a sintaxe de expressão do Rust. Discutiremos todos esses tipos de expressões neste capítulo. Os operadores são listados em ordem de precedência, do mais alto para o mais baixo.

(Como a maioria das linguagens de programação, o Rust tem uma *precedência de operadores* para determinar a ordem das operações quando uma expressão contém vários operadores adjacentes. Por exemplo, em `limit < 2 * broom.size + 1`, o operador `.` tem a precedência mais alta, então o acesso ao campo acontece primeiro.)

Tabela 6.1: Expressões

Tipo de expressão	Exemplo	Traits relacionados
Literal de array	<code>[1, 2, 3]</code>	
Literal de repetição de array	<code>[0; 50]</code>	
Tupla	<code>(6, "crullers")</code>	
Agrupamento	<code>(2 + 2)</code>	
Bloco	<code>{ f(); g() }</code>	
Expressões de fluxo de controle	<code>if ok { f() }</code>	
	<code>if ok { 1 } else { 0 }</code>	
	<code>if let Some(x) = f() { x } else { 0 }</code>	
	<code>match x { None => 0, _ => 1 }</code>	
	<code>for v in e { f(v); }</code>	<code>std::iter::IntoIterator</code>
	<code>while ok { ok = f(); }</code>	
	<code>while let Some(x) = it.next() { f(x); }</code>	
	<code>loop { next_event(); }</code>	
	<code>break</code>	
	<code>continue</code>	
	<code>return 0</code>	
Chamada de macro	<code>println!("ok")</code>	
Caminho	<code>std::f64::consts::PI</code>	
Literal de struct	<code>Point {x: 0, y: 0}</code>	
Acesso ao campo de tupla	<code>pair.0</code>	<code>Deref</code> , <code>DerefMut</code>
Acesso ao campo de struct	<code>point.x</code>	<code>Deref</code> , <code>DerefMut</code>
Chamada de método	<code>point.translate(50, 50)</code>	<code>Deref</code> , <code>DerefMut</code>
Chamada de função	<code>stdin()</code>	<code>Fn(Arg0, ...) -> T</code> , <code>FnMut(Arg0, ...) -> T</code> , <code>FnOnce(Arg0, ...) -> T</code>
Índice	<code>arr[0]</code>	<code>Index</code> , <code>IndexMutDeref</code> , <code>DerefMut</code>
Verificação de erro	<code>create_dir("tmp")?</code>	

Tipo de expressão	Exemplo	Traits relacionados
NOT lógico/bit a bit	!ok	Not
Negação	-num	Neg
Desreferência	*ptr	Deref, DerefMut
Empréstimo	&val	
Conversão de tipo	x as u32	
Multiplicação	n * 2	Mul
Divisão	n / 2	Div
Resto (módulo)	n % 2	Rem
Adição	n + 1	Add
Subtração	n - 1	Sub
Deslocamento para a esquerda	n << 1	Shl
Deslocamento para a direita	n >> 1	Shr
AND bit a bit	n & 1	BitAnd
OR exclusivo bit a bit	n ^ 1	BitXor
OR bit a bit	n 1	BitOr
Menor que	n < 1	std::cmp::PartialOrd
Menor ou igual	n <= 1	std::cmp::PartialOrd
Maior que	n > 1	std::cmp::PartialOrd
Maior ou igual	n >= 1	std::cmp::PartialOrd
Igual	n == 1	std::cmp::PartialEq
Não igual	n != 1	std::cmp::PartialEq
AND lógico	x.ok && y.ok	
OR lógico	x.ok backup.ok	
Intervalo não inclusivo	start .. stop	
Intervalo inclusivo	start ..= stop	
Atribuição	x = val	
Atribuição composta	x *= 1	MulAssign
	x /= 1	DivAssign
	x %= 1	RemAssign
	x += 1	AddAssign
	x -= 1	SubAssign
	x <<= 1	ShlAssign
	x >>= 1	ShrAssign
	x &= 1	BitAndAssign
	x ^= 1	BitXorAssign

Tipo de expressão	Exemplo	Traits relacionados
	<code>x = 1</code>	BitOrAssign
Closure	<code> x, y x + y</code>	

Todos os operadores que podem ser encadeados de forma útil são associativos à esquerda. Ou seja, uma cadeia de operações como `a - b - c` é agrupada como `(a - b) - c`, não `a - (b - c)`. Os operadores que podem ser encadeados dessa maneira são todos os que você pode esperar:

`* / % + - << >> & ^ | && || as`

Os operadores de comparação, os operadores de atribuição e os operadores de intervalo `..` e `..=` não podem ser encadeados de nenhuma forma.

Blocos e pontos e vírgulas

Blocos são o tipo mais geral de expressão. Um bloco produz um valor e pode ser utilizado em qualquer lugar em que um valor seja necessário:

```
let display_name = match post.author() {
    Some(author) => author.name(),
    None => {
        let network_info = post.get_network_metadata()?;
        let ip = network_info.client_address();
        ip.to_string()
    }
};
```

O código depois de `Some(author) =>` é a expressão simples `author.name()`. O código depois de `None =>` é uma expressão de bloco. Não faz diferença para Rust. O valor do bloco é o valor de sua última expressão, `ip.to_string()`.

Note que não há ponto e vírgula após a chamada de método `ip.to_string()`. A maioria das linhas de código Rust termina com um ponto e vírgula ou chaves, assim como C ou Java. E se um bloco se parece com o código C, com ponto e vírgula em todos os lugares familiares, ele será executado como um bloco C e seu valor será `()`. Como mencionamos no Capítulo 2, quando você deixa o ponto e vírgula fora da última linha de um bloco, isso torna o valor do bloco

o valor de sua expressão final, em vez do valor usual ().

Em algumas linguagens, particularmente JavaScript, você pode omitir o ponto e vírgula e a linguagem simplesmente os insere para você – uma pequena conveniência. Isso é diferente. No Rust, o ponto e vírgula realmente significa algo:

```
let msg = {  
    // declaração let: ponto e vírgula é sempre obrigatório  
    let dandelion_control = puffball.open();  
  
    // expressão + ponto e vírgula: o método é chamado, o valor de retorno é dropado  
    dandelion_control.release_all_seeds(launch_codes);  
  
    // expressão sem ponto e vírgula: o método é chamado,  
    // valor de retorno é armazenado em `msg`  
    dandelion_control.get_status()  
};
```

Essa capacidade dos blocos de conter declarações e, também, produzir um valor no final é um recurso interessante, que rapidamente parece natural. A única desvantagem é que isso leva a uma mensagem de erro estranha quando você omite um ponto e vírgula por acidente:

```
...  
if preferences.changed() {  
    page.compute_size() // ops, falta ponto e vírgula  
}  
...
```

Se você cometesse esse erro em um programa C ou Java, o compilador simplesmente apontaria que está faltando um ponto e vírgula. Eis o que o Rust diz:

```
error: mismatched types  
22 |     page.compute_size() // ops, falta ponto e vírgula  
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ help: try adding a semicolon: `;`  
   |     |  
   |     expected (), found tuple  
   |  
= note: expected unit type `()`  
        found tuple `(u32, u32)`
```

Com a falta do ponto e vírgula, o valor do bloco seria o que quer que `page.compute_size()` retorne, mas um `if` sem um `else` deve sempre retornar (). Felizmente, o Rust já viu esse tipo de coisa antes e

sugere adicionar o ponto e vírgula.

Declarações

Além de expressões e ponto e vírgulas, um bloco pode conter qualquer número de declarações. Os mais comuns são declarações `let`, que declaram variáveis locais:

```
let name: type = expr;
```

O tipo e o inicializador são opcionais. O ponto e vírgula é obrigatório. Como todos os identificadores no Rust, os nomes das variáveis devem começar com uma letra ou sublinhado e podem conter dígitos somente após o primeiro caractere. O Rust tem uma definição ampla de “letra”: isso inclui letras gregas, caracteres latinos acentuados e muitos outros símbolos – qualquer coisa que o Unicode Standard Annex #31 declare adequado. Emojis não são permitidos.

Uma declaração `let` pode declarar uma variável sem inicializá-la. A variável pode então ser inicializada com uma atribuição posterior. Isso é ocasionalmente útil, porque às vezes uma variável deve ser inicializada no meio de algum tipo de construção de fluxo de controle:

```
let name;
if user.has_nickname() {
    name = user.nickname();
} else {
    name = generate_unique_name();
    user.register(&name);
}
```

Aqui existem duas maneiras diferentes de se inicializar a variável local `name`, mas de qualquer maneira será inicializada exatamente uma vez e, portanto, `name` não precisa mais ser declarada `mut`.

É um erro utilizar uma variável antes de ser inicializada. (Isso está intimamente relacionado ao erro de utilizar um valor depois que ele foi movido. O Rust realmente quer que você utilize valores apenas enquanto eles existirem!)

Ocasionalmente, você pode ver um código que parece redeclarar

uma variável existente, como este:

```
for line in file.lines() {  
    let line = line?;  
    ...  
}
```

A declaração `let` cria uma nova segunda variável, de um tipo diferente. O tipo da primeira variável `line` é `Result<String, io::Error>`. O segundo `line` é um `String`. Sua definição substitui a primeira para o resto do bloco. Isso é chamado de *sombreamento* e é muito comum em programas Rust. O código é equivalente a:

```
for line_result in file.lines() {  
    let line = line_result?;  
    ...  
}
```

Neste livro, continuaremos utilizando um sufixo `_result` nessas situações para que as variáveis tenham nomes distintos.

Um bloco também pode conter *declarações de itens*. Um item é simplesmente qualquer declaração que pode aparecer globalmente em um programa ou módulo, como um `fn`, `struct` ou `use`.

Os capítulos posteriores abordarão os itens em detalhes. Por ora, `fn` dá um exemplo suficiente. Qualquer bloco pode conter uma `fn`:

```
use std::io;  
use std::cmp::Ordering;  
  
fn show_files() -> io::Result<()> {  
    let mut v = vec![];  
    ...  
  
    fn cmp_by_timestamp_then_name(a: &FileInfo, b: &FileInfo) -> Ordering {  
        a.timestamp.cmp(&b.timestamp) // primeiro, compara os registros de data/hora  
            .reverse() // arquivo mais novo primeiro  
            .then(a.path.cmp(&b.path)) // compara caminhos para decidir empates  
    }  
  
    v.sort_by(cmp_by_timestamp_then_name);  
    ...  
}
```

Quando uma `fn` é declarada dentro de um bloco, seu escopo é o bloco inteiro – ou seja, pode ser utilizado em todo o bloco envolvente. Mas uma `fn` aninhada não pode acessar variáveis locais

ou argumentos que estejam no escopo. Por exemplo, a função `cmp_by_timestamp_then_name` não poderia utilizar `v` diretamente. (O Rust também tem closures, que permitem a inclusão de escopos. Consulte o Capítulo 14.)

Um bloco pode até conter um módulo inteiro. Isso pode parecer um pouco demais – precisamos realmente ser capazes de aninhar *cada* fragmento da linguagem dentro de cada outro fragmento? –, mas os programadores (e particularmente os programadores que utilizam macros) têm uma maneira de encontrar usos para cada gota de ortogonalidade que a linguagem oferece.

if e match

A forma de uma expressão `if` é familiar:

```
if condition1 {  
    block1  
} else if condition2 {  
    block2  
} else {  
    block_n  
}
```

Cada `condition` deve ser uma expressão do tipo `bool`; fiel à forma, o Rust não converte implicitamente números ou ponteiros em valores booleanos.

Ao contrário de C, os parênteses não são necessários em torno das condições. Na verdade, `rustc` emitirá um aviso se parênteses desnecessários estiverem presentes. As chaves, porém, são necessárias.

Os blocos `else if`, bem como o `else final`, são opcionais. Uma expressão `if` sem bloco `else` se comporta exatamente como se tivesse um vazio bloco `else`.

Expressões `match` são algo como a instrução `switch` em C, porém mais flexível. Um exemplo simples:

```
match code {  
    0 => println!("OK"),  
    1 => println!("Wires Tangled"),  
    2 => println!("User Asleep"),
```

```

    _ => println!("Unrecognized Error {}", code)
}

```

Isso é algo que uma instrução `switch` poderia fazer. Exatamente um dos quatro braços¹ dessa expressão `match` será executado, dependendo do valor de `code`. O curinga `_` corresponde a tudo. Isso é como o caso `default`: em uma instrução `switch`, exceto que deve vir por último; colocar um padrão `_` antes de outros padrões significa que esse padrão terá precedência sobre os outros. Esses padrões nunca corresponderão com nada (e o compilador o avisará sobre isso).

O compilador pode otimizar esse tipo de `match` utilizando uma tabela de salto, assim como uma instrução `switch` em C++. Uma otimização semelhante é aplicada quando cada braço de um `match` produz um valor constante. Nesse caso, o compilador cria um array desses valores e o `match` é compilado em um acesso de array. Além de uma verificação de limites, não há nenhuma ramificação no código compilado.

A versatilidade de `match` decorre da variedade de padrões suportados que podem ser utilizados à esquerda de `=>` em cada braço. Acima, cada padrão é simplesmente um número inteiro constante. Também mostramos expressões `match` que distinguem os dois tipos de valor `Option`:

```

match params.get("name") {
    Some(name) => println!("Hello, {}", name),
    None => println!("Greetings, stranger.")
}

```

Essa é apenas uma sugestão do que os padrões podem fazer. Um padrão pode corresponder a um intervalo de valores. Pode desempacotar tuplas. Pode corresponder a campos individuais de structs. Pode buscar referências, emprestar partes de um valor e muito mais. Os padrões do Rust são uma minilinguagem própria. Dedicaremos várias páginas a eles no Capítulo 10.

A forma geral de uma expressão `match` é:

```

match value {
    pattern => expr,
    ...
}

```

A vírgula após um braço pode ser dropada se a *expr* é um bloco.

O Rust verifica o dado *value* contra cada padrão por vez, começando com o primeiro. Quando um padrão corresponde, a *expr* correspondente é avaliada e a expressão *match* está completa; nenhum outro padrão é verificado. Pelo menos um dos padrões deve corresponder. O Rust proíbe expressões *match* que não cobrem todos os valores possíveis:

```
let score = match card.rank {  
    Jack => 10,  
    Queen => 10,  
    Ace => 11  
}; // erro: padrões não exaustivos
```

Todos os blocos de uma expressão *if* devem produzir valores do mesmo tipo:

```
let suggested_pet =  
    if with_wings { Pet::Buzzard } else { Pet::Hyena }; // ok  
  
let favorite_number =  
    if user.is_hobbit() { "eleventy-one" } else { 9 }; // erro  
  
let best_sports_team =  
    if is_hockey_season() { "Predators" }; // erro
```

(O último exemplo é um erro porque em julho o resultado seria ().)

Da mesma forma, todos os braços de uma expressão *match* devem ter o mesmo tipo:

```
let suggested_pet =  
    match favorites.element {  
        Fire => Pet::RedPanda,  
        Air => Pet::Buffalo,  
        Water => Pet::Orca,  
        _ => None // erro: tipos incompatíveis  
    };
```

if let

Há mais uma forma de *if*, a expressão *if let*:

```
if let pattern = expr {  
    block1  
} else {  
    block2  
}
```

A *expr* fornecida deve corresponder ao *pattern* para que *block1* seja executado, caso contrário *block2* é executado. Às vezes, essa é uma boa maneira de obter dados de uma `Option` ou um `Result`:

```
if let Some(cookie) = request.session_cookie {  
    return restore_session(cookie);  
}  
  
if let Err(err) = show_cheesy_anti_robot_task() {  
    log_robot_attempt(err);  
    politely_accuse_user_of_being_a_robot();  
} else {  
    session.mark_as_human();  
}
```

Nunca é estritamente *necessário* utilizar `if let`, porque `match` pode fazer tudo que `if let` pode fazer. Uma expressão `if let` é uma abreviação para um `match` com apenas um padrão:

```
match expr {  
    pattern => { block1 }  
    _ => { block2 }  
}
```

Loops

Existem quatro expressões de loop:

```
while condition {  
    block  
}  
  
while let pattern = expr {  
    block  
}  
  
loop {  
    block  
}  
  
for pattern in iterable {  
    block  
}
```

Loops são expressões no Rust, mas o valor de um loop `while` ou `for` é sempre `()`, então seu valor não é muito útil. Uma expressão `loop` pode produzir um valor se você especificar um.

Um loop `while` se comporta exatamente como o equivalente em C,

exceto que, novamente, a *condition* deve ser exatamente do tipo `bool`.

O loop `while let` é análogo a `if let`. No início de cada iteração do loop, o valor de *expr* ou corresponde ao dado *pattern*, caso em que o bloco é executado; ou não, caso em que o loop é encerrado.

Utilize `loop` para escrever loops infinitos. Ele executa o *block* repetidamente para sempre (ou até que um `break` ou `return` é alcançado ou o thread gere um pânico).

Um loop `for` avalia a expressão *iterable* e, em seguida, avalia o *block* uma vez para cada valor no iterador resultante. Muitos tipos podem ser iterados, incluindo todas as coleções padrão como `Vec` e `HashMap`.

O loop `for` padrão em C:

```
for (int i = 0; i < 20; i++) {  
    printf("%d\n", i);  
}
```

é escrito assim no Rust:

```
for i in 0..20 {  
    println!("{}", i);  
}
```

Como em C, o último número impresso é 19.

O operador `..` produz um *intervalo*, um struct simples com dois campos: `start` e `end`. `0..20` é o mesmo que `std::ops::Range { start: 0, end: 20 }`. Os intervalos podem ser utilizados com loops `for` porque `Range` é um tipo iterável: ele implementa o `trait std::iter::IntoIterator`, que discutiremos no Capítulo 15. Todas as coleções padrão são iteráveis, assim como arrays e fatias.

De acordo com a semântica de movimento do Rust, um loop `for` sobre um valor consome o valor:

```
let strings: Vec<String> = error_messages();  
for s in strings {                      // cada String é movida para s aqui...  
    println!("{}", s);  
}                                       // ...e dropada aqui  
println!("{}", error(s), strings.len()); // erro: uso do valor movido
```

Isso pode ser inconveniente. A solução fácil é fazer um loop por uma referência à coleção. A variável do loop, então, será uma referência a cada item da coleção:

```
for rs in &strings {
```

```
println!("String {:?} is at address {:p}.", *rs, rs);
}
```

Aqui o tipo de `&strings` é `&Vec<String>` e o tipo de `rs` é `&String`.

Iterar sobre uma referência `mut` fornece uma referência `mut` a cada elemento:

```
for rs in &mut strings { // o tipo de rs é &mut String
    rs.push('\n'); // acrescente uma nova linha a cada string
}
```

O Capítulo 15 aborda loops `for` com mais detalhes e mostra muitas outras maneiras de utilizar iteradores.

Fluxo de controle em loops

Uma expressão `break` sai de um loop que a envolve. (No Rust, `break` funciona apenas em loops. Não é necessário em expressões `match`, que são diferentes da instrução `switch` nesse sentido.)

Dentro do corpo de um loop, você pode interromper uma expressão com `break`, e seu valor se torna o do loop:

```
// Cada chamada a `next_line` retorna `Some(line)`, onde `line` é uma linha
// de entrada, ou `None`, se chegamos ao final da entrada. Retorne a primeira
// linha que começa com "answer: ". Caso contrário, retorna "answer: nothing".
let answer = loop {
    if let Some(line) = next_line() {
        if line.starts_with("answer: ") {
            break line;
        }
    } else {
        break "answer: nothing";
    }
};
```

Naturalmente, todas as expressões `break` dentro de um loop devem produzir valores com o mesmo tipo, que se torna o tipo do loop em si.

Uma expressão `continue` salta para a próxima iteração do loop:

```
// Leia alguns dados, uma linha por vez
for line in input_lines {
    let trimmed = trim_comments_and_whitespace(line);
    if trimmed.is_empty() {
        // Salte de volta para o topo do loop e
        // passe para a próxima linha de entrada
    }
}
```

```

        continue;
    }
    ...
}

```

Em um loop `for`, `continue` avança para o próximo valor na coleção. Se não houver mais valores, o loop será encerrado. Da mesma forma, em um loop `while`, `continue` verifica novamente a condição do loop. Se agora for falso, o loop será encerrado.

Um loop pode ser *rotulado* com um tempo de vida. No exemplo a seguir, `'search:` é um rótulo para o loop `for` externo. Por isso, `break 'search` sai desse loop, não do loop interno:

```

'search:
for room in apartment {
    for spot in room.hiding_spots() {
        if spot.contains(keys) {
            println!("Your keys are {} in the {}. ", spot, room);
            break 'search;
        }
    }
}
}

```

Um `break` pode ter um rótulo e uma expressão de valor:

```

// Encontra a raiz quadrada do primeiro quadrado perfeito na série
let sqrt = 'outer: loop {
    let n = next_number();
    for i in 1.. {
        let square = i * i;
        if square == n {
            // Encontrou uma raiz quadrada
            break 'outer i;
        }
        if square > n {
            // `n` não é um quadrado perfeito, tente o próximo
            break;
        }
    }
};

```

Os rótulos também podem ser utilizados com `continue`.

Expressões `return`

Uma expressão `return` sai da função atual, retornando um valor para o

chamador.

`return` sem um valor é um atalho para `return ()`:

```
fn f() {    // tipo de retorno omitido: o padrão é ()
    return; // valor de retorno omitido: o padrão é ()
}
```

As funções não precisam ter uma expressão `return` explícita. O corpo de uma função funciona como uma expressão de bloco: se a última expressão não for seguida por um ponto e vírgula, seu valor será o valor de retorno da função. Na verdade, essa é a maneira preferida de fornecer o valor de retorno de uma função no Rust.

Mas isso não significa que `return` seja inútil, ou apenas uma concessão para usuários que não têm experiência com linguagens de expressão. Como uma expressão `break`, `return` pode abandonar o trabalho em andamento. Por exemplo, no Capítulo 2, utilizamos o operador `?` para verificar erros após chamar uma função que pode falhar:

```
let output = File::create(filename)?;
```

Explicamos que isso é uma abreviação para uma expressão `match`:

```
let output = match File::create(filename) {
    Ok(f) => f,
    Err(err) => return Err(err)
};
```

Esse código começa chamando `File::create(filename)`. Se isso retornar `Ok(f)`, então toda expressão `match` é avaliada como `f`, portanto `f` está armazenado em `output` e continuamos com a próxima linha de código seguindo o `match`.

Caso contrário, corresponderemos `Err(err)` e alcançaremos a expressão `return`. Quando isso acontece, não importa que estejamos avaliando uma expressão `match` para determinar o valor da variável `output`. Abandonamos tudo isso e saímos da função que a envolve, retornando qualquer erro que obtivemos de `File::create()`.

Vamos cobrir o operador `?` mais completamente em “Propagando erros”, na página 196.

Por que o Rust tem loops

Várias partes do compilador Rust analisam o fluxo de controle do seu programa:

- O Rust verifica se todo caminho ao longo de uma função retorna um valor do tipo de retorno esperado. Para fazer isso corretamente, ele precisa saber se é possível chegar ao final da função.
- O Rust verifica se as variáveis locais nunca são utilizadas não inicializadas. Isso envolve verificar cada caminho através de uma função para garantir que não haja como chegar a um local onde uma variável é utilizada sem já ter passado pelo código que a inicializa.
- O Rust avisa sobre código inacessível. O código é inacessível se *nenhum* caminho ao longo da função o alcança.

Isso é chamado de análise *sensível ao fluxo*. Não há nada de novo nisso; o Java tem uma análise de “atribuição definida”, semelhante à do Rust, há anos.

Ao impor esse tipo de regra, uma linguagem deve encontrar um equilíbrio entre a simplicidade, que torna mais fácil para os programadores descobrirem do que o compilador está falando às vezes, e a inteligência, que pode ajudar a eliminar falsos avisos e casos em que o compilador rejeita um programa perfeitamente seguro. O Rust optou pela simplicidade. Suas análises sensíveis ao fluxo não examinam as condições do loop, simplesmente assumem que qualquer condição em um programa pode ser verdadeira ou falsa.

Isso faz com que o Rust rejeite alguns programas seguros:

```
fn wait_for_process(process: &mut Process) -> i32 {  
    while true {  
        if process.wait() {  
            return process.exit_code();  
        }  
    }  
} // erro: tipos incompatíveis: i32 esperado, encontrado ()
```

O erro aqui é falso. Essa função só sai através da instrução `return`, então o fato de que o loop `while` não produz um `i32` é irrelevante.

A expressão `loop` é oferecida como uma solução do tipo “diga o que

“você quer dizer” para esse problema.

O sistema de tipos do Rust também é afetado pelo fluxo de controle. Anteriormente, dissemos que todos os ramos de uma expressão `if` devem ter o mesmo tipo. Mas seria bobagem impor essa regra em blocos que terminam com uma expressão `break` ou `return`, um loop infinito, ou uma chamada a `panic!()` ou `std::process::exit()`. O que todas essas expressões têm em comum é que elas nunca terminam da maneira usual, produzindo um valor. Um `break` ou `return` sai do bloco atual abruptamente, um loop infinito nunca termina e assim por diante.

Portanto, no Rust, essas expressões não têm um tipo normal. As expressões que não terminam normalmente recebem o tipo especial `!` e estão isentas das regras sobre os tipos que precisam corresponder. Você pode ver `!` na assinatura de função de `std::process::exit()`:

```
fn exit(code: i32) -> !
```

O `!` significa que `exit()` nunca retorna. É uma *função divergente*.

Você pode escrever suas próprias funções divergentes utilizando a mesma sintaxe e isso é perfeitamente natural em alguns casos:

```
fn serve_forever(socket: ServerSocket, handler: ServerHandler) -> ! {  
    socket.listen();  
    loop {  
        let s = socket.accept();  
        handler.handle(s);  
    }  
}
```

Naturalmente, o Rust então considera um erro se a função puder retornar normalmente.

Com esses blocos de construção de fluxo de controle de larga escala no lugar, podemos passar para as expressões mais refinadas normalmente utilizadas dentro desse fluxo, como chamadas de função e operadores aritméticos.

Chamadas de função e método

A sintaxe para chamar funções e métodos é a mesma no Rust e em

muitas outras linguagens:

```
let x = gcd(1302, 462); // chamada de função
```

```
let room = player.location(); // chamada de método
```

No segundo exemplo aqui, `player` é uma variável inventada do tipo `Player`, que tem um método `.location()` também inventado. (Mostraremos como definir seus próprios métodos quando começarmos a falar sobre tipos definidos pelo usuário no Capítulo 9.)

O Rust geralmente faz uma distinção nítida entre referências e os valores que elas referenciam. Se você passar um `&i32` para uma função que espera um `i32`, isso é um erro de tipo. Você notará que o operador `.` relaxa um pouco essas regras. Na chamada do método `player.location()`, `player` pode ser um `Player`, uma referência do tipo `&Player`, ou um ponteiro inteligente do tipo `Box<Player>` ou `Rc<Player>`. O método `.location()` pode aceitar o jogador por valor ou por referência. A mesma sintaxe `.location()` funciona em todos os casos, porque o operador `.` do Rust desreferencia automaticamente `player` ou empresta uma referência a ele, conforme necessário.

Uma terceira sintaxe é utilizada para chamar funções associadas a tipos, como `Vec::new()`:

```
let mut numbers = Vec::new(); // chamada de função associada ao tipo
```

Estes são semelhantes a métodos estáticos em linguagens orientadas a objetos: métodos comuns são chamados em valores (como `my_vec.len()`) e funções associadas a tipos são chamadas em tipos (como `Vec::new()`).

Naturalmente, as chamadas de método podem ser encadeadas:

```
// Do servidor da Web baseado em Actix no Capítulo 2:  
server  
    .bind("127.0.0.1:3000").expect("error binding server to address")  
    .run().expect("error running server");
```

Uma peculiaridade da sintaxe do Rust é que, em uma chamada de função ou chamada de método, a sintaxe usual para tipos genéricos, `Vec<T>`, não funciona:

```
return Vec<i32>::with_capacity(1000); // erro: algo sobre comparações encadeadas
```

```
let ramp = (0 .. n).collect<Vec<i32>>(); // mesmo erro
```

O problema é que, nas expressões, `<` é o operador menor que. O compilador Rust sugere útil escrever `::<T>` em vez de `<T>` neste caso e isso resolve o problema:

```
return Vec::i32::with_capacity(1000); // ok, utilizando ::<
```

```
let ramp = (0 .. n).collect::
```

O símbolo `::<...>` é carinhosamente conhecido na comunidade Rust como *turbofish*, ou *turbopeixe*.

Como alternativa, muitas vezes é possível dropar os parâmetros de tipo e deixar o Rust inferi-los:

```
return Vec::with_capacity(10); // ok, se o tipo de retorno fn for Vec<i32>
```

```
let ramp: Vec<i32> = (0 .. n).collect(); // ok, o tipo da variável é dado
```

É considerado um bom estilo omitir os tipos sempre que eles puderem ser inferidos.

Campos e elementos

Os campos de um struct são acessados utilizando uma sintaxe familiar. As tuplas são iguais, exceto que seus campos têm números em vez de nomes:

```
game.black_pawns // campo de struct  
coords.1         // elemento de tupla
```

Se o valor à esquerda do ponto for uma referência ou um tipo de ponteiro inteligente, ele será automaticamente desreferenciado, assim como ocorre com chamadas de método.

Os colchetes acessam os elementos de um array, uma fatia ou um vetor:

```
pieces[i]        // elemento de array
```

O valor à esquerda dos colchetes é automaticamente desreferenciado.

Expressões como essas três são chamadas *lvalues*, porque elas podem aparecer no lado esquerdo de uma atribuição:

```
game.black_pawns = 0x00ff0000_00000000_u64;  
coords.1 = 0;  
pieces[2] = Some(Piece::new(Black, Knight, coords));
```

Naturalmente, isso só é permitido se `game`, `coords` e `pieces` são

declarados como variáveis `mut`.

Extrair uma fatia de um array ou vetor é simples:

```
let second_half = &game_moves[midpoint .. end];
```

Aqui `game_moves` pode ser um array, uma fatia ou um vetor; o resultado, independentemente de tudo, é uma fatia emprestada de comprimento `end - midpoint`. `game_moves` é considerado emprestado para o tempo de vida de `second_half`.

O operador `..` permite que qualquer operando seja omitido; produz até quatro tipos diferentes de objetos dependendo de quais operandos estão presentes:

```
..      // RangeFull
a ..    // RangeFrom { start: a }
.. b    // RangeTo { end: b }
a .. b  // Range { start: a, end: b }
```

As duas últimas formas são *não inclusivas* (ou *meio abertas*): o valor final não está incluído no intervalo representado. Por exemplo, o intervalo `0 .. 3` inclui os números 0, 1 e 2.

O operador `..=` produz intervalos *inclusivos* (ou *fechados*), que incluem o valor final:

```
..= b    // RangeToInclusive { end: b }
a ..= b  // RangeInclusive::new(a, b)
```

Por exemplo, o intervalo `0 ..= 3` inclui os números 0, 1, 2 e 3.

Somente intervalos que incluem um valor inicial são iteráveis, pois um loop deve ter um ponto de início. Mas no fatiamento de array, todas as seis formas são úteis. Se o início ou o fim do intervalo for omitido, o padrão será o início ou o fim dos dados que estão sendo divididos.

Portanto, uma implementação de quicksort, o clássico algoritmo de ordenação de divisão e conquista, pode parecer, em parte, assim:

```
fn quicksort<T: Ord>(slice: &mut [T]) {
    if slice.len() <= 1 {
        return; // Nada para resolver
    }

    // Divida a fatia em duas partes, frontal e traseira
    let pivot_index = partition(slice);

    // Ordene recursivamente a metade frontal de `slice`
```

```
quicksort(&mut slice[.. pivot_index]);  
  
// E a metade traseira  
quicksort(&mut slice[pivot_index + 1 ..]);  
}
```

Operadores de referência

Os operadores de endereço & e &mut são abordados no Capítulo 5.

O operador unário * é utilizado para acessar o valor apontado por uma referência. Como vimos, o Rust segue referências automaticamente quando você utiliza o operador . para acessar um campo ou método, então o operador * é necessário apenas quando queremos ler ou gravar todo o valor para o qual a referência aponta. Por exemplo, às vezes um iterador produz referências, mas o programa precisa dos valores subjacentes:

```
let padovan: Vec<u64> = compute_padovan_sequence(n);  
for elem in &padovan {  
    draw_triangle(turtle, *elem);  
}
```

Neste exemplo, o tipo de elem é &u64, então *elem é um u64.

Operadores aritméticos, de bit a bit, de comparação e lógicos

Os operadores binários do Rust são como os de muitas outras linguagens. Para economizar tempo, assumimos familiaridade com uma dessas linguagens e nos concentramos nos poucos pontos em que o Rust se afasta da tradição.

O Rust tem os operadores aritméticos usuais, +, -, *, / e %. Conforme mencionado no Capítulo 3, o estouro (overflow/underflow) de número inteiro é detectado e gera um pânico nas compilações de depuração. A biblioteca padrão fornece métodos como a.wrapping_add(b) para aritmética não verificada.

A divisão de inteiros é arredondada para zero e a divisão de um número inteiro por zero aciona um pânico, mesmo em compilações em modo release. Os inteiros têm um método a.checked_div(b) que

retorna um `Option` (`None` se `b` for zero) e nunca gera um pânico.

Unário - nega um número. É compatível com todos os tipos numéricos, exceto inteiros sem sinal. Não existe o operador unário `+`.

```
println!("{}", -100); // -100
println!("{}", -100u32); // erro: não é possível aplicar `-` unário ao tipo `u32`
println!("{}", +100); // erro: expressão esperada, encontrado ``
```

Como em C, `a % b` calcula o resto com sinal, ou módulo, do arredondamento da divisão em direção a zero. O resultado tem o mesmo sinal do operando esquerdo. Observe que `%` pode ser utilizado em números de ponto flutuante, bem como inteiros:

```
let x = 1234.567 % 10.0; // aproximadamente 4.567
```

O Rust também herda os operadores de inteiros bit a bit do C, `&`, `|`, `^`, `<<` e `>>`. Contudo, o Rust utiliza `!` em vez de `~` para NOT bit a bit:

```
let hi: u8 = 0xe0;
let lo = !hi; // 0x1f
```

Isso significa que `!n` não pode ser utilizado em um número inteiro `n` para significar “`n` é zero”. Para tanto, escreva `n == 0`.

O deslocamento de bit é sempre estendido por sinal em tipos inteiros com sinal e estendido por zero em tipos inteiros sem sinal. Como o Rust possui inteiros sem sinal, ele não precisa de um operador de deslocamento sem sinal, como o operador `>>>` do Java.

As operações bit a bit têm maior precedência do que as comparações, ao contrário do C; portanto, se você escrever `x & BIT != 0`, isso significa `(x & BIT) != 0`, como você provavelmente pretendia. Isso é muito mais útil do que a interpretação do C, `x & (BIT != 0)`, que testa o bit errado!

Os operadores de comparação do Rust são `==`, `!=`, `<`, `<=`, `>` e `>=`. Os dois valores que estão sendo comparados devem ter o mesmo tipo.

O Rust também tem os dois operadores lógicos de curto-circuito `&&` e `||`. Ambos os operandos devem ter o tipo `bool` exato.

Atribuição

O operador `=` pode ser utilizado para atribuir a variáveis `mut` e seus

campos ou elementos. Mas a atribuição não é tão comum no Rust quanto em outras linguagens, pois as variáveis são imutáveis por padrão.

Conforme descrito no Capítulo 4, se o valor tiver um tipo não-Copy, a atribuição o move para o destino. A posse do valor é transferida da origem para o destino. O valor anterior do destino, se houver, é dropado.

A atribuição composta é suportada:

```
total += item.price;
```

Isso é equivalente a `total = total + item.price;`. Outros operadores também são suportados: `-=`, `*=` e assim por diante. A lista completa é fornecida na Tabela 6.1, anteriormente neste capítulo.

Ao contrário do C, o Rust não suporta atribuição de encadeamento: você não pode escrever `a = b = 3` para atribuir o valor 3 tanto para `a` como para `b`. A atribuição é rara o suficiente no Rust para que você não perca essa abreviação.

O Rust não possui operadores de incremento e decremento do C `++` e `--`.

Coerções de tipo

Converter um valor de um tipo para outro geralmente requer uma conversão explícita no Rust. Coerções utilizam a palavra-chave `as`:

```
let x = 17;           // x é do tipo i32
let index = x as usize; // converte para usize
```

Vários tipos de coerções são permitidos:

- Os números podem ser convertidos de qualquer um dos tipos numéricos integrados para qualquer outro, por coerção.

A coerção de um inteiro para outro tipo inteiro é sempre bem definida. A conversão para um tipo mais restrito resulta em truncamento. Um inteiro com sinal convertido para um tipo mais amplo é estendido por sinal, um inteiro sem sinal é estendido por zero e assim por diante. Resumindo, não há surpresas.

A conversão de um tipo de ponto flutuante para um tipo inteiro arredonda para zero: o valor de `-1.99` `as i32` é `-1`. Se o valor for muito

grande para caber no tipo inteiro, a conversão produzirá o valor mais próximo que o tipo inteiro pode representar: o valor de `1e6` as `u8` é 255.

- Valores do tipo `bool` ou `char`, ou de um tipo `enum` parecido com C, podem ser convertidos em qualquer tipo inteiro. (Abordaremos `enums` no Capítulo 10.)

Fazer a coerção na outra direção não é permitido, pois os tipos `bool`, `char` e `enum` têm restrições sobre seus valores que teriam de ser aplicadas com verificações em tempo de execução. Por exemplo, fazer a coerção de um tipo `u16` em um `char` é proibido porque alguns valores `u16`, como `0xd800`, correspondem a pontos de código substitutos Unicode e, portanto, não assumiria valores `char` válidos. Existe um método padrão, `std::char::from_u32()`, que executa a verificação em tempo de execução e retorna um `Option<char>`; contudo, mais especificamente, a necessidade desse tipo de conversão tornou-se rara. Normalmente, convertemos strings de caracteres ou fluxos inteiros de uma só vez e os algoritmos em texto Unicode geralmente não são triviais e é melhor deixá-los para as bibliotecas.

Como exceção, um `u8` pode ser convertido para um tipo `char`, já que todos os números inteiros de 0 a 255 são pontos de código Unicode válidos para `char` armazenar.

- Algumas conversões envolvendo tipos de ponteiro inseguros também são permitidas. Ver “Ponteiros brutos”, na página 90.

Dissemos que uma conversão geralmente requer uma coerção. Algumas conversões envolvendo tipos de referência são tão diretas que a linguagem as realiza mesmo sem conversão. Um exemplo trivial é a conversão de uma referência `mut` em uma referência `não-mut`.

Várias conversões automáticas mais significativas, porém, podem acontecer:

- Valores do tipo `&String` são automaticamente convertidos para o tipo `&str` sem coerção.
- Valores do tipo `&Vec<i32>` são automaticamente convertidos para `&`

[i32].

- Valores do tipo `&Box<Chessboard>` são automaticamente convertidos para `&Chessboard`.

Estas são chamadas de *coerções deref*, porque elas se aplicam a tipos que implementam o trait `Deref` interno. O propósito da coerção `Deref` é fazer com que tipos de ponteiro inteligentes, como `Box`, comportem-se o mais próximo possível do valor subjacente. Utilizar um `Box<Chessboard>` é basicamente como utilizar um simples `Chessboard`, graças a `Deref`.

Tipos definidos pelo usuário também podem implementar o trait `Deref`. Quando precisar escrever seu próprio tipo de ponteiro inteligente, consulte “`Deref` e `DerefMut`”, na página 362.

Closures

O Rust tem *closures*, valores semelhantes a funções leves. Uma closure geralmente consiste em uma lista de argumentos, fornecida entre barras verticais, seguida por uma expressão:

```
let is_even = |x| x % 2 == 0;
```

O Rust infere os tipos de argumento e o tipo de retorno. Você também pode escrevê-los explicitamente, como faria para uma função. Se você especificar um tipo de retorno, o corpo da closure deve ser um bloco, por uma questão de sanidade sintática:

```
let is_even = |x: u64| -> bool x % 2 == 0; // erro
```

```
let is_even = |x: u64| -> bool { x % 2 == 0 }; // ok
```

Chamar uma closure utiliza a mesma sintaxe que chamar uma função:

```
assert_eq!(is_even(14), true);
```

As closures são um dos recursos mais encantadores do Rust e há muito mais a ser dito sobre elas. Faremos isso no Capítulo 14.

A seguir

Expressões são o que pensamos como “código em execução”. Elas são a parte de um programa Rust que compila as instruções de

máquina. Entretanto, são uma pequena fração de toda a linguagem. O mesmo é verdade na maioria das linguagens de programação. A primeira tarefa de um programa é executar, mas essa não é sua única tarefa. Os programas precisam se comunicar. Eles têm de ser testáveis. Eles precisam se manter organizados e flexíveis para que possam continuar a evoluir. Eles precisam interoperar com códigos e serviços criados por outras equipes. E mesmo apenas para executar, programas em uma linguagem estaticamente tipada como o Rust precisam de mais algumas ferramentas para organizar dados do que apenas tuplas e arrays.

A seguir, passaremos vários capítulos falando sobre recursos nessa área: módulos e crates, que fornecem a estrutura do seu programa e, em seguida, structs e enums, que fazem o mesmo com seus dados.

Primeiro, vamos dedicar algumas páginas ao importante tópico sobre o que fazer quando as coisas dão errado.

¹ N.R.: Rust chama de braço cada par padrão => Expressão. Em C/C++ os braços seriam chamados de casos.

CAPÍTULO 7

Tratamento de erros

Eu sabia que, se ficasse por aí tempo suficiente, algo assim aconteceria.

– George Bernard Shaw sobre a morte

A abordagem do Rust para o tratamento de erros é incomum o suficiente para justificar um capítulo curto sobre o assunto. Não há ideias difíceis aqui, apenas ideias que podem ser novas para você. Este capítulo aborda os dois tipos diferentes de tratamento de erros no Rust: pânico e `Results`.

Erros comuns são tratados utilizando o tipo `Result`. `Results` normalmente representam problemas causados por coisas fora do programa, como entrada incorreta, interrupção de rede ou problema de permissão. Que tais situações ocorram não depende de nós; mesmo um programa livre de bugs os encontrará de tempos em tempos. A maior parte deste capítulo é dedicada a esse tipo de erro. Abordaremos o pânico primeiro, porque é o mais simples dos dois.

O pânico é para o outro tipo de erro, o tipo que *nunca deveria acontecer*.

Pânico

Um programa gera um pânico quando encontra algo tão confuso que deve haver um bug no próprio programa. Algo como:

- Acesso ao array fora dos limites.
- Divisão de inteiro por zero.
- Chamar `.expect()` sobre um `Result` que é um `Err`.
- Falha na asserção.

(Há também a macro `panic!()`, para casos em que seu próprio código descobre que falhou e, portanto, você precisa acionar um pânico

diretamente. `panic!()` aceita argumentos opcionais no estilo `println!()`, para construir uma mensagem de erro.)

O que essas condições têm em comum é que são todas – para não exagerar – culpa do programador. Uma boa regra de ouro é: “Não gerar pânico”.

Mas todos cometemos erros. Quando esses erros que não deveriam acontecer acontecem – o que acontece? Notavelmente, o Rust oferece uma escolha. O Rust pode desempilhar o stack (pilha) quando ocorre um pânico ou abortar o processo. Desempilhar é o padrão.

Desempilhamento

Quando os piratas dividem os objetos saqueados de um ataque, o capitão fica com metade do saque. Os tripulantes comuns ganham partes iguais da outra metade. (Piratas odeiam frações; então, se qualquer uma das divisões não sair igual, o resultado é arredondado para baixo, com o resto indo para o papagaio do navio.)

```
fn pirate_share(total: u64, crew_size: usize) -> u64 {  
    let half = total / 2;  
    half / crew_size as u64  
}
```

Isso pode ter funcionado bem por séculos até que um dia descobrimos que o capitão é o único sobrevivente de um ataque. Se passarmos um `crew_size` de zero a essa função, ele será dividido por zero. Em C++ isso seria um comportamento indefinido. No Rust, desencadeia um pânico, que normalmente ocorre da seguinte maneira:

- Uma mensagem de erro é impressa no terminal:

```
thread 'main' panicked at 'attempt to divide by zero', pirates.rs:3780  
note: Run with `RUST_BACKTRACE=1` for a backtrace.
```

Se você definir a variável de ambiente `RUST_BACKTRACE`, como as mensagens sugerem, o Rust também vai mostrar o stack (pilha) neste ponto.

- O stack (pilha) é desempilhado. Isso é muito parecido com o tratamento de exceções em C++.

Quaisquer valores temporários, variáveis locais ou argumentos que a função atual estava utilizando são dropados, na ordem inversa em que foram criados. Dropar um valor significa simplesmente limpá-lo: qualquer `String` ou `Vec` que o programa estava utilizando são dropados, qualquer arquivo `File` aberto é fechado e assim por diante. Métodos `drop` definidos pelo usuário também são chamados; veja “Drop”, na página 353. No caso particular de `pirate_share()`, não há nada para limpar.

Depois que a chamada de função atual é limpa, passamos para seu chamador, dropando suas variáveis e argumentos da mesma maneira. Então passamos para esse chamador da função e assim por diante no stack (pilha).

- Por fim, o thread sai. Se o thread em pânico for o thread principal, todo o processo será encerrado (com um código de saída diferente de zero).

Talvez pânico seja um nome enganoso para esse processo ordenado. Um pânico não é um acidente. Não é um comportamento indefinido. É mais como um `RuntimeException` em Java ou um `std::logic_error` em C++. O comportamento é bem definido; simplesmente não deveria estar acontecendo.

O pânico é seguro. Não viola nenhuma das regras de segurança do Rust; mesmo se você gerar um pânico no meio de um método de biblioteca padrão, ele nunca deixará um ponteiro perdido ou um valor inicializado pela metade na memória. A ideia é que o Rust pegue o acesso inválido ao array, ou seja lá o que for, *antes de* qualquer coisa ruim acontecer. Seria inseguro continuar, então o Rust desmonta o stack (pilha). Mas o restante do processo pode continuar em execução.

O pânico é por thread. Um thread pode gerar um pânico enquanto outros threads continuam funcionando normalmente. No Capítulo 19, mostraremos como um thread pai pode descobrir quando um thread filho gera um pânico e lidar com o erro normalmente.

Existe também uma maneira de *capturar* o desempilhamento do

stack (pilha), permitindo que o thread sobreviva e continue em execução. A função `std::panic::catch_unwind()` da biblioteca padrão faz isso. Não abordaremos como usá-lo, mas esse é o mecanismo utilizado pelo mecanismo de teste do Rust para se recuperar quando uma asserção falha em um teste. (Também pode ser necessário ao escrever código Rust que possa ser chamado de C ou C++ porque fazer o desempilhamento em código não Rust é um comportamento indefinido; consulte o Capítulo 22.)

Se fosse possível, todos nós teríamos um código livre de bugs que nunca gera um pânico. Mas ninguém é perfeito. Você pode utilizar `threads` e `catch_unwind()` para lidar com o pânico, tornando seu programa mais robusto. Uma ressalva importante é que essas ferramentas capturam apenas pânicos que desempilham o stack (pilha). Nem todo pânico procede dessa maneira.

Abortando

O desempilhamento do stack é o comportamento de pânico padrão, mas há duas circunstâncias nas quais o Rust não tenta desempilhar o stack.

Se um método `.drop()` aciona um segundo pânico enquanto o Rust ainda está tentando limpar após o primeiro, isso é considerado fatal. O Rust para de desempilhar e aborta todo o processo.

Além disso, o comportamento de pânico do Rust é personalizável. Se você compilar com `-C panic=abort`, o *primeiro* pânico em seu programa imediatamente aborta o processo. (Com essa opção, o Rust não precisa saber como desempilhar o stack (pilha), então isso pode reduzir o tamanho do seu código compilado.)

Isso conclui nossa discussão sobre pânico no Rust. Não há muito a dizer, porque o código Rust comum não tem obrigação de lidar com o pânico. Mesmo se você utilizar `threads` ou `catch_unwind()`, todo o seu código de gerenciamento de pânico provavelmente estará concentrado em alguns lugares. Não é razoável esperar que todas as funções de um programa antecipem e lidem com bugs em seu próprio código. Erros causados por outros fatores são outro

problema.

Result

O Rust não tem exceções. Em vez disso, as funções que podem falhar têm um tipo de retorno que diz isto:

```
fn get_weather(location: LatLng) -> Result<WeatherReport, io::Error>
```

O tipo `Result` indica possível falha. Quando chamamos a função `get_weather()`, ela retornará um *resultado de sucesso* `Ok(weather)`, em que `weather` é um novo valor `WeatherReport` ou um *resultado do erro* `Err(error_value)`, em que `error_value` é um `io::Error` explicando o que está errado.

O Rust exige que escrevamos algum tipo de tratamento de erro sempre que chamarmos essa função. Não podemos chegar ao `WeatherReport` sem fazer *algo* com o `Result` e você receberá um aviso do compilador se um valor de `Result` não for utilizado.

No Capítulo 10, veremos como a biblioteca padrão define `Result` e como você pode definir seus próprios tipos semelhantes. Por enquanto, vamos adotar uma abordagem de “livro de receitas” e focar em como utilizar `Results` para obter o comportamento de tratamento de erros que você deseja. Veremos como detectar, propagar e relatar erros, bem como padrões comuns para organizar e trabalhar com tipos `Result`.

Capturando erros

A maneira mais completa de lidar com um `Result` é a maneira que mostramos no Capítulo 2: utilize uma expressão `match`.

```
match get_weather(hometown) {  
    Ok(report) => {  
        display_weather(hometown, &report);  
    }  
    Err(err) => {  
        println!("error querying the weather: {}", err);  
        schedule_weather_retry();  
    }  
}
```

Esse é o equivalente do Rust a `try/catch` em outras linguagens. É o

que você utiliza quando deseja lidar com erros diretamente e não os repassa ao chamador.

`match` é um pouco prolixo, então `Result<T, E>` oferece uma variedade de métodos que são úteis em casos comuns específicos. Cada um desses métodos tem uma expressão `match` na sua execução. (Para a lista completa de métodos `Result`, consulte a documentação on-line. Os métodos listados aqui são os que mais utilizamos.)

`result.is_ok()`, `result.is_err()`

Retorna um `bool` dizendo se `result` é um resultado de sucesso ou um resultado de erro.

`result.ok()`

Retorna o valor de sucesso, se houver, como um `Option<T>`. Se `result` é um resultado de sucesso, isso retorna `Some(success_value)`; caso contrário, retorna `None`, descartando o valor do erro.

`result.err()`

Retorna o valor do erro, se houver, como um `Option<E>`.

`result.unwrap_or(fallback)`

Retorna o valor de sucesso, se `result` é um resultado de sucesso. Caso contrário, retorna `fallback`, descartando o valor do erro.

```
// Uma previsão bastante segura para o sul da Califórnia
const THE_USUAL: WeatherReport = WeatherReport::Sunny(72);
```

```
// Obtém um boletim meteorológico real, se possível
// Se não, recorre ao habitual (fallback)
let report = get_weather(los_angeles).unwrap_or(THE_USUAL);
display_weather(los_angeles, &report);
```

Essa é uma boa alternativa a `.ok()` porque o tipo de retorno é `T`, não `Option<T>`. Naturalmente, funciona apenas quando há um valor de `fallback` apropriado.

`result.unwrap_or_else(fallback_fn)`

Isso é o mesmo, mas em vez de passar um valor de `fallback` diretamente, você passa uma função ou uma closure. Isso é para casos em que seria um desperdício calcular um valor de `fallback` se você não for usá-lo. A `fallback_fn` é chamada apenas se tivermos um resultado de erro.

```
let report =
```

```
get_weather(hometown)
.unwrap_or_else(|_err| vague_prediction(hometown));
```

(O Capítulo 14 aborda as closures em detalhes.)

```
result.unwrap()
```

Também retorna o valor de sucesso, se `result` é um resultado de sucesso. Contudo, se `result` é um resultado de erro, esse método gera um pânico. Esse método tem seus usos; falaremos mais sobre isso depois.

```
result.expect(message)
```

Isso é o mesmo que `.unwrap()`, mas permite fornecer uma mensagem em caso de pânico.

Por fim, métodos para trabalhar com referências em um `Result`:

```
result.as_ref()
```

Converte um `Result<T, E>` em um `Result<&T, &E>`.

```
result.as_mut()
```

Isso é o mesmo, mas empresta uma referência mutável. O tipo de retorno é `Result<&mut T, &mut E>`.

Uma razão pela qual esses dois últimos métodos são úteis é que todos os outros métodos listados aqui, exceto `.is_ok()` e `.is_err()`, *consomem* o `result` sobre o qual eles operam. Ou seja, eles aceitam o argumento `self` por valor. Às vezes é muito útil acessar dados dentro de um `result` sem destruí-lo e é isso que `.as_ref()` e `.as_mut()` faz por nós. Por exemplo, suponha que você gostaria de chamar `result.ok()`, mas precisa que `result` seja deixado intacto. Você pode escrever `result.as_ref().ok()`, que apenas toma emprestado `result`, retornando um `Option<&T>` em vez de um `Option<T>`.

Aliases do tipo Result

Às vezes, você verá a documentação do Rust que parece omitir o tipo de erro de um `Result`:

```
fn remove_file(path: &Path) -> Result<()>
```

Isso significa que um alias do tipo `Result` está sendo utilizado.

Um alias de tipo é uma espécie de atalho para nomes de tipo. Os módulos geralmente definem um alias do tipo `Result` para evitar a

repetição de um tipo de erro que é utilizado consistentemente por quase todas as funções no módulo. Por exemplo, o módulo `std::io` da biblioteca padrão inclui esta linha de código:

```
pub type Result<T> = result::Result<T, Error>;
```

Isso define um tipo público `std::io::Result<T>`. É um alias para `Result<T, E>`, mas codifica diretamente `std::io::Error` como o tipo de erro. Em termos práticos, isso significa que, se você escrever `use std::io;`, então o Rust entenderá `io::Result<String>` como abreviação para `Result<String, io::Error>`.

Quando algo como `Result<>` aparece na documentação on-line, você pode clicar no identificador `Result` para ver qual alias de tipo está sendo utilizado e aprender o tipo de erro. Na prática, geralmente é óbvio pelo contexto.

Imprimindo erros

Às vezes, a única maneira de lidar com um erro é mostrando-o no terminal e seguindo em frente. Já mostramos uma maneira de fazer isso:

```
println!("error querying the weather: {}", err);
```

A biblioteca padrão define vários tipos de erros com nomes enfadonhos: `std::io::Error`, `std::fmt::Error`, `std::str::Utf8Error` e assim por diante. Todos eles implementam uma interface comum, o trait `std::error::Error`, o que significa que eles compartilham os seguintes recursos e métodos:

```
println!()
```

Todos os tipos de erro podem ser impressos utilizando isso. Imprimir um erro com o especificador de formato `{}` normalmente exibe apenas uma breve mensagem de erro. Alternativamente, você pode imprimir com o especificador de formato `{:?}`, para obter uma visualização `Debug` do erro. Isso é menos amigável, mas inclui informações técnicas extras.

```
// resultado de `println!("error: {}", err);`
```

```
error: failed to look up address information: No address associated with  
hostname
```

```
// resultado de `println!("error: {:?}", err);`
```

```
error: Error { repr: Custom(Custom { kind: Other, error: StringError(
"failed to look up address information: No address associated with
hostname") }) }
```

`err.to_string()`

Retorna a mensagem de erro como `String`.

`err.source()`

Retorna uma `Option` do erro subjacente, se houver, que causou o `err`. Por exemplo, um erro de rede pode fazer com que uma transação bancária falhe, o que, por sua vez, pode fazer com que você perca o barco que acabou de comprar. Se `err.to_string()` é "boat was repossessed", então `err.source()` pode retornar um erro sobre a transação malsucedida. Esse erro é `.to_string()` pode ser "failed to transfer \$300 to United Yacht Supply" e sua `.source()` pode ser um `io::Error` com detalhes sobre a interrupção de rede específica que causou toda a confusão. Esse terceiro erro é a causa raiz, então seu método `.source()` retornaria `None`. Como a biblioteca padrão inclui apenas recursos de baixo nível, a origem dos erros retornados da biblioteca padrão geralmente é `None`.

A impressão de um valor de erro também não imprime sua origem. Se você quiser ter certeza de imprimir todas as informações disponíveis, utilize esta função:

```
use std::error::Error;
use std::io::{Write, stderr};

/// Escreve uma mensagem de erro em `stderr`
/// Se outro erro ocorrer durante a criação da mensagem de erro
/// ou a escrita em `stderr`, ele é ignorado
fn print_error(mut err: &dyn Error) {
    let _ = writeln!(stderr(), "error: {}", err);
    while let Some(source) = err.source() {
        let _ = writeln!(stderr(), "caused by: {}", source);
        err = source;
    }
}
```

A macro `writeln!` funciona como `println!`, exceto que escreve os dados em um fluxo de sua escolha. Aqui, escrevemos as mensagens de erro no fluxo de erro padrão, `std::io::stderr`. Poderíamos utilizar a macro `eprintln!` para fazer a mesma coisa, mas `eprintln!` gera um pânico se ocorrer um erro. No `print_error`, queremos ignorar os erros que surgem

ao escrever a mensagem; explicamos porque em “Ignorando erros”, na página 201, mais adiante no capítulo.

Os tipos de erro da biblioteca padrão não incluem um rastreamento do stack (pilha), mas o popular crate `anyhow` fornece um tipo de erro pronto, quando utilizado com uma versão instável do compilador Rust. (No Rust 1.50, as funções da biblioteca padrão para capturar backtraces ainda não estavam estabilizadas.)

Propagando erros

Na maioria dos lugares onde tentamos algo que pode falhar, não queremos detectar e tratar o erro imediatamente. É simplesmente muito código para utilizar uma instrução `match` de 10 linhas em todos os lugares onde algo pode dar errado.

Em vez disso, se ocorrer um erro, geralmente queremos deixar nosso chamador lidar com isso. Queremos que os erros *se propaguem* no call stack (pilha de chamadas).

O Rust tem um operador `?` que faz isso. Você pode adicionar um `?` a qualquer expressão que produza um `Result`, como o resultado de uma chamada de função:

```
let weather = get_weather(hometown)?;
```

O comportamento de `?` depende se essa função retornar um resultado de sucesso ou um resultado de erro:

- Com sucesso, ele desencapsula o `Result` para obter o valor de sucesso dentro. O tipo de `weather` aqui não é `Result<WeatherReport, io::Error>` mas simplesmente `WeatherReport`.
- Em caso de erro, ele retorna imediatamente da função que o envolve, passando o resultado do erro para cima na cadeia de chamadas. Para garantir que isso funcione, `?` só pode ser utilizado em um `Result` em funções que possuem um tipo `Result` de retorno.

Não há nada de mágico no operador `?`. Você pode expressar a mesma coisa utilizando uma expressão `match`, embora seja muito mais prolixo:

```
let weather = match get_weather(hometown) {  
    Ok(success_value) => success_value,
```

```
Err(err) => return Err(err)
};
```

As únicas diferenças entre isso e o operador `?` são alguns pontos delicados envolvendo tipos e conversões. Abordaremos esses detalhes na próxima seção.

Em código mais antigo, você pode ver a macro `try!()`, que era a maneira usual de propagar erros até que o operador `?` foi introduzido no Rust 1.13:

```
let weather = try!(get_weather(hometown));
```

A macro se expande para uma expressão `match`, como a anterior.

É fácil esquecer o quão difundida é a possibilidade de erros em um programa, especialmente no código que faz interface com o sistema operacional. Às vezes, o operador `?` aparece em quase todas as linhas de uma função:

```
use std::fs;
use std::io;
use std::path::Path;
fn move_all(src: &Path, dst: &Path) -> io::Result<()> {
    for entry_result in src.read_dir()? { // a abertura do diretório pode falhar
        let entry = entry_result?;      // a leitura do diretório pode falhar
        let dst_file = dst.join(entry.file_name());
        fs::rename(entry.path(), dst_file)?; // a renomeação pode falhar
    }
    Ok(()) // ufa!
}
```

`?` também funciona de forma semelhante com o tipo `Option`. Em uma função que retorna `Option`, você pode utilizar `?` para desencapsular um valor ou retornar mais cedo no caso de `None`:

```
let weather = get_weather(hometown).ok()?;
```

Trabalhando com vários tipos de erro

Muitas vezes, mais de uma coisa pode dar errado. Suponha que estamos simplesmente lendo números de um arquivo de texto:

```
use std::io::{self, BufRead};

/// Lê números inteiros de um arquivo de texto
/// O arquivo deve ter um número em cada linha
fn read_numbers(file: &mut dyn BufRead) -> Result<Vec<i64>, io::Error> {
    let mut numbers = vec![];
```

```

    for line_result in file.lines() {
        let line = line_result?;    // a leitura das linhas pode falhar
        numbers.push(line.parse()?); // a análise de números inteiros pode falhar
    }
    Ok(numbers)
}

```

O Rust nos dá um erro de compilação:

error: ``?` couldn't convert the error to `std::io::Error`

```

    numbers.push(line.parse()?);    // a análise de números inteiros pode falhar
                        ^
    the trait `std::convert::From<std::num::ParseIntError>`
    is not implemented for `std::io::Error`
note: the question mark operation (`?`) implicitly performs a conversion
on the error value using the `From` trait

```

Os termos nesta mensagem de erro farão mais sentido quando chegarmos ao Capítulo 11, que aborda traits. Por enquanto, apenas observe que o Rust está reclamando que o operador `?` não pode converter um valor `std::num::ParseIntError` para o tipo `std::io::Error`.

O problema aqui é que a leitura de uma linha de um arquivo e a análise de um inteiro produzem dois tipos diferentes de erros em potencial. O tipo de `line_result` é `Result<String, std::io::Error>`. O tipo de `line.parse()` é `Result<i64, std::num::ParseIntError>`. O tipo de retorno da nossa função `read_numbers()` acomoda apenas `io::Errors`. O Rust tenta lidar com o `ParseIntError` convertendo-o em um `io::Error`, mas não há tal conversão, então obtemos um erro de tipo.

Existem várias maneiras de lidar com isso. Por exemplo, o crate `image` que utilizamos no Capítulo 2 para criar arquivos de imagem do conjunto de Mandelbrot define seu próprio tipo de erro, `ImageError`, e implementa conversões de `io::Error` e vários outros tipos de erro para `ImageError`. Se você gostaria de ir por esse caminho, tente o crate `thiserror`, que é projetado para ajudá-lo a definir bons tipos de erro com apenas algumas linhas de código.

Uma abordagem mais simples é utilizar o que está embutido no Rust. Todos os tipos de erro da biblioteca padrão podem ser convertidos para o tipo `Box<dyn std::error::Error + Send + Sync + 'static>`. Isso é um bocado de código, mas `dyn std::error::Error` representa “qualquer erro” e `Send + Sync + 'static` torna segura a passagem entre threads, o

que muitas vezes você desejará.¹ Por conveniência, você pode definir aliases de tipo:

```
type GenericError = Box<dyn std::error::Error + Send + Sync + 'static>;  
type GenericResult<T> = Result<T, GenericError>;
```

Em seguida, altere o tipo de retorno de `read_numbers()` para `GenericResult<Vec<i64>>`. Com essa alteração, a função compila. O operador `?` converte automaticamente qualquer tipo de erro em um `GenericError` conforme necessário.

Aliás, o operador `?` faz essa conversão automática utilizando um método padrão que você mesmo pode utilizar. Para converter qualquer erro para o tipo `GenericError`, chame `GenericError::from()`:

```
let io_error = io::Error::new(      // cria o nosso próprio io::Error  
    io::ErrorKind::Other, "timed out");  
return Err(GenericError::from(io_error)); // converte manualmente para GenericError
```

Vamos abordar o trait `From` e seu método `from()` extensamente no Capítulo 13.

A desvantagem da abordagem `GenericError` é que o tipo de retorno não comunica mais com precisão quais tipos de erros o chamador pode esperar. O chamador deve estar pronto para qualquer coisa.

Se você estiver chamando uma função que retorna um `GenericResult` e quer lidar com um tipo específico de erro, mas deixar todos os outros se propagarem, utilize o método genérico `error.downcast_ref::<ErrorType>()`. Ele toma emprestada uma referência ao erro e se acontecer de ser o tipo específico de erro que você está procurando:

```
loop {  
    match compile_project() {  
        Ok(()) => return Ok(()),  
        Err(err) => {  
            if let Some(mse) = err.downcast_ref::<MissingSemicolonError>() {  
                insert_semicolon_in_source_code(mse.file(), mse.line());  
                continue; // tenta novamente!  
            }  
            return Err(err);  
        }  
    }  
}
```

Muitas linguagens têm sintaxe interna para fazer isso, mas raramente é necessário. O Rust tem um método para isso.

Lidando com erros que “não podem acontecer”

Às vezes nós apenas *sabemos* que um erro não pode acontecer. Por exemplo, suponha que estamos escrevendo um código para analisar um arquivo de configuração e, a certa altura, descobrimos que a próxima coisa no arquivo é uma string de dígitos:

```
if next_char.is_digit(10) {
    let start = current_index;
    current_index = skip_digits(&line, current_index);
    let digits = &line[start..current_index];
    ...
}
```

Queremos converter essa string de dígitos em um número real. Existe um método padrão que faz isso:

```
let num = digits.parse::();
```

Agora o problema: o método `str.parse::u64()` não retorna um `u64`. Ele retorna um `Result`. Isso pode falhar, porque algumas strings não são numéricas:

```
"bleen".parse::<u64>() // ParseIntError: dígito inválido
```

Mas sabemos que, neste caso, `digits` consiste inteiramente de dígitos. O que deveríamos fazer?

Se o código que estamos escrevendo já retorna um `GenericResult`, podemos acrescentar um `?` e esquecê-lo. Caso contrário, enfrentaremos a perspectiva irritante de ter de escrever um código de tratamento de erros para um erro que não pode acontecer. A melhor escolha então seria utilizar `.unwrap()`, um método `Result` que gera um pânico se o resultado for um `Err`, mas simplesmente retorna o valor de sucesso de um `Ok`:

```
let num = digits.parse::().unwrap();
```

Isso é como ? exceto que, se estivermos errados sobre esse erro, se *puder* acontecer, então, nesse caso, resultaria em um pânico.

Na verdade, estamos errados sobre esse caso particular. Se a entrada contiver uma string de dígitos longa o suficiente, o número será muito grande para caber em um `u64`:

```
"999999999999999999".parse::<u64>() // erro de estouro
```

Utilizar `.unwrap()` neste caso particular seria, portanto, um bug. A entrada falsa não deve causar pânico.

Dito isso, surgem situações em que um valor `Result` realmente não pode ser um erro. Por exemplo, no Capítulo 18, você verá que o `trait Write` define um conjunto comum de métodos (`.write()` e outros) para saída de texto e saída binária. Todos esses métodos retornam `io::ResultS`, mas, se você estiver gravando em um `Vec<u8>`, eles não podem falhar. Nesses casos, é aceitável utilizar `.unwrap()` ou `.expect(message)` para dispensar os `ResultS`.

Esses métodos também são úteis quando um erro indica uma condição tão grave ou bizarra que um pânico é exatamente como você deseja lidar com isso:

```
fn print_file_age(filename: &Path, last_modified: SystemTime) {  
    let age = last_modified.elapsed().expect("system clock drift");  
    ...  
}
```

Aqui o método `.elapsed()` pode falhar apenas se a hora do sistema for *anterior* à da criação do arquivo. Isso pode acontecer se o arquivo foi criado recentemente e o relógio do sistema foi ajustado para trás enquanto nosso programa estava em execução. Dependendo de como esse código é utilizado, é uma decisão razoável gerar um pânico nesse caso, em vez de lidar com o erro ou propagá-lo ao chamador.

Ignorando erros

Ocasionalmente, queremos apenas ignorar um erro completamente. Por exemplo, em nossa função `print_error()`, tivemos de lidar com a situação improvável em que imprimir o erro aciona outro erro. Isso pode acontecer, por exemplo, se `stderr` é redirecionado para outro processo e esse processo está morto. O erro original que estávamos tentando relatar é provavelmente mais importante para propagarem então queremos apenas ignorar os problemas com `stderr`, mas o compilador Rust avisa sobre valores `Result`:

```
writeln!(stderr(), "error: {}", err); // aviso: resultado não utilizado
```

A expressão `let _ = ...` é utilizada para silenciar esse aviso:

```
let _ = writeln!(stderr(), "error: {}", err); // ok, ignore o resultado
```

Tratando erros em main()

Na maioria dos lugares onde um `Result` é retornado, deixar o erro borbulhar para o chamador é o comportamento correto. Esse é o motivo pelo qual `?` é um caractere único no Rust. Como vimos, em alguns programas isso é utilizado em muitas linhas de código seguidas.

Mas, se você propagar um erro por tempo suficiente, ao final ele atingirá `main()` e algo terá de ser feito com isso. Normalmente, `main()` não pode utilizar `?` porque seu tipo de retorno não é `Result`:

```
fn main() {  
    calculate_tides()?; // erro: não pode passar mais a responsabilidade  
}
```

A maneira mais simples de lidar com erros em `main()` é utilizar `.expect()`:

```
fn main() {  
    calculate_tides().expect("error"); // a bola para aqui  
}
```

Se `calculate_tides()` retorna um resultado de erro, o método `.expect()` gera um pânico. Um pânico gerado no thread principal imprime uma mensagem de erro e sai com um código de saída diferente de zero, que é aproximadamente o comportamento desejado. Utilizamos isso o tempo todo para pequenos programas. É um começo.

A mensagem de erro é um pouco intimidante:

```
$ tidecalc --planet mercury  
thread 'main' panicked at 'error: "moon not found"', src/main.rs:2:23  
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

A mensagem de erro se perde no ruído. Além disso, `RUST_BACKTRACE=1` é um mau conselho neste caso particular.

Mas você também pode alterar a assinatura de tipo de `main()` para retornar um tipo `Result`, então pode utilizar `?`:

```
fn main() -> Result<(), TideCalcError> {  
    let tides = calculate_tides()?;  
    print_tides(tides);  
    Ok(())  
}
```

Isso funciona para qualquer tipo de erro que possa ser impresso com o formatador `{:?}`, do qual todos os tipos de erro padrão, como `std::io::Error`, podem ser. Essa técnica é fácil de utilizar e fornece uma mensagem de erro um pouco mais agradável, mas não é a ideal:

```
$ tidecalc --planet mercury
Error: TideCalcError { error_type: NoMoon, message: "moon not found" }
```

Se você tiver tipos de erro mais complexos ou quiser incluir mais detalhes em sua mensagem, vale a pena imprimir você mesmo a mensagem de erro:

```
fn main() {
    if let Err(err) = calculate_tides() {
        print_error(&err);
        std::process::exit(1);
    }
}
```

Esse código utiliza uma expressão `if let` para imprimir a mensagem de erro somente se a chamada para `calculate_tides()` retornar um resultado de erro. Para detalhes sobre `if let` expressões, ver Capítulo 10. A função `print_error` está listada em “Imprimindo erros”, na página 194.

Agora a saída está elegante e organizada:

```
$ tidecalc --planet mercury
error: moon not found
```

Declarando um tipo de erro personalizado

Suponha que você esteja escrevendo um novo analisador JSON e queira que ele tenha seu próprio tipo de erro. (Ainda não cobrimos os tipos definidos pelo usuário; isso será abordado em alguns capítulos. Mas os tipos de erro são úteis, então vamos incluir uma prévia aqui.)

Aproximadamente o código mínimo que você escreveria seria:

```
// json/src/error.rs

#[derive(Debug, Clone)]
pub struct JsonError {
    pub message: String,
    pub line: usize,
    pub column: usize,
}
```

Esse struct será chamado de `json::error::JsonError` e, quando quiser gerar um erro desse tipo, você pode escrever:

```
return Err(JsonError {
    message: "expected ']' at end of array".to_string(),
    line: current_line,
    column: current_column
});
```

Isso vai funcionar bem. No entanto, se deseja que seu tipo de erro funcione como os tipos de erro padrão, como os usuários de sua biblioteca esperam, então você tem um pouco mais de trabalho a fazer:

```
use std::fmt;

// Os erros devem ser imprimíveis
impl fmt::Display for JsonError {
    fn fmt(&self, f: &mut fmt::Formatter) -> Result<(), fmt::Error> {
        write!(f, "{} ({}:{})", self.message, self.line, self.column)
    }
}

// Erros devem implementar o trait std::error::Error,
// mas as definições padrão para os métodos de erro estão ok
impl std::error::Error for JsonError { }
```

Mais uma vez, o significado da palavra-chave `impl`, `self` e todo o resto será explicado nos próximos capítulos.

Como em muitos aspectos da linguagem Rust, os crates existem para tornar o tratamento de erros muito mais fácil e conciso. Existem muitas variedades, mas uma das mais utilizadas é `thiserror`, que faz todo o trabalho anterior para você, permitindo que você escreva erros como este:

```
use thiserror::Error;
#[derive(Error, Debug)]
#[error("{message:} ({line:}, {column:})")]
pub struct JsonError {
    message: String,
    line: usize,
    column: usize,
}
```

A diretiva `#[derive(Error)]` diz `thiserror` para gerar o código mostrado anteriormente, o que pode economizar muito tempo e esforço.

Por que Results?

Agora sabemos o suficiente para entender o que o Rust quer dizer ao preferir `Results` a exceções. Eis os pontos-chave do projeto:

- O Rust exige que o programador tome algum tipo de decisão e a registre no código, em cada ponto onde um erro pode ocorrer. Isso é bom porque, caso contrário, é fácil lidar com erros de maneira incorreta por negligência.
- A decisão mais comum é permitir que os erros se propaguem, e isso é escrito com um único caractere, `?`. Portanto, o redirecionamento de erros não sobrecarrega seu código como acontece em C e Go. Contudo, ainda é visível: você pode olhar para um fragmento de código e ver rapidamente todos os lugares onde os erros são propagados.
- Como a possibilidade de erros faz parte do tipo de retorno de cada função, fica claro quais funções podem falhar e quais não podem. Se você alterar uma função para ser falível, estará alterando seu tipo de retorno, portanto o compilador fará com que você atualize os usuários dessa função.
- O Rust verifica que valores `Result` são utilizados, então você não pode acidentalmente deixar um erro passar silenciosamente (um engano comum em C).
- Como `Result` é um tipo de dados como qualquer outro, é fácil armazenar resultados de sucesso e erro na mesma coleção. Isso facilita a modelagem de sucesso parcial. Por exemplo, se você está escrevendo um programa que carrega milhões de registros de um arquivo de texto e precisa de uma maneira de lidar com o resultado provável de que a maioria terá sucesso, e que alguns falharão, você pode representar essa situação na memória utilizando um vetor de `Results`.

O custo é que você se verá pensando e lidando com erros de engenharia mais no Rust do que em outras linguagens. Como em muitas outras áreas, a abordagem do Rust no tratamento de erros é um pouco mais rígida do que você está acostumado. Para programação de sistemas, vale a pena.

- [1](#) Você também deve considerar o uso do popular crate anyhow, que fornece tipos de erro e resultado muito parecidos com o nosso GenericError e GenericResult, mas com alguns recursos adicionais interessantes.

CAPÍTULO 8

Crates e módulos

*Essa é uma nota em um tema do Rust:
programadores de sistemas podem ter coisas boas.*

*– Robert O’Callahan, Random Thoughts on Rust: crates.io and IDEs”
(<https://oreil.ly/Y22sV>)*

Suponha que você esteja escrevendo um programa que simule o crescimento de samambaias, a partir do nível de células individuais. Seu programa, como uma samambaia, começará muito simples, com todo o código, talvez, em um único arquivo – apenas o esporo de uma ideia. À medida que cresce, ele começa a ter uma estrutura interna. Diferentes partes terão diferentes finalidades. Ele se ramificará em vários arquivos. Pode abranger toda uma árvore de diretórios. Com o tempo, pode se tornar uma parte significativa de todo um ecossistema de software. Para qualquer programa que cresça além de algumas estruturas de dados ou algumas centenas de linhas, alguma organização é necessária.

Este capítulo aborda os recursos do Rust que ajudam a manter seu programa organizado: crates e módulos. Também abordaremos outros tópicos relacionados à estrutura e distribuição de um crate Rust, incluindo como documentar e testar o código Rust, como silenciar avisos indesejados do compilador, como utilizar Cargo para gerenciar dependências e versões do projeto, como publicar bibliotecas de código aberto no repositório público de crates do Rust, crates.io, como o Rust evolui por meio das edições da linguagem, e muito mais, utilizando nosso simulador de crescimento de uma samambaia como exemplo.

Crates

Os programas do Rust são feitos de *crates*. Cada crate é uma

unidade completa e coesa: todo o código-fonte para uma única biblioteca ou executável, além de quaisquer testes associados, exemplos, ferramentas, configuração e outras tranqueiras. Para seu simulador de samambaia, você pode utilizar bibliotecas de terceiros para gráficos 3D, bioinformática, computação paralela e assim por diante. Essas bibliotecas são distribuídas como crates (consulte a Figura 8.1).

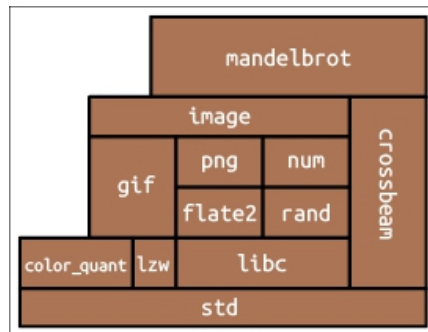


Figura 8.1: Um crate e suas dependências.

A maneira mais fácil de ver o que são os crates e como funcionam juntos é utilizar `cargo build` com o flag `--verbose` para construir um projeto existente que tenha algumas dependências. Fizemos isso utilizando “Um programa de Mandelbrot concorrente”, na página 56, como nosso exemplo. Os resultados são mostrados aqui:

```
$ cd mandelbrot
$ cargo clean # excluir código compilado anteriormente
$ cargo build --verbose
  Updating registry `https://github.com/rust-lang/crates.io-index`
  Downloading autocfg v1.0.0
  Downloading semver-parser v0.7.0
  Downloading gif v0.9.0
  Downloading png v0.7.0
  ... (downloading and compiling many more crates)

  Compiling jpeg-decoder v0.1.18
    Running `rustc
      --crate-name jpeg_decoder
      --crate-type lib
      ...
      --extern byteorder=.../libbyteorder-29efdd0b59c6f920.rmeta
      ...
    Compiling image v0.13.0
```

```

Running `rustc
  --crate-name image
  --crate-type lib
  ...
  --extern byteorder=.../libbyteorder-29efdd0b59c6f920.rmeta
  --extern gif=.../libgif-a7006d35f1b58927.rmeta
  --extern jpeg_decoder=.../libjpeg_decoder-5c10558d0d57d300.rmeta
Compiling mandelbrot v0.1.0 (/tmp/rustbook-test-files/mandelbrot)
Running `rustc
  --edition=2021
  --crate-name mandelbrot
  --crate-type bin
  ...
  --extern crossbeam=.../libcrossbeam-f87b4b3d3284acc2.rlib
  --extern image=.../libimage-b5737c12bd641c43.rlib
  --extern num=.../libnum-1974e9a1dc582ba7.rlib -C link-arg=-fuse-ld=lld`
Finished dev [unoptimized + debuginfo] target(s) in 16.94s
$

```

Reformatamos as linhas do comando `rustc` para facilitar a leitura e excluímos várias opções do compilador que não são relevantes para nossa discussão, substituindo-as por reticências (...).

Você deve se lembrar de que, quando terminamos, a função *main.rs* do programa Mandelbrot continha várias declarações `use` para itens de outros crates:

```

use num::Complex;
// ...
use image::ColorType;
use image::png::PNGEncoder;

```

Também especificamos em nosso arquivo *Cargo.toml* qual versão de cada crate queríamos:

```

[dependencies]
num = "0.4"
image = "0.13"
crossbeam = "0.8"

```

A palavra *dependencies* (dependências) aqui significa apenas outros crates que esse projeto utiliza: código do qual dependemos. Encontramos esses crates em crates.io (<https://crates.io>), o site da comunidade Rust para crates de código aberto. Por exemplo, descobrimos a biblioteca de imagens acessando *crates.io* e procurando por uma biblioteca `image`. A página de cada crate em *crates.io* mostra seu arquivo *README.md* e links para documentação

e fonte, bem como uma linha de configuração como `image = "0.13"` que você pode copiar e adicionar ao seu *Cargo.toml*. Os números de versão mostrados aqui são simplesmente as versões mais recentes desses três pacotes, no momento em que escrevíamos o programa.

A transcrição do Cargo conta a história de como essas informações são utilizadas. Quando rodamos `cargo build`, Cargo começa baixando o código-fonte para as versões especificadas desses crates de crates.io. Então, ele lê os arquivos *Cargo.toml* desses crates, baixa suas dependências e assim por diante recursivamente. Por exemplo, o código-fonte da versão 0.13.0 do crate `image` contém um arquivo *Cargo.tom* que inclui isto:

```
[dependencies]
byteorder = "1.0.0"
num-iter = "0.1.32"
num-rational = "0.1.32"
num-traits = "0.1.32"
enum_primitive = "0.1.0"
```

Vendo isso, Cargo sabe que, antes de utilizar `image`, ele também deve buscar esses crates. Posteriormente, veremos como dizer ao Cargo que busque o código-fonte de um repositório Git ou do sistema de arquivos local em vez de crates.io.

Como `mandelbrot` depende desses crates indiretamente, devido ao uso do crate `image`, nós os chamamos de dependências *transitivas* de `mandelbrot`. A coleção de todos esses relacionamentos de dependência, que informa ao Cargo tudo o que ele precisa saber sobre quais crates construir e em que ordem, é conhecida como *grafo de dependências* do crate. A manipulação automática do grafo de dependências e das dependências transitivas por Cargo é uma grande vitória em termos de tempo e esforço do programador.

Assim que tiver o código-fonte, Cargo compila todos os crates, e executa `rustc`, o compilador do Rust, uma vez para cada crate no grafo de dependências do projeto. Ao compilar bibliotecas, Cargo utiliza a opção `--crate-type lib`. Isso diz para `rustc` não procurar uma função `main()`, mas, em vez disso, produzir um arquivo *.rlib* contendo código compilado que pode ser utilizado para criar arquivos binários e outros arquivos *.rlib*.

Ao compilar um programa, Cargo utiliza `--crate-type bin` e o resultado é um executável binário para a plataforma de destino: *mandelbrot.exe* no Windows, por exemplo.

Com cada comando `rustc`, Cargo passa opções `--extern`, fornecendo o nome de arquivo de cada biblioteca que o crate utilizará. Dessa forma, quando `rustc` vê uma linha de código como `use image::png::PNGEncoder`, ele pode descobrir que `image` é o nome de outro crate e, graças ao Cargo, ele sabe onde encontrar esse crate compilado no disco. O compilador do Rust precisa de acesso a esses arquivos *.rlib* porque eles contêm o código compilado da biblioteca. O Rust vinculará (link) estaticamente esse código ao executável final. A *.rlib* também contém informações de tipo para que o Rust possa verificar se os recursos da biblioteca que estamos utilizando em nosso código realmente existem no crate e se os estamos utilizando corretamente. Ele também contém uma cópia das funções inline públicas do crate, generics e macros, recursos que não podem ser totalmente compilados para código de máquina até que o Rust veja como os utilizamos.

`cargo build` oferece suporte a todos os tipos de opções, a maioria das quais está além do escopo deste livro, mas mencionaremos uma aqui: `cargo build --release` produz uma versão otimizada. As versões em modo `release` são executadas mais rapidamente, mas demoram mais para compilar, não verificam o estouro de número inteiro, pulam `debug_assert!()` asserções, e os rastreamentos de `stack` (pilha) que geram em caso de pânico são geralmente menos confiáveis.

Edições

O Rust tem garantias de compatibilidade extremamente fortes. Qualquer código compilado no Rust 1.0 deve compilar tão bem no Rust 1.50 ou, se for lançado, no Rust 1.900.

Mas, às vezes, há propostas convincentes de extensões para a linguagem que fariam com que o código antigo não fosse mais compilado. Por exemplo, depois de muita discussão, o Rust estabeleceu uma sintaxe para suporte de programação assíncrona

que reaproveita os identificadores `async` e `await` como palavras-chave (ver Capítulo 20). Mas essa mudança de linguagem quebraria qualquer código existente que utilize `async` ou `await` como o nome de uma variável.

Para evoluir sem quebrar o código existente, o Rust utiliza *edições*. A edição 2015 do Rust é compatível com o Rust 1.0. A edição de 2018 transformou `async` e `await` em palavras-chave e simplificou o sistema de módulos, enquanto a edição de 2021 melhorou a ergonomia do array e disponibilizou algumas definições de biblioteca amplamente utilizadas em todos os lugares por padrão. Todas essas foram melhorias importantes para a linguagem, mas quebrariam o código existente. Para evitar isso, cada crate indica em qual edição do Rust esse crate foi escrito com uma linha como esta na seção `[package]` na parte superior do seu arquivo *Cargo.toml*:

```
edition = "2021"
```

Se essa palavra-chave estiver ausente, a edição de 2015 será assumida, de modo que os crates antigos não precisem ser alterados. Mas, se quiser utilizar funções assíncronas ou o novo sistema de módulos, você precisará de `edition = "2018"` ou posterior em seu arquivo *Cargo.toml*.

O Rust promete que o compilador sempre aceitará todas as edições existentes da linguagem e os programas podem misturar livremente crates escritos em diferentes edições. É até bom que um crate da edição de 2015 dependa de um crate da edição de 2021. Em outras palavras, a edição de um crate afeta apenas como seu código-fonte é interpretado; distinções de edição desaparecem no momento em que o código é compilado. Isso significa que não há pressão para atualizar crates antigos apenas para continuar a participar do ecossistema Rust moderno. Da mesma forma, não há pressão para manter seu crate em uma edição mais antiga para evitar inconvenientes para seus usuários. Você só precisa alterar as edições quando quiser utilizar novos recursos de linguagem em seu próprio código.

As edições não saem todos os anos, apenas quando o projeto Rust decide que é necessário. Por exemplo, não há edição de 2020.

Configurar `edition` como "2020" causa um erro. O Rust Edition Guide (<https://oreil.ly/bKEO7>) cobre as mudanças introduzidas em cada edição e fornece um bom histórico sobre o sistema de edição.

Quase sempre é uma boa ideia utilizar a edição mais recente, especialmente para novos códigos. `cargo new` cria novos projetos na última edição por padrão. Este livro utiliza a edição de 2021 em todo o livro.

Se você tiver um crate escrito em uma edição mais antiga do Rust, o comando `cargo fix` pode ajudá-lo a atualizar automaticamente seu código para a edição mais recente. O Rust Edition Guide explica o comando `cargo fix` em detalhes.

Perfis de compilação

Existem várias definições de configuração que você pode colocar em seu arquivo *Cargo.toml* que afetam as linhas de comando `rustc` que `cargo` gera (Tabela 8.1).

Tabela 8.1: Seções de definição de configuração do Cargo.toml

Linha de comando	Seção Cargo.toml utilizada
<code>cargo build</code>	<code>[profile.dev]</code>
<code>cargo build --release</code>	<code>[profile.release]</code>
<code>cargo test</code>	<code>[profile.test]</code>

Os padrões geralmente são bons, mas encontramos uma exceção quando você deseja utilizar um profiler – uma ferramenta que mede onde seu programa está gastando seu tempo de CPU. Para obter os melhores dados de um profiler, você precisa de otimizações (geralmente habilitadas apenas em compilações em modo `release`) e símbolos de depuração (geralmente habilitados apenas em compilações em modo de depuração/debug). Para habilitar ambos, adicione isto ao seu *Cargo.toml*:

```
[profile.release]
```

```
debug = true # habilita símbolos de depuração em compilações de release
```

A configuração `debug` controla a opção `-g` para `rustc`. Com essa configuração, ao digitar `cargo build --release`, você obterá um binário com

símbolos de depuração (debug). As configurações de otimização não são afetadas.

A documentação do Cargo (<https://oreil.ly/mTNiN>) lista muitas outras configurações que você pode ajustar em *Cargo.toml*.

Módulos

Considerando que os crates são sobre compartilhamento de código entre projetos, *módulos* são sobre organização de código *dentro de* um projeto. Eles atuam como namespaces do Rust, contêineres para as funções, tipos, constantes etc. que compõem seu programa ou biblioteca Rust. Um módulo se parece com isto:

```
mod spores {
    use cells::{Cell, Gene};

    /// Uma célula gerada por uma samambaia adulta. Ela se dispersa com
    /// o vento como parte do ciclo de vida da samambaia. Um esporo cresce
    /// em um prótalo -- um organismo completamente separado, com até 5 mm
    /// de diâmetro -- que produz o zigoto que se desenvolve em uma nova
    /// samambaia. (O sexo das plantas é complicado.)
    pub struct Spore {
        ...
    }
    /// Simula a produção de um esporo por meiose
    pub fn produce_spore(factory: &mut Sporangium) -> Spore {
        ...
    }
    /// Extrai os genes em um determinado esporo
    pub(crate) fn genes(spore: &Spore) -> Vec<Gene> {
        ...
    }
    /// Mistura genes para se preparar para a meiose (parte da interfase)
    fn recombine(parent: &mut Cell) {
        ...
    }
    ...
}
```

Um módulo é uma coleção de *itens*, características nomeadas como o struct `Spore` e as duas funções neste exemplo. A palavra-chave `pub` torna um item público, para que possa ser acessado de fora do módulo.

Uma função é marcada `pub(crate)`, o que significa que está disponível em qualquer lugar dentro desse `crate`, mas não é exposta como parte da interface externa. Ela não pode ser utilizada por outros `crates` e não aparecerá na documentação desse `crate`.

Qualquer coisa que não esteja marcada como `pub` é privada e só pode ser utilizada no mesmo módulo em que está definida ou em qualquer módulo filho:

```
let s = spores::produce_spore(&mut factory); // ok
```

```
spores::recombine(&mut cell); // erro: `recombine` é privado
```

Marcar um item como `pub` é muitas vezes conhecido como “exportar” esse item.

O restante desta seção cobre os detalhes que você precisa saber para fazer uso total dos módulos:

- Mostramos como aninhar módulos e distribuí-los em diferentes arquivos e diretórios, se necessário.
- Explicamos a sintaxe de `path` que o Rust utiliza para referenciar itens de outros módulos e mostramos como importar itens para que você possa usá-los sem ter de escrever seus caminhos completos.
- Abordamos o controle refinado do Rust para campos `struct`.
- Apresentamos os módulos de *prelúdio*, que reduzem o código que você precisaria escrever reunindo importações comuns que quase qualquer usuário usará.
- Apresentamos as definições de *constantes* e *variáveis estáticas*, duas maneiras de definir valores nomeados, para clareza e consistência.

Módulos aninhados

Os módulos podem ser aninhados e é bastante comum ver um módulo que é apenas uma coleção de submódulos:

```
mod plant_structures {  
    pub mod roots {  
        ...  
    }  
    pub mod stems {
```

```

    ...
}
pub mod leaves {
    ...
}
}

```

Se você deseja que um item em um módulo aninhado fique visível para outros crates, certifique-se de marcá-lo, *e também todos os módulos envolventes*, como públicos. Caso contrário, você pode ver um aviso como este:

```

warning: function is never used: `is_square`
|
23 | /      pub fn is_square(root: &Root) -> bool {
24 | |      root.cross_section_shape().is_square()
25 | |      }
   | |_____^
   |

```

Talvez essa função realmente seja um código morto no momento. Mas, se você pretendia utilizá-la em outros crates, o Rust está informando que ela não é realmente visível para eles. Você deve certificar-se de que seus módulos envolventes sejam todos marcados com `pub` também.

Também é possível especificar `pub(super)`, tornando um item visível apenas para o módulo pai e `pub(in <path>)`, o que o torna visível em um módulo pai específico e seus descendentes. Isso é especialmente útil com módulos profundamente aninhados:

```

mod plant_structures {
    pub mod roots {
        pub mod products {
            pub(in crate::plant_structures::roots) struct Cytokinin {
                ...
            }
        }
    }

    use products::Cytokinin; // ok: no módulo `roots`
}

use roots::products::Cytokinin; // erro: `Cytokinin` é privado
}

// erro: `Cytokinin` é privado
use plant_structures::roots::products::Cytokinin;

```

Dessa forma, poderíamos escrever um programa inteiro, com uma enorme quantidade de código e toda uma hierarquia de módulos, relacionados da maneira que quiséssemos, tudo em um único arquivo-fonte.

De fato, trabalhar dessa maneira é complicado, então há uma alternativa.

Módulos em arquivos separados

Um módulo também pode ser escrito assim:

```
mod spores;
```

Anteriormente, incluímos o corpo do módulo `spores`, entre chaves. Aqui, estamos dizendo ao compilador do Rust que o módulo `spores` reside em um arquivo separado, chamado *spores.rs*:

```
// spores.rs

/// Uma célula produzida por uma samambaia adulta...
pub struct Spore {
    ...
}
/// Simula a produção de um esporo por meiose
pub fn produce_spore(factory: &mut Sporangium) -> Spore {
    ...
}
/// Extrai os genes em um determinado esporo
pub(crate) fn genes(spore: &Spore) -> Vec<Gene> {
    ...
}
/// Mistura genes para se preparar para a meiose (parte da interfase)
fn recombine(parent: &mut Cell) {
    ...
}
```

spores.rs contém apenas os itens que compõem o módulo. Esse arquivo não precisa de nenhum tipo de código para declarar que é um módulo.

A localização do código é a *única* diferença entre esse módulo `spores` e a versão que mostramos na seção anterior. As regras sobre o que é público e o que é privado são exatamente as mesmas de qualquer maneira. E o Rust nunca compila os módulos separadamente, mesmo que estejam em arquivos separados: quando você cria um

crate do Rust, está recompilando todos os seus módulos.

Um módulo pode ter o próprio diretório. Quando Rust vê `mod spores;`, ele procura os dois arquivos, *spores.rs* e *spores/mod.rs*; se nenhum arquivo existir, ou ambos existirem, isso é um erro. Para esse exemplo, utilizamos *spores.rs*, porque o módulo `spores` não tinha nenhum submódulo. Mas considere o módulo `plant_structures` que escrevemos anteriormente. Se decidirmos dividir esse módulo e seus três submódulos em seus próprios arquivos, o projeto resultante ficaria assim:

```
fern_sim/
├── Cargo.toml
└── src/
    ├── main.rs
    ├── spores.rs
    └── plant_structures/
        ├── mod.rs
        ├── leaves.rs
        ├── roots.rs
        └── stems.rs
```

Em *main.rs*, declaramos o módulo `plant_structures`:

```
pub mod plant_structures;
```

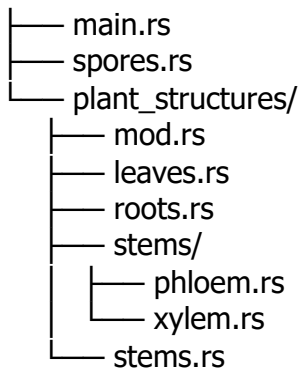
Isso faz com que o Rust carregue *plant_structs/mod.rs*, que declara os três submódulos:

```
// em plant_structures/mod.rs
pub mod roots;
pub mod stems;
pub mod leaves;
```

O conteúdo desses três módulos é armazenado em arquivos separados chamados *leaves.rs*, *roots.rs*, e *stems.rs*, armazenados com *mod.rs* no diretório *plant_structures*.

Também é possível utilizar um arquivo e um diretório com o mesmo nome para compor um módulo. Por exemplo, se `stems` precisasse incluir os módulos chamados `xylem` e `phloem`, poderíamos optar por manter `stems` em *plant_structures/stems.rs* e adicionar um diretório `stems`:

```
fern_sim/
├── Cargo.toml
└── src/
```



Então, em *stems.rs*, declaramos os dois novos submódulos:

```
// em plant_structures/stems.rs
pub mod xylem;
pub mod phloem;
```

Essas três opções – módulos em seu próprio arquivo, módulos em seu próprio diretório com um *mod.rs* e módulos em seu próprio arquivo com um diretório suplementar contendo submódulos – dão ao sistema de módulos flexibilidade suficiente para suportar quase qualquer estrutura de projeto que você desejar.

Caminhos e importações

O operador `::` é utilizado para acessar os recursos de um módulo. O código em qualquer lugar do seu projeto pode referenciar qualquer recurso de biblioteca padrão escrevendo seu caminho:

```
if s1 > s2 {
    std::mem::swap(&mut s1, &mut s2);
}
```

`std` é o nome da biblioteca padrão. O caminho `std` referencia o módulo de nível superior da biblioteca padrão. `std::mem` é um submódulo dentro da biblioteca padrão e `std::mem::swap` é uma função pública nesse módulo.

Você poderia escrever todo o seu código dessa maneira, digitando `std::f64::consts::PI` e `std::collections::HashMap::new` toda vez que quisesse um círculo ou um dicionário, mas isso seria tedioso e difícil de ler. A alternativa é *importar* recursos nos módulos em que são utilizados:

```
use std::mem;

if s1 > s2 {
    mem::swap(&mut s1, &mut s2);
}
```

```
}
```

A declaração `use` faz com que o nome `mem` seja um alias local para `std::mem` ao longo do bloco ou módulo envolvente.

Poderíamos escrever `use std::mem::swap;` para importar a função `swap` em si em vez do módulo `mem`. Mas o que fizemos anteriormente geralmente é considerado o melhor estilo: importar tipos, traits e módulos (como `std::mem`) e, em seguida, utilizar caminhos relativos para acessar as funções, constantes e outros membros dentro deles.

Vários nomes podem ser importados de uma só vez:

```
use std::collections::{HashMap, HashSet}; // importa ambos
use std::fs::{self, File}; // importa `std::fs` e `std::fs::File`
use std::io::prelude::*; // importa tudo
```

Esse é apenas um atalho para escrever todas as importações individuais:

```
use std::collections::HashMap;
use std::collections::HashSet;

use std::fs;
use std::fs::File;

// todos os itens públicos em std::io::prelude:
use std::io::prelude::Read;
use std::io::prelude::Write;
use std::io::prelude::BufRead;
use std::io::prelude::Seek;
```

Você pode utilizar `as` para importar um item, mas dar a ele um nome diferente localmente:

```
use std::io::Result as IOResult;

// Esse tipo de retorno é apenas outra maneira de escrever `std::io::Result<()>`:
fn save_spore(spore: &Spore) -> IOResult<()>
...

```

Módulos *não* herdam automaticamente nomes de seus módulos pais. Por exemplo, suponha que temos isto em nosso *proteins/mod.rs*:

```
// proteins/mod.rs
pub enum AminoAcid { ... }
pub mod synthesis;
```

Então o código em *synthesis.rs* não vê automaticamente o tipo `AminoAcid`:

```
// proteins/synthesis.rs
pub fn synthesize(seq: &[AminoAcid]) // erro: não foi possível encontrar o tipo
    `AminoAcid`
...

```

Em vez disso, cada módulo começa com uma lousa em branco e deve importar os nomes que utiliza:

```
// proteins/synthesis.rs
use super::AminoAcid; // importa explicitamente do pai

pub fn synthesize(seq: &[AminoAcid]) // ok
...

```

Por padrão, os caminhos são relativos ao módulo atual:

```
// em proteins/mod.rs

// importa de um submódulo
use synthesis::synthesize;

```

`self` também é um sinônimo para o módulo atual, então podemos escrever:

```
// em proteins/mod.rs

// importa nomes de uma enumeração,
// então podemos escrever `Lys` para lisina, ao invés de `AminoAcid::Lys`
use self::AminoAcid::*;

```

ou simplesmente:

```
// em proteins/mod.rs
use AminoAcid::*;

```

(O exemplo `AminoAcid` aqui é, obviamente, um desvio da regra de estilo que mencionamos anteriormente sobre importar apenas tipos, traits e módulos. Se nosso programa incluir longas sequências de aminoácidos, isso é justificado pela Sexta Regra de Orwell: “Quebre qualquer uma dessas regras antes de dizer qualquer coisa completamente bárbara.”)

As palavras-chave `super` e `crate` têm um significado especial em caminhos: `super` referencia o módulo pai e `crate` referencia o crate que contém o módulo atual.

O uso de paths relativos à raiz do crate em vez do módulo atual facilita a movimentação do código no projeto, pois todas as importações não serão interrompidas se o caminho do módulo atual for alterado. Por exemplo, poderíamos escrever *synthesis.rs*

utilizando crate:

```
// proteins/synthesis.rs
use crate::proteins::AminoAcid; // importa explicitamente em relação à raiz do crate

pub fn synthesize(seq: &[AminoAcid]) // ok
    ...
```

Os submódulos podem acessar itens privados em seus módulos pais com `use super::*`.

Se você tiver um módulo com o mesmo nome de um crate que está utilizando, então referenciar seu conteúdo requer algum cuidado. Por exemplo, se o seu programa listar o crate `image` como uma dependência em seu arquivo *Cargo.toml*, mas também tem um módulo chamado `image`, então caminhos começando com `image` são ambíguos:

```
mod image {
    pub struct Sampler {
        ...
    }
}
// erro: Isso referencia nosso módulo `image` ou o crate `image`?
use image::Pixels;
```

Mesmo que o módulo `image` não tenha o tipo `Pixels`, a ambiguidade ainda é considerada um erro: seria confuso se, ao adicionar uma definição assim mais tarde, pudesse mudar silenciosamente os caminhos referenciados em outras partes do programa.

Para resolver a ambiguidade, o Rust tem um tipo especial de path chamado path *absoluto*, começando com `::`, que sempre referencia um crate externo. Para referenciar o tipo `Pixels` no crate `image`, você pode escrever:

```
use ::image::Pixels; // os `Pixels` do crate `image`
```

Para referenciar o tipo `Sampler` do seu próprio módulo, você pode escrever:

```
use self::image::Sampler; // o `Sampler` do módulo `image`
```

Os módulos não são a mesma coisa que os arquivos, mas há uma analogia natural entre os módulos e os arquivos e diretórios de um sistema de arquivos Unix. A palavra-chave `use` cria aliases, assim como o comando `ln` cria links. Caminhos, como nomes de arquivos,

vêm em formas absolutas e relativas. `self` e `super` são como `o` e `..` diretórios especiais.

Prelúdio padrão

Dissemos há pouco que cada módulo começa com uma “folha em branco”, no que diz respeito aos nomes importados. Mas a lousa não é *completamente* em branco.

Por um lado, a biblioteca padrão `std` é vinculada automaticamente a cada projeto. Isso significa que você sempre pode ir com `use std::whatever` ou consultar `std` itens por nome, como `std::mem::swap()` inline no seu código. Além disso, alguns nomes particularmente úteis, como `Vec` e `Result`, estão incluídos no *prelúdio padrão* e são importados automaticamente. O Rust se comporta como se cada módulo, incluindo o módulo raiz, começasse com a seguinte importação:

```
use std::prelude::v1::*;
```

O prelúdio padrão contém algumas dezenas de traits e tipos comumente utilizados.

No Capítulo 2, mencionamos que às vezes as bibliotecas fornecem módulos chamados `prelude`. Mas `std::prelude::v1` é o único prelúdio que é importado automaticamente. Nomear um módulo `prelude` é apenas uma convenção que informa aos usuários que ele deve ser importado utilizando `*`.

Criando declarações `use` públicas

Ainda que declarações `use` sejam apenas aliases, elas podem ser públicas:

```
// em plant_structures/mod.rs
...
pub use self::leaves::Leaf;
pub use self::roots::Root;
```

Isso significa que `Leaf` e `Root` são itens públicos do módulo `plant_structures`. Eles ainda são aliases simples para `plant_structures::leaves::Leaf` e `plant_structures::roots::Root`.

O prelúdio padrão é escrito exatamente como uma série de `pub`

importações.

Criando campos struct públicos

Um módulo pode incluir tipos struct definidos pelo usuário, que são introduzidos utilizando a palavra-chave `struct`. Abordamos isso em detalhes no Capítulo 9, mas este é um bom ponto para mencionar como os módulos interagem com a visibilidade dos campos struct.

Um struct simples se parece com isto:

```
pub struct Fern {  
    pub roots: RootSet,  
    pub stems: StemSet  
}
```

Os campos de um struct, mesmo campos privados, são acessíveis em todo o módulo onde o struct é declarado e seus submódulos. Fora do módulo, apenas os campos públicos são acessíveis.

Ocorre que impor o controle de acesso por módulo, em vez de por classe como em Java ou C++, é surpreendentemente útil para o design de software. Ele reduz os métodos padronizados “getter” e “setter” e elimina em grande parte a necessidade de qualquer coisa como declarações `friend` do C++. Um único módulo pode definir vários tipos que trabalham juntos, como talvez `frond::LeafMap` e `frond::LeafMapIter`, acessando os campos privados uns dos outros conforme necessário, enquanto ainda oculta esses detalhes de implementação do resto do seu programa.

Variáveis estáticas e constantes

Além de funções, tipos e módulos aninhados, os módulos também podem definir *constantes* e variáveis *estáticas*.

A palavra-chave `const` introduz uma constante. A sintaxe é como `let` exceto que pode ser marcada como `pub` e o tipo é obrigatório. Além disso, `UPPERCASE_NAMES` são convencionais para constantes:

```
pub const ROOM_TEMPERATURE: f64 = 20.0; // graus Celsius
```

A palavra-chave `static` introduz um item estático, que é quase a mesma coisa:

```
pub static ROOM_TEMPERATURE: f64 = 68.0; // graus Fahrenheit
```

Uma constante é um pouco parecida com um `#define` do C++: o valor é compilado em seu código em todos os lugares em que é utilizado. Uma variável estática é uma variável que é configurada antes que seu programa comece a rodar e dura até que ele termine. Use constantes para números mágicos e strings em seu código. Use variáveis estáticas para quantidades maiores de dados ou sempre que precisar pedir uma referência emprestada ao valor constante.

Não há constantes `mut`. Variáveis estáticas podem ser marcadas como `mut`, mas, conforme discutido no Capítulo 5, o Rust não tem como impor suas regras sobre acesso exclusivo em variáveis estáticas `mut`. Elas são, portanto, inerentemente não thread-safe e um código seguro não pode utilizá-las:

```
static mut PACKETS_SERVED: usize = 0;
```

```
println!("{}", served, PACKETS_SERVED); // erro: uso de variável estática mutável
```

O Rust desencoraja o estado mutável global. Para uma discussão sobre as alternativas, ver “Variáveis globais”, na página 606.

Transformando um programa em uma biblioteca

À medida que seu simulador de samambaia começa a decolar, você decide que precisa de mais do que um único programa. Suponha que você tenha um programa de linha de comando que execute a simulação e salve os resultados em um arquivo. Agora, você deseja escrever outros programas para realizar análises científicas dos resultados salvos, exibindo renderizações 3D das plantas em crescimento em tempo real, renderizando imagens fotorrealistas e assim por diante. Todos esses programas precisam compartilhar o código básico da simulação de samambaia. Você precisa criar uma biblioteca.

A primeira etapa é fatorar seu projeto existente em duas partes: um crate de biblioteca, que contém todo o código compartilhado, e um executável, que contém o código necessário apenas para seu programa de linha de comando existente.

Para mostrar como você pode fazer isso, vamos utilizar um programa de exemplo grosseiramente simplificado:

```
struct Fern {
    size: f64,
    growth_rate: f64
}

impl Fern {
    /// Simula uma samambaia crescendo por um dia
    fn grow(&mut self) {
        self.size *= 1.0 + self.growth_rate;
    }
}

/// Executa uma simulação de samambaia por alguns dias
fn run_simulation(fern: &mut Fern, days: usize) {
    for _ in 0 .. days {
        fern.grow();
    }
}

fn main() {
    let mut fern = Fern {
        size: 1.0,
        growth_rate: 0.001
    };
    run_simulation(&mut fern, 1000);
    println!("final fern size: {}", fern.size);
}
```

Assumiremos que esse programa tem um arquivo *Cargo.toml* trivial:

```
[package]
name = "fern_sim"
version = "0.1.0"
authors = ["You <you@example.com>"]
edition = "2021"
```

Transformar esse programa em uma biblioteca é fácil. Eis os passos:

1. Renomeie o arquivo *src/main.rs* para *src/lib.rs*.
2. Adicione a palavra-chave `pub` para itens em *src/lib.rs* que serão recursos públicos de nossa biblioteca.
3. Mova a função `main` para um arquivo temporário em algum lugar. Voltaremos a isso em um minuto.

O arquivo *src/lib.rs* resultante fica assim:

```
pub struct Fern {
```

```

    pub size: f64,
    pub growth_rate: f64
}

impl Fern {
    /// Simula uma samambaia crescendo por um dia
    pub fn grow(&mut self) {
        self.size *= 1.0 + self.growth_rate;
    }
}

/// Executa uma simulação da samambaia por alguns dias
pub fn run_simulation(fern: &mut Fern, days: usize) {
    for _ in 0 .. days {
        fern.grow();
    }
}

```

Observe que não precisamos alterar nada em *Cargo.toml*. Isso ocorre porque nosso mínimo arquivo *Cargo.toml* deixa Cargo com seu comportamento padrão. Por padrão, `cargo build` examina os arquivos em nosso diretório de origem e descobre o que construir. Quando ele vê o arquivo *src/lib.rs*, sabe construir uma biblioteca.

O código em *src/lib.rs* forma o *módulo raiz* da biblioteca. Outros crates que utilizam nossa biblioteca só podem acessar os itens públicos desse módulo raiz.

Diretório *src/bin*

Fazer o programa original de linha de comando `fern_sim` funcionar novamente também é simples: Cargo tem algum suporte integrado para pequenos programas que vivem no mesmo crate de uma biblioteca.

Na verdade, o próprio Cargo é escrito dessa maneira. A maior parte do código está em uma biblioteca Rust. O programa de linha de comando `cargo` que utilizamos ao longo deste livro é um programa encapsulador simples que chama a biblioteca para todo o trabalho pesado. Tanto a biblioteca quanto o programa de linha de comando residem no mesmo repositório de origem (<https://oreil.ly/aJKOk>).

Também podemos manter nosso programa e nossa biblioteca no mesmo crate. Coloque esse código em um arquivo chamado

src/bin/efern.rs:

```
use fern_sim::{Fern, run_simulation};

fn main() {
    let mut fern = Fern {
        size: 1.0,
        growth_rate: 0.001
    };
    run_simulation(&mut fern, 1000);
    println!("final fern size: {}", fern.size);
}
```

A função `main` é aquela que separamos anteriormente. Acrescentamos uma declaração `use` de alguns itens do crate `fern_sim`, `Fern` e `run_simulation`. Em outras palavras, estamos utilizando esse crate como uma biblioteca.

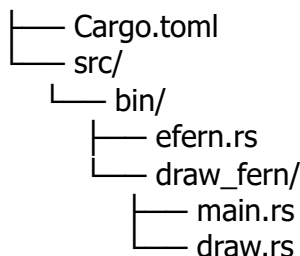
Como colocamos esse arquivo em *src/bin*, Cargo compilará a biblioteca `fern_sim` e esse programa na próxima vez que executarmos `cargo build`. Podemos executar o programa `efern` utilizando `cargo run --bin efern`. Eis como fica o código, utilizando `--verbose` para mostrar os comandos que Cargo está executando:

```
$ cargo build --verbose
Compiling fern_sim v0.1.0 (file:///.../fern_sim)
Running `rustc src/lib.rs --crate-name fern_sim --crate-type lib ...`
Running `rustc src/bin/efern.rs --crate-name efern --crate-type bin ...`
$ cargo run --bin efern --verbose
Fresh fern_sim v0.1.0 (file:///.../fern_sim)
Running `target/debug/efern`
final fern size: 2.7169239322355985
```

Ainda não tivemos que fazer nenhuma alteração em *Cargo.toml*, porque, novamente, o padrão do Cargo é examinar seus arquivos de origem e descobrir as coisas. Ele automaticamente trata arquivos *.rs* em *src/bin* como programas extras para compilar.

Também podemos compilar programas maiores no diretório *src/bin* utilizando subdiretórios. Suponha que desejamos fornecer um segundo programa que desenha uma samambaia na tela, mas o código de desenho é grande e modular, portanto tem seu próprio arquivo. Podemos dar ao segundo programa seu próprio subdiretório:

`fern_sim/`



Isso tem a vantagem de permitir que binários maiores tenham seus próprios submódulos sem sobrecarregar o código da biblioteca ou o diretório *src/bin*.

Naturalmente, agora que *fern_sim* é uma biblioteca, também temos outra opção. Poderíamos ter colocado esse programa em seu próprio projeto isolado, em um diretório completamente separado, com sua própria listagem de *Cargo.toml* *fern_sim* como uma dependência:

```
[dependencies]
fern_sim = { path = "../fern_sim" }
```

Talvez seja isso que você fará para outros programas de simulação de samambaias no futuro. O diretório *src/bin* é ideal para programas simples como *efern* e *draw_fern*.

Atributos

Qualquer item em um programa Rust pode ser decorado com *atributos*. Atributos são a sintaxe genérica do Rust para escrever diversas instruções e avisos para o compilador. Por exemplo, suponha que você esteja recebendo este aviso:

```
libgit2.rs: warning: type `git_revspec` should have a camel case name
such as `GitRevspec`, #[warn(non_camel_case_types)] on by default
```

Mas você escolheu esse nome por um motivo e gostaria que o Rust parasse de falar sobre isso. Você pode desativar o aviso adicionando um atributo `#[allow]` no tipo:

```
#[allow(non_camel_case_types)]
pub struct git_revspec {
    ...
}
```

A compilação condicional é outro recurso escrito utilizando um atributo, ou seja, `#[cfg]`:

```
// Só inclua esse módulo no projeto se estivermos compilando para Android
```

```
#[cfg(target_os = "android")]
mod mobile;
```

A sintaxe completa de `#[cfg]` é especificada no Rust Reference (<https://oreil.ly/F7ggB>); as opções mais utilizadas estão listadas na Tabela 8.2.

Tabela 8.2: Opções de `#[cfg]` mais comumente utilizadas

Opção de <code>#[cfg(...)]</code>	Habilitada quando
Test	Os testes estão ativados (compilando com <code>cargo test</code> ou <code>rustc --test</code>).
debug_assertions	As asserções de depuração estão habilitadas (normalmente em compilações não otimizadas).
Unix	Compilando para Unix, incluindo macOS.
Windows	Compilando para Windows.
target_pointer_width = "64"	Visando a uma plataforma de 64 bits. O outro valor possível é "32".
target_arch = "x86_64"	Segmentação x86-64 em particular. Outros valores: "x86", "arm", "aarch64", "powerpc", "powerpc64", "mips".
target_os = "macos"	Compilando para macOS. Outros valores: "windows", "ios", "android", "linux", "freebsd", "openbsd", "netbsd", "dragonfly".
feature = "robots"	O recurso definido pelo usuário chamado "robots" está ativado (compilando com <code>cargo build --feature robots</code> ou <code>rustc --cfg feature=""robots"</code>). As características são declaradas na Seção [features] de <i>Cargo.toml</i> (https://oreil.ly/IfEpj).
not(A)	A não é satisfeita. Para fornecer duas implementações diferentes de uma função, marque uma delas com <code>#[cfg(X)]</code> e a outra com <code>#[cfg(not(X))]</code> .
all(A,B)	Ambas, <i>A</i> e <i>B</i> , são satisfeitas (o equivalente a <code>&&</code>).
any(A,B)	Qualquer <i>A</i> ou <i>B</i> é satisfeita (o equivalente a <code> </code>).

Ocasionalmente, precisamos microgerenciar a expansão inline das funções, uma otimização que normalmente deixamos para o compilador. Podemos utilizar o atributo `#[inline]` para isto:

```
/// Ajusta níveis de íons etc. em duas células adjacentes devido à osmose entre elas
#[inline]
fn do_osmosis(c1: &mut Cell, c2: &mut Cell) {
    ...
}
```

Há uma situação em que o uso `inline` não acontecerá sem `#[inline]`. Quando uma função ou método definido em um `crate` é chamado

em outro crate, o Rust não o colocará inline, a menos que seja genérico (tenha parâmetros de tipo) ou seja explicitamente marcado como `#[inline]`.

Caso contrário, o compilador trata `#[inline]` como uma sugestão. O Rust também suporta os mais insistentes `#[inline(always)]`, para solicitar que uma função seja expandida em linha em cada local de chamada e `#[inline(never)]`, para solicitar que uma função nunca seja colocada inline.

Alguns atributos, como `#[cfg]` e `#[allow]`, podem ser anexados a um módulo inteiro e aplicados a tudo nele. Outros, como `#[test]` e `#[inline]`, devem ser anexados a itens individuais. Como você pode esperar de um recurso que faz tudo, cada atributo é feito sob medida e tem o próprio conjunto de argumentos suportados. A Rust Reference documenta o conjunto completo de atributos suportados (<https://oreil.ly/FtJWN>) em detalhe.

Para anexar um atributo a um crate inteiro, acrescente-o na parte superior do arquivo *main.rs* ou *lib.rs*, antes de qualquer item, e escreva `#!` em vez de `#`, como aqui:

```
// libgit2_sys/lib.rs
#![allow(non_camel_case_types)]

pub struct git_revspec {
    ...
}

pub struct git_error {
    ...
}
```

O `#!` diz ao Rust que anexe um atributo ao item que o envolve, em vez do que vem a seguir: neste caso, o atributo `#![allow]` se anexa a todo o crate `libgit2_sys`, não apenas a `struct git_revspec`.

`#!` também pode ser utilizado dentro de funções, structs e assim por diante, mas normalmente é utilizado apenas no início de um arquivo, para anexar um atributo a todo o módulo ou crate. Alguns atributos sempre utilizam a sintaxe `#!` porque só podem ser aplicados a um crate inteiro.

Por exemplo, o atributo `#![feature]` é utilizado para ativar recursos

instáveis da linguagem e bibliotecas do Rust, recursos que são experimentais e, portanto, podem ter bugs ou podem ser alterados ou removidos no futuro. Por exemplo, enquanto escrevemos isso, o Rust tem suporte experimental para rastrear a expansão de macros como `assert!`, mas como esse suporte é experimental, você só pode usá-lo (1) instalando a versão nightly do Rust e (2) declarando explicitamente que seu crate utiliza rastreamento de macro:

```
#![feature(trace_macros)]

fn main() {
    // Eu me pergunto qual código do Rust real
    // substituiria este uso de assert_eq!
    trace_macros!(true);
    assert_eq!(10*10*10 + 9*9*9, 12*12*12 + 1*1*1);
    trace_macros!(false);
}
```

Com o tempo, a equipe do Rust às vezes *estabiliza* um recurso experimental para que se torne uma parte padrão da linguagem. O atributo `#![feature]` então se torna supérfluo e o Rust gera um aviso aconselhando você a removê-lo.

Testes e documentação

Como vimos em “Escrevendo e executando testes unitários”, na página 30, uma estrutura de testes unitários simples é incorporada ao Rust. Testes são funções comuns marcadas com o atributo `#[test]`:

```
#[test]
fn math_works() {
    let x: i32 = 1;
    assert!(x.is_positive());
    assert_eq!(x + 1, 2);
}
```

`cargo test` executa todos os testes em seu projeto:

```
$ cargo test
Compiling math_test v0.1.0 (file:///.../math_test)
Running target/release/math_test-e31ed91ae51ebf22

running 1 test
test math_works ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

(Você também verá alguns resultados sobre “doc-tests”, que abordaremos em um minuto.)

Isso funciona da mesma forma se seu crate for um executável ou uma biblioteca. Você pode executar testes específicos passando argumentos para Cargo: `cargo test math` executa todos os testes que contêm `math` em algum lugar em seu nome.

Os testes geralmente utilizam as macros `assert!` e `assert_eq!` da biblioteca padrão do Rust. `assert!(expr)` será bem-sucedido se `expr` for verdade. Caso contrário, ele gera um pânico, o que faz com que o teste falhe. `assert_eq!(v1, v2)` é como `assert!(v1 == v2)` exceto que, se a asserção falhar, a mensagem de erro mostrará ambos os valores.

Você pode utilizar essas macros em código comum para verificar invariantes, mas observe que `assert!` e `assert_eq!` estão incluídos mesmo em compilações de release. Utilize `debug_assert!` e `debug_assert_eq!` em vez disso para escrever asserções que são verificadas apenas em compilações de depuração.

Para testar casos de erro, adicione o atributo `#[should_panic]` ao seu teste:

```
/// Esse teste passa somente se a divisão por zero causar pânico,  
/// como afirmamos no capítulo anterior  
#[test]  
#[allow(unconditional_panic, unused_must_use)]  
#[should_panic(expected="divide by zero")]  
fn test_divide_by_zero_error() {  
    1 / 0; // deve dar pânico!  
}
```

Neste caso, também precisamos adicionar um atributo `allow` para dizer ao compilador que nos deixe fazer coisas que ele pode provar estaticamente que criarão pânico, e fazer divisões e simplesmente jogar fora a resposta, porque, normalmente, ele tenta impedir esse tipo de tolice.

Você também pode devolver um `Result<(), E>` de seus testes. Desde que a variante de erro seja `Debug`, o que geralmente é o caso, você pode simplesmente retornar um `Result` utilizando `?` para jogar fora a variante `Ok`:

```
use std::num::ParseIntError;
```

```

/// O código passará por este teste se "1024" for um número válido, o que ele é
#[test]
fn explicit_radix() -> Result<(), ParseIntError> {
    i32::from_str_radix("1024", 10)?;
    Ok(())
}

```

Funções marcadas com `#[test]` são compiladas condicionalmente. Um `cargo build` simples ou `cargo build --release` ignora o código de teste. Mas, quando você roda `cargo test`, Cargo constrói seu programa duas vezes: uma vez da maneira normal e uma vez com seus testes e o mecanismo de teste ativado. Isso significa que seus testes unitários podem ficar juntos com o código que eles testam, acessando detalhes internos da implementação, se necessário, e ainda sem custo de tempo de execução. Mas alguns avisos podem aparecer. Por exemplo:

```

fn roughly_equal(a: f64, b: f64) -> bool {
    (a - b).abs() < 1e-6
}

#[test]
fn trig_works() {
    use std::f64::consts::PI;
    assert!(roughly_equal(PI.sin(), 0.0));
}

```

Em compilações que omitem o código de teste, `roughly_equal` parece não utilizado e o Rust reclamará:

```

$ cargo build
Compiling math_test v0.1.0 (file:///.../math_test)
warning: function is never used: `roughly_equal`
|
7 | / fn roughly_equal(a: f64, b: f64) -> bool {
8 | |     (a - b).abs() < 1e-6
9 | | }
  | |_^
  |
= note: #[warn(dead_code)] on by default

```

Portanto, a convenção, quando seus testes se tornam substanciais o suficiente para exigir código de suporte, é colocá-los em um módulo `tests` e declarar o módulo inteiro como somente teste utilizando o atributo `#[cfg]`:

```

#[cfg(test)] // inclua este módulo apenas ao testar

```

```

mod tests {
    fn roughly_equal(a: f64, b: f64) -> bool {
        (a - b).abs() < 1e-6
    }

    #[test]
    fn trig_works() {
        use std::f64::consts::PI;
        assert!(roughly_equal(PI.sin(), 0.0));
    }
}

```

O mecanismo de teste do Rust utiliza vários threads para executar vários testes ao mesmo tempo, um bom benefício colateral de seu código Rust ser thread-safe por padrão. Para desativar isso, execute um único teste, `cargo test testname`, ou execute `cargo test -- --test-threads 1`. (O primeiro `--` assegura que `cargo test` passa a opção `--test-threads` até o executável de teste.) Isso significa que, tecnicamente, o programa Mandelbrot que mostramos no Capítulo 2 não era o segundo programa multithread daquele capítulo, mas o terceiro! A execução de `cargo test` em “Escrevendo e executando testes unitários”, na página 30, foi o primeiro.

Normalmente, o mecanismo de teste mostra apenas a saída dos testes que falharam. Para mostrar a saída dos testes que passaram no teste, execute:

```
cargo test -- --no-capture.
```

Testes de integração

Seu simulador de samambaia continua a crescer. Você decidiu colocar toda a funcionalidade principal em uma biblioteca que pode ser utilizada por vários executáveis. Seria bom ter alguns testes vinculados à biblioteca da mesma forma que um usuário final faria, utilizando *fern_sim.rlib* como crate externo. Além disso, você tem alguns testes que começam carregando uma simulação salva de um arquivo binário e é estranho ter esses arquivos de teste grandes em seu diretório *src*. Os testes de integração ajudam com esses dois problemas.

Os testes de integração são arquivos *.rs* que vivem em um diretório

de testes no mesmo diretório *src* do seu projeto. Quando você executa `cargo test`, Cargo compila cada teste de integração como um crate independente e separado, vinculado à sua biblioteca e ao mecanismo de teste Rust. Eis um exemplo:

```
// tests/unfurl.rs - os brotos da samambaia se desenrolam à luz do sol

use fern_sim::Terrarium;
use std::time::Duration;

#[test]
fn test_fiddlehead_unfurling() {
    let mut world = Terrarium::load("tests/unfurl_files/fiddlehead.tm");
    assert!(world.fern(0).is_furled());
    let one_hour = Duration::from_secs(60 * 60);
    world.apply_sunlight(one_hour);
    assert!(world.fern(0).is_fully_unfurled());
}
```

Os testes de integração são valiosos em parte porque eles veem seu crate de fora, assim como um usuário faria. Eles testam a API pública do crate.

`cargo test` executa testes unitários e testes de integração. Para executar apenas os testes de integração em um determinado arquivo – por exemplo, *tests/unfurl.rs* –, utilize o comando `cargo test --test unfurl`.

Documentação

O comando `cargo doc` cria documentação HTML para sua biblioteca:

```
$ cargo doc --no-deps --open
Documenting fern_sim v0.1.0 (file:///.../fern_sim)
```

A opção `--no-deps` diz ao Cargo que gere documentação apenas para o próprio *fern_sim*, e não para todos os crates de que depende.

A opção `--open` diz ao Cargo que abra a documentação em seu navegador posteriormente.

Você pode ver o resultado na Figura 8.2. Cargo salva os novos arquivos de documentação em *target/doc*. A página inicial é *target/doc/fern_sim/index.html*.

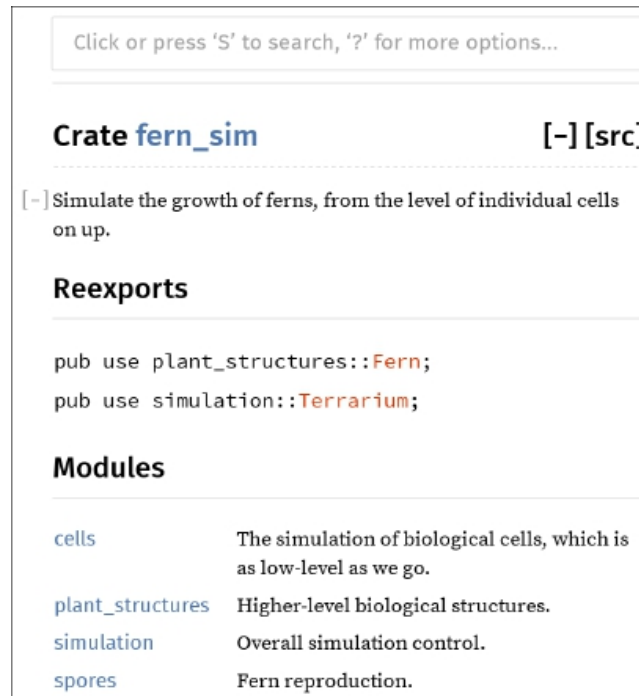


Figura 8.2: Exemplo de documentação gerada por rustdoc.

A documentação é gerada a partir dos recursos públicos de sua biblioteca, além de qualquer *comentários do documento* que você anexou a eles. Já vimos alguns comentários de documentos neste capítulo. Parecem comentários:

```
/// Simula a produção de um esporo por meiose
pub fn produce_spore(factory: &mut Sporangium) -> Spore {
    ...
}
```

Mas, quando o Rust vê comentários que começam com três barras, ele os trata como um atributo `#[doc]`. O Rust trata o exemplo anterior exatamente da mesma forma:

```
#[doc = "Simulate the production of a spore by meiosis."]
pub fn produce_spore(factory: &mut Sporangium) -> Spore {
    ...
}
```

Quando você compila uma biblioteca ou binário, esses atributos não mudam nada, mas, quando você gera a documentação, os comentários do documento sobre os recursos públicos são incluídos na saída.

Da mesma forma, os comentários que começam com `//!` são tratados

como atributos `#![doc]` e são anexados ao recurso envolvente, geralmente um módulo ou crate. Por exemplo, seu arquivo *fern_sim/src/lib.rs* pode começar assim:

```
/// Simula o crescimento de samambaias, desde o nível de
/// células individuais para cima
```

O conteúdo de um comentário de documento é tratado como Markdown, uma notação abreviada para formatação HTML simples. Os asteriscos são utilizados para *itálico* e **negrito**, uma linha em branco é tratada como uma quebra de parágrafo e assim por diante. Você também pode incluir tags HTML, que são copiadas literalmente na documentação formatada.

Um recurso especial dos comentários do documento no Rust é que os links Markdown podem utilizar caminhos de itens do Rust, como `leaves::Leaf`, em vez de URLs relativos, para indicar o que referenciam. Cargo procurará o que o caminho referencia e o substituirá por um link no lugar certo na página de documentação certa. Por exemplo, a documentação gerada a partir desse código está vinculada às páginas de documentação para `VascularPath`, `Leaf` e `Root`:

```
/// Cria e retorne um [VascularPath`] que representa o caminho dos
/// nutrientes do dado [Root`][r] para o dado [Leaf`](leaves::Leaf).
///
/// [r]: roots::Root
pub fn trace_path(leaf: &leaves::Leaf, root: &roots::Root) -> VascularPath {
    ...
}
```

Você também pode adicionar aliases de pesquisa para facilitar a localização de coisas utilizando o recurso de pesquisa integrado. Pesquisar por “path” ou “route” na documentação desse crate levará a `VascularPath`:

```
#![doc(alias = "route")]
pub struct VascularPath {
    ...
}
```

Para blocos de documentação mais longos ou para simplificar seu fluxo de trabalho, você pode incluir arquivos externos em sua documentação. Por exemplo, se o seu repositório de arquivos *README.md* contém o mesmo texto que você gostaria de utilizar

como documentação de nível superior do seu crate, você pode colocá-lo na parte superior de `lib.rs` ou `main.rs`:

```
#![doc = include_str!("../README.md")]
```

Você pode utilizar ``backticks`` para definir trechos de código no meio do texto. Na saída, esses trechos serão formatados em uma fonte de largura fixa. Exemplos de código maiores podem ser adicionadas recuando quatro espaços:

```
/// Um bloco de código em um comentário de documento:
///
/// if samples::everything().works() {
///     println!("ok");
/// }
```

Você também pode utilizar blocos de código protegidos por Markdown. Isso tem exatamente o mesmo efeito:

```
/// Outro trecho, o mesmo código, mas escrito de forma diferente:
///
/// ```
/// if samples::everything().works() {
///     println!("ok");
/// }
/// ```
```

Seja qual for o formato que você utiliza, uma coisa interessante acontece quando você inclui um bloco de código em um comentário de documento. O Rust transforma-o automaticamente em um teste.

Doc-Tests

Quando você executa testes em um crate de biblioteca Rust, o Rust verifica se todo o código que aparece em sua documentação realmente é executado e funciona. Ele faz isso pegando cada bloco de código que aparece em um comentário de documento, compilando-o como um crate executável separado, vinculando-o à sua biblioteca e executando-o.

Eis um exemplo independente de um doc-test. Crie um novo projeto executando `cargo new --lib ranges` (o flag `--lib` informa ao Cargo que estamos criando um crate de biblioteca, não um crate executável) e coloque o seguinte código em `ranges/src/lib.rs`:

```
use std::ops::Range;
```

```

/// Retorna verdadeiro se dois intervalos se sobrepõem.
///
/// assert_eq!(ranges::overlap(0..7, 3..10), true);
/// assert_eq!(ranges::overlap(1..5, 101..105), false);
///
/// Se um dos intervalos estiver vazio, eles não serão considerados sobrepostos.
///
/// assert_eq!(ranges::overlap(0..0, 0..10), false);
///
pub fn overlap(r1: Range<usize>, r2: Range<usize>) -> bool {
    r1.start < r1.end && r2.start < r2.end &&
    r1.start < r2.end && r2.start < r1.end
}

```

Os dois pequenos blocos de código no comentário do documento aparecem na documentação gerada por cargo doc, como mostra a Figura 8.3.



Figura 8.3: Documentação mostrando alguns doc-tests.

Eles também se tornam dois testes separados:

```

$ cargo test
  Compiling ranges v0.1.0 (file:///.../ranges)
...
Doc-tests ranges

running 2 tests
test overlap_0 ... ok
test overlap_1 ... ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

```

Se passar o flag `--verbose` para Cargo, você verá que ele está utilizando `rustdoc --test` para executar esses dois testes. `rustdoc`

armazena cada amostra de código em um arquivo separado, adicionando algumas linhas de código básico, para produzir dois programas. Eis o primeiro:

```
use ranges;
fn main() {
    assert_eq!(ranges::overlap(0..7, 3..10), true);
    assert_eq!(ranges::overlap(1..5, 101..105), false);
}
```

E aqui está o segundo:

```
use ranges;
fn main() {
    assert_eq!(ranges::overlap(0..0, 0..10), false);
}
```

Os testes passam se esses programas forem compilados e executados com êxito.

Esses dois exemplos de código contêm asserções, mas isso ocorre apenas porque, neste caso, as asserções constituem uma documentação decente. A ideia por trás do doc-tests não é colocar todos os seus testes em comentários. Em vez disso, você escreve a melhor documentação possível e o Rust garante que os exemplos de código em sua documentação realmente sejam compilados e executados.

Muitas vezes, um exemplo mínimo funcional inclui alguns detalhes, como importações ou código de configuração, que são necessários para compilar o código, mas não são importantes o suficiente para mostrar na documentação. Para ocultar uma linha de um exemplo de código, coloque um `#` seguido por um espaço no início dessa linha:

```
/// Deixe o sol brilhar e execute a simulação por
/// um determinado tempo.
///
/// # use fern_sim::Terrarium;
/// # use std::time::Duration;
/// # let mut tm = Terrarium::new();
/// tm.apply_sunlight(Duration::from_secs(60));
///
pub fn apply_sunlight(&mut self, time: Duration) {
    ...
}
```

Às vezes é útil mostrar um programa de exemplo completo na

documentação, incluindo uma função `main`. Obviamente, se esses trechos de código aparecerem em seu exemplo de código, você também não deseja que o `rustdoc` adicione-os novamente. O resultado não iria compilar. `Rustdoc`, portanto, trata qualquer bloco de código contendo a string exata `fn main` como um programa completo e não acrescenta nada a ele.

O teste pode ser desabilitado para blocos específicos de código. Para dizer ao Rust que compile seu exemplo, mas pare antes de realmente o executar, utilize um bloco de código cercado com a anotação `no_run`:

```
/// Carrega todos os terrários locais na galeria on-line.
///
/// ```no_run
/// let mut session = fern_sim::connect();
/// session.upload_all();
/// ```
pub fn upload_all(&mut self) {
    ...
}
```

Se você espera que o código nem venha a ser compilado, utilize `ignore` em vez de `no_run`. Blocos marcados com `ignore` não aparecem na saída de cargo `run`, mas os testes aparecem como `Ok` se forem compilados. Se o bloco de código não for um código Rust, utilize o nome da linguagem, como `c++` ou `sh`, ou `text` para texto simples. `rustdoc` não conhece os nomes das centenas de linguagens de programação existentes; em vez disso, ele trata qualquer anotação que não reconheça com a indicação de que o bloco de código não é Rust. Isso desativa o realce de código, bem como o teste de documentação.

Especificando dependências

Vimos uma maneira de dizer ao Cargo onde obter o código-fonte dos crates dos quais seu projeto depende: pelo número da versão.

```
image = "0.13"
```

Existem várias maneiras de especificar dependências e algumas coisas bastante sutis que você pode querer dizer sobre quais versões

utilizar, então vale a pena gastar algumas páginas nisso.

Em primeiro lugar, você pode querer utilizar dependências que não são publicadas no crates.io. Uma maneira de fazer isso é especificando uma URL e uma revisão do repositório Git:

```
image = { git = "https://github.com/Piston/image.git", rev = "528f19c" }
```

Esse crate específico é de código aberto, hospedado no GitHub, mas você pode facilmente apontar para um repositório Git privado hospedado em sua rede corporativa. Como mostrado aqui, você pode especificar para utilizar `rev`, `tag` ou `branch`. (Todas essas são maneiras de dizer ao Git qual revisão do código-fonte verificar.)

Outra alternativa é especificar um diretório que contenha o código-fonte do crate:

```
image = { path = "vendor/image" }
```

Isso é conveniente quando sua equipe tem um único repositório de controle de versão que contém o código-fonte de vários crates ou talvez todo o grafo de dependências. Cada crate pode especificar suas dependências utilizando caminhos relativos.

Ter esse nível de controle sobre suas dependências é poderoso. Se você decidir que qualquer um dos crates de código aberto que você utiliza não é exatamente do seu agrado, você pode bifurcá-lo trivialmente: basta clicar no botão Fork no GitHub e alterar uma linha no seu arquivo *Cargo.toml*. Seu próximo cargo build utilizará perfeitamente o fork do crate em vez da versão oficial.

Versões

Quando você escreve algo como `image = "0.13.0"` em seu arquivo *Cargo.toml*, Cargo interpreta isso de forma bastante vaga. Ele utiliza a versão mais recente do `image` que é considerado compatível com a versão 0.13.0.

As regras de compatibilidade são adaptadas do Versionamento Semântico (<http://semver.org>).

- Um número de versão que começa com 0.0 é tão cru que Cargo nunca assume que é compatível com qualquer outra versão.
- Um número de versão que começa com 0.x, em que x é diferente

de zero, é considerado compatível com outras versões pontuais na série 0.x. Especificamos `image` versão 0.6.1, mas Cargo utilizaria 0.6.3 se disponível. (Isso não é o que o padrão de versão semântica diz sobre números de versão 0.x, mas a regra provou ser muito útil para ser deixada de fora.)

- Quando um projeto atinge 1.0, apenas novas versões principais quebram a compatibilidade. Portanto, se você solicitar a versão 2.0.1, Cargo poderá utilizar a versão 2.17.99, mas não a 3.0.

Os números de versão são flexíveis por padrão porque, caso contrário, o problema de qual versão utilizar se tornaria rapidamente restrito. Suponha que uma biblioteca, `libA`, utilizava `num = "0.1.31"` enquanto outra, `libB`, utilizava `num = "0.1.29"`. Se os números de versão exigissem correspondências exatas, nenhum projeto poderia utilizar essas duas bibliotecas juntas. Permitir que Cargo utilize qualquer versão compatível é um padrão muito mais prático.

Ainda assim, projetos diferentes têm necessidades diferentes quando se trata de dependências e controle de versão. Você pode especificar uma versão exata ou intervalo de versões utilizando operadores, conforme ilustrado na Tabela 8.3.

Tabela 8.3: Especificando versões em um arquivo Cargo.toml

Linha Cargo.toml	Significado
<code>image = "=0.10.0"</code>	Use apenas a versão exata 0.10.0
<code>image = ">=1.0.5"</code>	Use 1.0.5 ou <i>qualquer</i> versão superior (mesmo 2.9, se estiver disponível)
<code>image = ">1.0.5 <1.1.9"</code>	Use uma versão superior a 1.0.5, mas inferior a 1.1.9
<code>image = "<=2.7.10"</code>	Use qualquer versão até 2.7.10

Outra especificação de versão que você verá ocasionalmente é o curinga `*`. Isso informa ao Cargo que qualquer versão serve. A menos que algum outro arquivo *Cargo.toml* contenha uma restrição mais específica, Cargo utilizará a versão mais recente disponível. A documentação de Cargo em *doc.crates.io* (<https://oreil.ly/gI1Lq>) abrange especificações de versão com mais detalhes.

Observe que as regras de compatibilidade significam que os

números de versão não podem ser escolhidos apenas por motivos de marketing. Eles realmente significam algo. São um contrato entre os mantenedores de um crate e seus usuários. Se você mantiver um crate que está na versão 1.7 e decidir remover uma função ou fazer qualquer outra alteração que não seja totalmente compatível com versões anteriores, deve aumentar o número da versão para 2.0. Se fosse chamá-lo de 1.8, estaria afirmando que a nova versão é compatível com 1.7 e seus usuários podem se deparar com compilações quebradas.

Cargo.lock

Os números da versão em *Cargo.toml* são deliberadamente flexíveis, mas não queremos que Cargo nos atualize para as versões mais recentes da biblioteca toda vez que fizermos build (construirmos). Imagine estar no meio de uma intensa sessão de depuração quando de repente `cargo build` atualiza você para uma nova versão de uma biblioteca. Isso pode ser incrivelmente perturbador. Qualquer mudança no meio da depuração é ruim. Na verdade, quando se trata de bibliotecas, nunca é um bom momento para uma mudança inesperada.

Cargo possui, portanto, um mecanismo integrado para evitar isso. Na primeira vez que você cria um projeto, Cargo gera um arquivo *Cargo.lock* que registra a versão exata de cada crate utilizada. Compilações posteriores consultarão esse arquivo e continuarão a utilizar as mesmas versões. Cargo atualiza para versões mais recentes somente quando você solicitar, aumentando manualmente o número da versão em seu arquivo *Cargo.toml* ou executando `cargo update`:

```
$ cargo update
  Updating registry `https://github.com/rust-lang/crates.io-index`
  Updating libc v0.2.7 -> v0.2.11
  Updating png v0.4.2 -> v0.4.3
```

`cargo update` apenas atualiza para as versões mais recentes compatíveis com o que você especificou em *Cargo.toml*. Se você especificou `image = "0.6.1"` e deseja atualizar para a versão 0.10.0, terá

de mudar isso em *Cargo.toml*. Na próxima vez que você compilar, Cargo atualizará para a nova versão da biblioteca *image* e armazenará o novo número de versão em *Cargo.lock*.

O exemplo anterior mostra Cargo atualizando dois crates hospedados em crates.io. Algo muito semelhante acontece com as dependências armazenadas no Git. Suponha que nosso arquivo *Cargo.toml* contenha isto:

```
image = { git = "https://github.com/Piston/image.git", branch = "master" }
```

cargo build não extrairá novas alterações do repositório Git se perceber que temos um arquivo *Cargo.lock*. Em vez disso, ele lê *Cargo.lock* e utiliza a mesma revisão da última vez. Mas cargo update vai puxar de master para que nossa próxima compilação utilize a revisão mais recente.

Cargo.lock é gerado automaticamente para você e você normalmente não o editará manualmente. Contudo, se o seu projeto for um executável, você deve adicionar *Cargo.lock* ao controle de versão. Dessa forma, todos que criarem seu projeto obterão consistentemente as mesmas versões. A história do seu arquivo *Cargo.lock* vai registrar suas atualizações de dependência.

Se o seu projeto é uma biblioteca Rust comum, não se preocupe em comitar *Cargo.lock*. Os usuários posteriores da sua biblioteca terão arquivos *Cargo.lock* que contêm informações de versão para todo o grafo de dependências; eles vão ignorar o arquivo *Cargo.lock* da sua biblioteca. No caso raro de seu projeto ser uma biblioteca compartilhada (ou seja, a saída é um arquivo *.dll*, *.dylib* ou *.so*), não existe tal usuário cargo posterior e você deve, portanto, comitar *Cargo.lock*.

Os especificadores de versão flexíveis de *Cargo.toml* facilitam o uso de bibliotecas Rust em seu projeto e maximizam a compatibilidade entre as bibliotecas. O controle de compilação de *Cargo.lock* oferece suporte a compilações consistentes e reproduzíveis em todas as máquinas. Juntos, eles percorrem um longo caminho para ajudá-lo a evitar o inferno das dependências.

Publicando crates em crates.io

Você decidiu publicar sua biblioteca de simulação de samambaia como software de código aberto. Parabéns! Essa parte é fácil.

Primeiro, certifique-se de que a Cargo possa empacotar o crate para você.

```
$ cargo package
warning: manifest has no description, license, license-file, documentation,
homepage or repository. See http://doc.crates.io/manifest.html#package-metadata
for more info.
Packaging fern_sim v0.1.0 (file:///.../fern_sim)
Verifying fern_sim v0.1.0 (file:///.../fern_sim)
Compiling fern_sim v0.1.0 (file:///.../fern_sim/target/package/fern_sim-0.1.0)
```

O comando `cargo package` cria um arquivo (neste caso, *target/package/fern_sim-0.1.0.crate*) contendo todos os arquivos-fonte da sua biblioteca, incluindo *Cargo.toml*. Esse é o arquivo que você enviará para crates.io para compartilhar com o mundo. (Você pode utilizar `cargo package --list` para ver quais arquivos estão incluídos.) Cargo então verifica novamente seu trabalho construindo sua biblioteca a partir do arquivo *.crate*, assim como seus eventuais usuários farão.

Cargo adverte que na seção `[package]` de *Cargo.toml* estão faltando algumas informações que serão importantes para usuários posteriores, como a licença sob a qual você está distribuindo o código. A URL no aviso é um excelente recurso, então não explicaremos todos os campos em detalhes aqui. Resumindo, você pode corrigir o aviso adicionando algumas linhas a *Cargo.toml*:

```
[package]
name = "fern_sim"
version = "0.1.0"
edition = "2021"
authors = ["You <you@example.com>"]
license = "MIT"
homepage = "https://fernsim.example.com/"
repository = "https://gitlair.com/sporeador/fern_sim"
documentation = "http://fernsim.example.com/docs"
description = ""
Fern simulation, from the cellular level up.
""
```



Depois de publicar esse crate em crates.io, qualquer pessoa que baixar seu crate poderá ver o arquivo *Cargo.toml*. Então, se o campo `authors` contiver um endereço de e-mail que você prefere manter privado, agora é a hora de alterá-lo.

Outro problema que às vezes surge nesta fase é que seu arquivo *Cargo.toml* pode estar especificando a localização de outros crates por `path`, como mostrado em “Especificando dependências”, na página 238:

```
image = { path = "vendor/image" }
```

Para você e sua equipe, isso pode funcionar bem. Mas, naturalmente, quando outras pessoas baixam biblioteca `fern_sim`, elas não terão os mesmos arquivos e diretórios em seus computadores que você. Cargo, portanto, ignora a chave `path` em bibliotecas baixadas automaticamente e isso pode causar erros de compilação. A correção, porém, é direta: se sua biblioteca for publicada em crates.io, suas dependências também devem estar em crates.io. Especifique um número de versão em vez de um `path`:

```
image = "0.13.0"
```

Se preferir, você pode especificar tanto um `path`, que tem precedência para suas próprias compilações locais, e uma `version` para todos os outros usuários:

```
image = { path = "vendor/image", version = "0.13.0" }
```

É claro que, nesse caso, é sua responsabilidade garantir que os dois permaneçam sincronizados.

Por fim, antes de publicar um crate, você precisará fazer login em crates.io e obter uma chave de API. Essa etapa é simples: assim que você tiver uma conta no crates.io, sua página “Account Settings” mostrará um comando `cargo login`, como este:

```
$ cargo login 5j0dV54BjIXBpUUbfIj7G9DvNI1vsWW1
```

Cargo salva a chave em um arquivo de configuração e a chave da API deve ser mantida em segredo, como uma senha. Portanto, execute esse comando apenas em um computador que você

controla.

Feito isso, o passo final é executar `cargo publish`:

```
$ cargo publish
  Updating registry `https://github.com/rust-lang/crates.io-index`
  Uploading fern_sim v0.1.0 (file:///.../fern_sim)
```

Com isso, sua biblioteca se junta a milhares de outras no crates.io.

Espaços de trabalho

À medida que seu projeto continua a crescer, você acaba escrevendo muitos crates. Eles vivem lado a lado em um único repositório de origem:

```
fernsoft/
├── .git/...
├── fern_sim/
│   ├── Cargo.toml
│   ├── Cargo.lock
│   ├── src/...
│   └── target/...
├── fern_img/
│   ├── Cargo.toml
│   ├── Cargo.lock
│   ├── src/...
│   └── target/...
└── fern_video/
    ├── Cargo.toml
    ├── Cargo.lock
    ├── src/...
    └── target/...
```

Do jeito que Cargo funciona, cada crate tem o próprio diretório de construção, `target`, que contém uma compilação separada de todas as dependências desse crate. Esses diretórios de compilação são completamente independentes. Mesmo que dois crates tenham uma dependência comum, elas não podem compartilhar nenhum código compilado. Isso é um desperdício.

Você pode economizar tempo de compilação e espaço em disco utilizando um *workspace* do Cargo, uma coleção de crates que compartilham um diretório de compilação comum e um arquivo *Cargo.lock*.

Tudo o que você precisa fazer é criar um arquivo *Cargo.toml* no diretório raiz do seu repositório e colocar estas linhas nele:

```
[workspace]
members = ["fern_sim", "fern_img", "fern_video"]
```

Aqui *fern_sim* etc. são os nomes dos subdiretórios que contêm seus crates. Exclua quaisquer outros arquivos *Cargo.lock* e diretórios *target* que existem nesses subdiretórios.

Depois de fazer isso, `cargo build` em qualquer crate criará e utilizará automaticamente um diretório de compilação compartilhado no diretório raiz (neste caso, *fernsoft/target*). O comando `cargo build --workspace` constrói todas as crates no espaço de trabalho atual. `Cargo test` e `cargo doc` aceitam a `--workspace` opção também.

Mais coisas legais

Caso ainda não esteja satisfeito, a comunidade Rust tem mais algumas vantagens e desvantagens:

- Quando você publica um crate de código aberto em *crates.io* (<https://crates.io>), sua documentação é renderizada e hospedada automaticamente em *docs.rs* graças a Onur Aslan.
- Se o seu projeto estiver no GitHub, Travis CI pode criar e testar seu código a cada push. Ele é surpreendentemente fácil de configurar; consulte <https://travis-ci.org> para detalhes. Se você já conhece o Travis, este arquivo *.travis.yml* vai ajudá-lo a começar:

```
language: rust
rust:
  - stable
```

- Você pode gerar um arquivo *README.md* a partir do document de nível superior do seu crate. Esse recurso é oferecido como um plug-in Cargo de terceiros por Livio Ribeiro. Execute `cargo install cargo-readme` para instalar o plug-in, então `cargo readme --help` para aprender a usá-lo.

Poderíamos continuar.

O Rust é novo, mas foi projetado para dar suporte a projetos grandes e ambiciosos. Ele tem ótimas ferramentas e uma comunidade ativa. Programadores de sistema *podem* ter coisas

legais.

CAPÍTULO 9

Structs

Há muito tempo, quando os pastores queriam ver se dois rebanhos de ovelhas eram isomórficos, eles procuravam um isomorfismo explícito.

*– John C. Baez e James Dolan, Categorification
(<https://oreil.ly/EpGpb>)*

Os structs do Rust, às vezes chamados de estruturas, se assemelham aos tipos `struct` em C e C++, às classes em Python e aos objetos em JavaScript. Um struct reúne valores de tipos variados em um único valor para que você possa tratá-los como uma unidade. Dado um struct, você pode ler e modificar seus componentes individuais. E um struct pode ter métodos associados a ele que operam sobre seus componentes.

O Rust tem três tipos de struct, *campo nomeado*, *tipo tupla* e *tipo unidade*, que diferem no modo como você referencia seus componentes: um struct de campo nomeado dá um nome a cada componente, enquanto um struct tipo tupla os identifica pela ordem em que aparecem. Structs tipo unidade não possuem nenhum componente; não são comuns, mas são mais úteis do que você imagina.

Neste capítulo, explicaremos cada tipo em detalhes e mostraremos como eles aparecem na memória. Abordaremos como adicionar métodos ao struct, como definir tipos struct genéricos que funcionam com vários tipos de componente diferentes e como solicitar ao Rust que gere implementações de traits úteis comuns para seus structs.

Structs de campo nomeado

A definição de um tipo de struct de campo nomeado se parece com

isto:

```
/// Um retângulo de pixels em tons de cinza de oito bits
struct GrayscaleMap {
    pixels: Vec<u8>,
    size: (usize, usize)
}
```

Isso declara um tipo `GrayscaleMap` com dois campos nomeados, `pixels` e `size`, dos tipos especificados. A convenção no Rust é que todos os tipos, incluindo structs, tenham nomes com a primeira letra de cada palavra em maiúscula, como `GrayscaleMap`, uma convenção chamada *CamelCaseName* (ou *PascalCaseName*). Campos e métodos são escritos em letras minúsculas, com palavras separadas por sublinhados. Isso é chamado *snake_case*.

Você pode construir um valor desse tipo com uma *expressão struct*, como esta:

```
let width = 1024;
let height = 576;
let image = GrayscaleMap {
    pixels: vec![0; width * height],
    size: (width, height)
};
```

Uma expressão *struct* começa com o nome do tipo (`GrayscaleMap`) e lista o nome e o valor de cada campo, tudo entre chaves. Também há atalhos para preencher campos de variáveis locais ou argumentos com o mesmo nome:

```
fn new_map(size: (usize, usize), pixels: Vec<u8>) -> GrayscaleMap {
    assert_eq!(pixels.len(), size.0 * size.1);
    GrayscaleMap { pixels, size }
}
```

A expressão `struct GrayscaleMap { pixels, size }` é abreviação para `GrayscaleMap { pixels: pixels, size: size }`. Você pode utilizar sintaxe `key: value` para alguns campos e abreviação para outros, na mesma expressão *struct*.

Para acessar os campos de um struct, utilize o familiar operador `.`:

```
assert_eq!(image.size, (1024, 576));
assert_eq!(image.pixels.len(), 1024 * 576);
```

Como todos os outros itens, os structs são privados por padrão, visíveis apenas no módulo onde são declarados e seus submódulos.

Você pode tornar um struct visível fora de seu módulo prefixando sua definição com `pub`. O mesmo vale para cada um de seus campos, que também são privados por padrão:

```
/// Um retângulo de pixels em tons de cinza de oito bits
pub struct GrayscaleMap {
    pub pixels: Vec<u8>,
    pub size: (usize, usize)
}
```

Mesmo que um struct seja declarado `pub`, seus campos podem ser privados:

```
/// Um retângulo de pixels em tons de cinza de oito bits
pub struct GrayscaleMap {
    pixels: Vec<u8>,
    size: (usize, usize)
}
```

Outros módulos podem utilizar esse struct e quaisquer funções públicas associadas que ela possa ter, mas não podem acessar os campos privados por nome ou utilizar expressões struct para criar novos valores `GrayscaleMap`. Ou seja, criar um valor struct requer que todos os campos do struct estejam visíveis. É por isso que você não pode escrever uma expressão struct para criar uma nova `String` ou um novo `Vec`. Esses tipos padrão são structs, mas todos os seus campos são privados. Para criar um, você deve utilizar funções públicas associadas ao tipo, como `Vec::new()`.

Ao criar um valor struct de campo nomeado, você pode utilizar outro struct do mesmo tipo para fornecer valores para os campos omitidos. Em uma expressão struct, se os campos nomeados forem seguidos por `.. EXPR`, todos os campos não mencionados recebem seus valores de `EXPR`, que deve ser outro valor do mesmo tipo de struct. Suponha que temos um struct representando um monstro em um jogo:

```
// Neste jogo, as vassouras são monstros. Você vai ver
struct Broom {
    name: String,
    height: u32,
    health: u32,
    position: (f32, f32, f32),
    intent: BroomIntent
}
```



```

/// Duas alternativas possíveis para o que uma `Broom` poderia estar fazendo
#[derive(Copy, Clone)]
enum BroomIntent { FetchWater, DumpWater }

```

O melhor conto de fadas para programadores é *O aprendiz de feiticeiro*: um mágico novato encanta uma vassoura para fazer seu trabalho por ele, mas não sabe como pará-la quando o trabalho está feito. Cortar a vassoura ao meio com um machado produz apenas duas vassouras, cada uma com metade do tamanho, mas continuando a tarefa com a mesma dedicação cega da original:

```

// Recebe a vassoura de entrada por valor, tomando sua posse
fn chop(b: Broom) -> (Broom, Broom) {
  // Inicialize `broom1` principalmente a partir de `b`, alterando apenas `height`.
  // Como a `String` não é `Copy`, `broom1` toma posse do nome de `b`.
  let mut broom1 = Broom { height: b.height / 2, .. b };
  // Inicialize `broom2` principalmente a partir de `broom1`. Como `String`
  // não é `Copy`, devemos clonar `name` explicitamente.
  let mut broom2 = Broom { name: broom1.name.clone(), .. broom1 };

  // Dá a cada fragmento um nome distinto
  broom1.name.push_str(" I");
  broom2.name.push_str(" II");

  (broom1, broom2)
}

```

Com essa definição, podemos criar uma vassoura, cortá-la em duas e ver o que obtemos:

```

let hokey = Broom {
  name: "Hokey".to_string(),
  height: 60,
  health: 100,
  position: (100.0, 200.0, 0.0),
  intent: BroomIntent::FetchWater
};

let (hokey1, hokey2) = chop(hokey);
assert_eq!(hokey1.name, "Hokey I");
assert_eq!(hokey1.height, 30);
assert_eq!(hokey1.health, 100);

assert_eq!(hokey2.name, "Hokey II");
assert_eq!(hokey2.height, 30);
assert_eq!(hokey2.health, 100);

```

As novas vassouras `hokey1` e `hokey2` receberam nomes ajustados, metade da altura e toda a saúde da original.

Structs do tipo tupla

O segundo tipo de struct é chamado de struct tipo tupla, porque se assemelha a uma tupla:

```
struct Bounds(usize, usize);
```

Você constrói um valor desse tipo da mesma forma que construiria uma tupla, exceto que deve incluir o nome do struct:

```
let image_bounds = Bounds(1024, 768);
```

Os valores mantidos por um struct tipo tupla são chamados de elementos, assim como os valores de uma tupla. Você os acessa da mesma forma que acessaria uma tupla:

```
assert_eq!(image_bounds.0 * image_bounds.1, 786432);
```

Elementos individuais de um struct do tipo tupla podem ser públicos ou não:

```
pub struct Bounds(pub usize, pub usize);
```

A expressão `Bounds(1024, 768)` parece uma chamada de função e de fato é: definir o tipo também define implicitamente uma função:

```
fn Bounds(elem0: usize, elem1: usize) -> Bounds { ... }
```

No nível mais fundamental, os structs de campo nomeado e do tipo tupla são muito semelhantes. A escolha de qual utilizar se resume a questões de legibilidade, ambiguidade e brevidade. Se você vai utilizar o `.` para obter os componentes de um valor, identificar os campos pelo nome fornece ao leitor mais informações e provavelmente é mais robusto contra erros de digitação. Se você costuma utilizar correspondência de padrões para encontrar os elementos, os structs do tipo tupla podem funcionar bem.

Mas o Rust do tipo tupla é bom para *novos tipos*, structs com um único componente que você define para obter uma verificação de tipo mais rígida. Por exemplo, se estiver trabalhando com texto somente ASCII, você pode definir um novo tipo assim:

```
struct Ascii(Vec<u8>);
```

Utilizar esse tipo para suas strings ASCII é muito melhor do que simplesmente passar buffers `Vec<u8>` e explicar o que são nos comentários. O novo tipo ajuda o Rust a detectar erros quando algum outro buffer de bytes é passado para uma função que espera

texto ASCII. Daremos um exemplo de uso de novos tipos para conversões de tipo eficientes no Capítulo 22.

Structs do tipo unidade

O terceiro tipo de struct é um pouco obscuro: ele declara um tipo de struct sem nenhum elemento:

```
struct Onesuch;
```

Um valor desse tipo não ocupa memória, de maneira muito parecida com o tipo unidade (). O Rust não se preocupa em armazenar valores struct do tipo unidade na memória ou gerar código para operar neles, porque pode dizer tudo o que precisa saber sobre o valor simplesmente vendo seu tipo. Mas, logicamente, um struct vazio é um tipo com valores como qualquer outro – ou, mais precisamente, um tipo para o qual existe apenas um único valor:

```
let o = Onesuch;
```

Você já encontrou um struct do tipo unidade ao ler sobre o operador .. de intervalo em “Campos e elementos”, na página 181. Considerando que uma expressão como 3..5 é uma abreviação para o valor struct Range { start: 3, end: 5 }, a expressão .., um intervalo que omite ambas as extremidades, é uma abreviação para o valor struct do tipo unidade RangeFull.

Structs tipo unidade também podem ser úteis ao trabalhar com traits, que descreveremos no Capítulo 11.

Disposição de um struct na memória

Na memória, os structs de campo nomeado e de tipo tupla são a mesma coisa: uma coleção de valores, de tipos possivelmente mistos, dispostos de uma maneira particular na memória. Por exemplo, no início do capítulo, definimos este struct:

```
struct GrayscaleMap {  
    pixels: Vec<u8>,  
    size: (usize, usize)  
}
```

Um valor GrayscaleMap é disposto na memória conforme diagramado na

Figura 9.1.

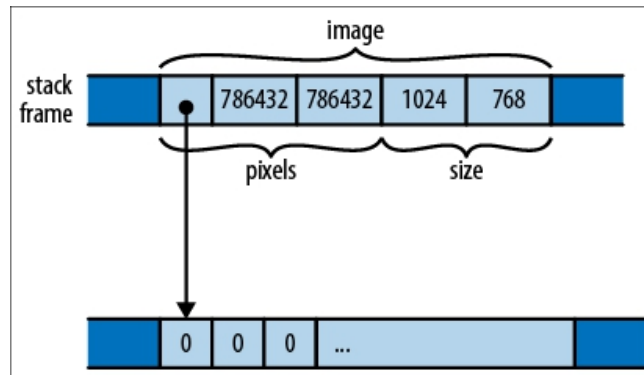


Figura 9.1: Uma estrutura `GrayscaleMap` na memória.

Ao contrário de C e C++, o Rust não faz promessas específicas sobre como ordenará os campos ou elementos de um struct na memória; esse diagrama mostra apenas um arranjo possível. Contudo, o Rust promete armazenar os valores dos campos diretamente no bloco de memória do struct. Considerando que JavaScript, Python e Java colocariam os valores `pixels` e `size` cada um em seus próprios blocos alocados no heap e com os campos de `GrayscaleMap` apontando para eles, o Rust incorpora `pixels` e `size` diretamente no valor `GrayscaleMap`. Somente o buffer alocado no heap possuído pelo vetor `pixels` permanece em seu próprio bloco.

Você pode pedir ao Rust que crie estruturas de maneira compatível com C e C++ utilizando o atributo `#[repr(C)]`. Abordaremos isso em detalhes no Capítulo 23.

Definindo métodos com `impl`

Ao longo do livro, chamamos métodos sobre todos os tipos de valores. Colocamos elementos em vetores com `v.push(e)`, buscamos seu comprimento com `v.len()`, verificamos valores `Result` para erros com `r.expect("msg")` e assim por diante. Você também pode definir métodos em seus próprios tipos de struct. Em vez de aparecer dentro da definição do struct, como em C++ ou Java, os métodos do Rust aparecem em um bloco `impl`.

Um bloco `impl` é simplesmente uma coleção de definições `fn`, cada

uma das quais se torna um método no tipo struct nomeado na parte superior do bloco. Aqui, por exemplo, definimos um struct público Queue e, em seguida, fornecemos dois métodos públicos, push e pop:

```
/// Uma fila de caracteres do tipo primeiro a entrar, primeiro a sair
pub struct Queue {
    older: Vec<char>, // elementos mais antigos, mais antigo por último
    younger: Vec<char> // elementos mais jovens, o mais jovem por último
}

impl Queue {
    /// Insere um caractere no final de uma fila
    pub fn push(&mut self, c: char) {
        self.younger.push(c);
    }

    /// Remove um caractere da frente de uma fila. Retorna `Some(c)` se houver
    /// era um caractere a ser exibido ou `None` se a fila estava vazia
    pub fn pop(&mut self) -> Option<char> {
        if self.older.is_empty() {
            if self.younger.is_empty() {
                return None;
            }
            // Move os elementos mais novos sobre os mais antigos
            // e coloca-os na ordem prometida
            use std::mem::swap;
            swap(&mut self.older, &mut self.younger);
            self.older.reverse();
        }
        // Agora é garantido que o mais antigo contenha algo. Método pop
        // de Vec já retornou uma opção, então estamos prontos
        self.older.pop()
    }
}
```

Funções definidas em um bloco impl são chamadas *funções associadas*, pois estão associadas a um tipo específico. O oposto de uma função associada é uma *função livre*, aquela que não é definida como um item do bloco impl.

O Rust passa a um método o valor que está sendo chamado como seu primeiro argumento, que deve ter o nome especial self. Como o tipo de self é obviamente aquele nomeado na parte superior do bloco impl, ou uma referência a isso, o Rust permite que você omita o tipo e escreva self, &self ou &mut self como abreviação para self: Queue, self:

`&Queue` e `self: &mut Queue`. Você pode utilizar a forma longa se quiser, mas quase todo o código Rust utiliza a forma abreviada, como mostrado anteriormente.

No nosso exemplo, os métodos `push` e `pop` referem-se aos campos de `Queue` como `self.older` e `self.younger`. Ao contrário de C++ e Java, em que os membros do objeto “this” são diretamente visíveis no corpo dos métodos, como identificadores não qualificados, um método Rust deve utilizar explicitamente `self` para referenciar o valor em que foi chamado, semelhante à maneira como os métodos Python utilizam `self` e a maneira como os métodos JavaScript utilizam `this`.

Como `push` e `pop` precisam modificar a `Queue`, ambos aceitam `&mut self`. No entanto, ao chamar um método, você não precisa pegar emprestada a referência mutável; a sintaxe de chamada de método comum cuida disso implicitamente. Assim, com essas definições em vigor, você pode utilizar `Queue` assim:

```
let mut q = Queue { older: Vec::new(), younger: Vec::new() };
```

```
q.push('0');  
q.push('1');  
assert_eq!(q.pop(), Some('0'));
```

```
q.push('∞');  
assert_eq!(q.pop(), Some('1'));  
assert_eq!(q.pop(), Some('∞'));  
assert_eq!(q.pop(), None);
```

Simplesmente escrever `q.push(...)` empresta uma referência mutável a `q`, como se você tivesse escrito `(&mut q).push(...)`, já que é isso que os métodos `push` de `self` exigem.

Se um método não precisa modificar seu `self`, então você pode defini-lo para obter uma referência compartilhada. Por exemplo:

```
impl Queue {  
    pub fn is_empty(&self) -> bool {  
        self.older.is_empty() && self.younger.is_empty()  
    }  
}
```

Novamente, a expressão de chamada de método sabe que tipo de referência deve ser emprestada:

```
assert!(q.is_empty());
```

```
q.push('⊙');
assert!(!q.is_empty());
```

Ou, se um método quiser tomar posse de `self`, pode receber `self` por valor:

```
impl Queue {
    pub fn split(self) -> (Vec<char>, Vec<char>) {
        (self.older, self.younger)
    }
}
```

A chamada de método de `split` se parece com as outras chamadas de método:

```
let mut q = Queue { older: Vec::new(), younger: Vec::new() };

q.push('P');
q.push('D');
assert_eq!(q.pop(), Some('P'));
q.push('X');

let (older, younger) = q.split();
// q agora não foi inicializado
assert_eq!(older, vec!['D']);
assert_eq!(younger, vec!['X']);
```

Mas note que, como `split` recebe seu `self` por valor, isso *move* a `Queue` para fora de `q`, deixando `q` não inicializada. Como o `self` de `split` agora possui a fila, ele é capaz de mover os vetores individuais para fora dela e devolvê-los ao chamador.

Às vezes, receber `self` por valor dessa maneira, ou mesmo por referência, não é suficiente, então o Rust também permite que você passe `self` por meio de tipos de ponteiros inteligentes.

Passando Self como um Box, Rc ou Arc

O argumento de um método `self` também pode ser um `Box<Self>`, `Rc<Self>` ou `Arc<Self>`. Esse método só pode ser chamado sobre um valor do tipo de ponteiro fornecido. Chamar o método passa a posse do ponteiro para ele.

Normalmente, você não precisará fazer isso. Um método que espera `self` por referência funciona bem quando chamado em qualquer um desses tipos de ponteiro:

```
let mut bq = Box::new(Queue::new());
```

```
// `Queue::push` espera uma `&mut Queue`, mas `bq` é uma `Box<Queue>`.
// Isso é bom: O Rust pega emprestado um `&mut Queue` da `Box` pela
// duração da chamada.
bq.push('■');
```

Para chamadas de método e acesso a campos, o Rust pega automaticamente uma referência de tipos de ponteiro como `Box`, `Rc` e `Arc`, então `&self` e `&mut self` são quase sempre a coisa certa em uma assinatura de método, com os ocasionais `self`.

Mas se acontecer de algum método precisar possuir um ponteiro para `self`, e seus chamadores tiverem esse ponteiro à mão, o Rust permitirá que você o passe como o argumento do método `self`. Para fazer isso, você deve especificar o tipo de `self`, como se fosse um parâmetro comum:

```
impl Node {
    fn append_to(self: Rc<Self>, parent: &mut Node) {
        parent.children.push(self);
    }
}
```

Funções associadas a tipos

Um bloco `impl` para um determinado tipo também pode definir funções que simplesmente não aceitam `self` como um argumento. Essas ainda são funções associadas, pois estão em um bloco `impl`, mas não são métodos, já que não recebem um argumento `self`. Para distingui-las dos métodos, nós as chamamos de *funções associadas ao tipo*.

Eles geralmente são utilizados para fornecer funções de construtor, como esta:

```
impl Queue {
    pub fn new() -> Queue {
        Queue { older: Vec::new(), younger: Vec::new() }
    }
}
```

Para utilizar essa função, nós a referenciamos como `Queue::new`: o nome do tipo, dois pontos duplos e o nome da função. Agora nosso código de exemplo se torna um pouco mais apresentável:

```
let mut q = Queue::new();
```



```
q.push('*');
```

```
...
```

É convencional no Rust que as funções do construtor sejam nomeadas `new`; nós já vimos `Vec::new`, `Box::new`, `HashMap::new` e outros. Mas não há nada de especial no nome `new`. Não é uma palavra-chave e os tipos geralmente têm outras funções associadas que servem como construtores, como `Vec::with_capacity`.

Embora você possa ter muitos blocos `impl` separados para um único tipo, eles devem estar todos na mesma `crate` que define esse tipo. Contudo, o Rust permite anexar seus próprios métodos a outros tipos; explicaremos como no Capítulo 11.

Se você está acostumado com C++ ou Java, separar os métodos de um tipo de sua definição pode parecer incomum, mas há várias vantagens em fazer isso:

- É sempre fácil encontrar os membros de dados de um tipo. Em grandes definições de classe C++, pode ser necessário examinar centenas de linhas de definições de função de membro para ter certeza de que não perdeu nenhum dos membros de dados da classe; no Rust, eles estão todos em um só lugar.
- Embora se possa imaginar encaixar métodos na sintaxe para `structs` de campo nomeado, não é tão simples para `structs` do tipo `tupla` e do tipo `unidade`. Colocar métodos em um bloco `impl` permite uma única sintaxe para todos os três. Na verdade, o Rust utiliza essa mesma sintaxe para definir métodos em tipos que não são `structs`, como tipos `enum` e tipos primitivos como `i32`. (O fato de que qualquer tipo pode ter métodos é uma das razões pelas quais Rust não utiliza muito o termo *objeto*, preferindo chamar tudo de *valor*.)
- A mesma sintaxe `impl` também serve perfeitamente para implementar `traits`, que abordaremos no Capítulo 11.

Consts associadas

Outra característica de linguagens como C# e Java, que o Rust adota em seu sistema de tipos, é a ideia de valores associados a um

tipo, em vez de uma instância específica desse tipo. No Rust, eles são conhecidos como *consts associadas*.

Como o nome indica, as consts associadas são valores constantes. Em geral, são utilizados para especificar valores comumente utilizados de um tipo. Por exemplo, você pode definir um vetor bidimensional para uso em álgebra linear com um vetor unitário associado:

```
pub struct Vector2 {  
    x: f32,  
    y: f32,  
}  
  
impl Vector2 {  
    const ZERO: Vector2 = Vector2 { x: 0.0, y: 0.0 };  
    const UNIT: Vector2 = Vector2 { x: 1.0, y: 0.0 };  
}
```

Esses valores são associados ao próprio tipo e você pode usá-los sem referenciar outra instância de `Vector2`. Assim como as funções associadas, elas são acessadas nomeando o tipo ao qual estão associadas, seguido de seu nome:

```
let scaled = Vector2::UNIT.scaled_by(2.0);
```

Tampouco uma const associada precisa ser do mesmo tipo que o tipo ao qual está associada; poderíamos utilizar esse recurso para adicionar IDs ou nomes a tipos. Por exemplo, se houvesse vários tipos semelhantes a `Vector2`, que precisassem ser gravados em um arquivo e carregados na memória posteriormente, uma const associada poderia ser utilizada para adicionar nomes ou IDs numéricos, que poderiam ser gravados ao lado dos dados para identificar seu tipo:

```
impl Vector2 {  
    const NAME: &'static str = "Vector2";  
    const ID: u32 = 18;  
}
```

Structs genéricos

Nossa definição anterior de `Queue` é insatisfatória: foi escrita para armazenar caracteres, mas não há nada em sua estrutura ou em

seus métodos que seja específico para caracteres. Se fôssemos definir outro struct que contivesse, digamos, valores `String`, o código pode ser idêntico, exceto que `char` seria substituído por `String`. Isso seria uma perda de tempo.

Felizmente, structs em Rust podem ser *genéricos*, o que significa que a definição deles é um template no qual você pode inserir qualquer tipo que quiser. Por exemplo, eis uma definição para `Queue` que pode conter valores de qualquer tipo:

```
pub struct Queue<T> {  
    older: Vec<T>,  
    younger: Vec<T>  
}
```

Você pode ler o `<T>` em `Queue<T>` como “para qualquer tipo de elemento `T`...”. Portanto, essa definição diz: “Para qualquer tipo `T`, uma `Queue<T>` são dois campos do tipo `Vec<T>`”. Por exemplo, em `Queue<String>`, `T` é `String` e, portanto, `older` e `younger` são do tipo `Vec<String>`. Em `Queue<char>`, `T` é `char` e obtemos um struct idêntico à definição específica de `char` com que começamos. Na verdade, `Vec` em si é um struct genérico, definido exatamente dessa maneira.

Nas definições de um struct genérico, os nomes de tipo utilizados entre os símbolos de menor (`<`) e maior (`>`) são chamados de *parâmetros de tipo*. Um bloco `impl` para um struct genérico se parece com isto:

```
impl<T> Queue<T> {  
    pub fn new() -> Queue<T> {  
        Queue { older: Vec::new(), younger: Vec::new() }  
    }  
  
    pub fn push(&mut self, t: T) {  
        self.younger.push(t);  
    }  
  
    pub fn is_empty(&self) -> bool {  
        self.older.is_empty() && self.younger.is_empty()  
    }  
  
    ...  
}
```

Você pode ler a linha `impl<T> Queue<T>` como “para qualquer tipo `T`,

aqui estão algumas funções associadas disponíveis em `Queue<T>`.” Então, você pode utilizar o parâmetro de tipo `T` como um tipo nas definições de função associadas.

A sintaxe pode parecer um pouco redundante, mas o `impl<T>` deixa claro que o bloco `impl` cobre qualquer tipo `T`, o que o distingue de um bloco `impl` escrito para um tipo específico de `Queue`, como este:

```
impl Queue<f64> {  
    fn sum(&self) -> f64 {  
        ...  
    }  
}
```

Esse cabeçalho de bloco `impl` diz: “Eis algumas funções associadas especificamente para `Queue<f64>`”. Isso dá a `Queue<f64>` um método `sum`, não disponível em nenhum outro tipo de `Queue`.

Utilizamos a abreviação do Rust para parâmetros `self` no código anterior; escrever `Queue<T>` em todos os lugares torna-se trabalhoso e distrativo. Como outra abreviação, cada bloco `impl`, genérico ou não, define o parâmetro de tipo especial `Self` (note o nome `CamelCase`) para ser do tipo ao qual estamos adicionando métodos. No código anterior, `Self` seria `Queue<T>`, então podemos abreviar a definição de `Queue::new` um pouco mais adiante:

```
pub fn new() -> Self {  
    Queue { older: Vec::new(), younger: Vec::new() }  
}
```

Você deve ter notado que, no corpo de `new`, não precisamos escrever o parâmetro de tipo na expressão de construção; simplesmente escrever `Queue { ... }` foi suficiente. Essa é a inferência de tipo do Rust em ação: já que há apenas um tipo que funciona para o valor de retorno dessa função – ou seja, `Queue<T>` –, o Rust fornece o parâmetro para nós. Mas você sempre precisará fornecer parâmetros de tipo em assinaturas de função e definições de tipo. O Rust não infere estes; em vez disso, utiliza esses tipos explícitos como base a partir da qual infere os tipos no corpo das funções.

`Self` também pode ser utilizado dessa forma; poderíamos ter escrito `Self { ... }` em seu lugar. Cabe a você decidir qual acha mais fácil de entender.

Para chamadas de função associadas, pode fornecer o parâmetro de tipo explicitamente utilizando a notação `::<>` (turbopeixe/turbofish):

```
let mut q = Queue::<char>::new();
```

Mas, na prática, geralmente pode deixar o Rust descobrir isso para você:

```
let mut q = Queue::new();  
let mut r = Queue::new();
```

```
q.push("CAD"); // aparentemente uma Fila<&'static str>  
r.push(0.74); // aparentemente uma fila<f64>  
q.push("BTC"); // Bitcoins por USD, 2019-6  
r.push(13764.0); // O Rust falha em detectar a exuberância irracional
```

Na verdade, isso é exatamente o que temos feito com `Vec`, outro tipo de struct genérico, ao longo do livro.

Não são apenas structs que podem ser genéricos. Enums também podem receber parâmetros de tipo, com uma sintaxe muito semelhante. Mostraremos isso em detalhes em “Enums”, na página 271.

Structs genéricos com parâmetros de tempo de vida

Como discutimos em “Structs contendo referências”, na página 147, se um tipo de struct contiver referências, você deverá nomear o tempo de vida dessas referências. Por exemplo, eis uma estrutura que pode conter referências aos maiores e menores elementos de alguma fatia:

```
struct Extrema<'elt> {  
    greatest: &'elt i32,  
    least: &'elt i32  
}
```

Anteriormente, convidamos você a pensar em uma declaração como `struct Queue<T>` no sentido de que, dado qualquer tipo específico `T`, você pode criar um `Queue<T>` que contém esse tipo. Da mesma forma, você pode pensar em `struct Extrema<'elt>` no sentido de que, dado qualquer tempo de vida específico `'elt`, você pode criar um `Extrema<'elt>` que contém referências com esse tempo de vida.

Eis uma função para escanear uma fatia e retornar um valor `Extrema` cujos campos referem-se aos seus elementos:

```
fn find_extrema<'s>(slice: &'s [i32]) -> Extrema<'s> {  
    let mut greatest = &slice[0];  
    let mut least = &slice[0];  
  
    for i in 1..slice.len() {  
        if slice[i] < *least { least = &slice[i]; }  
        if slice[i] > *greatest { greatest = &slice[i]; }  
    }  
    Extrema { greatest, least }  
}
```

Aqui, como `find_extrema` empresta elementos de `slice`, que tem tempo de vida `'s`, o struct `Extrema` que retornamos também utiliza `'s` como o tempo de vida de suas referências. O Rust sempre infere parâmetros de tempo de vida para chamadas, então chamadas a `find_extrema` não precisam mencioná-los:

```
let a = [0, -3, 0, 15, 48];  
let e = find_extrema(&a);  
assert_eq!(*e.least, -3);  
assert_eq!(*e.greatest, 48);
```

Como é muito comum que o tipo de retorno utilize o mesmo tempo de vida como um argumento, o Rust nos permite omitir os tempos de vida quando há um candidato óbvio. Poderíamos também ter escrito a assinatura de `find_extrema` assim, sem mudança de significado:

```
fn find_extrema(slice: &[i32]) -> Extrema {  
    ...  
}
```

De certo, *poderíamos* querer dizer `Extrema<'static>`, mas isso é bastante incomum. O Rust fornece uma abreviação para o caso comum.

Structs genéricos com parâmetros constantes

Um struct genérico também pode receber parâmetros que são valores constantes. Por exemplo, você pode definir um tipo representando polinômios de grau arbitrário da seguinte forma:

```
/// Um polinômio de grau N - 1.
```

```

struct Polynomial<const N: usize> {
    /// Os coeficientes do polinômio.
    ///
    /// Para um polinômio  $a + bx + cx^2 + \dots + zx^{n-1}$ ,
    /// o  $i^{\text{ésimo}}$  elemento é o coeficiente de  $x^i$ .
    coefficients: [f64; N]
}

```

Com essa definição, `Polynomial<3>` é um polinômio quadrático, por exemplo. A cláusula `<const N: usize>` diz que o tipo `Polynomial` espera um valor `usize` como seu parâmetro genérico, que ele utiliza para decidir quantos coeficientes armazenar.

Diferente de `Vec`, que possui campos contendo seu comprimento e capacidade e armazena seus elementos no heap, `Polynomial` armazena seus coeficientes diretamente no valor e nada mais. O comprimento é dado pelo tipo. (A capacidade não é necessária, porque `Polynomials` não podem crescer dinamicamente.)

Podemos utilizar o parâmetro `N` nas funções associadas do tipo:

```

impl<const N: usize> Polynomial<N> {
    fn new(coefficients: [f64; N]) -> Polynomial<N> {
        Polynomial { coefficients }
    }

    /// Avalie o polinômio em x.
    fn eval(&self, x: f64) -> f64 {
        // O método de Horner é numericamente estável, eficiente e simples:
        //  $c_0 + x(c_1 + x(c_2 + x(c_3 + \dots x(c_{n-1} + x c_n))))$ 
        let mut sum = 0.0;
        for i in (0..N).rev() {
            sum = self.coefficients[i] + x * sum;
        }

        sum
    }
}

```

Aqui a função `new` aceita um array de comprimento `N` e recebe seus elementos como os coeficientes de um novo valor `Polynomial`. O método `eval` itera pelo intervalo `0..N` para encontrar o valor do polinômio em um determinado ponto `x`.

Tal como acontece com os parâmetros de tipo e o tempo de vida, o Rust geralmente pode inferir os valores corretos para parâmetros

constantes:

```
use std::f64::consts::FRAC_PI_2; //  $\pi/2$ 

// Aproxima a função `sin` :  $\sin x \cong x - 1/6 x^3 + 1/120 x^5$ 
// Em torno de zero, é bastante preciso!
let sine_poly = Polynomial::new([0.0, 1.0, 0.0, -1.0/6.0, 0.0,
                                1.0/120.0]);
assert_eq!(sine_poly.eval(0.0), 0.0);
assert!((sine_poly.eval(FRAC_PI_2) - 1.).abs() < 0.005);
```

Como passamos um array `Polynomial::new` com seis elementos, o Rust sabe que devemos estar construindo um `Polynomial<6>`. O método `eval` sabe quantas iterações o loop `for` deve ser executado simplesmente consultando seu tipo `Self`. Como o comprimento é conhecido em tempo de compilação, o compilador provavelmente substituirá o loop inteiramente por código diretamente.

Um parâmetro `const` genérico pode ser qualquer tipo inteiro, `char` ou `bool`. Números de ponto flutuante, enumerações e outros tipos não são permitidos.

Se o `struct` tiver outros tipos de parâmetros genéricos, os parâmetros de tempo de vida devem vir primeiro, seguidos por tipos, seguidos por qualquer valor `const`. Por exemplo, um tipo que contém um array de referências pode ser declarado assim:

```
struct LumpOfReferences<'a, T, const N: usize> {
    the_lump: [&'a T; N]
}
```

Parâmetros genéricos constantes são uma adição relativamente nova ao Rust e seu uso é um pouco restrito por enquanto. Por exemplo, teria sido melhor definir `Polynomial` assim:

```
/// Um polinômio de grau N
struct Polynomial<const N: usize> {
    coefficients: [f64; N + 1]
}
```

Mas o Rust rejeita essa definição:

```
error: generic parameters may not be used in const operations
|
6 |     coefficients: [f64; N + 1]
|                      ^ cannot perform const operation using `N`
|
= help: const parameters may only be used as standalone arguments, i.e. `N`
```


Embora seja bom dizer `[f64; N]`, um tipo como `[f64; N + 1]` é aparentemente muito ousado para o Rust. Mas o Rust impõe essa restrição por enquanto para evitar problemas como este:

```
struct Ketchup<const N: usize> {  
    tomayto: [i32; N & !31],  
    tomahto: [i32; N - (N % 32)],  
}
```

Acontece que `N & !31` e `N - (N % 32)` são iguais para todos os valores de `N`, então `tomayto` e `tomahto` têm sempre o mesmo tipo. Deveria ser permitido atribuir um ao outro, por exemplo. Mas ensinar ao verificador de tipos do Rust a álgebra complicada de que ele precisaria para ser capaz de reconhecer esse fato corre o risco de introduzir casos confusos em um aspecto da linguagem que já é bastante complicado. Naturalmente, expressões simples como `N + 1` são muito mais bem-comportadas e há um trabalho em andamento para dar suporte ao Rust para lidar com isso sem problemas.

Como a preocupação aqui é com o comportamento do verificador de tipo, essa restrição se aplica apenas a parâmetros constantes que aparecem em tipos, como o comprimento de um array. Em uma expressão comum, você pode utilizar `N` como quiser: `N + 1` e `N & !31` são perfeitamente aceitáveis.

Se o valor que você deseja fornecer para um parâmetro `const` genérico não é simplesmente um literal ou um identificador único, então deve escrevê-lo entre chaves, como em `Polynomial<{5 + 1}>`. Essa regra permite que o Rust relate erros de sintaxe com mais precisão.

Derivando traits comuns para tipos de struct

Struct podem ser muito fáceis de escrever:

```
struct Point {  
    x: f64,  
    y: f64  
}
```

Contudo, se começasse a utilizar esse tipo `Point`, você notaria rapidamente que é um pouco trabalhoso. Como escrito, `Point` não é

copiável ou clonável. Você não pode imprimi-lo com `println!("{}", point);` e não suporta os operadores `==` e `!=`.

Cada um desses recursos tem um nome no Rust – `Copy`, `Clone`, `Debug` e `PartialEq`. Eles são chamados *traits*. No Capítulo 11, mostraremos como implementar traits manualmente para seus próprios structs. Mas no caso desses traits padrão e várias outros, você não precisa implementá-los manualmente, a menos que queira algum tipo de comportamento personalizado. O Rust pode implementá-los automaticamente para você, com precisão mecânica. Basta adicionar um atributo `#[derive]` para o struct:

```
#[derive(Copy, Clone, Debug, PartialEq)]
struct Point {
    x: f64,
    y: f64
}
```

Cada um desses traits pode ser implementado automaticamente para um struct, desde que cada um de seus campos implemente o trait. Podemos pedir ao Rust que derive `PartialEq` para `Point` porque seus dois campos são ambos do tipo `f64`, que já implementa `PartialEq`.

O Rust também pode derivar `PartialOrd`, o que adicionaria suporte para os operadores de comparação `<`, `>`, `<=` e `>=`. Não fizemos isso aqui, porque comparar dois pontos para ver se um é “menor que” o outro é realmente uma coisa muito estranha de se fazer. Não há uma ordem convencional de pontos. Portanto, optamos por não oferecer suporte a esses operadores para valores `Point`. Casos como esse são uma das razões pelas quais o Rust nos faz escrever o atributo `#[derive]` em vez de derivar automaticamente todos os traits possíveis. Outro motivo é que a implementação de um trait é automaticamente um recurso público, portanto copiabilidade, clonagem e assim por diante fazem parte da API pública de seu struct e devem ser escolhidos deliberadamente.

Descreveremos os traits padrão do Rust em detalhes e explicaremos quais são capazes de serem utilizados com `#[derive]` no Capítulo 13.

Mutabilidade interior

Mutabilidade é como qualquer outra coisa: em excesso, causa problemas, mas muitas vezes você quer só um pouquinho. Por exemplo, digamos que seu sistema de controle de um robô aranha tenha um struct central, `SpiderRobot`, que contém configurações e identificadores de E/S. Ele é configurado quando o robô inicializa, e os valores nunca mudam:

```
pub struct SpiderRobot {
    species: String,
    web_enabled: bool,
    leg_devices: [fd::FileDesc; 8],
    ...
}
```

Cada sistema principal do robô é tratado por um struct diferente e cada um tem um ponteiro de volta para o `SpiderRobot`:

```
use std::rc::Rc;

pub struct SpiderSenses {
    robot: Rc<SpiderRobot>, // <-- ponteiro para configurações e E/S
    eyes: [Camera; 32],
    motion: Accelerometer,
    ...
}
```

Os structs para construção da teia, predação, controle de fluxo de veneno e assim por diante também têm um ponteiro inteligente `Rc<SpiderRobot>`. Lembre-se de que `Rc` significa contagem de referência e um valor em um box `Rc` é sempre compartilhado e, portanto, sempre imutável.

Agora suponha que você queira adicionar um log ao `SpiderRobot` struct, utilizando o tipo padrão `File`. Há um problema: um `File` tem de ser `mut`. Todos os métodos para escrever nele requerem uma referência `mut`.

Esse tipo de situação surge com bastante frequência. O que precisamos é de um pouco de dados mutáveis (um `File`) dentro de um valor imutável (o struct `SpiderRobot`). Isso é chamado *mutabilidade interior*. O Rust oferece vários sabores de mutabilidade; nesta seção, discutiremos os dois tipos mais diretos: `Cell<T>` e `RefCell<T>`, no módulo `std::cell`.

Um `Cell<T>` é um struct que contém um único valor privado do tipo `T`. A única coisa especial sobre um `Cell` é que você pode obter e definir o

campo mesmo que não tenha acesso mut ao Cell em si:

```
Cell::new(value)
```

Cria uma nova Cell (célula), movendo o value para dentro dela.

```
cell.get()
```

Retorna uma cópia do valor na cell.

```
cell.set(value)
```

Armazena o value na cell, dropando o valor armazenado anteriormente.

Esse método leva self como uma referência não-mut:

```
fn set(&self, value: T) // nota: não `&mut self`
```

Isso, obviamente, é incomum para métodos chamados set. Até agora, o Rust nos treinou para esperar que precisamos de acesso mut se quisermos fazer alterações nos dados. Mas, da mesma forma, esse detalhe incomum é o ponto principal de Cells. Elas são simplesmente uma maneira segura de quebrar as regras de imutabilidade – nem mais, nem menos.

As Cells também têm alguns outros métodos, sobre os quais você pode ler na documentação (<https://oreil.ly/WqRrt>).

Uma Cell seria útil se você estivesse adicionando um contador simples ao seu SpiderRobot. Você poderia escrever:

```
use std::cell::Cell;

pub struct SpiderRobot {
    ...
    hardware_error_count: Cell<u32>,
    ...
}
```

Então, mesmo métodos não-mut de SpiderRobot podem acessar esse u32, utilizando os métodos .get() e .set():

```
impl SpiderRobot {
    /// Aumenta a contagem de erros em 1
    pub fn add_hardware_error(&self) {
        let n = self.hardware_error_count.get();
        self.hardware_error_count.set(n + 1);
    }

    /// Verdadeiro se algum erro de hardware tiver sido relatado
    pub fn has_hardware_errors(&self) -> bool {
        self.hardware_error_count.get() > 0
    }
}
```

```
}  
}
```

Isso é bastante fácil, mas não resolve nosso problema de logging. `Cell` não deixa você chamar métodos `mut` em um valor compartilhado. O método `.get()` retorna uma cópia do valor na célula, portanto funciona apenas se `T` implementa o trait `Copy`. Para registro em log, precisamos de um `File` mutável e `File` não é copiável.

A ferramenta certa neste caso é um `RefCell`. Como `Cell<T>`, `RefCell<T>` é um tipo genérico que contém um único valor do tipo `T`. Diferentemente de `Cell`, `RefCell` suporta o empréstimo de referências a seu valor `T`:

`RefCell::new(value)`

Cria uma nova `RefCell`, movendo `value` para dentro dela.

`ref_cell.borrow()`

Retorna uma `Ref<T>`, que é essencialmente apenas uma referência compartilhada ao valor armazenado em `ref_cell`.

Esse método gera um pânico se o valor já estiver mutavelmente emprestado; veja os detalhes a seguir.

`ref_cell.borrow_mut()`

Retorna um `RefMut<T>`, essencialmente uma referência mutável ao valor em `ref_cell`.

Esse método gera um pânico se o valor já estiver emprestado; veja os detalhes a seguir.

`ref_cell.try_borrow()`, `ref_cell.try_borrow_mut()`

Trabalha como `borrow()` e `borrow_mut()`, mas devolve um `Result`. Em vez de gerar um pânico se o valor já estiver mutavelmente emprestado, eles retornam um valor `Err`.

Novamente, `RefCell` tem alguns outros métodos, que você pode encontrar na documentação (<https://oreil.ly/FtnIO>).

Os dois métodos `borrow` geram um pânico apenas se você tentar quebrar a regra do Rust em que referências `mut` são referências exclusivas. Por exemplo, isso causaria pânico:

```
use std::cell::RefCell;
```

```
let ref_cell: RefCell<String> = RefCell::new("hello".to_string());
```

```
let r = ref_cell.borrow();    // ok, retorna uma Ref<String>
let count = r.len();          // ok, retorna "hello".len()
assert_eq!(count, 5);
```

```
let mut w = ref_cell.borrow_mut(); // pânico: já emprestado
w.push_str(" world");
```

Para evitar pânico, você pode colocar esses dois empréstimos em blocos separados. Dessa maneira, `r` seria dropado antes de tentar emprestar `w`.

Isso é muito parecido com o funcionamento das referências normais. A única diferença é que normalmente, quando você pega uma referência a uma variável, o Rust verifica em *tempo de compilação* para garantir que você está utilizando a referência com segurança. Se as verificações falharem, você receberá um erro do compilador. `RefCell` impõe a mesma regra utilizando verificações em tempo de execução. Então, se você está quebrando as regras, terá um pânico (ou `Err`, para `try_borrow` e `try_borrow_mut`).

Agora estamos prontos para colocar `RefCell` para trabalhar em nosso tipo `SpiderRobot`:

```
pub struct SpiderRobot {
    ...
    log_file: RefCell<File>,
    ...
}

impl SpiderRobot {
    /// Grava uma linha no arquivo de log
    pub fn log(&self, message: &str) {
        let mut file = self.log_file.borrow_mut();
        // `writeln!` é como `println!`, mas envia
        // saída para o arquivo fornecido
        writeln!(file, "{}", message).unwrap();
    }
}
```

A variável `file` tem tipo `RefMut<File>`. Ele pode ser utilizado apenas como uma referência mutável a um `File`. Para obter detalhes sobre como gravar em arquivos, consulte o Capítulo 18.

As células são fáceis de utilizar. Ter de chamar `.get()` e `.set()` ou `.borrow()` e `.borrow_mut()` é um pouco estranho, mas é apenas o preço que pagamos por quebrar as regras. A outra desvantagem é menos

óbvia e mais séria: as células – e quaisquer tipos que as contenham – não são thread-safe. O Rust, portanto, não permitirá que vários threads os acessem ao mesmo tempo. Descreveremos os tipos de mutabilidade interna thread-safe no Capítulo 19, quando discutirmos “Mutex<T>”, na página 596, “Atomics”, na página 604, e “Variáveis globais”, na página 606.

Quer um struct tenha campos nomeados ou seja do tipo tupla, ele é uma agregação de outros valores: se eu tiver um struct `SpiderSenses`, então tenho um ponteiro `Rc` para um struct `SpiderRobot` compartilhado, e tenho olhos, e tenho um acelerômetro etc. Portanto, a essência de um struct é a palavra “e”: Eu tenho um X *e* um Y. Mas e se houvesse outro tipo de tipo construído em torno da palavra “ou”? Ou seja, quando você tem um valor desse tipo, teria *ora* um X *ou* um Y? Esses tipos acabam sendo tão úteis que são onipresentes no Rust e são o assunto do próximo capítulo.

CAPÍTULO 10

Enums e padrões

É surpreendente o quanto as linguagens de computador fazem sentido quando vistas como uma trágica privação de tipos de soma (cf. privação de lambdas).

–Graydon Hoare (<https://oreil.ly/cyYQc>)

O primeiro tópico deste capítulo é poderoso, tão antigo quanto as montanhas, pronto para ajudá-lo a fazer muito em pouco tempo (por um preço) e conhecido por muitos nomes em muitas culturas. Mas não é o diabo. É um tipo de dados definido pelo usuário, há muito conhecido pelos hackers ML e Haskell como tipos de soma, uniões discriminadas ou tipos de dados algébricos. No Rust, eles são chamados de *enumerações*, ou simplesmente *enums*. Ao contrário do diabo, eles estão bastante seguros e o preço que cobram não é uma grande privação.

C++ e C# têm enums; você pode usá-los para definir seu próprio tipo cujos valores são um conjunto de nomes constantes. Por exemplo, você pode definir um tipo chamado `Color` com valores `Red`, `Orange`, `Yellow` e assim por diante. Esse tipo de enum também funciona no Rust. Mas o Rust leva os enums muito mais longe. Uma enumeração do Rust também pode conter dados, mesmo dados de vários tipos. Por exemplo, o tipo `Result<String, io::Error>` do Rust é uma enumeração; tal valor é um valor `Ok` contendo uma `String` ou um valor `Err` contendo um `io::Error`. Isso está além do que as enumerações em C++ e C# podem fazer. É mais como um `union` do C++ – mas, ao contrário das uniões, as enumerações do Rust são seguras para tipos (type-safe).

Enums são úteis sempre que um valor pode ser uma coisa ou outra. O “preço” de usá-los é que você deve acessar os dados com segurança, utilizando correspondência de padrões, nosso tópico para

a segunda metade deste capítulo.

Os padrões também podem ser familiares se você já usou desempacotamento no Python ou desestruturar em JavaScript, mas o Rust leva os padrões adiante. Os padrões do Rust são um pouco como expressões regulares para todos os seus dados. Eles são utilizados para testar se um valor tem ou não uma forma específica desejada. Eles podem extrair vários campos de um struct ou tupla em variáveis locais de uma só vez. E como as expressões regulares, eles são concisos, geralmente fazendo tudo em uma única linha de código.

Este capítulo começa com as noções básicas de enums, mostrando como os dados podem ser associados a variantes de enums e como os enums são armazenados na memória. Em seguida, mostraremos como os padrões do Rust e as instruções `match` podem especificar de forma concisa a lógica com base em enums, structs, arrays e fatias. Os padrões também podem incluir referências, movimentos e condições `if`, tornando-os ainda mais capazes.

Enums

Enumerações simples no estilo C são diretas:

```
enum Ordering {  
    Less,  
    Equal,  
    Greater,  
}
```

Isso declara um tipo `Ordering` com três valores possíveis, chamados *variantes* ou *construtores*: `Ordering::Less`, `Ordering::Equal` e `Ordering::Greater`. Esse enum específico faz parte da biblioteca padrão, portanto o código Rust pode importá-lo sozinho:

```
use std::cmp::Ordering;  
  
fn compare(n: i32, m: i32) -> Ordering {  
    if n < m {  
        Ordering::Less  
    } else if n > m {  
        Ordering::Greater  
    } else {
```

```

        Ordering::Equal
    }
}

```

ou com todos os seus construtores:

```

use std::cmp::Ordering::{self, *}; // `*` para importar todos os filhos

fn compare(n: i32, m: i32) -> Ordering {
    if n < m {
        Less
    } else if n > m {
        Greater
    } else {
        Equal
    }
}

```

Depois de importar os construtores, podemos escrever `Less` em vez de `Ordering::Less` e assim por diante, mas, como isso é menos explícito, geralmente é considerado um estilo melhor *não* os importar, exceto quando torna seu código muito mais legível.

Para importar os construtores de um enum declarado no módulo atual, utilize um `self` no import:

```

enum Pet {
    Orca,
    Giraffe,
    ...
}

use self::Pet::*;

```

Na memória, os valores de enums no estilo C são armazenados como inteiros. Ocasionalmente, é útil dizer ao Rust quais números inteiros utilizar:

```

enum HttpStatus {
    Ok = 200,
    NotModified = 304,
    NotFound = 404,
    ...
}

```

Caso contrário, o Rust atribuirá os números para você, começando em 0.

Por padrão, o Rust armazena enums no estilo C utilizando o menor tipo inteiro interno que pode acomodá-los. A maioria cabe em um

único byte:

```
use std::mem::size_of;
assert_eq!(size_of::<Ordering>(), 1);
assert_eq!(size_of::<HttpStatus>(), 2); // 404 não cabe em um u8
```

Você pode substituir a escolha de representação na memória do Rust adicionando um atributo `#[repr]` para a enumeração. Para detalhes, consulte “Encontrando representações de dados comuns”, na página 747.

A conversão de uma enumeração estilo C para um número inteiro é permitida:

```
assert_eq!(HttpStatus::Ok as i32, 200);
```

No entanto, converter na outra direção, do número inteiro para a enumeração, não é permitido. Ao contrário de C e C++, o Rust garante que um valor de enum seja apenas um dos valores especificados na declaração `enum`. Uma conversão não verificada de um tipo inteiro para um tipo enum pode quebrar essa garantia, portanto não é permitida. Você pode escrever sua própria conversão verificada:

```
fn http_status_from_u32(n: u32) -> Option<HttpStatus> {
    match n {
        200 => Some(HttpStatus::Ok),
        304 => Some(HttpStatus::NotModified),
        404 => Some(HttpStatus::NotFound),
        ...
        _ => None,
    }
}
```

ou utilizar o crate `enum_primitive` (crates.io/crates/enum_primitive). Ele contém uma macro que gera automaticamente esse tipo de código de conversão para você.

Tal como acontece com structs, o compilador vai implementar recursos como o operador `==` para você, mas você tem de pedir:

```
#[derive(Copy, Clone, Debug, PartialEq, Eq)]
enum TimeUnit {
    Seconds, Minutes, Hours, Days, Months, Years,
}
```

Enums podem ter métodos, assim como structs:

```
impl TimeUnit {
```

```

/// Retorna o substantivo plural para essa unidade de tempo
fn plural(self) -> &'static str {
    match self {
        TimeUnit::Seconds => "seconds",
        TimeUnit::Minutes => "minutes",
        TimeUnit::Hours => "hours",
        TimeUnit::Days => "days",
        TimeUnit::Months => "months",
        TimeUnit::Years => "years",
    }
}
/// Retorna o substantivo singular para essa unidade de tempo
fn singular(self) -> &'static str {
    self.plural().trim_end_matches('s')
}
}

```

Mas isso já é o suficiente para enums no estilo C. O tipo mais interessante de enum no Rust é aquele cujas variantes contêm dados. Mostraremos como eles são armazenados na memória, como torná-los genéricos adicionando parâmetros de tipo e como construir estruturas de dados complexas a partir de enums.

Enums com dados

Alguns programas sempre precisam exibir datas e horários completos em milissegundos, mas, para a maioria dos aplicativos, é mais fácil utilizar uma aproximação, como “dois meses atrás”. Podemos escrever um enum para ajudar com isso, utilizando o enum definido anteriormente:

```

/// Um registro de data/hora que foi deliberadamente arredondado, então nosso
/// programa diz "6 months ago" em vez de "February 9, 2016, at 9:49 AM"
#[derive(Copy, Clone, Debug, PartialEq)]
enum RoughTime {
    InThePast(TimeUnit, u32),
    JustNow,
    InTheFuture(TimeUnit, u32),
}

```

Duas das variantes nesta enumeração, `InThePast` e `InTheFuture`, aceitam argumentos. Estes são chamados *variantes de tupla*. Como structs de tupla, esses construtores são funções que criam novos valores `RoughTime`:

```

let four_score_and_seven_years_ago =

```

```
RoughTime::InThePast(TimeUnit::Years, 4 * 20 + 7);
```

```
let three_hours_from_now =  
    RoughTime::InTheFuture(TimeUnit::Hours, 3);
```

Enums também podem ter *variantes de struct*, que contêm campos nomeados, assim como structs comuns:

```
enum Shape {  
    Sphere { center: Point3d, radius: f32 },  
    Cuboid { corner1: Point3d, corner2: Point3d },  
}  
  
let unit_sphere = Shape::Sphere {  
    center: ORIGIN,  
    radius: 1.0,  
};
```

Ao todo, o Rust tem três tipos de variantes de enum, ecoando os três tipos de struct que mostramos no capítulo anterior. Variantes sem dados correspondem a structs do tipo unidade. As variantes de tupla se parecem e funcionam exatamente como structs de tupla. As variantes struct têm chaves e campos nomeados. Uma única enumeração pode ter variantes de todos os três tipos:

```
enum RelationshipStatus {  
    Single,  
    InARelationship,  
    ItsComplicated(Option<String>),  
    ItsExtremelyComplicated {  
        car: DifferentialEquation,  
        cdr: EarlyModernistPoem,  
    },  
}
```

Todos os construtores e campos de uma enumeração compartilham a mesma visibilidade da própria enumeração.

Enums na memória

Na memória, enums com dados são armazenados como uma *tag* de inteiro pequeno, além de memória suficiente para conter todos os campos da maior variante. O campo tag é para uso interno do Rust. Ele informa qual construtor criou o valor e, portanto, quais campos ele possui.

A partir do Rust 1.56, `RoughTime` cabe em 8 bytes, conforme mostra a

Figura 10.1.

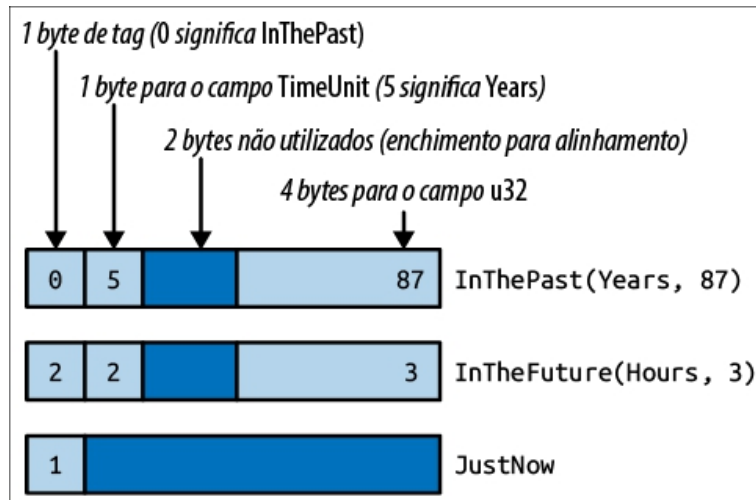


Figura 10.1: Valores `RoughTime` na memória.

O Rust não faz promessas sobre o layout de enum na memória, deixando a porta aberta para futuras otimizações. Em alguns casos, seria possível compactar uma enumeração com mais eficiência do que a figura sugere. Por exemplo, alguns structs genéricos podem ser armazenadas sem nenhuma tag, como veremos mais adiante.

Estruturas de dados ricas usando enums

Enums também são úteis para implementar rapidamente estruturas de dados do tipo árvore. Por exemplo, suponha que um programa Rust precise trabalhar com dados JSON arbitrários. Na memória, qualquer documento JSON pode ser representado como um valor deste tipo Rust:

```
use std::collections::HashMap;

enum Json {
    Null,
    Boolean(bool),
    Number(f64),
    String(String),
    Array(Vec<Json>),
    Object(Box<HashMap<String, Json>>),
}
```

A explicação dessa estrutura de dados em inglês não pode melhorar muito em relação ao código Rust. O padrão JSON especifica os

vários tipos de dados que podem aparecer em um documento JSON: null, valores booleanos, números, strings, arrays de valores JSON e objetos com chaves de string e valores JSON. O enum `Json` simplesmente explica esses tipos.

Esse não é um exemplo hipotético. Uma enumeração muito semelhante pode ser encontrada em `serde_json`, uma biblioteca de serialização para structs Rust que é um dos crates mais baixados em `crates.io`.

O `Box` em volta do `HashMap` que representa um `Object` serve apenas para tornar todos os valores `Json` mais compactos. Na memória, valores do tipo `Json` ocupam quatro palavras de máquina. Valores `String` e `Vec` têm três palavras e o Rust adiciona um byte de tag. Valores `Null` e `Boolean` não têm dados suficientes para utilizar todo esse espaço, mas todos os valores `Json` devem ser do mesmo tamanho. O espaço extra não é utilizado. A Figura 10.2 mostra alguns exemplos de como os valores `Json` realmente aparecem na memória.

Um `HashMap` é maior ainda. Se tivéssemos de deixar espaço para isso em cada valor `Json`, eles seriam bem grandes, oito palavras ou mais. Mas um `Box<HashMap>` é uma única palavra: é apenas um ponteiro para dados alocados no heap. Poderíamos tornar `Json` ainda mais compacto colocando mais campos em boxes.

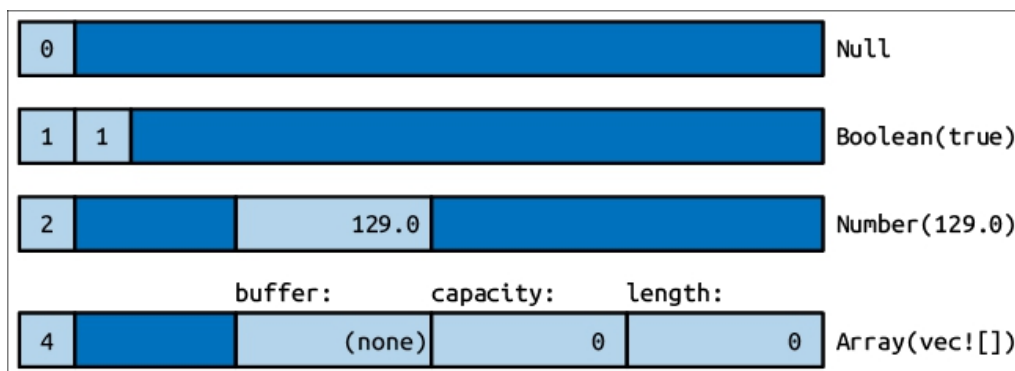


Figura 10.2: Valores `Json` na memória.

O que é notável aqui é como foi fácil configurar isso. Em C++, pode-se escrever uma classe assim:

```
class JSON {  
private:
```

```

enum Tag {
    Null, Boolean, Number, String, Array, Object
};
union Data {
    bool boolean;
    double number;
    shared_ptr<string> str;
    shared_ptr<vector<JSON>> array;
    shared_ptr<unordered_map<string, JSON>> object;

    Data() {}
    ~Data() {}

    ...
};

Tag tag;
Data data;

public:
    bool is_null() const { return tag == Null; }
    bool is_boolean() const { return tag == Boolean; }
    bool get_boolean() const {
        assert(is_boolean());
        return data.boolean;
    }
    void set_boolean(bool value) {
        this->~JSON(); // limpar valor string/array/object
        tag = Boolean;
        data.boolean = value;
    }
    ...
};

```

Com 30 linhas de código, mal começamos o trabalho. Essa classe precisará de construtores, um destruidor e um operador de atribuição. Uma alternativa seria criar uma hierarquia de classes com uma classe de base JSON e subclasses JSONBoolean, JSONString e assim por diante. De qualquer forma, quando terminar, nossa biblioteca C++ JSON terá mais de uma dúzia de métodos. Será necessário um pouco de leitura para outros programadores começarem a baixá-la e utilizá-la. Toda a enumeração do Rust tem oito linhas de código.

Enums genéricos

Enums podem ser genéricos. Dois exemplos da biblioteca padrão

estão entre os tipos de dados mais utilizados na linguagem:

```
enum Option<T> {  
    None,  
    Some(T),  
}  
  
enum Result<T, E> {  
    Ok(T),  
    Err(E),  
}
```

Esses tipos são bastante familiares agora e a sintaxe para enums genéricos é a mesma para structs genéricos.

Um detalhe nada óbvio é que o Rust pode eliminar o campo tag de `Option<T>` quando o tipo `T` for uma referência, `Box`, ou outro tipo de ponteiro inteligente. Como nenhum desses tipos de ponteiro pode ser zero, o Rust pode representar `Option<Box<i32>>`, digamos, como uma única palavra de máquina: 0 para `None` e diferente de zero para o ponteiro `Some`. Isso torna esses tipos `Option` análogos próximos dos valores de ponteiro em C ou C++, que podem ser nulos. A diferença é que o sistema de tipos do Rust exige que você verifique se um `Option` é `Some` antes de utilizar seu conteúdo. Isso efetivamente elimina desreferenciamento a ponteiros nulos.

Estruturas de dados genéricas podem ser construídas com apenas algumas linhas de código:

```
// Uma coleção ordenada de `T`s  
enum BinaryTree<T> {  
    Empty,  
    NonEmpty(Box<TreeNode<T>>),  
}  
  
// Uma parte de uma BinaryTree  
struct TreeNode<T> {  
    element: T,  
    left: BinaryTree<T>,  
    right: BinaryTree<T>,  
}
```

Essas poucas linhas de código definem um tipo `BinaryTree` que pode armazenar qualquer número de valores de tipo `T`.

Uma grande quantidade de informações está contida nessas duas definições, portanto dedicaremos um tempo para traduzir o código

palavra por palavra para o inglês. Cada valor `BinaryTree` é ou `Empty` ou `NonEmpty`. Se é `Empty`, então ele não contém nenhum dado. Se `NonEmpty`, então tem um `Box`, um ponteiro para um `TreeNode` alocado no heap.

Cada valor `TreeNode` contém um elemento real, bem como mais dois valores `BinaryTree`. Isso significa que uma árvore pode conter subárvores e, portanto, uma árvore `NonEmpty` pode ter qualquer número de descendentes.

Um esboço de um valor do tipo `BinaryTree<T>` é mostrado na Figura 10.3. Como com `Option<Box<T>>`, o Rust elimina o campo de tag, então um valor `BinaryTree` é apenas uma palavra de máquina.

Construir qualquer nó específico nesta árvore é simples:

```
use self::BinaryTree::*;
let jupiter_tree = NonEmpty(Box::new(TreeNode {
    element: "Jupiter",
    left: Empty,
    right: Empty,
}));
```

Árvores maiores podem ser construídas a partir de árvores menores:

```
let mars_tree = NonEmpty(Box::new(TreeNode {
    element: "Mars",
    left: jupiter_tree,
    right: mercury_tree,
}));
```

Naturalmente, essa atribuição transfere a posse de `jupiter_node` e `mercury_node` para seu novo nó pai.

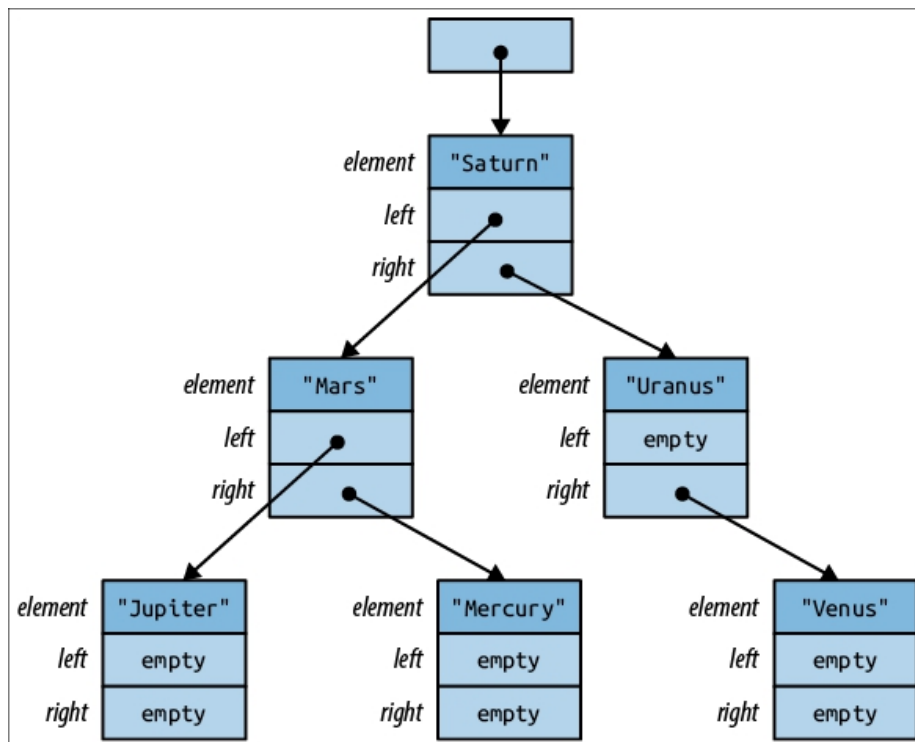


Figura 10.3: Um BinaryTree contendo seis strings.

As partes restantes da árvore seguem os mesmos padrões. O nó raiz não é diferente dos outros:

```

let tree = NonEmpty(Box::new(TreeNode {
    element: "Saturn",
    left: mars_tree,
    right: uranus_tree,
}));

```

Mais adiante neste capítulo, mostraremos como implementar um método `add` no tipo `BinaryTree` para que possamos escrever:

```

let mut tree = BinaryTree::Empty;
for planet in planets {
    tree.add(planet);
}

```

Não importa de qual linguagem você vem, criar estruturas de dados como `BinaryTree` no Rust provavelmente exigirá alguma prática. Não será óbvio a princípio onde colocar os `Boxes`. Uma maneira de encontrar um projeto que funcione é fazer um desenho como a Figura 10.3, que mostra como você deseja que as coisas sejam organizadas na memória. Em seguida, trabalhe de trás para a frente, da imagem para o código. Cada coleção de retângulos é um struct

ou tupla; cada flecha é um `Box` ou outro ponteiro inteligente. Descobrir o tipo de cada campo é um quebra-cabeça, mas administrável. A recompensa por resolver o quebra-cabeça é o controle sobre o uso de memória do seu programa.

Agora vem o “preço” que mencionamos na introdução. O campo `tag` de um `enum` custa um pouco de memória, até oito bytes no pior caso, mas isso geralmente é insignificante. A verdadeira desvantagem de `enums` (se é que pode ser chamado assim) é que o código `Rust` não pode arriscar e tentar acessar campos independentemente de estarem realmente presentes no valor:

```
let r = shape.radius; // erro: nenhum campo `radius` no tipo `Shape`
```

A única maneira de acessar os dados em um `enum` é a maneira segura: utilizando padrões.

Padrões

Lembre-se da definição de nosso tipo `RoughTime` do início deste capítulo:

```
enum RoughTime {  
    InThePast(TimeUnit, u32),  
    JustNow,  
    InTheFuture(TimeUnit, u32),  
}
```

Suponha que você tenha um valor `RoughTime` e queira exibi-lo em uma página da web. Você precisa acessar os campos `TimeUnit` e `u32` dentro do valor. O `Rust` não permite que você os acesse diretamente, escrevendo `rough_time.0` e `rough_time.1`, porque, afinal, o valor pode ser `RoughTime::JustNow`, que não tem campos. Mas então, como você pode obter os dados?

Você precisa de uma expressão `match`:

```
1 fn rough_time_to_english(rt: RoughTime) -> String {  
2     match rt {  
3         RoughTime::InThePast(units, count) =>  
4             format!("{}", {} ago", count, units.plural()),  
5         RoughTime::JustNow =>  
6             format!("just now"),  
7         RoughTime::InTheFuture(units, count) =>  
8             format!("{}", {} from now", count, units.plural()),
```

```
9 }  
10 }
```

`match` realiza correspondência de padrões; neste exemplo, os *padrões* são as partes que aparecem antes do símbolo `=>` nas linhas 3, 5 e 7. Padrões que combinam os valores se parecem com as expressões utilizadas para criar valores `RoughTime`. Isso não é coincidência. Expressões *produzem* valores; padrões *consomem* valores. Os dois utilizam muito da mesma sintaxe.

Vamos ver o que acontece quando essa expressão `match` é executada. Suponha que `rt` é o valor `RoughTime::InTheFuture(TimeUnit::Months, 1)`. O Rust primeiro tenta corresponder esse valor com o padrão na linha 3. Como você pode ver na Figura 10.4, ele não corresponde.

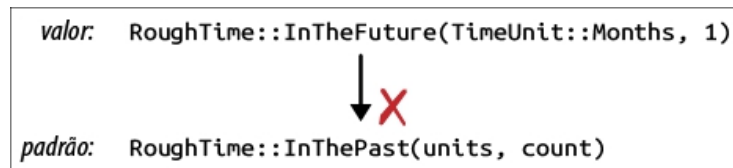


Figura 10.4: Um valor `RoughTime` e o padrão que não correspondem.

A correspondência de padrão de um enum, struct ou tupla funciona como se o Rust estivesse fazendo uma varredura simples da esquerda para a direita, verificando cada componente do padrão para ver se o valor corresponde a ele. Caso contrário, o Rust passa para o próximo padrão.

Os padrões nas linhas 3 e 5 não correspondem. Mas o padrão na linha 7 é bem-sucedido (Figura 10.5).

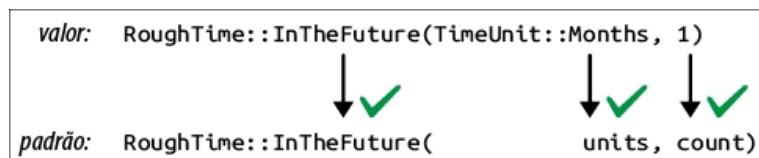


Figura 10.5: Uma correspondência bem-sucedida.

Quando um padrão contém identificadores simples como `units` e `count`, elas se tornam variáveis locais no código, seguindo o padrão. O que quer que esteja presente no valor é copiado ou movido para as novas variáveis. O Rust armazena `TimeUnit::Months` em `units` e `1` em `count`, executa a linha 8 e retorna a string `"1 months from now"`.

Essa saída tem um pequeno problema gramatical, que pode ser corrigido adicionando outro braço ao `match`:

```
RoughTime::InTheFuture(unit, 1) =>  
    format!("a {} from now", unit.singular()),
```

Esse braço corresponde apenas se o campo `count` é exatamente 1. Observe que esse novo código deve ser adicionado antes da linha 7. Se adicioná-lo no final, o Rust nunca chegará a ele, porque o padrão na linha 7 corresponde a todos valores `InTheFuture`. O compilador do Rust avisará sobre um “padrão inacessível” se você cometer esse tipo de erro.

Mesmo com o novo código, `RoughTime::InTheFuture(TimeUnit::Hours, 1)` ainda apresenta um problema: o resultado "a hour from now" não está certo. Assim é a língua inglesa. Isso também pode ser corrigido adicionando outro braço ao `match`.

Como mostra esse exemplo, a correspondência de padrões funciona de mãos dadas com enums e pode até testar os dados que eles contêm, tornando `match` um substituto poderoso e flexível para instrução `switch` do C. Até agora, vimos apenas padrões que correspondem a valores de enumeração. Há mais do que isso. Os padrões do Rust têm sua própria linguagem, resumida na Tabela 10.1. Passaremos a maior parte do restante do capítulo nos recursos mostrados nesta tabela.

Tabela 10.1: Padrões

Tipo de padrão	Exemplo	Notas
Literal	100 "name"	Corresponde a um valor exato; o nome de um <code>const</code> também é permitido
Intervalo	0 ..= 100 'a' ..= 'k' 256..	Corresponde a qualquer valor no intervalo, incluindo o valor final, se fornecido
Curinga	_	Corresponde a qualquer valor e o ignora
Variável	name mut count	Como _ mas move ou copia o valor para uma nova variável local
Variável ref	ref field ref mut field	Empresta uma referência ao valor correspondente em vez de movê-lo ou copiá-lo
Vinculação com subpadrão	val @ 0 ..= 99 ref circle @	Corresponde ao padrão à direita de @, utilizando o nome da variável à esquerda

Tipo de padrão	Exemplo	Notas
	Shape::Circle { .. }	
Padrão de enumeração	Some(value) None Pet::Orca	
Padrão de tupla	(key, value) (r, g, b)	
Padrão de array	[a, b, c, d, e, f, g] [heading, carom, correction]	
Padrão de fatia	[first, second] [first, _, third] [first, .., nth] []	
Padrão de struct	Color(r, g, b) Point { x, y } Card { suit: Clubs, rank: n } Account { id, name, .. }	
Referência	&value &(k, v)	Corresponde apenas aos valores de referência
Padrões Ou	'a' 'A' Some("left" "right")	
Expressão guarda	x if x * x <= r2	No match apenas (não é válido em let etc.)

Literais, variáveis e curingas em padrões

Até agora, mostramos expressões `match` trabalhando com enums. Outros tipos também podem ser combinados. Quando você precisa de algo como uma instrução `switch` do C, use `match` com um valor inteiro. Literais inteiros como 0 e 1 podem servir como padrões:

```
match meadow.count_rabbits() {
  0 => {} // nada a dizer
  1 => println!("A rabbit is nosing around in the clover."),
  n => println!("There are {} rabbits hopping about in the meadow", n),
}
```

O padrão 0 corresponde se não houver coelhos no campo. 1 corresponde se houver apenas um. Se houver dois ou mais coelhos, chegamos ao terceiro padrão, n. Esse padrão é apenas um nome de

variável. Ele pode corresponder a qualquer valor e o valor correspondente é movido ou copiado para uma nova variável local. Portanto, neste caso, o valor de `meadow.count_rabbits()` é armazenado em uma nova variável local `n`, que então imprimimos.

Outros literais também podem ser utilizados como padrões, incluindo booleanos, caracteres e até strings:

```
let calendar = match settings.get_string("calendar") {  
    "gregorian" => Calendar::Gregorian,  
    "chinese" => Calendar::Chinese,  
    "ethiopian" => Calendar::Ethiopian,  
    other => return parse_error("calendar", other),  
};
```

Neste exemplo, `other` serve como um padrão universal como `n` no exemplo anterior. Esses padrões desempenham o mesmo papel que um caso `default` em uma instrução `switch`, correspondendo a valores que não correspondem a nenhum dos outros padrões.

Se você precisa de um padrão genérico, mas não se importa com o valor correspondente, pode utilizar um único sublinhado `_` como padrão, o *padrão curinga*:

```
let caption = match photo.tagged_pet() {  
    Pet::Tyrannosaur => "RRRAAAAAHHHHHHH",  
    Pet::Samoyed => "*dog thoughts*",  
    _ => "I'm cute, love me", // legenda genérica, funciona para qq animal de estimação  
};
```

O padrão curinga corresponde a qualquer valor, mas sem armazená-lo em nenhum lugar. Como o Rust requer que toda expressão `match` trate todos os valores possíveis, geralmente é necessário um curinga no final. Mesmo se você tiver certeza de que os casos restantes não podem ocorrer, você deve pelo menos adicionar um braço alternativo, talvez um que gere um pânico:

```
// Existem muitos Shapes, mas oferecemos suporte apenas para "selecionar"  
// algum texto ou tudo em uma área retangular.  
// Você não pode selecionar uma elipse ou um trapézio.  
match document.selection() {  
    Shape::TextSpan(start, end) => paint_text_selection(start, end),  
    Shape::Rectangle(rect) => paint_rect_selection(rect),  
    _ => panic!("unexpected selection type"),  
}
```


Padrões com tupla e struct

Os padrões com tupla fazem a correspondência de tuplas. Eles são úteis sempre que você deseja obter vários dados envolvidos em um único match:

```
fn describe_point(x: i32, y: i32) -> &'static str {
    use std::cmp::Ordering::*;
    match (x.cmp(&0), y.cmp(&0)) {
        (Equal, Equal) => "at the origin",
        (_, Equal) => "on the x axis",
        (Equal, _) => "on the y axis",
        (Greater, Greater) => "in the first quadrant",
        (Less, Greater) => "in the second quadrant",
        _ => "somewhere else",
    }
}
```

Os padrões com struct utilizam chaves, assim como as expressões struct. Eles contêm um subpadrão para cada campo:

```
match balloon.location {
    Point { x: 0, y: height } =>
        println!("straight up {} meters", height),
    Point { x: x, y: y } =>
        println!("at ({}m, {}m)", x, y),
}
```

Neste exemplo, se o primeiro braço corresponder, então `balloon.location.y` é armazenado na nova variável local `height`.

Suponha que `balloon.location` seja `Point { x: 30, y: 40 }`. Como sempre, o Rust verifica cada componente de cada padrão por vez (Figura 10.6).

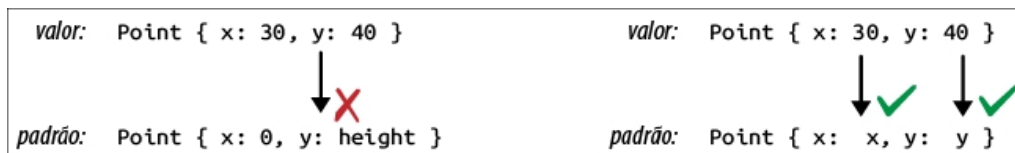


Figura 10.6: Correspondência de padrões com structs.

O segundo braço corresponde, então a saída seria `at (30m, 40m)`.

Padrões como `Point { x: x, y: y }` são comuns ao combinar structs e os nomes redundantes são visualmente desnecessários, então o Rust tem uma abreviação para isso: `Point {x, y}`. O significado é o mesmo.

Esse padrão ainda armazena o campo `x` de um ponto em uma variável local nova `x` e o seu campo `y` em uma variável local nova `y`.

Mesmo com a abreviação, é complicado combinar um struct grande quando nos preocupamos apenas com alguns campos:

```
match get_account(id) {
    ...
    Some(Account {
        name, language, // <--- as 2 coisas que nos preocupam
        id: _, status: _, address: _, birthday: _, eye_color: _,
        pet: _, security_question: _, hashed_innermost_secret: _,
        is_adamantium_preferred_customer: _, }) =>
        language.show_custom_greeting(name),
}
```

Para evitar isso, utilize `..` para dizer ao Rust que você não se importa com nenhum dos outros campos:

```
Some(Account { name, language, .. }) =>
    language.show_custom_greeting(name),
```

Padrões de array e fatia

Padrões de array fazem a correspondência com arrays. Eles geralmente são utilizados para filtrar alguns valores de casos especiais e são úteis sempre que você estiver trabalhando com arrays cujos valores têm um significado diferente com base na posição.

Por exemplo, ao converter valores de cor de matiz, saturação e luminosidade (HSL) em valores de cor vermelho, verde, azul (RGB), cores com luminosidade zero ou luminosidade total são apenas preto ou branco. Poderíamos utilizar uma expressão `match` para lidar com esses casos de forma simples.

```
fn hsl_to_rgb(hsl: [u8; 3]) -> [u8; 3] {
    match hsl {
        [_, _, 0] => [0, 0, 0],
        [_, _, 255] => [255, 255, 255],
        ...
    }
}
```

Os padrões de fatia são semelhantes, mas, diferentemente dos arrays, as fatias têm comprimentos variáveis, portanto os padrões de

fatia correspondem não apenas aos valores, mas também ao comprimento. .. em um padrão de fatia corresponde a qualquer número de elementos:

```
fn greet_people(names: &[&str]) {  
    match names {  
        [] => { println!("Hello, nobody.") },  
        [a] => { println!("Hello, {}. ", a) },  
        [a, b] => { println!("Hello, {} and {}. ", a, b) },  
        [a, .., b] => { println!("Hello, everyone from {} to {}. ", a, b) }  
    }  
}
```

Padrões de referência

Os padrões Rust oferecem suporte a dois recursos para trabalhar com referências. Os padrões `ref` emprestam partes de um valor correspondente. Os padrões `&` correspondem às referências. Abordaremos padrões `ref` primeiro.

A correspondência de um valor não copiável move o valor. Continuando com o exemplo da conta, esse código seria inválido:

```
match account {  
    Account { name, language, .. } => {  
        ui.greet(&name, &language);  
        ui.show_settings(&account); // erro: empréstimo de valor movido: `account`  
    }  
}
```

Aqui, os campos `account.name` e `account.language` são movidos para variáveis locais `name` e `language`. O restante de `account` é dropado. É por isso que não podemos emprestar uma referência a ele depois.

Se `name` e `language` fossem ambos os valores copiáveis, o Rust copiaria os campos em vez de movê-los e esse código ficaria bem. Mas suponha que esses são `Strings`. O que podemos fazer?

Precisamos de um tipo de padrão que *empresta* valores correspondentes em vez de movê-los. A palavra-chave `ref` faz exatamente isso:

```
match account {  
    Account { ref name, ref language, .. } => {  
        ui.greet(name, language);  
        ui.show_settings(&account); // ok  
    }  
}
```

```
}
}
```

Agora as variáveis locais `name` e `language` são referências aos campos correspondentes em `account`. Como a `account` está apenas sendo emprestada, não consumida, não há problema em continuar chamando métodos nela.

Você pode utilizar `ref mut` para emprestar referências `mut`:

```
match line_result {
    Err(ref err) => log_error(err), // `err` é &Error (ref compartilhada)
    Ok(ref mut line) => {           // `line` é &mut String (mut ref)
        trim_comments(line);       // modifique a String no lugar
        handle(line);
    }
}
```

O padrão `Ok(ref mut line)` corresponde a qualquer resultado de sucesso e toma emprestada uma referência `mut` ao valor de sucesso armazenado dentro dela.

O tipo oposto de padrão de referência é o padrão `&`. Um padrão começando com `&` corresponde a uma referência:

```
match sphere.center() {
    &Point3d { x, y, z } => ...
}
```

Neste exemplo, suponha que `sphere.center()` retorne uma referência a um campo privado de `sphere`, um padrão comum no Rust. O valor retornado é o endereço de um `Point3d`. Se o centro está na origem, então `sphere.center()` retorna `&Point3d { x: 0.0, y: 0.0, z: 0.0 }`.

A correspondência de padrões ocorre conforme mostrado na Figura 10.7.

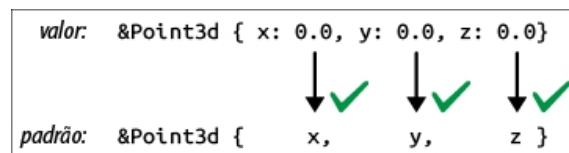


Figura 10.7: Correspondência de padrões com referências.

Isso é um pouco complicado porque o Rust está seguindo um ponteiro aqui, uma ação que geralmente associamos ao operador `*`, não ao operador `&`. O que deve ser lembrado é que padrões e

expressões são opostos naturais. A expressão `(x, y)` transforma dois valores em uma nova tupla, mas o padrão `(x, y)` faz o contrário: casa com uma tupla e separa os dois valores. É o mesmo com `&`. Em uma expressão, `&` cria uma referência. Em um padrão, `&` corresponde a uma referência.

A correspondência de uma referência segue todas as regras que esperamos. Os tempos de vida são aplicados. Você não pode obter acesso `mut` por meio de uma referência compartilhada. E você não pode mover um valor de uma referência, mesmo uma referência `mut`. Quando combinamos `&Point3d { x, y, z }`, as variáveis `x`, `y` e `z` recebem cópias das coordenadas, deixando o valor `Point3d` original intacto. Isso funciona porque esses campos são copiáveis. Se tentarmos a mesma coisa em um struct com campos não copiáveis, obteremos um erro:

```
match friend.borrow_car() {
    Some(&Car { engine, .. }) => // erro: não pode ser movido do empréstimo
    ...
    None => {}
}
```

Desmontar um carro emprestado para pegar as peças não é legal e o Rust não vai tolerar isso. Você pode utilizar um padrão `ref` para emprestar uma referência a uma parte. Você simplesmente não é o proprietário:

```
Some(&Car { ref engine, .. }) => // ok, o motor é uma referência
```

Vejamos mais um exemplo de um padrão `&`. Suponha que temos um iterador `chars` pelos caracteres em uma string e tem um método `chars.peek()` que retorna um `Option<&char>`: uma referência ao próximo caractere, se houver. (Iteradores `peekable`, de fato, retornam um `Option<&ItemType>`, como veremos no Capítulo 15.)

Um programa pode utilizar um padrão `&` para obter o caractere apontado:

```
match chars.peek() {
    Some(&c) => println!("coming up: {:?}", c),
    None => println!("end of chars"),
}
```

Guardas de correspondência

Às vezes, um braço do `match` tem condições adicionais que devem ser atendidas antes de ser considerado uma correspondência. Suponha que estamos implementando um jogo de tabuleiro com espaços hexagonais e o jogador apenas clicou para mover uma peça. Para confirmar que o clique foi válido, podemos tentar algo como isto:

```
fn check_move(current_hex: Hex, click: Point) -> game::Result<Hex> {
    match point_to_hex(click) {
        None =>
            Err("That's not a game space."),
        Some(current_hex) => // tenta corresponder se o usuário clicou no current_hex
                            // (não funciona: veja a explicação abaixo)
            Err("You are already there! You must click somewhere else."),
        Some(other_hex) =>
            Ok(other_hex)
    }
}
```

Isso falha porque os identificadores em padrões introduzem *novas* variáveis. O padrão `Some(current_hex)` aqui cria uma nova variável local `current_hex`, escondendo o argumento `current_hex`. O Rust emite vários avisos sobre esse código – em particular, o último braço do `match` é inalcançável. Uma maneira de corrigir isso é simplesmente utilizar uma expressão `if` no braço de correspondência:

```
match point_to_hex(click) {
    None => Err("That's not a game space."),
    Some(hex) => {
        if hex == current_hex {
            Err("You are already there! You must click somewhere else")
        } else {
            Ok(hex)
        }
    }
}
```

Mas o Rust também fornece *guardas de correspondência*, condições extras que devem ser verdadeiras para que um braço de correspondência seja aplicado, escrito como `if CONDITION`, entre o padrão e o token `=>` do braço:

```
match point_to_hex(click) {
    None => Err("That's not a game space."),
    Some(hex) if hex == current_hex =>
        Err("You are already there! You must click somewhere else"),
    Some(hex) => Ok(hex)
}
```

```
}
```

Se o padrão corresponder, mas a condição for falsa, a correspondência continua com o próximo braço.

Combinando múltiplas possibilidades

Um padrão na forma *pat1* | *pat2* corresponde se qualquer subpadrão corresponder:

```
let at_end = match chars.peek() {  
    Some(&'r' | &'n') | None => true,  
    _ => false,  
};
```

Em uma expressão, | é o operador OR bit a bit, mas aqui funciona mais como o símbolo | em uma expressão regular. *at_end* está configurado como *true* se *chars.peek()* for *None*, ou um *Some* armazenando um caractere de retorno de carro ou de nova linha.

Utilize *..=* para fazer a correspondência com todo um intervalo de valores. Os padrões de intervalo incluem os valores inicial e final, portanto *'0' ..= '9'* corresponde a todos os dígitos ASCII:

```
match next_char {  
    '0' ..= '9' => self.read_number(),  
    'a' ..= 'z' | 'A' ..= 'Z' => self.read_word(),  
    ' ' | '\t' | '\n' => self.skip_whitespace(),  
    _ => self.handle_punctuation(),  
}
```

O Rust também permite padrões de intervalo como *x..*, que correspondem a qualquer valor de *x* até o valor máximo do tipo. No entanto, as outras variedades de intervalos não inclusivos, como *0..100* ou *..100*, e intervalos ilimitados como *..* ainda não são permitidos em padrões.

Vinculando com padrões @

Por fim, *x @ pattern* corresponde exatamente ao *pattern* especificado, mas em caso de sucesso, em vez de criar variáveis para partes do valor correspondente, ele cria uma única variável *x* e move ou copia todo o valor para ela. Por exemplo, digamos que você tenha este código:

```

match self.get_selection() {
    Shape::Rect(top_left, bottom_right) => {
        optimized_paint(&Shape::Rect(top_left, bottom_right))
    }
    other_shape => {
        paint_outline(other_shape.get_outline())
    }
}

```

Observe que o primeiro caso descompacta um valor `Shape::Rect`, apenas para reconstruir um idêntico valor `Shape::Rect` na próxima linha. Isso pode ser reescrito para utilizar um padrão `@`:

```

rect @ Shape::Rect(..) => {
    optimized_paint(&rect)
}

```

Padrões `@` também são úteis com intervalos:

```

match chars.next() {
    Some(digit @ '0'..'9') => read_number(digit, chars),
    ...
},

```

Onde os padrões são permitidos

Embora os padrões sejam mais proeminentes em expressões `match`, eles também são permitidos em vários outros lugares, geralmente no lugar de um identificador. O significado é sempre o mesmo: em vez de apenas armazenar um valor em uma única variável, o Rust utiliza correspondência de padrões para separar o valor.

Isso significa que os padrões podem ser utilizados para...

```

// ... descompactar um struct em três novas variáveis locais
let Track { album, track_number, title, .. } = song;

```

```

// ... desempacotar um argumento de função que é uma tupla
fn distance_to((x, y): (f64, f64)) -> f64 { ... }

```

```

// ... iterar por chaves e valores de um HashMap
for (id, document) in &cache_map {
    println!("Document #{}: {}", id, document.title);
}

```

```

// ... desreferenciar automaticamente um argumento para uma closure
// (útil porque às vezes outro código passa uma referência para você
// quando você prefere ter uma cópia)
let sum = numbers.fold(0, |a, &num| a + num);

```


Cada um deles economiza duas ou três linhas de código básico que você não precisa escrever. O mesmo conceito existe em algumas outras linguagens: em JavaScript, é chamado de *desestruturar*, enquanto no Python é *desempacotar*.

Observe que, em todos os quatro exemplos, utilizamos padrões com correspondência garantida. O padrão `Point3d { x, y, z }` corresponde a todos os valores possíveis do tipo `struct, Point3d`, `(x, y)` corresponde a qualquer par `(f64, f64)` e assim por diante. Padrões que sempre combinam são especiais no Rust. Eles são chamados *padrões irrefutáveis* e são os únicos padrões permitidos nos quatro lugares mostrados aqui (após `let`, em argumentos de função, após `for`, e em argumentos de closure).

Um *padrão refutável* é aquele que pode não corresponder, como `Ok(x)`, que não corresponde a um resultado de erro ou `'0' ..= '9'`, que não corresponde ao caractere `'Q'`. Padrões refutáveis podem ser utilizados em braços `match`, porque `match` foi projetado para eles: se um padrão não corresponder, fica claro o que acontecerá a seguir. Os quatro exemplos anteriores são locais em programas Rust onde um padrão pode ser útil, mas a linguagem não permite falhas de correspondência.

Padrões refutáveis também são permitidos em expressões `if let` e `while let` que podem ser utilizadas para...

```
// ... tratar apenas uma variante de enumeração especialmente
if let RoughTime::InTheFuture(_, _) = user.date_of_birth() {
    user.set_time_traveler(true);
}

// ... executar algum código somente se uma pesquisa de tabela for bem-sucedida
if let Some(document) = cache_map.get(&id) {
    return send_cached_response(document);
}

// ...tentar repetidamente algo até conseguir
while let Err(err) = present_cheesy_anti_robot_task() {
    log_robot_attempt(err);
    // deixa o usuário tentar novamente (ainda pode ser um humano)
}

// ...fazer um loop manualmente sobre um iterador
while let Some(_) = lines.peek() {
```

```
    read_paragraph(&mut lines);  
}
```

Para obter detalhes sobre essas expressões, consulte “if let”, na página 173, e “Loops”, na página 173.

Preenchendo uma árvore binária

Anteriormente, prometemos mostrar como implementar um método, `BinaryTree::add()`, que adiciona um nó a um `BinaryTree` deste tipo:

```
// Uma coleção ordenada de `T`s.  
enum BinaryTree<T> {  
    Empty,  
    NonEmpty(Box<TreeNode<T>>),  
}  
  
// Uma parte de um BinaryTree.  
struct TreeNode<T> {  
    element: T,  
    left: BinaryTree<T>,  
    right: BinaryTree<T>,  
}
```

Agora você sabe o suficiente sobre padrões para escrever esse método. Uma explicação sobre árvores de busca binária está além do escopo deste livro, mas para os leitores já familiarizados com o tópico, vale a pena ver como funciona no Rust.

```
1 impl<T: Ord> BinaryTree<T> {  
2     fn add(&mut self, value: T) {  
3         match *self {  
4             BinaryTree::Empty => {  
5                 *self = BinaryTree::NonEmpty(Box::new(TreeNode {  
6                     element: value,  
7                     left: BinaryTree::Empty,  
8                     right: BinaryTree::Empty,  
9                 })))  
10            }  
11            BinaryTree::NonEmpty(ref mut node) => {  
12                if value <= node.element {  
13                    node.left.add(value);  
14                } else {  
15                    node.right.add(value);  
16                }  
17            }  
18        }  
}
```

```
19     }  
20 }
```

A linha 1 diz ao Rust que estamos definindo um método em `BinaryTrees` de tipos ordenados. Essa é exatamente a mesma sintaxe que utilizamos para definir métodos em structs genéricos, explicadas em “Definindo métodos com impl”, na página 252.

Se a árvore `*self` existente estiver vazia, esse é o caso trivial. As linhas 5–9 executam, alterando a árvore `Empty` para uma `NonEmpty`. A chamada para `Box::new()` aqui aloca um novo `TreeNode` no heap. Quando terminamos, a árvore contém um elemento. Suas subárvores esquerda e direita são ambas `Empty`.

Se `*self` não estiver vazio, correspondemos ao padrão na linha 11:

```
BinaryTree::NonEmpty(ref mut node) => {
```

Esse padrão empresta uma referência mutável ao `Box<TreeNode<T>>`, para que possamos acessar e modificar os dados nesse nó da árvore. Essa referência é nomeada `node` e está no escopo da linha 12 à linha 16. Como já existe um elemento neste nó, o código deve chamar recursivamente `.add()` para adicionar o novo elemento à subárvore esquerda ou direita.

O novo método pode ser utilizado assim:

```
let mut tree = BinaryTree::Empty;  
tree.add("Mercury");  
tree.add("Venus");  
...
```

Visão geral

As enumerações do Rust podem ser novas na programação de sistemas, mas não são uma ideia nova. Viajando sob vários nomes que soam acadêmicos, como *tipos de dados algébricos*, eles são utilizados em linguagens de programação funcionais há mais de 40 anos. Não se sabe direito por que tão poucas outras linguagens na tradição C nunca os tiveram. Talvez seja simplesmente porque, para um projetista de linguagem de programação, combinar variantes, referências, mutabilidade e segurança de memória é extremamente desafiador. As linguagens de programação funcional

dispensam a mutabilidade. `unions` do C, por outro lado, têm variantes, ponteiros e mutabilidade – mas são tão espetacularmente inseguros que, mesmo em C, são o último recurso. O *borrow checker* do Rust é a mágica que torna possível combinar todos os quatro sem comprometer.

Programação é processamento de dados. Colocar os dados na forma certa pode ser a diferença entre um programa pequeno, rápido e elegante, e um emaranhado lento e gigantesco de fita adesiva e chamadas de método virtuais.

Esse é o endereço de `enums` do espaço do problema. Eles são uma ferramenta de design para colocar os dados na forma certa. Para casos em que um valor pode ser uma coisa, ou outra coisa, ou talvez nada, `enums` são melhores que hierarquias de classes em todos os eixos: mais rápido, mais seguro, menos código, mais fácil de documentar.

O fator limitante é a flexibilidade. Os usuários finais de uma enumeração não podem estendê-la para adicionar novas variantes. As variantes podem ser adicionadas apenas alterando a declaração `enum`. E, quando isso acontece, o código existente quebra. Cada expressão `match` que corresponde individualmente a cada variante da enumeração deve ser revisada – ela precisa de um novo braço para lidar com a nova variante. Em alguns casos, trocar flexibilidade por simplicidade é apenas bom senso. Afinal, não se espera que a estrutura do JSON mude. E, em alguns casos, visitar todos os usos de um `enum` quando ele muda é exatamente o que queremos. Por exemplo, quando um `enum` é utilizado em um compilador para representar os vários operadores de uma linguagem de programação, adicionar um novo operador *deve* envolver mexer em todo o código que manipula os operadores.

Mas às vezes mais flexibilidade é necessária. Para essas situações, o Rust possui `traits`, o tópico de nosso próximo capítulo.

CAPÍTULO 11

Traits e genéricos

[Um] cientista da computação tende a ser capaz de lidar com estruturas não uniformes – caso 1, caso 2, caso 3 – enquanto um matemático tende a querer um axioma unificador que governe todo um sistema.

– Donald Knuth

Uma das grandes descobertas da programação é que é possível escrever código que opere em valores de muitos tipos diferentes, *mesmo tipos que ainda não foram inventados*. Eis dois exemplos:

- `Vec<T>` é genérico: você pode criar um vetor de qualquer tipo de valor, incluindo tipos definidos em seu programa que os autores de `Vec` nunca anteciparam.
- Muitos tipos têm métodos `.write()`, incluindo `File` e `TcpStreamS`. Seu código pode pegar um escritor (`writer`)¹ por referência, qualquer escritor (`writer`), e enviar dados para ele. Seu código não precisa se preocupar com o tipo de escritor (`writer`). Posteriormente, se alguém adicionar um novo tipo de escritor (`writer`), seu código já o suportará.

Obviamente, esse recurso não é novidade no Rust. É chamado *polimorfismo* e foi a nova tecnologia de linguagens de programação da década de 1970. Até agora é efetivamente universal. O Rust oferece suporte a polimorfismo com dois recursos relacionados: traits e genéricos. Esses conceitos serão familiares para muitos programadores, mas o Rust adota uma nova abordagem inspirada nas typeclasses de Haskell.

Traits são as interfaces do Rust ou classes base abstratas. A princípio, eles se parecem com interfaces em Java ou C#. O trait para escrever bytes é chamado `std::io::Write` e sua definição na biblioteca padrão começa assim:

```

trait Write {
    fn write(&mut self, buf: &[u8]) -> Result<usize>;
    fn flush(&mut self) -> Result<()>;
    fn write_all(&mut self, buf: &[u8]) -> Result<()> { ... }
    ...
}

```

Esse trait oferece vários métodos; mostramos apenas os três primeiros.

Os tipos padrão `File` e `TcpStream` ambos implementam `std::io::Write`. Assim como `Vec<u8>`. Todos os três tipos fornecem métodos nomeados `.write()`, `.flush()` e assim por diante. O código que utiliza um escritor (writer) sem se importar com seu tipo se parece com isto:

```

use std::io::Write;

fn say_hello(out: &mut dyn Write) -> std::io::Result<()> {
    out.write_all(b"hello world\n");
    out.flush()
}

```

O tipo de `out` é `&mut dyn Write`, significando “uma referência mutável a qualquer valor que implemente o trait `Write`”. Podemos passar a `say_hello` uma referência mutável a qualquer valor:

```

use std::fs::File;
let mut local_file = File::create("hello.txt");
say_hello(&mut local_file); // funciona

let mut bytes = vec![];
say_hello(&mut bytes); // também funciona
assert_eq!(bytes, b"hello world\n");

```

Este capítulo começa mostrando como os traits são utilizados, como funcionam e como definir os seus próprios. Mas há mais traits do que sugerimos até agora. Nós os utilizaremos para adicionar métodos de extensão a tipos existentes, até mesmo tipos internos como `str` e `bool`. Explicaremos por que adicionar um trait a um tipo não custa memória extra e como utilizar traits sem overhead de chamada de método virtual. Veremos que os traits internos são o gancho para a linguagem que o Rust fornece para sobrecarga de operador e outros recursos. E vamos abordar o tipo `Self`, funções associadas e tipos associados, três recursos que o Rust extraiu do Haskell que resolvem elegantemente problemas que outras

linguagens abordam com soluções alternativas e hacks.

Genéricos são o outro tipo de polimorfismo no Rust. Como um template em C++, uma função ou tipo genérico pode ser utilizado com valores de muitos tipos diferentes:

```
/// Dados dois valores, escolha o que for menor
fn min<T: Ord>(value1: T, value2: T) -> T {
    if value1 <= value2 {
        value1
    } else {
        value2
    }
}
```

O `<T: Ord>` nesta função significa que `min` pode ser utilizado com argumentos de qualquer tipo `T` que implementa o trait `Ord` – ou seja, qualquer tipo ordenado. Um requisito como esse é chamado de limite, porque define limites sobre quais tipos `T` possivelmente poderia ser. O compilador gera código de máquina personalizado para cada tipo `T` que você realmente utiliza.

Genéricos e traits estão intimamente relacionados: funções genéricas utilizam traits em limites para escrever explicitamente a quais tipos de argumentos eles podem ser aplicados. Também falaremos sobre como `&mut dyn Write` e `<T: Write>` são semelhantes, como são diferentes e como escolher entre essas duas formas de utilizar os traits.

Utilizando traits

Um trait é uma característica que qualquer tipo pode ou não suportar. Na maioria das vezes, um trait representa uma capacidade: algo que um tipo pode fazer.

- Um valor que implementa `std::io::Write` pode escrever bytes.
- Um valor que implementa `std::iter::Iterator` pode produzir uma sequência de valores.
- Um valor que implementa `std::clone::Clone` pode fazer clones de si mesmo na memória.
- Um valor que implementa `std::fmt::Debug` pode ser impresso utilizando `println!()` com o especificador de formato `{:?}`.

Esses quatro traits fazem parte da biblioteca padrão do Rust e muitos tipos padrão os implementam. Por exemplo:

- `std::fs::File` implementa o trait `Write`; ele escreve bytes em um arquivo local. `std::net::TcpStream` escreve em uma conexão de rede. `Vec<u8>` também implementa `Write`. Cada chamada de `.write()` sobre um vetor de bytes acrescenta alguns dados ao final.
- `Range<i32>` (o tipo de 0..10) implementa o trait `Iterator`, assim como alguns tipos de iteradores associados a fatias, tabelas hash e assim por diante.
- A maioria dos tipos de biblioteca padrão implementa `Clone`. As exceções são principalmente tipos como `TcpStream` que representam mais do que apenas dados na memória.
- Da mesma forma, a maioria dos tipos de biblioteca padrão suporta `Debug`.

Há uma regra incomum sobre métodos de trait: a própria trait deve estar no escopo. Caso contrário, todos os seus métodos estão ocultos:

```
let mut buf: Vec<u8> = vec![];
buf.write_all(b"hello"); // erro: nenhum método chamado `write_all`
```

Nesse caso, o compilador imprime uma mensagem de erro amigável que sugere adicionar `use std::io::Write;` e de fato isso corrige o problema:

```
use std::io::Write;

let mut buf: Vec<u8> = vec![];
buf.write_all(b"hello"); // ok
```

O Rust tem essa regra porque, como veremos mais adiante neste capítulo, você pode utilizar traits para adicionar novos métodos a qualquer tipo – até mesmo tipos de biblioteca padrão como `u32` e `str`. Crates de terceiros podem fazer a mesma coisa. Claramente, isso pode levar a conflitos de nomenclatura! Mas, como o Rust faz você importar os traits que planeja utilizar, os crates são livres para aproveitar esse superpoder. Para obter um conflito, você teria de importar dois traits que adicionam um método com o mesmo nome ao mesmo tipo. Isso é raro na prática. (No caso de se deparar com um conflito, você pode escrever explicitamente o que deseja

utilizando a sintaxe de método totalmente qualificada, abordada posteriormente neste capítulo.)

A razão por que os métodos `Clone` e `Iterator` funcionam sem nenhuma importação especial é que eles estão sempre no escopo por padrão: eles fazem parte do prelúdio padrão, nomes que o Rust importa automaticamente para cada módulo. Na verdade, o prelúdio é principalmente uma seleção cuidadosamente escolhida de traits. Abordaremos muitos deles no Capítulo 13.

Os programadores C++ e C# já devem ter notado que os métodos `trait` são como métodos virtuais. Ainda assim, chamadas como a mostrada acima são rápidas, tão rápidas quanto qualquer outra chamada de método. Simplificando, não há polimorfismo aqui. É óbvio que `buf` é um vetor, não um arquivo ou uma conexão de rede. O compilador pode emitir uma chamada simples para `Vec<u8>::write()`. Pode até mesmo colocar inline o método. (C++ e C# costumam fazer o mesmo, embora a possibilidade de subclasses às vezes impeça isso.) Apenas chamadas por meio de `&mut dyn Write` incorrem na sobrecarga de um despacho dinâmico, também conhecido como chamada de método virtual, que é indicado pela palavra-chave `dyn` no tipo. `dyn Write` é conhecido como um *objeto trait*; veremos os detalhes técnicos dos objetos `trait` e como eles se comparam às funções genéricas nas seções a seguir.

Objetos trait

Existem duas maneiras de utilizar traits para escrever código polimórfico no Rust: objetos `trait` e genéricos. Apresentaremos primeiro os objetos `trait` e passaremos aos genéricos na próxima seção.

O Rust não permite variáveis do tipo `dyn Write`:

```
use std::io::Write;
```

```
let mut buf: Vec<u8> = vec![];
```

```
let writer: dyn Write = buf; // erro: `Write` não tem um tamanho constante
```

O tamanho de uma variável deve ser conhecido em tempo de compilação e os tipos que implementam `Write` podem ser de qualquer

tamanho.

Isso pode ser surpreendente se você vier de C# ou Java, mas o motivo é simples. Em Java, uma variável do tipo `OutputStream` (a interface padrão Java análoga a `std::io::Write`) é uma referência a qualquer objeto que implementa `OutputStream`. O fato de ser uma referência é óbvio. É o mesmo com interfaces em C# e na maioria das outras linguagens.

O que queremos no Rust é a mesma coisa, mas no Rust as referências são explícitas:

```
let mut buf: Vec<u8> = vec![];  
let writer: &mut dyn Write = &mut buf; // ok
```

Uma referência a um tipo de `trait`, como `writer`, é chamado de objeto `trait`. Como qualquer outra referência, um objeto `trait` aponta para algum valor, tem um tempo de vida e pode ser `mut` ou compartilhado.

O que torna um objeto `trait` diferente é que o Rust geralmente não conhece o tipo do referente em tempo de compilação. Assim, um objeto `trait` inclui um pouco de informação extra sobre o tipo do referente. Isso é estritamente para uso próprio do Rust nos bastidores: quando você chama `writer.write(data)`, o Rust precisa das informações de tipo para chamar dinamicamente o método `write` dependendo do tipo de `*writer`. Você não pode consultar as informações de tipo diretamente e o Rust não oferece suporte ao downcasting do objeto `trait` `&mut dyn Write` de volta a um tipo concreto como `Vec<u8>`.

Layout do objeto trait

Na memória, um objeto `trait` é um ponteiro gordo que consiste em um ponteiro para o valor, mais um ponteiro para uma tabela que representa o tipo desse valor. Cada objeto `trait`, portanto, ocupa duas palavras de máquina, conforme mostrado na Figura 11.1.

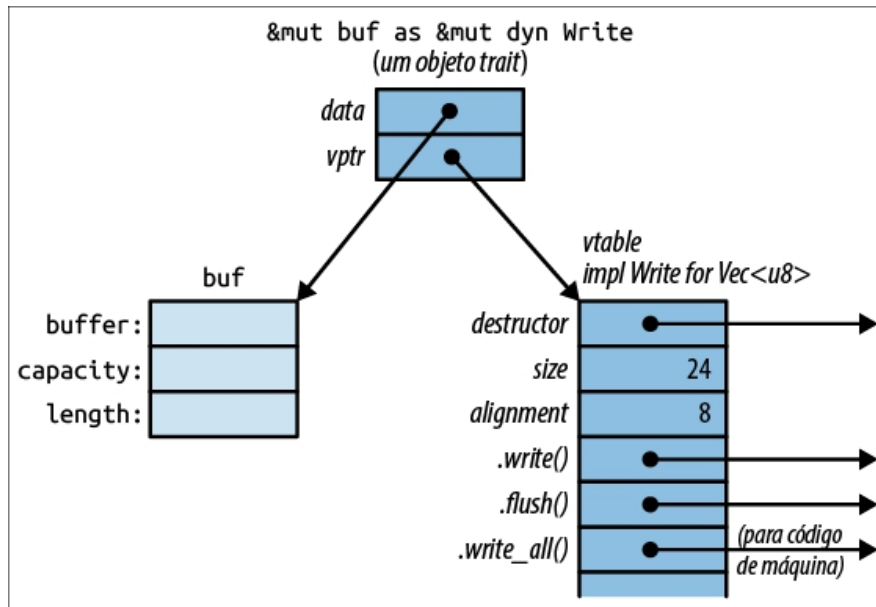


Figura 11.1: Objetos trait na memória.

O C++ também possui esse tipo de informação de tipo de tempo de execução. É chamado de tabela virtual ou vtable. No Rust, como em C++ a vtable é gerada uma vez, em tempo de compilação e compartilhada por todos os objetos do mesmo tipo. Tudo mostrado na sombra mais escura na Figura 11.1, incluindo o vtable, é um detalhe de implementação privada do Rust. Novamente, esses não são campos e estruturas de dados que você pode acessar diretamente. Em vez disso, a linguagem utiliza automaticamente a vtable quando você chama um método de um objeto trait, para determinar qual implementação chamar.

Programadores C++ experientes perceberão que o Rust e C++ utilizam a memória de maneira um pouco diferente. Em C++, o ponteiro vtable, ou vptr, é armazenado como parte do struct. Em vez disso, o Rust utiliza ponteiros gordos. O struct em si não contém nada além de seus campos. Dessa forma, um struct pode implementar dezenas de traits sem conter dezenas de vptrs. Mesmo tipos como `i32`, que não são grandes o suficiente para acomodar um vptr, podem implementar traits.

O Rust converte automaticamente referências comuns em objetos trait quando necessário. É por isso que somos capazes de passar `&mut local_file` para `say_hello` neste exemplo:

```
let mut local_file = File::create("hello.txt");  
say_hello(&mut local_file)?;
```

O tipo de `&mut local_file` é `&mut File` e o tipo do argumento para `say_hello` é `&mut dyn Write`. Como uma `File` é uma espécie de escritor, o Rust permite isso, convertendo automaticamente a referência simples em um objeto `trait`.

Da mesma forma, o Rust converterá um `Box<File>` para um `Box<dyn Write>`, um valor que possui um escritor no heap:

```
let w: Box<dyn Write> = Box::new(local_file);
```

`Box<dyn Write>`, como `&mut dyn Write`, é um ponteiro gordo: contém o endereço do próprio escritor e o endereço da `vtable`. O mesmo vale para outros tipos de ponteiro, como `Rc<dyn Write>`.

Esse tipo de conversão é a única maneira de criar um objeto `trait`. O que o compilador está realmente fazendo aqui é muito simples. No ponto em que a conversão acontece, o Rust conhece o verdadeiro tipo do referente (neste caso, `File`), então ele apenas adiciona o endereço da `vtable` apropriada, transformando o ponteiro regular em um ponteiro gordo.

Funções genéricas e parâmetros de tipo

No início deste capítulo, mostramos uma função `say_hello()` que tomou um objeto `trait` como um argumento. Vamos reescrever essa função como uma função genérica:

```
fn say_hello<W: Write>(out: &mut W) -> std::io::Result<()> {  
    out.write_all(b"hello world\n")?;  
    out.flush()  
}
```

Apenas a assinatura de tipo mudou:

```
fn say_hello(out: &mut dyn Write)    // função simples
```

```
fn say_hello<W: Write>(out: &mut W)  // função genérica
```

A frase `<W: Write>` é o que torna a função genérica. Esse é um parâmetro de tipo. Isso significa que, em todo o corpo dessa função, `w` representa algum tipo que implementa o `trait Write`. Parâmetros de tipo são geralmente letras maiúsculas únicas, por convenção.

Qual tipo `w` significa depende de como a função genérica é utilizada:

```
say_hello(&mut local_file)?; // chama say_hello::<File>
say_hello(&mut bytes)?;      // chama say_hello::<Vec<u8>>
```

Ao passar `&mut local_file` para a função genérica `say_hello()`, você está chamando `say_hello::<File>()`. O Rust gera código de máquina para essa função que chama `File::write_all()` e `File::flush()`. Ao passar `&mut bytes`, você está chamando `say_hello::<Vec<u8>>()`. O Rust gera código de máquina separado para essa versão da função, chamando os métodos `Vec<u8>` correspondentes. Em ambos os casos, o Rust infere o tipo `w` a partir do tipo do argumento. Esse processo é conhecido como monomorfização e o compilador lida com tudo isso automaticamente.

Você sempre pode escrever explicitamente os parâmetros de tipo:

```
say_hello::<File>(&mut local_file)?;
```

Isso raramente é necessário, porque o Rust em geral pode deduzir os parâmetros de tipo observando os argumentos. Aqui a função genérica `say_hello` espera um argumento `&mut W` e estamos passando um `&mut File`, então o Rust infere que `W = File`.

Se a função genérica que você está chamando não tiver nenhum argumento que forneça pistas úteis, talvez seja necessário deixar explícito:

```
// chamando um método genérico collect<C> () que não aceita argumentos
let v1 = (0 .. 1000).collect(); // erro: não é possível inferir o tipo
let v2 = (0 .. 1000).collect::<Vec<i32>>(); // ok
```

Às vezes, precisamos de várias habilidades de um parâmetro de tipo. Por exemplo, se quisermos imprimir os dez valores mais comuns em um vetor, precisaremos que esses valores sejam imprimíveis:

```
use std::fmt::Debug;
```

```
fn top_ten<T: Debug>(values: &Vec<T>) { ... }
```

Mas isso não é bom o suficiente. Como estamos planejando determinar quais valores são os mais comuns? A maneira usual é utilizar os valores como chaves em uma tabela hash. Isso significa que os valores precisam apoiar o `Hash` e `Eq` operações. Os limites em `T` devem incluir esses, bem como `Debug`. A sintaxe para isso utiliza o sinal `+`:

```
use std::hash::Hash;
use std::fmt::Debug;
```

```
fn top_ten<T: Debug + Hash + Eq>(values: &Vec<T>) { ... }
```

Alguns tipos implementam `Debug`, alguns implementam `Hash`, alguns suportam `Eq`, e alguns, como `u32` e `String`, implementam todos os três, conforme mostrado na Figura 11.2.

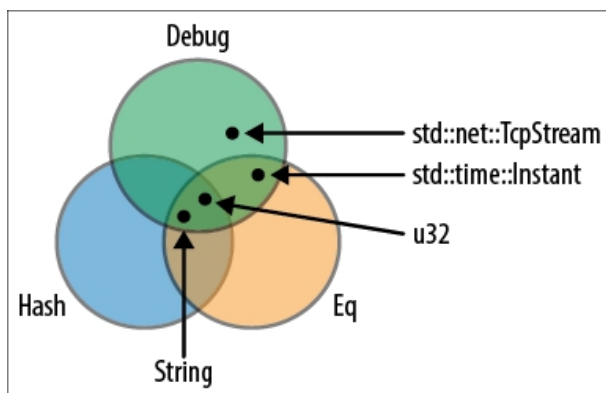


Figura 11.2: Traits como conjuntos de tipos.

Também é possível que um parâmetro de tipo não tenha nenhum limite, mas você não pode fazer muito com um valor se não tiver especificado nenhum limite para ele. Você pode movê-lo, pode colocá-lo em um box ou vetor. E isso é tudo que você pode fazer com o parâmetro de tipo sem limite.

As funções genéricas podem ter vários parâmetros de tipo:

```
/// Execute uma consulta em um conjunto de dados grande e particionado.
/// Ver <http://research.google.com/archive/mapreduce.html>.
fn run_query<M: Mapper + Serialize, R: Reducer + Serialize>(
    data: &DataSet, map: M, reduce: R) -> Results
{ ... }
```

Como esse exemplo mostra, os limites podem chegar a ser tão longos que são difíceis para os olhos. O Rust fornece uma sintaxe alternativa utilizando a palavra-chave `where`:

```
fn run_query<M, R>(data: &DataSet, map: M, reduce: R) -> Results
    where M: Mapper + Serialize,
          R: Reducer + Serialize
{ ... }
```

Os parâmetros de tipo `M` e `R` ainda são declarados na frente, mas os limites são movidos para linhas separadas. Esse tipo de cláusula `where` também é permitido em structs, enums, aliases de tipo e métodos genéricos – em qualquer lugar, limites são permitidos.

Naturalmente, uma alternativa a cláusulas `where` é mantê-las simples: encontre uma maneira de escrever o programa sem utilizar genéricos tão intensivamente.

“Recebendo referências como argumentos de função”, na página 142, introduziu a sintaxe para parâmetros de tempo de vida. Uma função genérica pode ter parâmetros de tempo de vida e parâmetros de tipo. Os parâmetros de tempo de vida vêm primeiro:

```
/// Retorne uma referência ao ponto em `candidates` que é
/// mais próximo do ponto `target`.
fn nearest<'t, 'c, P>(target: &'t P, candidates: &'c [P]) -> &'c P
  where P: MeasureDistance
{
  ...
}
```

Essa função recebe dois argumentos, `target` e `candidates`. Ambos são referências e damos a eles tempos de vida distintos `'t` e `'c` (conforme discutido em “Parâmetros de tempo de vida distintos”, na página 150). Além disso, a função funciona com qualquer tipo `P` que implementa o trait `MeasureDistance`, então podemos utilizá-la em valores `Point2d` em um programa e valores `Point3d` em outro.

Os tempos de vida nunca têm qualquer impacto no código de máquina. Duas chamadas a `nearest()` utilizando o mesmo tipo `P`, mas tempos de vida diferentes, chamarão a mesma função compilada. Somente tipos diferentes fazem com que o Rust compile várias cópias de uma função genérica.

Além de tipos e tempos de vida, as funções genéricas também podem receber parâmetros constantes, como o struct `Polynomial` que apresentamos em “Structs genéricos com parâmetros constantes”, na página 261:

```
fn dot_product<const N: usize>(a: [f64; N], b: [f64; N]) -> f64 {
  let mut sum = 0.;
  for i in 0..N {
    sum += a[i] * b[i];
  }
  sum
}
```

Aqui, a frase `<const N: usize>` indica que a função `dot_product` espera um

parâmetro genérico `N`, que deve ser um `usize`. Dado `N`, a função recebe dois argumentos do tipo `[f64; N]` e soma os produtos de seus elementos correspondentes. O que distingue `N` de um argumento `usize` comum é que você pode usá-lo nos tipos da assinatura e no corpo da função `dot_product`.

Assim como os parâmetros de tipo, você pode fornecer parâmetros constantes explicitamente ou deixar Rust inferi-los:

```
// Fornece explicitamente `3` como o valor para `N`  
dot_product::<3>([0.2, 0.4, 0.6], [0., 0., 1.])
```

```
// Deixa o Rust inferir que `N` deve ser `2`  
dot_product([3., 4.], [-5., 1.])
```

Naturalmente, as funções não são o único tipo de código genérico no Rust:

- Já cobrimos tipos genéricos em “Structs genéricos”, na página 257, e “Enums genéricos”, na página 278.
- Um método individual pode ser genérico, mesmo que o tipo em que está definido não seja genérico:

```
impl PancakeStack {  
    fn push<T: Topping>(&mut self, goop: T) -> PancakeResult<()> {  
        goop.pour(&self);  
        self.absorb_topping(goop)  
    }  
}
```

- Os aliases de tipo também podem ser genéricos:

```
type PancakeResult<T> = Result<T, PancakeError>;
```

- Abordaremos os traits genéricos mais adiante neste capítulo.

Todos os recursos introduzidos nesta seção – limites, cláusulas `where`, parâmetros de tempo de vida e assim por diante – podem ser utilizados em todos os itens genéricos, não apenas em funções.

Qual utilizar

A escolha de utilizar objetos `trait` ou código genérico é sutil. Como ambos os recursos são baseados em traits, eles têm muito em comum.

Objetos `trait` são a escolha certa sempre que você precisar de uma

coleção de valores de tipos mistos, todos juntos. É tecnicamente possível fazer uma salada genérica:

```
trait Vegetable {  
    ...  
}  
  
struct Salad<V: Vegetable> {  
    veggies: Vec<V>  
}
```

Contudo, esse é um design bastante rigoroso. Cada uma dessas saladas consiste inteiramente em um único tipo de vegetal. Nem todo mundo é talhado para esse tipo de coisa. Um de seus autores uma vez pagou US\$ 14 por uma `Salad<IcebergLettuce>` e nunca superou a experiência.

Como podemos construir uma salada melhor? Como valores `Vegetable` podem ser de tamanhos diferentes, não podemos pedir ao Rust um `Vec<dyn Vegetable>`:

```
struct Salad {  
    veggies: Vec<dyn Vegetable> // erro: `dyn Vegetal`  
                                // não tem um tamanho constante  
}
```

Objetos `trait` são a solução:

```
struct Salad {  
    veggies: Vec<Box<dyn Vegetable>>  
}
```

Cada `Box<dyn Vegetable>` pode possuir qualquer tipo de vegetal, mas a `box` em si tem um tamanho constante – dois ponteiros – adequado para armazenar em um vetor. Além da infeliz metáfora mista de ter caixas na comida de alguém, isso é exatamente o que é necessário e funcionaria tão bem para formas em um aplicativo de desenho, monstros em um jogo, algoritmos de roteamento conectáveis em um roteador de rede e assim por diante.

Outra razão possível para utilizar objetos `trait` é reduzir a quantidade total de código compilado. O Rust pode ter de compilar uma função genérica muitas vezes, uma vez para cada tipo com o qual é utilizado. Isso poderia tornar o binário grande, um fenômeno chamado inchaço do código em círculos de C++. Hoje em dia, a memória é abundante e a maioria de nós pode se dar ao luxo de

ignorar o tamanho do código; mas existem ambientes restritos.

Fora das situações que envolvem salads ou ambientes com poucos recursos, os genéricos têm três vantagens importantes sobre os objetos `trait`, com o resultado de que, no Rust, os genéricos são a escolha mais comum.

A primeira vantagem é a velocidade. Observe a ausência da palavra-chave `dyn` em assinaturas de função genérica. Como você especifica os tipos em tempo de compilação, explicitamente ou por meio de inferência de tipo, o compilador sabe exatamente qual método `write` chamar. A palavra-chave `dyn` não é utilizada porque não há objetos `trait` – e, portanto, nenhum despacho dinâmico – envolvido.

A função genérica `min()` mostrada na introdução é tão rápida como se tivéssemos escrito as funções `min_u8`, `min_i64`, `min_string` etc. separadamente. O compilador pode colocá-la inline, como qualquer outra função; portanto, em uma versão de release, uma chamada a `min::<i32>` tem provavelmente apenas duas ou três instruções. Uma chamada com argumentos constantes, como `min(5, 3)`, será ainda mais rápida: O Rust pode avaliá-lo em tempo de compilação, de modo que não haja nenhum custo de tempo de execução.

Ou considere esta chamada de função genérica:

```
let mut sink = std::io::sink();
say_hello(&mut sink)?;
```

`std::io::sink()` retorna um escritor do tipo `Sink`, que silenciosamente descarta todos os bytes escritos nele.

Quando o Rust gera código de máquina para isso, ele poderia emitir um código que chama `Sink::write_all`, verificar se há erros e, em seguida, chamar `Sink::flush`. Isso é o que o corpo da função genérica diz para fazer.

Ou o Rust pode olhar para esses métodos e perceber o seguinte:

- `Sink::write_all()` não faz nada.
- `Sink::flush()` não faz nada.
- Nenhum dos métodos retorna um erro.

Resumindo, o Rust tem todas as informações necessárias para otimizar totalmente essa chamada de função.

Compare isso com o comportamento de objetos trait. O Rust nunca sabe para qual tipo de valor um objeto trait aponta, até o tempo de execução. Portanto, mesmo que você passe um `Sink`, o overhead de chamar métodos virtuais e verificar erros ainda se aplica.

A segunda vantagem dos genéricos é que nem todo trait pode suportar objetos trait. Os traits suportam vários recursos, como funções associadas, que funcionam apenas com genéricos: eles excluem totalmente os objetos trait. Apontaremos esses recursos à medida que nos deparmos com eles.

A terceira vantagem dos genéricos é que é fácil vincular um parâmetro de tipo genérico com vários traits de uma só vez, como nossa função `top_ten` fez quando exigia seu parâmetro `T` para implementar `Debug + Hash + Eq`. Objetos trait não podem fazer isso: tipos como `&mut (dyn Debug + Hash + Eq)` não são suportados no Rust. (Você pode contornar isso com subtraits, definidos posteriormente neste capítulo, mas é um pouco complicado.)

Definindo e implementando traits

Definir um trait é simples. Dê um nome a ele e liste as assinaturas de tipo dos métodos do trait. Se estivermos escrevendo um jogo, podemos ter um trait como este:

```
/// Um trait para caracteres, itens e cenários -  
/// qualquer coisa no mundo do jogo que seja visível na tela  
trait Visible {  
    /// Renderiza esse objeto na tela fornecida  
    fn draw(&self, canvas: &mut Canvas);  
  
    /// Retorna verdadeiro se clicando em (x, y) deve  
    /// selecionar este objeto  
    fn hit_test(&self, x: i32, y: i32) -> bool;  
}
```

Para implementar um trait, utilize a sintaxe `impl TraitName for Type`:

```
impl Visible for Broom {  
    fn draw(&self, canvas: &mut Canvas) {  
        for y in self.y - self.height - 1 .. self.y {  
            canvas.write_at(self.x, y, '|');  
        }  
        canvas.write_at(self.x, self.y, 'M');  
    }  
}
```

```

    }

    fn hit_test(&self, x: i32, y: i32) -> bool {
        self.x == x
        && self.y - self.height - 1 <= y
        && y <= self.y
    }
}

```

Observe que esse `impl` contém uma implementação para cada método do `trait Visible` e nada mais. Com tudo definido em um `trait`, `impl` deve realmente ser um recurso do `trait`; se quiséssemos adicionar um método auxiliar para dar suporte a `Broom::draw()`, teríamos de defini-lo em um bloco `impl`:

```

impl Broom {
    /// Função auxiliar utilizada por Broom::draw() abaixo
    fn broomstick_range(&self) -> Range<i32> {
        self.y - self.height - 1 .. self.y
    }
}

```

Essas funções auxiliares podem ser utilizadas dentro dos blocos `impl` do `trait`:

```

impl Visible for Broom {
    fn draw(&self, canvas: &mut Canvas) {
        for y in self.broomstick_range() {
            ...
        }
        ...
    }
    ...
}

```

Métodos padrão

O tipo de escritor `Sink` que discutimos anteriormente pode ser implementado em algumas linhas de código. Primeiro, definimos o tipo:

```

/// Um Writer que ignora todos os dados que você escreve nele
pub struct Sink;

```

`Sink` é um `struct` vazio, pois não precisamos armazenar nenhum dado nele. Em seguida, apresentamos uma implementação do `trait Write` para `Sink`:

```

use std::io::{Write, Result};

impl Write for Sink {
    fn write(&mut self, buf: &[u8]) -> Result<usize> {
        // Afirma ter escrito com sucesso todo o buffer
        Ok(buf.len())
    }

    fn flush(&mut self) -> Result<()> {
        Ok(())
    }
}

```

Até agora, isso é muito parecido com o trait `Visible`. Mas também vimos que o trait `Write` tem um método `write_all`:

```

let mut out = Sink;
out.write_all(b"hello world\n");

```

Por que o Rust nos deixa `impl Write for Sink` sem definir esse método? A resposta é que a definição da biblioteca padrão do trait `Write` contém uma implementação padrão para `write_all`:

```

trait Write {
    fn write(&mut self, buf: &[u8]) -> Result<usize>;
    fn flush(&mut self) -> Result<()>;

    fn write_all(&mut self, buf: &[u8]) -> Result<()> {
        let mut bytes_written = 0;
        while bytes_written < buf.len() {
            bytes_written += self.write(&buf[bytes_written..]);
        }
        Ok(())
    }
}
...
}

```

Os métodos `write` e `flush` são os métodos básicos que todo escritor deve implementar. Um escritor também pode implementar `write_all`, mas, se não, a implementação padrão mostrada anteriormente será utilizada.

Seus próprios traits podem incluir implementações padrão utilizando a mesma sintaxe.

O uso mais dramático de métodos padrão na biblioteca padrão é o trait `Iterator`, que tem um método obrigatório (`.next()`) e dezenas de métodos padrão. O Capítulo 15 explica por quê.

Traits e tipos de outras pessoas

O Rust permite implementar qualquer trait em qualquer tipo, desde que o trait ou o tipo seja introduzido no crate atual.

Isso significa que, sempre que você quiser adicionar um método a qualquer tipo, poderá utilizar um trait para fazer isso:

```
trait IsEmoji {
    fn is_emoji(&self) -> bool;
}

/// Implementa IsEmoji como o tipo de caractere integrado
impl IsEmoji for char {
    fn is_emoji(&self) -> bool {
        ...
    }
}

assert_eq!('$'.is_emoji(), false);
```

Como qualquer outro método de trait, esse novo método `is_emoji` só é visível quando `IsEmoji` está no escopo.

O único propósito dessa trait em particular é adicionar um método a um tipo existente, `char`. Isso é chamado de trait de extensão. Naturalmente, você também pode adicionar esse trait aos tipos, escrevendo `impl IsEmoji for str { ... }` e assim por diante.

Você pode até utilizar um bloco `impl` genérico para adicionar um trait de extensão a toda uma família de tipos de uma só vez. Esse trait pode ser implementado em qualquer tipo:

```
use std::io::{self, Write};

/// Trait para valores para os quais você pode enviar HTML
trait WriteHtml {
    fn write_html(&mut self, html: &HtmlDocument) -> io::Result<()>;
}
```

A implementação do trait para todos os escritores o torna um trait de extensão, adicionando um método a todos os escritores Rust:

```
/// Você pode escrever HTML em qualquer escritor std::io
impl<W: Write> WriteHtml for W {
    fn write_html(&mut self, html: &HtmlDocument) -> io::Result<()> {
        ...
    }
}
```

A linha `impl<W: Write> WriteHtml for W` significa “para cada tipo `w` que implementa `Write`, eis uma implementação de `WriteHtml` para `w`”.

A biblioteca `serde` oferece um bom exemplo de como pode ser útil implementar traits definidos pelo usuário em tipos padrão. `serde` é uma biblioteca de serialização. Ou seja, você pode usá-lo para escrever estruturas de dados do Rust no disco e recarregá-las posteriormente. A biblioteca define um trait, `Serialize`, que é implementado para cada tipo de dados compatível com a biblioteca. Então, no código-fonte `serde`, há implementação do código `Serialize` para `bool`, `i8`, `i16`, `i32`, tipos de array e tupla etc., e todas as estruturas de dados padrão, como `Vec` e `HashMap`.

O resultado de tudo isso é que `serde` adiciona um método `.serialize()` para todos esses tipos. Pode ser utilizado assim:

```
use serde::Serialize;
use serde_json;

pub fn save_configuration(config: &HashMap<String, String>)
    -> std::io::Result<()>
{
    // Cria um serializador JSON para gravar os dados em um arquivo
    let writer = File::create(config_filename())?;
    let mut serializer = serde_json::Serializer::new(writer);

    // O método serde`serialize()` faz o resto
    config.serialize(&mut serializer)?;

    Ok(())
}
```

Dissemos anteriormente que, quando você implementa um trait, o trait ou o tipo deve ser novo no crate atual. Isso é chamado de regra órfã. Isso ajuda o Rust a garantir que as implementações de traits sejam únicas. Seu código não pode `impl Write` for `u8`, porque tanto `Write` como `u8` são definidos na biblioteca padrão. Se o Rust permitisse que os crates fizessem isso, poderia haver várias implementações de `Write` para `u8`, em crates diferentes, e o Rust não teria uma maneira razoável de decidir qual implementação utilizar para uma determinada chamada de método.

(O C++ tem uma restrição de unicidade semelhante: a Regra de uma definição. No estilo C++ típico, ela não é imposta pelo

compilador, exceto nos casos mais simples e você obtém um comportamento indefinido se a quebrar.)

Self em traits

Um trait pode utilizar a palavra-chave `self` como um tipo. O trait `Clone` padrão, por exemplo, se parece com isto (ligeiramente simplificado):

```
pub trait Clone {  
    fn clone(&self) -> Self;  
    ...  
}
```

Utilizar `self` como o tipo de retorno aqui significa que o tipo de `x.clone()` é igual ao tipo de `x`, seja lá o que for. Se `x` é um `String`, então o tipo de `x.clone()` é `String` — não `dyn Clone` ou qualquer outro tipo clonável.

Da mesma forma, se definirmos esse trait:

```
pub trait Spliceable {  
    fn splice(&self, other: &Self) -> Self;  
}
```

com duas implementações:

```
impl Spliceable for CherryTree {  
    fn splice(&self, other: &Self) -> Self {  
        ...  
    }  
}  
  
impl Spliceable for Mammoth {  
    fn splice(&self, other: &Self) -> Self {  
        ...  
    }  
}
```

então dentro do primeiro `impl`, `Self` é simplesmente um alias para `CherryTree` e, no segundo, é um alias para `Mammoth`. Isso significa que podemos unir duas cerejeiras ou dois mamutes, não que possamos criar um híbrido mamute-cereja. O tipo de `self` e o tipo de `other` devem coincidir.

Um trait que utiliza o tipo `self` é incompatível com objetos trait:

```
// erro: o trait `Splicable` não pode ser transformada em um objeto  
fn splice_anything(left: &dyn Spliceable, right: &dyn Spliceable) {  
    let combo = left.splice(right);  
    // ...  
}
```



```
}
```

A razão é algo que veremos repetidas vezes à medida que nos aprofundarmos nos recursos avançados dos traits. O Rust rejeita esse código porque não tem como verificar o tipo da chamada `left.splice(right)`. O objetivo dos objetos trait é que o tipo não é conhecido até o tempo de execução. O Rust não tem como saber em tempo de compilação se `left` e `right` serão do mesmo tipo, conforme necessário.

Os objetos trait são realmente destinados aos tipos mais simples de traits, os tipos que podem ser implementados utilizando interfaces em Java ou classes base abstratas em C++. Os recursos mais avançados de traits são úteis, mas não podem coexistir com objetos trait porque, com objetos trait, você perde as informações de tipo de que o Rust precisa para verificar o tipo do seu programa.

Agora, se quiséssemos um splicing geneticamente improvável, poderíamos ter projetado um trait amigável ao objeto:

```
pub trait MegaSpliceable {  
    fn splice(&self, other: &dyn MegaSpliceable) -> Box<dyn MegaSpliceable>;  
}
```

Esse trait é compatível com objetos trait. Não há problema em verificar o tipo de chamadas para esse método `.splice()` porque o tipo do argumento `other` não precisa corresponder ao tipo de `self`, desde que ambos os tipos sejam `MegaSpliceable`.

Subtraits

Podemos declarar que um trait é uma extensão de outro trait:

```
/// Alguém no mundo do jogo, seja o jogador ou algum outro  
/// duende, gárgula, esquilo, ogro etc.  
trait Creature: Visible {  
    fn position(&self) -> (i32, i32);  
    fn facing(&self) -> Direction;  
    ...  
}
```

A frase `trait Creature: Visible` significa que todas as criaturas são visíveis. Todo tipo que implementa `Creature` também deve implementar o trait `Visible`:

```
impl Visible for Broom {
    ...
}

impl Creature for Broom {
    ...
}
```

Podemos implementar os dois traits em qualquer ordem, mas é um erro implementar `Creature` para um tipo sem também implementar `Visible`. Aqui dizemos que `Creature` é um subtrait de `Visible` e que `Visible` é um supertrait de `Creature`.

Os subtraits se assemelham a subinterfaces em Java ou C#, pois os usuários podem presumir que qualquer valor que implemente um subtrait também implemente seu supertrait. Mas no Rust, um subtrait não herda os itens associados de seu supertrait; cada trait ainda precisa estar no escopo se você quiser chamar seus métodos.

Na verdade, os subtraits do Rust são apenas uma abreviação para um limite em `Self`. Uma definição de `Creature` assim é exatamente equivalente ao mostrado anteriormente:

```
trait Creature where Self: Visible {
    ...
}
```

Funções associadas a tipos

Na maioria das linguagens orientadas a objetos, as interfaces não podem incluir métodos ou construtores estáticos, mas os traits podem incluir funções associadas a tipos, análogos do Rust a métodos estáticos:

```
trait StringSet {
    /// Retornar um novo conjunto vazio
    fn new() -> Self;

    /// Retorna um conjunto que contém todas as strings em `strings`
    fn from_slice(strings: &[&str]) -> Self;

    /// Descubra se esse conjunto contém um `valor` específico
    fn contains(&self, string: &str) -> bool;

    /// Adicione uma string a esse conjunto
    fn add(&mut self, string: &str);
}
```

Todo tipo que implementa o trait `StringSet` deve implementar essas quatro funções associadas. Os dois primeiros, `new()` e `from_slice()`, não aceitam um argumento `self`. Eles servem como construtores. Em código não genérico, essas funções podem ser chamadas utilizando sintaxe `::`, assim como qualquer outra função associada ao tipo:

```
// Cria conjuntos de dois tipos hipotéticos que impl StringSet:
let set1 = SortedStringSet::new();
let set2 = HashedStringSet::new();
```

No código genérico, é o mesmo, exceto que o tipo geralmente é uma variável de tipo, como na chamada a `S::new()` mostrada aqui:

```
/// Retorna o conjunto de palavras em `document` que não estão em `wordlist`
fn unknown_words<S: StringSet>(document: &[String], wordlist: &S) -> S {
    let mut unknowns = S::new();
    for word in document {
        if !wordlist.contains(word) {
            unknowns.add(word);
        }
    }
    unknowns
}
```

Assim como as interfaces em Java e C#, os objetos trait não oferecem suporte a funções associadas a tipos. Se quiser utilizar objetos trait `&dyn StringSet`, você deve alterar o trait, adicionando o limite `where Self: Sized` a cada função associada que não recebe um argumento `self` por referência:

```
trait StringSet {
    fn new() -> Self
    where Self: Sized;

    fn from_slice(strings: &[&str]) -> Self
    where Self: Sized;

    fn contains(&self, string: &str) -> bool;

    fn add(&mut self, string: &str);
}
```

Esse limite diz ao Rust que os objetos trait são dispensados de oferecer suporte a essa função associada específica. Com essas adições, objetos trait `StringSet` são permitidos; eles ainda não suportam `new` nem `from_slice`, mas você pode criá-los e usá-los para chamar `.contains()` e `.add()`. O mesmo truque funciona para qualquer

outro método que seja incompatível com objetos `trait`. (Renunciaremos à explicação técnica bastante tediosa de por que isso funciona, mas o `trait Sized` é abordado no Capítulo 13.)

Chamadas de método totalmente qualificadas

Todas as maneiras de chamar métodos de `traits` que vimos até agora dependem do Rust preenchendo algumas peças que faltam para você. Por exemplo, suponha que você escreva o seguinte:

```
"hello".to_string()
```

É entendido que `to_string` referencia o método `to_string` do `trait ToString`, do qual estamos chamando a implementação do tipo `str`. Portanto, há quatro jogadores nesse jogo: o `trait`, o método desse `trait`, a implementação desse método e o valor ao qual essa implementação está sendo aplicada. É ótimo não termos de escrever explicitamente tudo isso toda vez que queremos chamar um método. Mas, em alguns casos, você precisa de uma maneira de dizer exatamente o que quer dizer. Chamadas de método totalmente qualificadas servem a esse propósito.

Em primeiro lugar, ajuda saber que um método é apenas um tipo especial de função. Essas duas chamadas são equivalentes:

```
"hello".to_string()
```

```
str::to_string("hello")
```

A segunda forma se parece exatamente com uma chamada de função associada. Isso funciona mesmo que o método `to_string` aceite um argumento `self`. Simplesmente passe `self` como o primeiro argumento da função.

Como `to_string` é um método do `trait ToString` padrão, existem mais duas formas que você pode utilizar:

```
ToString::to_string("hello")
```

```
<str as ToString>::to_string("hello")
```

Todas essas quatro chamadas de método chamam exatamente a mesma coisa. Na maioria das vezes, você apenas escreverá

`value.method()`. As outras formas são chamadas de método qualificadas. Elas especificam o tipo ou trait ao qual um método está associado. A última forma, com os símbolos de menor e maior, especifica ambos: uma chamada de método totalmente qualificada.

Quando você escreve `"hello".to_string()`, utilizando o operador `.`, não diz exatamente qual método `to_string` você está chamando. O Rust tem um algoritmo de pesquisa de método que descobre isso, dependendo dos tipos, coerções de deref e assim por diante. Com chamadas totalmente qualificadas, você pode dizer exatamente qual método deseja e isso pode ajudar em alguns casos estranhos:

- Quando dois métodos têm o mesmo nome. O exemplo dramático clássico é o `Outlaw` (fora da lei) com dois métodos `.draw()`² de dois traits diferentes, um para desenhar na tela e outro para interagir com a lei:

```
outlaw.draw(); // erro: desenhar na tela ou sacar a pistola?
```

```
Visible::draw(&outlaw); // ok: desenhar na tela
```

```
HasPistol::draw(&outlaw); // tudo bem: defenda-se
```

Normalmente, é melhor renomear um dos métodos, mas às vezes não é possível.

- Quando o tipo do argumento `self` não pode ser inferido:

```
let zero = 0; // tipo não especificado; pode ser `i8`, `u8`, ...
```

```
zero.abs(); // erro: não é possível chamar o método `abs`  
// no tipo numérico ambíguo
```

```
i64::abs(zero); // ok
```

- Ao utilizar a própria função como um valor de função:

```
let words: Vec<String> =  
    line.split_whitespace() // o iterador produz valores &str  
        .map(ToString::to_string) // ok  
        .collect();
```

- Ao chamar métodos de trait em macros. Explicaremos no Capítulo 21.

A sintaxe totalmente qualificada também funciona para funções associadas. Na seção anterior, escrevemos `S::new()` para criar um novo conjunto em uma função genérica. Poderíamos também ter escrito `StringSet::new()` OU `<S as StringSet>::new()`.

Traits que definem relacionamentos entre tipos

Até agora, todos os traits que examinamos são independentes: um trait é um conjunto de métodos que os tipos podem implementar. Os traits também podem ser utilizados em situações em que existem vários tipos que precisam trabalhar juntos. Eles podem descrever relacionamentos entre tipos.

- O trait `std::iter::Iterator` relaciona cada tipo de iterador com o tipo de valor que ele produz.
- O trait `std::ops::Mul` relaciona tipos que podem ser multiplicados. Na expressão `a * b`, os valores `a` e `b` podem ser do mesmo tipo ou de tipos diferentes.
- O crate `rand` inclui um trait para geradores de números aleatórios (`rand::Rng`) e um trait para tipos que podem ser gerados aleatoriamente (`rand::Distribution`). Os próprios traits definem exatamente como esses tipos funcionam juntos.

Você não precisará criar traits como esses todo dia, mas os encontrará em toda biblioteca padrão e em crates de terceiros. Nesta seção, mostraremos como cada um desses exemplos é implementado, selecionando os recursos relevantes da linguagem Rust conforme necessário. A principal habilidade aqui é a capacidade de ler traits e assinaturas de métodos e descobrir o que eles dizem sobre os tipos envolvidos.

Tipos associados (ou como funcionam os iteradores)

Começaremos com iteradores. Até agora, toda linguagem orientada a objetos tem algum tipo de suporte integrado para iteradores, objetos que representam o percurso de alguma sequência de valores.

O Rust tem um padrão trait `Iterator`, definido assim:

```
pub trait Iterator {  
    type Item;
```

```

    fn next(&mut self) -> Option<Self::Item>;
    ...
}

```

O primeiro recurso desse trait, `type Item`, é um tipo associado. Cada tipo que implementa `Iterator` deve especificar que tipo de item produz. O segundo recurso, o método `next()`, utiliza o tipo associado em seu valor de retorno. `next()` retorna um `Option<Self::Item>`: qualquer `Some(item)`, o próximo valor na sequência, ou `None` quando não houver mais valores para visitar. O tipo é escrito como `Self::Item`, não apenas `Item`, pois `Item` é um recurso de cada tipo de iterador, não um tipo autônomo. Como sempre, `self` e o tipo `Self` aparecem explicitamente no código em todos os lugares em que seus campos, métodos e assim por diante são utilizados.

Eis o que parece para implementar `Iterator` para um tipo:

```

// (código do módulo da biblioteca padrão std::env)
impl Iterator for Args {
    type Item = String;

    fn next(&mut self) -> Option<String> {
        ...
    }
    ...
}

```

`std::env::Args` é o tipo de iterador retornado pela função de biblioteca padrão `std::env::args()` que utilizamos no Capítulo 2 para acessar argumentos de linha de comando. Produz valores `String`, então a `impl` declara `type Item = String`.

O código genérico pode utilizar tipos associados:

```

/// Faça um loop por um iterador, armazenando os valores em um novo vetor.
fn collect_into_vector<I: Iterator>(iter: I) -> Vec<I::Item> {
    let mut results = Vec::new();
    for value in iter {
        results.push(value);
    }
    results
}

```

Dentro do corpo dessa função, o Rust infere o tipo de `value` para nós, o que é bom; mas devemos escrever explicitamente o tipo de retorno de `collect_into_vector`, e o tipo `Item` associado é a única maneira

de fazer isso. (`Vec<I>` seria simplesmente errado: estaríamos alegando retornar um vetor de iteradores!)

O exemplo anterior não é um código que você mesmo escreveria, porque, depois de ler o Capítulo 15, você saberá que os iteradores já possuem um método padrão que faz isso: `iter.collect()`. Então, vamos ver mais um exemplo antes de prosseguir:

```
/// Imprime todos os valores gerados por um iterador
fn dump<I>(iter: I)
  where I: Iterator
{
  for (index, value) in iter.enumerate() {
    println!("{:?}", index, value); // erro
  }
}
```

Isso quase funciona. Há apenas um problema: `value` pode não ser um tipo imprimível.

```
error: `<I as Iterator>::Item` doesn't implement `Debug`
|
8 |     println!("{:?}", index, value); // erro
|                               ^^^^^^
|                               `<I as Iterator>::Item` cannot be formatted
|                               using `{:?}` because it doesn't implement `Debug`
|
= help: the trait `Debug` is not implemented for `<I as Iterator>::Item`
= note: required by `std::fmt::Debug::fmt`
help: consider further restricting the associated type
5 |     where I: Iterator, <I as Iterator>::Item: Debug
|                               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

A mensagem de erro é ligeiramente ofuscada pelo uso da sintaxe `<I as Iterator>::Item` do Rust, que é uma maneira explícita, mas prolixa, de dizer `I::Item`. Essa é uma sintaxe Rust válida, mas você raramente precisará escrever um tipo dessa maneira.

A essência da mensagem de erro é que, para compilar essa função genérica, devemos garantir que `I::Item` implementa o `trait Debug`, o recurso para formatar valores com `{:?}`. Como sugere a mensagem de erro, podemos fazer isso colocando um limite em `I::Item`:

```
use std::fmt::Debug;

fn dump<I>(iter: I)
```



```

    where I: Iterator, I::Item: Debug
{
    ...
}

```

Ou, poderíamos escrever, “I deve ser um iterador sobre valores String”:

```

fn dump<I>(iter: I)
    where I: Iterator<Item=String>
{
    ...
}

```

`Iterator<Item=String>` é em si um trait. Se você pensar em `Iterator` como o conjunto de todos os tipos de iteradores, então `Iterator<Item=String>` é um subconjunto de `Iterator`: o conjunto de tipos de iteradores que produzem `Strings`. Essa sintaxe pode ser utilizada em qualquer lugar onde o nome de um trait pode ser utilizado, incluindo tipos de objeto trait:

```

fn dump(iter: &mut dyn Iterator<Item=String>) {
    for (index, s) in iter.enumerate() {
        println!("{:?}", index, s);
    }
}

```

Traits com tipos associados, como `Iterator`, são compatíveis com métodos de trait, mas somente se todos os tipos associados forem especificados, conforme mostrado aqui. Caso contrário, o tipo de `s` poderia ser qualquer coisa e, novamente, o Rust não teria como verificar o tipo desse código.

Mostramos muitos exemplos envolvendo iteradores. É difícil não mostrar, uma vez que iteradores são de longe o uso mais proeminente de tipos associados. Mas os tipos associados geralmente são úteis sempre que um trait precisa cobrir mais do que apenas métodos:

- Em uma biblioteca de pool de threads, um trait `Task`, representando uma unidade de trabalho, poderia ter um tipo `Output`.
- Um trait `Pattern`, representando uma forma de pesquisar uma string, pode ter um tipo `Match` associado, representando todas as

informações reunidas ao combinar o padrão com a string:

```
trait Pattern {  
    type Match;  
  
    fn search(&self, string: &str) -> Option<Self::Match>;  
}  
  
/// Você pode pesquisar uma string por um caractere específico  
impl Pattern for char {  
    /// Uma "correspondência" é apenas o local onde o  
    /// caractere foi encontrado  
    type Match = usize;  
  
    fn search(&self, string: &str) -> Option<usize> {  
        ...  
    }  
}
```

Se você estiver familiarizado com expressões regulares, é fácil ver como `impl Pattern for RegExp` teria um tipo `Match` mais elaborado, provavelmente um `struct` que incluiria o início e o comprimento da correspondência, os locais onde os grupos entre parênteses correspondiam, e assim por diante.

- Uma biblioteca para trabalhar com bancos de dados relacionais pode ter um `trait DatabaseConnection` com tipos associados que representam transações, cursores, instruções preparadas e assim por diante.

Tipos associados são perfeitos para casos em que cada implementação tem um tipo relacionado específico: cada tipo de `Task` produz um determinado tipo de `Output`; cada tipo de `Pattern` procura um determinado tipo de `Match`. No entanto, como veremos, alguns relacionamentos entre tipos não são assim.

Traits genéricos (ou como funciona a sobrecarga do operador)

A multiplicação no Rust utiliza este `trait`:

```
/// std::ops::Mul, o trait para tipos que suportam `*`  
pub trait Mul<RHS> {  
    /// O tipo resultante após a aplicação do operador `*`  
    type Output;
```

```

    /// O método para o operador `*`
    fn mul(self, rhs: RHS) -> Self::Output;
}

```

Mul é um trait genérico. O parâmetro de tipo, RHS, é a abreviação de *righthand side* (lado direito).

O parâmetro de tipo aqui significa a mesma coisa que significa em um struct ou função: Mul é um trait genérico e suas instâncias Mul<f64>, Mul<String>, Mul<Size> etc. são todos traits diferentes, assim como min::<i32> e min::<String> são funções diferentes e Vec<i32> e Vec<String> são tipos diferentes.

Um único tipo – digamos, WindowSize – pode implementar tanto Mul<f64> como Mul<i32> e muitos mais. Você então seria capaz de multiplicar um WindowSize por muitos outros tipos. Cada implementação teria seu próprio tipo Output associado.

Os traits genéricos recebem uma dispensa especial quando se trata da regra órfã: você pode implementar um trait externo para um tipo externo, desde que um dos parâmetros de tipo do trait seja um tipo definido no crate atual. Então, se você mesmo definiu WindowSize, pode implementar Mul<WindowSize> para f64, mesmo que também não tenha definido Mul ou f64. Essas implementações podem até ser genéricas, como impl<T> Mul<WindowSize> for Vec<T>. Isso funciona porque não há como nenhum outro crate definir Mul<WindowSize> em nada e, portanto, nenhum conflito entre as implementações poderia surgir. (Introduzimos a regra órfã em “Traits e tipos de outras pessoas”, na página 312.) É assim que crates como nalgebra definem operações aritméticas sobre vetores.

No trait mostrado anteriormente, falta um pequeno detalhe. O trait Mul real se parece com isto:

```

pub trait Mul<RHS=Self> {
    ...
}

```

A sintaxe RHS=Self significa que RHS assume Self por padrão. Se eu escrever impl Mul for Complex, sem especificar o parâmetro do tipo Mul, isso significa impl Mul<Complex> for Complex. Em um limite, se eu escrever where T: Mul, isso significa where T: Mul<T>.

No Rust, a expressão `lhs * rhs` é uma abreviação para `Mul::mul(lhs, rhs)`. Sobrecarregar assim o operador `*` no Rust é tão simples quanto implementar o trait `Mul`. Mostraremos exemplos no próximo capítulo.

impl Trait

Como você pode imaginar, combinações de muitos tipos genéricos podem se tornar confusas. Por exemplo, combinar apenas alguns iteradores utilizando combinadores da biblioteca padrão rapidamente transforma seu tipo de retorno em uma monstruosidade:

```
use std::iter;
use std::vec::IntoIter;
fn cyclical_zip(v: Vec<u8>, u: Vec<u8>) ->
    iter::Cycle<iter::Chain<IntoIter<u8>, IntoIter<u8>>> {
    v.into_iter().chain(u.into_iter()).cycle()
}
```

Poderíamos facilmente substituir esse tipo de retorno complicado por um objeto trait:

```
fn cyclical_zip(v: Vec<u8>, u: Vec<u8>) -> Box<dyn Iterator<Item=u8>> {
    Box::new(v.into_iter().chain(u.into_iter()).cycle())
}
```

Entretanto, assumir o overhead do despacho dinâmico e uma alocação de heap inevitável toda vez que essa função é chamada apenas para evitar uma assinatura de tipo feia não parece uma boa escolha, na maioria dos casos.

O Rust tem um recurso chamado `impl Trait` concebido precisamente para essa situação. `impl Trait` nos permite “apagar” o tipo de um valor de retorno, especificando apenas o trait ou traits que ele implementa, sem despacho dinâmico ou alocação de heap:

```
fn cyclical_zip(v: Vec<u8>, u: Vec<u8>) -> impl Iterator<Item=u8> {
    v.into_iter().chain(u.into_iter()).cycle()
}
```

Agora, em vez de especificar um tipo específico aninhado de iteradores de struct combinador, a assinatura de `cyclical_zip` apenas afirma que ele retorna algum tipo de iterador sobre `u8`. O tipo de retorno expressa a intenção da função, em vez de seus detalhes de implementação.

Isso definitivamente limpou o código e o tornou mais legível, mas `impl Trait` é mais do que apenas uma abreviação conveniente. Utilizar `impl Trait` significa que você pode alterar o tipo real que está sendo retornado no futuro, desde que ainda implemente `Iterator<Item=u8>`, e qualquer código que chame a função continuará a compilar sem problemas. Isso fornece muita flexibilidade para os autores da biblioteca, porque apenas a funcionalidade relevante é codificada na assinatura de tipo.

Por exemplo, se a primeira versão de uma biblioteca utiliza iteradores combinadores como anteriormente, mas um algoritmo melhor para o mesmo processo é descoberto, o autor da biblioteca pode utilizar diferentes combinadores ou até mesmo criar um tipo personalizado que implemente `Iterator` e os usuários da biblioteca podem obter as melhorias de desempenho sem alterar seu código.

Pode ser tentador utilizar `impl Trait` para fazer uma versão despachada estaticamente se aproximar do padrão `Factory` que é comumente utilizado em linguagens orientadas a objetos. Por exemplo, você pode definir um `trait` assim:

```
trait Shape {  
    fn new() -> Self;  
    fn area(&self) -> f64;  
}
```

Depois de implementá-lo para alguns tipos, você pode querer utilizar diferentes `shapes` dependendo de um valor de tempo de execução, como uma `string` que um usuário insere. Isso não funciona com `impl Shape` como o tipo de retorno:

```
fn make_shape(shape: &str) -> impl Shape {  
    match shape {  
        "circle" => Circle::new(),  
        "triangle" => Triangle::new(), // erro: tipos incompatíveis  
        "shape" => Rectangle::new(),  
    }  
}
```

Do ponto de vista do chamador, uma função como essa não faz muito sentido. `Impl Trait` é uma forma de despacho estático, então o compilador precisa saber o tipo que está sendo retornado da função em tempo de compilação para alocar a quantidade certa de espaço

no stack (pilha) e acessar corretamente os campos e métodos desse tipo. Aqui pode ser `Circle`, `Triangle`, ou `Rectangle`, que podem ocupar diferentes quantidades de espaço e todos têm diferentes implementações de `area()`.

É importante observar que o Rust não permite que métodos de traits utilizem valores de retorno `impl Trait`. O suporte a isso exigirá algumas melhorias no sistema de tipos da linguagem. Até que esse trabalho seja concluído, apenas funções livres e funções associadas a tipos específicos podem utilizar retornos `impl Trait`.

`impl Trait` também pode ser utilizado em funções que recebem argumentos genéricos. Por exemplo, considere esta função genérica simples:

```
fn print<T: Display>(val: T) {  
    println!("{}", val);  
}
```

É idêntica a esta versão utilizando `impl Trait`:

```
fn print(val: impl Display) {  
    println!("{}", val);  
}
```

Há uma exceção importante. O uso de genéricos permite que os chamadores da função especifiquem o tipo dos argumentos genéricos, como `print::<i32>(42)`, enquanto utilizar `impl Trait` não permite.

Cada argumento `impl Trait` recebe o próprio parâmetro de tipo anônimo, então `impl Trait` para argumentos é limitado apenas às funções genéricas mais simples, sem relacionamentos entre os tipos de argumentos.

Consts associadas

Como structs e enums, traits podem ter constantes associadas. Você pode declarar um trait com uma constante associada, utilizando a mesma sintaxe de um struct ou enum:

```
trait Greet {  
    const GREETING: &'static str = "Hello";  
    fn greet(&self) -> String;  
}
```

Mas consts associadas em traits têm um poder especial. Assim como

os tipos e funções associados, você pode declará-las, mas não atribuir um valor a elas:

```
trait Float {  
    const ZERO: Self;  
    const ONE: Self;  
}
```

Em seguida, os implementadores do trait podem definir estes valores:

```
impl Float for f32 {  
    const ZERO: f32 = 0.0;  
    const ONE: f32 = 1.0;  
}  
  
impl Float for f64 {  
    const ZERO: f64 = 0.0;  
    const ONE: f64 = 1.0;  
}
```

Isso permite que você escreva código genérico que utiliza esses valores:

```
fn add_one<T: Float + Add<Output=T>>(value: T) -> T {  
    value + T::ONE  
}
```

Observe que as constantes associadas não podem ser utilizadas com objetos trait, pois o compilador depende das informações de tipo sobre a implementação para escolher o valor correto em tempo de compilação.

Mesmo um trait simples sem nenhum comportamento, como `Float`, pode fornecer informações suficientes sobre um tipo, em combinação com alguns operadores, para implementar funções matemáticas comuns como Fibonacci:

```
fn fib<T: Float + Add<Output=T>>(n: usize) -> T {  
    match n {  
        0 => T::ZERO,  
        1 => T::ONE,  
        n => fib::<T>(n - 1) + fib::<T>(n - 2)  
    }  
}
```

Nas duas últimas seções, mostramos diferentes maneiras pelas quais os traits podem descrever relacionamentos entre tipos. Tudo isso também pode ser visto como uma forma de evitar overhead e

downcasts de métodos virtuais, já que permitem ao Rust conhecer tipos mais concretos em tempo de compilação.

Engenharia reversa de limites

Escrever um código genérico pode ser uma tarefa árdua quando não há um único trait que faz tudo o que você precisa. Suponha que tenhamos escrito essa função não genérica para fazer algum cálculo:

```
fn dot(v1: &[i64], v2: &[i64]) -> i64 {  
    let mut total = 0;  
    for i in 0 .. v1.len() {  
        total = total + v1[i] * v2[i];  
    }  
    total  
}
```

Agora queremos utilizar o mesmo código com valores de ponto flutuante. Podemos tentar algo assim:

```
fn dot<N>(v1: &[N], v2: &[N]) -> N {  
    let mut total: N = 0;  
    for i in 0 .. v1.len() {  
        total = total + v1[i] * v2[i];  
    }  
    total  
}
```

Mas não é possível: O Rust reclama do uso de `*` e do tipo de `0`. Podemos exigir que `N` seja um tipo que suporta `+` e `*` utilizando os traits `Add` e `Mul`. Nosso uso de `0` precisa mudar, porque `0` é sempre um número inteiro no Rust; o valor de ponto flutuante correspondente é `0.0`. Felizmente, existe um padrão trait `Default` para tipos que possuem valores padrão. Para tipos numéricos, o padrão é sempre `0`:

```
use std::ops::{Add, Mul};  
  
fn dot<N: Add + Mul + Default>(v1: &[N], v2: &[N]) -> N {  
    let mut total = N::default();  
    for i in 0 .. v1.len() {  
        total = total + v1[i] * v2[i];  
    }  
    total  
}
```


Isso está mais próximo, mas ainda não funciona bem:

error: mismatched types

```
5 | fn dot<N: Add + Mul + Default>(v1: &[N], v2: &[N]) -> N {
|     - this type parameter
```

```

...
8 |     total = total + v1[i] * v2[i];
  |           ^^^^^^^^^^^^^^^^^ expected type parameter `N`,
  |                               found associated type

```

```
= note: expected type parameter `N`
      found associated type `::Output`
```

help: consider further restricting this bound

```
5 | fn dot<N: Add + Mul + Default + Mul<Output = N>>(v1: &[N], v2: &[N]) -> N {
    | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Nosso novo código assume que a multiplicação de dois valores do tipo `N` produz outro valor do tipo `N`. Esse não é necessariamente o caso. Você pode sobrecarregar o operador de multiplicação para retornar qualquer tipo que desejar. Precisamos de alguma forma dizer ao Rust que essa função genérica só funciona com tipos que têm o tipo normal de multiplicação, em que multiplicar `N * N` retorna um `N`. A sugestão na mensagem de erro está quase certa: podemos fazer isso substituindo `Mul` por `Mul<Output=N>` e o mesmo para `Add`:

```
fn dot<N: Add<Output=N> + Mul<Output=N> + Default>(v1: &[N], v2: &[N]) -> N
{
    ...
}
```

Neste ponto, os limites estão começando a se acumular, tornando o código difícil de ler. Vamos mover os limites para uma cláusula `where`:

```
fn dot<N>(v1: &[N], v2: &[N]) -> N
  where N: Add<Output=N> + Mul<Output=N> + Default
{
  ...
}
```

Excelente. Mas o Rust ainda reclama desta linha de código:

error: cannot move out of type `[N]`, a non-copy slice

```
8 |         total = total + v1[i] * v2[i];
   |               ^^^^
```

```
|
|      cannot move out of here
|      move occurs because `v1[_]` has type `N`,
|      which does not implement the `Copy` trait
```

Como não exigimos que `N` seja um tipo copiável, o Rust interpreta `v1[i]` como uma tentativa de mover um valor para fora da fatia, o que é proibido. Mas não queremos modificar a fatia; queremos apenas copiar os valores para operar nelas. Felizmente, todos os tipos numéricos internos do Rust implementam `Copy`, então podemos simplesmente adicionar isso às nossas restrições em `N`:

```
where N: Add<Output=N> + Mul<Output=N> + Default + Copy
```

Com isso, o código compila e roda. O código final fica assim:

```
use std::ops::{Add, Mul};

fn dot<N>(v1: &[N], v2: &[N]) -> N
    where N: Add<Output=N> + Mul<Output=N> + Default + Copy
{
    let mut total = N::default();
    for i in 0 .. v1.len() {
        total = total + v1[i] * v2[i];
    }
    total
}

#[test]
fn test_dot() {
    assert_eq!(dot(&[1, 2, 3, 4], &[1, 1, 1, 1]), 10);
    assert_eq!(dot(&[53.0, 7.0], &[1.0, 5.0]), 88.0);
}
```

Isso ocasionalmente acontece no Rust: há um período de intensa discussão com o compilador, no final do qual o código parece bastante bom, como se tivesse sido fácil de escrever e funciona lindamente.

O que estamos fazendo aqui é a engenharia reversa dos limites `N`, utilizando o compilador para guiar e verificar nosso trabalho. A razão pela qual foi um pouco trabalhoso é que não havia um único trait `Number` na biblioteca padrão que incluísse todos os operadores e métodos que queríamos utilizar. Acontece que existe um crate de código aberto popular chamado `num` que define tal trait! Se soubéssemos, poderíamos ter adicionado `num` ao nosso *Cargo.toml* e escrito:

```

use num::Num;

fn dot<N: Num + Copy>(v1: &[N], v2: &[N]) -> N {
    let mut total = N::zero();
    for i in 0 .. v1.len() {
        total = total + v1[i] * v2[i];
    }
    total
}

```

Assim como na programação orientada a objetos, a interface certa torna tudo legal; na programação genérica, o trait certo torna tudo legal.

Ainda assim, por que se dar a todo esse trabalho? Por que os projetistas do Rust não fizeram os genéricos mais parecidos com os templates do C++, onde as restrições são deixadas implícitas no código, à la “tipagem pato”?

Uma vantagem da abordagem do Rust é a compatibilidade futura do código genérico. Você pode alterar a implementação de uma função ou método genérico público e, se não alterou a assinatura, não quebrou nenhum de seus usuários.

Outra vantagem dos limites é que, quando você obtém um erro do compilador, pelo menos o compilador pode lhe dizer onde está o problema. As mensagens de erro do compilador do C++ envolvendo templates podem ser muito mais longas que as do Rust, apontando para muitas linhas de código diferentes, porque o compilador não tem como dizer quem é o culpado por um problema: o template, ou seu chamador, que também pode ser um template, ou o chamador desse template...

Talvez a vantagem mais importante de escrever explicitamente os limites seja simplesmente que eles estão lá, no código e na documentação. Você pode olhar para a assinatura de uma função genérica no Rust e ver exatamente que tipo de argumentos ela aceita. O mesmo não pode ser dito para os templates. O trabalho necessário para documentar totalmente os tipos de argumento em bibliotecas C++ como o Boost é ainda mais árduo do que o que passamos aqui. Os desenvolvedores do Boost não têm um compilador que verifique seu trabalho.

Traits como uma fundação

Os traits são um dos principais recursos de organização no Rust e por uma boa razão. Não há nada melhor para projetar um programa ou biblioteca do que uma boa interface.

Este capítulo foi uma nevasca de sintaxe, regras e explicações. Agora que estabelecemos uma base, podemos começar a falar sobre as várias maneiras como traits e genéricos são utilizados no código Rust. O fato é que apenas começamos a arranhar a superfície. Os próximos dois capítulos abordam traits comuns fornecidas pela biblioteca padrão. Os próximos capítulos abordam closures, iteradores, entrada/saída e concorrência. Traits e genéricos desempenham um papel central em todos esses tópicos.

¹ N.R.: Os métodos que implementam o Write (escrita) são chamados em inglês de writers. Daí o termo escritor (writer) em português. Ou seja, um escritor é um código que implementa o trait Write.

² N.R.: O verbo to draw em inglês pode significar sacar uma arma ou desenhar. Daí a metáfora usada pelo autor ao explicar a ambiguidade de .draw() para um Outlaw (fora da lei, bandido) no contexto de um jogo.

CAPÍTULO 12

Sobrecarga de operador

No `plotter` de conjunto de Mandelbrot que mostramos no Capítulo 2, utilizamos o tipo `Complex` do `crate num` para representar um número no plano complexo:

```
#[derive(Clone, Copy, Debug)]
struct Complex<T> {
    /// Parte real do número complexo
    re: T,

    /// Parte imaginária do número complexo
    im: T,
}
```

Conseguimos somar e multiplicar números `Complex` como qualquer tipo numérico interno, utilizando os operadores `+` e `*` do Rust:

```
z = z * z + c;
```

Você pode fazer seus próprios tipos suportarem aritmética, bem como outros operadores, simplesmente implementando alguns `traits` internos. Isso se chama *sobrecarga de operador* e o efeito é muito parecido com a *sobrecarga de operador* em C++, C#, Python e Ruby.

Os `traits` de *sobrecarga de operador* se enquadram em algumas categorias, dependendo da parte da linguagem que elas suportam, conforme mostrado na Tabela 12.1. Neste capítulo, abordaremos cada categoria. Nosso objetivo não é apenas o ajudar a integrar bem seus próprios tipos na linguagem, mas também dar a você uma noção melhor de como escrever funções genéricas como a função de produto escalar descrita em “Engenharia reversa de limites”, na página 329, que operam nos tipos mais utilizados naturalmente por meio desses operadores. O capítulo também deve fornecer algumas dicas sobre como alguns recursos da própria linguagem são implementados.

Tabela 12.1: Resumo dos traits para sobrecarga de operador

Categoria	Trait	Operador
Operadores unários	std::ops::Neg	-x
	std::ops::Not	!x
Operadores aritméticos	std::ops::Add	x + y
	std::ops::Sub	x - y
	std::ops::Mul	x * y
	std::ops::Div	x / y
	std::ops::Rem	x % y
Operadores bit a bit	std::ops::BitAnd	x & y
	std::ops::BitOr	x y
	std::ops::BitXor	x ^ y
	std::ops::Shl	x << y
	std::ops::Shr	x >> y
Operadores aritméticos de atribuição composta	std::ops::AddAssign	x += y
	std::ops::SubAssign	x -= y
	std::ops::MulAssign	x *= y
	std::ops::DivAssign	x /= y
	std::ops::RemAssign	x %= y
Operadores bit a bit de atribuição composta	std::ops::BitAndAssign	x &= y
	std::ops::BitOrAssign	x = y
	std::ops::BitXorAssign	x ^= y
	std::ops::ShlAssign	x <<= y
	std::ops::ShrAssign	x >>= y
Comparação	std::cmp::PartialEq	x == y, x != y
	std::cmp::PartialOrd	x < y, x <= y, x > y, x >= y
Indexação	std::ops::Index	x[y], &x[y]
	std::ops::IndexMut	x[y] = z, &mut x[y]

Operadores aritméticos e de bit a bit

No Rust, a expressão `a + b` é, na realidade, uma abreviação para `a.add(b)`, uma chamada para o método `add` do trait `std::ops::Add` da biblioteca padrão. Todos os tipos numéricos padrão do Rust implementam `std::ops::Add`. Para fazer a expressão `a + b` funcionar para

valores `Complex`, o `crate num` implementa esse `trait` para `Complex` também. Traits semelhantes cobrem os outros operadores: `a * b` é uma abreviação para `a.mul(b)`, um método do `trait std::ops::Mul`, `std::ops::Neg` cobre o prefixo operador `-` de negação e assim por diante. Se quiser tentar escrever explicitamente `z.add(c)`, precisará trazer o `trait Add` ao escopo para que seu método seja visível. Feito isso, você pode tratar toda aritmética como chamadas de função:^{[1](#)}

```
use std::ops::Add;

assert_eq!(4.125f32.add(5.75), 9.875);
assert_eq!(10.add(20), 10 + 20);
```

Eis a definição de `std::ops::Add`:

```
trait Add<Rhs = Self> {
    type Output;
    fn add(self, rhs: Rhs) -> Self::Output;
}
```

Em outras palavras, o `trait Add<T>` é a capacidade de adicionar um valor `T` para si mesmo. Por exemplo, se você quiser adicionar valores `i32` e `u32` ao seu tipo, seu tipo deve implementar tanto `Add<i32>` como `Add<u32>`. O parâmetro de tipo do `trait Rhs` assume por padrão o valor de `Self`; portanto, se estiver implementando adição entre dois valores do mesmo tipo, basta escrever `Add` para esse caso. O tipo `Output` associado descreve o resultado da adição.

Por exemplo, para somar valores `Complex<i32>`, `Complex<i32>` deve implementar `Add<Complex<i32>>`. Como estamos adicionando um tipo a ele mesmo, apenas escrevemos `Add`:

```
use std::ops::Add;

impl Add for Complex<i32> {
    type Output = Complex<i32>;
    fn add(self, rhs: Self) -> Self {
        Complex {
            re: self.re + rhs.re,
            im: self.im + rhs.im,
        }
    }
}
```

Naturalmente, não deveríamos ter de implementar `Add` separadamente para `Complex<i32>`, `Complex<f32>`, `Complex<f64>` etc. Todas

as definições seriam exatamente iguais, exceto pelos tipos envolvidos, portanto deveríamos ser capazes de escrever uma única implementação genérica que abrangesse todas elas, desde que o tipo dos próprios componentes complexos suporte a adição:

```
use std::ops::Add;

impl<T> Add for Complex<T>
where
    T: Add<Output = T>,
{
    type Output = Self;
    fn add(self, rhs: Self) -> Self {
        Complex {
            re: self.re + rhs.re,
            im: self.im + rhs.im,
        }
    }
}
```

Escrevendo `where T: Add<Output=T>`, restringimos `T` a tipos que podem ser adicionados a si mesmos, gerando outro valor `T`. Essa é uma restrição razoável, mas poderíamos afrouxar ainda mais as coisas: o `trait Add` não requer que ambos os operandos `+` sejam do mesmo tipo, nem restringe o tipo de resultado. Portanto, uma implementação maximamente genérica permitiria que os operandos esquerdo e direito variassem independentemente e produziria um valor `Complex` de qualquer tipo de componente que a adição produz:

```
use std::ops::Add;

impl<L, R> Add<Complex<R>> for Complex<L>
where
    L: Add<R>,
{
    type Output = Complex<L::Output>;
    fn add(self, rhs: Complex<R>) -> Self::Output {
        Complex {
            re: self.re + rhs.re,
            im: self.im + rhs.im,
        }
    }
}
```

Na prática, porém, o Rust tende a evitar o suporte a operações de tipo misto. Como nosso parâmetro de tipo `L` deve implementar

Add<R>, geralmente se segue que L e R serão do mesmo tipo: simplesmente não há muitos tipos disponíveis para L que implementam qualquer outra coisa. Portanto, no final, essa versão maximamente genérica pode não ser muito mais útil do que a definição genérica anterior, mais simples.

Os traits internos do Rust para operadores aritméticos e de bit a bit dividem-se em três grupos: operadores unários, operadores binários e operadores de atribuição composta. Dentro de cada grupo, os traits e seus métodos têm todos a mesma forma, então abordaremos um exemplo de cada um.

Operadores unários

Além do operador de desreferenciação *, que abordaremos separadamente em “Deref e DerefMut”, na página 362, o Rust tem dois operadores unários que você pode personalizar, mostrados na Tabela 12.2.

Tabela 12.2: Traits internos para operadores unários

Nome do trait	Expressão	Expressão equivalente
std::ops::Neg	-x	x.neg()
std::ops::Not	!x	x.not()

Todos os tipos numéricos com sinal do Rust implementam std::ops::Neg, para o operador de negação unária - ; os tipos inteiros e bool implementam std::ops::Not, para o operador de complemento unário !. Também há implementações para referências a esses tipos.

Observe que ! complementa valores bool e produz um complemento bit a bit (ou seja, inverte os bits) quando aplicado a inteiros; ele desempenha o papel dos operadores ! e ~ de C e C++.

As definições desses traits são simples:

```
trait Neg {  
    type Output;  
    fn neg(self) -> Self::Output;  
}  
  
trait Not {
```

```

    type Output;
    fn not(self) -> Self::Output;
}

```

Negar um número complexo simplesmente nega cada um de seus componentes. Veja como podemos escrever uma implementação genérica de negação para valores `Complex`:

```

use std::ops::Neg;

impl<T> Neg for Complex<T>
where
    T: Neg<Output = T>,
{
    type Output = Complex<T>;
    fn neg(self) -> Complex<T> {
        Complex {
            re: -self.re,
            im: -self.im,
        }
    }
}

```

Operadores binários

Os operadores binários aritméticos e de bit a bit do Rust e seus traits internos correspondentes aparecem na Tabela 12.3.

Tabela 12.3: Traits internos para operadores binários

Categoria	Nome do trait	Expressão	Expressão equivalente
Operadores aritméticos	<code>std::ops::Add</code>	<code>x + y</code>	<code>x.add(y)</code>
	<code>std::ops::Sub</code>	<code>x - y</code>	<code>x.sub(y)</code>
	<code>std::ops::Mul</code>	<code>x * y</code>	<code>x.mul(y)</code>
	<code>std::ops::Div</code>	<code>x / y</code>	<code>x.div(y)</code>
	<code>std::ops::Rem</code>	<code>x % y</code>	<code>x.rem(y)</code>
Operadores bit a bit	<code>std::ops::BitAnd</code>	<code>x & y</code>	<code>x.bitand(y)</code>
	<code>std::ops::BitOr</code>	<code>x y</code>	<code>x.bitor(y)</code>
	<code>std::ops::BitXor</code>	<code>x ^ y</code>	<code>x.bitxor(y)</code>
	<code>std::ops::Shl</code>	<code>x << y</code>	<code>x.shl(y)</code>
	<code>std::ops::Shr</code>	<code>x >> y</code>	<code>x.shr(y)</code>

Todos os tipos numéricos do Rust implementam os operadores

aritméticos. Tipos inteiros e `bool` do Rust implementam os operadores de bit a bit. Também existem implementações que aceitam referências a esses tipos como um ou ambos os operandos.

Todos os traits aqui têm a mesma forma geral. A definição de `std::ops::BitXor`, para o operador `^`, fica assim:

```
trait BitXor<Rhs = Self> {  
    type Output;  
    fn bitxor(self, rhs: Rhs) -> Self::Output;  
}
```

No início deste capítulo, também mostramos `std::ops::Add`, outro trait nessa categoria, com várias implementações de exemplo.

Você pode utilizar o operador `+` para concatenar um `String` com uma fatia `&str` ou outra `String`. Contudo, o Rust não permite que o operando esquerdo de `+` seja uma `&str`, para desencorajar a construção de strings longas concatenando repetidamente pequenos pedaços à esquerda. (Isso funciona mal, exigindo tempo quadrático no comprimento final da string.) Em geral, a macro `write!` é melhor para construir strings peça por peça; mostramos como fazer isso em “Acrescentando e inserindo texto”, na página 499.

Operadores de atribuição composta

Uma expressão de atribuição composta é como `x += y` ou `x &= y`: ela pega dois operandos, executa alguma operação neles como adição ou um AND bit a bit e armazena o resultado de volta no operando esquerdo. No Rust, o valor de uma expressão de atribuição composta é sempre `()`, nunca o valor armazenado.

Muitas linguagens têm operadores como esses e geralmente os definem como atalhos para expressões como `x = x + y` ou `x = x & y`. Mas o Rust não adota essa abordagem. Em vez disso, `x += y` é uma abreviação para a chamada de método `x.add_assign(y)`, em que `add_assign` é o único método do trait `std::ops::AddAssign`:

```
trait AddAssign<Rhs = Self> {  
    fn add_assign(&mut self, rhs: Rhs);  
}
```

A Tabela 12.4 mostra todos os operadores de atribuição composta do Rust e os traits internos que os implementam.

Tabela 12.4: Traits internos para operadores de atribuição composta

Categoria	Nome do trait	Expressão	Expressão equivalente
Operadores aritméticos	<code>std::ops::AddAssign</code>	<code>x += y</code>	<code>x.add_assign(y)</code>
	<code>std::ops::SubAssign</code>	<code>x -= y</code>	<code>x.sub_assign(y)</code>
	<code>std::ops::MulAssign</code>	<code>x *= y</code>	<code>x.mul_assign(y)</code>
	<code>std::ops::DivAssign</code>	<code>x /= y</code>	<code>x.div_assign(y)</code>
	<code>std::ops::RemAssign</code>	<code>x %= y</code>	<code>x.rem_assign(y)</code>
Operadores bit a bit	<code>std::ops::BitAndAssign</code>	<code>x &= y</code>	<code>x.bitand_assign(y)</code>
	<code>std::ops::BitOrAssign</code>	<code>x = y</code>	<code>x.bitor_assign(y)</code>
	<code>std::ops::BitXorAssign</code>	<code>x ^= y</code>	<code>x.bitxor_assign(y)</code>
	<code>std::ops::ShlAssign</code>	<code>x <<= y</code>	<code>x.shl_assign(y)</code>
	<code>std::ops::ShrAssign</code>	<code>x >>= y</code>	<code>x.shr_assign(y)</code>

Todos os tipos numéricos do Rust implementam os operadores de atribuição composta aritméticos. Tipos inteiros e `bool` do Rust implementam os operadores de atribuição composta de bit a bit.

Uma implementação genérica de `AddAssign` para nosso tipo `Complex` é simples e direta:

```
use std::ops::AddAssign;

impl<T> AddAssign for Complex<T>
where
    T: AddAssign<T>,
{
    fn add_assign(&mut self, rhs: Complex<T>) {
        self.re += rhs.re;
        self.im += rhs.im;
    }
}
```

O trait interno para um operador de atribuição composto é completamente independente do trait interno para o operador binário correspondente. Implementar `std::ops::Add` não implementa automaticamente `std::ops::AddAssign`; se quiser que o Rust permita que seu tipo seja o operando esquerdo de um operador `+=`, deve implementar `AddAssign` você mesmo.

Comparações de equivalência

Os operadores de igualdade do Rust, `==` e `!=`, são atalhos para chamadas aos métodos `std::cmp::PartialEq` `eq` e `ne` do trait:

```
assert_eq!(x == y, x.eq(&y));
assert_eq!(x != y, x.ne(&y));
```

Eis a definição de `std::cmp::PartialEq`:

```
trait PartialEq<Rhs = Self>
where
    Rhs: ?Sized,
{
    fn eq(&self, other: &Rhs) -> bool;
    fn ne(&self, other: &Rhs) -> bool {
        !self.eq(other)
    }
}
```

Como o método `ne` tem uma definição padrão, você só precisa definir `eq` para implementar o trait `PartialEq`, então aqui está uma implementação completa para `Complex`:

```
impl<T: PartialEq> PartialEq for Complex<T> {
    fn eq(&self, other: &Complex<T>) -> bool {
        self.re == other.re && self.im == other.im
    }
}
```

Em outras palavras, para qualquer tipo de componente τ que pode ser comparado por igualdade, isso implementa comparação para `Complex<T>`. Supondo que também tenhamos implementado `std::ops::Mul` para `Complex` em algum lugar ao longo da linha, agora podemos escrever:

```
let x = Complex { re: 5, im: 2 };
let y = Complex { re: 2, im: 5 };
assert_eq!(x * y, Complex { re: 0, im: 29 });
```

Implementações de `PartialEq` são quase sempre da forma mostrada aqui: comparam cada campo do operando da esquerda com o campo correspondente da direita. Torna-se logo tedioso escrevê-los e a igualdade é uma operação comum para suportar; então, se pedir, o Rust vai gerar uma implementação de `PartialEq` para você automaticamente. Basta adicionar `PartialEq` para o atributo `derive` da definição de tipo assim:

```
#[derive(Clone, Copy, Debug, PartialEq)]
struct Complex<T> {
```

```
} ...
```

A implementação gerada automaticamente pelo Rust é essencialmente idêntica ao nosso código escrito à mão, comparando cada campo ou elemento do tipo por vez. O Rust também pode derivar implementações `PartialEq` para tipos `enum`. Naturalmente, cada um dos valores que o tipo contém (ou pode conter, no caso de um `enum`) deve implementar `PartialEq`.

Ao contrário dos traits aritméticos e bit a bit, que recebem seus operandos por valor, `PartialEq` recebe seus operandos por referência. Isso significa que comparar valores não-Copy como `StringS`, `VecS` ou `HashMapS` não faz com que eles sejam movidos, o que seria problemático:

```
let s = "d\x6fv\x65t\x61i\x6c".to_string();
let t = "\x64o\x76e\x74a\x69l".to_string();
assert!(s == t); // s e t são apenas emprestados...

// ... portanto, ainda têm seus valores aqui
assert_eq!(format!("{}", s), t, "dovetail dovetail");
```

Isso nos leva ao limite do trait no parâmetro de tipo `Rhs`, que é de um tipo que não vimos antes:

```
where
    Rhs: ?Sized,
```

Isso relaxa o requisito usual do Rust de que os parâmetros de tipo devem ser tipos dimensionados, permitindo-nos escrever traits como `PartialEq<str>` ou `PartialEq<[T]>`. Os métodos `eq` e `ne` recebem parâmetros de tipo `&Rhs`, e comparar algo com uma `&str` ou um `&[T]` é completamente razoável. Como `str` implementa `PartialEq<str>`, as seguintes afirmações são equivalentes:

```
assert!("ungula" != "ungulate");
assert!("ungula".ne("ungulate"));
```

Aqui, tanto `Self` como `Rhs` seriam do tipo não dimensionado `str`, fazendo o `self` de `ne` e o parâmetro `rhs` valores `&str`. Discutiremos tipos dimensionados, tipos não dimensionados e os traits `Sized` em detalhes em “Sized”, na página 357.

Por que esse trait se chama `PartialEq`? A definição matemática tradicional de uma *relação de equivalência*, da qual a igualdade é

uma instância, impõe três requisitos. Para quaisquer valores x e y :

- Se $x == y$ é verdadeiro, então $y == x$ também deve ser verdadeiro. Em outras palavras, trocar os dois lados de uma comparação de igualdade não afeta o resultado.
- Se $x == y$ e $y == z$, então deve ser o caso que $x == z$. Dada qualquer cadeia de valores, cada um igual ao próximo, cada valor na cadeia é diretamente igual a todos os outros. A igualdade é contagiosa.
- Deve ser sempre verdade que $x == x$.

Esse último requisito pode parecer óbvio demais para valer a pena ser declarado, mas é exatamente aí que as coisas dão errado. Os valores `f32` e `f64` do Rust são de ponto flutuante padrão IEEE. De acordo com esse padrão, expressões como `0.0/0.0` e outras sem valor adequado devem produzir valores *não-é-número*, geralmente referidos como valores NaN. O padrão exige ainda que um valor NaN seja tratado como diferente de todos os outros valores – incluindo ele mesmo. Por exemplo, o padrão requer todos os seguintes comportamentos:

```
assert!(f64::is_nan(0.0 / 0.0));  
assert_eq!(0.0 / 0.0 == 0.0 / 0.0, false);  
assert_eq!(0.0 / 0.0 != 0.0 / 0.0, true);
```

Além disso, qualquer comparação ordenada com um valor NaN deve retornar `false`:

```
assert_eq!(0.0 / 0.0 < 0.0 / 0.0, false);  
assert_eq!(0.0 / 0.0 > 0.0 / 0.0, false);  
assert_eq!(0.0 / 0.0 <= 0.0 / 0.0, false);  
assert_eq!(0.0 / 0.0 >= 0.0 / 0.0, false);
```

Então, enquanto o operador `==` do Rust atende aos dois primeiros requisitos para relações de equivalência, ele claramente não atende ao terceiro quando utilizado em valores de ponto flutuante IEEE. Isso é chamado de *relação de equivalência parcial*, então o Rust utiliza o nome `PartialEq` para o trait interno do operador `==`. Se escrever código genérico com parâmetros de tipo conhecidos apenas por serem `PartialEq`, você pode presumir que os dois primeiros requisitos são válidos, mas não deve presumir que os valores sempre são iguais.

Isso pode ser um pouco contraintuitivo e pode levar a bugs se você

não estiver vigilante. Se preferir que seu código genérico exija uma relação de equivalência completa, pode utilizar o `trait std::cmp::Eq` como um limite, que representa uma relação de equivalência completa: se um tipo implementa `Eq`, então `x == x` deve ser `true` para cada valor `x` desse tipo. Na prática, quase todo tipo que implementa `PartialEq` também deve implementar `Eq`; `f32` e `f64` são os únicos tipos na biblioteca padrão que são `PartialEq`, mas não `Eq`.

A biblioteca padrão define `Eq` como uma extensão de `PartialEq`, sem adicionar novos métodos:

```
trait Eq: PartialEq<Self> {}
```

Se seu tipo é `PartialEq` e gostaria que fosse `Eq` também, deve implementar explicitamente `Eq`, mesmo que não precise realmente definir novas funções ou tipos para fazer isso. Portanto, implementar `Eq` para nosso tipo `Complex` é rápido:

```
impl<T: Eq> Eq for Complex<T> {}
```

Poderíamos implementá-lo de forma ainda mais sucinta, apenas incluindo `Eq` no atributo `derive` na definição do tipo `Complex`:

```
#[derive(Clone, Copy, Debug, Eq, PartialEq)]
struct Complex<T> {
    ...
}
```

As implementações derivadas em um tipo genérico podem depender dos parâmetros do tipo. Com o atributo `derive`, `Complex<i32>` implementaria `Eq`, como `i32` o faz, mas `Complex<f32>` só implementaria `PartialEq`, já que `f32` não implementa `Eq`.

Quando você implementa `std::cmp::PartialEq` por conta própria, o Rust não pode verificar se suas definições para os métodos `eq` e `ne` realmente se comportam conforme necessário para equivalência parcial ou total. Eles podem fazer o que você quiser. O Rust simplesmente aceita sua palavra de que implementou a igualdade de uma forma que atende às expectativas dos usuários do `trait`.

Embora a definição de `PartialEq` forneça uma definição padrão para `ne`, você pode fornecer sua própria implementação, se quiser. Mas deve garantir que `ne` e `eq` são complementos exatos um do outro. Usuários do `trait PartialEq` assumirão que é assim.

Comparações ordenadas

O Rust especifica o comportamento dos operadores de comparação ordenada `<`, `>`, `<=` e `>=` em termos de um único trait, `std::cmp::PartialOrd`:

```
trait PartialOrd<Rhs = Self>: PartialEq<Rhs>
where
    Rhs: ?Sized,
{
    fn partial_cmp(&self, other: &Rhs) -> Option<Ordering>;

    fn lt(&self, other: &Rhs) -> bool { ... }
    fn le(&self, other: &Rhs) -> bool { ... }
    fn gt(&self, other: &Rhs) -> bool { ... }
    fn ge(&self, other: &Rhs) -> bool { ... }
}
```

Observe que `PartialOrd<Rhs>` estende `PartialEq<Rhs>`: você pode fazer comparações ordenadas apenas em tipos que também podem ser comparados quanto a igualdade.

O único método de `PartialOrd` que você mesmo deve implementar é `partial_cmp`. Quando `partial_cmp` retorna `Some(o)`, então `o` indica relação de `self` com `other`:

```
enum Ordering {
    Less,      // self < other
    Equal,     // self == other
    Greater,   // self > other
}
```

Mas se `partial_cmp` retorna `None`, isso significa que `self` e `other` estão desordenados um em relação ao outro: nenhum é maior que o outro, nem são iguais. Entre todos os tipos primitivos do Rust, apenas comparações entre valores de ponto flutuante retornam `None`: especificamente, comparar um valor NaN (não é número) com qualquer outra coisa retorna `None`. Fornecemos mais informações sobre os valores NaN em “Comparações de equivalência”, na página 341.

Assim como os outros operadores binários, para comparar valores de dois tipos `Left` e `Right`, `Left` deve implementar `PartialOrd<Right>`. Expressões como `x < y` ou `x >= y` são atalhos para chamadas para métodos `PartialOrd`, conforme mostrado na Tabela 12.5.

Tabela 12.5: Operadores de comparação ordenada e métodos

PartialOrd

Expressão	Chamada de método equivalente	Definição padrão
$x < y$	<code>x.lt(y)</code>	<code>x.partial_cmp(&y) == Some(Less)</code>
$x > y$	<code>x.gt(y)</code>	<code>x.partial_cmp(&y) == Some(Greater)</code>
$x \leq y$	<code>x.le(y)</code>	<code>matches!(x.partial_cmp(&y), Some(Less Equal))</code>
$x \geq y$	<code>x.ge(y)</code>	<code>matches!(x.partial_cmp(&y), Some(Greater Equal))</code>

Como nos exemplos anteriores, o código de chamada de método equivalente mostrado assume que `std::cmp::PartialOrd` e `std::cmp::Ordering` estão no escopo.

Se você sabe que os valores de dois tipos são sempre ordenados um em relação ao outro, então pode implementar o `trait std::cmp::Ord` mais estrito:

```
trait Ord: Eq + PartialOrd<Self> {
    fn cmp(&self, other: &Self) -> Ordering;
}
```

O método `cmp` aqui simplesmente retorna um `Ordering`, em vez de um `Option<Ordering>` como `partial_cmp`: `cmp` sempre declara seus argumentos iguais ou indica sua ordem relativa. Quase todos os tipos que implementam `PartialOrd` também devem implementar `Ord`. Na biblioteca padrão, `f32` e `f64` são as únicas exceções a essa regra.

Como não há ordem natural em números complexos, não podemos utilizar o tipo `Complex` das seções anteriores para mostrar uma implementação de exemplo de `PartialOrd`. Em vez disso, suponha que esteja trabalhando com o seguinte tipo, representando o conjunto de números dentro de um determinado intervalo semiaberto:

```
#[derive(Debug, PartialEq)]
struct Interval<T> {
    lower: T, // inclusivo
    upper: T, // não inclusivo
}
```

Você gostaria de fazer com que os valores desse tipo fossem parcialmente ordenados: um intervalo é menor que outro se cair inteiramente antes do outro, sem sobreposição. Se dois intervalos

desiguais se sobrepõem, eles estão desordenados: algum elemento de cada lado é menor que algum elemento do outro. E dois intervalos iguais são simplesmente iguais. A seguinte implementação de `PartialOrd` implementa essas regras:

```
use std::cmp::{Ordering, PartialOrd};
impl<T: PartialOrd> PartialOrd<Interval<T>> for Interval<T> {
    fn partial_cmp(&self, other: &Interval<T>) -> Option<Ordering> {
        if self == other {
            Some(Ordering::Equal)
        } else if self.lower >= other.upper {
            Some(Ordering::Greater)
        } else if self.upper <= other.lower {
            Some(Ordering::Less)
        } else {
            None
        }
    }
}
```

Com essa implementação em vigor, você pode escrever o seguinte:

```
assert!(Interval { lower: 10, upper: 20 } < Interval { lower: 20, upper: 40 });
assert!(Interval { lower: 7, upper: 8 } >= Interval { lower: 0, upper: 1 });
assert!(Interval { lower: 7, upper: 8 } <= Interval { lower: 7, upper: 8 });
// Os intervalos sobrepostos não são ordenados uns em relação aos outros
let left = Interval { lower: 10, upper: 30 };
let right = Interval { lower: 20, upper: 40 };
assert!(!(left < right));
assert!(!(left >= right));
```

Enquanto `PartialOrd` é o que você normalmente verá, pedidos totais definidos com `Ord` são necessários em alguns casos, como os métodos de ordenação implementados na biblioteca padrão. Por exemplo, ordenar intervalos não é possível com apenas uma implementação de `PartialOrd`. Se você quiser classificá-los, terá de preencher as lacunas dos casos não ordenados. Você pode querer ordenar por limite superior, por exemplo, e é fácil fazer isso com `sort_by_key`:

```
intervals.sort_by_key(|i| i.upper);
```

O tipo encapsulador `Reverse` tira proveito disso implementando `Ord` com um método que simplesmente inverte qualquer ordem. Para qualquer tipo `T` que implementa `Ord`, `std::cmp::Reverse<T>` também implementa `Ord`, mas com ordem inversa. Por exemplo, ordenar

nossos intervalos de alto a baixo pelo limite inferior é simples:

```
use std::cmp::Reverse;
intervals.sort_by_key(|i| Reverse(i.lower));
```

Index e IndexMut

Você pode especificar como uma expressão de indexação como `a[i]` funciona em seu tipo, implementando os traits `std::ops::Index` e `std::ops::IndexMut`. Arrays suportam o operador `[]` diretamente, mas em qualquer outro tipo, a expressão `a[i]` normalmente é uma abreviação para `*a.index(i)`, em que `index` é um método do trait `std::ops::Index`. Contudo, se a expressão estiver sendo atribuída ou emprestada mutavelmente, é uma abreviação para `*a.index_mut(i)`, uma chamada para o método do trait `std::ops::IndexMut`.

Eis as definições dos traits:

```
trait Index<Idx> {
    type Output: ?Sized;
    fn index(&self, index: Idx) -> &Self::Output;
}

trait IndexMut<Idx>: Index<Idx> {
    fn index_mut(&mut self, index: Idx) -> &mut Self::Output;
}
```

Observe que esses traits utilizam o tipo da expressão de índice como um parâmetro. Você pode indexar uma fatia com um único `usize`, referindo-se a um único elemento, porque as fatias implementam `Index<usize>`. Mas você pode referenciar uma subfatia com uma expressão como `a[i..j]` porque eles também implementam `Index<Range<usize>>`. Essa expressão é uma abreviação para:

```
*a.index(std::ops::Range { start: i, end: j })
```

As coleções `HashMap` e `BTreeMap` do Rust permitem que você utilize qualquer tipo hashável ² ou ordenado como o índice. O código a seguir funciona porque `HashMap<&str, i32>` implementa `Index<&str>`:

```
use std::collections::HashMap;
let mut m = HashMap::new();
m.insert("十", 10);
m.insert("百", 100);
m.insert("千", 1000);
m.insert("万", 1_0000);
```

```
m.insert("億", 1_0000_0000);  
  
assert_eq!(m["十"], 10);  
assert_eq!(m["千"], 1000);
```

Essas expressões de indexação são equivalentes a:

```
use std::ops::Index;  
assert_eq!(*m.index("十"), 10);  
assert_eq!(*m.index("千"), 1000);
```

O tipo associado `Output` do `trait Index` especifica que tipo uma expressão de indexação produz: para o nosso `HashMap`, o tipo `Output` da implementação de `Index` é `i32`.

O `trait IndexMut` estende `Index` com um método `index_mut` que utiliza uma referência mutável para `self` e retorna uma referência mutável para um valor `Output`. O Rust seleciona automaticamente `index_mut` quando a expressão de indexação ocorre em um contexto em que é necessário. Por exemplo, suponha que escrevemos o seguinte:

```
let mut desserts =  
    vec!["Howalon".to_string(), "Soan papdi".to_string()];  
desserts[0].push_str(" (fictional)");  
desserts[1].push_str(" (real)");
```

Como o método `push_str` opera em `&mut self`, essas duas últimas linhas são equivalentes a:

```
use std::ops::IndexMut;  
(*desserts.index_mut(0)).push_str(" (fictional)");  
(*desserts.index_mut(1)).push_str(" (real)");
```

Uma limitação de `IndexMut` é que, por projeto, ele deve retornar uma referência mutável para algum valor. É por isso que você não pode utilizar uma expressão como `m["十"] = 10;` para inserir um valor no `HashMap m`: a tabela precisaria criar uma entrada para "十" primeiro, com algum valor padrão, e retornar uma referência mutável para isso. Mas nem todos os tipos têm valores padrão baratos, e alguns podem ser caros de dropar; seria um desperdício criar tal valor apenas para ser imediatamente dropado pela atribuição. (Existem planos para melhorar isso em versões futuras da linguagem.)

O uso mais comum da indexação é para coleções. Por exemplo, suponha que estamos trabalhando com imagens de bitmap, como as que criamos no `plotter` do conjunto de `Mandelbrot` no Capítulo 2.

Lembre-se de que nosso programa continha um código como este:

```
pixels[row * bounds.0 + column] = ...;
```

Seria melhor ter um tipo `Image<u8>` que age como um array bidimensional, permitindo acessar pixels sem ter de escrever toda a aritmética:

```
image[row][column] = ...;
```

Para fazer isso, precisaremos declarar um struct:

```
struct Image<P> {
    width: usize,
    pixels: Vec<P>,
}

impl<P: Default + Copy> Image<P> {
    /// Crie uma nova imagem do tamanho especificado
    fn new(width: usize, height: usize) -> Image<P> {
        Image {
            width,
            pixels: vec![P::default(); width * height],
        }
    }
}
```

E aqui estão as implementações de `Index` e `IndexMut` mais apropriadas:

```
impl<P> std::ops::Index<usize> for Image<P> {
    type Output = [P];
    fn index(&self, row: usize) -> &[P] {
        let start = row * self.width;
        &self.pixels[start..start + self.width]
    }
}

impl<P> std::ops::IndexMut<usize> for Image<P> {
    fn index_mut(&mut self, row: usize) -> &mut [P] {
        let start = row * self.width;
        &mut self.pixels[start..start + self.width]
    }
}
```

Quando você indexa em um `Image`, recupera uma fatia de pixels; indexar a fatia fornece um pixel individual.

Note que, quando escrevemos `image[row][column]` e se `row` está fora dos limites, nosso método `.index()` tentará indexar `self.pixels` fora do intervalo, gerando um pânico. É assim que as implementações de

`Index` e `IndexMut` devem se comportar: o acesso fora dos limites é detectado e causa um pânico, o mesmo de quando você indexa um array, fatia ou vetor fora dos limites.

Outros operadores

Nem todos os operadores podem ser sobrecarregados no Rust. A partir do Rust 1.56, o operador de verificação de erros `?` funciona apenas com `Result` e alguns outros tipos da biblioteca padrão, mas o trabalho está em andamento para expandir isso também para tipos definidos pelo usuário. Da mesma forma, os operadores lógicos `&&` e `||` são limitados apenas a valores booleanos. Os operadores `..` e `..=` sempre criam um struct representando os limites do intervalo, o operador `&` sempre pega referências emprestadas e o operador `=` sempre move ou copia valores. Nenhum deles pode ser sobrecarregado.

O operador de desreferenciação, `*val` e o operador ponto para acessar campos e métodos de chamada, como em `val.field` e `val.method()`, pode ser sobrecarregado utilizando os traits `Deref` e `DerefMut`, que serão abordados no próximo capítulo. (Não os incluímos aqui porque esses traits fazem mais do que apenas sobrecarregar alguns operadores.)

O Rust não suporta sobrecarga de operador de chamada de função, `f(x)`. Em vez disso, quando você precisar de um valor que possa ser chamado (invocado), normalmente apenas escreverá uma closure. Explicaremos como isso funciona e abordaremos os traits especiais `Fn`, `FnMut` e `FnOnce` no Capítulo 14.

¹ Programadores de Lisp deliciam-se! A expressão `<i32 as Add>::add` é o operador `+` em `i32`, capturado como um valor de função.

² N.R.: Um tipo `hasheável` é um tipo em que podemos calcular um hash de seu valor.

CAPÍTULO 13

Traits utilitários

A ciência nada mais é do que a busca para descobrir a unidade na variedade selvagem da natureza – ou, mais precisamente, na variedade de nossa experiência. A poesia, a pintura, as artes são a mesma busca, nas palavras de Coleridge, pela unidade na variedade.

– Jacob Bronowski

Este capítulo descreve o que chamamos de traits “utilitários” do Rust, uma coleção de vários traits da biblioteca padrão que têm grande impacto na maneira como o Rust é escrito, de modo que você precisará estar familiarizado com eles para escrever código idiomático e projetar interfaces públicas para seus crates, que os usuários julgarão ser propriamente “rústicas”. Eles se enquadram em três grandes categorias:

Traits de extensão da linguagem

Assim como os traits de sobrecarga de operador que abordamos no capítulo anterior permitem que você utilize os operadores de expressão do Rust em seus próprios tipos, existem vários outros traits de biblioteca padrão que servem como pontos de extensão do Rust, permitindo que você integre seus próprios tipos mais próximos da linguagem. Isso inclui `Drop`, `Deref` e `DerefMut`, e os traits de conversão `From` e `Into`. Vamos descrevê-los neste capítulo.

Traits marcadores

Estes são traits utilizados principalmente para vincular variáveis de tipo genérico para expressar restrições que você não pode capturar de outra forma. Isso inclui `Sized` e `Copy`.

Traits de vocabulário público

Não têm nenhuma integração mágica com o compilador; você pode definir traits equivalentes em seu próprio código. Mas servem ao

importante objetivo de estabelecer soluções convencionais para problemas comuns. São especialmente valiosos em interfaces públicas entre crates e módulos: ao reduzir a variação desnecessária, tornam as interfaces mais fáceis de entender, mas também aumentam a probabilidade de que recursos de diferentes crates possam ser simplesmente conectados diretamente, sem código boilerplate ou código de colagem¹ personalizado. Isso inclui `Default`, os traits de empréstimo de referência `AsRef`, `AsMut`, `Borrow` e `BorrowMut`; os traits de conversão que podem falhar `TryFrom` e `TryInto`; e o trait `ToOwned`, uma generalização de `Clone`.

Estes estão resumidos na Tabela 13.1.

Tabela 13.1: Resumo dos traits utilitários

Trait	Descrição
Drop	Destrutores. Código de limpeza que o Rust executa automaticamente sempre que um valor é dropado.
Sized	O trait marcador para tipos com um tamanho fixo conhecido em tempo de compilação, em oposição a tipos (como fatias) que são dimensionados dinamicamente.
Clone	Tipos que dão suporte a clonagem de valores.
Copy	Trait marcador para tipos que podem ser clonados simplesmente fazendo uma cópia byte por byte da memória que contém o valor.
Deref e DerefMut	Traits para tipos de ponteiro inteligente.
Default	Tipos que têm um “valor padrão”.
AsRef e AsMut	Traits de conversão para pegar emprestado um tipo de referência de outro.
Borrow e BorrowMut	Traits de conversão, como <code>AsRef/AsMut</code> , mas também garantindo hash, ordenação e igualdade consistentes.
From e Into	Traits de conversão para transformar um tipo de valor em outro.
TryFrom e TryInto	Traits de conversão para transformar um tipo de valor em outro, para transformações que podem falhar.
ToOwned	Trait de conversão para converter uma referência em um valor com posse.

Existem outros traits importantes da biblioteca padrão também. Abordaremos `Iterator` e `IntoIterator` no Capítulo 15. O trait `Hash`, para calcular códigos hash, é abordado no Capítulo 16. E um par de traits que marcam os tipos thread-safe, `Send` e `Sync`, são abordados no Capítulo 19.

Drop

Quando o proprietário de um valor desaparece, dizemos que o Rust *dropa* o valor. Dropar um valor envolve a liberação de quaisquer outros valores, armazenamento de heap e recursos do sistema pertencentes ao valor. Dropagens ocorrem em várias circunstâncias: quando uma variável sai do escopo; no final de uma instrução de expressão; quando você trunca um vetor, removendo elementos de sua extremidade; e assim por diante.

Na maioria das vezes, o Rust lida com a dropagem de valores para você automaticamente. Por exemplo, suponha que você defina o seguinte tipo:

```
struct Appellation {  
    name: String,  
    nicknames: Vec<String>  
}
```

Um `Appellation` possui armazenamento em heap para o conteúdo das strings e o buffer de elementos do vetor. O Rust se encarrega de limpar tudo isso sempre que um `Appellation` é dropado, sem qualquer codificação adicional necessária de sua parte. Mas, se quiser, você pode personalizar o modo como o Rust dropa os valores do seu tipo, implementando o `trait std::ops::Drop`:

```
trait Drop {  
    fn drop(&mut self);  
}
```

Uma implementação de `Drop` é análoga a um destrutor em C++ ou um finalizador em outras linguagens. Quando um valor é dropado, se ele implementa `std::ops::Drop`, o Rust chama seu método `drop`, antes de prosseguir para dropar quaisquer valores que seus campos ou elementos possuam, como faria normalmente. Essa invocação implícita de `drop` é a única maneira de chamar esse método; se você tentar invocá-lo explicitamente, o Rust sinaliza isso como um erro.

Como o Rust chama `Drop::drop` sobre um valor antes de dropar seus campos ou elementos, o valor que o método recebe sempre está completamente inicializado. Uma implementação de `Drop` para nosso tipo `Appellation` pode fazer pleno uso de seus campos:

```

impl Drop for Appellation {
    fn drop(&mut self) {
        print!("Dropping {}", self.name);
        if !self.nicknames.is_empty() {
            print!(" (AKA {})", self.nicknames.join(", "));
        }
        println!("");
    }
}

```

Dada essa implementação, podemos escrever o seguinte:

```

{
    let mut a = Appellation {
        name: "Zeus".to_string(),
        nicknames: vec!["cloud collector".to_string(),
                        "king of the gods".to_string()]
    };

    println!("before assignment");
    a = Appellation { name: "Hera".to_string(), nicknames: vec![] };
    println!("at end of block");
}

```

Quando atribuímos o segundo `Appellation` para `a`, o primeiro é dropado e, quando deixamos o escopo de `a`, o segundo é dropado. Esse código imprime o seguinte:

```

before assignment
Dropping Zeus (AKA cloud collector, king of the gods)
at end of block
Dropping Hera

```

Uma vez que nossa implementação de `std::ops::Drop` para `Appellation` não faz nada além de imprimir uma mensagem, como, exatamente, sua memória é limpa? O tipo `Vec` implementa `Drop`, dropando cada um de seus elementos e, em seguida, liberando o buffer alocado no heap que eles ocupavam. Uma `String` utiliza um `Vec<u8>` internamente para manter seu texto, então `String` não precisa implementar `Drop` em si; isso deixa seu `Vec` cuidar de liberar os caracteres. O mesmo princípio se estende a valores `Appellation`: quando um é dropado, no final é a implementação `Vec` de `Drop` que realmente se encarrega de liberar o conteúdo de cada uma das strings e, finalmente, liberar o buffer que contém os elementos do vetor. Quanto à memória que armazena o valor `Appellation` em si, ele também tem algum proprietário, talvez uma

variável local ou alguma estrutura de dados, que é responsável por liberá-lo.

Se o valor de uma variável for movido para outro lugar, de modo que a variável não seja inicializada quando sair do escopo, o Rust não tentará dropar essa variável: não há valor nela para dropar.

Esse princípio vale mesmo quando uma variável pode ou não ter seu valor deslocado, dependendo do fluxo de controle. Em casos como esse, o Rust acompanha o estado da variável com um sinalizador invisível indicando se o valor da variável precisa ser dropado ou não:

```
let p;
{
    let q = Appellation { name: "Cardamine hirsuta".to_string(),
                          nicknames: vec!["shotweed".to_string(),
                                           "bittercress".to_string()] };

    if complicated_condition() {
        p = q;
    }
}
println!("Sproing! What was that?");
```

Dependendo se `complicated_condition` retorna `true` ou `false`, `p` ou `q` acabará por possuir o `Appellation`, com a outra variável não inicializada. O local onde ele cai determina se ele será dropado antes ou depois do `println!`, uma vez que `q` sai do escopo antes do `println!`, e `p` depois. Embora um valor possa ser movido de um lugar para outro, o Rust o dropa apenas uma vez.

Normalmente, você não precisará implementar `std::ops::Drop` a menos que esteja definindo um tipo que possua recursos que o Rust ainda não conheça. Por exemplo, em sistemas Unix, a biblioteca padrão do Rust utiliza o seguinte tipo internamente para representar um descritor de arquivo do sistema operacional:

```
struct FileDesc {
    fd: c_int,
}
```

O campo `fd` de um `FileDesc` é simplesmente o número do descritor de arquivo que deve ser fechado quando o programa terminar; `c_int` é um alias para `i32`. A biblioteca padrão implementa `Drop` para `FileDesc` do seguinte modo:

```
impl Drop for FileDesc {
    fn drop(&mut self) {
        let _ = unsafe { libc::close(self.fd) };
    }
}
```

Aqui, `libc::close` é o nome Rust para a função `close` da biblioteca C. O código Rust pode chamar funções C apenas dentro de blocos `unsafe`, então a biblioteca utiliza um aqui.

Se um tipo implementa `Drop`, não pode implementar o trait `Copy`. Se um tipo é `Copy`, isso significa que a duplicação simples byte por byte é suficiente para produzir uma cópia independente do valor. Mas normalmente é um erro chamar o mesmo método `drop` mais de uma vez sobre os mesmos dados.

O prelúdio padrão inclui uma função para dropar um valor, `drop`, mas sua definição é tudo, menos mágica:

```
fn drop<T>(_x: T) { }
```

Em outras palavras, ele recebe seu argumento por valor, tomando posse do chamador – e depois não faz nada com ele. O Rust dropa o valor de `_x` quando sai do escopo, como faria com qualquer outra variável.

Sized

Um *tipo dimensionado* (*sized type*) é aquele cujos valores têm todos o mesmo tamanho na memória. Quase todos os tipos no Rust são dimensionados: todo `u64` tem oito bytes, toda tupla `(f32, f32, f32)` tem doze. Mesmo enums são dimensionados: não importa qual variante esteja realmente presente, um enum sempre ocupa espaço suficiente para conter sua maior variante. E, embora um `Vec<T>` possua um buffer alocado em heap cujo tamanho pode variar, o valor de `vec` em si é um ponteiro para o buffer, sua capacidade e seu comprimento; portanto, `Vec<T>` é um tipo dimensionado.

Todos os tipos dimensionados implementam o trait `std::marker::Sized`, que não possui métodos ou tipos associados. O Rust o implementa automaticamente para todos os tipos aos quais se aplica; você não pode implementá-lo sozinho. O único uso para `Sized` é como um limite

para variáveis de tipo: um limite como `T: Sized` requer que `T` seja um tipo cujo tamanho é conhecido em tempo de compilação. Traits desse tipo são chamados *traits marcadores*, porque a própria linguagem Rust os utiliza para marcar certos tipos como tendo características de interesse.

Entretanto, o Rust também tem alguns *tipos* não dimensionados cujos valores não são todos do mesmo tamanho. Por exemplo, o tipo de fatia de string `str` (note, sem `&`) é não dimensionado. Os literais de string `"diminutive"` e `"big"` são referências a fatias `str` que ocupam dez e três bytes. Ambos são mostrados na Figura 13.1. Tipos de fatia de array como `[T]` (novamente, sem `&`) também são não dimensionados: uma referência compartilhada como `&[u8]` pode apontar para uma fatia `[u8]` de qualquer tamanho. Como os tipos `str` e `[T]` denotam conjuntos de valores de tamanhos variados, eles são tipos não dimensionados.

O outro tipo comum de tipo não dimensionado no Rust é um tipo `dyn`, o referente de um objeto `trait`. Como explicamos em “Objetos `trait`”, na página 301, um objeto `trait` é um ponteiro para algum valor que implementa uma determinada `trait`. Por exemplo, os tipos `&dyn std::io::Write` e `Box<dyn std::io::Write>` são ponteiros para algum valor que implementa o `trait Write`. O referente pode ser um arquivo ou um soquete de rede ou algum tipo para o qual você mesmo implementou `Write`. Como o conjunto de tipos que implementam `Write` é aberto, `dyn Write` é considerado um tipo não dimensionado: seus valores têm vários tamanhos.

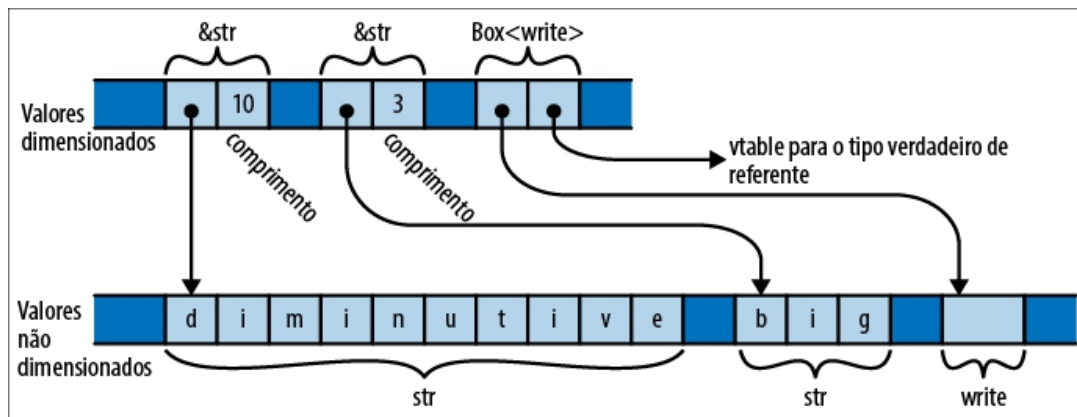


Figura 13.1: Referências a valores não dimensionados.

O Rust não pode armazenar valores não dimensionados em variáveis ou passá-los como argumentos. Você só pode lidar com eles por meio de ponteiros como `&str` ou `Box<dyn Write>`, que são eles próprios dimensionados. Conforme mostrado na Figura 13.1, um ponteiro para um valor não dimensionado é sempre um *ponteiro gordo*, duas palavras de largura: um ponteiro para uma fatia também carrega o comprimento da fatia, e um objeto `trait` também carrega um ponteiro para uma `vtable` de implementações de método.

Objetos `trait` e ponteiros para fatias são perfeitamente simétricos. Em ambos os casos, o tipo carece de informações necessárias para usá-lo: você não pode indexar um `[u8]` sem saber seu comprimento, nem pode invocar um método em um `Box<dyn Write>` sem conhecer a implementação de `Write` adequada ao valor específico a que ela se refere. E em ambos os casos, o ponteiro gordo preenche as informações que faltam no tipo, carregando um comprimento ou um ponteiro `vtable`. A informação estática omitida é substituída por informação dinâmica.

Como os tipos não dimensionados são tão limitados, a maioria das variáveis de tipo genérico deve ser restrita a tipos `Sized`. Na verdade, isso é necessário com tanta frequência que é o padrão implícito no Rust: se você escrever `struct S<T> { ... }`, o Rust entende que você quer dizer `struct S<T: Sized> { ... }`. Se você não quiser restringir `T` dessa forma, deve cancelar explicitamente, escrevendo `struct S<T: ?Sized> { ... }`. A sintaxe `?Sized` é específica para esse caso e significa “não necessariamente `Sized`”. Por exemplo, se você escrever `struct S<T: ?Sized> { b: Box<T> }`, então o Rust permitirá que escreva `S<str>` e `S<dyn Write>`, em que o `box` se torna um ponteiro gordo, assim como `S<i32>` e `S<String>`, em que o `box` é um ponteiro comum.

Apesar de suas restrições, os tipos não dimensionados fazem com que o sistema de tipos do Rust funcione de maneira mais suave. Lendo a documentação da biblioteca padrão, você ocasionalmente encontrará um limite `?Sized` sobre uma variável de tipo; isso quase sempre significa que o tipo fornecido é apenas apontado e permite que o código associado funcione com fatias e objetos `trait`, bem

como valores comuns. Quando uma variável de tipo `Vec` tem o limite `Vec::MAX`, as pessoas costumam dizer que é *questionavelmente dimensionado*: pode ser `Vec`, ou talvez não.

Além de fatias e objetos `trait`, há mais uma espécie de tipo não dimensionado. O último campo de um tipo `struct` (mas apenas o último) pode ser não dimensionado e tal `struct` é ele próprio não dimensionado. Por exemplo, um ponteiro de contagem de referência `Rc<T>` é implementado internamente como um ponteiro para o tipo privado `RcBox<T>`, que armazena a contagem de referência ao lado do `T`. Eis uma definição simplificada de `RcBox`:

```
struct RcBox<T: ?Sized> {  
    ref_count: usize,  
    value: T,  
}
```

O campo `value` é o `T` cujas referências `Rc<T>` está contando; `Rc<T>` cancela a referência a um ponteiro para esse campo. O campo `ref_count` contém o contador de referências.

O `RcBox` real é apenas um detalhe de implementação da biblioteca padrão e não está disponível para uso público. Mas suponha que estamos trabalhando com a definição anterior. Você pode utilizar esse `RcBox` com tipos dimensionados, como `RcBox<String>`; o resultado é um tipo `struct` dimensionado. Ou pode usá-lo com tipos não dimensionados, como `RcBox<dyn std::fmt::Display>` (em que `Display` é o `trait` para tipos que podem ser formatados por `println!` e macros semelhantes); `RcBox<dyn Display>` é um tipo `struct` não dimensionado.

Você não pode construir um valor `RcBox<dyn Display>` diretamente. Em vez disso, primeiro precisa criar um `RcBox` dimensionado cujo tipo `value` implementa `Display`, como `RcBox<String>`. O Rust então permite que converta uma referência `&RcBox<String>` para uma referência gorda `&RcBox<dyn Display>`:

```
let boxed_lunch: RcBox<String> = RcBox {  
    ref_count: 1,  
    value: "lunch".to_string()  
};  
  
use std::fmt::Display;  
let boxed_displayable: &RcBox<dyn Display> = &boxed_lunch;
```


Essa conversão acontece implicitamente ao passar valores para funções, então você pode passar um `&RcBox<String>` para uma função que espera um `&RcBox<dyn Display>`:

```
fn display(boxed: &RcBox<dyn Display>) {  
    println!("For your enjoyment: {}", &boxed.value);  
}  
  
display(&boxed_lunch);
```

Isso produziria a seguinte saída:

```
For your enjoyment: lunch
```

Clone

O trait `std::clone::Clone` é para tipos que podem fazer cópias de si mesmos. `Clone` é definido da seguinte forma:

```
trait Clone: Sized {  
    fn clone(&self) -> Self;  
    fn clone_from(&mut self, source: &Self) {  
        *self = source.clone()  
    }  
}
```

O método `clone` deve construir uma cópia independente de `self` e retorná-la. Como o tipo de retorno desse método é `Self` e as funções podem não retornar valores não dimensionados, o próprio trait `Clone` estende o trait `Sized`: isso tem o efeito de limitar as implementações dos tipos `Self` para que sejam `Sized`.

A clonagem de um valor geralmente envolve a alocação de cópias de tudo o que ele possui, portanto um `clone` pode ser caro, tanto em tempo quanto em memória. Por exemplo, a clonagem de um `Vec<String>` não apenas copia o vetor, mas também copia cada um de seus elementos `String`. É por isso que o Rust não apenas clona valores automaticamente, mas exige que você faça uma chamada de método explícita. Os tipos de ponteiro contados por referência, como `Rc<T>` e `Arc<T>`, são exceções: a clonagem de um deles simplesmente incrementa a contagem de referência e fornece a você um novo ponteiro.

O método `clone_from` transforma `self` em uma cópia de `source`. A definição padrão de `clone_from` simplesmente clona `source` e depois

move este para `*self`. Isso sempre funciona, mas, para alguns tipos, existe uma maneira mais rápida de obter o mesmo efeito. Por exemplo, suponha que `s` e `t` sejam `Strings`. A instrução `s = t.clone()`; deve clonar `t`, dropar o valor antigo de `s` e, então, mover o valor clonado para `s`; isso é uma alocação de heap e uma desalocação de heap. Mas, se o buffer de heap pertencente ao `s` original tem capacidade suficiente para armazenar o conteúdo de `t`, nenhuma alocação ou desalocação é necessária: você pode simplesmente copiar o texto de `t` para o buffer de `s` e ajustar o comprimento. No código genérico, você deve utilizar `clone_from` sempre que possível para aproveitar as implementações otimizadas quando presentes.

Se sua implementação de `Clone` simplesmente aplica `clone` a cada campo ou elemento do seu tipo e, em seguida, constrói um novo valor desses clones, e a definição padrão de `clone_from` é boa o suficiente, então o Rust vai implementar isso para você: basta colocar `#[derive(Clone)]` acima da sua definição de tipo.

Praticamente todos os tipos na biblioteca padrão que fazem sentido copiar implementam `Clone`. Tipos primitivos como `bool` e `i32` o fazem. Tipos contêineres como `String`, `Vec<T>` e `HashMap` também o fazem. Não faz sentido copiar alguns tipos, como `std::sync::Mutex`; aqueles não implementam `Clone`. Alguns tipos como `std::fs::File` podem ser copiados, mas a cópia pode falhar se o sistema operacional não tiver os recursos necessários; esses tipos não implementam `Clone`, uma vez que `clone` deve ser infalível. Em vez disso, `std::fs::File` fornece um método `try_clone`, que retorna um `std::io::Result<File>`, que pode relatar uma falha.

Copy

No Capítulo 4, explicamos que, para a maioria dos tipos, a atribuição move os valores, em vez de copiá-los. Mover valores torna muito mais simples rastrear os recursos que eles possuem. Mas em “Tipos de cópia: A exceção aos movimentos”, na página 122, apontamos a exceção: tipos simples que não possuem nenhum recurso podem ser tipos `Copy`, em que a atribuição faz uma cópia da origem, em vez de

mover o valor e deixar a origem não inicializada.

Naquele momento, deixamos vago exatamente o que `Copy` era, mas agora podemos dizer a você: um tipo é `Copy` se implementa o trait marcador `std::marker::Copy`, que é definido da seguinte forma:

```
trait Copy: Clone { }
```

Isso certamente é fácil de implementar para seus próprios tipos:

```
impl Copy for MyType { }
```

Mas como `Copy` é um trait marcador com significado especial para a linguagem, o Rust permite que um tipo implemente `Copy` somente se uma cópia rasa byte por byte copiar tudo o que ele precisa. Tipos que possuem quaisquer outros recursos, como buffers de heap ou identificadores do sistema operacional, não podem implementar `Copy`.

Qualquer tipo que implemente o trait `Drop` não pode ser `Copy`. O Rust presume que, se um tipo precisa de código de limpeza especial, ele também deve exigir código de cópia especial e, portanto, não pode ser `Copy`.

Como com `Clone`, pode pedir ao Rust que derive `Copy` para você, utilizando `#[derive(Copy)]`. Muitas vezes você verá ambos derivados de uma só vez, com `#[derive(Copy, Clone)]`.

Pense bem antes de fazer um tipo `Copy`. Embora isso torne o tipo mais fácil de utilizar, ele impõe pesadas restrições sobre sua implementação. Cópias implícitas também podem ser caras. Explicamos esses fatores em detalhes em “Tipos de cópia: a exceção aos movimentos”, na página 122.

Deref e DerefMut

Você pode especificar como os operadores de desreferenciação, como `*` e `.`, comportam-se em seus tipos, implementando o traits `std::ops::Deref` e `std::ops::DerefMut`. Tipos de ponteiro como `Box<T>` e `Rc<T>` implementam esses traits para que eles possam se comportar como os tipos de ponteiro internos do Rust. Por exemplo, se você tiver um valor `Box<Complex> b`, então `*b` referencia valor `Complex` para o qual `b` aponta, e `b.re` referencia seu componente real. Se o contexto atribui ou empresta uma referência mutável ao referente, o Rust utiliza o

`trait DerefMut` (“desreferenciar mutavelmente”); caso contrário, o acesso somente leitura é suficiente e utiliza `Deref`.

Os traits são definidos assim:

```
trait Deref {  
    type Target: ?Sized;  
    fn deref(&self) -> &Self::Target;  
}  
  
trait DerefMut: Deref {  
    fn deref_mut(&mut self) -> &mut Self::Target;  
}
```

Os métodos `deref` e `deref_mut` aceitam uma referência `&Self` e retornam uma referência `&Self::Target`. `Target` deve ser algo que `Self` contém, possui ou referencia: para `Box<Complex>` o tipo `Target` é `Complex`. Observe que `DerefMut` estende `Deref`: se você pode desreferenciar algo e modificá-lo, certamente também deve ser capaz de emprestar uma referência compartilhada a ele. Como os métodos retornam uma referência com o mesmo tempo de vida que `&self`, `self` permanece emprestado enquanto a referência retornada existir.

O traits `Deref` e `DerefMut` desempenham outro papel também. Como a `deref` aceita uma referência `&Self` e retorna uma referência `&Self::Target`, o Rust utiliza isso para converter automaticamente as referências do primeiro tipo no segundo. Em outras palavras, se inserir uma chamada `deref` impediria uma incompatibilidade de tipos, o Rust insere uma para você. Implementar `DerefMut` habilita a conversão correspondente para referências mutáveis. Estas são chamadas de *coerções deref*: um tipo está sendo “coagido” a se comportar como outro.

Embora as coerções `deref` não sejam nada que você não possa escrever explicitamente, elas são convenientes:

- Se você tiver algum valor `Rc<String> r` e quiser aplicar `String::find` a isso, pode simplesmente escrever `r.find('?')`, em vez de `(*r).find('?')`: a chamada de método empresta implicitamente `r`, e `&Rc<String>` força para `&String`, pois `Rc<T>` implementa `Deref<Target=T>`.
- Você pode utilizar métodos como `split_at` em valores `String`, embora `split_at` seja um método do tipo de fatia `str`, porque `String` implementa

`Deref<Target=str>`. Não há necessidade de `String` reimplementar todos os métodos de `str`, já que você pode coagir uma `&str` a partir de uma `&String`.

- Se você tiver um vetor de bytes `v` e quiser passá-lo para uma função que espera uma fatia de bytes `&[u8]`, pode simplesmente passar `&v` como argumento, já que `Vec<T>` implementa `Deref<Target=[T]>`.

O Rust aplicará várias coerções `deref` em sucessão, se necessário. Por exemplo, utilizando as coerções mencionadas anteriormente, você pode aplicar `split_at` diretamente a uma `Rc<String>`, uma vez que `&Rc<String>` é desreferenciado para `&String`, que por sua vez é desreferenciado para `&str`, a qual tem o método `split_at`.

Por exemplo, suponha que você tenha o seguinte tipo:

```
struct Selector<T> {  
    /// Elementos disponíveis neste `Selector`  
    elements: Vec<T>,  
  
    /// O índice do elemento "atual" em `elements`. Um `Selector`  
    /// se comporta como um ponteiro para o elemento atual  
    current: usize  
}
```

Para fazer o `Selector` comportar-se como afirma o comentário do documento, você deve implementar `Deref` e `DerefMut` para o tipo:

```
use std::ops::{Deref, DerefMut};  
  
impl<T> Deref for Selector<T> {  
    type Target = T;  
    fn deref(&self) -> &T {  
        &self.elements[self.current]  
    }  
}  
  
impl<T> DerefMut for Selector<T> {  
    fn deref_mut(&mut self) -> &mut T {  
        &mut self.elements[self.current]  
    }  
}
```

Dadas essas implementações, você pode utilizar um `Selector` assim:

```
let mut s = Selector { elements: vec!['x', 'y', 'z'],  
                      current: 2 };
```

```
// Como `Selector` implementa `Deref`, podemos utilizar o operador `*`
// para referenciar seu elemento atual
assert_eq!(*s, 'z');

// Assegura que 'z' é alfabético, utilizando um método `char` diretamente
// em um `Selector`, via coerção deref
assert!(s.is_alphabetic());

// Muda o 'z' para um 'w', atribuindo ao referente do `Seleto`
*s = 'w';

assert_eq!(s.elements, ['x', 'y', 'w']);
```

O traits `Deref` e `DerefMut` são projetados para implementar tipos de ponteiro inteligente, como `Box`, `Rc` e `Arc`, e tipos que servem como versões proprietárias de algo que você também utilizaria com frequência por referência, assim como `Vec<T>` e `String` servem como proprietárias de versões de `[T]` e `str`. Você não deve implementar `Deref` e `DerefMut` para um tipo apenas para fazer os métodos `Target` do tipo aparecerem nele automaticamente, da mesma forma como os métodos de uma classe de base em C++ são visíveis em uma subclasse. Isso nem sempre funcionará como você espera e pode ser confuso quando der errado.

As coerções `deref` vêm com uma ressalva que pode causar alguma confusão: O Rust as aplica para resolver conflitos de tipo, mas não para satisfazer limites em variáveis de tipo. Por exemplo, o seguinte código funciona bem:

```
let s = Selector { elements: vec!["good", "bad", "ugly"],
                  current: 2 };

fn show_it(thing: &str) { println!("{}", thing); }
show_it(&s);
```

Na chamada `show_it(&s)`, o Rust vê um argumento do tipo `&Selector<&str>` e um parâmetro do tipo `&str`, encontra a implementação de `Deref<Target=&str>` e reescreve a chamada como `show_it(s.deref())`, conforme necessário.

Contudo, se você mudar `show_it` em uma função genérica, o Rust de repente não é mais cooperativo:

```
use std::fmt::Display;
fn show_it_generic<T: Display>(thing: T) { println!("{}", thing); }
show_it_generic(&s);
```

O Rust reclama:

```
error: `Selector<&str>` doesn't implement `std::fmt::Display`
|
31 | show_it_generic(&s);
|               ^^
|               |
|               `Selector<&str>` cannot be formatted with
|               the default formatter
|               help: consider adding dereference here: `&*s`
|
note: required by a bound in `show_it_generic`
|
30 | fn show_it_generic<T: Display>(thing: T) { println!("{}", thing); }
|               ^^^^^^^^^ required by this bound
|               in `show_it_generic`
```

Isso pode ser desconcertante: Como o simples fato de tornar uma função genérica poderia introduzir um erro? Verdade, `Selector<&str>` não implementa `Display` em si, mas desreferencia para `&str`, o que certamente o faz.

Já que você está passando um argumento do tipo `&Selector<&str>` e o tipo de parâmetro da função é `&T`, a variável de tipo `T` deve ser `Selector<&str>`. Então, o Rust verifica se o limite `T: Display` é satisfeito: uma vez que ele não aplica coerções `deref` para satisfazer limites em variáveis de tipo, essa verificação falha.

Para contornar esse problema, você pode escrever explicitamente a coerção utilizando o operador `as`:

```
show_it_generic(&s as &str);
```

Ou, como sugere o compilador, você pode forçar a coerção com `&*`:

```
show_it_generic(&*s);
```

Default

Alguns tipos têm um valor padrão razoavelmente óbvio: o vetor ou string padrão está vazio, o número padrão é zero, a `Option` padrão é `None` e assim por diante. Tipos como esses podem implementar o `trait` `std::default::Default`:

```
trait Default {
    fn default() -> Self;
}
```

O método `default` simplesmente retorna um novo valor do tipo `Self`. A implementação `String` de `Default` é simples e direta:

```
impl Default for String {  
    fn default() -> String {  
        String::new()  
    }  
}
```

Todos os tipos de coleção do Rust – `Vec`, `HashMap`, `BinaryHeap` e assim por diante – implementam `Default`, com métodos `default` que retornam uma coleção vazia. Isso é útil quando você precisa construir uma coleção de valores, mas quer deixar seu chamador decidir exatamente que tipo de coleção construir. Por exemplo, método `partition` do trait `Iterator` divide os valores que o iterador produz em duas coleções, utilizando uma closure para decidir onde cada valor vai:

```
use std::collections::HashSet;  
let squares = [4, 9, 16, 25, 36, 49, 64];  
let (powers_of_two, impure): (HashSet<i32>, HashSet<i32>)  
    = squares.iter().partition(|&n| n & (n-1) == 0);  
  
assert_eq!(powers_of_two.len(), 3);  
assert_eq!(impure.len(), 4);
```

A closure `|&n| n & (n-1) == 0` utiliza alguma manipulação de bits para reconhecer números que são potências de dois, e `partition` utiliza isso para produzir dois `HashSets`. Mas, naturalmente, `partition` não é específico para `HashSets`; você pode utilizá-la para produzir qualquer tipo de coleção que quiser, desde que o tipo de coleção implemente `Default`, a fim de produzir uma coleção vazia para começar, e `Extend<T>`, para adicionar um `T` à coleção. `String` implementa `Default` e `Extend<char>`, então você pode escrever:

```
let (upper, lower): (String, String)  
    = "Great Teacher Onizuka".chars().partition(|&c| c.is_uppercase());  
assert_eq!(upper, "GTO");  
assert_eq!(lower, "reat eacher nizuka");
```

Outro uso comum de `Default` é produzir valores padrão para structs que representam uma grande coleção de parâmetros, a maioria dos quais você normalmente não precisa alterar. Por exemplo, o crate `glum` fornece ligações do Rust com a poderosa e complexa biblioteca

de gráficos OpenGL. O struct `glium::DrawParameters` inclui 24 campos, cada um controlando um detalhe diferente de como o OpenGL deve renderizar alguns gráficos. A função `glium draw` espera um struct `DrawParameters` como um argumento. Como `DrawParameters` implementa `Default`, você pode criar um para passar para `draw`, mencionando apenas os campos que deseja alterar:

```
let params = glium::DrawParameters {  
    line_width: Some(0.02),  
    point_size: Some(0.02),  
    .. Default::default()  
};
```

```
target.draw(..., &params).unwrap();
```

Isso chama `Default::default()` para criar um valor `DrawParameters` inicializado com os valores padrão para todos os seus campos e, em seguida, utiliza a sintaxe `..` para structs a fim de criar um novo com os campos `line_width` e `point_size` alterados, prontos para você passar para `target.draw`.

Se um tipo `T` implementa `Default`, então a biblioteca padrão implementa `Default` automaticamente para `Rc<T>`, `Arc<T>`, `Box<T>`, `Cell<T>`, `RefCell<T>`, `Cow<T>`, `Mutex<T>` e `RwLock<T>`. O valor padrão para o tipo `Rc<T>`, por exemplo, é um `Rc` apontando para o valor padrão para o tipo `T`.

Se todos os tipos de elemento de um tipo de tupla implementam `Default`, o tipo de tupla também o faz, assumindo valor padrão de cada elemento.

O Rust não implementa implicitamente `Default` para tipos struct, mas, se todos os campos de um struct implementarem `Default`, você pode implementar `Default` para o struct utilizando automaticamente `#[derive(Default)]`.

AsRef e AsMut

Quando um tipo implementa `AsRef<T>`, isso significa que você pode emprestar um `&T` a partir dele de forma eficiente. `AsMut` é o análogo para referências mutáveis. Suas definições são as seguintes:

```
trait AsRef<T: ?Sized> {
```

```

    fn as_ref(&self) -> &T;
}
trait AsMut<T: ?Sized> {
    fn as_mut(&mut self) -> &mut T;
}

```

Assim, por exemplo, `Vec<T>` implementa `AsRef<[T]>` e `String` implementa `AsRef<str>`. Você também pode emprestar conteúdo de uma `String` como um array de bytes, então `String` também implementa `AsRef<[u8]>`. `AsRef` é normalmente utilizado para tornar as funções mais flexíveis nos tipos de argumento que aceitam. Por exemplo, a função `std::fs::File::open` é declarada assim:

```
fn open<P: AsRef<Path>>>(path: P) -> Result<File>
```

O que `open` realmente quer é um `&Path`, o tipo que representa um caminho do sistema de arquivos. Mas, com essa assinatura, `open` aceita qualquer coisa da qual se pode emprestar um `&Path` – isto é, qualquer coisa que implemente `AsRef<Path>`. Tais tipos incluem `String` e `str`, os tipos de string da interface do sistema operacional `OsString` e `OsStr` e, naturalmente, `PathBuf` e `Path`; consulte a documentação da biblioteca para obter a lista completa. Isso é o que permite que você passe literais de string para `open`:

```
let dot_emacs = std::fs::File::open("/home/jimb/.emacs"?);
```

Todas as funções de acesso ao sistema de arquivos da biblioteca padrão aceitam argumentos de path dessa maneira. Para chamadores, o efeito se assemelha ao de uma função sobrecarregada em C++ embora o Rust tenha uma abordagem diferente para estabelecer quais tipos de argumento são aceitáveis.

Mas isso não pode ser a história completa. Um literal de string é uma `&str`, mas o tipo que implementa `AsRef<Path>` é `str`, sem um `&`. E, como explicamos em “Deref e DerefMut”, na página 186, o Rust não tenta coerções `deref` para satisfazer limites de variável de tipo, portanto elas também não ajudarão aqui.

Felizmente, a biblioteca padrão inclui a implementação geral:

```

impl<'a, T, U> AsRef<U> for &'a T
    where T: AsRef<U>,
           T: ?Sized, U: ?Sized
{
    fn as_ref(&self) -> &U {

```

```

        (*self).as_ref()
    }
}

```

Em outras palavras, para quaisquer tipos T e U , se $T: \text{AsRef}<U>$, então $\&T: \text{AsRef}<U>$ também é possível: basta seguir a referência e proceder como antes. Em particular, uma vez que $\text{str}: \text{AsRef}<\text{Path}>$, então $\&\text{str}: \text{AsRef}<\text{Path}>$ também é possível. Em certo sentido, essa é uma maneira de obter uma forma limitada de coerção `deref` na verificação de limites `AsRef` em variáveis de tipo.

Você pode supor que, se um tipo implementa `AsRef<T>`, deve também implementar `AsMut<T>`. Mas há casos em que isso não é apropriado. Por exemplo, mencionamos que `String` implementa `AsRef<[u8]>`; isso faz sentido, pois cada `String` certamente tem um buffer de bytes que pode ser útil para acessar como dados binários. Contudo, `String` garante ainda que esses bytes sejam uma codificação UTF-8 bem formada de texto Unicode; se `String` implementasse `AsMut<[u8]>`, isso permitiria que os chamadores alterassem os bytes de `String` para qualquer coisa que eles quisessem e você não poderia mais confiar em uma `String` para obter UTF-8 bem formado. Só faz sentido para um tipo implementar `AsMut<T>` se modificar o dado T não puder violar as invariantes do tipo.

Embora `AsRef` e `AsMut` sejam bastante simples, fornecer traits padrão e genéricos para conversão de referência evita a proliferação de traits de conversão mais específicas. Você deve evitar definir seus próprios traits `AsFoo` quando poderia apenas implementar `AsRef<Foo>`.

Borrow e BorrowMut

O trait `std::borrow::Borrow` é semelhante a `AsRef`: se um tipo implementa `Borrow<T>`, então seu método `borrow` empresta eficientemente um $\&T$ dele. Mas `Borrow` impõe mais restrições: um tipo deve implementar `Borrow<T>` somente quando um $\&T$ obtém sua chave hash e comparado da mesma maneira que o valor do qual é emprestado. (O Rust não impõe isso; é apenas a intenção documentada do trait.) Isso torna `Borrow` valioso ao lidar com chaves em tabelas hash e árvores ou ao lidar com valores que utilizam hash ou que são

comparados por algum outro motivo.

Essa distinção é importante quando emprestamos de `String`s, por exemplo: `String` implementa `AsRef<str>`, `AsRef<[u8]>` e `AsRef<Path>`, mas esses três tipos de destino geralmente terão valores de hash diferentes. Apenas a fatia `&str` tem garantidamente o mesmo hash da `String` equivalente, assim `String` implementa apenas `Borrow<str>`.

A definição de `Borrow` é idêntica à de `AsRef`; apenas os nomes foram alterados:

```
trait Borrow<Borrowed: ?Sized> {  
    fn borrow(&self) -> &Borrowed;  
}
```

`Borrow` foi projetado para lidar com uma situação específica com tabelas hash genéricas e outros tipos de coleção associativa. Por exemplo, suponha que você tenha um `std::collections::HashMap<String, i32>`, mapeando strings para números. As chaves dessa tabela são `String`s; cada entrada possui uma. Qual deve ser a assinatura do método que procura uma entrada nessa tabela? Eis uma primeira tentativa:

```
impl<K, V> HashMap<K, V> where K: Eq + Hash  
{  
    fn get(&self, key: K) -> Option<&V> { ... }  
}
```

Isso faz sentido: para procurar uma entrada, você deve fornecer uma chave do tipo apropriado para a tabela. Mas, nesse caso, `K` é `String`; essa assinatura iria forçá-lo a passar uma `String` por valor para cada chamada a `get`, o que é claramente um desperdício. Você realmente só precisa de uma referência à chave:

```
impl<K, V> HashMap<K, V> where K: Eq + Hash  
{  
    fn get(&self, key: &K) -> Option<&V> { ... }  
}
```

Isso é um pouco melhor, mas agora você tem de passar a chave como um `&String`; portanto, se quiser procurar uma constante string, precisará escrever:

```
hashtable.get(&"twenty-two".to_string())
```

Isso é ridículo: aloca uma `String` buffer no heap e copia o texto para ela, apenas para que possa emprestá-la como uma `&String`, passá-la

para `get` e, então, dropá-la.

Isso deve ser bom o suficiente para passar qualquer coisa que suporte hash e deva ser comparado com nosso tipo de chave; uma `&str` deve ser perfeitamente adequada, por exemplo. Então aqui está a iteração final, que é o que você encontrará na biblioteca padrão:

```
impl<K, V> HashMap<K, V> where K: Eq + Hash
{
    fn get<Q: ?Sized>(&self, key: &Q) -> Option<&V>
        where K: Borrow<Q>,
              Q: Eq + Hash
    { ... }
}
```

Em outras palavras, se você pode pegar emprestada a chave de uma entrada como `&Q` e a referência resultante tem o mesmo hash e pode ser comparada exatamente como ocorreria com a própria chave, então claramente `&Q` deve ser um tipo de chave aceitável. Como a `String` implementa `Borrow<str>` e `Borrow<String>`, essa versão final de `get` permite que você passe qualquer uma, `&String` ou `&str`, como uma chave, conforme necessário.

`Vec<T>` e `[T; N]` implementam `Borrow<[T]>`. Cada tipo semelhante a string permite o empréstimo de seu tipo de fatia correspondente: `String` implementa `Borrow<str>`, `PathBuf` implementa `Borrow<Path>` e assim por diante. E todos os tipos de coleção associativa da biblioteca padrão utilizam `Borrow` para decidir quais tipos podem ser passados para suas funções de pesquisa.

A biblioteca padrão inclui uma implementação geral para que cada tipo `T` seja emprestado de si mesmo: `T: Borrow<T>`. Isso garante que `&K` é sempre um tipo aceitável para procurar entradas em um `HashMap<K, V>`.

Por conveniência, cada tipo `&mut T` também implementa `Borrow<T>`, retornando uma referência compartilhada `&T` como sempre. Isso permite que você passe referências mutáveis para funções de pesquisa de coleção sem ter de emprestar novamente uma referência compartilhada, emulando a coerção implícita usual do Rust de referências mutáveis para referências compartilhadas.

O trait `BorrowMut` é o análogo de `Borrow` para referências mutáveis:

```
trait BorrowMut<Borrowed: ?Sized>: Borrow<Borrowed> {
    fn borrow_mut(&mut self) -> &mut Borrowed;
}
```

As mesmas expectativas descritas para `Borrow` aplicam-se a `BorrowMut` também.

From e Into

Os traits `std::convert::From` e `std::convert::Into` representam conversões que consomem um valor de um tipo e retornam um valor de outro. Considerando que os traits `AsRef` e `AsMut` emprestam uma referência de um tipo de outro, `From` e `Into` tomam posse de seu argumento, transformam-no e, em seguida, retornam a posse do resultado de volta ao chamador.

Suas definições são bem simétricas:

```
trait Into<T>: Sized {
    fn into(self) -> T;
}
trait From<T>: Sized {
    fn from(other: T) -> Self;
}
```

A biblioteca padrão implementa automaticamente a conversão trivial de cada tipo para si mesmo: cada tipo `T` implementa `From<T>` e `Into<T>`.

Embora os traits simplesmente forneçam duas maneiras de fazer a mesma coisa, elas se prestam a usos diferentes.

Você geralmente utiliza `Into` para tornar suas funções mais flexíveis nos argumentos que aceitam. Por exemplo, se você escrever:

```
use std::net::Ipv4Addr;
fn ping<A>(address: A) -> std::io::Result<bool>
    where A: Into<Ipv4Addr>
{
    let ipv4_address = address.into();
    ...
}
```

então `ping` pode aceitar não apenas um `Ipv4Addr` como argumento, mas também `u32` ou um array `[u8; 4]`, já que esses tipos convenientemente implementam `Into<Ipv4Addr>`. (Às vezes é útil tratar

um endereço IPv4 como um único valor de 32 bits ou um array de 4 bytes.) Como a única coisa que `ping` sabe sobre `address` é que implementa `Into<Ipv4Addr>`, não há necessidade de especificar qual tipo você deseja ao chamar `into`; há apenas um que poderia funcionar, então a inferência de tipo o preenche para você.

Como ocorre com `AsRef` na seção anterior, o efeito é muito parecido com o de sobrecarregar uma função em C++. Com a definição de `ping` de antes, podemos fazer qualquer uma dessas chamadas:

```
println!("{:?}", ping(Ipv4Addr::new(23, 21, 68, 141))); // passa um Ipv4Addr
println!("{:?}", ping([66, 146, 219, 98]));           // passa um [u8; 4]
println!("{:?}", ping(0xd076eb94_u32));              // passa um u32
```

O `trait From`, porém, desempenha um papel diferente. O método `from` serve como um construtor genérico para produzir uma instância de um tipo de algum outro valor único. Por exemplo, em vez de `Ipv4Addr` tendo dois métodos nomeados `from_array` e `from_u32`, ele simplesmente implementa `From<[u8;4]>` e `From<u32>`, permitindo-nos escrever:

```
let addr1 = Ipv4Addr::from([66, 146, 219, 98]);
let addr2 = Ipv4Addr::from(0xd076eb94_u32);
```

Podemos deixar que a inferência de tipo resolva qual implementação se aplica.

Dada uma implementação de `From` apropriada, a biblioteca padrão implementa automaticamente o `trait Into` correspondente. Ao definir seu próprio tipo, se ele tiver construtores de argumento único, você deve escrevê-los como implementações de `From<T>` para os tipos apropriados; você receberá as implementações de `Into` correspondente de graça.

Como os métodos de conversão `from` e `into` assumem a posse de seus argumentos, uma conversão pode reutilizar os recursos do valor original para construir o valor convertido. Por exemplo, suponha que você escreva:

```
let text = "Beautiful Soup".to_string();
let bytes: Vec<u8> = text.into();
```

A implementação de `Into<Vec<u8>>` para `String` simplesmente pega o heap do buffer `String` e o reaproveita, inalterado, como o buffer de elementos do vetor retornado. A conversão não tem necessidade de

alocar ou copiar o texto. Esse é outro caso em que as movimentações permitem implementações eficientes.

Essas conversões também fornecem uma boa maneira de relaxar um valor de um tipo restrito em algo mais flexível, sem enfraquecer as garantias do tipo restrito. Por exemplo, uma `String` garante que seu conteúdo seja sempre UTF-8 válido; seus métodos de mutação são cuidadosamente restritos para garantir que nada que você possa fazer introduzirá um UTF-8 inválido. Mas esse exemplo eficientemente “rebaixa” um `String` para um bloco de bytes simples com o qual você pode fazer o que quiser: talvez o comprima ou combine com outros dados binários que não sejam UTF-8. Como `into` toma seu argumento por valor, `text` não é mais inicializado após a conversão, o que significa que podemos acessar livremente a antiga `String` do buffer sem ser capaz de corromper qualquer existente `String`.

Contudo, conversões baratas não fazem parte do contrato de `Into` e `From`. Enquanto espera-se que as conversões de `AsRef` e `AsMut` sejam baratas, as conversões `From` e `Into` podem alocar, copiar ou processar o conteúdo do valor. Por exemplo, `String` implementa `From<&str>`, que copia a fatia de string em um novo buffer alocado em heap para a `String`. E `std::collections::BinaryHeap<T>` implementa `From<Vec<T>>`, que compara e reordena os elementos de acordo com os requisitos de seu algoritmo.

O operador `?` utiliza `From` e `Into` para ajudar a limpar o código em funções que podem falhar de várias maneiras, convertendo automaticamente de tipos de erros específicos para gerais quando necessário.

Por exemplo, imagine um sistema que precisa ler dados binários e converter parte deles de números de base 10 escritos como texto UTF-8. Isso significa utilizar `std::str::from_utf8` e a implementação de `FromStr` para `i32`, o que pode retornar erros de tipos diferentes. Supondo que utilizamos os tipos `GenericError` e `GenericResult` que definimos no Capítulo 7 ao discutir o tratamento de erros, o operador `?` fará a conversão para nós:

```
type GenericError = Box<dyn std::error::Error + Send + Sync + 'static>;  
type GenericResult<T> = Result<T, GenericError>;
```



```
fn parse_i32_bytes(b: &[u8]) -> GenericResult<i32> {
    Ok(std::str::from_utf8(b)?.parse::<i32>())?
}
```

Como a maioria dos tipos de erro, `Utf8Error` e `ParseIntError` implementam o `trait Error` e a biblioteca padrão nos dá uma cobertura do `From impl` para converter de qualquer coisa que implemente `Error` para um `Box<dyn Error>`, o qual ? utiliza automaticamente:

```
impl<'a, E: Error + Send + Sync + 'a> From<E>
for Box<dyn Error + Send + Sync + 'a> {
    fn from(err: E) -> Box<dyn Error + Send + Sync + 'a> {
        Box::new(err)
    }
}
```

Isso transforma o que teria sido uma função razoavelmente grande com duas instruções `match` em uma única linha.

Antes de `From` e `Into` serem adicionados à biblioteca padrão, o código Rust estava cheio de traits de conversão *ad hoc* e métodos de construção, cada um específico a um único tipo. `From` e `Into` codificam as convenções que você pode seguir para tornar seus tipos mais fáceis de utilizar, pois seus usuários já estão familiarizados com eles. Outras bibliotecas e a própria linguagem também podem contar com esses traits como uma maneira canônica e padronizada de codificar conversões.

`From` e `Into` são traits infalíveis – sua API exige que as conversões não falhem. Infelizmente, muitas conversões são mais complexas do que isso. Por exemplo, inteiros grandes como `i64` podem armazenar números muito maiores do que `i32`, e converter um número como `2_000_000_000_000i64` em um `i32` não faz muito sentido sem algumas informações adicionais. Fazer uma conversão bit a bit simples, na qual os primeiros 32 bits são descartados, geralmente não produz o resultado que esperamos:

```
let huge = 2_000_000_000_000i64;
let smaller = huge as i32;
println!("{}", smaller); // -1454759936
```

Existem muitas opções para lidar com essa situação. Dependendo do contexto, essa conversão por “encapsulamento (wrapping)” pode ser apropriada. Por outro lado, aplicações como processamento de sinal

digital e sistemas de controle muitas vezes podem se contentar com uma conversão de “saturação”, na qual números maiores que o valor máximo possível são limitados a esse máximo.

TryFrom e TryInto

Como não está claro como essa conversão deve se comportar, o Rust não implementa `From<i64>` para `i32`, ou qualquer outra conversão entre tipos numéricos que perderiam informações. Em vez disso, `i32` implementa `TryFrom<i64>`. `TryFrom` e `TryInto` são os primos que podem falhar de `From` e `Into` e são igualmente recíprocos; implementar `TryFrom` significa que `TryInto` também é implementado.

Suas definições são apenas um pouco mais complexas do que `From` e `Into`.

```
pub trait TryFrom<T>: Sized {
    type Error;
    fn try_from(value: T) -> Result<Self, Self::Error>;
}

pub trait TryInto<T>: Sized {
    type Error;
    fn try_into(self) -> Result<T, Self::Error>;
}
```

O método `try_into()` nos dá um `Result`, para que possamos escolher o que fazer em casos excepcionais, como um número muito grande para caber no tipo resultante:

```
// Satura no caso de estouro, em vez de encapsular
let smaller: i32 = huge.try_into().unwrap_or(i32::MAX);
```

Se quisermos também lidar com o caso negativo, podemos utilizar o método `unwrap_or_else()` de `Result`:

```
let smaller: i32 = huge.try_into().unwrap_or_else(|_| {
    if huge >= 0 {
        i32::MAX
    } else {
        i32::MIN
    }
});
```

Implementar conversões que podem falhar para seus próprios tipos também é fácil. O tipo `Error` pode ser tão simples, ou tão complexo,

quanto exigir uma determinada aplicação. A biblioteca padrão utiliza um struct vazio, não fornecendo nenhuma informação além do fato de que ocorreu um erro, pois o único erro possível é um estouro. Por outro lado, as conversões entre tipos mais complexos podem querer retornar mais informações:

```
impl TryInto<LinearShift> for Transform {
    type Error = TransformError;

    fn try_into(self) -> Result<LinearShift, Self::Error> {
        if !self.normalized() {
            return Err(TransformError::NotNormalized);
        }
        ...
    }
}
```

Onde `From` e `Into` relacionam tipos com conversões simples, `TryFrom` e `TryInto` estendem a simplicidade das conversões `From` e `Into` com o expressivo tratamento de erros proporcionado por `Result`. Esses quatro traits podem ser utilizados juntos para relacionar vários tipos em um único crate.

ToOwned

Dada uma referência, a maneira usual de produzir uma cópia com posse de seu referente é chamar `clone`, assumindo que o tipo implementa `std::clone::Clone`. Mas e se você quiser clonar um `&str` ou um `&[i32]`? O que você provavelmente quer é uma `String` ou um `Vec<i32>`, mas a definição de `Clone` não permite isso: por definição, a clonagem de um `&T` deve sempre retornar um valor do tipo `T`, e `str` e `[u8]` são não dimensionados; eles nem são tipos que uma função poderia retornar.

O trait `std::borrow::ToOwned` fornece uma maneira um pouco mais flexível de converter uma referência em um valor possuído:

```
trait ToOwned {
    type Owned: Borrow<Self>;
    fn to_owned(&self) -> Self::Owned;
}
```

Diferentemente de `clone`, que deve retornar exatamente `Self`, `to_owned` pode devolver qualquer coisa da qual que você poderia emprestar

um `&Self`: o tipo `Owned` deve implementar `Borrow<Self>`. Você pode emprestar um `&[T]` a partir de um `Vec<T>`, então `[T]` pode implementar `ToOwned<Owned=Vec<T>>`, desde que `T` implemente `Clone`, para que possamos copiar os elementos da fatia para o vetor. De forma similar, `str` implementa `ToOwned<Owned=String>`, `Path` implementa `ToOwned<Owned=PathBuf>` e assim por diante.

Borrow e ToOwned em funcionamento: Cow (“clone on write”)

Fazer bom uso do Rust envolve pensar em questões de posse, como se uma função deve receber um parâmetro por referência ou por valor. Normalmente, você pode escolher uma abordagem ou outra e o tipo de parâmetro reflete sua decisão. Mas, em alguns casos, você não pode decidir se quer tomar emprestado ou possuir até que o programa esteja em execução; o tipo `std::borrow::Cow` (para “clone on write”) fornece uma maneira de fazer isso.

Sua definição é mostrada aqui:

```
enum Cow<'a, B: ?Sized>
    where B: ToOwned
{
    Borrowed(&'a B),
    Owned(<B as ToOwned>::Owned),
}
```

Um `Cow<B>` ou toma emprestada uma referência compartilhada a um `B` ou possui um valor do qual poderíamos emprestar tal referência. Como o `Cow` implementa `Deref`, você pode chamar métodos nele como se fosse uma referência compartilhada para um `B`: se for `Owned`, ele toma emprestada uma referência compartilhada ao valor possuído; e se for `Borrowed`, ele apenas entrega a referência que está segurando.

Você também pode obter uma referência mutável para o valor de um `Cow` chamando seu método `to_mut`, que retorna um `&mut B`. Se `Cow` acontece de ser `Cow::Borrowed`, `to_mut` simplesmente chama o método `to_owned` da referência para obter sua própria cópia do referente, altera `Cow` dentro de `Cow::Owned` e toma emprestada uma referência

mutável ao valor recém-possuído. Esse é o comportamento “clone on write” ao qual o nome do tipo se refere.

De forma similar, `Cow` tem um método `into_owned` que promove a referência a um valor possuído, se necessário, e depois o retorna, movendo a posse para o chamador e consumindo o `Cow` no processo.

Um uso comum para `Cow` é retornar uma constante de string alocada estaticamente ou uma string computada. Por exemplo, suponha que você precise converter uma enumeração de erro em uma mensagem. A maioria das variantes pode ser tratada com strings fixas, mas algumas delas possuem dados adicionais que devem ser incluídos na mensagem. Você pode devolver um `Cow<'static, str>`:

```
use std::path::PathBuf;
use std::borrow::Cow;
fn describe(error: &Error) -> Cow<'static, str> {
    match *error {
        Error::OutOfMemory => "out of memory".into(),
        Error::StackOverflow => "stack overflow".into(),
        Error::MachineOnFire => "machine on fire".into(),
        Error::Unfathomable => "machine bewildered".into(),
        Error::FileNotFound(ref path) => {
            format!("file not found: {}", path.display()).into()
        }
    }
}
```

Esse código utiliza implementação `Cow` de `Into` para construir os valores. A maioria dos braços dessa instrução `match` retorna um `Cow::Borrowed` referenciando uma string alocada estaticamente. Mas, quando recebemos uma variante `FileNotFound`, utilizamos `format!` para construir uma mensagem incorporando o nome de arquivo dado. Esse braço da instrução `match` produz um valor `Cow::Owned`.

Chamadores de `describe` que não precisam alterar o valor podem simplesmente tratar o `Cow` como um `&str`:

```
println!("Disaster has struck: {}", describe(&error));
```

Os chamadores que precisam de um valor possuído podem prontamente produzir um:

```
let mut log: Vec<String> = Vec::new();
...
log.push(describe(&error).into_owned());
```

Utilizar `Cow` ajuda `describe` e seus chamadores a adiar a alocação até o momento em que ela se torna necessária.

¹ N.R.: Código de colagem (glue code) é um código que não contribui para nenhuma funcionalidade do programa, e serve somente para unir diferentes partes de um código que de outra forma não seriam compatíveis. Fonte: Wikipédia.

CAPÍTULO 14

Closures

Salve o meio ambiente! Crie uma closure hoje!
– Cormac Flanagan

Ordenar um vetor de números inteiros é fácil:

```
integers.sort();
```

É, portanto, um triste fato que, quando queremos alguns dados ordenados, dificilmente é um vetor de números inteiros. Normalmente, temos algum tipo de registro e o método `sort` interno normalmente não funciona:

```
struct City {  
    name: String,  
    population: i64,  
    country: String,  
    ...  
}
```

```
fn sort_cities(cities: &mut Vec<City>) {  
    cities.sort(); // erro: como você quer que eles sejam ordenados?  
}
```

O Rust reclama que `City` não implementa `std::cmp::Ord`. Precisamos especificar a ordem de classificação, assim:

```
/// Função auxiliar para ordenar cidades por população  
fn city_population_descending(city: &City) -> i64 {  
    -city.population  
}  
  
fn sort_cities(cities: &mut Vec<City>) {  
    cities.sort_by_key(city_population_descending); // ok  
}
```

A função auxiliar, `city_population_descending`, aceita um registro `City` e extrai a chave, o campo pelo qual queremos ordenar nossos dados. (Ela retorna um número negativo porque `sort` organiza os números em ordem crescente e queremos ordem decrescente: a cidade mais populosa primeiro.) O método `sort_by_key` utiliza essa função-chave

como um parâmetro.

Isso funciona bem, mas é mais conciso escrever a função auxiliar como uma closure, uma expressão de função anônima:

```
fn sort_cities(cities: &mut Vec<City>) {  
    cities.sort_by_key(|city| -city.population);  
}
```

A closure aqui é `|city| -city.population`. Ela recebe um argumento `city` e retorna `-city.population`. O Rust infere o tipo de argumento e o tipo de retorno de como a closure é utilizada.

Outros exemplos de recursos de biblioteca padrão que aceitam closures incluem:

- Métodos `Iterator` como `map` e `filter`, para trabalhar com dados sequenciais. Abordaremos esses métodos no Capítulo 15.
- APIs de `threading` como `thread::spawn`, que inicia um novo thread do sistema. A concorrência tem tudo a ver com mover o trabalho para outros threads e as closures representam convenientemente unidades de trabalho. Abordaremos esses recursos no Capítulo 19.
- Alguns métodos que precisam calcular condicionalmente um valor padrão, como o método `or_insert_with` de entradas `HashMap`. Esse método obtém ou cria uma entrada em um `HashMap` e é utilizado quando o valor padrão é caro para calcular. O valor padrão é passado como uma closure, que é chamada somente se uma nova entrada deve ser criada.

É claro que as funções anônimas estão por toda parte hoje em dia, mesmo em linguagens como Java, C#, Python e C++, que originalmente não as tinham. De agora em diante, assumiremos que você já viu funções anônimas antes e focaremos no que torna as closures do Rust um pouco diferentes. Neste capítulo, você aprenderá os três tipos de closures, como utilizar closures com métodos de biblioteca padrão, como uma closure pode “capturar” variáveis em seu escopo, como escrever suas próprias funções e métodos que aceitam closures como argumentos, e como armazenar closures para uso posterior, como chamadas de retorno. Também explicaremos como as closures do Rust são implementadas e por que são mais rápidas do que você imagina.

Capturando variáveis

Uma closure pode utilizar dados que pertencem a uma função que a contém. Por exemplo:

```
/// Ordena por qualquer uma das várias estatísticas diferentes
fn sort_by_statistic(cities: &mut Vec<City>, stat: Statistic) {
    cities.sort_by_key(|city| -city.get_statistic(stat));
}
```

A closure aqui utiliza `stat`, que existe na função que contém a closure (que a envolve), `sort_by_statistic`. Dizemos que a closure “captura” `stat`. Esse é um dos recursos clássicos das closures, então, naturalmente, o Rust o suporta; mas, no Rust, esse recurso vem com um detalhe importante.

Na maioria das linguagens com closures, a coleta de lixo desempenha um papel importante. Por exemplo, considere este código JavaScript:

```
// Inicia uma animação que reorganiza as linhas em uma tabela de cidades
function startSortingAnimation(cities, stat) {
    // Função auxiliar que utilizaremos para ordenar a tabela
    // Observe que esta função referencia stat
    function keyfn(city) {
        return city.get_statistic(stat);
    }

    if (pendingSort)
        pendingSort.cancel();

    // Agora inicia uma animação, passando keyfn para ela
    // O algoritmo de ordenação chamará keyfn posteriormente
    pendingSort = new SortingAnimation(cities, keyfn);
}
```

A closure `keyfn` é armazenada no novo objeto `SortingAnimation`. É para ser chamada depois que `startSortingAnimation` retorna. Agora, normalmente quando uma função retorna, todas as suas variáveis e argumentos saem do escopo e são descartadas. Mas, aqui, o mecanismo JavaScript deve manter `stat` por perto de alguma forma, já que a closure o utiliza. A maioria dos mecanismos JavaScript faz isso alocando `stat` no heap e deixando o coletor de lixo recuperá-lo mais tarde.

O Rust não tem coleta de lixo. Como isso vai funcionar? Para

responder a essa pergunta, veremos dois exemplos.

Closures que pedem emprestado

Primeiro, vamos repetir o exemplo de abertura desta seção:

```
/// Ordena por qualquer uma das várias estatísticas diferentes
fn sort_by_statistic(cities: &mut Vec<City>, stat: Statistic) {
    cities.sort_by_key(|city| -city.get_statistic(stat));
}
```

Neste caso, quando o Rust cria a closure, ele automaticamente pega emprestada uma referência a `stat`. Isso tem uma razão: a closure referencia `stat`, portanto deve ter uma referência a ele.

O resto é simples. A closure está sujeita às regras sobre empréstimo e tempo de vida que descrevemos no Capítulo 5. Em particular, uma vez que a closure contém uma referência a `stat`, o Rust não vai deixá-la sobreviver a `stat`. Como a closure é utilizada apenas durante a ordenação, esse exemplo é bom.

Resumindo, o Rust garante segurança utilizando tempos de vida em vez de coleta de lixo (CL). O caminho do Rust é mais rápido: mesmo uma alocação rápida do coletor de lixo seria mais lenta do que armazenar `stat` no stack, como o Rust faz nesse caso.

Closures que roubam

O segundo exemplo é mais complicado:

```
use std::thread;

fn start_sorting_thread(mut cities: Vec<City>, stat: Statistic)
    -> thread::JoinHandle<Vec<City>>
{
    let key_fn = |city: &City| -> i64 { -city.get_statistic(stat) };
    thread::spawn(|| {
        cities.sort_by_key(key_fn);
        cities
    })
}
```

Isso é um pouco mais parecido com o que nosso exemplo de JavaScript estava fazendo: `thread::spawn` pega uma closure e a chama em um novo thread do sistema. Observe que `||` é a lista de

argumentos vazia da closure.

O novo thread é executado em paralelo com o chamador. Quando a closure retorna, o novo thread sai. (O valor de retorno da closure é enviado de volta para o thread de chamada como um valor `JoinHandle`. Abordaremos isso no Capítulo 19.)

Mais uma vez, a closure `key_fn` contém uma referência a `stat`. Mas, dessa vez, o Rust não pode garantir que a referência seja utilizada com segurança. O Rust, portanto, rejeita esse programa:

```
error: closure may outlive the current function, but it borrows `stat`,
       which is owned by the current function
33 | let key_fn = |city: &City| -> i64 { -city.get_statistic(stat) };
   |               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |               |                                     `stat` is borrowed here
   |               |                                     may outlive borrowed value `stat`
```

Na verdade, há dois problemas aqui, porque `cities` também é compartilhado de forma insegura. Simplesmente, não se pode esperar que o novo thread criada por `thread::spawn` termine seu trabalho antes que `cities` e `stat` sejam destruídos no final da função.

A solução para ambos os problemas é a mesma: diga ao Rust que mova `cities` e `stat` para as closures que os utilizam em vez de pegar emprestadas referências a eles.

```
fn start_sorting_thread(mut cities: Vec<City>, stat: Statistic)
-> thread::JoinHandle<Vec<City>>
{
    let key_fn = move |city: &City| -> i64 { -city.get_statistic(stat) };

    thread::spawn(move || {
        cities.sort_by_key(key_fn);
        cities
    })
}
```

A única coisa que mudamos é adicionar o palavra-chave `move` antes de cada um dos dois closures. A palavra-chave `move` diz ao Rust que uma closure não pega emprestadas as variáveis que utiliza: ela as rouba.

A primeira closure, `key_fn`, toma posse de `stat`. Em seguida, a segunda closure toma posse tanto de `cities` como de `key_fn`.

O Rust, portanto, oferece duas maneiras para closures obter dados de escopos envoltivos: movimentos e empréstimos. Realmente não há nada mais a dizer do que isso; closures seguem as mesmas regras sobre movimentos e empréstimos que já abordamos nos capítulos 4 e 5. Alguns casos em destaque:

- Assim como em qualquer outra parte da linguagem, se uma closure `move` um valor de um tipo copiável, como `i32`, ela copia o valor. Então, se `Statistic` passou a ser um tipo copiável, poderíamos continuar utilizando `stat` mesmo depois de criar uma closure `move` que o utiliza.
- Valores de tipos não copiáveis, como `Vec<City>`, são realmente movidos: o código precedente transfere `cities` para o novo segmento, por meio da closure `move`. O Rust não nos deixava acessar `cities` pelo nome depois de criar a closure.
- Acontece que esse código não precisa utilizar `cities` após o ponto onde a closure o `move`. Se o fizéssemos, porém, a solução alternativa seria fácil: poderíamos dizer ao Rust que clonasse `cities` e armazenasse a cópia em uma variável diferente. A closure roubaria apenas uma das cópias – qualquer que seja a que ela referencia.

Obtemos algo importante ao aceitar as regras estritas do Rust: segurança de threads. É precisamente porque o vetor é movido, em vez de ser compartilhado entre os threads, que sabemos que o thread antigo não liberará o vetor enquanto o novo thread o estiver modificando.

Função e tipos de closure

Ao longo deste capítulo, vimos funções e closures utilizadas como valores. Naturalmente, isso significa que elas têm tipos. Por exemplo:

```
fn city_population_descending(city: &City) -> i64 {  
    -city.population  
}
```

Essa função recebe um argumento (um `&City`) e retorna um `i64`. Tem o tipo `fn(&City) -> i64`.

Você pode fazer as mesmas coisas com funções que faz com outros valores. Pode armazená-las em variáveis. Pode utilizar toda a sintaxe usual do Rust para calcular os valores da função:

```
let my_key_fn: fn(&City) -> i64 =
    if user.prefs.by_population {
        city_population_descending
    } else {
        city_monster_attack_risk_descending
    };
```

```
cities.sort_by_key(my_key_fn);
```

Structs podem ter campos tipados como função. Tipos genéricos como `Vec` podem armazenar um grande número de funções, desde que todas compartilhem o mesmo tipo `fn`. E o custo de armazenagem de valores do tipo função é pequeno: um valor do tipo `fn` é apenas o endereço de memória do código de máquina da função, assim como um ponteiro de função em C++.

Uma função pode receber outra função como argumento. Por exemplo:

```
/// Dada uma lista de cidades e uma função de teste,
/// retorna quantas cidades passam no teste
fn count_selected_cities(cities: &Vec<City>,
                        test_fn: fn(&City) -> bool) -> usize
{
    let mut count = 0;
    for city in cities {
        if test_fn(city) {
            count += 1;
        }
    }
    count
}

/// Um exemplo de uma função de teste. Observe que o tipo dessa
/// função é `fn(&City) -> bool`, o mesmo que
/// o argumento `test_fn` para `count_selected_cities`.
fn has_monster_attacks(city: &City) -> bool {
    city.monster_attack_risk > 0.0
}
```

```
// Quantas cidades estão em risco de ataque de monstros?
let n = count_selected_cities(&my_cities, has_monster_attacks);
```

Se você estiver familiarizado com ponteiros de função em C/C++,

verá que os valores de função do Rust são exatamente a mesma coisa.

Depois de tudo isso, pode ser uma surpresa que as closures não tenham o mesmo tipo que as funções:

```
let limit = preferences.acceptable_monster_risk();
let n = count_selected_cities(
    &my_cities,
    |city| city.monster_attack_risk > limit); // erro: tipo incompatível
```

O segundo argumento causa um erro de tipo. Para oferecer suporte a closures, devemos alterar a assinatura de tipo dessa função. Precisa ficar assim:

```
fn count_selected_cities<F>(cities: &Vec<City>, test_fn: F) -> usize
where F: Fn(&City) -> bool
{
    let mut count = 0;
    for city in cities {
        if test_fn(city) {
            count += 1;
        }
    }
    count
}
```

Alteramos apenas a assinatura de tipo de `count_selected_cities`, não o corpo. A nova versão é genérica. Ela aceita um `test_fn` de qualquer tipo `F` desde que `F` implemente o trait especial `Fn(&City) -> bool`. Esse trait é implementado automaticamente por todas as funções e a maioria das closures que levam um único `&City` como um argumento e retornam um valor booleano:

```
fn(&City) -> bool    // tipo fn (somente funções)
Fn(&City) -> bool    // Trait Fn (funções e closures)
```

Essa sintaxe especial é parte da linguagem. O `->` e o tipo de retorno são opcionais; se omitidos, o tipo de retorno é `()`.

A nova versão de `count_selected_cities` aceita uma função ou uma closure:

```
count_selected_cities(
    &my_cities,
    has_monster_attacks); // ok

count_selected_cities(
    &my_cities,
```

```
|city| city.monster_attack_risk > limit); // também ok
```

Por que nossa primeira tentativa não funcionou? Bem, uma closure pode ser chamada, mas não é uma `fn`. A closure `|city| city.monster_attack_risk > limit` tem seu próprio tipo que não é um tipo `fn`.

Na verdade, cada closure que você escreve tem seu próprio tipo, porque uma closure pode conter dados: valores emprestados ou roubados de escopos envoltivos. Isso pode ser qualquer número de variáveis, em qualquer combinação de tipos. Portanto, cada closure tem um tipo *ad hoc* criado pelo compilador, grande o suficiente para conter esses dados. Não há duas closures exatamente do mesmo tipo. Mas toda closure implementa um trait `Fn`; a closure em nossos exemplos implementa `Fn(&City) -> i64`.

Como cada closure tem seu próprio tipo, o código que funciona com closures geralmente precisa ser genérico, como `count_selected_cities`. É um pouco chato escrever os tipos genéricos um de cada vez, mas, para ver as vantagens desse design, continue lendo.

Desempenho de closure

As closures do Rust são projetadas para serem rápidas: mais rápidas do que os ponteiros de função, rápidas o suficiente para que você possa utilizá-las mesmo em códigos sensíveis ao desempenho. Se você estiver familiarizado com lambdas do C++, descobrirá que as closures do Rust são tão rápidas e compactas, porém mais seguras.

Na maioria das linguagens, as closures são alocadas no heap, despachadas dinamicamente e liberadas pelo coletor de lixo. Portanto, criar, chamar e coletar cada uma delas custa um pouquinho de tempo extra de CPU. Pior ainda, as closures tendem a descartar o *inlining*, uma técnica chave que os compiladores utilizam para eliminar o overhead da chamada de função e habilitar uma série de outras otimizações. No final das contas, as closures são lentas o suficiente nessas linguagens para que valha a pena removê-las manualmente de loops internos apertados¹.

As closures do Rust não têm nenhuma dessas desvantagens de desempenho. Elas não sofrem coleta de lixo. Como tudo no Rust,

elas não são alocadas no heap, a menos que você as coloque em um `Box`, `Vec` ou outro contêiner. E, como cada closure tem um tipo distinto, sempre que o compilador do Rust souber o tipo de closure que você está chamando, ele pode incorporar o código para essa closure específica. Isso torna aceitável o uso de closures em loops apertados e os programas Rust costumam fazer isso com entusiasmo, como você verá no Capítulo 15.

A Figura 14.1 mostra como as closures do Rust são dispostas na memória. Na parte superior da figura, mostramos algumas variáveis locais que nossas closures vão referenciar: uma string `food` e uma enumeração simples `weather`, cujo valor numérico casualmente é 27.

A closure (a) utiliza ambas as variáveis. Aparentemente, estamos procurando cidades que tenham tacos e tornados. Na memória, essa closure parece um pequeno struct contendo referências às variáveis que ele utiliza.

Observe que ela não contém um ponteiro para seu código! Isso não é necessário: contanto que o Rust conheça o tipo da closure, ele saberá qual código executar quando você a chamar.

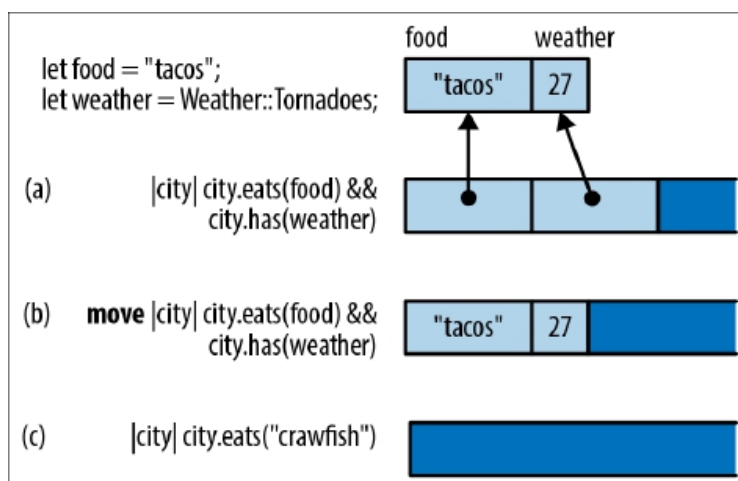


Figura 14.1: Layout de closures na memória.

A closure (b) é exatamente a mesma, exceto que é uma closure `move`, portanto contém valores em vez de referências.

A closure (c) não utiliza nenhuma variável do seu ambiente. O struct está vazio, então essa closure não ocupa nenhuma memória.

Como mostra a figura, essas closures não ocupam muito espaço. Mas mesmo esses poucos bytes nem sempre são necessários na prática. Frequentemente, o compilador pode colocar inline todas as chamadas a uma closure e, em seguida, até mesmo os pequenos structs mostrados nessa figura são otimizados.

Em “Callbacks”, na página 395, mostraremos como alocar closures no heap e chamá-los dinamicamente, utilizando objetos `trait`. Isso é um pouco mais lento, mas ainda é tão rápido quanto qualquer outro método de objeto `trait`.

Closures e segurança

Ao longo deste capítulo até aqui, falamos sobre como o Rust garante que as closures respeitem as regras de segurança da linguagem quando elas pegam emprestadas ou movem variáveis do código ao redor. Mas há algumas outras consequências que não são exatamente óbvias. Nesta seção, explicaremos um pouco mais sobre o que acontece quando uma closure dropa ou modifica um valor capturado.

Closures que matam

Vimos closures que emprestam valores e closures que os roubam; era apenas uma questão de tempo até que elas ficassem totalmente ruins.

Naturalmente, matar não é realmente a terminologia certa. No Rust, nós dropamos *valores*. A maneira mais direta de fazer isso é chamar `drop()`:

```
let my_str = "hello".to_string();  
let f = || drop(my_str);
```

Quando `f` é chamada, `my_str` é dropado.

Então, o que acontece se a chamarmos duas vezes?

```
f();  
f();
```

Vamos pensar bem. A primeira vez que chamamos `f`, ela dropa `my_str`, o que significa que a memória onde a string está armazenada é

dropada, retornada ao sistema. A segunda vez que chamamos `f`, a mesma coisa acontece. É um *double free*, um erro clássico na programação C++ que aciona um comportamento indefinido.

Dropar uma `String` duas vezes seria uma ideia igualmente ruim no Rust. Felizmente, o Rust não pode ser enganado tão facilmente:

```
f(); // ok
f(); // erro: uso de valor movido
```

O Rust sabe que essa closure não pode ser chamada duas vezes.

Uma closure que pode ser chamada apenas uma vez pode parecer uma coisa bastante extraordinária, mas temos falado ao longo deste livro sobre posse e tempos de vida. A ideia de valores sendo utilizados (ou seja, movidos) é um dos conceitos centrais do Rust. Funciona da mesma forma com closures como com todo o resto.

FnOnce

Vamos tentar uma vez mais, para enganar o Rust e fazê-lo dropar uma `String` duas vezes. Dessa vez, utilizaremos esta função genérica:

```
fn call_twice<F>(closure: F) where F: Fn() {
    closure();
    closure();
}
```

Essa função genérica pode receber qualquer closure que implemente o trait `Fn()`: isto é, closures que não recebem argumentos e retornam `()`. (Assim como com funções, o tipo de retorno pode ser omitido se for `()`; `Fn()` é uma abreviação para `Fn() -> ()`.)

Agora, o que acontece se passarmos nossa closure insegura para essa função genérica?

```
let my_str = "hello".to_string();
let f = || drop(my_str);
call_twice(f);
```

Mais uma vez, a closure vai dropar `my_str` quando for chamado. Chamar duas vezes seria um *double free*. Mas, novamente, o Rust não se deixa enganar:

```
error: expected a closure that implements the `Fn` trait, but
      this closure only implements `FnOnce`
|
```

```

8 | let f = || drop(my_str);
  |           ^^^^^^^^^^-----^
  |           |           |
  |           |           | closure is `FnOnce` because it moves the variable `my_str`
  |           |           | out of its environment
  |           |           | this closure implements `FnOnce`, not `Fn`
9 | call_twice(f);
  | ----- the requirement to implement `Fn` derives from here

```

Essa mensagem de erro nos diz mais sobre como o Rust lida com “closures que matam”. Elas poderiam ter sido totalmente banidas da linguagem, mas às vezes as closures de limpeza são úteis. Então, em vez disso, o Rust restringe seu uso. Closures que dropam valores, como `f`, não têm permissão para ter `Fn`. Elas não são, literalmente, uma `Fn` de forma alguma. Elas implementam um trait menos poderoso, `FnOnce`, o trait de closures que podem ser chamados uma vez.

Na primeira vez que você chama uma closure `FnOnce`, *a closure em si é consumida*. É como se os dois traits, `Fn` e `FnOnce`, tivessem sido definidos assim:

```

// Pseudocódigo para traits `Fn` e `FnOnce` sem argumentos
trait Fn() -> R {
    fn call(&self) -> R;
}

trait FnOnce() -> R {
    fn call_once(self) -> R;
}

```

Assim como uma expressão aritmética como `a + b` é uma abreviação para uma chamada de método, `Add::add(a, b)`, o Rust trata `closure()` como abreviação para um dos dois métodos de trait mostrados no exemplo anterior. Para uma closure `Fn`, `closure()` expande para `closure.call()`. Esse método recebe `self` por referência, então a closure não é movida. Mas, se a closure for segura apenas para chamar uma vez, então `closure()` expande-se para `closure.call_once()`. Esse método recebe `self` por valor, então a closure é consumida.

É claro que estamos deliberadamente criando problemas aqui utilizando `drop()`. Na prática, você entrará nessa situação por acidente. Isso não acontece com frequência, mas de vez em quando você escreverá algum código de closure que inadvertidamente utiliza

um valor:

```
let dict = produce_glossary();
let debug_dump_dict = || {
  for (key, value) in dict { // ops!
    println!("{:?}", key, value);
  }
};
```

Então, quando você chamar `debug_dump_dict()` mais de uma vez, receberá uma mensagem de erro como esta:

```
error: use of moved value: `debug_dump_dict`
|
19 |   debug_dump_dict();
|   ----- `debug_dump_dict` moved due to this call
20 |   debug_dump_dict();
|   ^^^^^^^^^^^^^^^^^ value used here after move
|
note: closure cannot be invoked more than once because it moves the variable
`dict` out of its environment
|
13 |   for (key, value) in dict {
|   ^^^^^
```

Para depurar isso, temos de descobrir por que a closure é uma `FnOnce`. Qual valor está sendo utilizado aqui? O compilador prestativamente aponta que é `dict`, que neste caso é o único que estamos referenciando. Ah, aí está o bug: estamos gastando `dict` iterando por ele diretamente. Deveríamos estar fazendo um loop por `&dict`, em vez de um simples `dict`, para acessar os valores por referência:

```
let debug_dump_dict = || {
  for (key, value) in &dict { // não utiliza dict
    println!("{:?}", key, value);
  }
};
```

Isso corrige o erro; a função agora é uma `Fn` e pode ser chamada quantas vezes você quiser.

FnMut

Existe mais um tipo de closure, o tipo que contém dados mutáveis ou referências `mut`.

O Rust considera valores não-`mut` seguros para compartilhar entre threads. Mas não seria seguro compartilhar closures não-`mut` que contêm dados `mut`: chamar tal closure de vários threads pode levar a todo tipo de condição de concorrência (race condition), pois vários threads tentam ler e gravar os mesmos dados ao mesmo tempo.

Portanto, o Rust tem mais uma categoria de closure, `FnMut`, a categoria de closures que escrevem, gravam ou modificam valores. Closures `FnMut` são chamadas por referência `mut`, como se fossem definidas assim:

```
// Pseudocódigo para os traits `Fn`, `FnMut` e `FnOnce`
trait Fn() -> R {
    fn call(&self) -> R;
}

trait FnMut() -> R {
    fn call_mut(&mut self) -> R;
}

trait FnOnce() -> R {
    fn call_once(self) -> R;
}
```

Qualquer closure que exija acesso `mut` a um valor, mas não dropa nenhum valor, é uma closure `FnMut`. Por exemplo:

```
let mut i = 0;
let incr = || {
    i += 1; // incr toma emprestada uma referência mut a i
    println!("Ding! i is now: {}", i);
};
call_twice(incr);
```

A maneira como escrevemos `call_twice` requer uma `Fn`. Como a `incr` é uma `FnMut` e não uma `Fn`, esse código falha ao compilar. Mas há uma solução fácil. Para entender a correção, vamos dar um passo atrás e resumir o que você aprendeu sobre as três categorias de closures do Rust.

- `Fn` é a família de closures e funções que você pode chamar várias vezes sem restrição. Essa categoria mais elevada também inclui todas as funções `fn`.
- `FnMut` é a família de closures que podem ser chamadas várias vezes se a própria closure for declarada `mut`.

- `FnOnce` é a família de closures que podem ser chamadas uma vez, se o chamador for o proprietário da closure.

Cada `Fn` atende aos requisitos de `FnMut`, e cada `FnMut` atende aos requisitos de `FnOnce`. Conforme mostrado na Figura 14.2, elas não são três categorias separadas.

Em vez disso, `Fn()` é um subtrait de `FnMut()`, que é um subtrait de `FnOnce()`. Isso torna `Fn` a categoria mais exclusiva e poderosa. `FnMut` e `FnOnce` são categorias mais amplas que incluem closures com restrições de uso.

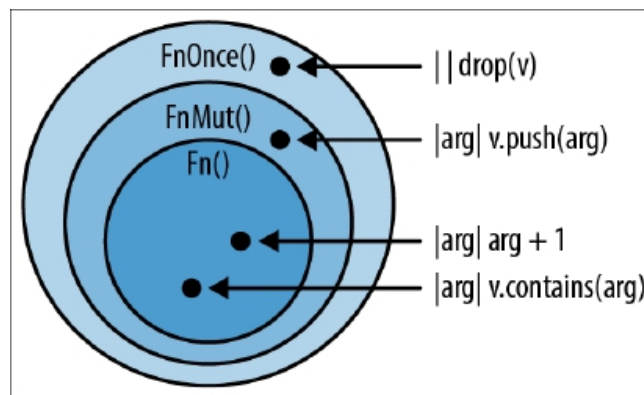


Figura 14.2: Diagrama de Venn das três categorias de closure.

Agora que organizamos o que sabemos, está claro que, para aceitar a maior faixa possível de closures, nossa função `call_twice` realmente deve aceitar todas as closures `FnMut`, assim:

```
fn call_twice<F>(mut closure: F) where F: FnMut() {
    closure();
    closure();
}
```

O limite na primeira linha era `F: Fn()` e agora é `F: FnMut()`. Com essa mudança, ainda aceitamos todas as closures `Fn` e também podemos utilizar `call_twice` em closures que modificam (mutam) dados:

```
let mut i = 0;
call_twice(|i| i += 1); // ok!
assert_eq!(i, 2);
```

Copy e Clone para closures

Assim como o Rust descobre automaticamente quais closures podem

ser chamadas apenas uma vez, ele pode descobrir quais closures podem ser implementadas `Copy` e `Clone` e quais não podem.

Como explicamos anteriormente, as closures são representadas como structs que contêm os valores (para closures `move`) ou referências aos valores (para closures `não-move`) das variáveis que eles capturam. As regras para `Copy` e `Clone` em closures são como as regras para `Copy` e `Clone` em structs normais. Uma closure `não-move` que não modifica variáveis contém apenas referências compartilhadas, que são `Clone` e `Copy`, de modo que a closure é tanto `Clone` como `Copy`:

```
let y = 10;
let add_y = |x| x + y;
let copy_of_add_y = add_y;           // essa closure é `Copy`, então...
assert_eq!(add_y(copy_of_add_y(22)), 42); // ... podemos chamar ambos
```

Por outro lado, uma closure `não-move` que *realmente* modifica valores têm referências mutáveis dentro de sua representação interna. Referências mutáveis não são nem `Clone` nem `Copy`, assim como nenhuma closure que as utiliza:

```
let mut x = 0;
let mut add_to_x = |n| { x += n; x };

let copy_of_add_to_x = add_to_x;      // isso move, em vez de copiar
assert_eq!(add_to_x(copy_of_add_to_x(1)), 2); // erro: uso de valor movido
```

Para a closure `move`, as regras são ainda mais simples. Se tudo que uma closure `move` captura é `Copy`, ela é `Copy`. Se tudo que ela captura é `Clone`, ela é `Clone`. Por exemplo:

```
let mut greeting = String::from("Hello, ");
let greet = move |name| {
    greeting.push_str(name);
    println!("{}", greeting);
};
greet.clone()("Alfred");
greet.clone()("Bruce");
```

Essa sintaxe de `.clone()(...)` é um pouco estranha, mas significa apenas que clonamos a closure e depois chamamos o clone. Esse programa produz:

```
Hello, Alfred
Hello, Bruce
```

Quando `greeting` é utilizado em `greet`, ele é movido para o struct que representa `greet` internamente, porque é uma closure `move`. Então, quando clonamos `greet`, tudo dentro dele também é clonado. Existem duas cópias de `greeting`, que são modificadas separadamente quando os clones de `greet` são chamados. Isso não é tão útil por si só, mas, quando você precisa passar a mesma closure para mais de uma função, pode ser muito útil.

Callbacks

Muitas bibliotecas utilizam chamadas de retorno (callbacks) como parte de sua API: funções fornecidas pelo usuário, para a biblioteca chamar posteriormente. Na verdade, você já viu algumas APIs como essa neste livro. De volta ao Capítulo 2, utilizamos o framework `actix-web` para escrever um servidor web simples. Uma parte importante desse programa era o roteador, que se parecia com isto:

```
App::new()
    .route("/", web::get().to(get_index))
    .route("/gcd", web::post().to(post_gcd))
```

O objetivo do roteador é rotear as solicitações recebidas da internet para a parte do código Rust que lida com esse tipo específico de solicitação. Nesse exemplo, `get_index` e `post_gcd` eram os nomes das funções que declaramos em outra parte do programa, utilizando a palavra-chave `fn`. Mas poderíamos ter passado closures, assim:

```
App::new()
    .route("/", web::get().to(|| {
        HttpResponse::Ok()
            .content_type("text/html")
            .body("<title>GCD Calculator</title>...")
    }))
    .route("/gcd", web::post().to(|form: web::Form<GcdParameters>| {
        HttpResponse::Ok()
            .content_type("text/html")
            .body(format!("The GCD of {} and {} is {}.",
                form.n, form.m, gcd(form.n, form.m)))
    })))
```

Isso é porque `actix-web` foi escrito para aceitar qualquer `Fn` thread-safe como argumento.

Como podemos fazer isso em nossos próprios programas? Vamos tentar escrever nosso próprio roteador muito simples do zero, sem utilizar nenhum código de `actix-web`. Podemos começar declarando alguns tipos para representar solicitações e respostas HTTP:

```
struct Request {  
    method: String,  
    url: String,  
    headers: HashMap<String, String>,  
    body: Vec<u8>  
}  
  
struct Response {  
    code: u32,  
    headers: HashMap<String, String>,  
    body: Vec<u8>  
}
```

Agora, o trabalho de um roteador é simplesmente armazenar uma tabela que mapeia URLs para chamadas de retorno, para que a chamada de retorno correta possa ser feita sob demanda. (Para simplificar, permitiremos apenas que os usuários criem rotas que correspondam a um único URL exato.)

```
struct BasicRouter<C> where C: Fn(&Request) -> Response {  
    routes: HashMap<String, C>  
}  
  
impl<C> BasicRouter<C> where C: Fn(&Request) -> Response {  
    /// Cria um roteador vazio  
    fn new() -> BasicRouter<C> {  
        BasicRouter { routes: HashMap::new() }  
    }  
  
    /// Adiciona uma rota ao roteador  
    fn add_route(&mut self, url: &str, callback: C) {  
        self.routes.insert(url.to_string(), callback);  
    }  
}
```

Infelizmente, cometemos um erro. Você notou isso?

Esse roteador funciona bem, desde que adicionemos apenas uma rota a ele:

```
let mut router = BasicRouter::new();  
router.add_route("/", |_| get_form_response());
```

Isso compila e executa. Infelizmente, se adicionarmos outra rota:

```
router.add_route("/gcd", |req| get_gcd_response(req));
```

então obtemos erros:

error: mismatched types

```
|
41 | router.add_route("/gcd", |req| get_gcd_response(req));
    |                               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    |                               expected closure, found a different closure
    |
= note: expected type `[closure@closures_bad_router.rs:40:27: 40:50]`
      found type `[closure@closures_bad_router.rs:41:30: 41:57]`
note: no two closures, even if identical, have the same type
help: consider boxing your closure and/or using it as a trait object
```

Nosso erro estava em como definimos o tipo `BasicRouter`:

```
struct BasicRouter<C> where C: Fn(&Request) -> Response {
    routes: HashMap<String, C>
}
```

Declaramos involuntariamente que cada `BasicRouter` tem um único tipo de chamada de retorno `c` e todas as chamadas de retorno no `HashMap` são desse tipo. Lá atrás, em “Qual utilizar”, na página 307, mostramos um tipo `Salad` que tinha o mesmo problema:

```
struct Salad<V: Vegetable> {
    veggies: Vec<V>
}
```

A solução aqui é a mesma da salada: como queremos suportar uma variedade de tipos, precisamos utilizar boxes e objetos `trait`:

```
type BoxedCallback = Box<dyn Fn(&Request) -> Response>;

struct BasicRouter {
    routes: HashMap<String, BoxedCallback>
}
```

Cada box pode conter um tipo diferente de closure, portanto um único `HashMap` pode conter todos os tipos de chamada de retorno. Observe que o parâmetro de tipo `c` sumiu.

Isso requer alguns ajustes nos métodos:

```
impl BasicRouter {
    // Cria um roteador vazio
    fn new() -> BasicRouter {
        BasicRouter { routes: HashMap::new() }
    }

    // Adiciona uma rota ao roteador
```

```

fn add_route<C>(&mut self, url: &str, callback: C)
  where C: Fn(&Request) -> Response + 'static
{
  self.routes.insert(url.to_string(), Box::new(callback));
}

```



Observe os dois limites em `c` na assinatura de tipo para `add_route`: um trait `Fn` específico e o tempo de vida `'static`. O Rust nos faz adicionar esse limite `'static`. Sem isso, a chamada a `Box::new(callback)` seria um erro, porque não é seguro armazenar uma closure se ela contiver referências emprestadas a variáveis que estão prestes a sair do escopo.

Por fim, nosso roteador simples está pronto para lidar com as solicitações recebidas:

```

impl BasicRouter {
  fn handle_request(&self, request: &Request) -> Response {
    match self.routes.get(&request.url) {
      None => not_found_response(),
      Some(callback) => callback(request)
    }
  }
}

```

Ao custo de alguma flexibilidade, também poderíamos escrever uma versão mais eficiente em termos de espaço desse roteador que, em vez de armazenar objetos `trait`, utilizamos ponteiros de função ou tipos `fn`. Esses tipos, como `fn(u32) -> u32`, comportam-se de maneira muito parecida com as closures:

```

fn add_ten(x: u32) -> u32 {
  x + 10
}

let fn_ptr: fn(u32) -> u32 = add_ten;
let eleven = fn_ptr(1); //11

```

Na verdade, as closures que não capturam nada de seu ambiente são idênticas aos ponteiros de função, pois não precisam conter nenhuma informação extra sobre as variáveis capturadas. Se você especificar o tipo `fn` apropriado, seja em uma vinculação ou em uma

assinatura de função, o compilador ficará feliz em permitir que você os utilize dessa maneira:

```
let closure_ptr: fn(u32) -> u32 = |x| x + 1;
let two = closure_ptr(1); // 2
```

Ao contrário da captura de closures, esses ponteiros de função ocupam apenas um único `usize`.

Uma tabela de roteamento que contém ponteiros de função ficaria assim:

```
struct FnPointerRouter {
    routes: HashMap<String, fn(&Request) -> Response>
}
```

Aqui o `HashMap` está armazenando apenas um único `usize` por `String` e, criticamente, não há nenhum `Box`. Além do `HashMap` em si, não há nenhuma alocação dinâmica. Naturalmente, os métodos também precisam ser ajustados:

```
impl FnPointerRouter {
    // Cria um roteador vazio
    fn new() -> FnPointerRouter {
        FnPointerRouter { routes: HashMap::new() }
    }
    // Adiciona uma rota ao roteador
    fn add_route(&mut self, url: &str, callback: fn(&Request) -> Response)
    {
        self.routes.insert(url.to_string(), callback);
    }
}
```

Conforme apresentado na Figura 14.1, as closures têm tipos únicos porque cada uma captura uma variável diferente; portanto, entre outras coisas, cada uma delas tem um tamanho diferente. Se elas não capturarem nada, porém, não há nada para armazenar. Utilizando ponteiros `fn` em funções que recebem chamadas de retorno, você pode restringir um chamador a utilizar apenas essas closures sem captura, ganhando algum desempenho e flexibilidade dentro do código, utilizando chamada de retorno ao custo de flexibilidade para os usuários de sua API.

Usando closures de forma eficaz

Como vimos, as closures do Rust são diferentes das closures da maioria das outras linguagens. A maior diferença é que, em linguagens com Coleta de Lixo (Garbage Collection), você pode utilizar variáveis locais em uma closure sem ter de pensar em tempo de vida ou posse. Sem Coleta de Lixo, as coisas são diferentes. Alguns padrões de design comuns em Java, C# e JavaScript não funcionarão no Rust sem alterações.

Por exemplo, pegue o padrão de projeto Model-View-Controller (MVC), ilustrado na Figura 4.3. Para cada elemento de uma interface de usuário, um framework MVC cria três objetos: um *modelo* representando o estado desse elemento de interface do usuário, uma *visualização* que é responsável por sua aparência e um *controlador* que lida com a interação do usuário. Inúmeras variações do MVC foram implementadas ao longo dos anos, mas a ideia geral é que três objetos dividem as responsabilidades da interface do usuário de alguma forma.

Eis o problema. Normalmente, cada objeto tem uma referência a um ou a ambos os outros, diretamente ou por meio de uma chamada de retorno, conforme mostrado na Figura 14.3. Sempre que algo acontece com um dos objetos, ele avisa os outros, então tudo é atualizado prontamente. A questão de qual objeto “possui” os outros nunca surge.

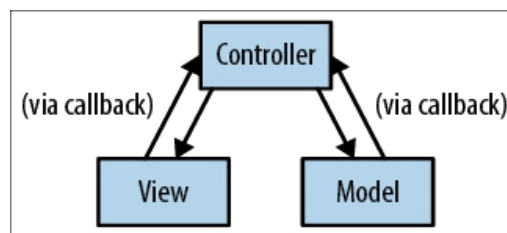


Figura 14.3: Padrão de projeto Model-View-Controller.

Você não pode implementar esse padrão no Rust sem fazer algumas alterações. A posse deve ser explicitada e os ciclos de referência devem ser eliminados. O modelo e o controlador não podem ter referências diretas entre si.

A aposta radical do Rust é que existem bons projetos alternativos. Às vezes, você pode corrigir um problema com a posse e os tempos

de vida da closure fazendo com que cada closure receba as referências necessárias como argumentos. Às vezes, você pode atribuir um número a cada coisa no sistema e passar os números em vez de referências. Ou você pode implementar uma das muitas variações no MVC onde os objetos não têm referências uns aos outros. Ou modele seu kit de ferramentas de acordo com um sistema não MVC com fluxo de dados unidirecional, como o da arquitetura Flux do Facebook, mostrada na Figura 14.4.

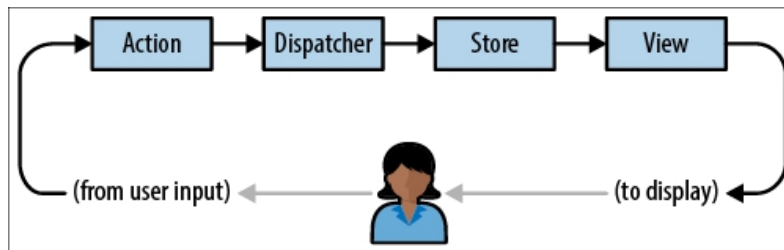


Figura 14.4: Arquitetura Flux, uma alternativa ao MVC.

Resumindo, se você tentar utilizar closures do Rust para fazer um “mar de objetos”, terá dificuldades. Mas existem alternativas. Nesse caso, parece que a engenharia de software como disciplina já está gravitando para as alternativas de qualquer maneira, porque são mais simples.

No próximo capítulo, abordaremos um tópico em que as closures realmente se destacam. Vamos escrever um tipo de código que tira o máximo proveito da concisão, velocidade e eficiência das closures do Rust e que é divertido de escrever, fácil de ler e eminentemente prático. A seguir: Iteradores do Rust.

¹ N.R.: Loop apertado (tight loop) é um tipo de loop que tem poucas instruções e itera muitas vezes.

CAPÍTULO 15

Iteradores

Era o fim de um dia muito longo.

– Phil

Um *iterador* é um valor que produz uma sequência de valores, geralmente para um loop operar. A biblioteca padrão do Rust fornece iteradores que percorrem vetores, strings, tabelas hash e outras coleções, mas também iteradores para produzir linhas de texto de um fluxo de entrada, conexões que chegam a um servidor de rede, valores recebidos de outros threads em um canal de comunicação e assim por diante. E, naturalmente, você pode implementar iteradores para seus próprios propósitos. O loop `for` do Rust fornece uma sintaxe natural para utilizar iteradores, mas os próprios iteradores também fornecem um rico conjunto de métodos para mapear, filtrar, unir, coletar etc.

Os iteradores do Rust são flexíveis, expressivos e eficientes. Considere a seguinte função, que retorna a soma dos primeiros n inteiros positivos (muitas vezes chamados de *n -ésimo número triangular*):

```
fn triangle(n: i32) -> i32 {  
    let mut sum = 0;  
    for i in 1..=n {  
        sum += i;  
    }  
    sum  
}
```

A expressão `1..=n` é um valor `RangeInclusive<i32>`. Um `RangeInclusive<i32>` é um iterador que produz os inteiros a partir de seu valor inicial até seu valor final (inclusive), então você pode utilizá-lo como o operando do loop `for` para somar os valores de 1 até n .

Mas os iteradores também têm um método `fold`, que você pode

utilizar na definição equivalente:

```
fn triangle(n: i32) -> i32 {  
    (1..=n).fold(0, |sum, item| sum + item)  
}
```

Começando com 0 como o total corrente, `fold` toma cada valor que `1..=n` produz e aplica a closure `|sum, item| sum + item` ao total acumulado e ao valor. O valor de retorno da closure é considerado o novo total corrente. O último valor que retorna é o que o próprio `fold` retorna – nesse caso, o total da sequência inteira. Isso pode parecer estranho se você estiver acostumado com loops `for` e `while`, mas, uma vez que você se acostumou com isso, `fold` é uma alternativa legível e concisa.

Isso é um padrão comum para linguagens de programação funcionais, que valorizam a expressividade. Mas os iteradores do Rust foram cuidadosamente projetados para garantir que o compilador também possa traduzi-los em excelente código de máquina. Em uma versão release da segunda definição mostrada anteriormente, o Rust conhece a definição de `fold` e a coloca inline em `triangle`. A seguir, a closure `|sum, item| sum + item` também é colocada inline. Por fim, o Rust examina o código combinado e reconhece que existe uma maneira mais simples de somar os números de 1 a n : a soma é sempre igual a $n * (n+1) / 2$. O Rust traduz todo o corpo de `triangle`, `loop`, closure e tudo, em uma única instrução de multiplicação e alguns outros bits de aritmética.

Esse exemplo envolve aritmética simples, mas os iteradores também funcionam bem quando utilizados de forma mais complexa. Eles são outro exemplo do Rust fornecendo abstrações flexíveis que impõem pouco ou nenhum overhead no uso típico.

Neste capítulo, vamos explicar:

- Os traits `Iterator` e `IntoIterator`, que são a base dos iteradores do Rust.
- Os três estágios de um pipeline iterador típico: criar um iterador a partir de algum tipo de origem de valor; adaptar um tipo de iterador para outro, selecionar ou processar valores conforme eles passam; e, em seguida, consumir os valores que o iterador produz.

- Como implementar iteradores para seus próprios tipos.

Existem muitos métodos, então é ok dar apenas uma olhada rápida na seção depois de ter uma ideia geral. Mas os iteradores são muito comuns no Rust idiomático e estar familiarizado com as ferramentas que os acompanham é essencial para dominar a linguagem.

Traits `Iterator` e `IntoIterator`

Um iterador é qualquer valor que implementa o trait `std::iter::Iterator`:

```
trait Iterator {  
    type Item;  
    fn next(&mut self) -> Option<Self::Item>;  
    ... // muitos métodos padrão  
}
```

`Item` é o tipo de valor que o iterador produz. O método `next` retorna `Some(v)`, em que `v` é o próximo valor do iterador, ou retorna `None` para indicar o fim da sequência. Aqui omitimos muitos métodos padrão de `Iterator`; vamos abordá-los individualmente ao longo deste capítulo.

Se houver uma maneira natural de iterar por algum tipo, esse tipo pode implementar `std::iter::IntoIterator`, cujo método `into_iter` pega um valor e retorna um iterador sobre ele:

```
trait IntoIterator where Self::IntoIter: Iterator<Item=Self::Item> {  
    type Item;  
    type IntoIter: Iterator;  
    fn into_iter(self) -> Self::IntoIter;  
}
```

`IntoIter` é o tipo do próprio valor do iterador e `Item` é o tipo de valor que ele produz. Chamamos qualquer tipo que implementa `IntoIterator`, um *iterável*, porque é algo pelo qual você poderia iterar se quisesse.

O loop `for` do Rust reúne todas essas partes muito bem. Para iterar sobre os elementos de um vetor, você pode escrever:

```
println!("There's:");  
let v = vec!["antimony", "arsenic", "aluminum", "selenium"];  
  
for element in &v {  
    println!("{}", element);  
}
```

Na verdade, cada loop `for` é apenas um atalho para chamadas aos

métodos `IntoIterator` e `Iterator`:

```
let mut iterator = (&v).into_iter();
while let Some(element) = iterator.next() {
    println!("{}", element);
}
```

O loop `for` utiliza `IntoIterator::into_iter` para converter seu operando `&v` em um iterador e, em seguida, chama `Iterator::next` repetidamente. Cada vez que retorna `Some(element)`, o loop `for` executa seu corpo; e, se retornar `None`, o loop termina.

Com esse exemplo em mente, eis alguma terminologia para iteradores:

- Como dissemos, um *iterador* é qualquer tipo que implementa `Iterator`.
- Um *iterável* é qualquer tipo que implementa `IntoIterator`: você pode obter um iterador sobre ele chamando seu método `into_iter`. A referência do vetor `&v` é o iterável nesse caso.
- Um iterador *produz* valores.
- Os valores que um iterador produz são *Itens*. Aqui, os itens são "antimony", "arsenic" etc.
- O código que recebe os itens que um iterador produz é o *consumidor*. Nesse exemplo, o loop `for` é o consumidor.

Embora um loop `for` sempre chame `into_iter` em seu operando, você também pode passar iteradores para loops `for` diretamente; isso ocorre quando você faz um loop sobre um `Range`, por exemplo. Todos os iteradores implementam automaticamente `IntoIterator`, com um método `into_iter` que simplesmente retorna o iterador.

Se você chamar um método iterador `next` novamente depois que ele retornou `None`, o trait `Iterator` não especifica o que deve fazer. A maioria dos iteradores apenas retornará `None` novamente, mas não todos. (Se isso causar problemas, o adaptador `fuse` abordado em "fuse", na página 422, pode ajudar.)

Criando iteradores

A documentação da biblioteca padrão Rust explica em detalhes que

tipo de iteradores cada tipo fornece, mas a biblioteca segue algumas convenções gerais para ajudá-lo a se orientar e encontrar o que precisa.

Métodos `iter` e `iter_mut`

A maioria dos tipos de coleção fornece métodos `iter` e `iter_mut`, que retornam os iteradores naturais sobre o tipo, produzindo uma referência compartilhada ou mutável para cada item. Fatias de array como `&[T]` e `&mut [T]` também têm métodos `iter` e `iter_mut`. Esses métodos são a maneira mais comum de obter um iterador, se não vai deixar um loop `for` cuidar disso para você:

```
let v = vec![4, 20, 12, 8, 6];
let mut iterator = v.iter();
assert_eq!(iterator.next(), Some(&4));
assert_eq!(iterator.next(), Some(&20));
assert_eq!(iterator.next(), Some(&12));
assert_eq!(iterator.next(), Some(&8));
assert_eq!(iterator.next(), Some(&6));
assert_eq!(iterator.next(), None);
```

O tipo de item desse iterador é `&i32`: cada chamada a `next` produz uma referência ao próximo elemento, até chegarmos ao final do vetor.

Cada tipo é livre para implementar `iter` e `iter_mut` da maneira que fizer mais sentido para o seu propósito. O método `iter` em `std::path::Path` retorna um iterador que produz um componente `path` por vez:

```
use std::ffi::OsStr;
use std::path::Path;

let path = Path::new("C:/Users/JimB/Downloads/Fedora.iso");
let mut iterator = path.iter();
assert_eq!(iterator.next(), Some(OsStr::new("C:")));
assert_eq!(iterator.next(), Some(OsStr::new("Users")));
assert_eq!(iterator.next(), Some(OsStr::new("JimB")));
...
```

O tipo de item desse iterador é `&std::ffi::OsStr`, uma fatia emprestada de uma string do tipo aceito pelas chamadas do sistema operacional. Se houver mais de uma maneira comum de iterar por um tipo, o tipo geralmente fornece métodos específicos para cada tipo de percurso,

pois um simples método `iter` seria ambíguo. Por exemplo, não há método `iter` no tipo de fatia `&str` de `string`. Em vez disso, se `s` é um `&str`, então `s.bytes()` retorna um iterador que produz cada byte de `s`, enquanto `s.chars()` interpreta o conteúdo como UTF-8 e produz cada caractere Unicode.

Implementações de `IntoIterator`

Quando um tipo implementa `IntoIterator`, você mesmo pode chamar seu método `into_iter`, assim como um loop `for` seria:

```
// Você normalmente deve utilizar HashSet, mas sua ordem de iteração é
// não determinista, então BtreeSet funciona melhor em exemplos
use std::collections::BTreeSet;
let mut favorites = BTreeSet::new();
favorites.insert("Lucy in the Sky With Diamonds".to_string());
favorites.insert("Liebesträume No. 3".to_string());

let mut it = favorites.into_iter();
assert_eq!(it.next(), Some("Liebesträume No. 3".to_string()));
assert_eq!(it.next(), Some("Lucy in the Sky With Diamonds".to_string()));
assert_eq!(it.next(), None);
```

A maioria das coleções realmente fornece várias implementações de `IntoIterator`, para referências compartilhadas (`&T`), referências mutáveis (`&mut T`) e movimentos (`T`):

- Dada uma *referência compartilhada* à coleção, `into_iter` retorna um iterador que produz referências compartilhadas para seus itens. Por exemplo, no código anterior, `(&favorites).into_iter()` retornaria um iterador cujo tipo `Item` é `&String`.
- Dada uma *referência mutável* à coleção, `into_iter` retorna um iterador que produz referências mutáveis aos itens. Por exemplo, se `vector` é algum `Vec<String>`, a chamada a `(&mut vector).into_iter()` retorna um iterador cujo tipo `Item` é `&mut String`.
- Quando recebe a coleção *por valor*, `into_iter` retorna um iterador que se apropria da coleção e retorna itens por valor; a posse dos itens passa da coleção para o consumidor e a coleção original é consumida no processo. Por exemplo, a chamada `favorites.into_iter()` no código anterior retorna um iterador que produz cada string por valor; o consumidor recebe a posse de cada string. Quando o

iterador é dropado, todos os elementos restantes no `BTreeSet` também são dropados e a estrutura agora vazia do conjunto é dropada.

Como um loop `for` aplica `IntoIterator::into_iter` ao seu operando, essas três implementações são o que criam os seguintes protocolos para iterar por referências compartilhadas ou mutáveis a uma coleção ou consumir a coleção e tomar posse de seus elementos:

```
for element in &collection { ... }  
for element in &mut collection { ... }  
for element in collection { ... }
```

Cada um deles simplesmente resulta em uma chamada para uma das implementações de `IntoIterator` listadas aqui.

Nem todo tipo fornece todas as três implementações. Por exemplo, `HashSet`, `BTreeSet` e `BinaryHeap` não implementam `IntoIterator` em referências mutáveis, pois modificar seus elementos provavelmente violaria as invariantes do tipo: o valor modificado pode ter um valor hash diferente ou ser ordenado de maneira diferente em relação a seus vizinhos; portanto, modificá-lo o deixaria posicionado incorretamente. Outros tipos suportam mutação, mas apenas parcialmente. Por exemplo, `HashMap` e `BTreeMap` produzem referências mutáveis aos valores de suas entradas, mas apenas referências compartilhadas às suas chaves, por motivos semelhantes aos dados anteriormente.

O princípio geral é que a iteração deve ser eficiente e previsível; portanto, em vez de fornecer implementações caras ou que possam exibir um comportamento surpreendente (por exemplo, refazer o hash das entradas `HashSet` e potencialmente as encontrar novamente mais tarde na iteração), o Rust as omite completamente.

Fatias implementam duas das três variantes de `IntoIterator`; já que não possuem seus elementos, não há caso “por valor”. Em vez disso, `into_iter` para `&[T]` e `&mut [T]` retorna um iterador que produz referências compartilhadas e mutáveis aos elementos. Se você imaginar o tipo de fatia subjacente `[T]` como uma coleção de algum tipo, isso se encaixa perfeitamente no padrão geral.

Você deve ter notado que as duas primeiras variantes `IntoIterator`, para

referências compartilhadas e mutáveis, são equivalentes a chamar `iter` ou `iter_mut` no referente. Por que o Rust fornece ambos?

`IntoIterator` é o que faz loops `for` funcionarem, então isso é obviamente necessário. Mas, quando você não está utilizando um loop `for`, é mais claro escrever `favorites.iter()` que `(&favorites).into_iter()`. A iteração por referência compartilhada é algo de que você precisará com frequência, então `iter` e `iter_mut` ainda são valiosos por sua ergonomia.

`IntoIterator` também pode ser útil em código genérico : você pode utilizar um limite como `T: IntoIterator` para restringir a variável de tipo `T` a tipos que podem ser iterados. Ou você pode escrever `T: IntoIterator<Item=U>` para exigir ainda mais: que a iteração produza um tipo específico `U`. Por exemplo, essa função despeja valores de qualquer iterável cujos itens são imprimíveis com o formato "{:?}",

```
use std::fmt::Debug;

fn dump<T, U>(t: T)
  where T: IntoIterator<Item=U>,
        U: Debug
{
  for u in t {
    println!("{:?}", u);
  }
}
```

Você não pode escrever essa função genérica utilizando `iter` e `iter_mut`, já que não são métodos de nenhum `trait`: a maioria dos tipos iteráveis simplesmente tem métodos com esses nomes.

from_fn e sucessores

Uma maneira simples e geral de produzir uma sequência de valores é fornecer uma closure que os retorne.

Dada uma função que retorna `Option<T>`, `std::iter::from_fn` retorna um iterador que simplesmente chama a função para produzir seus itens. Por exemplo:

```
use rand::random; // Nas dependências de Cargo.toml: rand = "0.7"
use std::iter::from_fn;

// Gera os comprimentos de 1000 segmentos de linha aleatórios cujos pontos
// finais são uniformemente distribuídos ao longo do intervalo [0, 1].
```

```
// (Isso não é uma distribuição que você encontrará no crate `rand_distr`,
// mas é fácil de fazer você mesmo.)
let lengths: Vec<f64> =
    from_fn(|| Some((random::<f64>() - random::<f64>()).abs()))
        .take(1000)
        .collect();
```

Isso chama `from_fn` para fazer um iterador produzir números aleatórios. Como o iterador sempre retorna `Some`, a sequência nunca termina, mas chamamos `take(1000)` para limitá-lo aos primeiros 1.000 elementos. Então `collect` constrói o vetor da iteração resultante. Essa é uma maneira eficiente de construir vetores inicializados; nós explicamos por que em “Construindo coleções: `collect` e `FromIterator`”, na página 440, mais adiante neste capítulo.

Se cada item depende do anterior, a função `std::iter::successors` funciona bem. Você fornece um item inicial e uma função que pega um item e retorna uma `Option` do próximo. Se retornar `None`, a iteração termina. Por exemplo, eis outra maneira de escrever a função `escape_time` do nosso plotter do conjunto de Mandelbrot no Capítulo 2:

```
use num::Complex;
use std::iter::successors;

fn escape_time(c: Complex<f64>, limit: usize) -> Option<usize> {
    let zero = Complex { re: 0.0, im: 0.0 };
    successors(Some(zero), |&z| { Some(z * z + c) })
        .take(limit)
        .enumerate()
        .find(|(_i, z)| z.norm_sqr() > 4.0)
        .map(|(i, _z)| i)
}
```

Começando com zero, a chamada a `successors` produz uma sequência de pontos no plano complexo, elevando repetidamente o último ponto ao quadrado e adicionando o parâmetro `c`. Ao plotar o conjunto de Mandelbrot, queremos ver se essa sequência orbita perto da origem para sempre ou voa para o infinito. A chamada a `take(limit)` estabelece um limite de quanto tempo vamos perseguir a sequência e `enumerate` numera cada ponto, convertendo cada ponto `z` em uma tupla `(i, z)`. Utilizamos `find` para procurar o primeiro ponto que se afasta o suficiente da origem para escapar. O método `find` retorna um `Option`: `Some((i, z))` se existir, ou `None` caso contrário. A chamada a

`Option::map` transforma `Some((i, z))` em `Some(i)`, mas retorna `None` inalterado: esse é exatamente o valor de retorno que queremos.

Tanto `from_fn` como `successors` aceitam closures `FnMut`, para que suas closures possam capturar e modificar variáveis dos escopos circundantes. Por exemplo, esta função `fibonacci` utiliza uma closure `move` para capturar uma variável e utilizá-la como seu estado de execução:

```
fn fibonacci() -> impl Iterator<Item=usize> {
    let mut state = (0, 1);
    std::iter::from_fn(move || {
        state = (state.1, state.0 + state.1);
        Some(state.0)
    })
}

assert_eq!(fibonacci().take(8).collect::<Vec<_>>(),
           vec![1, 1, 2, 3, 5, 8, 13, 21]);
```

Uma nota de advertência: os métodos `from_fn` e `successors` são flexíveis o suficiente para que você possa transformar praticamente qualquer uso de iteradores em uma única chamada para um ou outro, passando closures complexas para obter o comportamento necessário. Mas fazer isso negligencia a oportunidade que os iteradores fornecem para esclarecer como os dados fluem pela computação e utiliza nomes padrão para padrões comuns. Certifique-se de estar familiarizado com os outros métodos iteradores deste capítulo antes de se apoiar nesses dois; muitas vezes há maneiras melhores de fazer o trabalho.

Métodos drain

Muitos tipos de coleção fornecem um método `drain` que utiliza uma referência mutável para a coleção e retorna um iterador que passa a posse de cada elemento para o consumidor. No entanto, ao contrário do método `into_iter()`, que pega a coleção por valor e a consome, `drain` simplesmente pega emprestada uma referência mutável à coleção e, quando o iterador é dropado, ele remove todos os elementos restantes da coleção e a deixa vazia.

Em tipos que podem ser indexados por um intervalo, como `Strings`,

vetores e `VecDeque`, o método `drain` pega uma série de elementos para remover, em vez de drenar toda a sequência:

```
let mut outer = "Earth".to_string();
let inner = String::from_iter(outer.drain(1..4));

assert_eq!(outer, "Eh");
assert_eq!(inner, "art");
```

Se você precisar drenar toda a sequência, utilize todo o intervalo, `..`, como o argumento.

Outras fontes do iterador

As seções anteriores tratam principalmente de tipos de coleção, como vetores e `HashMap`, mas há muitos outros tipos na biblioteca padrão que oferecem suporte à iteração. A Tabela 15.1 resume os mais interessantes, mas há muitos mais. Cobrimos alguns desses métodos com mais detalhes nos capítulos dedicados aos tipos específicos (a saber, capítulos 16, 17 e 18).

Tabela 15.1: Outros iteradores na biblioteca padrão

Tipo ou trait	Expressão	Notas
<code>std::ops::Range</code>	<code>1..10</code>	Os pontos de extremidade devem ser um tipo inteiro para serem iteráveis. O intervalo inclui o valor inicial e exclui o valor final.
	<code>(1..10).step_by(2)</code>	Produz 1, 3, 5, 7, 9.
<code>std::ops::RangeFrom</code>	<code>1..</code>	Iteração ilimitada. O início deve ser um número inteiro. Pode gerar um pânico ou overflow se o valor atingir o limite do tipo.
<code>std::ops::RangeInclusive</code>	<code>1..=10</code>	Como <code>Range</code> , mas inclui o valor final.
<code>Option<T></code>	<code>Some(10).iter()</code>	Comporta-se como um vetor cujo comprimento é 0 (<code>None</code>) ou 1 (<code>Some(v)</code>).
<code>Result<T, E></code>	<code>Ok("blah").iter()</code>	Igual a <code>Option</code> , produzindo valores <code>Ok</code> .
<code>Vec<T>, &[T]</code>	<code>v.windows(16)</code>	Produz cada fatia contígua de um determinado comprimento, da esquerda para a direita. As

Tipo ou trait	Expressão	Notas
		janelas se sobrepõem.
	v.chunks(16)	Produz fatias contíguas e não sobrepostas de um determinado comprimento, da esquerda para a direita.
	v.chunks_mut(1024)	Como chunks, mas as fatias são mutáveis.
	v.split(byte byte & 1 != 0)	Produz fatias separadas por elementos que correspondem ao predicado fornecido.
	v.split_mut(...)	Como acima, mas produz fatias mutáveis.
	v.rsplit(...)	Como split, mas produz fatias da direita para a esquerda.
	v.splitn(n, ...)	Como split, mas produz no máximo n fatias.
String, &str	s.bytes()	Produz os bytes da string.
	s.chars()	Produz os chars que o UTF-8 representa.
	s.split_whitespace()	Divide a string por espaço em branco e produz fatias de caracteres que não são espaços.
	s.lines()	Produz fatias das linhas da string.
	s.split('/')	Divide a string em um determinado padrão, produzindo as fatias entre as correspondências. Os padrões podem ser: caracteres, strings, closures.
	s.matches(char::is_numeric)	Produz fatias correspondentes ao padrão fornecido.
std::collections::HashMap, std::collections::BTreeMap	map.keys(), map.values()	Produz referências compartilhadas para chaves ou valores do mapa.
	map.values_mut()	Produz referências mutáveis aos valores das entradas.
std::collections::HashSet, std::collections::BTreeSet	set1.union(set2)	Produz referências compartilhadas a elementos de união de set1 e set2.
	set1.intersection(set2)	Produz referências compartilhadas a elementos de

Tipo ou trait	Expressão	Notas
		intersecção de set1 e set2.
std::sync::mpsc::Receiver	recv.iter()	Produz valores enviados de outro thread no correspondente Sender.
std::io::Read	stream.bytes()	Produz bytes de um fluxo de E/S.
	stream.chars()	Analisa o fluxo como UTF-8 e produz chars.
std::io::BufRead	bufstream.lines()	Analisa o fluxo como UTF-8, produz linhas como Strings.
	bufstream.split(0)	Divide o fluxo em determinado byte, produz buffers de Vec<u8> entre bytes.
std::fs::ReadDir	std::fs::read_dir(path)	Produz entradas de diretório.
std::net::TcpListener	listener.incoming()	Produz conexões de rede de entrada.
Funções livres	std::iter::empty()	Retorna None imediatamente.
	std::iter::once(5)	Produz o valor dado e depois termina.
	std::iter::repeat("#9")	Produz o valor dado para sempre.

Adaptadores iteradores

Depois de ter um iterador em mãos, o trait `Iterator` fornece uma ampla seleção de *métodos adaptadores*, ou simplesmente *adaptadores*, que consomem um iterador e criam um novo com comportamentos úteis. Para ver como os adaptadores funcionam, começaremos com dois dos adaptadores mais populares, `map` e `filter`. Em seguida, abordaremos o restante da caixa de ferramentas do adaptador, abrangendo quase todas as formas que você imaginar para criar sequências de valores a partir de outras sequências: truncamento, salto, combinação, reversão, concatenação, repetição e muito mais.

map e filter

O adaptador `map` do trait `Iterator` permite que você transforme um iterador aplicando uma closure a seus itens. O adaptador `filter`

permite filtrar itens de um iterador, utilizando uma closure para decidir quais manter e quais dropar.

Por exemplo, suponha que você esteja iterando sobre linhas de texto e queira omitir os espaços em branco iniciais e finais de cada linha. O método `str::trim` da biblioteca padrão dropa os espaços em branco à esquerda e à direita de uma única `&str`, retornando uma nova `&str` reduzida que empresta da original. Você pode utilizar o adaptador `map` para aplicar `str::trim` a cada linha do iterador:

```
let text = " ponies \n giraffes\niguanas \nsquid".to_string();
let v: Vec<&str> = text.lines()
    .map(str::trim)
    .collect();
assert_eq!(v, ["ponies", "giraffes", "iguanas", "squid"]);
```

A chamada a `text.lines()` retorna um iterador que produz as linhas da string. Chamar `map` nesse iterador retorna um segundo iterador que aplica `str::trim` a cada linha e produz os resultados como seus itens. Por fim, `collect` reúne esses itens em um vetor.

O iterador que `map` retorna é, naturalmente, um candidato para uma adaptação posterior. Se você deseja excluir iguanas do resultado, pode escrever o seguinte:

```
let text = " ponies \n giraffes\niguanas \nsquid".to_string();
let v: Vec<&str> = text.lines()
    .map(str::trim)
    .filter(|s| *s != "iguanas")
    .collect();
assert_eq!(v, ["ponies", "giraffes", "squid"]);
```

Aqui, `filter` retorna um terceiro iterador que produz apenas os itens do iterador `map` para o qual a closure `|s| *s != "iguanas"` retorna `true`. Uma cadeia de adaptadores iteradores é como um pipeline no shell do Unix: cada adaptador tem um único propósito e fica claro como a sequência está sendo transformada à medida que se lê da esquerda para a direita.

As assinaturas desses adaptadores são as seguintes:

```
fn map<B, F>(self, f: F) -> impl Iterator<Item=B>
    where Self: Sized, F: FnMut(Self::Item) -> B;

fn filter<P>(self, predicate: P) -> impl Iterator<Item=Self::Item>
    where Self: Sized, P: FnMut(&Self::Item) -> bool;
```

Na biblioteca padrão, `map` e `filter` na verdade retornam tipos `struct` opacos específicos chamados `std::iter::Map` e `std::iter::Filter`. Contudo, simplesmente ver seus nomes não é muito informativo; portanto, neste livro, vamos apenas escrever `-> impl Iterator<Item=...>` em vez disso, já que isso nos diz o que realmente queremos saber: o método retorna um `Iterator` que produz itens do tipo dado.

Como a maioria dos adaptadores aceita `self` por valor, eles exigem que `Self` seja `Sized` (como são todos os iteradores comuns).

Um iterador `map` passa cada item para sua closure por valor e, por sua vez, repassa a posse do resultado da closure para seu consumidor. Um iterador `filter` passa cada item para sua closure por referência compartilhada, retendo a posse caso o item seja selecionado para ser repassado ao seu consumidor. É por isso que o exemplo deve desreferenciar `s` e compará-lo com "iguanas": o tipo de item do iterador `filter` é `&str`, então o tipo de argumento da closure `s` é `&&str`.

Há dois pontos importantes a serem observados sobre adaptadores iteradores.

Primeiro, simplesmente chamar um adaptador em um iterador não consome nenhum item; apenas retorna um novo iterador, pronto para produzir seus próprios itens derivando do primeiro iterador conforme necessário. Em uma cadeia de adaptadores, a única maneira de fazer qualquer trabalho realmente ser feito é chamar `next` no iterador final.

Portanto, em nosso exemplo anterior, a chamada de método `text.lines()` em si não analisa nenhuma linha da string; apenas retorna um iterador que analisaria as linhas, se solicitado. De forma similar, `map` e `filter` simplesmente retornam novos iteradores que mapeariam ou filtrariam, se solicitados. Nenhum trabalho ocorre até que `collect` comece a chamar `next` no iterador `filter`.

Esse ponto é especialmente importante se você utilizar adaptadores que tenham efeitos colaterais. Por exemplo, este código imprime nada:

```
["earth", "water", "air", "fire"]
```

```
.iter().map(|elt| println!("{}", elt));
```

A chamada a `iter` retorna um iterador sobre os elementos do array e a chamada a `map` retorna um segundo iterador que aplica a closure a cada valor que o primeiro produz. Mas não há nada aqui que realmente exija um valor de toda a cadeia, então nenhum método `next` é executado. Na verdade, o Rust vai avisá-lo sobre isso:

```
warning: unused `std::iter::Map` that must be used
|
7 | /   ["earth", "water", "air", "fire"]
8 | |   .iter().map(|elt| println!("{}", elt));
  | |_____^
  |
= note: iterators are lazy and do nothing unless consumed
```

O termo “lazy” (“preguiçoso”) na mensagem de erro não é um termo depreciativo; é apenas um jargão para qualquer mecanismo que adia uma computação até que seu valor seja necessário. É convenção do Rust que os iteradores devem fazer o mínimo de trabalho necessário para satisfazer cada chamada a `next`; no exemplo, não há tais chamadas, então nenhum trabalho ocorre.

O segundo ponto importante é que os adaptadores iteradores são uma abstração de overhead zero. Como a `map`, `filter` e seus complementares são genéricos, aplicá-los a um iterador especializa seu código para o tipo de iterador específico envolvido. Isso significa que o Rust tem informações suficientes para colocar inline cada método `next` do iterador em seu consumidor e, em seguida, traduz todo o arranjo em código de máquina como uma unidade. Então a cadeia de iteradores `lines/map/filter` que mostramos antes é tão eficiente quanto o código que você provavelmente escreveria à mão:

```
for line in text.lines() {
    let line = line.trim();
    if line != "iguanas" {
        v.push(line);
    }
}
```

O restante desta seção cobre os vários adaptadores disponíveis no `trait Iterator`.

filter_map e flat_map

O adaptador `map` funciona bem em situações em que cada item de entrada produz um item de saída. Mas e se você quiser excluir certos itens da iteração em vez de processá-los ou substituir itens únicos por zero ou mais itens? Os adaptadores `filter_map` e `flat_map` garantem essa flexibilidade.

O adaptador `filter_map` é semelhante a `map` exceto que permite que sua closure transforme o item em um novo item (como `map` faz) ou dropar o item da iteração. Assim, é um pouco como uma combinação de `filter` e `map`. A sua assinatura é a seguinte:

```
fn filter_map<B, F>(self, f: F) -> impl Iterator<Item=B>
  where Self: Sized, F: FnMut(Self::Item) -> Option<B>;
```

Isso é o mesmo que assinatura de `map`, exceto que aqui a closure retorna `Option<B>`, não simplesmente `B`. Quando a closure voltar `None`, o item é dropado da iteração; quando voltar `Some(b)`, então `b` é o próximo item o iterador `filter_map` produz.

Por exemplo, suponha que você queira verificar uma string em busca de palavras separadas por espaços em branco que possam ser analisadas como números e processar os números, dropando as outras palavras. Você pode escrever:

```
use std::str::FromStr;

let text = "1\nfrond .25 289\n3.1415 estuary\n";
for number in text
    .split_whitespace()
    .filter_map(|w| f64::from_str(w).ok())
{
    println!("{:4.2}", number.sqrt());
}
```

Isso imprime o seguinte:

```
1.00
0.50
17.00
1.77
```

A closure dada a `filter_map` tenta analisar cada fatia separada por espaços em branco utilizando `f64::from_str`. Isso retorna um `Result<f64, ParseFloatError>`, que `.ok()` se transforma em um `Option<f64>`: um erro de análise torna-se `None`, enquanto um resultado de análise bem-sucedido torna-se `Some(v)`. O iterador `filter_map` dropa todos os valores

None e produz o valor `v` para cada `Some(v)`.

Mas qual é o ponto em fundir `map` e `filter` em uma única operação como essa, em vez de apenas utilizar esses adaptadores diretamente? O adaptador `filter_map` mostra seu valor em situações como a que acabamos de mostrar, quando a melhor maneira de decidir se deve incluir o item na iteração é realmente tentar processá-lo. Você pode fazer a mesma coisa com apenas `filter` e `map`, mas é algo um pouco desajeitado:

```
text.split_whitespace()
  .map(|w| f64::from_str(w))
  .filter(|r| r.is_ok())
  .map(|r| r.unwrap())
```

Você pode pensar no adaptador `flat_map` como continuando na mesma linha que `map` e `filter_map`, exceto que agora a closure pode retornar não apenas um item (como com `map`) ou zero ou um item (como com `filter_map`), mas uma sequência de qualquer número de itens. O iterador `flat_map` produz a concatenação das sequências que a closure retorna.

A assinatura de `flat_map` é mostrada aqui:

```
fn flat_map<U, F>(self, f: F) -> impl Iterator<Item=U::Item>
  where F: FnMut(Self::Item) -> U, U: IntoIterator;
```

A closure passada para `flat_map` deve retornar um iterável, mas qualquer tipo de iterável serve.^{[1](#)}

Por exemplo, suponha que temos uma tabela mapeando países para suas principais cidades. Dada uma lista de países, como podemos iterar por suas principais cidades?

```
use std::collections::HashMap;

let mut major_cities = HashMap::new();
major_cities.insert("Japan", vec!["Tokyo", "Kyoto"]);
major_cities.insert("The United States", vec!["Portland", "Nashville"]);
major_cities.insert("Brazil", vec!["São Paulo", "Brasília"]);
major_cities.insert("Kenya", vec!["Nairobi", "Mombasa"]);
major_cities.insert("The Netherlands", vec!["Amsterdam", "Utrecht"]);
let countries = ["Japan", "Brazil", "Kenya"];
for &country in countries.iter().flat_map(|country| &major_cities[country]) {
    println!("{}", city);
}
```


Isso imprime o seguinte:

```
Tokyo  
Kyoto  
São Paulo  
Brasília  
Nairobi  
Mombasa
```

Uma maneira de ver isso seria dizer que, para cada país, recuperamos o vetor de suas cidades, concatenamos todos os vetores em uma única sequência e a imprimimos.

Mas lembre-se de que os iteradores são preguiçosos: são apenas as chamadas de loop `for` para o método `next` do iterador `flat_map` que fazem com que o trabalho seja realizado. A sequência concatenada completa nunca é construída na memória. Em vez disso, o que temos aqui é uma pequena máquina de estado que extrai do iterador de cidades, um item de cada vez, até que se esgote e só então produz um novo iterador de cidades para o próximo país. O efeito é o de um loop aninhado, mas empacotado para uso como um iterador.

flatten

O adaptador `flatten` concatena os itens de um iterador, assumindo que cada item é iterável:

```
use std::collections::BTreeMap;  
  
// Uma tabela mapeando cidades para seus parques: cada valor é um vetor  
let mut parks = BTreeMap::new();  
parks.insert("Portland", vec!["Mt. Tabor Park", "Forest Park"]);  
parks.insert("Kyoto", vec!["Tadasu-no-Mori Forest", "Maruyama Koen"]);  
parks.insert("Nashville", vec!["Percy Warner Park", "Dragon Park"]);  
  
// Constrói um vetor de todos os parques. `values` nos dá um iterador produzindo  
// vetores e, em seguida, `flatten` produz os elementos de cada vetor por vez  
let all_parks: Vec<_> = parks.values().flatten().cloned().collect();  
  
assert_eq!(all_parks,  
            vec!["Tadasu-no-Mori Forest", "Maruyama Koen", "Percy Warner Park",  
                 "Dragon Park", "Mt. Tabor Park", "Forest Park"]);
```

O nome “flatten” vem da imagem de achatamento de uma estrutura de dois níveis em uma estrutura de um nível: o `BTreeMap` e seus `Vecs` de

nomes são achatados em um iterador produzindo todos os nomes.

A assinatura de `flatten` é a seguinte:

```
fn flatten(self) -> impl Iterator<Item=Self::Item::Item>  
    where Self::Item: IntoIterator;
```

Em outras palavras, os itens do iterador subjacente devem eles próprios implementar `IntoIterator` de modo que seja efetivamente uma sequência de sequências. O método `flatten` então retorna um iterador sobre a concatenação dessas sequências. Naturalmente, isso é feito preguiçosamente, derivando um novo item de `self` somente quando terminarmos de iterar pelo último.

O método `flatten` é utilizado de algumas maneiras surpreendentes. Se você tem um `Vec<Option<...>>` e deseja iterar apenas pelos valores `Some`, `flatten` funciona lindamente:

```
assert_eq!(vec![None, Some("day"), None, Some("one")]  
    .into_iter()  
    .flatten()  
    .collect::<Vec<_>>(),  
    vec!["day", "one"]);
```

Isso funciona porque a própria `Option` implementa `IntoIterator`, representando uma sequência de elementos zero ou um. Os elementos `None` não contribuem em nada para a iteração, enquanto cada elemento `Some` contribui com um único valor. Da mesma forma, você pode utilizar `flatten` para iterar por valores `Option<Vec<...>>`: `None` se comporta da mesma forma que um vetor vazio.

`Result` também implementa `IntoIterator`, com `Err` representando uma sequência vazia, então aplicar `flatten` a um iterador de valores `Result` efetivamente espreme todos os `Errs` e os descarta, resultando em um fluxo de valores de sucesso desencapsulados. Não recomendamos ignorar erros em seu código, mas esse é um truque interessante que as pessoas utilizam quando acham que sabem o que está acontecendo.

Você pode se encontrar usando `flatten` quando o que você realmente precisa é `flat_map`. Por exemplo, o método `str::to_uppercase` da biblioteca padrão, que converte uma string em letras maiúsculas, funciona mais ou menos assim:

```
fn to_uppercase(&self) -> String {
```

```

self.chars()
  .map(char::to_uppercase)
  .flatten() // há uma maneira melhor
  .collect()
}

```

A razão pela qual `flatten` é necessário é que `ch.to_uppercase()` não retorna um único caractere, mas um iterador produzindo um ou mais caracteres. Mapear cada caractere em seu equivalente em maiúsculas resulta em um iterador de iteradores de caracteres e o `flatten` cuida de juntá-los todos em algo que podemos finalmente coletar com `collect` dentro de uma `String`.

Mas essa combinação de `map` e `flatten` é tão comum que `Iterator` fornece o adaptador `flat_map` apenas para esse caso. (Na verdade, `flat_map` foi adicionado à biblioteca padrão antes de `flatten`.) Portanto, o código anterior poderia ser escrito:

```

fn to_uppercase(&self) -> String {
  self.chars()
    .flat_map(char::to_uppercase)
    .collect()
}

```

take e take_while

Os adaptadores `take` e `take_while` do trait `Iterator` permitem que você termine uma iteração após um certo número de itens ou quando uma closure decide interromper as coisas. Suas assinaturas são as seguintes:

```

fn take(self, n: usize) -> impl Iterator<Item=Self::Item>
  where Self: Sized;

fn take_while<P>(self, predicate: P) -> impl Iterator<Item=Self::Item>
  where Self: Sized, P: FnMut(&Self::Item) -> bool;

```

Ambos tomam posse de um iterador e retornam um novo iterador que repassa os itens do primeiro, possivelmente encerrando a sequência mais cedo. O iterador `take` retorna `None` depois de produzir no máximo `n` itens. O iterador `take_while` aplica `predicate` a cada item e retorna `None` no lugar do primeiro item para o qual `predicate` retorna `false` e em cada chamada subsequente a `next`.

Por exemplo, dada uma mensagem de e-mail com uma linha em

branco separando os cabeçalhos do corpo da mensagem, você pode utilizar `take_while` para iterar apenas pelos cabeçalhos:

```
let message = "To: jimb\r\n\
               From: superego <editor@oreilly.com>\r\n\
               \r\n\
               Did you get any writing done today?\r\n\
               When will you stop wasting time plotting fractals?\r\n";
for header in message.lines().take_while(|| !l.is_empty()) {
    println!("{}", header);
}
```

Lembre-se de “Literais de string”, na página 98, que, quando uma linha em uma string termina com uma barra invertida, o Rust não inclui o recuo da próxima linha na string, então nenhuma das linhas na string tem qualquer espaço em branco à esquerda. Isso significa que a terceira linha de `message` está em branco. O adaptador `take_while` encerra a iteração assim que vê essa linha em branco, então esse código imprime apenas as duas linhas:

```
To: jimb
From: superego <editor@oreilly.com>
```

skip e skip_while

Os métodos `skip` e `skip_while` do trait `Iterator` são o complemento de `take` e `take_while`: eles dropam um certo número de itens desde o início de uma iteração ou dropam itens até que uma closure encontre um aceitável e, em seguida, passam os itens restantes inalterados. Suas assinaturas são as seguintes:

```
fn skip(self, n: usize) -> impl Iterator<Item=Self::Item>
    where Self: Sized;

fn skip_while<P>(self, predicate: P) -> impl Iterator<Item=Self::Item>
    where Self: Sized, P: FnMut(&Self::Item) -> bool;
```

Um uso comum para o adaptador `skip` é ignorar o nome do comando ao iterar pelos argumentos de linha de comando de um programa. No Capítulo 2, nossa calculadora de máximo denominador comum utilizou o código a seguir para fazer um loop por seus argumentos de linha de comando:

```
for arg in std::env::args().skip(1) {
    ...
}
```

A função `std::env::args` retorna um iterador que produz os argumentos do programa como `Strings`, sendo o primeiro item o nome do próprio programa. Essa não é uma string que queremos processar nesse loop. Chamar `skip(1)` nesse iterador retorna um novo iterador que dropa o nome do programa na primeira vez em que é chamado e, em seguida, produz todos os argumentos subsequentes.

O adaptador `skip_while` utiliza uma closure para decidir quantos itens dropar desde o início da sequência. Você pode iterar pelas linhas do corpo da mensagem da seção anterior assim:

```
for body in message.lines()
    .skip_while(|| !.is_empty())
    .skip(1) {
    println!("{}", body);
}
```

Isso utiliza `skip_while` para pular linhas não em branco, mas esse iterador produz a própria linha em branco – afinal, a closure retornou `false` para essa linha. Então utilizamos o método `skip` também para dropar isso, o que nos deixa com um iterador cujo primeiro item será a primeira linha do corpo da mensagem. Com a declaração de `message` da seção anterior, esse código imprime:

```
Did you get any writing done today?
When will you stop wasting time plotting fractals?
```

peekable

Um iterador `peekable` permite que você espie o próximo item que será produzido sem realmente o consumir. Você pode transformar qualquer iterador em um iterador `peekable` chamando o método `peekable` do `trait Iterator`:

```
fn peekable(self) -> std::iter::Peekable<Self>
    where Self: Sized;
```

Aqui, `Peekable<Self>` é um `struct` que implementa `Iterator<Item=Self::Item>` e `Self` é o tipo do iterador subjacente.

Um iterador `Peekable` tem um método adicional `peek` que retorna um `Option<&Item>`: `None` se o iterador subjacente terminar, e `Some(r)` caso contrário, em que `r` é uma referência compartilhada para o próximo item. (Observe que, se o tipo de item do iterador já for uma

referência a algo, isso acaba sendo uma referência a uma referência.)

Chamar `peek` tenta derivar o próximo item do iterador subjacente e, se houver, o armazena em cache até a próxima chamada a `next`. Todos os outros métodos `Iterator` em `Peekable` conhecem esse cache: por exemplo, `iter.last()` em um iterador `peekable` `iter` sabe verificar o cache depois de esgotar o iterador subjacente.

Iteradores `peekable` são essenciais quando você não consegue decidir quantos itens consumir de um iterador até que tenha ido longe demais. Por exemplo, se você estiver analisando números de um fluxo de caracteres, não poderá decidir onde o número termina até ver o primeiro caractere não numérico que o segue:

```
use std::iter::Peekable;

fn parse_number<I>(tokens: &mut Peekable<I>) -> u32
  where I: Iterator<Item=char>
{
  let mut n = 0;
  loop {
    match tokens.peek() {
      Some(r) if r.is_digit(10) => {
        n = n * 10 + r.to_digit(10).unwrap();
      }
      _ => return n
    }
    tokens.next();
  }
}

let mut chars = "226153980,1766319049".chars().peekable();
assert_eq!(parse_number(&mut chars), 226153980);
// Observe, `parse_number` não consumiu a vírgula! Então nós faremos isso
assert_eq!(chars.next(), Some(','));
assert_eq!(parse_number(&mut chars), 1766319049);
assert_eq!(chars.next(), None);
```

A função `parse_number` utiliza `peek` para verificar o próximo caractere e o consome somente se for um dígito. Se não for um dígito ou o iterador estiver esgotado (ou seja, se `peek` retorna `None`), retornamos o número que analisamos e deixamos o próximo caractere no iterador, pronto para ser consumido.

fuse

Uma vez que `Iterator` retornou `None`, o `trait` não especifica como ele deve se comportar se você chamar seu método `next` novamente. A maioria dos iteradores apenas retorna `None` de novo, mas não todos. Se o seu código conta com esse comportamento, você pode se surpreender.

O adaptador `fuse` pega qualquer iterador e produz um que definitivamente continuará retornando `None` uma vez que tenha feito isso pela primeira vez:

```
struct Flaky(bool);

impl Iterator for Flaky {
    type Item = &'static str;
    fn next(&mut self) -> Option<Self::Item> {
        if self.0 {
            self.0 = false;
            Some("totally the last item")
        } else {
            self.0 = true; // D'oh!
            None
        }
    }
}

let mut flaky = Flaky(true);
assert_eq!(flaky.next(), Some("totally the last item"));
assert_eq!(flaky.next(), None);
assert_eq!(flaky.next(), Some("totally the last item"));

let mut not_flaky = Flaky(true).fuse();
assert_eq!(not_flaky.next(), Some("totally the last item"));
assert_eq!(not_flaky.next(), None);
assert_eq!(not_flaky.next(), None);
```

O adaptador `fuse` é provavelmente mais útil em código genérico que precisa trabalhar com iteradores de origem incerta. Em vez de esperar que todos os iteradores com os quais você terá de lidar sejam bem-comportados, você pode utilizar `fuse` para ter a certeza.

Iteradores reversíveis e rev

Alguns iteradores são capazes de produzir itens de ambas as

extremidades da sequência. Você pode reverter esses iteradores utilizando o adaptador `rev`. Por exemplo, um iterador sobre um vetor poderia facilmente produzir itens desde o final do vetor quanto desde o início. Esses iteradores podem implementar o `trait std::iter::DoubleEndedIterator`, que se estende `Iterator`:

```
trait DoubleEndedIterator: Iterator {  
    fn next_back(&mut self) -> Option<Self::Item>;  
}
```

Você pode pensar em um iterador de ponta dupla como tendo dois dedos marcando a frente e o verso atuais da sequência. Pegar itens de cada extremidade avança esse dedo em direção ao outro; quando os dois se encontram, a iteração é feita:

```
let bee_parts = ["head", "thorax", "abdomen"];  
  
let mut iter = bee_parts.iter();  
assert_eq!(iter.next(), Some(&"head"));  
assert_eq!(iter.next_back(), Some(&"abdomen"));  
assert_eq!(iter.next(), Some(&"thorax"));  
  
assert_eq!(iter.next_back(), None);  
assert_eq!(iter.next(), None);
```

A estrutura de um iterador sobre uma fatia facilita a implementação desse comportamento: é literalmente um par de ponteiros para o início e o fim do intervalo de elementos que ainda não produzimos; `next` e `next_back` simplesmente derivam um item de um ou do outro. Iteradores para coleções ordenadas como `BTreeSet` e `BTreeMap` também têm duas extremidades: seus métodos `next_back` derivam os maiores elementos ou entradas primeiro. Em geral, a biblioteca padrão fornece iteração dupla sempre que for prático.

Mas nem todos os iteradores podem fazer isso tão facilmente: um iterador produzindo valores de outros threads chegando a um canal `Receiver` não tem como prever qual pode ser o último valor recebido. Em geral, você precisará verificar a documentação da biblioteca padrão para ver quais iteradores implementam `DoubleEndedIterator` e quais não.

Se um iterador for duplo, você pode reverter-lo com o adaptador `rev`:

```
fn rev(self) -> impl Iterator<Item=Self>  
    where Self: Sized + DoubleEndedIterator;
```


O iterador retornado também é duplo: seus métodos `next` e `next_back` são simplesmente trocados:

```
let meals = ["breakfast", "lunch", "dinner"];

let mut iter = meals.iter().rev();
assert_eq!(iter.next(), Some(&"dinner"));
assert_eq!(iter.next(), Some(&"lunch"));
assert_eq!(iter.next(), Some(&"breakfast"));
assert_eq!(iter.next(), None);
```

A maioria dos adaptadores iteradores, se aplicados a um iterador reversível, retorna outro iterador reversível. Por exemplo, `map` e `filter` preservam a reversibilidade.

inspect

O adaptador `inspect` é útil para depurar sequências (pipelines) de adaptadores de iteradores, mas não é muito utilizado em código de produção. Ele simplesmente aplica uma closure a uma referência compartilhada para cada item e, em seguida, passa o item. A closure não pode afetar os itens, mas pode fazer coisas como imprimi-los ou fazer afirmações sobre eles.

Esse exemplo mostra um caso em que a conversão de uma string para letras maiúsculas altera seu comprimento:

```
let upper_case: String = "große".chars()
    .inspect(|c| println!("before: {:?}", c))
    .flat_map(|c| c.to_uppercase())
    .inspect(|c| println!(" after:  {:?}", c))
    .collect();
assert_eq!(upper_case, "GROSSE");
```

O equivalente maiúsculo da letra alemã minúscula “ß” é “SS” e é por isso que `char::to_uppercase` retorna um iterador sobre caracteres, não um único caractere de substituição. O código anterior utiliza `flat_map` para concatenar todas as sequências que `to_uppercase` retorna em uma única `String`, imprimindo o seguinte conforme faz isso:

```
before: 'g'
after:   'G'
before: 'r'
after:   'R'
before: 'o'
after:   'O'
```

```
before: 'ß'
after:   'S'
after:   'S'
before: 'e'
after:   'E'
```

chain

O adaptador `chain` junta um iterador a outro. Mais precisamente, `i1.chain(i2)` retorna um iterador que consome itens de `i1` até que se esgote e, em seguida, consome itens de `i2`.

A assinatura do adaptador `chain` é a seguinte:

```
fn chain<U>(self, other: U) -> impl Iterator<Item=Self::Item>
    where Self: Sized, U: IntoIterator<Item=Self::Item>;
```

Em outras palavras, você pode encadear um iterador com qualquer iterável que produza o mesmo tipo de item.

Por exemplo:

```
let v: Vec<i32> = (1..4).chain([20, 30, 40]).collect();
assert_eq!(v, [1, 2, 3, 20, 30, 40]);
```

Um iterador `chain` é reversível, se ambos os iteradores subjacentes forem:

```
let v: Vec<i32> = (1..4).chain([20, 30, 40]).rev().collect();
assert_eq!(v, [40, 30, 20, 3, 2, 1]);
```

Um iterador `chain` acompanha quando cada um dos dois iteradores subjacentes retorna `None` e dirige as chamadas a `next` e `next_back` para um ou outro, conforme apropriado.

enumerate

O adaptador `enumerate` do trait `Iterator` anexa um índice corrente à sequência, utilizando um iterador que produz itens `A`, `B`, `C`, ... e retornando um iterador que produz pares `(0, A)`, `(1, B)`, `(2, C)`, Parece trivial à primeira vista, mas é utilizado com uma frequência surpreendente.

Os consumidores podem utilizar esse índice para distinguir um item do outro e estabelecer o contexto no qual processar cada um. Por exemplo, o plotter de conjunto de Mandelbrot no Capítulo 2 divide a imagem em oito faixas horizontais e atribui cada uma a um thread

diferente. Esse código utiliza `enumerate` para dizer a cada thread a qual parte da imagem sua faixa corresponde.

Ele começa com um buffer retangular de pixels:

```
let mut pixels = vec![0; columns * rows];
```

A seguir, utiliza `chunks_mut` para dividir a imagem em faixas horizontais, uma por thread:

```
let threads = 8;
```

```
let band_rows = rows / threads + 1;
```

```
...
```

```
let bands: Vec<&mut [u8]> = pixels.chunks_mut(band_rows * columns).collect();
```

E então itera sobre as faixas, iniciando um thread para cada uma:

```
for (i, band) in bands.into_iter().enumerate() {
```

```
    let top = band_rows * i;
```

```
    // inicia um thread para renderizar linhas `top..top + band_rows`
```

```
    ...
```

```
}
```

Cada iteração obtém um par `(i, band)`, em que `band` é a fatia `&mut [u8]` do buffer de pixels em que o thread deve desenhar e `i` é o índice dessa faixa na imagem geral, cortesia do adaptador `enumerate`. Dados os limites do gráfico e o tamanho das faixas, isso é informação suficiente para o thread determinar qual parte da imagem lhe foi atribuído e, portanto, o que desenhar em `band`.

Você pode pensar nos pares `(index, item)` que `enumerate` produz como análogos aos pares `(key, value)` que você obtém ao iterar por um `HashMap` ou outra coleção associativa. Se você estiver iterando sobre uma fatia ou vetor, o `index` é a “chave” sob a qual o `item` aparece.

zip

O adaptador `zip` combina dois iteradores em um único iterador que produz pares contendo um valor de cada iterador, como um zíper unindo seus dois lados em uma única costura. O iterador compactado termina quando um dos dois iteradores subjacentes termina.

Por exemplo, você pode obter o mesmo efeito que o adaptador `enumerate` compactando o intervalo ilimitado `0..` com o outro iterador:

```
let v: Vec<_> = (0..).zip("ABCD".chars()).collect();
```

```
assert_eq!(v, vec![(0, 'A'), (1, 'B'), (2, 'C'), (3, 'D')]);
```

Nesse sentido, você pode pensar em `zip` como uma generalização de `enumerate`: enquanto `enumerate` anexa índices à sequência, `zip` anexa os itens de qualquer iterador arbitrário. Sugerimos antes que `enumerate` pode ajudar a fornecer contexto para itens de processamento; `zip` é uma maneira mais flexível de fazer o mesmo.

O argumento para `zip` não precisa ser um iterador em si; pode ser qualquer iterável:

```
use std::iter::repeat;
```

```
let endings = ["once", "twice", "chicken soup with rice"];
let rhyme: Vec<_> = repeat("going")
    .zip(endings)
    .collect();
assert_eq!(rhyme, vec!["going", "once"),
            ("going", "twice"),
            ("going", "chicken soup with rice")]);
```

by_ref

Ao longo desta seção, adicionamos adaptadores a iteradores. Depois de fazer isso, você pode remover o adaptador novamente? Normalmente, não: os adaptadores tomam posse do iterador subjacente e não fornecem nenhum método para devolvê-lo.

O método `by_ref` de um iterador pega emprestada uma referência mutável ao iterador para que você possa aplicar adaptadores à referência. Quando terminar de consumir itens desses adaptadores, você os dropa, o empréstimo termina e você recupera o acesso ao seu iterador original.

Por exemplo, no início do capítulo, mostramos como utilizar `take_while` e `skip_while` para processar as linhas de cabeçalho e o corpo de uma mensagem de e-mail. Mas e se você quiser fazer as duas coisas, utilizando o mesmo iterador subjacente? Utilizando `by_ref`, podemos utilizar `take_while` para lidar com os cabeçalhos e, quando terminar, recuperar o iterador subjacente, que `take_while` deixou exatamente em posição de lidar com o corpo da mensagem:

```
let message = "To: jimb\r\n\
               From: id\r\n\
```

```

        \r\n\
        Ooooooh, donuts!!\r\n";

let mut lines = message.lines();

println!("Headers:");
for header in lines.by_ref().take_while(|| !l.is_empty()) {
    println!("{}", header);
}

println!("\nBody:");
for body in lines {
    println!("{}", body);
}

```

A chamada a `lines.by_ref()` toma emprestada uma referência mutável ao iterador e é dessa referência que o iterador `take_while` toma posse. Esse iterador sai do escopo no final do primeiro loop `for`, o que significa que o empréstimo terminou, então você pode utilizar `lines` novamente no segundo loop `for`. Isso imprime o seguinte:

```

Headers:
To: jimb
From: id

Body:
Ooooooh, donuts!!

```

A definição do adaptador `by_ref` é trivial: ele retorna uma referência mutável ao iterador. Então, a biblioteca padrão inclui essa pequena implementação estranha:

```

impl<'a, I: Iterator + ?Sized> Iterator for &'a mut I {
    type Item = I::Item;
    fn next(&mut self) -> Option<I::Item> {
        (**self).next()
    }
    fn size_hint(&self) -> (usize, Option<usize>) {
        (**self).size_hint()
    }
}

```

Em outras palavras, se `I` é algum tipo de iterador, então `&mut I` também é um iterador, cujos métodos `next` e `size_hint` concedem ao seu referente. Quando você chama um adaptador em uma referência mutável para um iterador, o adaptador assume a posse da referência, não o próprio iterador. Isso é apenas um empréstimo que termina quando o adaptador sai do escopo.

cloned, copied

O adaptador `cloned` pega um iterador que produz referências e retorna um iterador que produz valores clonados dessas referências, muito parecido com `iter.map(|item| item.clone())`. Naturalmente, o tipo de referência deve implementar `Clone`. Por exemplo:

```
let a = ['1', '2', '3', '∞'];

assert_eq!(a.iter().next(),      Some(&'1'));
assert_eq!(a.iter().cloned().next(), Some('1'));
```

O adaptador `copied` é a mesma ideia, mas mais restritiva: o tipo de referência deve implementar `Copy`. Uma chamada como `iter.copied()` é aproximadamente o mesmo que `iter.map(|r| *r)`. Como todo tipo que implementa `Copy` também implementa `Clone`, `cloned` é estritamente mais geral, mas, dependendo do tipo de item, uma chamada `clone` pode fazer quantidades arbitrárias de alocação e cópia. Se você está assumindo que isso nunca aconteceria porque seu tipo de item é algo simples, é melhor utilizar `copied` para fazer o verificador de tipos verificar suas suposições.

cycle

O adaptador `cycle` retorna um iterador que repete infinitamente a sequência produzida pelo iterador subjacente. O iterador subjacente deve implementar `std::clone::Clone` de modo que `cycle` possa salvar seu estado inicial e reutilizá-lo sempre que o ciclo recomeçar.

Por exemplo:

```
let dirs = ["North", "East", "South", "West"];
let mut spin = dirs.iter().cycle();
assert_eq!(spin.next(), Some(&"North"));
assert_eq!(spin.next(), Some(&"East"));
assert_eq!(spin.next(), Some(&"South"));
assert_eq!(spin.next(), Some(&"West"));
assert_eq!(spin.next(), Some(&"North"));
assert_eq!(spin.next(), Some(&"East"));
```

Ou, para um uso realmente gratuito de iteradores:

```
use std::iter::{once, repeat};

let fizzes = repeat("").take(2).chain(once("fizz")).cycle();
let buzzes = repeat("").take(4).chain(once("buzz")).cycle();
```

```

let fizzes_buzzes = fizzes.zip(buzzes);

let fizz_buzz = (1..100).zip(fizzes_buzzes)
  .map(|tuple|
    match tuple {
      (i, ("", "")) => i.to_string(),
      (_, (fizz, buzz)) => format!("{}", fizz, buzz)
    });

for line in fizz_buzz {
  println!("{}", line);
}

```

Isso representa um jogo de palavras para crianças, agora às vezes utilizado como uma pergunta de entrevista de emprego para codificadores, no qual os jogadores se revezam na contagem, substituindo qualquer número divisível por três pela palavra `fizz` e qualquer número divisível por cinco por `buzz`. Números divisíveis por ambos se tornam `fizzbuzz`.

Consumindo iteradores

Até agora, cobrimos a criação de iteradores e sua adaptação em novos iteradores; aqui finalizamos o processo mostrando formas de consumi-los.

Naturalmente, você pode consumir um iterador com um loop `for` ou com uma chamada `next` explicitamente, mas há muitas tarefas comuns que você não deveria ter de escrever tudo várias vezes. O `trait Iterator` fornece uma ampla seleção de métodos para cobrir muitas delas.

Acumulação simples: contagem, soma, produto

O método `count` consome itens de um iterador até retornar `None` e diz quantos itens conseguiu consumir. Eis um pequeno programa que conta o número de linhas em sua entrada padrão:

```

use std::io::prelude::*;

fn main() {
  let stdin = std::io::stdin();

```

```
println!("{}", stdin.lock().lines().count());
}
```

Os métodos `sum` e `product` calculam a soma ou produto dos itens do iterador, que devem ser inteiros ou números de ponto flutuante:

```
fn triangle(n: u64) -> u64 {
    (1..=n).sum()
}
assert_eq!(triangle(20), 210);

fn factorial(n: u64) -> u64 {
    (1..=n).product()
}
assert_eq!(factorial(20), 2432902008176640000);
```

(Você pode estender `sum` e `product` para trabalhar com outros tipos, implementando os traits `std::iter::Sum` e `std::iter::Product`, que não descreveremos neste livro.)

max, min

Os métodos `min` e `max` em `Iterator` retornam o menor ou o maior item que o iterador produz. O tipo de item do iterador deve implementar `std::cmp::Ord` para que os itens possam ser comparados uns com os outros. Por exemplo:

```
assert_eq!([-2, 0, 1, 0, -2, -5].iter().max(), Some(&1));
assert_eq!([-2, 0, 1, 0, -2, -5].iter().min(), Some(&-5));
```

Esses métodos retornam uma `Option<Self::Item>` para que possam retornar `None` se o iterador não produzir itens.

Conforme explicado em “Comparações de equivalência”, na página 341, tipos de ponto flutuante `f32` e `f64` do Rust implementam apenas `std::cmp::PartialOrd`, e não implementam `std::cmp::Ord`, então você não pode utilizar os métodos `min` e `max` para calcular o menor ou o maior de uma sequência de números de ponto flutuante. Esse não é um aspecto popular do design do Rust, mas é deliberado: não está claro o que tais funções devem fazer com os valores IEEE NaN. Simplesmente os ignorar correria o risco de mascarar problemas mais sérios no código.

Se você sabe como gostaria de lidar com valores NaN, pode utilizar o `max_by` e `min_by` em vez disso, métodos iteradores, que permitem

fornecer sua própria função de comparação.

max_by, min_by

Os métodos `max_by` e `min_by` retornam o item máximo ou mínimo que o iterador produz, conforme determinado por uma função de comparação que você fornece:

```
use std::cmp::Ordering;

// Compara dois valores f64. Gera um pânico se receber um NaN
fn cmp(lhs: &f64, rhs: &f64) -> Ordering {
    lhs.partial_cmp(rhs).unwrap()
}

let numbers = [1.0, 4.0, 2.0];
assert_eq!(numbers.iter().copied().max_by(cmp), Some(4.0));
assert_eq!(numbers.iter().copied().min_by(cmp), Some(1.0));

let numbers = [1.0, 4.0, std::f64::NAN, 2.0];
assert_eq!(numbers.iter().copied().max_by(cmp), Some(4.0)); // pânico
```

Os métodos `max_by` e `min_by` passam itens para a função de comparação por referência para que possam funcionar de forma eficiente com qualquer tipo de iterador, de modo que `cmp` espera tomar seus argumentos por referência, mesmo que tenhamos utilizado `copied` para obter um iterador que produza itens `f64`.

max_by_key, min_by_key

Os métodos `max_by_key` e `min_by_key` em `Iterator` permitem selecionar o item máximo ou mínimo conforme determinado por uma closure aplicado a cada item. A closure pode selecionar algum campo do item ou realizar uma computação nos itens. Como você geralmente está interessado em dados associados a algum mínimo ou máximo, não apenas ao extremo em si, essas funções costumam ser mais úteis do que `min` e `max`. Suas assinaturas são as seguintes:

```
fn min_by_key<B: Ord, F>(self, f: F) -> Option<Self::Item>
    where Self: Sized, F: FnMut(&Self::Item) -> B;

fn max_by_key<B: Ord, F>(self, f: F) -> Option<Self::Item>
    where Self: Sized, F: FnMut(&Self::Item) -> B;
```

Ou seja, dada uma closure que pega um item e retorna qualquer tipo `B` ordenado, retorne o item para o qual a closure retornou o

valor máximo ou mínimo B, ou None se nenhum item foi gerado.

Por exemplo, se você precisar varrer uma tabela hash de cidades para encontrar as cidades com as maiores e menores populações, pode escrever:

```
use std::collections::HashMap;

let mut populations = HashMap::new();
populations.insert("Portland", 583_776);
populations.insert("Fossil", 449);
populations.insert("Greenhorn", 2);
populations.insert("Boring", 7_762);
populations.insert("The Dalles", 15_340);

assert_eq!(populations.iter().max_by_key(|&(_name, pop)| pop),
           Some((&"Portland", &583_776)));
assert_eq!(populations.iter().min_by_key(|&(_name, pop)| pop),
           Some((&"Greenhorn", &2)));
```

A closure `|&(_name, pop)| pop` é aplicada a cada item que o iterador produz e retorna o valor a ser utilizado para comparação – nesse caso, a população da cidade. O valor retornado é o item inteiro, não apenas o valor que a closure retorna. (Naturalmente, se você estiver fazendo consultas como essa com frequência, provavelmente desejará encontrar uma maneira mais eficiente de localizar as entradas do que fazer uma pesquisa linear na tabela.)

Comparando sequências de itens

Você pode utilizar os operadores `<` e `==` para comparar strings, vetores e fatias, supondo que seus elementos individuais possam ser comparados. Embora os iteradores não suportem os operadores de comparação do Rust, eles fornecem métodos como `eq` e `lt` que fazem o mesmo trabalho, produzindo pares de itens dos iteradores e comparando-os até que uma decisão seja alcançada. Por exemplo:

```
let packed = "Helen of Troy";
let spaced = "Helen  of  Troy";
let obscure = "Helen of Sandusky"; // boa pessoa, só não é famosa

assert!(packed != spaced);
assert!(packed.split_whitespace().eq(spaced.split_whitespace()));
// Isso é verdadeiro porque ' ' < 'o'
assert!(spaced < obscure);
```

```
// Isso é verdadeiro porque 'Troy' > 'Sandusky'
assert!(spaced.split_whitespace().gt(obscurer.split_whitespace()));
```

As chamadas a `split_whitespace` retornam iteradores sobre as palavras separadas por espaços em branco da string. Utilizar os métodos `eq` e `gt` nesses iteradores executa uma comparação palavra por palavra, em vez de uma comparação caractere por caractere. Tudo isso é possível porque `&str` implementa `PartialOrd` e `PartialEq`.

Os iteradores fornecem os métodos `eq` e `ne` para comparações de igualdade e os métodos `lt`, `le`, `gt` e `ge` para comparações ordenadas. Os métodos `cmp` e `partial_cmp` se comportam como os métodos correspondentes dos traits `Ord` e `PartialOrd`.

any e all

Os métodos `any` e `all` aplicam uma closure a cada item que o iterador produz e retornam `true` se a closure voltar `true` para qualquer item (`any`), ou para todos os itens (`all`):

```
let id = "Iterator";

assert!(id.chars().any(char::is_uppercase));
assert!(!id.chars().all(char::is_uppercase));
```

Esses métodos consomem apenas os itens necessários para determinar a resposta. Por exemplo, se a closure retornar `true` para um determinado item, então `any` retorna `true` imediatamente, sem extrair mais itens do iterador.

position, rposition e ExactSizeIterator

O método `position` aplica uma closure a cada item do iterador e retorna o índice do primeiro item para o qual a closure retorna `true`. Mais precisamente, ele retorna uma `Option` do índice: se a closure retornar `true` para nenhum item, `position` retorna `None`. Ele para de pegar itens assim que a closure retorna `true`. Por exemplo:

```
let text = "Xerxes";
assert_eq!(text.chars().position(|c| c == 'e'), Some(1));
assert_eq!(text.chars().position(|c| c == 'z'), None);
```

O método `rposition` é o mesmo, exceto que ele pesquisa a partir da direita. Por exemplo:

```
let bytes = b"Xerxes";
assert_eq!(bytes.iter().rposition(|&c| c == b'e'), Some(4));
assert_eq!(bytes.iter().rposition(|&c| c == b'X'), Some(0));
```

O método `rposition` requer um iterador reversível para que possa consumir itens da extremidade direita da sequência. Ele também requer um iterador de tamanho exato para que possa atribuir índices da mesma maneira que `position` faria, começando com 0 à esquerda. Um iterador de tamanho exato é aquele que implementa o `trait std::iter::ExactSizeIterator`:

```
trait ExactSizeIterator: Iterator {
    fn len(&self) -> usize { ... }
    fn is_empty(&self) -> bool { ... }
}
```

O método `len` retorna o número de itens restantes e o método `is_empty` retorna `true` se a iteração estiver completa.

Naturalmente, nem todo iterador sabe quantos itens produzirá com antecedência. Por exemplo, o iterador `str::chars` utilizado anteriormente não sabe (já que UTF-8 é uma codificação de tamanho variável), então você não pode utilizar `rposition` em strings. Mas um iterador em um array de bytes certamente conhece o comprimento do array, então ele pode implementar `ExactSizeIterator`.

fold e rfold

O método `fold` é uma ferramenta muito geral para acumular algum tipo de resultado em toda a sequência de itens que um iterador produz. Dado um valor inicial, que chamaremos de acumulador, e uma closure, `fold` aplica repetidamente a closure ao acumulador atual e ao próximo item do iterador. O valor que a closure retorna é tomado como o novo acumulador, a ser passado para a closure com o próximo item. O valor final do acumulador é o que `fold` ela mesma retorna. Se a sequência estiver vazia, `fold` simplesmente retorna o acumulador inicial.

Muitos dos outros métodos para consumir os valores de um iterador podem ser escritos como usos de `fold`:

```
let a = [5, 6, 7, 8, 9, 10];
```

```

assert_eq!(a.iter().fold(0, |n, _| n+1), 6);    // contagem
assert_eq!(a.iter().fold(0, |n, i| n+i), 45);   // soma
assert_eq!(a.iter().fold(1, |n, i| n*i), 151200); // produto

// max
assert_eq!(a.iter().cloned().fold(i32::min_value(), std::cmp::max),
          10);

```

A assinatura do método `fold` é a seguinte:

```

fn fold<A, F>(self, init: A, f: F) -> A
  where Self: Sized, F: FnMut(A, Self::Item) -> A;

```

Aqui, `A` é o tipo do acumulador. O argumento `init` é um `A`, assim como o primeiro argumento e o valor de retorno da closure, e o valor de retorno do próprio `fold`.

Observe que os valores do acumulador são movidos para dentro e para fora da closure, então você pode utilizar `fold` com tipos não-Copy de acumuladores:

```

let a = ["Pack", "my", "box", "with",
         "five", "dozen", "liquor", "jugs"];

// Veja também: o método `join` em fatias, que não
// dão a você esse espaço extra no final.
let pangram = a.iter()
  .fold(String::new(), |s, w| s + w + " ");
assert_eq!(pangram, "Pack my box with five dozen liquor jugs ");

```

O método `rfold` é o mesmo que `fold`, exceto que requer um iterador de dupla extremidade e processa seus itens do último ao primeiro:

```

let weird_pangram = a.iter()
  .rfold(String::new(), |s, w| s + w + " ");
assert_eq!(weird_pangram, "jugs liquor dozen five with box my Pack ");

```

try_fold e try_rfold

O método `try_fold` é o mesmo que `fold`, exceto que a iteração pode sair antecipadamente, sem consumir todos os valores do iterador. O valor retornado pela closure que você passa para `try_fold` indica se deve retornar imediatamente ou continuar consumindo os itens do iterador.

Sua closure pode retornar qualquer um dos vários tipos, indicando como o `fold` deve proceder:

- Se sua closure retornar `Result<T, E>`, talvez porque faça `E/S` ou

realize alguma outra operação falível, então retornar `Ok(v)` instrui `try_fold` a continuar, com `v` como o novo valor do acumulador. Retornar `Err(e)` faz com que o fold pare imediatamente. O valor final do fold é um `Result` carregando o valor final do acumulador, ou o erro retornado pela closure.

- Se sua closure retornar `Option<T>`, então `Some(v)` indica que o fold deve continuar com `v` como o novo valor do acumulador e `None` indica que a iteração deve parar imediatamente. O valor final do fold também é uma `Option`.
- Por fim, a closure pode retornar um valor `std::ops::ControlFlow`. Esse tipo é uma enumeração com duas variantes, `Continue(c)` e `Break(b)`, o que significa continuar com o novo valor do acumulador `c`, ou parar mais cedo. O resultado do fold é um valor `ControlFlow`: `Continue(v)` se o fold consumiu todo o iterador, gerando o valor final do acumulador `v`; ou `Break(b)`, se a closure retornou esse valor.

`Continue(c)` e `Break(b)` comportam-se exatamente como `Ok(c)` e `Err(b)`. A vantagem de utilizar `ControlFlow` em vez de `Result` é que isso torna seu código um pouco mais legível quando uma saída antecipada não indica um erro, mas apenas que a resposta está pronta antecipadamente. Mostramos um exemplo disso a seguir.

Eis um programa que soma números lidos de sua entrada padrão:

```
use std::error::Error;
use std::io::prelude::*;
use std::str::FromStr;

fn main() -> Result<(), Box<dyn Error>> {
    let stdin = std::io::stdin();
    let sum = stdin.lock()
        .lines()
        .try_fold(0, |sum, line| -> Result<u64, Box<dyn Error>> {
            Ok(sum + u64::from_str(&line?.trim())?)
        })?;
    println!("{}", sum);
    Ok(())
}
```

O iterador `lines` em fluxos de entrada bufferizada produz itens do tipo `Result<String, std::io::Error>` e a análise de `String` como um inteiro pode falhar também. Utilizar `try_fold` aqui deixa a closure retornar

`Result<u64, ...>`, então podemos utilizar o operador `?` para propagar falhas desde a closure até a função `main`.

Devido a `try_fold` ser tão flexível, ele é utilizado para implementar muitos dos outros métodos de consumo de `Iterator`. Por exemplo, eis uma implementação de `all`:

```
fn all<P>(&mut self, mut predicate: P) -> bool
  where P: FnMut(Self::Item) -> bool,
        Self: Sized
{
  use std::ops::ControlFlow::*;
  self.try_fold((), |_, item| {
    if predicate(item) { Continue(()) } else { Break(()) }
  }) == Continue(())
}
```

Observe que isso não pode ser escrito com `fold`: `all` que promete parar de consumir itens do iterador subjacente assim que o `predicate` retornar falso, mas `fold` sempre consome todo o iterador.

Se você estiver implementando seu próprio tipo de iterador, vale a pena investigar se seu iterador pode implementar `try_fold` mais eficientemente do que a definição padrão do trait `Iterator`. Se você pode acelerar `try_fold`, todos os outros métodos criados nele também serão beneficiados.

O método `try_rfold`, como o próprio nome sugere, é o mesmo que `try_fold`, exceto que ele consome valores na parte de trás, em vez da frente, e requer um iterador de duas extremidades.

`nth, nth_back`

O método `nth` pega um índice `n`, ignora muitos itens do iterador e retorna o próximo item ou `None` se a sequência terminar antes desse ponto. Chamar `.nth(0)` é equivalente a `.next()`.

Ele não toma posse do iterador da mesma forma que um adaptador faria, então você pode chamá-lo várias vezes:

```
let mut squares = (0..10).map(|i| i*i);

assert_eq!(squares.nth(4), Some(16));
assert_eq!(squares.nth(0), Some(25));
assert_eq!(squares.nth(6), None);
```

Sua assinatura é mostrada aqui:

```
fn nth(&mut self, n: usize) -> Option<Self::Item>
    where Self: Sized;
```

O método `nth_back` é praticamente o mesmo, exceto pelo fato de que extrai da parte de trás de um iterador de duas extremidades. Chamar `.nth_back(0)` é equivalente a `.next_back()`: retorna o último item, ou `None` se o iterador estiver vazio.

last

O método `last` retorna o último item que o iterador produz, ou `None` se estiver vazio. A sua assinatura é a seguinte:

```
fn last(self) -> Option<Self::Item>;
```

Por exemplo:

```
let squares = (0..10).map(|i| i*i);
assert_eq!(squares.last(), Some(81));
```

Isso consome todos os itens do iterador começando pela frente, mesmo que o iterador seja reversível. Se você tiver um iterador reversível e não precisar consumir todos os seus itens, basta escrever `iter.next_back()`.

find, rfind e find_map

O método `find` deriva itens de um iterador, retornando o primeiro item para o qual a closure fornecida retorna `true`, ou `None` se a sequência terminar antes que um item adequado seja encontrado. Sua assinatura é:

```
fn find<P>(&mut self, predicate: P) -> Option<Self::Item>
    where Self: Sized,
          P: FnMut(&Self::Item) -> bool;
```

O método `rfind` é semelhante, mas requer um iterador de duas extremidades e procura valores de trás para a frente, retornando o último item para o qual a closure retorna `true`.

Por exemplo, utilizando a tabela de cidades e populações de “`max_by_key`, `min_by_key`”, na página 432, você poderia escrever:

```
assert_eq!(populations.iter().find(|&(_name, &pop)| pop > 1_000_000),
           None);
```



```
assert_eq!(populations.iter().find(|&(_name, &pop)| pop > 500_000),
          Some(("Portland", &583_776)));
```

Nenhuma das cidades da tabela tem população acima de um milhão, mas há uma cidade com meio milhão de pessoas.

Às vezes, sua closure não é apenas um simples predicado lançando um julgamento booleano sobre cada item e seguindo em frente: pode ser algo mais complexo que produz um valor interessante por si só. Nesse caso, `find_map` é exatamente o que você quer. Sua assinatura é:

```
fn find_map<B, F>(&mut self, f: F) -> Option<B> where
    F: FnMut(Self::Item) -> Option<B>;
```

Isso é como `find`, exceto que, em vez de retornar `bool`, a closure deve retornar uma `Option` de algum valor. `find_map` retorna a primeira `Option` que é `Some`.

Por exemplo, se tivermos um banco de dados dos parques de cada cidade, podemos querer ver se algum deles é vulcânico e fornecer o nome do parque em caso afirmativo:

```
let big_city_with_volcano_park = populations.iter()
    .find_map(|(&city, _)| {
        if let Some(park) = find_volcano_park(city, &parks) {
            // find_map retorna esse valor, então nosso chamador sabe
            // *qual* parque encontramos.
            return Some((city, park.name));
        }

        // Rejeita este item e continua a pesquisa
        None
    });

assert_eq!(big_city_with_volcano_park,
          Some(("Portland", "Mt. Tabor Park")));
```

Construindo coleções: `collect` e `FromIterator`

Ao longo do livro, utilizamos o `collect` para construir vetores contendo os itens de um iterador. Por exemplo, no Capítulo 2, chamamos `std::env::args()` para obter um iterador sobre os argumentos de linha de comando do programa e, em seguida, chamar o método `collect` do

iterador para reuni-los em um vetor:

```
let args: Vec<String> = std::env::args().collect();
```

Mas `collect` não é específico para vetores: na verdade, ele pode construir qualquer tipo de coleção da biblioteca padrão do Rust, desde que o iterador produza um tipo de item adequado:

```
use std::collections::{HashSet, BTreeSet, LinkedList, HashMap, BTreeMap};
```

```
let args: HashSet<String> = std::env::args().collect();
let args: BTreeSet<String> = std::env::args().collect();
let args: LinkedList<String> = std::env::args().collect();
```

```
// A coleta de um mapa requer pares (chave, valor); portanto, para este exemplo,
// zipa a sequência de strings com uma sequência de inteiros.
let args: HashMap<String, usize> = std::env::args().zip(0..).collect();
let args: BTreeMap<String, usize> = std::env::args().zip(0..).collect();
```

```
// e assim por diante
```

Naturalmente, `collect` em si não sabe como construir todos esses tipos. Em vez disso, quando algum tipo de coleção como `Vec` ou `HashMap` sabe como se construir a partir de um iterador, ele implementa o `trait std::iter::FromIterator`, para a qual `collect` é apenas um verniz conveniente:

```
trait FromIterator<A>: Sized {
    fn from_iter<T: IntoIterator<Item=A>>(iter: T) -> Self;
}
```

Se um tipo de coleção implementa `FromIterator<A>`, então sua função associada ao tipo `from_iter` constrói um valor desse tipo a partir de um iterável produzindo itens do tipo `A`.

No caso mais simples, a implementação poderia simplesmente construir uma coleção vazia e adicionar os itens do iterador um por um. Por exemplo, implementação de `FromIterator` por `std::collections::LinkedList` funciona dessa forma.

Contudo, alguns tipos podem fazer melhor do que isso. Por exemplo, construir um vetor de algum iterador `iter` pode ser tão simples quanto:

```
let mut vec = Vec::new();
for item in iter {
    vec.push(item)
}
vec
```

Mas isso não é o ideal: à medida que o vetor cresce, pode ser necessário expandir seu buffer, exigindo uma chamada para o alocador de heap e uma cópia dos elementos existentes. Os vetores tomam medidas algorítmicas para manter esse overhead baixo, mas, se houvesse alguma maneira de simplesmente alocar um buffer do tamanho certo para começar, não haveria necessidade de redimensionar.

É aqui que entra o método `size_hint` do `trait Iterator`:

```
trait Iterator {  
    ...  
    fn size_hint(&self) -> (usize, Option<usize>) {  
        (0, None)  
    }  
}
```

Esse método retorna um limite inferior e um limite superior opcional para o número de itens que o iterador produzirá. A definição padrão retorna zero como o limite inferior e se recusa a nomear um limite superior, dizendo, na verdade, “Não faço ideia”, mas muitos iteradores podem fazer melhor do que isso. Um iterador sobre um `Range`, por exemplo, sabe exatamente quantos itens produzirá, assim como um iterador em um `Vec` ou `HashMap`. Esses iteradores fornecem suas próprias definições especializadas para `size_hint`.

Esses limites são exatamente as informações de que a implementação de `FromIterator` por `Vec` precisa para dimensionar o buffer do novo vetor corretamente desde o início. As inserções ainda verificam se o buffer é grande o suficiente; portanto, mesmo que a dica esteja incorreta, apenas o desempenho é afetado, não a segurança. Outros tipos podem seguir etapas semelhantes: por exemplo, `HashSet` e `HashMap` também utilizam `Iterator::size_hint` para escolher um tamanho inicial apropriado para sua tabela hash.

Uma observação sobre inferência de tipo: no início desta seção, é um pouco estranho ver a mesma chamada, `std::env::args().collect()`, produzir quatro tipos diferentes de coleções, dependendo de seu contexto. O tipo de retorno de `collect` é seu parâmetro de tipo, então as duas primeiras chamadas são equivalentes ao seguinte:

```
let args = std::env::args().collect::<Vec<String>>();
```

```
let args = std::env::args().collect::<HashSet<String>>();
```

Mas, enquanto houver apenas um tipo que poderia funcionar como `collect`, a inferência de tipo do Rust vai fornecê-lo para você. Quando você escrever explicitamente o tipo de `args`, garante que esse é o caso.

Trait Extend

Se um tipo implementa o trait `std::iter::Extend`, então seu método `extend` adiciona os itens de um iterável à coleção:

```
let mut v: Vec<i32> = (0..5).map(|i| 1 << i).collect();
v.extend([31, 57, 99, 163]);
assert_eq!(v, [1, 2, 4, 8, 16, 31, 57, 99, 163]);
```

Todas as coleções padrão implementam `Extend`, então todos eles têm esse método; assim como `String`. Arrays e fatias, que têm um comprimento fixo, não.

A definição do trait é a seguinte:

```
trait Extend<A> {
    fn extend<T>(&mut self, iter: T)
        where T: IntoIterator<Item=A>;
}
```

Obviamente, isso é muito parecido com `std::iter::FromIterator`: que cria uma nova coleção, enquanto `Extend` estende uma coleção existente. De fato, várias implementações de `FromIterator` na biblioteca padrão simplesmente criam uma nova coleção vazia e chamam `extend` para preenchê-la. Por exemplo, a implementação de `FromIterator` para `std::collections::LinkedList` funciona assim:

```
impl<T> FromIterator<T> for LinkedList<T> {
    fn from_iter<I: IntoIterator<Item = T>>(iter: I) -> Self {
        let mut list = Self::new();
        list.extend(iter);
        list
    }
}
```

partition

O método `partition` divide os itens de um iterador entre duas coleções, utilizando uma closure para decidir onde cada item deve entrar:

```
let things = ["doorknob", "mushroom", "noodle", "giraffe", "grapefruit"];

// Fato surpreendente: o nome de um ser vivo sempre começa
// com uma letra ímpar
let (living, nonliving): (Vec<&str>, Vec<&str>)
    = things.iter().partition(|name| name.as_bytes()[0] & 1 != 0);

assert_eq!(living, vec!["mushroom", "giraffe", "grapefruit"]);
assert_eq!(nonliving, vec!["doorknob", "noodle"]);
```

Assim como `collect`, `partition` pode fazer qualquer tipo de coleção que você quiser, embora ambas devam ser do mesmo tipo. E, assim como `collect`, você precisará especificar o tipo de retorno: o exemplo anterior escreve explicitamente o tipo de `living` e `nonliving` e permite que a inferência de tipo escolha os parâmetros de tipo corretos para a chamada `partition`.

A assinatura de `partition` é a seguinte:

```
fn partition<B, F>(self, f: F) -> (B, B)
    where Self: Sized,
           B: Default + Extend<Self::Item>,
           F: FnMut(&Self::Item) -> bool;
```

Enquanto `collect` requer seu tipo de resultado para implementar `FromIterator`, `partition` em vez disso requer `std::default::Default`, que todas as coleções Rust implementam retornando uma coleção vazia e `std::default::Extend`.

Outras linguagens oferecem operações `partition` que apenas dividem o iterador em dois iteradores, em vez de construir duas coleções. Mas isso não é uma boa opção para Rust: os itens extraídos do iterador subjacente, mas ainda não extraídos do iterador particionado apropriado, precisariam ser armazenados em algum lugar; você acabaria construindo uma coleção de algum tipo internamente, de qualquer maneira.

for_each e try_for_each

O método `for_each` simplesmente aplica uma closure a cada item:

```
["doves", "hens", "birds"].iter()
    .zip(["turtle", "french", "calling"])
    .zip(2..5)
    .rev()
    .map(|((item, kind), quantity)| {
```

```

        format!("{}", {}, {}, quantity, kind, item)
    })
    .for_each(|gift| {
        println!("You have received: {}", gift);
    });

```

Isso imprime:

```

You have received: 4 calling birds
You have received: 3 french hens
You have received: 2 turtle doves

```

Isso é muito semelhante a um simples loop `for`, no qual você também pode utilizar estruturas de controle como `break` e `continue`. Mas longas cadeias de chamadas de adaptador como essa são um pouco estranhas em loops `for`:

```

for gift in ["doves", "hens", "birds"].iter()
    .zip(["turtle", "french", "calling"])
    .zip(2..5)
    .rev()
    .map(|((item, kind), quantity)| {
        format!("{}", {}, {}, quantity, kind, item)
    })
{
    println!("You have received: {}", gift);
}

```

O padrão sendo vinculado, `gift`, pode acabar bem longe do corpo do loop no qual é utilizado.

Se sua closure precisa ser falível ou sair mais cedo, você pode utilizar `try_for_each`:

```

...
    .try_for_each(|gift| {
        writeln!(&mut output_file, "You have received: {}", gift)
    })?;

```

Implementando seus próprios iteradores

Você pode implementar os traits `IntoIterator` e `Iterator` para seus próprios tipos, tornando todos os adaptadores e consumidores mostrados neste capítulo disponíveis para uso, com muitas outras bibliotecas e código de crate escritos para funcionar com a interface do iterador padrão. Nesta seção, mostraremos um iterador simples sobre um tipo de intervalo e, a seguir, um iterador mais complexo sobre um

tipo de árvore binária.

Suponha que temos o seguinte tipo de intervalo (simplificado a partir do tipo `std::ops::Range<T>` da biblioteca padrão):

```
struct I32Range {  
    start: i32,  
    end: i32  
}
```

Iterar sobre um `I32Range` requer dois fragmentos de estado: o valor atual e o limite em que a iteração deve terminar. Isso se ajusta perfeitamente ao próprio tipo `I32Range`, utilizando `start` como o próximo valor e `end` como o limite. Então você pode implementar `Iterator` assim:

```
impl Iterator for I32Range {  
    type Item = i32;  
    fn next(&mut self) -> Option<i32> {  
        if self.start >= self.end {  
            return None;  
        }  
        let result = Some(self.start);  
        self.start += 1;  
        result  
    }  
}
```

Esse iterador produz itens `i32`, então esse é o tipo `Item`. Se a iteração estiver completa, `next` retorna `None`; caso contrário, produz o próximo valor e atualiza seu estado para se preparar para a próxima chamada.

Naturalmente, um loop `for` utiliza `IntoIterator::into_iter` para converter seu operando em um iterador. Mas a biblioteca padrão fornece uma implementação geral de `IntoIterator` para cada tipo que implementa `Iterator`; portanto, `I32Range` está pronto para uso:

```
let mut pi = 0.0;  
let mut numerator = 1.0;  
  
for k in (I32Range { start: 0, end: 14 }) {  
    pi += numerator / (2*k + 1) as f64;  
    numerator /= -3.0;  
}  
pi *= f64::sqrt(12.0);  
  
// O IEEE 754 especifica esse resultado exatamente  
assert_eq!(pi as f32, std::f32::consts::PI);
```

Mas `I32Range` é um caso especial, pois o iterável e o iterador são do mesmo tipo. Muitos casos não são tão simples. Por exemplo, eis o tipo de árvore binária do Capítulo 10:

```
enum BinaryTree<T> {
    Empty,
    NonEmpty(Box<TreeNode<T>>)
}

struct TreeNode<T> {
    element: T,
    left: BinaryTree<T>,
    right: BinaryTree<T>
}
```

A maneira clássica de percorrer uma árvore binária é recursivamente, utilizando o stack (pilha) de chamadas de função para acompanhar seu lugar na árvore e os nós ainda a serem visitados. Mas, ao implementar `Iterator` para `BinaryTree<T>`, cada chamada a `next` deve produzir exatamente um valor e retornar. Para acompanhar os nós da árvore que ele ainda não produziu, o iterador deve manter seu próprio stack (pilha). Eis um tipo de iterador possível para `BinaryTree`:

```
use self::BinaryTree::*;

// O estado de um percurso em ordem de uma `BinaryTree`
struct TreeIter<'a, T> {
    // Um stack de referências a nós de árvore. Como utilizamos os métodos
    // `push` e `pop` de `Vec`, o topo do stack é o fim do vetor
    //
    // O nó que o iterador visitará em seguida está na parte superior do
    // stack, com aqueles ancestrais ainda não visitados abaixo dele.
    // Se o stack estiver vazio, a iteração acabou.
    unvisited: Vec<&'a TreeNode<T>>
}
```

Quando criamos um novo `TreeIter`, seu estado inicial deve estar prestes a produzir o nó folha mais à esquerda na árvore. De acordo com as regras para o stack (pilha) `unvisited`, esta deve ter aquela folha na parte superior, seguida por seus ancestrais não visitados: os nós ao longo do lado esquerdo da árvore. Podemos inicializar `unvisited` caminhando pelo lado esquerdo da árvore a partir da raiz até a folha e inserindo cada nó que encontrarmos; portanto, definiremos um

método em `TreeIter` para fazer isso:

```
impl<'a, T: 'a> TreeIter<'a, T> {  
    fn push_left_edge(&mut self, mut tree: &'a BinaryTree<T>) {  
        while let NonEmpty(ref node) = *tree {  
            self.unvisited.push(node);  
            tree = &node.left;  
        }  
    }  
}
```

Escrever `mut tree` permite que o loop mude o nó para o qual `tree` aponta enquanto caminhamos pelo lado esquerdo; mas, como `tree` é uma referência compartilhada, não podemos modificar os próprios nós.

Com esse método auxiliar instalado, podemos fornecer a `BinaryTree` um método `iter` que retorna um iterador sobre a árvore:

```
impl<T> BinaryTree<T> {  
    fn iter(&self) -> TreeIter<T> {  
        let mut iter = TreeIter { unvisited: Vec::new() };  
        iter.push_left_edge(self);  
        iter  
    }  
}
```

O método `iter` constrói um `TreeIter` com um stack (pilha) `unvisited` vazio e depois chama `push_left_edge` para inicializá-lo. O nó mais à esquerda termina a parte superior, conforme exigido pelas regras do stack `unvisited`.

Seguindo as práticas da biblioteca padrão, podemos então implementar `IntoIterator` em uma referência compartilhada a uma árvore com uma chamada a `BinaryTree::iter`:

```
impl<'a, T: 'a> IntoIterator for &'a BinaryTree<T> {  
    type Item = &'a T;  
    type IntoIter = TreeIter<'a, T>;  
    fn into_iter(self) -> Self::IntoIter {  
        self.iter()  
    }  
}
```

A definição de `IntoIter` estabelece `TreeIter` como o tipo de iterador para uma `&BinaryTree`.

Por fim, na implementação de `Iterator`, podemos realmente andar na

árvore. Como método `iter` de `BinaryTree`, o método do iterador `next` é guiado pelas regras do stack:

```
impl<'a, T> Iterator for TreeIter<'a, T> {
  type Item = &'a T;
  fn next(&mut self) -> Option<&'a T> {
    // Encontra o nó que essa iteração deve produzir,
    // ou termina a iteração. (Utiliza o operador `?`
    // para retornar imediatamente se for `None`.)
    let node = self.unvisited.pop()?;

    // Depois de `node`, a próxima coisa que produzimos deve ser o filho mais à
    // esquerda na subárvore direita de `node`, então inserimos o caminho daqui
    // para baixo. Nosso método auxiliar acaba sendo exatamente o que precisamos
    self.push_left_edge(&node.right);

    // Produz uma referência para o valor deste nó
    Some(&node.element)
  }
}
```

Se o stack estiver vazio, a iteração está completa. Por outro lado, `node` é uma referência ao nó a ser visitado agora; essa chamada retornará uma referência ao seu campo `element`. Mas, primeiro, devemos avançar o estado do iterador para o próximo nó. Se esse nó tiver uma subárvore à direita, o próximo nó a ser visitado é o nó mais à esquerda da subárvore e podemos utilizar `push_left_edge` para inserir a ele e seus ancestrais não visitados, no stack (pilha). Mas, se esse nó não tiver uma subárvore direita, `push_left_edge` não tem efeito, que é exatamente o que queremos: podemos contar com que o novo topo do stack será o primeiro ancestral não visitado de `node`, se houver um.

Com as implementações de `IntoIterator` e `Iterator` em vigor, podemos finalmente utilizar um loop `for` para iterar por uma `BinaryTree` por referência. Utilizando o método `add` em `BinaryTree` a partir de “Preenchendo uma árvore binária”, na página 294:

```
// Constrói uma pequena árvore
let mut tree = BinaryTree::Empty;
tree.add("jaeger");
tree.add("robot");
tree.add("droid");
tree.add("mecha");
```

```
// Itera por ela
let mut v = Vec::new();
for kind in &tree {
    v.push(*kind);
}
assert_eq!(v, ["droid", "jaeger", "mecha", "robot"]);
```

A Figura 15.1 mostra como o stack `unvisited` se comporta à medida que iteramos por uma árvore de exemplo. A cada passo, o próximo nó a ser visitado está na parte superior do stack, com todos os seus ancestrais não visitados abaixo dele.

Todos os adaptadores de iteradores e consumidores usuais estão prontos para uso em nossas árvores:

```
assert_eq!(tree.iter()
    .map(|name| format!("mega-{}", name))
    .collect::<Vec<_>>(),
    vec!["mega-droid", "mega-jaeger",
        "mega-mecha", "mega-robot"]);
```

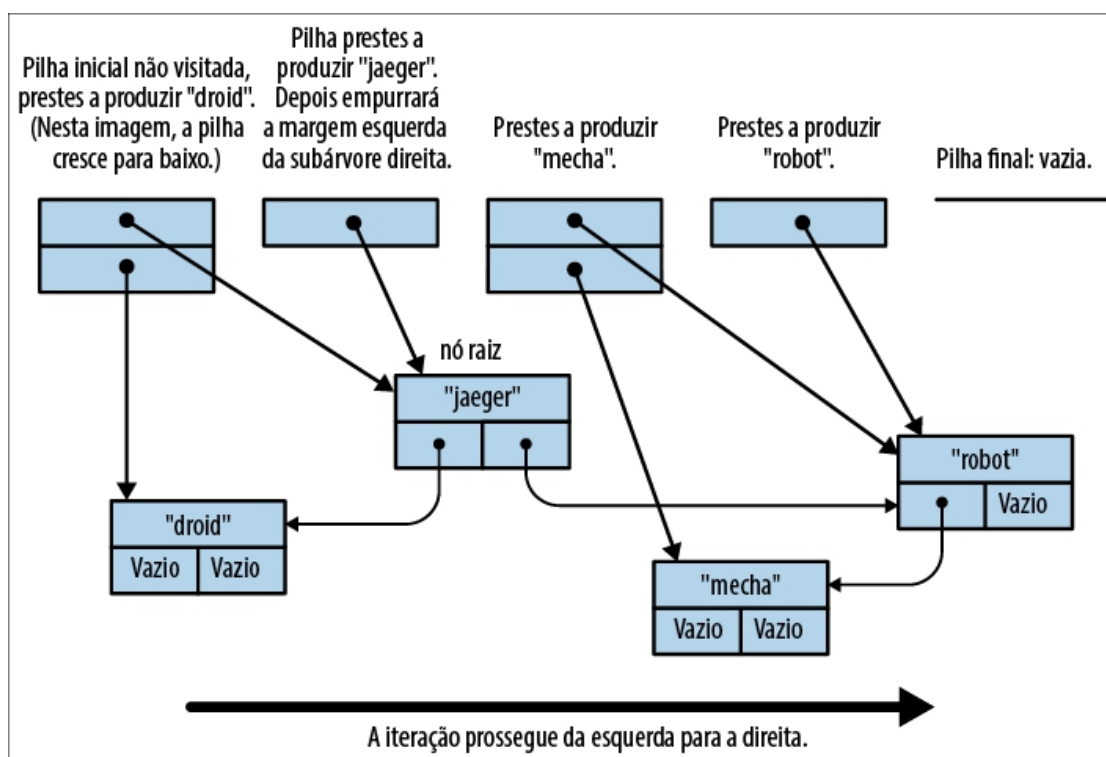


Figura 15.1: Iterando por uma árvore binária.

Os iteradores são a personificação da filosofia do Rust de fornecer abstrações poderosas e de custo zero que melhoram a expressividade e a legibilidade do código. Os iteradores não

substituem totalmente os loops, mas fornecem uma primitiva poderosa com avaliação lazy (preguiçosa) integrada e excelente desempenho.

[1](#) Na verdade, como Option é um iterável se comportando como uma sequência de zero ou um item, `iterator.filter_map(closure)` é equivalente a `iterator.flat_map(closure)`, assumindo que closure retorna uma `Option<T>`.

CAPÍTULO 16

Coleções

Todos nós nos comportamos como o demônio de Maxwell. Organismos se organizam. Na experiência cotidiana está a razão pela qual físicos sérios mantiveram viva essa fantasia de desenho animado ao longo de dois séculos. Separamos a correspondência, construímos castelos de areia, resolvemos quebra-cabeças, separamos o joio do trigo, reorganizamos as peças de xadrez, colecionamos selos, colocamos livros em ordem alfabética, criamos simetrias, compomos sonetos e sonatas, e colocamos nossos quartos em ordem; e tudo isso que fazemos não requer grande energia, desde que possamos aplicar a inteligência.

—James Gleick, The Information: A History, a Theory, a Flood

A biblioteca padrão do Rust contém várias *coleções*, tipos genéricos para armazenar dados na memória. Já utilizamos coleções, como `Vec` e `HashMap`, ao longo deste livro. Neste capítulo, abordaremos os métodos desses dois tipos em detalhes, com a outra meia dúzia de coleções padrão. Antes de começarmos, vamos abordar algumas diferenças sistemáticas entre as coleções do Rust e as de outras linguagens.

Primeiro, movimentos e empréstimos estão por toda parte. O Rust utiliza movimentos para evitar a cópia profunda (deep-copy) de valores. É por isso que o método `Vec<T>::push(item)` toma seu argumento por valor, não por referência. O valor é movido para o vetor. Os diagramas no Capítulo 4 mostram como isso funciona na prática: empurrar uma `String` do Rust para um `Vec<String>` é rápido, porque o Rust não precisa copiar os caracteres da string e a posse da string é sempre clara.

Em segundo lugar, o Rust não tem erros de invalidação – o tipo de erro de ponteiro perdido em que uma coleção é redimensionada ou

alterada de outra forma, enquanto o programa mantém um ponteiro para os dados dentro dela. Erros de invalidação são outra fonte de comportamento indefinido em C++ e causam o ocasional `ConcurrentModificationException`, mesmo em linguagens com segurança de memória. O verificador de empréstimo do Rust os descarta em tempo de compilação.

Por fim, o Rust não tem `null`, então veremos `Options` em lugares onde outras linguagens utilizariam `null`.

Além dessas diferenças, as coleções do Rust são o que você esperaria. Se você é um programador experiente com pressa, pode dar uma olhada aqui, mas não perca “Entradas”, na página 477.

Visão geral

A Tabela 16.1 mostra as oito coleções padrão do Rust. Todas elas são tipos genéricos.

Tabela 16.1: Resumo das coleções padrão

Coleção	Descrição	Tipo de coleção semelhante em...		
		C++	Java	Python
<code>Vec<T></code>	Array expansível	<code>vector</code>	<code>ArrayList</code>	<code>list</code>
<code>VecDeque<T></code>	Fila dupla (buffer circular expansível)	<code>deque</code>	<code>ArrayDeque</code>	<code>collections.deque</code>
<code>LinkedList<T></code>	Lista duplamente vinculada	<code>list</code>	<code>LinkedList</code>	—
<code>BinaryHeap<T></code> where <code>T: Ord</code>	Heap máximo	<code>priority_queue</code>	<code>PriorityQueue</code>	<code>heapq</code>
<code>HashMap<K, V></code> where <code>K: Eq + Hash</code>	Tabela hash de chave-valor	<code>unordered_map</code>	<code>HashMap</code>	<code>dict</code>
<code>BTreeMap<K, V></code> where <code>K: Ord</code>	Tabela de valores-chave ordenados	<code>map</code>	<code>TreeMap</code>	—
<code>HashSet<T></code> where <code>T: Eq + Hash</code>	Conjunto não ordenado, baseado em hash	<code>unordered_set</code>	<code>HashSet</code>	<code>set</code>
<code>BTreeSet<T></code> where <code>T: Ord</code>	Conjunto ordenado	<code>set</code>	<code>TreeSet</code>	—

`Vec<T>`, `HashMap<K, V>` e `HashSet<T>` são os tipos de coleção geralmente

mais úteis. O restante tem usos de nicho. Este capítulo discute cada tipo de coleção por vez:

`Vec<T>`

Um array expansível e alocado em heap de valores do tipo τ . Cerca de metade deste capítulo é dedicada a `Vec` e seus muitos métodos úteis.

`VecDeque<T>`

Como `Vec<T>`, mas melhor para uso como uma fila do tipo primeiro a entrar, primeiro a sair. Suporta a adição e remoção eficiente de valores na frente da lista, bem como na parte de trás. Isso tem o custo de tornar todas as outras operações um pouco mais lentas.

`BinaryHeap<T>`

Uma fila de prioridades. Os valores em um `BinaryHeap` são organizados para que seja sempre eficiente encontrar e remover o valor máximo.

`HashMap<K, V>`

Uma tabela de pares chave-valor. Procurar um valor por sua chave é rápido. As entradas são armazenadas em uma ordem arbitrária.

`BTreeMap<K, V>`

Como `HashMap<K, V>`, mas mantém as entradas ordenadas por chave. Um `BTreeMap<String, i32>` armazena suas entradas na ordem de comparação de `String`. A menos que você precise que as entradas fiquem organizadas, um `HashMap` é mais rápido.

`HashSet<T>`

Um conjunto de valores do tipo τ . Adicionar e remover valores é rápido e é rápido perguntar se um determinado valor está no conjunto ou não.

`BTreeSet<T>`

Como `HashSet<T>`, mas mantém os elementos ordenados por valor. Novamente, a menos que você precise ordenar os dados, um `HashSet` é mais rápido.

Como `LinkedList` é raramente utilizado (e existem alternativas melhores, tanto em desempenho quanto em interface, para a maioria dos casos de uso), não o descrevemos aqui.

Vec<T>

Vamos assumir alguma familiaridade com `vec`, já que o utilizamos ao longo do livro. Para uma introdução, consulte “Vetores”, na página 92. Aqui finalmente descreveremos seus métodos e seu funcionamento interno em profundidade.

A maneira mais fácil de criar um vetor é utilizar a macro `vec!`:

```
// Cria um vetor vazio
let mut numbers: Vec<i32> = vec![];

// Cria um vetor com o conteúdo fornecido
let words = vec!["step", "on", "no", "pets"];
let mut buffer = vec![0u8; 1024]; // 1024 bytes zerados
```

Conforme descrito no Capítulo 4, um vetor possui três campos: o comprimento, a capacidade e um ponteiro para uma alocação de heap onde os elementos são armazenados. A Figura 16.1 mostra como os vetores anteriores apareceriam na memória. O vetor vazio, `numbers`, inicialmente tem uma capacidade de 0. Nenhuma memória de heap é alocada para ele até que o primeiro elemento seja adicionado.

Como todas as coleções, `Vec` implementa `std::iter::FromIterator`, então você pode criar um vetor a partir de qualquer iterador utilizando o método `.collect()` do iterador, conforme descrito em “Construindo coleções: `collect` e `FromIterator`”, na página 440:

```
// Converte outra coleção em um vetor
let my_vec = my_set.into_iter().collect::<Vec<String>>();
```

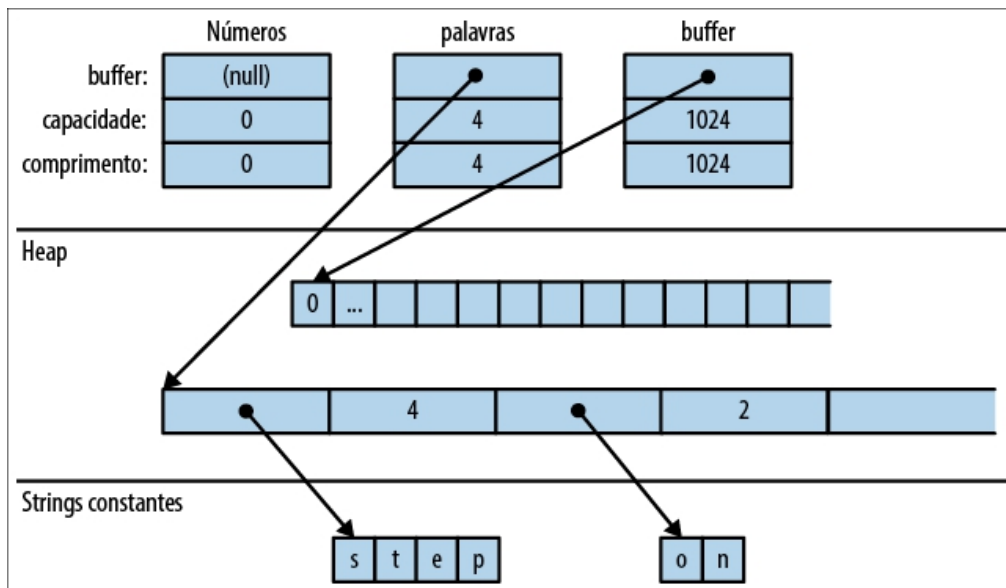



Figura 16.1: Disposição vetorial na memória: cada elemento das palavras é um valor `&str` que consiste em um ponteiro e um comprimento.

Acessando elementos

Obter elementos de um array, fatia ou vetor por índice é simples:

```
// Obtém uma referência a um elemento
let first_line = &lines[0];

// Obtém uma cópia de um elemento
let fifth_number = numbers[4];    // requer Copy
let second_line = lines[1].clone(); // requer Clone

// Obtém uma referência a uma fatia
let my_ref = &buffer[4..12];

// Obtém uma cópia de uma fatia
let my_copy = buffer[4..12].to_vec(); // requer Clone
```

Todas essas formas geram um pânico se um índice estiver fora dos limites.

O Rust é exigente quanto a tipos numéricos e não faz exceções para vetores. Comprimentos e índices vetoriais são do tipo `usize`. Tentar utilizar um `u32`, `u64` ou `isize` como um índice vetorial é um erro. Você pode utilizar uma conversão `as` `usize` para converter conforme necessário; veja “Coerções de tipo”, na página 185.

Vários métodos fornecem acesso fácil a elementos específicos de um vetor ou fatia (observe que todos os métodos de fatia também estão disponíveis em arrays e vetores):

`slice.first()`

Retorna uma referência ao primeiro elemento de `slice`, caso existam.

O tipo de retorno é `Option<&T>`, então o valor de retorno é `None` se `slice` estiver vazio e `Some(&slice[0])` se não estiver vazio:

```
if let Some(item) = v.first() {  
    println!("We got one! {}", item);  
}
```

`slice.last()`

Semelhante, mas retorna uma referência ao último elemento.

`slice.get(index)`

Retorna referência `Some` a `slice[index]`, se existir. Se `slice` tem menos que `index+1` elementos, isso retorna `None`:

```
let slice = [0, 1, 2, 3];  
assert_eq!(slice.get(2), Some(&2));  
assert_eq!(slice.get(4), None);
```

`slice.first_mut()`, `slice.last_mut()`, `slice.get_mut(index)`

Variações do precedente que emprestam referências `mut`:

```
let mut slice = [0, 1, 2, 3];  
{  
    let last = slice.last_mut().unwrap(); // tipo de last: &mut i32  
    assert_eq!(*last, 3);  
    *last = 100;  
}  
assert_eq!(slice, [0, 1, 2, 100]);
```

Como devolver um `T` por valor significaria movê-lo, métodos que acessam elementos no local normalmente retornam esses elementos por referência.

Uma exceção é o método `.to_vec()`, que faz cópias:

`slice.to_vec()`

Clona uma fatia inteira, retornando um novo vetor:

```
let v = [1, 2, 3, 4, 5, 6, 7, 8, 9];  
assert_eq!(v.to_vec(),  
    vec![1, 2, 3, 4, 5, 6, 7, 8, 9]);  
assert_eq!(v[0..6].to_vec(),  
    vec![1, 2, 3, 4, 5, 6]);
```

Esse método está disponível apenas se os elementos forem clonáveis, ou seja, where T: Clone.

Iteração

Vetores, arrays e fatias são iteráveis, seja por valor ou por referência, seguindo o padrão descrito em “Implementações de IntoIterator”, na página 405:

- Iterar por um `Vec<T>` ou array `[T; N]` produz itens do tipo `T`. Os elementos são movidos para fora do vetor ou array um a um, consumindo-o.
- Iterar por um valor do tipo `&[T; N]`, `&[T]` ou `&Vec<T>` – isto é, uma referência a um array, fatia ou vetor – produz itens do tipo `&T`, referências aos elementos individuais, que não são movidos.
- Iterar por um valor do tipo `&mut [T; N]`, `&mut [T]` ou `&mut Vec<T>` produz itens do tipo `&mut T`.

Arrays, fatias e vetores também têm métodos `.iter()` e `.iter_mut()` (descritos em “Métodos iter e iter_mut”, na página 404) para criar iteradores que produzem referências a seus elementos.

Abordaremos algumas maneiras mais sofisticadas de iterar por uma fatia em “Dividindo”, na página 461.

Crescendo e encolhendo vetores

O comprimento de um array, fatia ou vetor é o número de elementos que ele contém:

`slice.len()`

Retorna o comprimento do `slice`, como um `usize`.

`slice.is_empty()`

É verdade se `slice` não contém elementos (ou seja, `slice.len() == 0`).

Os métodos restantes nesta seção são sobre crescimento e encolhimento de vetores. Eles não estão presentes em arrays e fatias, que não podem ser redimensionadas depois de criadas.

Todos os elementos de um vetor são armazenados em um bloco contíguo de memória alocado em heap. A capacidade de um vetor é

o número máximo de elementos que caberiam nesse pedaço. `Vec` normalmente gerencia a capacidade para você, alocando automaticamente um buffer maior e movendo os elementos para ele quando mais espaço é necessário. Existem também alguns métodos para gerenciar a capacidade explicitamente:

`Vec::with_capacity(n)`

Cria um novo vetor vazio com capacidade `n`.

`vec.capacity()`

Retorna a capacidade de `vec`, como um `usize`. É sempre verdadeiro que `vec.capacity() >= vec.len()`.

`vec.reserve(n)`

Garante que o vetor tenha pelo menos capacidade sobressalente suficiente para mais `n` elementos: isto é, `vec.capacity()` tem pelo menos `vec.len() + n`. Se já houver espaço suficiente, isso não fará nada. Caso contrário, isso aloca um buffer maior e move o conteúdo do vetor para ele.

`vec.reserve_exact(n)`

Como `vec.reserve(n)`, mas instrui `vec` a não alocar nenhuma capacidade extra para crescimento futuro, além de `n`. Depois de chamado, `vec.capacity()` é exatamente `vec.len() + n`.

`vec.shrink_to_fit()`

Tenta liberar a memória extra se `vec.capacity()` for maior que `vec.len()`.

`Vec<T>` tem muitos métodos que adicionam ou removem elementos, alterando o comprimento do vetor. Cada um deles recebe seu argumento `self` por referência `mut`.

Esses dois métodos adicionam ou removem um único valor no final de um vetor:

`vec.push(value)`

Adiciona `value` ao final de `vec`.

`vec.pop()`

Remove e retorna o último elemento. O tipo de retorno é `Option<T>`. Isso retorna `Some(x)` se o elemento removido for `x` e `None` se o vetor já estiver vazio.

Observe que `.push()` toma seu argumento por valor, não por

referência. Da mesma maneira, `.pop()` retorna o valor exibido, não uma referência. O mesmo é verdadeiro para a maioria dos métodos restantes nesta seção. Eles movem valores para dentro e para fora dos vetores.

Esses dois métodos adicionam ou removem um valor em qualquer lugar em um vetor:

`vec.insert(index, value)`

Insere o dado `value` em `vec[index]`, deslocando quaisquer valores existentes em `vec[index..]` uma casa à direita para abrir espaço.

Gera um pânico se `index > vec.len()`.

`vec.remove(index)`

Remove e retorna `vec[index]`, deslocando quaisquer valores existentes em `vec[index+1..]` uma posição à esquerda para fechar a lacuna.

Gera um pânico se `index >= vec.len()`, pois nesse caso não há elemento `vec[index]` a remover.

Quanto mais longo o vetor, mais lenta fica essa operação. Se você se encontrar fazendo `vec.remove(0)` muitas vezes, considere utilizar um `VecDeque` (explicado em “`VecDeque<T>`”, na página 469) em vez de um `Vec`.

Ambos `.insert()` e `.remove()` são mais lentos quanto mais elementos precisam ser deslocados.

Quatro métodos alteram o comprimento de um vetor para um valor específico:

`vec.resize(new_len, value)`

Configura comprimento de `vec` como `new_len`. Se isso aumentar o comprimento de `vec`, cópias de `value` são adicionados para preencher o novo espaço. O tipo de elemento deve implementar o trait `Clone`.

`vec.resize_with(new_len, closure)`

Assim como `vec.resize`, mas chama a closure para construir cada novo elemento. Pode ser utilizado com vetores de elementos que não são `Clone`.

`vec.truncate(new_len)`

Reduz o comprimento de `vec` para `new_len`, dropando todos os elementos que estiverem no intervalo `vec[new_len..]`.

Se `vec.len()` já for menor ou igual a `new_len`, nada acontece.

`vec.clear()`

Remove todos os elementos de `vec`. É o mesmo que `vec.truncate(0)`.

Quatro métodos adicionam ou removem muitos valores de uma só vez:

`vec.extend(iterable)`

Adiciona todos os itens do dado valor `iterable` ao final de `vec`, em ordem. É como uma versão multivalor de `.push()`. O `iterable` argumento pode ser qualquer coisa que implemente `IntoIterator<Item=T>`.

Esse método é tão útil que existe um `trait` padrão para ele, o `trait Extend`, que todas as coleções padrão implementam. Infelizmente, isso faz `rustdoc` colocar `.extend()` com outros métodos de `trait` em uma grande seção na parte inferior do HTML gerado, por isso é difícil encontrá-lo quando você precisa dele. Você apenas tem de lembrar que está lá! Ver “`Trait Extend`”, na página 442, para mais.

`vec.split_off(index)`

Como `vec.truncate(index)`, exceto que ele retorna um `Vec<T>` contendo os valores removidos do final de `vec`. É como uma versão multivalor de `.pop()`.

`vec.append(&mut vec2)`

Isso move todos os elementos de `vec2` para `vec`, em que `vec2` é outro vetor do tipo `Vec<T>`. Depois disso, `vec2` está vazio.

Isso é como `vec.extend(vec2)` exceto que `vec2` ainda existe depois, com sua capacidade inalterada.

`vec.drain(range)`

Isso remove o `range` `vec[range]` a partir de `vec` e retorna um iterador sobre os elementos removidos, em que `range` é um valor de intervalo, como `..` ou `0..4`.

Existem também alguns métodos excêntricos para remover seletivamente alguns dos elementos de um vetor:

`vec.retain(test)`

Remove todos os elementos que não passam no teste fornecido. O argumento `test` é uma função ou closure que implementa `FnMut(&T) ->`

bool. Para cada elemento de `vec`, isso chama `test(&element)` e, se retornar `false`, o elemento é removido do vetor e dropado.

Além do desempenho, é como escrever:

```
vec = vec.into_iter().filter(test).collect();  
vec.dedup()
```

Dropa elementos repetidos. É como o utilitário de shell `uniq` do Unix. Ele varre `vec` à procura de lugares onde os elementos adjacentes são iguais e dropa os valores extras iguais para que apenas um seja deixado:

```
let mut byte_vec = b"Misssssssissippi".to_vec();  
byte_vec.dedup();  
assert_eq!(&byte_vec, b"Misisipi");
```

Note que ainda existem dois caracteres 's' na saída. Esse método remove apenas duplicatas adjacentes. Para eliminar todas as duplicatas, você tem três opções: ordenar o vetor antes de chamar `.dedup()`, mover os dados para um conjunto ou (para manter os elementos em sua ordem original) utilizar este truque com `.retain()`:

```
let mut byte_vec = b"Misssssssissippi".to_vec();  
  
let mut seen = HashSet::new();  
byte_vec.retain(|r| seen.insert(*r));
```

```
assert_eq!(&byte_vec, b"Misp");
```

Isso funciona porque `.insert()` retorna `false` quando o conjunto já contém o item que estamos inserindo.

```
vec.dedup_by(same)
```

O mesmo que `vec.dedup()`, mas utiliza a função ou closure `same(&mut elem1, &mut elem2)`, em vez de operador `==`, para verificar se dois elementos devem ser considerados iguais.

```
vec.dedup_by_key(key)
```

O mesmo que `vec.dedup()`, mas trata dois elementos como iguais se `key(&mut elem1) == key(&mut elem2)`.

Por exemplo, se `errors` é um `Vec<Box<dyn Error>>`, você pode escrever:

```
// Remove erros com mensagens redundantes  
errors.dedup_by_key(|err| err.to_string());
```

De todos os métodos abordados nesta seção, apenas `.resize()` nunca clona valores. Os outros trabalham movendo valores de um lugar

para outro.

Unindo

Dois métodos funcionam em arrays de arrays, o que significa qualquer array, fatia ou vetor cujos elementos sejam arrays, fatias ou vetores:

`slices.concat()`

Retorna um novo vetor feito pela concatenação de todas as fatias:

```
assert_eq!([[1, 2], [3, 4], [5, 6]].concat(),
            vec![1, 2, 3, 4, 5, 6]);
```

`slices.join(&separator)`

O mesmo, exceto que uma cópia do valor `separator` é inserida entre as fatias:

```
assert_eq!([[1, 2], [3, 4], [5, 6]].join(&0),
            vec![1, 2, 0, 3, 4, 0, 5, 6]);
```

Dividindo

É fácil acabar ficando com muitas referências não-mut em um array, fatia ou vetor de uma só vez:

```
let v = vec![0, 1, 2, 3];
let a = &v[i];
let b = &v[j];

let mid = v.len() / 2;
let front_half = &v[..mid];
let back_half = &v[mid..];
```

Obter várias referências mut não é tão fácil:

```
let mut v = vec![0, 1, 2, 3];
let a = &mut v[i];
let b = &mut v[j]; // erro: não é possível emprestar `v` como mutável
                  //      mais de uma vez por vez

*a = 6;           // as referências `a` e `b` são utilizadas aqui,
*b = 7;           // então seus tempos de vida devem se sobrepor
```

O Rust proíbe isso porque, se `i == j`, então `a` e `b` seriam duas referências mut ao mesmo número inteiro, violando as regras de segurança do Rust. (Ver “Compartilhamento versus mutação”, na página 153.)

O Rust tem vários métodos que podem emprestar referências `mut` a duas ou mais partes de um array, fatia ou vetor de uma só vez. Ao contrário do código anterior, esses métodos são seguros porque, por design, sempre dividem os dados em regiões não sobrepostas. Muitos desses métodos também são úteis para trabalhar com fatias `mut`; portanto, existem versões `mut` e não-`mut` de cada um.

A Figura 16.2 ilustra esses métodos.

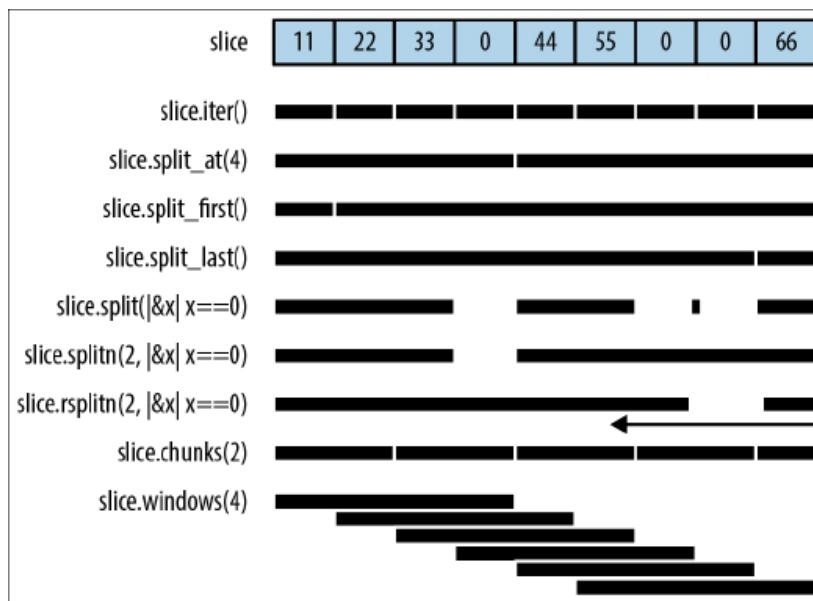


Figura 16.2: Métodos de divisão ilustrados (nota: o pequeno retângulo na saída de `slice.split()` é uma fatia vazia causada pelos dois separadores adjacentes e `rsplitn` produz sua saída na ordem do fim para o início, ao contrário dos outros).

Nenhum desses métodos modifica diretamente um array, fatia ou vetor; eles apenas retornam novas referências a partes dos dados dentro deles:

`slice.iter()`, `slice.iter_mut()`

Produz uma referência para cada elemento de `slice`. Abordamos isso em “Iteração”, na página 456.

`slice.split_at(index)`, `slice.split_at_mut(index)`

Quebre uma fatia em duas, retornando um par. `slice.split_at(index)` é equivalente a `(&slice[..index], &slice[index..])`. Esses métodos geram pânico se `index` estiver fora dos limites.

`slice.split_first()`, `slice.split_first_mut()`

Retorna também um par: uma referência ao primeiro elemento (`slice[0]`) e uma referência de fatia para todo o resto (`slice[1..]`).

O tipo de retorno de `.split_first()` é `Option<(&T, &[T])>`; o resultado é `None` se `slice` estiver vazia.

`slice.split_last()`, `slice.split_last_mut()`

Estes são análogos, mas separam o último elemento em vez do primeiro.

O tipo de retorno de `.split_last()` é `Option<(&T, &[T])>`.

`slice.split(is_sep)`, `slice.split_mut(is_sep)`

Dividir `slice` em uma ou mais subfatias, utilizando a função ou closure `is_sep` para descobrir onde dividir. Retornam um iterador sobre as subfatias.

À medida que você consome o iterador, ele chama `is_sep(&element)` para cada elemento da fatia. Se `is_sep(&element)` for `true`, o elemento é um separador. Os separadores não são incluídos em nenhuma subfatia de saída.

A saída sempre contém pelo menos uma subfatia, mais uma por separador. Subfatias vazias são incluídas sempre que separadores aparecem adjacentes uns aos outros ou nas extremidades `slice`.

`slice.split_inclusive(is_sep)`, `slice.split_inclusive_mut(is_sep)`

Estes funcionam como `split` e `split_mut`, mas incluem o separador no final da subfatia anterior em vez de excluí-lo.

`slice.rsplit(is_sep)`, `slice.rsplit_mut(is_sep)`

Assim como `slice` e `slice_mut`, mas começa no final da fatia.

`slice.splitn(n, is_sep)`, `slice.splitn_mut(n, is_sep)`

O mesmo, mas produzem no máximo `n` subfatias. Depois que as primeiras `n-1` fatias são encontradas, `is_sep` não é chamado novamente. A última subfatia contém todos os elementos restantes.

`slice.rsplitn(n, is_sep)`, `slice.rsplitn_mut(n, is_sep)`

Assim como `.splitn()` e `.splitn_mut()` exceto que a fatia é varrida na ordem inversa. Ou seja, esses métodos se dividem nos últimos `n-1`

separadores na fatia, em vez da primeira, e as subfatias são produzidas a partir do final.

`slice.chunks(n)`, `slice.chunks_mut(n)`

Retorna um iterador por subfatias não sobrepostas de comprimento `n`. Se `n` não divide `slice.len()` exatamente, o último pedaço conterá menos que `n` elementos.

`slice.rchunks(n)`, `slice.rchunks_mut(n)`

Como `slice.chunks` e `slice.chunks_mut`, mas começa no final da fatia.

`slice.chunks_exact(n)`, `slice.chunks_exact_mut(n)`

Retorna um iterador por subfatias não sobrepostas de comprimento `n`. Se `n` não divide `slice.len()`, o último pedaço (com menos de `n` elementos) está disponível no método `remainder()` do resultado.

`slice.rchunks_exact(n)`, `slice.rchunks_exact_mut(n)`

Como `slice.chunks_exact` e `slice.chunks_exact_mut`, mas começa no final da fatia.

Há mais um método para iterar subfatias:

`slice.windows(n)`

Retorna um iterador que se comporta como uma “janela deslizante” sobre os dados em `slice`. Produz subfatias que abrangem `n` elementos consecutivos de `slice`. O primeiro valor gerado é `&slice[0..n]`, o segundo é `&slice[1..n+1]` e assim por diante.

Se `n` for maior que o comprimento `slice`, então nenhuma fatia é produzida. Se `n` for 0, o método gera um pânico.

Por exemplo, se `days.len() == 31`, então podemos produzir todos os períodos de sete dias em `days` chamando `days.windows(7)`.

Uma janela deslizante de tamanho 2 é útil para explorar como uma série de dados muda de um ponto de dados para o próximo:

```
let changes = daily_high_temperatures
    .windows(2)           // obter temperaturas de dias adjacentes
    .map(|w| w[1] - w[0]) // quanto mudou?
    .collect::<Vec<_>>();
```

Como as subfatias se sobrepõem, não há variação desse método que retorna referências `mut`.

Permutando

Há métodos de conveniência para trocar o conteúdo das fatias:

```
slice.swap(i, j)
```

Troca os dois elementos `slice[i]` e `slice[j]`.

```
slice_a.swap(&mut slice_b)
```

Troca todo o conteúdo de `slice_a` e `slice_b`. `slice_a` e `slice_b` devem ter o mesmo comprimento.

Os vetores têm um método relacionado para remover eficientemente qualquer elemento:

```
vec.swap_remove(i)
```

Remove e retorna `vec[i]`. Isso é como `vec.remove(i)` exceto que, em vez de deslocar o restante dos elementos do vetor para fechar a lacuna, ele simplesmente move o último elemento de `vec` para a lacuna. É útil quando você não se preocupa com a ordem dos itens deixados no vetor.

Preenchimento

Existem dois métodos convenientes para substituir o conteúdo de fatias mutáveis:

```
slice.fill(value)
```

Preenche a fatia com clones de `value`.

```
slice.fill_with(function)
```

Preenche a fatia com valores criados pela chamada da função dada. Isso é especialmente útil para tipos que implementam `Default`, mas não são `Clone`, como `Option<T>` ou `Vec<T>` quando `T` não é `Clone`.

Ordenando e pesquisando

As fatias oferecem três métodos de ordenação:

```
slice.sort()
```

Classifica os elementos em ordem crescente. Esse método está presente apenas quando o tipo de elemento implementa `Ord`.

```
slice.sort_by(cmp)
```

Classifica os elementos de `slice` utilizando uma função ou closure `cmp`

para especificar a ordem de classificação. `cmp` deve implementar `Fn(&T, &T) -> std::cmp::Ordering`.

A implementação manual de `cmp` é tediosa, a menos que você delegue a um método `.cmp()`:

```
students.sort_by(|a, b| a.last_name.cmp(&b.last_name));
```

Para ordenar por um campo, utilizando um segundo campo como desempate, compare as tuplas:

```
students.sort_by(|a, b| {  
    let a_key = (&a.last_name, &a.first_name);  
    let b_key = (&b.last_name, &b.first_name);  
    a_key.cmp(&b_key)  
});
```

```
slice.sort_by_key(key)
```

Classifica os elementos de `slice` em ordem crescente por uma chave de ordenação, dada pela função ou closure `key`. O tipo de `key` deve implementar `Fn(&T) -> K` em que `K: Ord`.

Isso é útil quando `T` contém um ou mais campos ordenáveis, de forma que possa ser ordenada de várias maneiras:

```
// Classifica por média de notas, a mais baixa primeiro  
students.sort_by_key(|s| s.grade_point_average());
```

Observe que esses valores de chave de ordenação não são armazenados em cache durante a ordenação, portanto a função `key` pode ser chamada mais de `n` vezes.

Por razões técnicas, `key(element)` não pode retornar nenhuma referência emprestada do elemento. Isso não vai funcionar:

```
students.sort_by_key(|s| &s.last_name); // erro: não é possível inferir o tempo de vida
```

O Rust não consegue calcular os tempos de vida. Mas, nesses casos, é fácil recorrer a `.sort_by()`.

Todos os três métodos executam uma ordenação estável.

Para ordenar na ordem inversa, você pode utilizar `sort_by` com uma closure `cmp` que troca os dois argumentos. Tomar argumentos `|b, a|` em vez de `|a, b|` efetivamente produz a ordem oposta. Ou você pode simplesmente chamar para o método `.reverse()` após a ordenação:

```
slice.reverse()
```

Inverte uma fatia no mesmo lugar (sem alocar memória adicional).

Depois que uma fatia é ordenada, ela pode ser pesquisada com

eficiência:

```
slice.binary_search(&value),           slice.binary_search_by(&value,           cmp),  
slice.binary_search_by_key(&value, key)
```

Todas as buscas por `value` no dado `slice` ordenado. Observe que `value` é passado por referência.

O tipo de retorno desses métodos é `Result<usize, usize>`. Eles retornam `Ok(index)` se `slice[index]` for igual a `value` na ordem de classificação especificada. Se não houver esse índice, eles retornam `Err(insertion_point)` de forma que inserir `value` em `insertion_point` preservaria a ordem.

Obviamente, uma pesquisa binária só funciona se a fatia estiver de fato ordenada na ordem especificada. Caso contrário, os resultados são arbitrários – entra lixo, sai lixo.

Como `f32` e `f64` têm valores `NaN`, eles não implementam `Ord` e não podem ser utilizados diretamente como chaves com os métodos de ordenação e pesquisa binária. Para obter métodos semelhantes que funcionam em dados de ponto flutuante, utilize o crate `ord_subset`.

Existe um método para pesquisar um vetor que não está ordenado:

```
slice.contains(&value)
```

Retorna `true` se algum elemento de `slice` é igual a `value`. Isso simplesmente verifica cada elemento da fatia até que uma correspondência seja encontrada. Novamente, `value` é passado por referência.

Para encontrar a localização de um valor em uma fatia, como `array.indexOf(value)` em JavaScript, utilize um iterador:

```
slice.iter().position(|x| *x == value)
```

Isso retorna um `Option<usize>`.

Comparando fatias

Se um tipo `T` suporta os operadores `==` e `!=` (o trait `PartialEq`, descrito em “Comparações de equivalência”, na página 341), então arrays `[T; N]`, fatias `[T]` e vetores `Vec<T>` também os suportam. Duas fatias são iguais se tiverem o mesmo comprimento e seus elementos

correspondentes forem iguais. O mesmo vale para arrays e vetores.

Se `T` suporta os operadores `<`, `<=`, `>` e `>=` (o `trait PartialOrd`, descrito em “Comparações Ordenadas”, na página 345), então arrays, fatias e vetores de `T` também os suportam. As comparações de fatias são lexicográficas.

Dois métodos de conveniência realizam comparações comuns de fatias:

```
slice.starts_with(other)
```

Retorna `true` se `slice` começa com uma sequência de valores que são iguais aos elementos da fatia `other`:

```
assert_eq!([1, 2, 3, 4].starts_with(&[1, 2]), true);  
assert_eq!([1, 2, 3, 4].starts_with(&[2, 3]), false);
```

```
slice.ends_with(other)
```

Semelhante, mas verifica o final de `slice`:

```
assert_eq!([1, 2, 3, 4].ends_with(&[3, 4]), true);
```

Elementos aleatórios

Números aleatórios não são integrados na biblioteca padrão do Rust. O crate `rand`, que os fornece, oferece esses dois métodos para obter saída aleatória de um array, fatia ou vetor:

```
slice.choose(&mut rng)
```

Retorna uma referência a um elemento aleatório de uma fatia.

Assim como `slice.first()` e `slice.last()`, isso retorna um `Option<T>` que é `None` somente se a fatia estiver vazia.

```
slice.shuffle(&mut rng)
```

Reordena aleatoriamente os elementos de uma fatia no lugar. A fatia deve ser passada por referência `mut`.

Estes são métodos do `trait rand::Rng`, então você precisa de um `Rng`, um gerador de números aleatórios, para chamá-los. Felizmente, é fácil conseguir um chamando `rand::thread_rng()`. Para embaralhar o vetor `my_vec`, podemos escrever:

```
use rand::seq::SliceRandom;  
use rand::thread_rng;
```

```
my_vec.shuffle(&mut thread_rng());
```

O Rust descarta erros de invalidação

A maioria das linguagens de programação convencionais tem coleções e iteradores e todas elas têm alguma variação nesta regra: não modifique uma coleção enquanto estiver iterando por ela. Por exemplo, o equivalente Python de um vetor é uma lista:

```
my_list = [1, 3, 5, 7, 9]
```

Suponha que tentamos remover todos os valores maiores que 4 de `my_list`:

```
for index, val in enumerate(my_list):
    if val > 4:
        del my_list[index] # bug: modificando a lista durante a iteração

print(my_list)
```

(A função `enumerate` é o equivalente do Python do Rust método `.enumerate()`, descrito em “[enumerate](#)”, na página 426.)

Esse programa, surpreendentemente, imprime `[1, 3, 7]`. Mas sete é maior que quatro. Como isso escapou? Esse é um erro de invalidação: o programa modifica os dados enquanto itera sobre eles, invalidando o iterador. Em Java, o resultado seria uma exceção; em C++ é um comportamento indefinido. Em Python, embora o comportamento seja bem definido, não é intuitivo: o iterador pula um elemento. `val` nunca é 7.

Vamos tentar reproduzir esse bug no Rust:

```
fn main() {
    let mut my_vec = vec![1, 3, 5, 7, 9];

    for (index, &val) in my_vec.iter().enumerate() {
        if val > 4 {
            my_vec.remove(index); // erro: não é possível emprestar `my_vec` como
mutável
        }
    }
    println!("{}", my_vec);
}
```

Naturalmente, o Rust rejeita esse programa em tempo de compilação. Quando chamamos `my_vec.iter()`, ele toma emprestado uma referência compartilhada (não-mut) ao vetor. A referência dura tanto quanto o iterador, até o final do loop `for`. Não podemos

modificar o vetor chamando `my_vec.remove(index)` enquanto existe uma referência não-mut.

Ter um erro apontado para você é bom, mas é claro que você ainda precisa encontrar uma maneira de obter o comportamento desejado! A correção mais fácil aqui é escrever:

```
my_vec.retain(|&val| val <= 4);
```

Ou você pode fazer o que faria no Python ou em qualquer outra linguagem: criar um novo vetor utilizando um `filter`.

VecDeque<T>

`Vec` suporta adicionar e remover elementos de forma eficiente apenas no final. Quando um programa precisa de um local para armazenar valores que estão “esperando na fila”, `Vec` pode ser lento.

O Rust `std::collections::VecDeque<T>` é um deque, uma fila de duas extremidades. Ele suporta operações eficientes de adicionar e remover na frente e atrás:

```
deque.push_front(value)
```

Adiciona um valor na frente da fila.

```
deque.push_back(value)
```

Adiciona um valor no final. (Esse método é utilizado muito mais do que `.push_front()`, porque a convenção usual para filas é que os valores são adicionados na parte de trás e removidos na frente, como pessoas esperando em uma fila.)

```
deque.pop_front()
```

Remove e retorna o valor da frente da fila, retornando um `Option<T>` que é `None` se a fila estiver vazia, como `vec.pop()`.

```
deque.pop_back()
```

Remove e retorna o valor na parte de trás, retornando novamente um `Option<T>`.

```
deque.front(), deque.back()
```

Funciona como `vec.first()` e `vec.last()`. Retornam uma referência ao elemento no início ou no fim da fila. O valor de retorno é um `Option<&T>` que é `None` se a fila estiver vazia.

`deque.front_mut()`, `deque.back_mut()`

Funciona como `vec.first_mut()` e `vec.last_mut()`, retornando `Option<&mut T>`.

A implementação de `VecDeque` é um buffer circular, conforme mostrado na Figura 16.3.

Como um `Vec`, possui uma única alocação de heap onde os elementos são armazenados. Diferente de `Vec`, os dados nem sempre começam no início dessa região e podem “contornar” no final, conforme mostrado. Os elementos desse deque, em ordem, são ['A', 'B', 'C', 'D', 'E']. `VecDeque` tem campos privados, rotulados `start` e `stop` na figura, que ele utiliza para lembrar onde no buffer os dados começam e terminam.

Adicionar um valor à fila, em qualquer extremidade, significa reivindicar um dos slots não utilizados, ilustrados como os blocos mais escuros, contornando ou alocando um pedaço maior de memória, se necessário.

`VecDeque` gerencia o contorno, para que você não precise pensar nisso. A Figura 16.3 é uma visão dos bastidores de como Rust torna `.pop_front()` velozes.

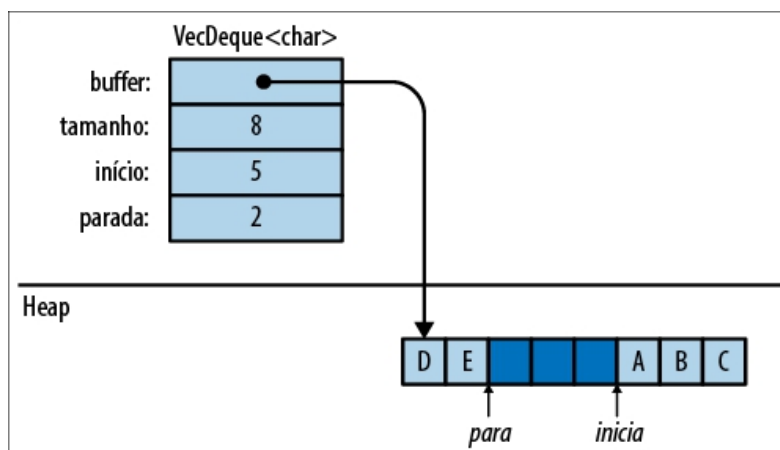


Figura 16.3: Como um `VecDeque` está armazenado na memória.

Muitas vezes, quando você precisa de um deque, `.push_back()` e `.pop_front()` são os dois únicos métodos necessários. As funções associadas a tipos `VecDeque::new()` e `VecDeque::with_capacity(n)`, para criar filas, são como suas contrapartes em `Vec`. Vários métodos `Vec` também são implementados para `VecDeque`: `.len()` e `.is_empty()`,

`.insert(index, value)`, `.remove(index)`, `.extend(iterable)` e assim por diante.

Dequeues, como vetores, podem ser iterados por valor, por referência compartilhada ou por referência mut. Eles têm os três métodos iteradores `.into_iter()`, `.iter()` e `.iter_mut()`. Podem ser indexados da maneira usual: `deque[index]`.

Como os dequeues não armazenam seus elementos continuamente na memória, não podem herdar todos os métodos das fatias. Mas, se você estiver disposto a pagar o custo de mudar o conteúdo, `VecDeque` fornece um método que corrigirá isso:

```
deque.make_contiguous()
```

Pega `&mut self` e reorganiza o `VecDeque` na memória contígua, retornando `&mut [T]`.

`VecS` e `VecDequeS` estão intimamente relacionados, e a biblioteca padrão fornece duas implementações de traits para converter facilmente entre as duas:

```
Vec::from(deque)
```

`Vec<T>` implementa `From<VecDeque<T>>`, então isso transforma um deque em um vetor. Isso custa um tempo $O(n)$, pois pode exigir a reorganização dos elementos.

```
VecDeque::from(vec)
```

`VecDeque<T>` implementa `From<Vec<T>>`, então isso transforma um vetor em um deque. Isso também é $O(n)$, mas geralmente é rápido, mesmo se o vetor for grande, porque a alocação de heap do vetor pode simplesmente ser movida para o novo deque.

Esse método facilita a criação de um deque com elementos especificados, mesmo que não haja um macro padrão `vec_deque![]`:

```
use std::collections::VecDeque;
```

```
let v = VecDeque::from(vec![1, 2, 3, 4]);
```

BinaryHeap<T>

Um `BinaryHeap` é uma coleção cujos elementos são mantidos frouxamente organizados para que o maior valor sempre apareça na frente da fila. Eis os três métodos mais utilizados de `BinaryHeap`:

```
heap.push(value)
```

Adiciona um valor ao heap.

```
heap.pop()
```

Remove e retorna o maior valor do heap. Retorna um `Option<T>` que é `None` se o heap estiver vazio.

```
heap.peek()
```

Retorna uma referência ao maior valor no heap. O tipo de retorno é `Option<&T>`.

```
heap.peek_mut()
```

Retorna um `PeekMut<T>`, que atua como uma referência mutável ao maior valor no heap e fornece a função associada ao tipo `pop()` para remover esse valor do heap. Utilizando esse método, podemos escolher remover ou não remover do heap com base no valor máximo:

```
use std::collections::binary_heap::PeekMut;
```

```
if let Some(top) = heap.peek_mut() {  
    if *top > 10 {  
        PeekMut::pop(top);  
    }  
}
```

`BinaryHeap` também suporta um subconjunto dos métodos em `Vec`, incluindo `BinaryHeap::new()`, `.len()`, `.is_empty()`, `.capacity()`, `.clear()` e `.append(&mut heap2)`.

Por exemplo, suponha que preenchamos um `BinaryHeap` com um monte de números:

```
use std::collections::BinaryHeap;
```

```
let mut heap = BinaryHeap::from(vec![2, 3, 8, 6, 9, 5, 4]);
```

O valor 9 está na parte superior do heap:

```
assert_eq!(heap.peek(), Some(&9));  
assert_eq!(heap.pop(), Some(9));
```

Remover o valor 9 também reorganiza ligeiramente os outros elementos de tal modo que 8 agora está na frente, e assim por diante:

```
assert_eq!(heap.pop(), Some(8));  
assert_eq!(heap.pop(), Some(6));  
assert_eq!(heap.pop(), Some(5));
```

...

É claro, `BinaryHeap` não se limita a números. Ele pode conter qualquer tipo de valor que implemente o `Ord` interno.

Isso torna `BinaryHeap` útil como uma fila de trabalho. Você pode definir um struct de tarefa que implementa `Ord` com base na prioridade para que as tarefas de maior prioridade tenham precedência (`Greater`) sobre as tarefas de menor prioridade. Em seguida, crie um `BinaryHeap` para manter todas as tarefas pendentes. Seu método `.pop()` sempre retornará o item mais importante, a tarefa na qual seu programa deve trabalhar a seguir.

Observação: `BinaryHeap` é iterável e tem um método `.iter()`, mas os iteradores produzem os elementos do heap em uma ordem arbitrária, não do maior para o menor. Para consumir valores de um `BinaryHeap` em ordem de prioridade, utilize um loop `while`:

```
while let Some(task) = heap.pop() {  
    handle(task);  
}
```

HashMap<K, V> e BTreeMap<K, V>

Um *mapa* é uma coleção de pares chave-valor (chamados *entradas*). Não há duas entradas com a mesma chave, e as entradas são mantidas organizadas para que, se você tiver uma chave, possa procurar com eficiência o valor correspondente em um mapa. Resumindo, um mapa é uma tabela de consulta (lookup table).

O Rust oferece dois tipos de mapa: `HashMap<K, V>` e `BTreeMap<K, V>`. Os dois compartilham muitos dos mesmos métodos; a diferença está em como os dois mantêm as entradas organizadas para pesquisa rápida.

Um `HashMap` armazena as chaves e valores em uma tabela hash, por isso requer um tipo de chave `K` que implementa `Hash` e `Eq`, os traits padrão para hashing e igualdade.

A Figura 16.4 mostra como um `HashMap` está organizado na memória. As regiões mais escuras não são utilizadas. Todas as chaves, valores e códigos hash em cache são armazenados em uma única tabela

alocada em heap. Adicionar entradas eventualmente força o `HashMap` a alocar uma tabela maior e mover todos os dados para ela.

Um `BTreeMap` armazena as entradas em ordem por chave, dentro de uma estrutura de árvore, por isso requer um tipo de chave κ que implementa `Ord`. A Figura 16.5 mostra uma `BTreeMap`. Novamente, as regiões mais escuras são capacidade sobressalente não utilizada.

Um `BTreeMap` armazena suas entradas em nós. A maioria dos nós em um `BTreeMap` contém apenas pares chave-valor. Os nós não folha (com nós filhos), como o nó raiz mostrado nesta figura, também têm espaço para ponteiros para os nós filhos. O ponteiro entre (20, 'q') e (30, 'r') aponta para um nó filho contendo chaves entre 20 e 30. A adição de entradas geralmente requer deslocar algumas das entradas existentes de um nó para a direita, para mantê-las ordenadas e, ocasionalmente, envolve a alocação de novos nós.

Essa imagem é um pouco simplificada para caber na página. Por exemplo, os nós reais têm espaço para 11 entradas, não 4.

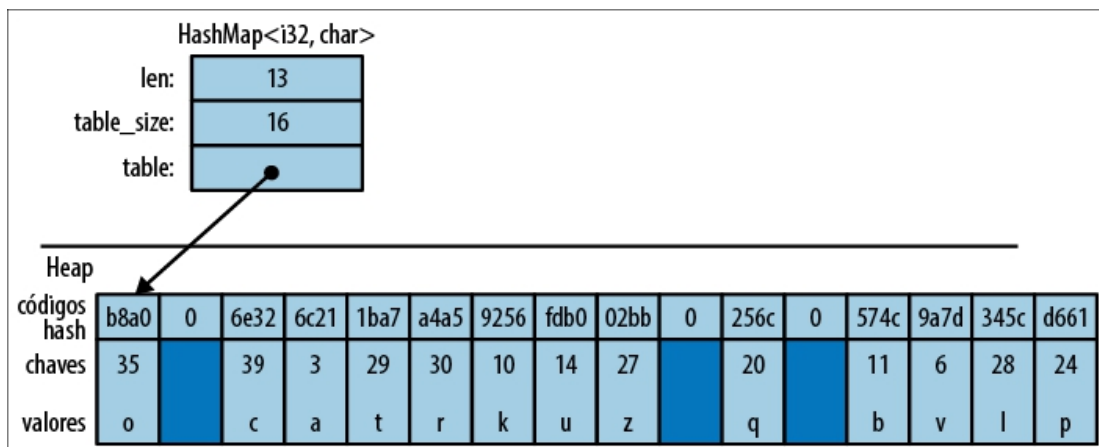


Figura 16.4: Um HashMap na memória.

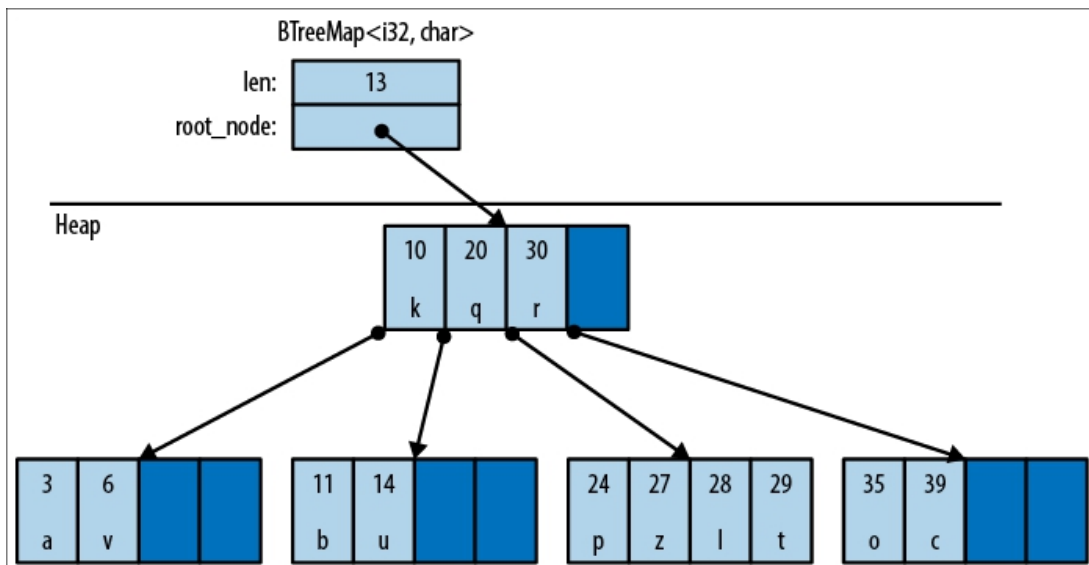


Figura 16.5: Um BTreeMap na memória.

A biblioteca padrão do Rust utiliza árvores B (B-Trees) em vez de árvores binárias balanceadas porque as árvores B são mais rápidas em hardware moderno. Uma árvore binária pode utilizar menos comparações por pesquisa do que uma árvore B, mas pesquisar uma árvore B tem melhor localização – ou seja, os acessos à memória são agrupados em vez de espalhados por todo o heap. Isso torna as falhas de cache da CPU mais raras. É um aumento de velocidade significativo.

Existem várias maneiras de criar um mapa:

`HashMap::new()`, `BTreeMap::new()`

Cria novos mapas vazios.

`iter.collect()`

Pode ser utilizado para criar e preencher um novo `HashMap` ou `BTreeMap` a partir de pares chave-valor. `iter` deve ser um `Iterator<Item=(K, V)>`.

`HashMap::with_capacity(n)`

Cria um novo mapa de hash vazio com espaço para pelo menos `n` entradas. `HashMaps`, como vetores, armazenam seus dados em uma única alocação de heap; portanto, têm uma capacidade e os métodos relacionados `hash_map.capacity()`, `hash_map.reserve(additional)` e `hash_map.shrink_to_fit()`. `BTreeMap` não têm.

HashMap e BTreeMaps têm os mesmos métodos principais para trabalhar com chaves e valores:

`map.len()`

Retorna o número de entradas.

`map.is_empty()`

Retorna `true` se `map` não tiver entradas.

`map.contains_key(&key)`

Retorna `true` se o mapa tiver uma entrada para a chave `key`.

`map.get(&key)`

Pesquisa `map` à procura de uma entrada com a chave `key`. Se uma entrada correspondente for encontrada, retornará `Some(r)`, em que `r` é uma referência ao valor correspondente. Caso contrário, retorna `None`.

`map.get_mut(&key)`

Semelhante, mas retorna uma referência `mut` ao valor.

Em geral, os mapas permitem que você tenha acesso `mut` aos valores armazenados dentro deles, mas não às chaves. Os valores são seus para modificar como quiser. As chaves pertencem ao próprio mapa; este precisa garantir que as chaves não mudem, porque as entradas são organizadas por suas chaves. Modificar uma chave no local seria um bug.

`map.insert(key, value)`

Insere a entrada `(key, value)` em `map` e retorna o valor antigo, se houver. O tipo de retorno é `Option<V>`. Se já houver uma entrada para `key` no mapa, o recém-inserido `value` substitui o antigo.

`map.extend(iterable)`

Itera sobre os itens `(K, V)` de `iterable` e insere cada um desses pares de chave-valor em `map`.

`map.append(&mut map2)`

Mova todas as entradas de `map2` em `map`. Depois disso, `map2` está vazio.

`map.remove(&key)`

Localiza e remove qualquer entrada com a chave `key` a partir de `map`, retornando o valor removido, se houver. O tipo de retorno é

`Option<V>.`

`map.remove_entry(&key)`

Localiza e remove qualquer entrada com a chave `key` a partir de `map`, retornando a chave e o valor removidos, se houver. O tipo de retorno é `Option<(K, V)>`.

`map.retain(test)`

Remove todos os elementos que não passam no teste fornecido. O argumento `test` é uma função ou closure que implementa `FnMut(&K, &mut V) -> bool`. Para cada elemento de `map`, chama `test(&key, &mut value)` e, se retornar `false`, o elemento é removido do mapa e dropado.

Exceto pelo desempenho, é como escrever:

```
map = map.into_iter().filter(test).collect();
```

`map.clear()`

Remove todas as entradas.

Um mapa também pode ser consultado utilizando colchetes: `map[&key]`. Ou seja, os mapas implementam o `trait Index` interno. Entretanto, isso gera um pânico se ainda não houver uma entrada para o determinado `key`, como um acesso de array fora dos limites; portanto, utilize essa sintaxe somente se a entrada que você está procurando for preenchida com certeza.

O argumento `key` para `.contains_key()`, `.get()`, `.get_mut()` e `.remove()` não precisa ser do tipo `&K` exato. Esses métodos são tipos genéricos que podem ser emprestados de `K`. É OK chamar `fish_map.contains_key("conger")` sobre um `HashMap<String, Fish>`, ainda que `"conger"` não seja exatamente uma `String`, porque `String` implementa `Borrow<&str>`. Para detalhes, consulte “Borrow e BorrowMut”, na página 369.

Como um `BTreeMap<K, V>` mantém suas entradas ordenadas por chave, ele suporta uma operação adicional:

`btree_map.split_off(&key)`

Divide `btree_map` em dois. Entradas com chaves menores que `key` são deixadas em `btree_map`. Retorna um novo `BTreeMap<K, V>` contendo as outras entradas.

Entradas

Ambos `HashMap` e `BTreeMap` têm um tipo `Entry` correspondente. O objetivo das entradas é eliminar pesquisas de mapa redundantes. Por exemplo, eis um código para obter ou criar um registro de aluno:

```
// Já temos um registro para esse aluno?
if !student_map.contains_key(name) {
    // Não: crie um
    student_map.insert(name.to_string(), Student::new());
}
// Agora um registro definitivamente existe
let record = student_map.get_mut(name).unwrap();
...
```

Isso funciona bem, mas acessa `student_map` duas ou três vezes, fazendo sempre a mesma pesquisa.

A ideia com as entradas é que façamos a pesquisa apenas uma vez, produzindo um valor `Entry` que é então utilizado para todas as operações subsequentes. Esse código de uma única linha é equivalente a todo o código anterior, exceto que faz a pesquisa apenas uma vez:

```
let record = student_map.entry(name.to_string()).or_insert_with(Student::new);
```

O valor `Entry` retornado por `student_map.entry(name.to_string())` age como uma referência mutável a um local dentro do mapa que está ocupado por um par chave-valor ou vago, o que significa que ainda não há entrada lá. Se estiver vago, o método `.or_insert_with()` da entrada insere um novo `Student`. A maioria dos usos de entradas é assim: curto e eficaz.

Todos os valores `Entry` são criados pelo mesmo método:

`map.entry(key)`

Retorna uma `Entry` para a dada `key`. Se não houver tal chave no mapa, isso retornará uma `Entry` vaga.

Esse método pega seu argumento `self` por referência `mut` e retorna uma `Entry` com um tempo de vida correspondente:

```
pub fn entry<'a>(&'a mut self, key: K) -> Entry<'a, K, V>
```

O tipo `Entry` tem um parâmetro de tempo de vida `'a` porque é efetivamente um tipo sofisticado de referência `mut` emprestada ao mapa. Contanto que `Entry` exista, o método tem acesso exclusivo ao mapa.

De volta a “Structs contendo referências”, na página 147, vimos como armazenar referências em um tipo e como isso afeta os tempos de vida. Agora estamos vendo como isso se parece da perspectiva do usuário. Isso é o que está acontecendo com `Entry`.

Infelizmente, não é possível passar uma referência do tipo `&str` a esse método se o mapa tiver chaves `String`. O método `.entry()`, nesse caso, requer uma `String` real.

Valores de `Entry` fornecem três métodos para lidar com entradas vagas:

```
map.entry(key).or_insert(value)
```

Assegura que `map` contém uma entrada com a dada `key`, inserindo uma nova entrada com o dado `value` se necessário. Ele retorna uma referência `mut` ao valor novo ou existente.

Suponha que precisamos contar votos. Podemos escrever:

```
let mut vote_counts: HashMap<String, usize> = HashMap::new();
for name in ballots {
    let count = vote_counts.entry(name).or_insert(0);
    *count += 1;
}
```

`.or_insert()` retorna uma referência `mut`, então o tipo de `count` é `&mut usize`.

```
map.entry(key).or_default()
```

Assegura que `map` contém uma entrada com a chave fornecida, inserindo uma nova entrada com o valor retornado por `Default::default()` se necessário. Isso só funciona para tipos que implementam `Default`. Como `or_insert`, esse método retorna uma referência `mut` ao valor novo ou existente.

```
map.entry(key).or_insert_with(default_fn)
```

Isso é o mesmo, exceto que, se precisar criar uma nova entrada, chama `default_fn()` para produzir o valor padrão. Se já houver uma entrada para `key` no `map`, então `default_fn` não é utilizado.

Suponha que queremos saber quais palavras aparecem em quais arquivos. Podemos escrever:

```
// Esse mapa contém, para cada palavra, o conjunto de arquivos em que ela aparece
let mut word_occurrence: HashMap<String, HashSet<String>> =
    HashMap::new();
```

```

for file in files {
  for word in read_words(file)? {
    let set = word_occurrence
      .entry(word)
      .or_insert_with(HashSet::new);
    set.insert(file.clone());
  }
}

```

Entry também fornece uma maneira conveniente de modificar apenas os campos existentes.

```
map.entry(key).and_modify(closure)
```

Chama `closure` se uma entrada com a chave `key` existir, passando uma referência mutável ao valor. Retorna a `Entry`, para que seja encadeada com outros métodos.

Por exemplo, poderíamos utilizar isso para contar o número de ocorrências de palavras em uma string:

```

// Esse mapa contém todas as palavras em uma determinada string,
// com o número de vezes que ocorrem
let mut word_frequency: HashMap<&str, u32> = HashMap::new();
for c in text.split_whitespace() {
  word_frequency.entry(c)
    .and_modify(|count| *count += 1)
    .or_insert(1);
}

```

O tipo `Entry` é um enum, definido assim para `HashMap` (e da mesma forma para `BTreeMap`):

```

// (in std::collections::hash_map)
pub enum Entry<'a, K, V> {
  Occupied(OccupiedEntry<'a, K, V>),
  Vacant(VacantEntry<'a, K, V>)
}

```

Os tipos `OccupiedEntry` e `VacantEntry` têm métodos para inserir, remover e acessar entradas sem repetir a pesquisa inicial. Você pode encontrá-los na documentação on-line. Os métodos extras podem ocasionalmente ser utilizados para eliminar uma ou duas pesquisas redundantes, mas `.or_insert()` e `.or_insert_with()` cobrem os casos comuns.

Iteração em mapas

Há várias maneiras de iterar por um mapa:

- Iterar por valor (`for (k, v) in map`) produz pares (K, V) . Isso consome o mapa.
- Iterar por uma referência compartilhada (`for (k, v) in &map`) produz pares $(\&K, \&V)$.
- Iterar por uma referência `mut` (`for (k, v) in &mut map`) produz pares $(\&K, \&mut V)$. (Novamente, não há como obter acesso `mut` às chaves armazenadas em um mapa, pois as entradas são organizadas por suas chaves.)

Assim como os vetores, os mapas têm métodos `.iter()` e `.iter_mut()` que retornam iteradores por referência, exatamente como iterar por `&map` ou `&mut map`. Além disso:

`map.keys()`

Retorna um iterador apenas sobre as chaves, por referência.

`map.values()`

Retorna um iterador sobre os valores, por referência.

`map.values_mut()`

Retorna um iterador sobre os valores, por referência `mut`.

`map.into_iter()`, `map.into_keys()`, `map.into_values()`

Consome o mapa, retornando um iterador sobre tuplas (K, V) de chaves e valores, chaves, ou valores, respectivamente.

Todos os iteradores `HashMap` visitam as entradas do mapa em uma ordem arbitrária. Os iteradores de `BTreeMap` os visitam em ordem por chave.

HashSet<T> e BTreeSet<T>

Conjuntos são coleções de valores organizados para testes rápidos de associação:

```
let b1 = large_vector.contains(&"needle"); // lento, verifica todos os elementos
let b2 = large_hash_set.contains(&"needle"); // rápido, pesquisa em hash
```

Um conjunto nunca contém várias cópias do mesmo valor.

Mapas e conjuntos têm métodos diferentes, mas, nos bastidores, um conjunto é como um mapa com apenas chaves, em vez de pares chave-valor. Na verdade, os dois tipos de conjunto do Rust,

`HashSet<T>` e `BTreeSet<T>`, são implementados como encapsuladores simples em torno de `HashMap<T, ()>` e `BTreeMap<T, ()>`.

`HashSet::new()`, `BTreeSet::new()`

Cria novos conjuntos.

`iter.collect()`

Pode ser utilizado para criar um novo conjunto a partir de qualquer iterador. Se `iter` produz qualquer valor mais de uma vez, as duplicatas são dropadas.

`HashSet::with_capacity(n)`

Cria um `HashSet` vazio com espaço para pelo menos `n` valores.

`HashSet<T>` e `BTreeSet<T>` têm todos os métodos básicos em comum:

`set.len()`

Retorna o número de valores em `set`.

`set.is_empty()`

Retorna `true` se o conjunto não contém elementos.

`set.contains(&value)`

Retorna `true` se o conjunto contém o dado `value`.

`set.insert(value)`

Adiciona um `value` ao conjunto. Retorna `true` se um valor foi adicionado, `false` se já era um membro do conjunto.

`set.remove(&value)`

Remove um `value` do conjunto. Retorna `true` se um valor foi removido, `false` se já não era um membro do conjunto.

`set.retain(test)`

Remove todos os elementos que não passam no teste fornecido. O argumento `test` é uma função ou closure que implementa `FnMut(&T) -> bool`. Para cada elemento de `set`, isso chama `test(&value)` e, se retornar `false`, o elemento é removido do conjunto e dropado.

Exceto pelo desempenho, é como escrever:

```
set = set.into_iter().filter(test).collect();
```

Assim como nos mapas, os métodos que procuram um valor por referência são tipos genéricos que podem ser emprestados de `T`. Para detalhes, consulte “Borrow e BorrowMut”, na página 369.

Iteração em conjuntos

Há duas maneiras de iterar por conjuntos:

- Iterar por valor ("for v in set") produz os membros do conjunto (e consome o conjunto).
- Iterar por referência compartilhada ("for v in &set") produz referências compartilhadas aos membros do conjunto.

A iteração de um conjunto por referência `mut` não é suportada. Não há como obter uma referência `mut` a um valor armazenado em um conjunto.

`set.iter()`

Retorna um iterador sobre os membros de `set` por referência.

Iteradores de `HashSet`, como iteradores de `HashMap`, produzem seus valores em uma ordem arbitrária. Iteradores de `BTreeSet` produzem valores em ordem, como um vetor ordenado.

Quando valores iguais são diferentes

Os conjuntos têm alguns métodos que você precisa utilizar somente se se preocupa com as diferenças entre valores "iguais".

Tais diferenças muitas vezes existem. Dois valores `String` idênticos, por exemplo, armazenam seus caracteres em diferentes posições na memória:

```
let s1 = "hello".to_string();
let s2 = "hello".to_string();
println!("{:p}", &s1 as &str); // 0x7f8b32060008
println!("{:p}", &s2 as &str); // 0x7f8b32060010
```

Normalmente, não nos importamos.

Mas, caso você se importe com isso, pode obter acesso aos valores reais armazenados em um conjunto utilizando os métodos a seguir. Cada um retorna um `Option` que é `None` se `set` não contiver um valor correspondente:

`set.get(&value)`

Retorna uma referência compartilhada ao membro de `set` que é igual a `value`, caso existam. Retorna uma `Option<T>`.

`set.take(&value)`

Como `set.remove(&value)`, mas retorna o valor removido, se houver. Retorna uma `Option<T>`.

`set.replace(value)`

Como `set.insert(value)`, mas, se `set` já contém um valor igual a `value`, substitui e retorna o valor antigo. Retorna uma `Option<T>`.

Operações sobre o conjunto inteiro

Até agora, a maioria dos métodos de conjunto que vimos é focada em um único valor em um único conjunto. Conjuntos também possuem métodos que operam em conjuntos inteiros:

`set1.intersection(&set2)`

Retorna um iterador sobre todos os valores que estão tanto em `set1` como em `set2`.

Por exemplo, se quisermos imprimir os nomes de todos os alunos que estão fazendo aulas de cirurgia cerebral e ciência espacial, poderíamos escrever:

```
for student in &brain_class {
    if rocket_class.contains(student) {
        println!("{}", student);
    }
}
```

Ou, mais curto:

```
for student in brain_class.intersection(&rocket_class) {
    println!("{}", student);
}
```

Surpreendentemente, existe um operador para isso.

`&set1 & &set2` retorna um novo conjunto que é a interseção de `set1` e `set2`. Esse é o operador AND bit a bit binário, aplicado a duas referências. Ele encontra valores que estão tanto em `set1` como em `set2`:

```
let overachievers = &brain_class & &rocket_class;
```

`set1.union(&set2)`

Retorna um iterador sobre valores que estão em `set1` ou `set2`, ou ambos.

`&set1 | &set2` retorna um novo conjunto contendo todos esses valores. Contendo valores que estão em `set1` ou em `set2`.

`set1.difference(&set2)`

Retorna um iterador sobre valores que estão em `set1`, mas não em `set2`.

`&set1 - &set2` retorna um novo conjunto contendo todos esses valores.

`set1.symmetric_difference(&set2)`

Retorna um iterador sobre valores que estão em `set1` ou `set2`, mas não ambos.

`&set1 ^ &set2` retorna um novo conjunto contendo todos esses valores.

E há três métodos para testar relacionamentos entre conjuntos:

`set1.is_disjoint(set2)`

Verdadeiro se `set1` e `set2` não têm valores em comum – a interseção entre eles é vazia.

`set1.is_subset(set2)`

Verdadeiro se `set1` é um subconjunto de `set2` – isto é, todos os valores em `set1` também estão em `set2`.

`set1.is_superset(set2)`

Esse é o inverso: é verdadeiro se `set1` é um superconjunto de `set2`.

Conjuntos também suportam testes de igualdade com `==` e `!=`; dois conjuntos são iguais se contêm os mesmos valores (elementos).

Hash

`std::hash::Hash` é o trait de biblioteca padrão para tipos hasheáveis. Chaves `HashMap` e elementos `HashSet` devem implementar tanto `Hash` como `Eq`.

A maioria dos tipos internos que implementam `Eq` também implementa `Hash`. Os tipos inteiros, `char` e `String` são todos hasheáveis; assim como tuplas, arrays, fatias e vetores, desde que seus elementos sejam passíveis de hash.

Um princípio da biblioteca padrão é que um valor deve ter o mesmo código hash independentemente de onde você o armazene ou de como você o aponte. Portanto, uma referência tem o mesmo código hash do valor que ela referencia e um `Box` tem o mesmo código hash

que o valor empacotado. Um vetor `vec` tem o mesmo código hash da fatia que contém todos os seus dados, `&vec[..]`. Uma `String` tem o mesmo código hash que uma `&str` com os mesmos caracteres.

Estruturas e enums não implementam `Hash` por padrão, mas uma implementação pode ser derivada:

```
/// O número de identificação de um objeto na coleção do Museu Britânico
#[derive(Clone, PartialEq, Eq, Hash)]
enum MuseumNumber {
    ...
}
```

Isso funciona desde que os campos do tipo sejam todos hasheáveis.

Se você implementar `PartialEq` manualmente para um tipo, também deve implementar `Hash` à mão. Por exemplo, suponha que temos um tipo que representa tesouros históricos inestimáveis:

```
struct Artifact {
    id: MuseumNumber,
    name: String,
    cultures: Vec<Culture>,
    date: RoughTime,
    ...
}
```

Dois `Artifact`s são considerados iguais se tiverem o mesmo ID:

```
impl PartialEq for Artifact {
    fn eq(&self, other: &Artifact) -> bool {
        self.id == other.id
    }
}

impl Eq for Artifact {}
```

Como comparamos artefatos puramente com base em seu ID, devemos fazer o hash deles da mesma maneira:

```
use std::hash::{Hash, Hasher};

impl Hash for Artifact {
    fn hash<H: Hasher>(&self, hasher: &mut H) {
        // Delega o hashing para o MuseumNumber
        self.id.hash(hasher);
    }
}
```

(Caso contrário, `HashSet<Artifact>` não funcionaria corretamente; como em todas as tabelas hash, ela requer que `hash(a) == hash(b)` se `a == b`.)

Isso nos permite criar um `HashSet` de `Artifacts`:

```
let mut collection = HashSet::<Artifact>::new();
```

Como mostra esse código, mesmo quando implementamos `Hash` manualmente, você não precisa saber nada sobre algoritmos hash. `.hash()` recebe uma referência a um `Hasher`, que representa o algoritmo de hash. Você simplesmente alimenta esse `Hasher` com todos os dados relevantes para o operador `==`. O `Hasher` calcula um código hash para qualquer coisa que você fornecer.

Usando um algoritmo de hash personalizado

O método `hash` é genérico, então as implementações de `Hash` mostradas anteriormente podem alimentar dados para qualquer tipo que implemente `Hasher`. É assim que o Rust suporta algoritmos de hash novos.

Um terceiro trait, `std::hash::BuildHasher`, é o trait para tipos que representam o estado inicial de um algoritmo de hash. Cada `Hasher` é de uso único, como um iterador: você utiliza uma vez e joga fora. Um `BuildHasher` é reutilizável.

Cada `HashMap` contém um `BuildHasher` que ele utiliza cada vez que precisa calcular um código hash. O valor `BuildHasher` contém a chave, estado inicial ou outros parâmetros que o algoritmo hash precisa toda vez que é executado.

O protocolo completo para calcular um código hash é assim:

```
use std::hash::{Hash, Hasher, BuildHasher};

fn compute_hash<B, T>(builder: &B, value: &T) -> u64
    where B: BuildHasher, T: Hash
{
    let mut hasher = builder.build_hasher(); // 1. inicia o algoritmo
    value.hash(&mut hasher);                // 2. alimenta-o com dados
    hasher.finish()                          // 3. finaliza, produzindo um u64
}
```

`HashMap` chama esses três métodos toda vez que precisa calcular um código hash. Todos os métodos são passíveis de serem colocados inline, por isso são muito rápidos.

O algoritmo de hash padrão do Rust é um algoritmo conhecido chamado SipHash-1-3. SipHash é rápido e é muito bom para minimizar colisões de hash. Na verdade, é um algoritmo criptográfico: não há nenhuma maneira eficiente conhecida de gerar colisões SipHash-1-3. Contanto que uma chave diferente e imprevisível seja utilizada para cada tabela hash, o Rust é seguro contra um tipo de ataque de negação de serviço chamado HashDoS, em que os invasores utilizam deliberadamente colisões de hash para acionar o pior desempenho em um servidor.

Mas talvez você não precise disso para sua aplicação. Se você estiver armazenando muitas chaves pequenas, como números inteiros ou strings muito curtas, é possível implementar uma função de hash mais rápida, à custa da segurança do HashDoS. O crate `fnv` implementa um desses algoritmos, o hash Fowler-Noll-Vo (FNV). Para experimentá-lo, adicione esta linha ao seu arquivo *Cargo.toml*:

```
[dependencies]
fnv = "1.0"
```

Em seguida, importe o mapa e defina os tipos de `fnv`:

```
use fnv::{FnvHashMap, FnvHashSet};
```

Você pode utilizar esses dois tipos como substitutos drop-in para `HashMap` e `HashSet`. Uma espiada dentro do código-fonte de `fnv` revela como eles são definidos:

```
/// Um `HashMap` utilizando um hasher FNV padrão
pub type FnvHashMap<K, V> = HashMap<K, V, FnvBuildHasher>;
```

```
/// Um `HashSet` utilizando um hasher FNV padrão
pub type FnvHashSet<T> = HashSet<T, FnvBuildHasher>;
```

As coleções `HashMap` e `HashSet` padrão aceitam um parâmetro de tipo extra opcional que especifica o algoritmo de hashing; `FnvHashMap` e `FnvHashSet` são aliases de tipo genérico para `HashMap` e `HashSet`, especificando um hasher FNV para esse parâmetro.

Além das coleções padrão

A criação de um novo tipo de coleção personalizada no Rust é praticamente a mesma que em qualquer outra linguagem. Você organiza os dados combinando as partes fornecidas pela linguagem:

structs e enums, coleções padrão, Options, Boxes e assim por diante. Para obter um exemplo, consulte o tipo `BinaryTree<T>` definido em “Enums genéricos”, na página 278.

Se está acostumado a implementar estruturas de dados em C++ utilizando ponteiros, gerenciamento manual da memória, `placement new` e chamadas destruidoras explícitas para obter o melhor desempenho possível, sem dúvida você achará o Rust seguro um tanto limitador. Todas essas ferramentas são inerentemente inseguras. Elas estão disponíveis no Rust, mas apenas se você optar pelo código não seguro (`unsafe`). O Capítulo 22 mostra como; ele inclui um exemplo que utiliza algum código não seguro (`unsafe`) para implementar uma coleção personalizada segura.

Por enquanto, vamos aproveitar o conforto das coleções padrão e suas APIs seguras e eficientes. Como grande parte da biblioteca padrão do Rust, elas são projetadas para garantir que a necessidade de escrever `unsafe` seja a mais rara possível.

CAPÍTULO 17

Strings e texto

A string de caracteres é uma estrutura de dados rígida e, em todos os lugares em que ela é passada, há muita duplicação de processo. É um veículo perfeito para esconder informações.

– Alan Perlis, epigrama #34

Estamos utilizando os tipos textuais principais do Rust, `String`, `str` e `char`, ao longo do livro. Em “Tipos de String”, na página 97, descrevemos a sintaxe para caracteres e literais de string e mostramos como as strings são representadas na memória. Neste capítulo, abordamos a manipulação de texto com mais detalhes.

Neste capítulo:

- Fornecemos algumas informações básicas sobre Unicode que devem ajudá-lo a entender o design da biblioteca padrão.
- Descrevemos o tipo `char`, representando um único ponto de código Unicode.
- Descrevemos os tipos `String` e `str`, representando sequências de caracteres Unicode que possuímos ou emprestamos. Eles têm uma ampla variedade de métodos para construir, pesquisar, modificar e iterar por seu conteúdo.
- Abordamos as facilidades de formatação de string do Rust, como as macros `println!` e `format!`. Você pode escrever as próprias macros que funcionam com strings de formatação e estendê-las para suportar os próprios tipos.
- Damos uma visão geral do suporte a expressões regulares do Rust.
- Por fim, falamos sobre a importância da normalização Unicode e mostramos como fazer isso no Rust.

Princípios básicos de Unicode

Este livro é sobre Rust, não Unicode, que já tem livros inteiros dedicados a ele. Mas os tipos de caractere e string do Rust são projetados em torno do Unicode. Eis algumas partes do Unicode que ajudam a explicar o Rust.

ASCII, Latin-1 e Unicode

Unicode e ASCII correspondem a todos os pontos de código¹ ASCII, de 0 a 0x7f: por exemplo, ambos atribuem ao caractere * o ponto de código 42. Da mesma forma, o Unicode atribui 0–0xff aos mesmos caracteres do conjunto de caracteres ISO/IEC 8859-1, um superconjunto de oito bits de ASCII para uso com linguagens da Europa Ocidental. O Unicode chama esse intervalo de pontos de código de *bloco de código Latin-1*, portanto vamos nos referir à ISO/IEC 8859-1 pelo nome mais sugestivo *Latin-1*.

Como o Unicode é um superconjunto do Latin-1, a conversão do Latin-1 para Unicode nem requer uma tabela:

```
fn latin1_to_char(latin1: u8) -> char {  
    latin1 as char  
}
```

A conversão inversa também é trivial, assumindo que os pontos de código caíam no intervalo Latin-1:

```
fn char_to_latin1(c: char) -> Option<u8> {  
    if c as u32 <= 0xff {  
        Some(c as u8)  
    } else {  
        None  
    }  
}
```

UTF-8

Os tipos `String` e `str` do Rust representam texto utilizando a forma de codificação UTF-8. UTF-8 codifica um caractere como uma sequência de um a quatro bytes (Figura 17.1).

Codificação UTF-8 (um a quatro bytes de comprimento)	Ponto de código representado	Intervalo
0 x x x x x x x	0bxxxxxx	0 to 0x7f
1 1 0 x x x x x 1 0 y y y y y y	0bxxxxxyyyyy	0x80 to 0x7ff
1 1 1 0 x x x x 1 0 y y y y y y 1 0 z z z z z z	0bxxxxxyyyyyzzzzz	0x800 to 0xffff
1 1 1 1 0 x x x 1 0 y y y y y y 1 0 z z z z z z 1 0 w w w w w w	0bxxxxxyyyyyzzzzzwwwwww	0x10000 to 0x10ffff

Figura 17.1: Codificação UTF-8.

Há duas restrições em sequências UTF-8 bem formadas. Primeiro, apenas a codificação mais curta para qualquer ponto de código é considerada bem formada; você não pode gastar quatro bytes codificando um ponto de código que caberia em três. Essa regra garante que haja exatamente uma codificação UTF-8 para um determinado ponto de código. Em segundo lugar, um UTF-8 bem formado não deve codificar números de 0xd800 a 0xdfff ou além de 0x10ffff: esses são reservados para fins não caractere ou totalmente fora do intervalo do Unicode.

A Figura 17.2 mostra alguns exemplos.

Codificação UTF-8 (um a quatro bytes de comprimento)	Ponto de código representado	Intervalo
0 0 1 0 1 0 1 0	0b0101010 == 0x2a	'*'
1 1 0 0 1 1 1 0 1 0 1 1 1 1 0 0	0b01110_111100 == 0x3bc	'μ'
1 1 1 0 1 0 0 1 1 0 0 0 1 1 0 0 1 0 0 0 0 1 1 0	0b1001_001100_000110 == 0x9306	'錆' (sabi: rust)
1 1 1 1 0 0 0 0 1 0 0 1 1 1 1 1 1 0 1 0 1 0 0 1 1 0 1 0 0 0 0 0 0 0	0b000_011111_100110_000000 == 0x1f980	'🐡' (emoji caranguejo)

Figura 17.2: Exemplos de UTF-8.

Observe que, embora o emoji de caranguejo tenha uma codificação cujo byte inicial contribui apenas com zeros para o ponto de código, ele ainda precisa de uma codificação de quatro bytes: as codificações UTF-8 de três bytes podem transmitir apenas pontos de código de 16 bits e 0x1f980 tem 17 bits de comprimento.

Eis um exemplo rápido de uma string contendo caracteres com codificações de comprimentos variados:

```
assert_eq!("うどん: udon".as_bytes(),
```



```

&[0xe3, 0x81, 0x86, // う
    0xe3, 0x81, 0xa9, // ど
    0xe3, 0x82, 0x93, // ん
    0x3a, 0x20, 0x75, 0x64, 0x6f, 0x6e // : udon
];

```

A Figura 17.2 também mostra algumas propriedades muito úteis do UTF-8:

- Como o UTF-8 codifica pontos de código 0 a 0x7f como nada mais do que os bytes 0 a 0x7f, um intervalo de bytes contendo texto ASCII é UTF-8 válido. E se uma string de UTF-8 incluir apenas caracteres do ASCII, o inverso também é verdadeiro: a codificação UTF-8 é ASCII válida.

O mesmo não é verdade para Latin-1: por exemplo, Latin-1 codifica é como o byte 0xe9, que o UTF-8 interpretaria como o primeiro byte de uma codificação de três bytes.

- Olhando para os bits superiores de qualquer byte, você pode dizer imediatamente se é o início da codificação UTF-8 de algum caractere ou um byte no meio de um.
- O primeiro byte de uma codificação sozinho informa o comprimento total da codificação, por meio de seus bits iniciais.
- Como nenhuma codificação tem mais de quatro bytes, o processamento UTF-8 nunca requer loops ilimitados, o que é bom ao trabalhar com dados não confiáveis.
- Em UTF-8 bem formado, você sempre pode dizer de forma inequívoca onde as codificações dos caracteres começam e terminam, mesmo se você começar de um ponto arbitrário no meio dos bytes. Os primeiros bytes UTF-8 e os bytes seguintes são sempre distintos, portanto uma codificação não pode começar no meio da outra. O primeiro byte determina o comprimento total da codificação, portanto nenhuma codificação pode ser um prefixo de outra. Isso tem muitas consequências interessantes. Por exemplo, pesquisar uma string UTF-8 para um caractere delimitador ASCII requer apenas uma varredura simples do byte do delimitador. Ele nunca pode aparecer como parte de uma codificação multibyte e, portanto, não há necessidade de rastrear a estrutura UTF-8. Da mesma forma, os algoritmos que procuram

uma string de bytes em outra funcionarão sem modificações em strings UTF-8, embora alguns nem sequer examinem cada byte do texto que está sendo pesquisado.

Embora as codificações de tamanho variável sejam mais complicadas do que as codificações de tamanho fixo, essas características tornam o UTF-8 mais confortável de se trabalhar do que você poderia esperar. A biblioteca padrão lida com a maioria dos aspectos para você.

Direcionalidade do texto

Enquanto textos em latim, cirílico e tailandês são escritos da esquerda para a direita, outros textos como em hebraico e árabe são escritos da direita para a esquerda. O Unicode armazena os caracteres na ordem em que normalmente seriam escritos ou lidos, de modo que os bytes iniciais de uma string contendo, digamos, texto em hebraico codificam o caractere que seria escrito à direita:

```
assert_eq!("ערב טוב".chars().next(), Some('ע'));
```

Caracteres (char)

Um `char` do Rust é um valor de 32 bits contendo um ponto de código Unicode. Garante-se que um `char` caia no intervalo de 0 a 0xd7ff ou no intervalo 0xe000 a 0x10ffff; todos os métodos para criar e manipular os valores garantem que isso seja verdade. O tipo `char` implementa `Copy` e `Clone`, com todos os traits usuais para comparação, hashing e formatação.

Uma fatia de string pode produzir um iterador sobre seus caracteres com `slice.chars()`:

```
assert_eq!("力二".chars().next(), Some('力'));
```

Nas descrições a seguir, a variável `ch` é sempre do tipo `char`.

Classificando caracteres

O tipo `char` tem métodos para classificar caracteres em algumas categorias comuns, conforme listado na Tabela 17.1. Todos eles derivam suas definições do Unicode.

Tabela 17.1: Métodos de classificação para tipo char

Método	Descrição	Exemplos
ch.is_numeric()	Um caractere numérico. Isso inclui as categorias gerais do Unicode "Number; digit" e "Number; letter" mas não "Number; other".	'4'.is_numeric() '1'.is_numeric() '8'.is_numeric()
ch.is_alphabetic()	Um caractere alfabético: Propriedade "Alphabetic" derivada do Unicode.	'q'.is_alphabetic() '七'.is_alphabetic()
ch.is_alphanumeric())	Numérico ou alfabético, conforme definido anteriormente.	'9'.is_alphanumeric() '饅'.is_alphanumeric() '*'.is_alphanumeric()
ch.is_whitespace()	Um caractere de espaço em branco: Propriedade de caractere Unicode "WSpace=Y".	' '.is_whitespace() '\n'.is_whitespace() '\u{A0}'.is_whitespace())
ch.is_control()	Um caractere de controle: Categoria geral "Other, control" do Unicode.	'\n'.is_control() '\u{85}'.is_control()

Um conjunto paralelo de métodos se restringe apenas a ASCII, retornando false para qualquer char não-ASCII (Tabela 17.2).

Tabela 17.2: Métodos de classificação ASCII para char

Método	Descrição	Exemplos
ch.is_ascii()	Um caractere ASCII: aquele cujo ponto de código fica entre 0 e 127 inclusive.	'n'.is_ascii() 'ñ'.is_ascii()
ch.is_ascii_alphabetic()	Uma letra ASCII maiúscula ou minúscula, no intervalo 'A'..'Z' ou 'a'..'z'.	'n'.is_ascii_alphabetic() '1'.is_ascii_alphabetic() 'ñ'.is_ascii_alphabetic()
ch.is_ascii_digit()	Um dígito ASCII, no	'8'.is_ascii_digit() '-'.is_ascii_digit()

Método	Descrição	Exemplos
	intervalo '0'..'9'.	!'@'.is_ascii_digit()
ch.is_ascii_hexdigit()	Qualquer caractere nos intervalos '0'..'9', 'A'..'F' ou 'a'..'f'.	
ch.is_ascii_alphanumeric()	Um dígito ASCII ou letra maiúscula ou minúscula.	'q'.is_ascii_alphanumeric() '0'.is_ascii_alphanumeric()
ch.is_ascii_control()	Um caractere de controle ASCII, incluindo 'DEL'.	'\n'.is_ascii_control() '\x7f'.is_ascii_control()
ch.is_ascii_graphic()	Qualquer caractere ASCII que deixe tinta na página: não um espaço nem um caractere de controle.	'Q'.is_ascii_graphic() '~'.is_ascii_graphic() ' '.is_ascii_graphic()
ch.is_ascii_uppercase(), ch.is_ascii_lowercase()	Letras maiúsculas e minúsculas ASCII.	'z'.is_ascii_lowercase() 'Z'.is_ascii_uppercase()
ch.is_ascii_punctuation()	Qualquer caractere gráfico ASCII que	

Método	Descrição	Exemplos
	não seja alfabético nem um dígito.	
<code>ch.is_ascii_whitespace()</code>	Um caractere de espaço em branco ASCII: um espaço, tabulação horizontal, nova linha, avanço de formulário ou retorno de carro.	<code>' '.is_ascii_whitespace()</code> <code>'\n'.is_ascii_whitespace()</code> <code>!\u{A0}'.is_ascii_whitespace()</code>

Todos os métodos `is_ascii...` também estão disponíveis no tipo `byte u8`:

```
assert!(32u8.is_ascii_whitespace());
assert!(b'9'.is_ascii_digit());
```

Tome cuidado ao utilizar essas funções para implementar uma especificação existente, como um padrão de linguagem de programação ou formato de arquivo, pois as classificações podem diferir de maneiras surpreendentes. Por exemplo, observe que `is_whitespace` e `is_ascii_whitespace` diferem no tratamento de certos caracteres:

```
let line_tab = '\u{000b}'; // 'tabulação de linha' ou 'tabulação vertical'
assert_eq!(line_tab.is_whitespace(), true);
assert_eq!(line_tab.is_ascii_whitespace(), false);
```

A função `char::is_ascii_whitespace` implementa definição de espaço em branco comum a muitos padrões da web, enquanto `char::is_whitespace` segue o padrão Unicode.

Manipulando dígitos

Para lidar com dígitos, você pode utilizar os seguintes métodos:

`ch.to_digit(radix)`

Decide se `ch` é um dígito na base `radix`. Se for, ele retorna `Some(num)`, em que `num` é um `u32`. Caso contrário, retorna `None`. Isso reconhece apenas dígitos ASCII, não a classe mais ampla de caracteres abrangida por `char::is_numeric`. O parâmetro `radix` pode variar de 2 a 36. Para bases maiores que 10, as letras ASCII de qualquer um dos casos são consideradas dígitos com valores de 10 a 35.

`std::char::from_digit(num, radix)`

Uma função que converte o valor `u32` do dígito `num` para um `char` se possível. Se `num` pode ser representado como um único dígito em `radix`, `from_digit` retorna `Some(ch)`, em que `ch` é o dígito. Quando `radix` é maior que 10, `ch` pode ser uma letra minúscula. Caso contrário, retorna `None`.

Esse é o inverso de `to_digit`. Se `std::char::from_digit(num, radix)` é `Some(ch)`, então `ch.to_digit(radix)` é `Some(num)`. Se `ch` é um dígito ASCII ou uma letra minúscula, o inverso também é válido.

`ch.is_digit(radix)`

Retorna `true` se `ch` é um dígito ASCII na base `radix`. Isso é equivalente a `ch.to_digit(radix) != None`.

Assim, por exemplo:

```
assert_eq!('F'.to_digit(16), Some(15));
assert_eq!(std::char::from_digit(15, 16), Some('f'));
assert!(char::is_digit('f', 16));
```

Conversão de maiúsculas e minúsculas para caracteres

Para lidar com maiúsculas e minúsculas:

`ch.is_lowercase()`, `ch.is_uppercase()`

Indicar se `ch` é um caractere alfabético maiúsculo ou minúsculo. Eles seguem as propriedades derivadas de maiúsculas e minúsculas do Unicode, portanto abrangem alfabetos não latinos, como grego e cirílico, e também fornecem os resultados esperados para ASCII.

`ch.to_lowercase()`, `ch.to_uppercase()`

Retorna iteradores que produzem os caracteres dos equivalentes em minúsculas e maiúsculas de `ch`, de acordo com os algoritmos Unicode Default Case Conversion:

```
let mut upper = 's'.to_uppercase();
assert_eq!(upper.next(), Some('S'));
assert_eq!(upper.next(), None);
```

Esses métodos retornam um iterador em vez de um único caractere porque a conversão de maiúsculas e minúsculas em Unicode nem sempre é um processo de um para um:

// A forma maiúscula da letra alemã "Eszett" é "SS"

```
let mut upper = 'ß'.to_uppercase();
assert_eq!(upper.next(), Some('S'));
assert_eq!(upper.next(), Some('S'));
assert_eq!(upper.next(), None);
```

// O Unicode determina que, em Turco, a forma maiúscula da letra
// minúscula 'i' é 'İ' com o ponto diacrítico seguido por ``\u{307}``,
// COMBINING DOT ABOVE (ponto combinado acima), de forma que uma
// conversão subsequente de volta para maiúsculas preserve o ponto

```
let ch = 'i'; // `\u{130}`  
let mut lower = ch.to_lowercase();  
assert_eq!(lower.next(), Some('i'));  
assert_eq!(lower.next(), Some("`u{307}`"));  
assert_eq!(lower.next(), None);
```

Por conveniência, esses iteradores implementam o `trait std::fmt::Display`, para que você possa passá-los diretamente para uma macro `println!` ou `write!`.

Conversões de e para inteiros

O operador `as` do Rust vai converter um `char` para qualquer tipo inteiro, mascarando silenciosamente quaisquer bits superiores:

```
assert_eq!('B' as u32, 66);  
assert_eq!('𩚑' as u8, 66); // bits superiores truncados  
assert_eq!('𐄂' as i8, -116); // idem
```

O operador `as` converte qualquer valor `u8` para um `char`, e `char` também implementa `From<u8>`, mas tipos inteiros mais amplos podem representar pontos de código inválidos, portanto para estes você deve utilizar `std::char::from_u32`, que retorna `Option<char>`:

```
assert_eq!(char::from(66), 'B');  
assert_eq!(std::char::from_u32(0x9942), Some('𩚑'));
```

```
assert_eq!(std::char::from_u32(0xd800), None); // reservado para UTF-16
```

String e str

Os tipos `String` e `str` do Rust têm a garantia de conter apenas UTF-8 bem formado. A biblioteca garante isso restringindo as formas que você pode criar valores `String` e `str` e as operações que você pode executar neles, de modo que os valores sejam bem formados quando introduzidos e permaneçam assim enquanto você trabalha com eles. Todos os seus métodos protegem essa garantia: nenhuma operação segura neles pode introduzir UTF-8 malformado. Isso simplifica o código que funciona com o texto.

O Rust coloca métodos de manipulação de texto em qualquer `str` ou `String` dependendo se o método precisa de um buffer redimensionável ou contenta-se com apenas utilizar o texto no local. Como `String` desreferencia para `&str`, todo método definido em `str` também está disponível diretamente em `String`. Essa seção apresenta métodos de ambos os tipos, agrupados por funcionalidade.

Esses métodos indexam o texto por posição de byte e medem seu comprimento em bytes, em vez de caracteres. Na prática, dada a natureza do Unicode, a indexação por caractere não é tão útil quanto parece e as posições de bytes são mais rápidas e simples. Se você tentar utilizar uma posição de byte que caia no meio da codificação UTF-8 de algum caractere, o método gera um pânico, portanto você não pode introduzir UTF-8 malformado dessa maneira.

Uma `String` é implementada como uma capa em torno de um `Vec<u8>`, o que garante que o conteúdo do vetor seja sempre UTF-8 bem formado. O Rust nunca mudará `String` para utilizar uma representação mais complicada, então você pode assumir que `String` compartilha características de desempenho de `Vec`.

Nessas explicações, as variáveis têm os tipos dados na Tabela 17.3.

Tabela 17.3: Tipos de variáveis utilizadas nas explicações

Variável	Tipo presumido
<code>string</code>	<code>String</code>

Variável	Tipo presumido
slice	&str ou algo que não referencia uma, como String ou Rc<String>
ch	char
n	usize, um comprimento
i, j	usize, um deslocamento de byte
range	Uma variedade de usize deslocamentos de byte, totalmente delimitados como i..j, ou parcialmente delimitados como i.., ..j ou ..
pattern	Qualquer tipo de padrão: char, String, &str, &[char] ou FnMut(char) -> bool

Descrevemos os tipos de padrão em “Padrões para pesquisar texto”, na página 502.

Criando valores string

Existem algumas maneiras comuns de criar valores String:

`String::new()`

Retorna uma string nova e vazia. Ela não tem buffer alocado no heap, mas alocará um conforme necessário.

`String::with_capacity(n)`

Retorna uma string nova e vazia com um buffer pré-alocado para conter pelo menos `n` bytes. Se você souber o comprimento da string que está construindo com antecedência, esse construtor permite obter o buffer corretamente dimensionado desde o início, em vez de redimensionar o buffer à medida que você cria a string. A string ainda aumentará seu buffer conforme necessário se seu comprimento exceder `n` bytes. Assim como os vetores, as strings têm métodos `capacity`, `reserve` e `shrink_to_fit`, mas geralmente a lógica de alocação padrão é boa.

`str_slice.to_string()`

Aloca uma nova String cujo conteúdo é uma cópia de `str_slice`. Temos utilizado expressões como `"literal text".to_string()` ao longo do livro para fazer Strings a partir de literais string.

`iter.collect()`

Constrói uma string concatenando os itens de um iterador, que podem ser valores `char`, `&str` ou `String`. Por exemplo, para remover todos os espaços de uma string, você pode escrever:

```
let spacey = "man hat tan";
let spaceless: String =
    spacey.chars().filter(|c| !c.is_whitespace()).collect();
assert_eq!(spaceless, "manhattan");
```

Utilizando `collect` dessa forma aproveita a implementação do `trait std::iter::FromIterator` por `String`.

`slice.to_owned()`

Retorna uma cópia da fatia como uma `String` recém-alocada. O tipo `str` não pode implementar `Clone`: o `trait` exigiria que sobre uma `&str` retornasse um valor `str`, mas `str` não tem tamanho. No entanto, `&str` implementa `ToOwned`, o que permite que o implementador especifique seu equivalente com posse.

Inspecção simples

Esses métodos obtêm informações básicas de fatias de string:

`slice.len()`

O comprimento do `slice`, em bytes.

`slice.is_empty()`

Verdadeiro se `slice.len() == 0`.

`slice[range]`

Retorna uma fatia emprestando a porção dada de `slice`. Intervalos parcialmente limitados e ilimitados estão OK; por exemplo:

```
let full = "bookkeeping";
assert_eq!(&full[..4], "book");
assert_eq!(&full[5..], "eeping");
assert_eq!(&full[2..4], "ok");
assert_eq!(full[..].len(), 11);
assert_eq!(full[5..].contains("boo"), false);
```

Observe que você não pode indexar uma fatia de string com uma única posição, como `slice[i]`. Buscar um único caractere em um determinado deslocamento de byte é um pouco desajeitado: você deve produzir um iterador de `chars` sobre a fatia e pedir para analisar o UTF-8 de um caractere:

```
let parenthesized = "Rust (餛)";
assert_eq!(parenthesized[6..].chars().next(), Some('餛'));
```

Contudo, você raramente precisa fazer isso. O Rust tem maneiras muito melhores de iterar por fatias, que descrevemos em “Iterando

por texto”, na página 504.

`slice.split_at(i)`

Retorna uma tupla de duas fatias compartilhadas emprestadas de `slice`: a porção até o deslocamento de byte `i` e a parte depois dela.

Em outras palavras, isso retorna `(slice[..i], slice[i..])`.

`slice.is_char_boundary(i)`

Verdadeiro se o deslocamento de bytes `i` cai entre os limites de caracteres e, portanto, é adequado como um deslocamento para `slice`.

Naturalmente, as fatias podem ser comparadas quanto à igualdade, ordenadas e hashadas. A comparação ordenada simplesmente trata a string como uma sequência de pontos de código Unicode e os compara em ordem lexicográfica.

Acrescentando e inserindo texto

Os métodos a seguir adicionam texto a uma `String`:

`string.push(ch)`

Anexa o caractere `ch` à string final.

`string.push_str(slice)`

Anexa o conteúdo completo de `slice`.

`string.extend(iter)`

Anexa os itens produzidos pelo iterador `iter` à string. O iterador pode produzir valores `char`, `str` ou `String`. Essas são implementações de `std::iter::Extend` por `String`:

```
let mut also_spaceless = "con".to_string();
also_spaceless.extend("tri but ion".split_whitespace());
assert_eq!(also_spaceless, "contribution");
```

`string.insert(i, ch)`

Insere o único caractere `ch` no deslocamento de bytes `i` na string. Isso envolve a mudança de quaisquer caracteres após `i` para abrir espaço para `ch`, portanto construir uma string dessa maneira pode exigir tempo quadrático no comprimento da string.

`string.insert_str(i, slice)`

Isso faz o mesmo para `slice`, com a mesma ressalva de desempenho.

`String` implementa `std::fmt::Write`, significado que as macros `write!` e `writeln!` podem anexar texto formatado a `Strings`:

```
use std::fmt::Write;
let mut letter = String::new();
writeln!(letter, "Whose {} these are I think I know", "rutabagas");
writeln!(letter, "His house is in the village though;");
assert_eq!(letter, "Whose rutabagas these are I think I know\n\
His house is in the village though;\n");
```

Como a `write!` e `writeln!` são projetados para escrever em fluxos de saída, eles retornam um `Result`, que o Rust reclama se você ignorar. Esse código utiliza o operador `?` para lidar com isso, mas escrever em uma `String` é realmente infalível, então, nesse caso, chamar `.unwrap()` ficaria bem também.

Como a `String` implementa `Add<&str>` e `AddAssign<&str>`, você pode escrever um código como este:

```
let left = "partners".to_string();
let mut right = "crime".to_string();
assert_eq!(left + " in " + &right, "partners in crime");

right += " doesn't pay";
assert_eq!(right, "crime doesn't pay");
```

Quando aplicado a strings, o operador `+` pega seu operando esquerdo por valor, então ele pode realmente reutilizar essa `String` como resultado da adição. Como consequência, se o buffer do operando esquerdo for grande o suficiente para conter o resultado, nenhuma alocação é necessária.

Em uma infeliz falta de simetria, o operando esquerdo de `+` não pode ser um `&str`, então você não pode escrever:

```
let parenthetical = "(" + string + ");
```

Em vez disso, deve escrever:

```
let parenthetical = "(" + &string + ");
```

No entanto, essa restrição desencoraja a criação de strings do final para trás. Essa abordagem funciona mal porque o texto deve ser deslocado repetidamente para o final do buffer.

Construir strings do começo para o fim anexando pequenos pedaços, porém, é eficiente. Uma `String` se comporta como um vetor, dobrando o tamanho de seu buffer quando precisa de mais capacidade. Isso

mantém o overhead de recopiar proporcional ao tamanho final. Mesmo assim, utilizar `String::with_capacity` para começar, cria strings com o tamanho de buffer correto, evita o redimensionamento e pode reduzir o número de chamadas para o alocador de heap.

Removendo e substituindo texto

`String` tem alguns métodos para remover texto (esses não afetam a capacidade da string; utilize `shrink_to_fit` se você precisar liberar memória):

`string.clear()`

Reinicia `string` como uma string vazia.

`string.truncate(n)`

Descarta todos os caracteres após o deslocamento de byte `n`, deixando `string` com um comprimento de no máximo `n`. Se `string` é mais curto do que `n` bytes, isso não tem efeito.

`string.pop()`

Remove o último caractere de `string`, se houver, e o retorna como um `Option<char>`.

`string.remove(i)`

Remove o caractere no deslocamento de byte `i` de `string` e o retorna, deslocando quaisquer caracteres seguintes para a frente. Isso leva um tempo linear no número de caracteres seguintes.

`string.drain(range)`

Retorna um iterador no intervalo fornecido de índices de bytes e remove os caracteres assim que o iterador é dropado. Os caracteres após o intervalo são deslocados para a frente:

```
let mut choco = "chocolate".to_string();
assert_eq!(choco.drain(3..6).collect::<String>(), "col");
assert_eq!(choco, "choate");
```

Se você quiser apenas remover o intervalo, basta dropar o iterador imediatamente, sem extrair nenhum item dele:

```
let mut winston = "Churchill".to_string();
winston.drain(2..6);
assert_eq!(winston, "Chill");
```

`string.replace_range(range, replacement)`

Substituí o intervalo fornecido em `string` com a fatia de `string` de substituição fornecida. A fatia não precisa ter o mesmo comprimento que o intervalo que está sendo substituído, mas, a menos que o intervalo que está sendo substituído vá até o final de `string`, exigirá mover todos os bytes após o final do intervalo:

```
let mut beverage = "a piña colada".to_string();  
beverage.replace_range(2..7, "kahlua"); // 'ñ' são dois bytes!  
assert_eq!(beverage, "a kahlua colada");
```

Convenções para pesquisa e iteração

As funções da biblioteca padrão do Rust para pesquisar texto e iterar pelo texto seguem algumas convenções de nomenclatura para torná-las mais fáceis de lembrar:

`r`

A maioria das operações processa texto do início ao fim, mas operações com nomes começando com `r` trabalham do fim para o início. Por exemplo, `rsplit` é a versão do fim para o início de `split`. Em alguns casos, mudar de direção pode afetar não apenas a ordem em que os valores são gerados, mas também os próprios valores. Veja o diagrama na Figura 17.3 para um exemplo disso.

`n`

Iteradores com nomes que terminam em `n` limitam-se a um determinado número de correspondências.

`_indices`

Iteradores com nomes que terminam em `_indices` produzem, com seus valores de iteração usuais, os deslocamentos de byte na fatia em que aparecem.

A biblioteca padrão não fornece todas as combinações para cada operação. Por exemplo, muitas operações não precisam de uma variante `n`, já que é mais fácil simplesmente terminar a iteração antecipadamente.

Padrões para pesquisar texto

Quando uma função de biblioteca padrão precisa pesquisar, combinar, dividir ou aparar texto, ela aceita vários tipos diferentes

para representar o que procurar:

```
let haystack = "One fine day, in the middle of the night";
assert_eq!(haystack.find(','), Some(12));
assert_eq!(haystack.find("night"), Some(35));
assert_eq!(haystack.find(char::is_whitespace), Some(3));
```

Esses tipos são chamados de padrões e a maioria das operações os suporta:

```
assert_eq!("## Elephants"
    .trim_start_matches(|ch: char| ch == '#' || ch.is_whitespace()),
    "Elephants");
```

A biblioteca padrão suporta quatro tipos principais de padrões:

- Um `char` como um padrão corresponde a esse caractere.
- Um `String` ou `&str` ou `&&str` como um padrão corresponde a uma substring igual ao padrão.
- Uma closure `FnMut(char) -> bool` como um padrão corresponde a um único caractere para o qual a closure retorna verdadeiro.
- Um `&[char]` como um padrão (não um `&str`, mas uma fatia de valores `char`) corresponde a qualquer caractere único que aparece na lista. Observe que, se você escrever a lista como um array literal, talvez seja necessário chamar `as_ref()` para acertar o tipo:

```
let code = "\t  function noodle() { ";
assert_eq!(code.trim_start_matches([' ', '\t'].as_ref()),
    "function noodle() { ");
// Equivalente mais curto: &[' ', '\t'][..]
```

Caso contrário, o Rust ficará confuso com o tipo de array de tamanho fixo `&[char; 2]`, que infelizmente não é um tipo de padrão.

No próprio código da biblioteca, um padrão é qualquer tipo que implementa o `trait std::str::Pattern`. Os detalhes de `Pattern` ainda não são estáveis, então você não pode implementá-lo para seus próprios tipos no Rust estável, mas a porta está aberta para permitir expressões regulares e outros padrões sofisticados no futuro. O Rust garante que os tipos de padrão suportados agora continuarão a funcionar no futuro.

Pesquisando e substituindo

O Rust tem alguns métodos para procurar padrões em fatias e

possivelmente os substituir por um novo texto:

`slice.contains(pattern)`

Retorna verdadeiro se `slice` contém uma correspondência para `pattern`.

`slice.starts_with(pattern), slice.ends_with(pattern)`

Retorna verdadeiro se, no `slice`, o texto inicial ou final corresponde a `pattern`:

```
assert!("2017".starts_with(char::is_numeric));
```

`slice.find(pattern), slice.rfind(pattern)`

Retorna `Some(i)` se `slice` contém uma correspondência para `pattern`, em que `i` é o deslocamento de byte no qual o padrão aparece. O método `find` retorna a primeira correspondência, `rfind` o último:

```
let quip = "We also know there are known unknowns";
assert_eq!(quip.find("know"), Some(8));
assert_eq!(quip.rfind("know"), Some(31));
assert_eq!(quip.find("ya know"), None);
assert_eq!(quip.rfind(char::is_uppercase), Some(0));
```

`slice.replace(pattern, replacement)`

Retorna uma nova `String` formada pela substituição “gulosa” de todas as correspondências de `pattern` por `replacement`:

```
assert_eq!("The only thing we have to fear is fear itself"
    .replace("fear", "spin"),
    "The only thing we have to spin is spin itself");
assert_eq!("`Borrow` and `BorrowMut`"
    .replace(|ch:char| !ch.is_alphanumeric(), ""),
    "BorrowandBorrowMut");
```

Como a substituição é “gulosa”, o comportamento de `.replace()` em correspondências sobrepostas pode ser surpreendente. Aqui, há quatro instâncias do padrão, “aba”, mas o segundo e o quarto não correspondem mais depois que o primeiro e o terceiro são substituídos:

```
assert_eq!("cabababababbage"
    .replace("aba", "***"),
    "c***b***babbage")
```

`slice.replacen(pattern, replacement, n)`

Isso faz o mesmo, mas substitui no máximo as primeiras `n` correspondências.

Iterando por texto

A biblioteca padrão fornece várias maneiras de iterar pelo texto de uma fatia. A Figura 17.3 mostra alguns exemplos.

Você pode pensar nas famílias `split` e `match` como sendo complementares umas das outras: as divisões são os intervalos entre as correspondências.

A maioria desses métodos retorna iteradores que são reversíveis (isto é, eles implementam `DoubleEndedIterator`): chamando seu método de adaptador `.rev()` fornece um iterador que produz os mesmos itens, mas na ordem inversa.

`slice.chars()`

Retorna um iterador sobre os caracteres de `slice`.

`slice.char_indices()`

Retorna um iterador sobre `slice` e seus deslocamentos de bytes:

```
assert_eq!("élan".char_indices().collect::<Vec<_>>(),
    vec![(0, 'é'), // tem uma codificação UTF-8 de dois bytes
        (2, 'l'),
        (3, 'a'),
        (4, 'n')]);
```

Observe que isso não é equivalente a `.chars().enumerate()`, pois fornece o deslocamento de byte de cada caractere dentro da fatia, em vez de apenas numerar os caracteres.

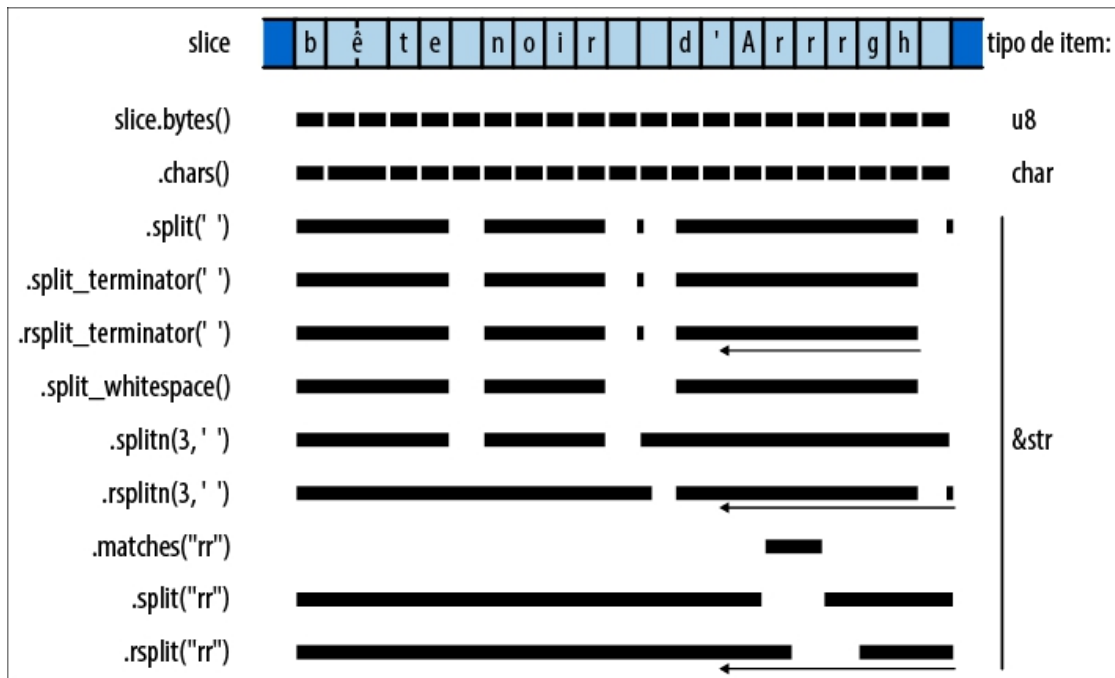


Figura 17.3: Algumas maneiras de iterar por uma fatia.

`slice.bytes()`

Retorna um iterador sobre os bytes individuais de `slice`, expondo a codificação UTF-8:

```
assert_eq!("élan".bytes().collect::<Vec<_>>(),
    vec![195, 169, b'l', b'a', b'n']);
```

`slice.lines()`

Retorna um iterador sobre as linhas de `slice`. As linhas são terminadas por `"\n"` ou `"\r\n"`. Cada item gerado é uma `&str` pedindo emprestado de `slice`. Os itens não incluem os caracteres de final de linha.

`slice.split(pattern)`

Retorna um iterador sobre as partes de `slice` separados por correspondências de `pattern`. Isso produz strings vazias entre correspondências imediatamente adjacentes, bem como para correspondências no início e no final de `slice`.

O iterador retornado não é reversível se `pattern` é uma `&str`. Esses padrões podem produzir diferentes sequências de correspondências, dependendo de qual direção você varre, o que os iteradores reversíveis são proibidos de fazer. Em vez disso, você pode utilizar o método `rsplit`, descrito a seguir.

`slice.rsplit(pattern)`

Esse método é o mesmo, mas varre `slice` do fim para o início, produzindo correspondências nessa ordem.

`slice.split_terminator(pattern)`, `slice.rsplit_terminator(pattern)`

Estes são semelhantes, exceto que o padrão `pattern` é tratado como um terminador, não um separador: se `pattern` corresponde ao final de `slice`, os iteradores não produzem uma fatia vazia representando a string vazia entre essa correspondência e o final da fatia, como `split` e `rsplit` fazem. Por exemplo:

```
// Os caracteres ':' são separadores aqui. Observe o "" final
assert_eq!("jimb:1000:Jim Blandy:".split(':').collect::<Vec<_>>(),
    vec!["jimb", "1000", "Jim Blandy", ""]);
```

```
// Os caracteres '\n' são terminadores aqui
assert_eq!("127.0.0.1 localhost\n\
127.0.0.1 www.reddit.com\n"
    .split_terminator('\n').collect::<Vec<_>>(),
    vec!["127.0.0.1 localhost",
        "127.0.0.1 www.reddit.com"]);
// Note, sem "" final!
```

`slice.splitn(n, pattern)`, `slice.rsplitn(n, pattern)`

Estes são como `split` e `rsplit`, exceto que dividem a string em no máximo `n` fatias, na primeira ou na última `n-1` correspondência a `pattern`.

`slice.split_whitespace()`, `slice.split_ascii_whitespace()`

Retorna um iterador sobre as partes separadas por espaços em branco de `slice`. Uma sequência de vários caracteres de espaço em branco é considerada um único separador. O espaço em branco à direita é ignorado.

O método `split_whitespace` utiliza a definição Unicode de espaço em branco, conforme implementado pelo método `is_whitespace` em `char`. Em vez disso, o método `split_ascii_whitespace` utiliza `char::is_ascii_whitespace`, que reconhece apenas caracteres de espaço em branco ASCII.

```
let poem = "This is just to say\n\
I have eaten\n\
the plums\n\
again\n";
assert_eq!(poem.split_whitespace().collect::<Vec<_>>(),
```

```
vec!["This", "is", "just", "to", "say",  
    "I", "have", "eaten", "the", "plums",  
    "again"]);
```

`slice.matches(pattern)`

Retorna um iterador sobre as correspondências para `pattern` em fatia. `slice.rmatches(pattern)` é o mesmo, mas itera do fim para o início.

`slice.match_indices(pattern)`, `slice.rmatch_indices(pattern)`

Estes são semelhantes, exceto que os itens gerados são pares (`offset`, `match`), e que `offset` é o deslocamento de byte no qual a correspondência começa e `match` é a fatia correspondente.

Recortando uma string

Recortar ou aparar uma string é remover o texto, geralmente espaços em branco, do início ou do final da string. Muitas vezes, é útil limpar a entrada lida de um arquivo em que o usuário pode ter recuado o texto para legibilidade ou acidentalmente deixado um espaço em branco à direita em uma linha.

`slice.trim()`

Retorna uma subfatia de `slice` que omite qualquer espaço em branco inicial e final. `slice.trim_start()` omite apenas espaços em branco à esquerda, `slice.trim_end()` apenas espaços em branco à direita:

```
assert_eq!("t*.rs ".trim(), "*.rs");  
assert_eq!("t*.rs ".trim_start(), "*.rs ");  
assert_eq!("t*.rs ".trim_end(), "t*.rs");
```

`slice.trim_matches(pattern)`

Retorna uma subfatia de `slice` que omite todas as correspondências de `pattern` do início ao fim. Os métodos `trim_start_matches` e `trim_end_matches` fazem o mesmo apenas para correspondências iniciais ou finais:

```
assert_eq!("001990".trim_start_matches('0'), "1990");
```

`slice.strip_prefix(pattern)`, `slice.strip_suffix(pattern)`

Se `slice` começa com `pattern`, `strip_prefix` retorna `Some` armazenando a fatia com o texto correspondente removido. Caso contrário, retorna `None`. O método `strip_suffix` é semelhante, mas verifica uma correspondência no final da string.

Esses são como `trim_start_matches` e `trim_end_matches`, exceto que retornam uma `Option` e apenas uma cópia de `pattern` é removida:

```
let slice = "banana";
assert_eq!(slice.strip_suffix("na"),
           Some("bana"))
```

Conversão de maiúsculas e minúsculas para strings

Os métodos `slice.to_uppercase()` e `slice.to_lowercase()` retornam uma `string` recém-allocada contendo o texto da `slice` convertido para maiúsculas ou minúsculas. O resultado pode não ter o mesmo comprimento que `slice`. Veja “Conversão de maiúsculas e minúsculas para caracteres”, na página 494, para detalhes.

Convertendo outros tipos a partir de strings

O Rust fornece traits padrão para analisar valores de strings e produzir representações textuais de valores.

Se um tipo implementa o trait `std::str::FromStr`, ele fornece uma maneira padrão de converter um valor de uma fatia de `string`:

```
pub trait FromStr: Sized {
    type Err;
    fn from_str(s: &str) -> Result<Self, Self::Err>;
}
```

Todos os tipos nativos usuais implementam `FromStr`:

```
use std::str::FromStr;

assert_eq!(usize::from_str("3628800"), Ok(3628800));
assert_eq!(f64::from_str("128.5625"), Ok(128.5625));
assert_eq!(bool::from_str("true"), Ok(true));

assert!(f64::from_str("not a float at all").is_err());
assert!(bool::from_str("TRUE").is_err());
```

O tipo `char` também implementa `FromStr`, para strings com apenas um caractere:

```
assert_eq!(char::from_str("é"), Ok('é'));
assert!(char::from_str("abcdefg").is_err());
```

O tipo `std::net::IpAddr`, um `enum` que armazena um endereço de Internet IPv4 ou IPv6, também implementa `FromStr`:

```
use std::net::IpAddr;

let address = IpAddr::from_str("fe80::0000:3ea9:f4ff:fe34:7a50")?;
assert_eq!(address,
    IpAddr::from([0xfe80, 0, 0, 0, 0x3ea9, 0xf4ff, 0xfe34, 0x7a50]));
```

As fatias de string têm um método `parse` que converte a fatia em qualquer tipo que você quiser, assumindo que implementa `FromStr`. Como com `Iterator::collect`, às vezes você precisará escrever explicitamente o tipo que deseja, então `parse` nem sempre é muito mais legível do que chamar `from_str` diretamente:

```
let address = "fe80::0000:3ea9:f4ff:fe34:7a50".parse::<IpAddr>()?;
```

Convertendo outros tipos em strings

Há três maneiras principais de converter valores não textuais em strings:

- Tipos que têm uma forma impressa natural legível por humanos podem implementar o `trait std::fmt::Display`, que permite que você utilize o especificador de formato `{}` na macro `format!`:

```
assert_eq!(format!("{}", wow, "doge"), "doge, wow");
assert_eq!(format!("{}", true), "true");
assert_eq!(format!("{}", {:.3}, {:.3}), 0.5, f64::sqrt(3.0)/2.0,
    "(0.500, 0.866)");
```

```
// Utilizando o `address` de cima
let formatted_addr: String = format!("{}", address);
assert_eq!(formatted_addr, "fe80::3ea9:f4ff:fe34:7a50");
```

Todos os tipos numéricos de máquina do Rust implementam `Display`, assim como caracteres, strings e fatias. Os tipos de ponteiro inteligente `Box<T>`, `Rc<T>` e `Arc<T>` implementam `Display` se o próprio `T` o faz: sua forma exibida é simplesmente a de seu referente. Recipientes como `Vec` e `HashMap` não implementam `Display`, já que não há uma única forma natural legível por humanos para esses tipos.

- Se um tipo implementa `Display`, a biblioteca padrão implementa automaticamente o `trait std::str::ToString` para ele, cujo único método `to_string` pode ser mais conveniente quando você não precisa da

flexibilidade de `format!`:

```
// Continuação de cima
assert_eq!(address.to_string(), "fe80::3ea9:f4ff:fe34:7a50");
```

O `trait ToString` é anterior à introdução de `Display` e é menos flexível. Para seus próprios tipos, você geralmente deve implementar `Display` em vez de `ToString`.

- Cada tipo público na biblioteca padrão implementa `std::fmt::Debug`, que pega um valor e o formata como uma string de maneira útil para os programadores. A maneira mais fácil de utilizar `Debug` para produzir uma string é por meio do especificador de formato `{:?}` da macro `format!`:

```
// Continuação de cima
let addresses = vec![address,
    IpAddr::from_str("192.168.0.1")?];
assert_eq!(format!("{:?}", addresses),
    "[fe80::3ea9:f4ff:fe34:7a50, 192.168.0.1]");
```

Isso tira vantagem de uma implementação geral de `Debug` para `Vec<T>`, para qualquer `T` que implementa `Debug`. Todos os tipos de coleção do Rust têm tais implementações.

Você deve implementar `Debug` para seus próprios tipos também. Geralmente é melhor deixar o Rust derivar uma implementação, como fizemos para o tipo `Complex` do Capítulo 12:

```
#[derive(Copy, Clone, Debug)]
struct Complex { re: f64, im: f64 }
```

Os traits de formatação `Display` e `Debug` são apenas dois dentre vários que a macro `format!` e seus parentes utilizam para formatar valores como texto. Abordaremos os outros e explicaremos como implementá-los, em “Formatando valores”, na página 515.

Emprestando como outros tipos semelhantes a texto

Você pode pegar emprestado o conteúdo de uma fatia de várias maneiras diferentes:

- Fatias e Strings implementam `AsRef<str>`, `AsRef<[u8]>`, `AsRef<Path>` e `AsRef<OsStr>`. Muitas funções de biblioteca padrão utilizam esses

traits como limites em seus tipos de parâmetro, então você pode passar fatias e strings para elas diretamente, mesmo quando o que elas realmente querem é algum outro tipo. Ver “AsRef e AsMut”, na página 368, para uma explicação mais detalhada.

- Fatias e strings também implementam o trait `std::borrow::Borrow<str>`. `HashMap` e `BTreeMap` utilizam `Borrow` para fazer `Strings` funcionarem bem como chaves em uma tabela. Ver “Borrow e BorrowMut”, na página 369 para detalhes.

Acessando texto como UTF-8

Existem duas maneiras principais de obter os bytes que representam o texto, dependendo se você deseja tomar posse dos bytes ou apenas tomá-los emprestados:

`slice.as_bytes()`

Toma emprestado bytes de `slice` como um `&[u8]`. Como essa não é uma referência mutável, `slice` pode assumir que seus bytes permanecerão UTF-8 bem formados.

`string.into_bytes()`

Toma posse de `string` e retorna um `Vec<u8>` dos bytes da string por valor. Essa é uma conversão barata, pois simplesmente entrega o `Vec<u8>` que a string estava utilizando como seu buffer. Como a string não existe mais, não há necessidade de os bytes continuarem sendo UTF-8 bem formados e o chamador é livre para modificar o `Vec<u8>` como quiser.

Produzindo texto a partir de dados UTF-8

Se você tiver um bloco de bytes que acredita conter dados UTF-8, terá algumas opções para convertê-los em `Strings` ou fatias, dependendo de como você deseja lidar com erros:

`str::from_utf8(byte_slice)`

Pega uma fatia de bytes `&[u8]` e retorna um `Result`, que pode ser `Ok(&str)` se `byte_slice` contiver UTF-8 bem formado, ou um erro caso contrário.

`String::from_utf8(vec)`

Tenta construir uma string a partir de um `Vec<u8>` passado por valor. Se `vec` contém UTF-8 bem formado, `from_utf8` retorna `Ok(string)`, em que `string` tomou posse de `vec` para uso como seu buffer. Nenhuma alocação de heap ou cópia do texto ocorre.

Se os bytes não forem UTF-8 válidos, isso retornará `Err(e)`, em que `e` é um valor `FromUtf8Error` de erro. A chamada a `e.into_bytes()` devolve o vetor original `vec`, portanto não é perdido quando a conversão falha:

```
let good_utf8: Vec<u8> = vec![0xe9, 0x8c, 0x86];
assert_eq!(String::from_utf8(good_utf8).ok(), Some("鏄".to_string()));
```

```
let bad_utf8: Vec<u8> = vec![0x9f, 0xf0, 0xa6, 0x80];
let result = String::from_utf8(bad_utf8);
assert!(result.is_err());
// Como String::from_utf8 falhou, não consumiu o vetor
// original, e o valor do erro o devolve intacto para nós
assert_eq!(result.unwrap_err().into_bytes(),
           vec![0x9f, 0xf0, 0xa6, 0x80]);
```

`String::from_utf8_lossy(byte_slice)`

Tenta construir uma `String` ou `&str` a partir de uma fatia compartilhada de bytes `&[u8]`. Essa conversão sempre é bem-sucedida, substituindo qualquer UTF-8 malformado por caracteres substitutos Unicode. O valor de retorno é um `Cow<str>` que ou pega emprestada uma `&str` diretamente de `byte_slice` se contiver UTF-8 bem formado, ou possui uma `String` recém-alocada com caracteres substitutos para os bytes malformados. Portanto, quando `byte_slice` está bem formado, nenhuma alocação de heap ou cópia ocorre. Discutimos `Cow<str>` com mais detalhes em “Adiando a alocação”, na página 512.

`String::from_utf8_unchecked(vec)`

Se você sabe com certeza que seu `Vec<u8>` contém UTF-8 bem formado, então pode chamar essa função insegura. Isso simplesmente encapsula `vec` como uma `String` e a retorna, sem examinar os bytes. Você é responsável por garantir que não introduziu UTF-8 malformado no sistema e é por isso que essa função está marcada como `unsafe`.

`str::from_utf8_unchecked(byte_slice)`

Da mesma forma, isso recebe um `&[u8]` e o devolve como um `&str`,

sem verificar se ele contém UTF-8 bem formado. Como ocorre com `String::from_utf8_unchecked`, você é responsável por garantir que isso seja seguro.

Adiando a alocação

Suponha que você queira que seu programa cumprimente o usuário. No Unix, você poderia escrever:

```
fn get_name() -> String {
    std::env::var("USER") // O Windows utiliza "USERNAME"
        .unwrap_or("whoever you are".to_string())
}

println!("Greetings, {}!", get_name());
```

Para usuários Unix, isso os saúda pelo nome de usuário. Para usuários do Windows, tragicamente sem nome, um texto alternativo é fornecido.

A função `std::env::var` retorna uma `String` – e tem boas razões para o fazer, que não discutiremos aqui. Mas isso significa que o texto alternativo também deve ser retornado como uma `String`. Isso é decepcionante: quando `get_name` retorna uma string estática, nenhuma alocação deveria ser necessária.

O cerne do problema é que às vezes o valor de retorno de `get_name` deve ser uma `String` com posse, às vezes deve ser uma `&'static str` e não podemos saber qual será até executarmos o programa. Esse caráter dinâmico é a dica para considerar o uso de `std::borrow::Cow`, o tipo clone-on-write que pode conter dados próprios ou emprestados.

Conforme explicado em “Borrow e ToOwned em funcionamento: Cow (“clone on write”)", na página 377, `Cow<'a, T>` é uma enumeração com duas variantes: `Owned` e `Borrowed`. `Borrowed` detém uma referência `&'a T` e `Owned` detém a versão proprietária de `&T`: `String` para `&str`, `Vec<i32>` para `&[i32]` e assim por diante. Quer seja `Owned` ou `Borrowed`, uma `Cow<'a, T>` sempre pode produzir um `&T` para você utilizar. Na verdade, `Cow<'a, T>` desreferencia para `&T`, comportando-se como uma espécie de ponteiro inteligente.

Mudar `get_name` para devolver um `Cow` resulta no seguinte:

```
use std::borrow::Cow;
```

```
fn get_name() -> Cow<'static, str> {
  std::env::var("USER")
    .map(|v| Cow::Owned(v))
    .unwrap_or(Cow::Borrowed("whoever you are"))
}
```

Se isso conseguir ler a variável de ambiente "USER", o `map` retorna a `String` resultante como um `Cow::Owned`. Se falhar, `unwrap_or` retorna sua `&str` estática como um `Cow::Borrowed`. O chamador pode permanecer inalterado:

```
println!("Greetings, {}", get_name());
```

Desde que `T` implemente o `trait std::fmt::Display`, exibir um `Cow<'a, T>` produz os mesmos resultados que exibir um `T`.

`Cow` também é útil quando você pode ou não precisar modificar algum texto emprestado. Quando nenhuma mudança for necessária, você pode continuar a pegá-lo emprestado. Mas o comportamento `clone-on-write` (clone ao escrever) homônimo de `Cow` pode fornecer a você uma cópia com posse e mutável do valor sob demanda. O método `to_mut` de `Cow` garante que o `Cow` é `Cow::Owned`, aplicando a implementação `ToOwned` do valor se necessário e, em seguida, retorna uma referência mutável ao valor.

Então, se você achar que alguns de seus usuários, mas não todos, têm títulos pelos quais eles preferem ser tratados, pode dizer:

```
fn get_title() -> Option<&'static str> { ... }

let mut name = get_name();
if let Some(title) = get_title() {
  name.to_mut().push_str(", ");
  name.to_mut().push_str(title);
}
```

```
println!("Greetings, {}", name);
```

Isso pode produzir uma saída como a seguinte:

```
$ cargo run
Greetings, jimb, Esq.!
$
```

O que é bom aqui é que, se `get_name()` retorna uma `string` estática e `get_title` retorna `None`, a `Cow` simplesmente carrega a `string` estática por todo o caminho até o `println!`. Você conseguiu adiar a alocação, a menos que seja realmente necessária, enquanto ainda escreve um

código direto.

Como a `Cow` é frequentemente utilizada para strings, a biblioteca padrão tem algum suporte especial para `Cow<'a, str>`. Ela fornece as conversões `From` e `Into` tanto de `String` como de `&str`, então você pode escrever `get_name` mais concisamente:

```
fn get_name() -> Cow<'static, str> {
    std::env::var("USER")
        .map(|v| v.into())
        .unwrap_or("whoever you are".into())
}
```

`Cow<'a, str>` também implementa `std::ops::Add` e `std::ops::AddAssign`; portanto, para adicionar o título ao nome, você poderia escrever:

```
if let Some(title) = get_title() {
    name += ", ";
    name += title;
}
```

Ou, uma vez que uma `String` pode ser o destino de uma macro `write!`:

```
use std::fmt::Write;

if let Some(title) = get_title() {
    write!(name.to_mut(), ", {}", title).unwrap();
}
```

Como antes, nenhuma alocação ocorre até que você tente modificar o `Cow`.

Tenha em mente que nem todo `Cow<..., str>` deve ser `'static`: você pode utilizar `Cow` para emprestar o texto previamente calculado até o momento em que uma cópia se torna necessária.

Strings como coleções genéricas

`String` implementa tanto `std::default::Default` como `std::iter::Extend`: `default` retorna uma string vazia e `extend` pode anexar caracteres, fatias de string, `Cow<..., str>`s, ou strings ao final de uma string. Essa é a mesma combinação de traits implementadas pelos outros tipos de coleção do Rust, como `Vec` e `HashMap` para padrões de construção genéricos, como `collect` e `partition`.

O tipo `&str` também implementa `Default`, retornando uma fatia vazia. Isso é útil em alguns casos extremos; por exemplo, permite derivar

Default para estruturas contendo fatias de string.

Formatando valores

Ao longo do livro, utilizamos macros de formatação de texto como `println!`:

```
println!("{:.3}µs: relocated {} at {:#x} to {:#x}, {} bytes",
        0.84391, "object",
        140737488346304_usize, 6299664_usize, 64);
```

Essa chamada produz a seguinte saída:

```
0.844µs: relocated object at 0x7fffffffddc0 to 0x602010, 64 bytes
```

O literal string serve como um template para a saída: cada `{...}` no template é substituído pela forma formatada de um dos seguintes argumentos. A string do template deve ser uma constante para que o Rust possa compará-la com os tipos de argumentos em tempo de compilação. Cada argumento deve ser utilizado; caso contrário, o Rust relata um erro em tempo de compilação.

Vários recursos de biblioteca padrão compartilham essa pequena linguagem para formatar strings:

- A macro `format!` o utiliza para construir Strings.
- As macros `println!` e `print!` escrevem texto formatado no fluxo de saída padrão.
- As macros `writeln!` e `write!` o escrevem em um fluxo de saída designado.
- A macro `panic!` o utiliza para construir uma expressão (idealmente informativa) de erro terminal.

Os recursos de formatação do Rust são projetados para serem abertos. Você pode estender essas macros para suportar os próprios tipos implementando os traits `std::fmt` de formatação do módulo. E você pode utilizar a macro `format_args!` e o tipo `std::fmt::Arguments` para fazer as próprias funções e macros suportarem a linguagem de formatação.

Macros de formatação sempre emprestam referências compartilhadas a seus argumentos; nunca tomam posse deles ou os transformam.

As formas {...} da template são chamadas de parâmetros de formato e assumem a forma {which:how}. Ambas as partes são opcionais; {} é frequentemente utilizado.

O valor `which` seleciona qual argumento após o template deve ocupar o lugar do parâmetro. Você pode selecionar argumentos por índice ou por nome. Parâmetros sem valor `which` são simplesmente pareados com os argumentos da esquerda para a direita.

O valor `how` diz como o argumento deve ser formatado: quanto preenchimento, com qual precisão, em qual raiz numérica e assim por diante. Se `how` estiver presente, o dois-pontos antes é necessário. A Tabela 17.4 apresenta alguns exemplos.

Tabela 17.4: Exemplos de string formatada

String de template	Lista de argumentos	Resultado
"number of {}: {}"	"elephants", 19	"number of elephants: 19"
"from {1} to {0}"	"the grave", "the cradle"	"from the cradle to the grave"
"v = {:?}"	vec![0,1,2,5,12,29]	"v = [0, 1, 2, 5, 12, 29]"
"name = {:?}"	"Nemo"	"name = \"Nemo\""
"{:8.2} km/s"	11.186	" 11.19 km/s"
"{:20} {:02x} {:02x}"	"adc #42", 105, 42	"adc #42 69 2a"
"{:1:02x} {2:02x} {0}"	"adc #42", 105, 42	"69 2a adc #42"
"{:lsb:02x} {msb:02x} {insn}"	insn="adc #42", lsb=105, msb=42	"69 2a adc #42"
"{:02?}"	[110, 11, 9]	"[110, 11, 09]"
"{:02x?}"	[110, 11, 9]	"[6e, 0b, 09]"

Se você quiser incluir os caracteres { ou } em sua saída, dobre os caracteres no template:

```
assert_eq!(format!("{}", {a, c}) iø {{a, b, c}}"),
           "{a, c} iø {a, b, c}");
```

Formatando valores de texto

Ao formatar um tipo textual como `&str` ou `String` (`char` é tratado como uma string de um único caractere), o valor `how` de um parâmetro tem várias partes, todas opcionais:

- Um *limite de comprimento de texto*. O Rust trunca seu argumento

se for mais longo do que isso. Se você não especificar nenhum limite, o Rust utilizará o texto completo.

- Um *tamanho mínimo de campo*. Após qualquer truncamento, se seu argumento for menor que isso, o Rust o preenche à direita (por padrão) com espaços (por padrão) para criar um campo desse tamanho. Se omitido, o Rust não preenche seu argumento.
- Um *alinhamento*. Se seu argumento precisar ser preenchido para atender ao tamanho mínimo do campo, isso diz onde seu texto deve ser colocado dentro do campo. <, ^ e > colocam seu texto no início, no meio e no fim, respectivamente.
- Um caractere de *preenchimento* a ser utilizado nesse processo de preenchimento. Se omitido, o Rust utiliza espaços. Se você especificar o caractere de preenchimento, também deverá especificar o alinhamento.

A Tabela 17.5 ilustra alguns exemplos mostrando como escrever as coisas e seus efeitos. Todos utilizam o mesmo argumento de oito caracteres, "bookends".

Tabela 17.5: Diretivas de string de formato para texto

Recursos em uso	String de template	Resultado
Default	"{}"	"bookends"
Largura mínima do campo	"{:4}"	"bookends"
	"{:12}"	"bookends "
Limite de comprimento do texto	"{:4.}"	"book"
	"{:12.}"	"bookends"
Largura do campo, limite de comprimento	"{:12.20}"	"bookends "
	"{:4.20}"	"bookends"
	"{:4.6}"	"booken"
	"{:6.4}"	"book "
Alinhado à esquerda, largura	"{:<12}"	"bookends "
Centralizado, largura	"{: ^12}"	" bookends "
Alinhado à direita, largura	"{:>12}"	" bookends"
Preenchimento com '=', centralizado, largura	"{: ^12}"	"==bookends=="
Preenchimento com '*', alinhado à direita, largura, limite	"{: *>12.4}"	"*****book"

O formatador do Rust tem uma compreensão ingênua de largura: ele assume que cada caractere ocupa uma coluna, sem levar em conta a combinação de caracteres, katakana de meia largura, espaços de largura zero ou outras realidades confusas do Unicode. Por exemplo:

```
assert_eq!(format!("{:4}", "th\u{e9}"), "th\u{e9} ");
assert_eq!(format!("{:4}", "the\u{301}"), "the\u{301}");
```

Embora o Unicode diga que essas strings são equivalentes a "thé", o formatador do Rust não sabe que caracteres como '\u{301}', COMBINING ACUTE ACCENT (combinação com acento agudo), necessitam de tratamento especial. Ele preenche a primeira string corretamente, mas assume que a segunda tem quatro colunas de largura e não adiciona nenhum preenchimento. Embora seja fácil ver como o Rust poderia melhorar nesse caso específico, a verdadeira formatação de texto multilíngue para todos os scripts do Unicode é uma tarefa monumental, mais bem tratada confiando nos kits de ferramentas de interface do usuário da sua plataforma ou talvez gerando HTML e CSS e fazer um navegador web resolver tudo. Existe um crate popular, `unicode-width`, que lida com alguns aspectos disso.

Com `&str` e `String`, você também pode passar tipos de ponteiro inteligente de macros de formatação com referências textuais, como `Rc<String>` OU `Cow<'a, str>`, sem cerimônia.

Como os caminhos de nome de arquivo não são necessariamente UTF-8 bem formados, `std::path::Path` não é exatamente um tipo textual; você não pode passar um `std::path::Path` diretamente para uma macro de formatação. Entretanto, um método `display` de `Path` retorna um valor que você pode formatar para ordenar as coisas de maneira apropriada para a plataforma:

```
println!("processing file: {}", path.display());
```

Formatando números

Quando o argumento de formatação tem um tipo numérico como `usize` OU `f64`, o valor `how` do parâmetro tem as seguintes partes, todas opcionais:

- Um *preenchimento* e *alinhamento*, que funcionam como fazem

com tipos textuais.

- Um caractere `+`, solicitando que o sinal do número seja sempre mostrado, mesmo quando o argumento for positivo.
- Um caractere `#`, solicitando um prefixo radix explícito como `0x` ou `0b`. Veja o último item desta lista, “notação”.
- Um caractere `0`, solicitando que a largura mínima do campo seja satisfeita incluindo zeros à esquerda no número, em vez da abordagem de preenchimento usual.
- Uma largura *mínima de campo*. Se o número formatado não tiver pelo menos essa largura, o Rust o preencherá à esquerda (por padrão) com espaços (por padrão) para criar um campo com a largura especificada.
- Uma *precisão* para argumentos de ponto flutuante, indicando quantos dígitos o Rust deve incluir após o ponto decimal. O Rust arredonda ou estende com zeros conforme necessário para produzir exatamente essa quantidade de dígitos fracionários. Se a precisão for omitida, o Rust tenta representar com precisão o valor utilizando o menor número de dígitos possível. Para argumentos do tipo inteiro, a precisão é ignorada.
- Uma *notação*. Para tipos inteiros, isso pode ser `b` para binário, `o` para octal, ou `x` ou `X` para hexadecimal com letras maiúsculas ou minúsculas. Se você incluiu o caractere `#`, eles incluem um prefixo radix explícito no estilo Rust, `0b`, `0o`, `0x` ou `0X`. Para tipos de ponto flutuante, uma radix de `e` ou `E` exige notação científica, com coeficiente normalizado, utilizando `e` ou `E` para o expoente. Se você não especificar nenhuma notação, o Rust formata os números em decimal.

A Tabela 17.6 mostra alguns exemplos de formatação do valor `i32` 1234.

Tabela 17.6: Diretivas de string de formato para números inteiros

Recursos em uso	String de template	Resultado
Default	"{"	"1234"
Sinal forçado	"{:+}"	"1234"

Recursos em uso	String de template	Resultado
Largura mínima do campo	"{:12}"	" 1234"
	"{:2}"	"1234"
Sinal, largura	"{:+12}"	" +1234"
Zeros à esquerda, largura	"{:012}"	"000000001234"
Sinal, zeros, largura	"{:+012}"	"+000000001234"
Alinhado à esquerda, largura	"{:<12}"	"1234 "
Centralizado, largura	"{: ^12}"	" 1234 "
Alinhado à direita, largura	"{:>12}"	" 1234"
Alinhado à esquerda, sinal, largura	"{:<+12}"	" +1234 "
Centralizado, sinal, largura	"{: ^+12}"	" +1234 "
Alinhado à direita, sinal, largura	"{:>+12}"	" +1234"
Preenchido com '=', centralizado, largura	"{: = ^12}"	"====1234===="
Notação binária	"{:b}"	"10011010010"
Largura, notação octal	"{:12o}"	" 2322"
Sinal, largura, notação hexadecimal	"{:+12x}"	" +4d2"
Sinal, largura, hexa com dígitos maiúsculos	"{:+12X}"	" +4D2"
Sinal, prefixo radix explícito, largura, hexadecimal	"{:+ #12x}"	" +0x4d2"
Sinal, radix, zeros, largura, hexa	"{:+ #012x}"	" +0x0000004d2"
	"{:+ #06x}"	" +0x4d2"

Como mostram os dois últimos exemplos, a largura mínima do campo se aplica ao número inteiro, sinal, prefixo de base e tudo mais.

Números negativos sempre incluem seu sinal. Os resultados são como os mostrados nos exemplos de "sinal forçado".

Quando você impõe zeros à esquerda, os caracteres de alinhamento e preenchimento são simplesmente ignorados, pois os zeros expandem o número para preencher todo o campo.

Utilizando o argumento 1234.5678, podemos mostrar efeitos específicos para tipos de ponto flutuante (Tabela 17.7).

Tabela 17.7: Diretivas de string de formato para números de ponto flutuante

Recursos em uso	String de template	Resultado
Default	"{}"	"1234.5678"
Precisão	"{:2}"	"1234.57"
	"{:6}"	"1234.567800"
Largura mínima do campo	"{:12}"	" 1234.5678"
Mínimo, precisão	"{:12.2}"	" 1234.57"
	"{:12.6}"	" 1234.567800"
Zeros à esquerda, mínimo, precisão	"{:012.6}"	"01234.567800"
Científico	"{:e}"	"1.2345678e3"
Científico, precisão	"{:3e}"	"1.235e3"
Científico, mínimo, precisão	"{:12.3e}"	" 1.235e3"
	"{:12.3E}"	" 1.235E3"

Formatando outros tipos

Além de strings e números, você pode formatar vários outros tipos da biblioteca padrão:

- Todos os tipos de erro podem ser formatados diretamente, facilitando sua inclusão em mensagens de erro. Todo tipo de erro deve implementar o trait `std::error::Error`, que estende o trait de formatação padrão `std::fmt::Display`. Como consequência, qualquer tipo que implemente `Error` está pronto para ser formatado.
- Você pode formatar tipos de endereço de protocolo de internet como `std::net::IpAddr` e `std::net::SocketAddr`.
- Os valores booleanos `true` e `false` podem ser formatados, embora geralmente não sejam as melhores strings para apresentar diretamente aos usuários finais.

Você deve utilizar os mesmos tipos de parâmetros de formato que utilizaria para strings. O limite de comprimento, a largura do campo e os controles de alinhamento funcionam conforme o esperado.

Valores de formatação para depuração

Para ajudar na depuração e logging, o parâmetro `{:?}` formata qualquer tipo público na biblioteca padrão do Rust de forma a ser

útil para os programadores. Você pode utilizar isso para inspecionar vetores, fatias, tuplas, tabelas hash, threads e centenas de outros tipos.

Por exemplo, você pode escrever o seguinte:

```
use std::collections::HashMap;
let mut map = HashMap::new();
map.insert("Portland", (45.5237606,-122.6819273));
map.insert("Taipei", (25.0375167, 121.5637));
println!("{:?}", map);
```

Isso imprime:

```
{"Taipei": (25.0375167, 121.5637), "Portland": (45.5237606, -122.6819273)}
```

Os tipos `HashMap` e `(f64, f64)` já sabem como se formatar, sem nenhum esforço de sua parte.

Se você incluir o caractere `#` no parâmetro de formato, o Rust imprimirá o valor em um formato mais fácil de ler (pretty-print). Mudar esse código para dizer `println!("{:#?}", map)` resulta nesta saída:

```
{
  "Taipei": (
    25.0375167,
    121.5637
  ),
  "Portland": (
    45.5237606,
    -122.6819273
  )
}
```

Essas formas exatas não são garantidas e às vezes mudam de uma versão do Rust para outra.

A formatação de depuração geralmente imprime números em decimal, mas você pode colocar um `x` ou `X` antes do ponto de interrogação para impor hexadecimal. A sintaxe do zero à esquerda e da largura do campo também é respeitada. Por exemplo, você pode escrever:

```
println!("ordinary: {:02?}", [9, 15, 240]);
println!("hex:      {:02x?}", [9, 15, 240]);
```

Isso imprime:

```
ordinary: [09, 15, 240]
hex:      [09, 0f, f0]
```

Como mencionamos, você pode utilizar a sintaxe `#[derive(Debug)]` para fazer seus próprios tipos funcionarem com `{:?}`:

```
#[derive(Copy, Clone, Debug)]
struct Complex { re: f64, im: f64 }
```

Com essa definição em vigor, podemos utilizar um formato `{:?}` para imprimir valores `Complex`:

```
let third = Complex { re: -0.5, im: f64::sqrt(0.75) };
println!("{:?}", third);
```

Isso imprime:

```
Complex { re: -0.5, im: 0.8660254037844386 }
```

Isso é bom para depuração, mas seria legal se `{}` pudesse imprimi-los de uma forma mais tradicional, como `-0.5 + 0.8660254037844386i`. Em “Formatando os próprios tipos”, na página 524, mostraremos como fazer exatamente isso.

Ponteiros de formatação para depuração

Normalmente, se você passar qualquer tipo de ponteiro para uma macro de formatação – uma referência, um `Box`, um `Rc` – a macro simplesmente segue o ponteiro e formata seu referente; o ponteiro em si não é de interesse. Mas, quando você está depurando, às vezes é útil ver o ponteiro: um endereço pode servir como um “nome” aproximado para um valor individual, que pode ser esclarecedor ao examinar estruturas com ciclos ou compartilhamento.

A notação `{:p}` formata referências, boxes e outros tipos semelhantes a ponteiro como endereços:

```
use std::rc::Rc;
let original = Rc::new("mazurka".to_string());
let cloned = original.clone();
let impostor = Rc::new("mazurka".to_string());
println!("text:   {}, {}, {}", original, cloned, impostor);
println!("pointers: {:p}, {:p}, {:p}", original, cloned, impostor);
```

Esse código imprime:

```
text:   mazurka, mazurka, mazurka
pointers: 0x7f99af80e000, 0x7f99af80e000, 0x7f99af80e030
```

Obviamente, os valores específicos do ponteiro variam de execução

para execução, mas, mesmo assim, comparar os endereços deixa claro que os dois primeiros são referências à mesma `String`, enquanto a terceira aponta para um valor distinto.

Os endereços tendem a parecer uma sopa hexadecimal, então visualizações mais refinadas podem valer a pena, mas o estilo `{:p}` ainda pode ser uma solução eficaz e rápida.

Referindo-se a argumentos por índice ou nome

Um parâmetro de formato pode selecionar explicitamente qual argumento ele utiliza. Por exemplo:

```
assert_eq!(format!("{1},{0},{2}", "zeroth", "first", "second"),
           "first,zeroth,second");
```

Você pode incluir parâmetros de formato após dois-pontos:

```
assert_eq!(format!("{2:#06x},{1:b},{0:=>10}", "first", 10, 100),
           "0x0064,1010,====first");
```

Você também pode selecionar argumentos por nome. Isso torna templates complexos com muitos parâmetros muito mais legíveis. Por exemplo:

```
assert_eq!(format!("{description:.<25}{quantity:2} @ {price:5.2}",
                   price=3.25,
                   quantity=3,
                   description="Maple Turmeric Latte"),
           "Maple Turmeric Latte..... 3 @ 3.25");
```

(Os argumentos nomeados aqui se assemelham a argumentos de palavra-chave no Python, mas isso é apenas um recurso especial das macros de formatação, não faz parte da sintaxe de chamada de função do Rust.)

Você pode misturar parâmetros indexados, nomeados e posicionais (ou seja, nenhum índice ou nome) juntos em um único uso de macro de formatação. Os parâmetros posicionais são pareados com argumentos da esquerda para a direita, como se os parâmetros indexados e nomeados não existissem:

```
assert_eq!(format!("{mode} {2} {} {}",
                   "people", "eater", "purple", mode="flying"),
           "flying purple people eater");
```

Os argumentos nomeados devem aparecer no final da lista.

Larguras dinâmicas e precisões

A largura mínima do campo, o limite de comprimento do texto e a precisão numérica de um parâmetro nem sempre precisam ser valores fixos; você pode escolhê-los em tempo de execução.

Estivemos analisando casos como essa expressão, que fornece a string `content` justificada à direita em um campo de 20 caracteres de largura:

```
format!("{:>20}", content)
```

Mas, se você quiser escolher a largura do campo em tempo de execução, pode escrever:

```
format!("{:>1$}", content, get_width())
```

Escrever `1$` para a largura mínima do campo instrui `format!` a utilizar o valor do segundo argumento como a largura. O argumento citado deve ser um `usize`. Você também pode referenciar o argumento pelo nome:

```
format!("{:>width$}", content, width=get_width())
```

A mesma abordagem também funciona para o limite de comprimento do texto:

```
format!("{:>width$.limit$}", content,  
width=get_width(), limit=get_limit())
```

No lugar do limite de comprimento do texto ou da precisão do ponto flutuante, você também pode escrever `*`, que diz para utilizar o próximo argumento posicional como a precisão. O seguinte exemplo picota `content` em `get_limit()` caracteres no máximo:

```
format!("{:.*}", get_limit(), content)
```

O argumento tomado como a precisão deve ser um `usize`. Não há sintaxe correspondente para a largura do campo.

Formatando os próprios tipos

As macros de formatação utilizam um conjunto de traits definidas no módulo `std::fmt` para converter valores em texto. Você pode fazer com que as macros de formatação do Rust formatem os próprios tipos,

implementando você mesmo um ou mais desses traits.

A notação de um parâmetro de formato indica qual trait o tipo de seu argumento deve implementar, conforme ilustrado na Tabela 17.8.

Tabela 17.8: Notação da diretiva de string de formato

Notação	Exemplo	Trait	Propósito
nenhum	{}	std::fmt::Display	Texto, números, erros: o trait genérico
b	{bits:#b}	std::fmt::Binary	Números em binário
o	{:#5o}	std::fmt::Octal	Números em octal
x	{:4x}	std::fmt::LowerHex	Números em hexadecimal, dígitos minúsculos
X	{:016X}	std::fmt::UpperHex	Números em hexadecimal, dígitos maiúsculos
e	{:.3e}	std::fmt::LowerExp	Números de ponto flutuante em notação científica
E	{:.3E}	std::fmt::UpperExp	Igual, E maiúsculo
?	{:#{?}}	std::fmt::Debug	Visualização de depuração, para desenvolvedores
p	{:p}	std::fmt::Pointer	Ponteiro como endereço, para desenvolvedores

Quando você coloca o atributo `#[derive(Debug)]` em uma definição de tipo para utilizar o parâmetro de formato `{:?}`, está simplesmente pedindo ao Rust que implemente o trait `std::fmt::Debug` para você.

Todos os traits de formatação têm a mesma estrutura, diferindo apenas no nome. Vamos utilizar `std::fmt::Display` como um exemplo:

```
trait Display {  
    fn fmt(&self, dest: &mut std::fmt::Formatter)  
        -> std::fmt::Result;  
}
```

O trabalho do método `fmt` é produzir uma representação formatada corretamente de `self` e gravar seus caracteres em `dest`. Além de servir como um fluxo de saída, o argumento `dest` também carrega detalhes analisados do parâmetro de formato, como o alinhamento e a largura mínima do campo.

Por exemplo, no início deste capítulo, sugerimos que seria bom se

valores `Complex` fossem impressos na forma usual $a + bi$. Eis uma implementação de `Display` que faz isso:

```
use std::fmt;

impl fmt::Display for Complex {
    fn fmt(&self, dest: &mut fmt::Formatter) -> fmt::Result {
        let im_sign = if self.im < 0.0 { '-' } else { '+' };
        write!(dest, "{} {} {}i", self.re, im_sign, f64::abs(self.im))
    }
}
```

Isso aproveita o fato de que `Formatter` é, ele próprio, um fluxo de saída, então a macro `write!` pode fazer a maior parte do trabalho para nós. Com essa implementação em vigor, podemos escrever o seguinte:

```
let one_twenty = Complex { re: -0.5, im: 0.866 };
assert_eq!(format!("{}", one_twenty),
           "-0.5 + 0.866i");

let two_forty = Complex { re: -0.5, im: -0.866 };
assert_eq!(format!("{}", two_forty),
           "-0.5 - 0.866i");
```

Às vezes é útil exibir números complexos na forma polar: se você imaginar uma linha desenhada no plano complexo desde a origem até o número, a forma polar fornecerá o comprimento da linha e seu ângulo no sentido horário em relação ao eixo x positivo. O caractere `#` em um parâmetro de formato normalmente seleciona alguma forma de exibição alternativa; a implementação de `Display` poderia tratá-la como uma solicitação para utilizar a forma polar:

```
impl fmt::Display for Complex {
    fn fmt(&self, dest: &mut fmt::Formatter) -> fmt::Result {
        let (re, im) = (self.re, self.im);
        if dest.alternate() {
            let abs = f64::sqrt(re * re + im * im);
            let angle = f64::atan2(im, re) / std::f64::consts::PI * 180.0;
            write!(dest, "{} ∠ {}°", abs, angle)
        } else {
            let im_sign = if im < 0.0 { '-' } else { '+' };
            write!(dest, "{} {} {}i", re, im_sign, f64::abs(im))
        }
    }
}
```

Utilizando essa implementação:

```
let ninety = Complex { re: 0.0, im: 2.0 };
assert_eq!(format!("{}", ninety),
           "0 + 2i");
assert_eq!(format!("{:?}", ninety),
           "2 ∠ 90°");
```

Embora os métodos `fmt` dos traits de formatação retornem um valor `fmt::Result` (um típico tipo `Result` específico do módulo), você deve propagar apenas falhas de operações no `Formatter`, enquanto a implementação de `fmt::Display` faz com suas chamadas a `write!`; suas funções de formatação nunca devem originar erros. Isso permite que macros como `format!` simplesmente retornem uma `String` em vez de um `Result<String, ...>`, pois anexar o texto formatado a uma `String` nunca falha. Também garante que quaisquer erros que você obtenha de `write!` ou `writeln!` refletem problemas reais do fluxo de E/S subjacente, não problemas de formatação.

`Formatter` tem muitos outros métodos úteis, incluindo alguns para lidar com dados estruturados como mapas, listas etc., que não abordaremos aqui; consulte a documentação on-line para obter todos os detalhes.

Usando a linguagem de formatação em seu próprio código

Você pode escrever as próprias funções e macros que aceitam templates de formato e argumentos utilizando a macro `format_args!` e o tipo `std::fmt::Arguments` do Rust. Por exemplo, suponha que seu programa precise registrar mensagens de status enquanto é executado e você quisesse utilizar a linguagem de formatação de texto do Rust para produzi-las. O programa seguinte seria um começo:

```
fn logging_enabled() -> bool { ... }

use std::fs::OpenOptions;
use std::io::Write;

fn write_log_entry(entry: std::fmt::Arguments) {
    if logging_enabled() {
```

```

// Mantenha as coisas simples por enquanto e apenas
// abra o arquivo a cada vez.
let mut log_file = OpenOptions::new()
    .append(true)
    .create(true)
    .open("log-file-name")
    .expect("failed to open log file");

log_file.write_fmt(entry)
    .expect("failed to write to log");
}
}

```

Você pode chamar `write_log_entry` assim:

```
write_log_entry(format_args!("Hark! {:?}\n", mysterious_value));
```

Em tempo de compilação, a macro `format_args!` analisa a string do template e a compara com os tipos de argumentos, relatando um erro se houver algum problema. Em tempo de execução, ele avalia os argumentos e constrói um valor `Arguments` carregando todas as informações necessárias para formatar o texto: uma forma pré-analisada do template, com referências compartilhadas aos valores do argumento.

Construir um valor de `Arguments` é barato: é só juntar alguns ponteiros. Nenhum trabalho de formatação ocorre ainda, apenas a coleta das informações necessárias para fazê-lo posteriormente. Isso pode ser importante: se o log não estiver ativado, qualquer tempo gasto convertendo números em decimais, preenchendo valores e assim por diante será desperdiçado.

O tipo `File` implementa o trait `std::io::Write`, cujo método `write_fmt` recebe um `Argument` e faz a formatação. Ele grava os resultados no fluxo subjacente.

Essa chamada a `write_log_entry` não é elegante. É aqui que uma macro pode ajudar:

```

macro_rules! log { // o ! não é necessário após o nome nas definições de macro
    ($format:tt, $($arg:expr),*) => (
        write_log_entry(format_args!($format, $($arg),*))
    )
}

```

Cobrimos macros em detalhes no Capítulo 21. Por enquanto,

acredite que isso define uma nova macro `log!` que passa seus argumentos para `format_args!` e depois chama sua função `write_log_entry` no valor `Arguments` resultante. As macros de formatação como `println!`, `writeln!` e `format!` têm mais ou menos a mesma ideia.

Você pode utilizar `log!` assim:

```
log!("O day and night, but this is wondrous strange! {:?}\n",
    mysterious_value);
```

Idealmente, isso parece um pouco melhor.

Expressões regulares

O crate `regex` externo é a biblioteca oficial de expressões regulares do Rust. Ele fornece as funções usuais de pesquisa e correspondência de padrões. Ele tem um bom suporte para Unicode, mas também pode pesquisar strings de bytes. Embora não suporte alguns recursos que você encontrará com frequência em outros pacotes de expressões regulares, como referências inversas e padrões *look-around*, essas simplificações permitem que `regex` garanta que as pesquisas tenham tempo linear no tamanho da expressão e no comprimento do texto que está sendo pesquisado. Essas garantias, entre outras, tornam `regex` seguro de utilizar mesmo com expressões não confiáveis pesquisando texto não confiável.

Neste livro, forneceremos apenas uma visão geral de `regex`; você deve consultar a documentação on-line para obter detalhes.

Apesar de o crate `regex` não estar em `std`, é mantido pela equipe da biblioteca do Rust, o mesmo grupo responsável por `std`. Para utilizar `regex`, coloque a seguinte linha na seção `[dependencies]` do arquivo *Cargo.toml* do seu crate:

```
regex = "1"
```

Nas seções a seguir, presumiremos que você tenha feito essa alteração.

Uso básico de Regex

Um valor `Regex` representa uma expressão regular analisada pronta para uso. O construtor `Regex::new` tenta analisar uma `&str` como uma

expressão regular e retorna um `Result`:

```
use regex::Regex;

// Um número de versão semver, como 0.2.1.
// Pode conter um sufixo de versão de pré-lançamento, como 0.2.1-alpha.
// (Sem sufixo de metadados de versão, para abreviar.)
//
// Observe o uso da sintaxe de string bruta r"...", para evitar
// excesso de barras invertidas
let semver = Regex::new(r"(\d+)\.(\d+)\.(\d+)(-[:alnum:]*)?")?;

// Pesquisa simples, com resultado booleano
let haystack = r#"regex = "0.2.5"#"#;
assert!(semver.is_match(haystack));
```

O método `Regex::captures` procura uma string pela primeira correspondência e retorna um valor `regex::Captures` contendo informações de correspondência para cada grupo na expressão:

```
// Você pode recuperar grupos de captura
let captures = semver.captures(haystack)
    .ok_or("semver regex should have matched")?;
assert_eq!(&captures[0], "0.2.5");
assert_eq!(&captures[1], "0");
assert_eq!(&captures[2], "2");
assert_eq!(&captures[3], "5");
```

Indexar um valor `Captures` gera pânico se o grupo solicitado não corresponder. Para testar se um determinado grupo correspondeu, você pode chamar `Captures::get`, que retorna um `Option<regex::Match>`. Um valor `Match` registra a correspondência de um único grupo:

```
assert_eq!(captures.get(4), None);
assert_eq!(captures.get(3).unwrap().start(), 13);
assert_eq!(captures.get(3).unwrap().end(), 14);
assert_eq!(captures.get(3).unwrap().as_str(), "5");
```

Você pode iterar por todas as correspondências em uma string:

```
let haystack = "In the beginning, there was 1.0.0. \
    For a while, we used 1.0.1-beta, \
    but in the end, we settled on 1.2.4.";

let matches: Vec<&str> = semver.find_iter(haystack)
    .map(|match_| match_.as_str())
    .collect();
assert_eq!(matches, vec!["1.0.0", "1.0.1-beta", "1.2.4"]);
```

O iterador `find_iter` produz um valor `Match` para cada correspondência

não sobreposta da expressão, trabalhando do início ao fim da string. O método `captures_iter` é semelhante, mas produz valores `Captures` registrando todos os grupos de captura. A pesquisa é mais lenta quando os grupos de captura devem ser gerados; portanto, se você não precisar deles, é melhor utilizar um dos métodos que não os retornam.

Construindo valores regex preguiçosamente (lazily)

O construtor `Regex::new` pode ser caro: construir um `Regex` para uma expressão regular de 1.200 caracteres pode levar quase um milissegundo em uma máquina de desenvolvedor rápida e mesmo uma expressão trivial recebe microssegundos. É melhor manter a construção `Regex` fora de loops computacionais pesados; em vez disso, você deve construir seu `Regex` uma vez e depois reutilizá-lo.

O crate `lazy_static` fornece uma boa maneira de construir valores estáticos preguiçosamente na primeira vez em que são utilizados. Para começar, observe a dependência em seu arquivo *Cargo.toml*:

```
[dependencies]
lazy_static = "1"
```

Esse crate fornece uma macro para declarar tais variáveis:

```
use lazy_static::lazy_static;

lazy_static! {
    static ref SEMVER: Regex
        = Regex::new(r"(\d+)\.(\d+)\.(\d+)(-[-[:alnum:]]*)?")
            .expect("error parsing regex");
}
```

A macro se expande para uma declaração de uma variável estática chamada `SEMVER`, mas seu tipo não é exatamente `Regex`. Em vez disso, é um tipo gerado por macro que implementa `Deref<Target=Regex>` e, portanto, expõe todos os mesmos métodos como um `Regex`. A primeira vez que `SEMVER` é desreferenciado, o inicializador é avaliado e o valor é salvo para uso posterior. Como `SEMVER` é uma variável estática, não apenas uma variável local, o inicializador é executado no máximo uma vez por execução do programa.

Com essa declaração em vigor, o uso de SEMVER é simples e direto:

```
use std::io::BufRead;

let stdin = std::io::stdin();
for line_result in stdin.lock().lines() {
    let line = line_result?;
    if let Some(match_) = SEMVER.find(&line) {
        println!("{}", match_.as_str());
    }
}
```

Você pode colocar a declaração `lazy_static!` em um módulo, ou mesmo dentro da função que utiliza o `Regex`, se esse for o escopo mais apropriado. A expressão regular ainda é sempre compilada apenas uma vez por execução do programa.

Normalização

A maioria dos usuários consideraria a palavra francesa para chá, thé, com três caracteres. Contudo, o Unicode na verdade tem duas maneiras de representar esse texto:

- Na forma *composta*, "thé" contém os três caracteres 't', 'h' e 'é', em que 'é' é um único caractere Unicode com ponto de código 0xe9.
- Na forma *decomposta*, "thé" compreende os quatro caracteres 't', 'h', 'e' e '\u{301}', em que o 'e' é o caractere ASCII simples, sem acento, e o ponto de código 0x301 é o caractere "COMBINING ACUTE ACCENT" (acento agudo combinado), que adiciona um acento agudo a qualquer caractere antes dele.

O Unicode não considera nem a forma composta nem a decomposta de "é" como a forma "correta"; em vez disso, considera ambas representações equivalentes do mesmo caráter abstrato. O Unicode diz que ambas as formas devem ser exibidas da mesma maneira e os métodos de entrada de texto podem produzir qualquer uma delas; portanto, os usuários geralmente não saberão qual forma estão vendo ou digitando. (O Rust permite que você utilize caracteres Unicode diretamente em literais string, então pode simplesmente escrever "thé" se não se importar com qual codificação vai obter. Aqui utilizaremos o escape `\u` para maior clareza.)

Mas, considerados como valores `&str` ou `String` do Rust, `"th\u{e9}"` e `"the\u{301}"` são completamente distintos. Eles têm comprimentos diferentes, são comparados como desiguais, têm valores hash diferentes e são ordenados de maneira diferente em relação a outras strings:

```
assert!("th\u{e9}" != "the\u{301}");
assert!("th\u{e9}" > "the\u{301}");

// Um Hasher é projetado para acumular o hash de uma série de valores,
// portanto o hashing é apenas um pouco desajeitado.
use std::hash::{Hash, Hasher};
use std::collections::hash_map::DefaultHasher;
fn hash<T: ?Sized + Hash>(t: &T) -> u64 {
    let mut s = DefaultHasher::new();
    t.hash(&mut s);
    s.finish()
}

// Esses valores podem mudar em versões futuras do Rust
assert_eq!(hash("th\u{e9}"), 0x53e2d0734eb1dff3);
assert_eq!(hash("the\u{301}"), 0x90d837f0a0928144);
```

Claramente, se você pretende comparar o texto fornecido pelo usuário ou utilizá-lo como uma chave em uma tabela hash ou árvore B, precisará primeiro colocar cada string em alguma forma canônica.

Felizmente, o Unicode especifica formas *normalizadas* para strings. Sempre que duas strings devem ser tratadas como equivalentes de acordo com as regras do Unicode, suas formas normalizadas são idênticas caractere por caractere. Quando codificados com UTF-8, eles são idênticos byte por byte. Isso significa que você pode comparar strings normalizadas com `==`, utilizá-las como chaves em um `HashMap` ou `HashSet` e assim por diante e obterá a noção de igualdade do Unicode.

A falha na normalização pode até ter consequências de segurança. Por exemplo, se seu site normaliza nomes de usuários em alguns casos, mas não em outros, você pode acabar com dois usuários distintos chamados bananasflambé, que algumas partes do seu código tratam como o mesmo usuário, mas outras o distinguem, resultando na extensão incorreta dos privilégios de um para o outro. É claro que

existem muitas maneiras de evitar esse tipo de problema, mas a história mostra que também há muitas maneiras de não o fazer.

Formas de normalização

O Unicode define quatro formas normalizadas, cada uma delas apropriada para diferentes usos. Há duas perguntas a responder:

- Primeiro, você prefere que os caracteres sejam tão *compostos* quanto possível ou tão *decompostos* quanto possível?

Por exemplo, a representação mais composta da palavra vietnamita *Pho* é a string de três caracteres `"Ph\u{1edf}"`, em que tanto a marca tonal `'` e a marca da vogal `'` são aplicados ao caractere de base `"o"` em um único caractere Unicode, `\u{1edf}`, que o Unicode diligentemente nomeia como LATIN SMALL LETTER O WITH HORN AND HOOK ABOVE (letra latina o minúscula com chifre e gancho acima).

A representação mais decomposta divide a letra base e suas duas marcas em três caracteres Unicode separados: `'o'`, `\u{31b}'` (COMBINING HORN – chifre combinado) e `\u{309}'` (COMBINING HOOK ABOVE – gancho acima combinado), resultando em `"Pho\u{31b}\u{309}"`. (Sempre que as marcas de combinação aparecem como caracteres separados, em vez de fazer parte de um caractere composto, todas as formas normalizadas especificam uma ordem fixa na qual devem aparecer; portanto, a normalização é bem especificada mesmo quando os caracteres têm vários acentos.)

A forma composta geralmente tem menos problemas de compatibilidade, uma vez que corresponde mais às representações que a maioria dos idiomas utilizava para seu texto antes de o Unicode se estabelecer. Também pode funcionar melhor com recursos de formatação de string ingênuos, como a macro `format!` do Rust. A forma decomposta, por outro lado, pode ser melhor para exibição de texto ou pesquisa, pois torna a estrutura detalhada do texto mais explícita.

- A segunda pergunta é: se duas sequências de caracteres

representam o mesmo texto fundamental, mas diferem na forma como o texto deve ser formatado, você deseja tratá-los como equivalentes ou mantê-los distintos?

O Unicode possui caracteres separados para o dígito comum 5, o dígito sobrescrito ⁵ (ou `\u{2075}`) e o dígito circulado ⑤ (ou `\u{2464}`), mas declara que todos os três são *equivalentes de compatibilidade*. Da mesma forma, o Unicode possui um único caractere para a ligadura *ffi* (`\u{fb03}`), mas declara que isso é o equivalente de compatibilidade à sequência de três caracteres *ffi*.

A equivalência de compatibilidade faz sentido para pesquisas: uma pesquisa por "difficult", utilizando apenas caracteres ASCII, deve corresponder à string `di\u{fb03}cult`, que utiliza a ligadura *ffi*. A aplicação da decomposição de compatibilidade à última string substituiria a ligadura pelas três letras simples "ffi", facilitando a busca. Mas a normalização do texto para uma forma equivalente de compatibilidade pode perder informações essenciais, portanto não deve ser aplicada de forma descuidada. Por exemplo, seria incorreto na maioria dos contextos armazenar "2⁵" Como "25".

Unicode Normalization Form C e Normalization Form D (NFC e NFD) utilizam as formas maximamente compostas e maximamente decompostas de cada caractere, mas não tentam unificar sequências equivalentes de compatibilidade. As formas de normalização NFKC e NFKD são como NFC e NFD, mas normalizam todas as sequências de equivalentes de compatibilidade para algum representante simples de sua classe.

O "Modelo de caracteres para a World Wide Web" do World Wide Web Consortium recomenda o uso de NFC para todo o conteúdo. O anexo "Identificador Unicode e sintaxe de padrão" sugere o uso de NFKC para identificadores em linguagens de programação e oferece princípios para adaptar a forma quando necessário.

Crate de normalização Unicode

O crate `unicode-normalization` do Rust fornece um trait que adiciona métodos a `&str` para colocar o texto em qualquer uma das quatro

formas normalizadas. Para utilizá-lo, adicione a seguinte linha à seção [dependencies] do seu arquivo *Cargo.toml*:

```
unicode-normalization = "0.1.17"
```

Com essa declaração, um `&str` tem quatro novos métodos que retornam iteradores sobre uma forma normalizada específica da string:

```
use unicode_normalization::UnicodeNormalization;

// Não importa qual representação a string da esquerda utilize
// (você não deve ser capaz de dizer apenas olhando),
// essas asserções vão se sustentar
assert_eq!("Pho".nfd().collect::<String>(), "Pho\u{31b}\u{309}");
assert_eq!("Pho".nfc().collect::<String>(), "Ph\u{1edf}");

// O lado esquerdo aqui utiliza o caractere de ligadura "ffi"
assert_eq!("① Di\u{fb03}culty".nfkc().collect::<String>(), "1 Difficulty");
```

Pegar uma string normalizada e normalizá-la novamente na mesma forma garante o retorno de um texto idêntico.

Embora qualquer substring de uma string normalizada seja normalizada, a concatenação de duas strings normalizadas não é necessariamente normalizada: por exemplo, a segunda string pode começar com a combinação de caracteres que deve ser colocada antes da combinação de caracteres no final da primeira string.

Contanto que um texto não utilize pontos de código não atribuídos quando for normalizado, o Unicode promete que sua forma normalizada não será alterada em versões futuras do padrão. Isso significa que as formas normalizadas geralmente são seguras para uso em armazenamento persistente, mesmo com a evolução do padrão Unicode.

¹ N.R.: Ponto de código é um termo usado na definição do Unicode para diferenciar um grafema de sua representação numérica. Foi uma solução encontrada para desvincular o caractere de seu valor numérico, que pode variar dependendo de como este é codificado em diferentes formatos.

CAPÍTULO 18

Entrada e saída

Doolittle: Que evidências concretas tem de que você existe?

Bomba nº 20: Hummm...bem... Penso, logo existo.

Doolittle: Isso é bom. Isso é muito bom. Mas como você sabe que qualquer outra coisa existe?

Bomba nº 20: Meu aparelho sensorial revela isso para mim.

– Dark Star

Recursos da biblioteca padrão do Rust para entrada e saída são organizados em torno de três traits, `Read`, `BufRead` e `Write`:

- Valores que implementam `Read` têm métodos para entrada orientada a bytes. São chamados *readers* (leitores).
- Valores que implementam `BufRead` são leitores *buffered*. Eles suportam todos os métodos de `Read`, além de métodos para ler linhas de texto etc.
- Valores que implementam `Write` suportam saída de texto orientada a bytes e UTF-8. São chamados *writers* (escritores).

A Figura 18.1 mostra esses três traits e alguns exemplos dos tipos leitor e escritor.

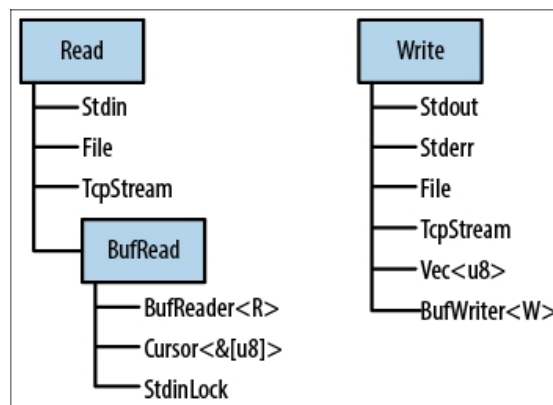


Figura 18.1: Principais traits de E/S do Rust e os tipos selecionados

que os implementam.

Neste capítulo, explicaremos como utilizar esses traits e seus métodos, discutiremos os tipos de leitor e escritor/escritor mostrados na figura e mostraremos outras formas de interagir com arquivos, o terminal e a rede.

Leitores e escritores

Leitores são valores dos quais o programa pode ler bytes. Exemplos incluem:

- Arquivos abertos utilizando `std::fs::File::open(filename)`
- `std::net::TcpStreamS`, para receber dados pela rede
- `std::io::stdin()`, para leitura do fluxo de entrada padrão do processo
- `std::io::Cursor<&[u8]>` e valores `std::io::Cursor<Vec<u8>>`, que são leitores que “leem” de um array de bytes ou vetor que já está na memória

Escritores são valores nos quais seu programa pode escrever (ou enviar) bytes. Exemplos incluem:

- Arquivos abertos utilizando `std::fs::File::create(filename)`
- `std::net::TcpStreamS`, para enviar dados pela rede
- `std::io::stdout()` e `std::io::stderr()`, para escrever no terminal
- `Vec<u8>`, um escritor cujos métodos `write` anexam ao vetor
- `std::io::Cursor<Vec<u8>>`, que é semelhante, mas permite ler e escrever dados e acessar diferentes posições dentro do vetor
- `std::io::Cursor<&mut [u8]>`, que é muito parecido com `std::io::Cursor<Vec<u8>>`, exceto que não pode aumentar o tamanho do buffer, pois é apenas uma fatia de algum array de bytes existente

Como existem traits padrão para leitores e escritores (`std::io::Read` e `std::io::Write`), é bastante comum escrever código genérico que funciona em uma variedade de canais de entrada ou saída. Por exemplo, eis uma função que copia todos os bytes de qualquer leitor para qualquer escritor:

```
use std::io::{self, Read, Write, ErrorKind};

const DEFAULT_BUF_SIZE: usize = 8 * 1024;
```

```

pub fn copy<R: ?Sized, W: ?Sized>(reader: &mut R, writer: &mut W)
-> io::Result<u64>
where R: Read, W: Write
{
    let mut buf = [0; DEFAULT_BUF_SIZE];
    let mut written = 0;
    loop {
        let len = match reader.read(&mut buf) {
            Ok(0) => return Ok(written),
            Ok(len) => len,
            Err(ref e) if e.kind() == ErrorKind::Interrupted => continue,
            Err(e) => return Err(e),
        };
        writer.write_all(&buf[..len])?;
        written += len as u64;
    }
}

```

Essa é a implementação de `std::io::copy()` da biblioteca padrão do Rust. Por ser genérica, você pode usá-la para copiar dados de um `File` para um `TcpStream`, de `Stdin` para um `Vec<u8>` na memória etc.

Se o código de tratamento de erros aqui não estiver claro, revise o Capítulo 7. Utilizaremos o tipo `Result` constantemente nas próximas páginas; é importante ter uma boa noção de como ele funciona.

Os três traits `std::io::Read`, `BufRead` e `Write`, com `Seek`, são tão comumente utilizados que há um módulo `prelude` contendo apenas esses traits:

```
use std::io::prelude::*;
```

Veremos isso uma ou duas vezes neste capítulo. O hábito é também importar o próprio módulo `std::io`:

```
use std::io::{self, Read, Write, ErrorKind};
```

A palavra-chave `self` aqui declara `io` como um aliás para o módulo `std::io`. Dessa maneira, `std::io::Result` e `std::io::Error` podem ser escritos de forma mais concisa como `io::Result` e `io::Error` e assim por diante.

Leitores

`std::io::Read` tem vários métodos para ler dados. Todos eles recebem o próprio leitor por referência `mut`.

```
reader.read(&mut buffer)
```

Lê alguns bytes da fonte de dados e os armazena no `buffer`

determinado. O tipo do argumento `buffer` é `&mut [u8]`. Isso lê até `buffer.len()` bytes.

O tipo de retorno é `io::Result<u64>`, que é um alias de tipo para `Result<u64, io::Error>`. Se bem-sucedido, o valor `u64` é o número de bytes lidos – que pode ser igual ou menor que `buffer.len()`, mesmo que haja mais dados por vir, conforme o capricho da fonte de dados. `Ok(0)` significa que não há mais entrada a ler.

Em caso de erro, `.read()` retorna `Err(err)`, em que `err` é um valor `io::Error`. Um `io::Error` é imprimível, para benefício dos humanos; para programas, tem um método `.kind()` que retorna um código de erro do tipo `io::ErrorKind`. Os membros desse enum têm nomes como `PermissionDenied` e `ConnectionReset`. A maioria indica erros graves que não podem ser ignorados, mas um tipo de erro deve ser tratado de forma especial. `io::ErrorKind::Interrupted` corresponde ao código de erro Unix `EINTR`, o que significa que a leitura foi interrompida por um sinal. A menos que o programa seja projetado para fazer algo inteligente com os sinais, ele deve apenas tentar novamente a leitura. O código para `copy()`, na seção anterior, mostra um exemplo disso.

Como podemos ver, o método `.read()` é de nível muito baixo, até mesmo herda peculiaridades do sistema operacional subjacente. Se você estiver implementando o trait `Read` para um novo tipo de fonte de dados, isso lhe dá muita margem de manobra. Se você estiver tentando ler alguns dados, é um problema. Portanto, o Rust fornece vários métodos de conveniência de alto nível. Todos eles têm implementações padrão em termos de `.read()`. Todos eles tratam `ErrorKind::Interrupted`, de modo que você não precise tratar.

```
reader.read_to_end(&mut byte_vec)
```

Lê todas as entradas restantes desse leitor, anexando-as a `byte_vec`, que é um `Vec<u8>`. Retorna um `io::Result<usize>`, o número de bytes lidos.

Não há limite quanto à quantidade de dados que esse método armazenará no vetor, portanto não o utilize em uma fonte não confiável. (Você pode impor um limite utilizando o método `.take()`, descrito na lista a seguir.)

`reader.read_to_string(&mut string)`

Isso é a mesma coisa, mas anexa os dados à `String` determinada. Se o fluxo não for UTF-8 válido, isso retornará um erro `ErrorKind::InvalidData`.

Em algumas linguagens de programação, a entrada de bytes e a entrada de caracteres são tratadas por tipos diferentes. Atualmente, o UTF-8 é tão dominante que o Rust reconhece esse padrão de fato e suporta UTF-8 em todos os lugares. Outros conjuntos de caracteres são suportados com o crate de código-fonte aberto `encoding`.

`reader.read_exact(&mut buf)`

Lê exatamente dados suficientes para preencher o buffer fornecido. O tipo de argumento é `&[u8]`. Se o leitor ficar sem dados antes de ler `buf.len()` bytes, isso retornará um erro `ErrorKind::UnexpectedEof`.

Esses são os principais métodos do trait `Read`. Além disso, existem três métodos adaptadores que utilizam o `reader` por valor, transformando-o em um iterador ou outro leitor:

`reader.bytes()`

Retorna um iterador sobre os bytes do fluxo de entrada. O tipo de item é `io::Result<u8>`, portanto uma verificação de erro é necessária para cada byte. Além disso, isso chama `reader.read()` uma vez por byte, o que será muito ineficiente se o leitor não estiver armazenado em um buffer.

`reader.chain(reader2)`

Retorna um novo leitor que produz todas as entradas de `reader`, seguidas por todas as entradas de `reader2`.

`reader.take(n)`

Retorna um novo leitor que lê da mesma fonte que `reader`, mas limita-se a `n` bytes de entrada.

Não há método para fechar um leitor. Leitores e escritores normalmente implementam `Drop` para que sejam fechados automaticamente.

Leitores bufferizados

Para eficiência, os leitores e escritores podem ser armazenados em buffer, o que significa simplesmente que eles têm um fragmento da memória (um buffer) que contém alguns dados de entrada ou saída na memória. Isso economiza chamadas de sistema, como mostrado na Figura 18.2. O aplicativo lê os dados do `BufReader`, nesse exemplo chamando o método `.read_line()`. O `BufReader` por sua vez obtém a entrada em blocos maiores do sistema operacional.

Essa imagem não está na escala. O tamanho padrão real de um buffer de `BufReader` é de vários kilobytes, então um único `read` do sistema pode atender a centenas de chamadas `.read_line()`. Isso é importante porque as chamadas de sistema são lentas.

(Como mostra a figura, o sistema operacional também possui um buffer, pelo mesmo motivo: as chamadas de sistema são lentas, mas a leitura de dados de um disco é mais lenta ainda.)

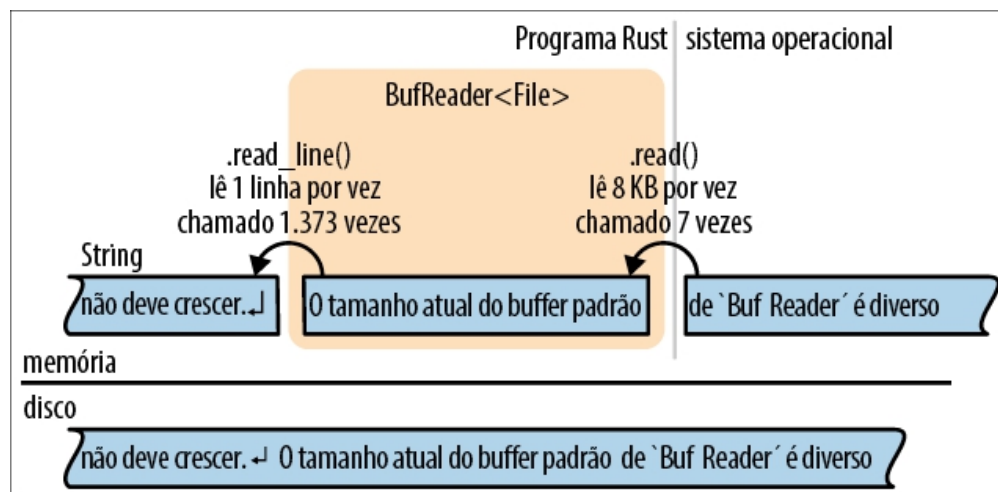


Figura 18.2: Um leitor de arquivo armazenado em buffer.

Leitores armazenados em buffer implementam tanto `Read` como um segundo trait, `BufRead`, que adiciona os seguintes métodos:

```
reader.read_line(&mut line)
```

Lê uma linha de texto e a anexa a `line`, que é uma `String`. O caractere de nova linha `\n` no final da linha é incluído em `line`. Se a entrada tiver terminações de linha no estilo do Windows, `"r\n"`, ambos os caracteres serão incluídos em `line`.

O valor de retorno é um `io::Result<usize>`, o número de bytes lidos,

incluindo o final da linha, se houver algum.

Se o leitor está no final da entrada, isso deixa `line` inalterado e retorna `Ok(0)`.

```
reader.lines()
```

Retorna um iterador nas linhas da entrada. O tipo de item é `io::Result<String>`. Os caracteres de nova linha não são incluídos nas strings. Se a entrada tiver terminações de linha no estilo do Windows, `"\r\n"`, ambos os caracteres serão removidos.

Esse método é quase sempre o que você quer para entrada de texto. As próximas duas seções mostram alguns exemplos de seu uso.

```
reader.read_until(stop_byte, &mut byte_vec), reader.split(stop_byte)
```

Esses são como `.read_line()` e `.lines()`, mas orientados a bytes, produzindo `Vec<u8>` em vez de `String`. Você escolhe o delimitador `stop_byte`.

`BufRead` também fornece um par de métodos de baixo nível `.fill_buf()` e `.consume(n)`, para acesso direto ao buffer interno do leitor. Para saber mais sobre esses métodos, consulte a documentação on-line.

As próximas duas seções abordam os leitores armazenados em buffer em mais detalhes.

Lendo linhas

Eis uma função que implementa o utilitário `grep` do Unix. Ela pesquisa muitas linhas de texto, normalmente redirecionado de outro comando, para uma determinada string:

```
use std::io;
use std::io::prelude::*;

fn grep(target: &str) -> io::Result<()> {
    let stdin = io::stdin();
    for line_result in stdin.lock().lines() {
        let line = line_result?;
        if line.contains(target) {
            println!("{}", line);
        }
    }
    Ok(())
}
```

```
}
```

Como queremos chamar `.lines()`, precisamos de uma fonte de entrada que implemente `BufRead`. Nesse caso, chamamos `io::stdin()` para obter os dados que estão sendo redirecionados para nós. Entretanto, a biblioteca padrão do Rust protege `stdin` com um mutex. Chamamos `.lock()` para bloquear `stdin` para uso exclusivo pelo thread atual; retornando um valor `StdinLock` que implementa `BufRead`. No final do loop, o `StdinLock` é dropado, liberando o mutex. (Sem um mutex, dois threads tentando ler de `stdin` ao mesmo tempo causariam um comportamento indefinido. O C tem o mesmo problema e o soluciona da mesma maneira: todas as funções de entrada e saída padrão do C obtêm um bloqueio nos bastidores. A única diferença é que no Rust o bloqueio é parte da API.)

O restante da função é simples: chama `.lines()` e faz um loop sobre o iterador resultante. Como esse iterador produz valores `Result`, utilizamos o operador `?` para verificar se há erros.

Suponha que queremos levar nosso programa `grep` um passo adiante e adicionar suporte para pesquisar arquivos no disco. Podemos tornar essa função genérica:

```
fn grep<R>(target: &str, reader: R) -> io::Result<()>
    where R: BufRead
{
    for line_result in reader.lines() {
        let line = line_result?;
        if line.contains(target) {
            println!("{}", line);
        }
    }
    Ok(())
}
```

Agora podemos passar para ela um `StdinLock` ou um `File` armazenado em buffer:

```
let stdin = io::stdin();
grep(&target, stdin.lock())?; // ok

let f = File::open(file)?;
grep(&target, BufReader::new(f))?; // também ok
```

Observe que um `File` não é automaticamente armazenado em buffer.

File implementa `Read`, mas não `BufRead`. Contudo, é fácil criar um leitor armazenado em buffer para um `File`, ou qualquer outro leitor não armazenado em buffer. `BufReader::new(reader)` faz isso. (Para definir o tamanho do buffer, utilize `BufReader::with_capacity(size, reader)`.)

Na maioria das linguagens, os arquivos são armazenados em buffer por padrão. Se você deseja entrada ou saída não armazenada em buffer, precisa descobrir como desativar o armazenamento em buffer. No Rust, `File` e `BufReader` são dois recursos de biblioteca distintos, porque às vezes você quer arquivos sem buffer e às vezes quer armazenar em buffer sem arquivos (por exemplo, pode querer armazenar em buffer a entrada da rede).

O programa completo, incluindo tratamento de erros e algumas análises grosseiras de argumentos, é mostrado aqui:

```
// grep - Pesquisa em stdin ou em alguns arquivos linhas
// que correspondem a uma determinada string

use std::error::Error;
use std::io::{self, BufReader};
use std::io::prelude::*;
use std::fs::File;
use std::path::PathBuf;

fn grep<R>(target: &str, reader: R) -> io::Result<()>
    where R: BufRead
{
    for line_result in reader.lines() {
        let line = line_result?;
        if line.contains(target) {
            println!("{}", line);
        }
    }
    Ok(())
}

fn grep_main() -> Result<(), Box<dyn Error>> {
    // Obtenha os argumentos de linha de comando. O primeiro argumento
    // é a string a procurar; os restantes são nomes de arquivo
    let mut args = std::env::args().skip(1);
    let target = match args.next() {
        Some(s) => s,
        None => Err("usage: grep PATTERN FILE...")?
    };
    let files: Vec<PathBuf> = args.map(PathBuf::from).collect();
```

```

    if files.is_empty() {
        let stdin = io::stdin();
        grep(&target, stdin.lock())?;
    } else {
        for file in files {
            let f = File::open(file)?;
            grep(&target, BufReader::new(f))?;
        }
    }
    Ok(())
}

fn main() {
    let result = grep_main();
    if let Err(err) = result {
        eprintln!("{}", err);
        std::process::exit(1);
    }
}

```

Coletando linhas

Vários métodos de leitores, incluindo `.lines()`, retornam iteradores que produzem valores `Result`. A primeira vez que você quiser coletar todas as linhas de um arquivo em um grande vetor, terá problemas para se livrar do `Results`:

```

// ok, mas é não o que você quer
let results: Vec<io::Result<String>> = reader.lines().collect();

// erro: não é possível converter a coleção de Results em Vec<String>
let lines: Vec<String> = reader.lines().collect();

```

A segunda tentativa não compila: o que aconteceria com os erros? A solução simples é escrever um loop `for` e verificar em cada item os erros:

```

let mut lines = vec![];
for line_result in reader.lines() {
    lines.push(line_result?);
}

```

Nada mal; mas seria bom utilizar `.collect()` aqui e acontece que podemos. Só temos de saber qual tipo solicitar:

```

let lines = reader.lines().collect::<io::Result<Vec<String>>>()?;

```

Como isso funciona? A biblioteca padrão contém uma

implementação de `FromIterator` para `Result` – fácil de ignorar na documentação on-line – que torna isso possível:

```
impl<T, E, C> FromIterator<Result<T, E>> for Result<C, E>
  where C: FromIterator<T>
{
  ...
}
```

Isso requer uma leitura cuidadosa, mas é um truque interessante. Suponha que `c` é qualquer tipo de coleção, como `Vec` ou `HashSet`. Desde que saibamos como construir um `c` a partir de um iterador de valores `T`, podemos construir um `Result<C, E>` a partir de um iterador produzindo valores `Result<T, E>`. Só precisamos extrair valores do iterador e construir a coleção a partir dos resultados `Ok`, mas, se alguma vez um `Err` é visto, pare e passe adiante.

Em outras palavras, `io::Result<Vec<String>>` é um tipo de coleção, então o método `.collect()` pode criar e preencher valores desse tipo.

Escritores

Como vimos, a entrada é principalmente produzida utilizando métodos. A saída é um pouco diferente.

Ao longo do livro, utilizamos `println!()` para produzir saída de texto simples:

```
println!("Hello, world!");

println!("The greatest common divisor of {:?} is {}", numbers, d);

println(); // imprime uma linha em branco
```

Também há uma macro `print!()`, que não adiciona um caractere de nova linha ao final e as macros `eprintln!` e `eprint!` que escrevem no fluxo de erros padrão. Os códigos de formatação para todas elas são os mesmos das macro `format!`, descritas em “Formatando valores”, na página 515.

Para enviar a saída para um escritor, utilize o `write!()` e macros `writeln!()`. Elas são os mesmos que `print!()` e `println!()`, exceto por duas diferenças:

```
writeln!(io::stderr(), "error: world not helloable");

writeln!(&mut byte_vec, "The greatest common divisor of {:?} is {}",
```

```
numbers, d)?;
```

Uma diferença é que cada macro `write` recebe um primeiro argumento extra, um escritor. A outra é que elas retornam um `Result`, portanto os erros devem ser tratados. É por isso que utilizamos o operador `?` no final de cada linha.

As macros `print` não retornam um `Result`; simplesmente geram pânico se a escrita falhar. Como elas escrevem no terminal, isso é raro.

O trait `Write` tem estes métodos:

```
writer.write(&buf)
```

Escreve alguns dos bytes na fatia `buf` no fluxo subjacente. Retorna um `io::Result<usize>`. Em caso de sucesso, isso fornece o número de bytes escritos, que pode ser menor que `buf.len()`, de acordo com o fluxo.

Como `Reader::read()`, esse é um método de baixo nível que você deve evitar utilizar diretamente.

```
writer.write_all(&buf)
```

Escreve todos os bytes na fatia `buf`. Retorna `Result<()>`.

```
writer.flush()
```

Libera todos os dados armazenados no buffer para o fluxo subjacente. Retorna `Result<()>`.

Note que, embora as macros `println!` e `eprintln!` liberem automaticamente o fluxo `stdout` e `stderr`, as macros `print!` e `eprint!` não liberam. Você pode ter de chamar `flush()` manualmente ao usá-las.

Assim como os leitores, os escritores são fechados automaticamente quando são dropados.

Assim como `BufReader::new(reader)` adiciona um buffer a qualquer leitor, `BufWriter::new(writer)` adiciona um buffer a qualquer escritor:

```
let file = File::create("tmp.txt"?);  
let writer = BufWriter::new(file);
```

Para definir o tamanho do buffer, utilize `BufWriter::with_capacity(size, writer)`.

Quando um `BufWriter` é dropado, todos os dados remanescentes armazenados em buffer são escritos no escritor subjacente. Mas, se ocorrer um erro durante essa escrita, o erro será ignorado. (Como

isso acontece dentro do método `.drop()` de `BufWriter`, não há lugar útil para relatar o erro.) Para garantir que seu aplicativo receba todos os erros de saída, chame manualmente `.flush()` dos escritores antes de dropá-los.

Arquivos

Já vimos duas maneiras de abrir um arquivo:

`File::open(filename)`

Abre um arquivo existente para leitura. Retorna um `io::Result<File>` e é um erro se o arquivo não existir.

`File::create(filename)`

Cria um novo arquivo para escrita. Se existir um arquivo com o nome de arquivo fornecido, ele será truncado (o conteúdo será apagado).

Observe que o tipo `File` está no módulo do sistema de arquivos, `std::fs`, não `std::io`.

Quando nenhum desses é apropriado, utilize `OpenOptions` para especificar o comportamento exato desejado:

```
use std::fs::OpenOptions;
```

```
let log = OpenOptions::new()
    .append(true) // se o arquivo existir, adiciona ao final
    .open("server.log");
```

```
let file = OpenOptions::new()
    .write(true)
    .create_new(true) // falha se o arquivo já existir
    .open("new_file.txt");
```

Os métodos `.append()`, `.write()`, `.create_new()` e assim por diante são projetados para serem encadeados desta maneira: cada um retorna `self`. Esse padrão de design de encadeamento de métodos é comum o suficiente para ter um nome no Rust: é chamado `builder` (construtor). `std::process::Command` é outro exemplo. Para mais detalhes sobre `OpenOptions`, consulte a documentação on-line.

Depois que `File` foi aberto, ele se comporta como qualquer outro leitor ou escritor. Você pode adicionar um buffer, se necessário. O `File` será fechado automaticamente quando você o dropar.

Posicionando

File também implementa o trait `Seek`, o que significa que você pode mudar a posição dentro de um `File` em vez de ler ou escrever em uma única passagem do começo ao fim. `Seek` é definido assim:

```
pub trait Seek {  
    fn seek(&mut self, pos: SeekFrom) -> io::Result<u64>;  
}  
  
pub enum SeekFrom {  
    Start(u64),  
    End(i64),  
    Current(i64)  
}
```

Graças ao enum, o método `seek` é bem expressivo: utilize `file.seek(SeekFrom::Start(0))` para voltar ao início e utilize `file.seek(SeekFrom::Current(-8))` para voltar alguns bytes e assim por diante.

O posicionamento (mudança da posição de leitura ou escrita) dentro de um arquivo é lento. Se estiver utilizando um disco rígido ou um disco de estado sólido (SSD), uma busca demora tanto quanto a leitura de vários megabytes de dados.

Outros tipos de leitores e escritores

Até agora, este capítulo usou `File` como exemplo de burro de carga, mas existem muitos outros tipos úteis de leitor e escritor:

`io::stdin()`

Retorna um leitor para o fluxo de entrada padrão. Seu tipo é `io::Stdin`. Como isso é compartilhado por todos os threads, cada leitura adquire e libera um mutex.

`Stdin` tem um método `.lock()` que adquire o mutex e retorna um `io::StdinLock`, um leitor em buffer que mantém o mutex até que seja dropado. Portanto, operações individuais no `StdinLock` evitam a sobrecarga do mutex. Mostramos um código de exemplo utilizando esse método em “Lendo linhas”, na página 542.

Por razões técnicas, `io::stdin().lock()` não funciona. O bloqueio contém uma referência ao valor `Stdin`, e isso significa que o valor `Stdin` deve ser armazenado em algum lugar para que exista por tempo

suficiente:

```
let stdin = io::stdin();  
let lines = stdin.lock().lines(); // ok
```

`io::stdout()`, `io::stderr()`

Retorna os tipos de escritor `Stdout` e `Stderr` para os fluxos da saída padrão e de erro padrão. Esses também têm mutexes e métodos `.lock()`.

`Vec<u8>`

Implementa `Write`. Escrever em um `Vec<u8>` estende o vetor com os novos dados.

(`String`, porém, não implementa `Write`. Para construir uma `string` utilizando `Write`, primeiro escreva em um `Vec<u8>` e depois utilize `String::from_utf8(vec)` para converter o vetor em uma `string`.)

`Cursor::new(buf)`

Cria um `Cursor`, um leitor em buffer que lê de `buf`. É assim que você cria um leitor que lê de uma `String`. O argumento `buf` pode ser qualquer tipo que implementa `AsRef<[u8]>`, assim você também pode passar um `&[u8]`, `&str` OU `Vec<u8>`.

`Cursors` são triviais internamente. Eles têm apenas dois campos: o próprio `buf` e um inteiro, o deslocamento em `buf` onde a próxima leitura começará. A posição é inicialmente 0.

`Cursors` implementam `Read`, `BufRead` e `Seek`. Se o tipo de `buf` é `&mut [u8]` OU `Vec<u8>`, então o `Cursor` também implementa `Write`. Escrever em um cursor sobrescreve os bytes em `buf` começando na posição atual. Se você tentar escrever depois do final de um `&mut [u8]`, obterá uma escrita parcial ou um `io::Error`. Utilizar um cursor para escrever após o final de um `Vec<u8>` é correto, porém aumenta o vetor. `Cursor<&mut [u8]>` e `Cursor<Vec<u8>>`, portanto implementam todos os quatro traits `std::io::prelude`.

`std::net::TcpStream`

Representa uma conexão de rede TCP. Como o TCP permite a comunicação bidirecional, ele é um leitor e um escritor.

A função associada ao tipo `TcpStream::connect(("hostname", PORT))` tenta se conectar a um servidor e retorna um `io::Result<TcpStream>`.

`std::process::Command`

Suporta lançar um processo filho e redirecionar dados para a sua entrada padrão, assim:

```
use std::process::{Command, Stdio};
```

```
let mut child =  
    Command::new("grep")  
        .arg("-e")  
        .arg("a.*e.*i.*o.*u")  
        .stdin(Stdio::piped())  
        .spawn()?;  
  
let mut to_child = child.stdin.take().unwrap();  
for word in my_words {  
    writeln!(to_child, "{}", word)?;  
}  
drop(to_child); // feche o stdin do grep, então ele sairá  
child.wait()?;
```

O tipo de `child.stdin` é `Option<std::process::ChildStdin>`; aqui nós utilizamos `.stdin(Stdio::piped())` ao configurar o processo filho, então `child.stdin` é definitivamente preenchido quando `.spawn()` é bem-sucedido. Se não utilizássemos, `child.stdin` seria `None`.

`Command` também tem métodos semelhantes `.stdout()` e `.stderr()`, que podem ser utilizados para solicitar leitores em `child.stdout` e `child.stderr`.

O módulo `std::io` também oferece algumas funções que retornam leitores e escritores triviais:

```
io::sink()
```

Esse é o escritor não operacional. Todos os métodos de escrita retornam `Ok`, mas os dados são simplesmente descartados.

```
io::empty()
```

Esse é o leitor não operacional. A leitura sempre é bem-sucedida, mas retorna o fim da entrada.

```
io::repeat(byte)
```

Retorna um leitor que repete o byte dado indefinidamente.

Dados binários, compactação e serialização

Muitos crates de código-fonte aberto são construídos no framework `std::io` para oferecer recursos extras.

O crate `byteorder` oferece traits de `ReadBytesExt` e `WriteBytesExt` que adicionam métodos a todos os leitores e escritores para entrada e saída binárias:

```
use byteorder::{ReadBytesExt, WriteBytesExt, LittleEndian};

let n = reader.read_u32::()?;
writer.write_i64::(n as i64)?;
```

O crate `flate2` fornece métodos adaptadores para leitura e escrita de dados compactados com `gzip`:

```
use flate2::read::GzDecoder;

let file = File::open("access.log.gz")?;
let mut gzip_reader = GzDecoder::new(file);
```

O crate `serde`, e os crates de formato associados, como `serde_json`, implementam serialização e desserialização: eles convertem entre structs Rust e bytes. Já mencionamos isso antes, em “Traits e tipos de outras pessoas”, na página 312. Agora podemos analisar mais atentamente.

Suponha que temos alguns dados – o mapa para um jogo de aventura em texto – armazenados em um `HashMap`:

```
type RoomId = String; // cada ambiente tem um nome único
type RoomExits = Vec<(char, RoomId)>; // ...e uma lista de saídas
type RoomMap = HashMap<RoomId, RoomExits>; // nomes de ambiente e saídas,
    simples

// Crie um mapa simples
let mut map = RoomMap::new();
map.insert("Cobble Crawl".to_string(),
    vec![( 'W', "Debris Room".to_string())]);
map.insert("Debris Room".to_string(),
    vec![( 'E', "Cobble Crawl".to_string()),
        ('W', "Sloping Canyon".to_string())]);

...
```

Transformar esses dados em JSON para saída é uma única linha de código:

```
serde_json::to_writer(&mut std::io::stdout(), &map)?;
```

Internamente, `serde_json::to_writer` utiliza o método `serialize` do trait `serde::Serialize`. A biblioteca atribui esse trait a todos os tipos que sabe como serializar, e isso inclui todos os tipos que aparecem em nossos dados: strings, caracteres, tuplas, vetores e `HashMapS`.

`serde` é flexível. Nesse programa, a saída são dados JSON, porque escolhemos o serializador `serde_json`. Outros formatos, como `MessagePack`, também estão disponíveis. Da mesma forma, você pode enviar essa saída para um arquivo, um `Vec<u8>` ou qualquer outro escritor. O código anterior imprime os dados em `stdout`. Eis:

```
{"Debris Room":[[{"E","Cobble Crawl"},{"W","Sloping Canyon"}], "Cobble Crawl":
[{"W","Debris Room"}]}
```

`serde` também inclui suporte para derivar os dois principais traits `serde`:

```
#[derive(Serialize, Deserialize)]
struct Player {
    location: String,
    items: Vec<String>,
    health: u32
}
```

Esse atributo `#[derive]` pode fazer com que as compilações demorem um pouco mais, assim você precisa solicitar explicitamente `serde` para suportá-lo ao listá-lo como uma dependência em seu arquivo *Cargo.toml*. Eis o que utilizamos para o código anterior:

```
[dependencies]
serde = { version = "1.0", features = ["derive"] }
serde_json = "1.0"
```

Veja na documentação sobre `serde` mais detalhes. Resumindo, o sistema de compilação gera automaticamente implementações de `serde::Serialize` e `serde::Deserialize` para `Player`, de modo que a serialização de um valor `Player` é simples:

```
serde_json::to_writer(&mut std::io::stdout(), &player)?;
```

A saída se parece com isto:

```
{"location":"Cobble Crawl","items":["a wand"],"health":3}
```

Arquivos e diretórios

Agora que mostramos como trabalhar com leitores e escritores, as próximas seções discutem os recursos do Rust para trabalhar com arquivos e diretórios, que residem nos módulos `std::path` e `std::fs`. Todos esses recursos envolvem trabalhar com nomes de arquivo, então começaremos com os tipos de nome de arquivo.

OsStr e Path

Inconvenientemente, seu sistema operacional não força os nomes de arquivo a serem Unicode válidos. Eis dois comandos shell do Linux que criam arquivos de texto. Somente o primeiro utiliza um nome de arquivo UTF-8 válido:

```
$ echo "hello world" > ô.txt
```

```
$ echo "O brave new world, that has such filenames in't" > $'\xf4'.txt
```

Ambos os comandos são passados sem comentários, porque o kernel do Linux não conhece UTF-8 de Ogg Vorbis. Para o kernel, qualquer string de bytes (excluindo bytes nulos e barras) é um nome de arquivo aceitável. É uma história semelhante no Windows: quase qualquer string de “caracteres de largura” de 16 bits é um nome de arquivo aceitável, mesmo strings que não são UTF-16 válidas. O mesmo se aplica a outras strings manipuladas pelo sistema operacional, como argumentos de linha de comando e variáveis de ambiente.

Strings do Rust são sempre Unicode válidas. Os nomes de arquivo são quase sempre Unicode na prática, mas o Rust de alguma forma tem de lidar com o raro caso em que não são. É por isso que o Rust tem `std::ffi::OsStr` e `OsString`.

`OsStr` é um tipo de string que é um superconjunto de UTF-8. Seu trabalho é ser capaz de representar todos os nomes de arquivo, argumentos de linha de comando e variáveis de ambiente no sistema atual, sejam eles Unicode válidos ou não. No Unix, um `OsStr` pode conter qualquer sequência de bytes. No Windows, um `OsStr` é armazenado utilizando uma extensão de UTF-8 que pode codificar qualquer sequência de valores de 16 bits, incluindo substitutos não correspondidos.

Portanto, temos dois tipos de string: `str` para strings Unicode reais; e `OsStr` para qualquer bobagem que seu sistema operacional possa oferecer. Vamos apresentar mais um: `std::path::Path`, para nomes de arquivo. Esse é puramente uma conveniência. `Path` é exatamente como `OsStr`, mas adiciona muitos métodos úteis relacionados ao nome de arquivo, que abordaremos na próxima seção. Utilize `Path`

para caminhos absolutos e relativos. Para um componente individual de um caminho, utilize `OsStr`.

Por fim, para cada tipo de string, há um correspondente tipo proprietário: um tipo `String` possui uma `str` alocada em heap, um `std::ffi::OsString` possui uma `OsStr` alocada em heap e um `std::path::PathBuf` possui uma `Path` alocada em heap. A Tabela 18.1 descreve algumas das características de cada tipo.

Tabela 18.1: Tipos de nome de arquivo

	<code>str</code>	<code>OsStr</code>	<code>Path</code>
Tipo não dimensionado, sempre passado por referência	Sim	Sim	Sim
Pode conter qualquer texto Unicode	Sim	Sim	Sim
Parece com UTF-8, normalmente	Sim	Sim	Sim
Pode conter dados não Unicode	Não	Sim	Sim
Métodos de processamento de texto	Sim	Não	Não
Métodos relacionados a nome de arquivo	Não	Não	Sim
Equivalente alocado no heap com posse e expansível	<code>String</code>	<code>OsString</code>	<code>PathBuf</code>
Converte em tipo proprietário	<code>.to_string()</code>	<code>.to_os_string()</code>	<code>.to_path_buf()</code>

Todos esses três tipos implementam um trait comum, `AsRef<Path>`, para que possamos declarar facilmente uma função genérica que aceita “qualquer tipo de nome de arquivo” como argumento. Isso utiliza uma técnica que mostramos em “`AsRef` e `AsMut`”, na página 368:

```
use std::path::Path;
use std::io;

fn swizzle_file<P>(path_arg: P) -> io::Result<()>
    where P: AsRef<Path>
{
    let path = path_arg.as_ref();
    ...
}
```

Todas as funções e métodos padrão que recebem argumentos `path` utilizam essa técnica, então você pode passar strings literais livremente para qualquer um deles.

Métodos Path e PathBuf

Path oferece os seguintes métodos, entre outros:

`Path::new(str)`

Converte um `&str` ou `&OsStr` em um `&Path`. Isso não copia a string. O novo `&Path` aponta para os mesmos bytes que o original `&str` ou `&OsStr`:

```
use std::path::Path;
let home_dir = Path::new("/home/fwolfe");
```

(O método semelhante `OsStr::new(str)` converte um `&str` em um `&OsStr`.)

`path.parent()`

Retorna o diretório pai do caminho, se houver um. O tipo de retorno é `Option<&Path>`.

Isso não copia o caminho. O diretório pai de `path` é sempre uma substring de `path`:

```
assert_eq!(Path::new("/home/fwolfe/program.txt").parent(),
           Some(Path::new("/home/fwolfe")));
```

`path.file_name()`

Retorna o último componente de `path`, se houver um. O tipo de retorno é `Option<&OsStr>`.

No caso típico, em que `path` consiste em um diretório, então uma barra e, em seguida, um nome de arquivo, retorna o nome de arquivo:

```
use std::ffi::OsStr;
assert_eq!(Path::new("/home/fwolfe/program.txt").file_name(),
           Some(OsStr::new("program.txt")));
```

`path.is_absolute()`, `path.is_relative()`

Informam se o arquivo é absoluto, como o caminho do Unix */usr/bin/advent* ou o caminho do Windows *C:\Arquivos de programas*, ou relativo, como *src/main.rs*.

`path1.join(path2)`

Associa dois caminhos, retornando um novo `PathBuf`:

```
let path1 = Path::new("/usr/share/dict");
assert_eq!(path1.join("words"),
           Path::new("/usr/share/dict/words"));
```

Se `path2` é um caminho absoluto, isso apenas retorna uma cópia de

path2, assim esse método pode ser utilizado para converter qualquer caminho em um caminho absoluto:

```
let abs_path = std::env::current_dir()?.join(any_path);
```

path.components()

Retorna um iterador sobre os componentes do caminho especificado, da esquerda para a direita. O tipo de item desse iterador é `std::path::Component`, uma enumeração que pode representar todas as diferentes partes que podem aparecer em nomes de arquivo:

```
pub enum Component<'a> {  
    Prefix(PrefixComponent<'a>), // uma letra de unidade ou compartilhamento (no  
    Windows)  
    RootDir, // o diretório raiz, '/' ou '\\'  
    CurDir, // o diretório especial '.'  
    ParentDir, // o diretório especial '..'  
    Normal(&'a OsStr) // nomes simples de arquivo e diretório  
}
```

Por exemplo, o caminho do Windows `\\venice\Music\A Love Supreme\04-Psalm.mp3` consiste em um `Prefix` representando `\\venice\Music`, seguido por um `RootDir` e então dois componentes `Normal` que representam `A LoveSupreme` e `04-Psalm.mp3`.

Para obter detalhes, consulte a documentação on-line (<https://oreil.ly/mtHCK>).

path.ancestors()

Retorna um iterador que vai de `path` até a raiz. Cada item gerado é um `Path`: primeiro o próprio `path`, então o pai, depois o avô e assim por diante:

```
let file = Path::new("/home/jimb/calendars/calendar-18x18.pdf");  
assert_eq!(file.ancestors().collect::<Vec<_>>(),  
    vec![Path::new("/home/jimb/calendars/calendar-18x18.pdf"),  
        Path::new("/home/jimb/calendars"),  
        Path::new("/home/jimb"),  
        Path::new("/home"),  
        Path::new("/")]);
```

Isso é quase como chamar `parent` repetidamente até retornar `None`. O item final é sempre um caminho raiz ou prefixo.

Esses métodos funcionam em strings na memória. `Paths` também têm alguns métodos que consultam o sistema de arquivos: `.exists()`,

`.is_file()`, `.is_dir()`, `.read_dir()`, `.canonicalize()` etc. Consulte informações adicionais na documentação on-line.

Existem três métodos para converter `Paths` em strings. Cada um permite a possibilidade de UTF-8 inválido em `Path`:

`path.to_str()`

Converte um `Path` em uma string, como uma `Option<&str>`. Se `path` não é UTF-8 válido, isso retorna `None`:

```
if let Some(file_str) = path.to_str() {  
    println!("{}", file_str);  
} // ...caso contrário, ignore esse arquivo de nome estranho
```

`path.to_string_lossy()`

Isso é basicamente a mesma coisa, mas consegue retornar algum tipo de string em todos os casos. Se `path` não é UTF-8 válido, esses métodos criam uma cópia, substituindo cada sequência de bytes

inválida pelo caractere de substituição Unicode, U+FFFD ('').

O tipo de retorno é `std::borrow::Cow<str>`: uma string emprestada ou possuída. Para obter uma `String` a partir desse valor, utilize o método `.to_owned()`. (Para saber mais sobre `Cow`, ver “Borrow e ToOwned em funcionamento: Cow (“clone on write”)”, na página 377.)

`path.display()`

Isso é para caminhos de impressão:

```
println!("Download found. You put it in: {}", dir_path.display());
```

O valor que isso retorna não é uma string, mas implementa `std::fmt::Display`, então pode ser utilizado com `format!()`, `println!()` e amigos. Se o caminho não é UTF-8 válido, a saída pode conter o

caractere .

Funções de acesso ao sistema de arquivos

A Tabela 18.2 mostra algumas das funções em `std::fs` e seus equivalentes aproximados no Unix e Windows. Todas essas funções retornam valores `io::Result`. São `Result<>` salvo indicação em contrário.

Tabela 18.2: Resumo das funções de acesso ao sistema de arquivos

	Função Rust	Unix	Windows
Criando e excluindo	<code>create_dir(path)</code>	<code>mkdir()</code>	<code>CreateDirectory()</code>
	<code>create_dir_all(path)</code>	como <code>mkdir -p</code>	como <code>mkdir</code>
	<code>remove_dir(path)</code>	<code>rmdir()</code>	<code>RemoveDirectory()</code>
	<code>remove_dir_all(path)</code>	como <code>rm -r</code>	como <code>rmdir /s</code>
	<code>remove_file(path)</code>	<code>unlink()</code>	<code>DeleteFile()</code>
Copiar, mover e vincular	<code>copy(src_path, dest_path) -> Result<u64></code>	como <code>cp -p</code>	<code>CopyFileEx()</code>
	<code>rename(src_path, dest_path)</code>	<code>rename()</code>	<code>MoveFileEx()</code>
	<code>hard_link(src_path, dest_path)</code>	<code>link()</code>	<code>CreateHardLink()</code>
Inspecionando	<code>canonicalize(path) -> Result<PathBuf></code>	<code>realpath()</code>	<code>GetFinalPathNameByHandle()</code>
	<code>metadata(path) -> Result<Metadata></code>	<code>stat()</code>	<code>GetFileInformationByHandle()</code>
	<code>symlink_metadata(path) -> Result<Metadata></code>	<code>lstat()</code>	<code>GetFileInformationByHandle()</code>
	<code>read_dir(path) -> Result<ReadDir></code>	<code>opendir()</code>	<code>FindFirstFile()</code>
	<code>read_link(path) -> Result<PathBuf></code>	<code>readlink()</code>	<code>FSCTL_GET_REPARSE_POINT</code>
Permissões	<code>set_permissions(path, perm)</code>	<code>chmod()</code>	<code>SetFileAttributes()</code>

(O número retornado por `copy()` é o tamanho do arquivo copiado, em bytes. Para criar links simbólicos, consulte “Recursos específicos da plataforma”, na página 560.)

Como podemos ver, o Rust se esforça para fornecer funções portáteis que operam de forma previsível no Windows, bem como no macOS, Linux e outros sistemas Unix.

Um tutorial completo sobre sistemas de arquivos está além do escopo deste livro, mas, se estiver curioso sobre qualquer uma dessas funções, você poderá facilmente encontrar mais sobre elas on-line. Mostraremos alguns exemplos na próxima seção.

Todas essas funções são implementadas de acordo com o sistema operacional. Por exemplo, `std::fs::canonicalize(path)` não utiliza apenas o processamento de strings para eliminar `.` e `..` do `path` especificado. Resolve caminhos relativos utilizando o diretório de trabalho atual e procura links simbólicos. É um erro se o caminho não existir.

O tipo `Metadata` que é gerado por `std::fs::metadata(path)` e `std::fs::symlink_metadata(path)` contém informações como tipo e tamanho do arquivo, permissões e registros de data e hora. Como sempre, consulte detalhes na documentação.

Por conveniência, no tipo `Path` há alguns deles integrados como métodos: `path.metadata()`, por exemplo, é a mesma coisa que `std::fs::metadata(path)`.

Lendo diretórios

Para listar o conteúdo de um diretório, utilize `std::fs::read_dir` ou, de forma equivalente, o método `.read_dir()` de um `Path`:

```
for entry_result in path.read_dir()? {  
    let entry = entry_result?;  
    println!("{}", entry.file_name().to_string_lossy());  
}
```

Observe os dois usos de `?` nesse código. A primeira linha verifica se há erros ao abrir o diretório. A segunda linha verifica erros ao ler a próxima entrada.

O tipo de `entry` é `std::fs::DirEntry` e é um struct com apenas alguns métodos:

`entry.file_name()`

O nome de arquivo ou diretório, como um `OsString`.

`entry.path()`

Isso é a mesma coisa, mas com o caminho original associado a ele, produzindo um novo `PathBuf`. Se o diretório que estamos listando fosse `"/home/jimb"` e `entry.file_name()` é `".emacs"`, então `entry.path()` retornaria `PathBuf::from("/home/jimb/.emacs")`.

`entry.file_type()`

Retorna um `io::Result<FileType>`. `FileType` tem métodos `.is_file()`, `.is_dir()` e `.is_symlink()`.

`entry.metadata()`

Obtém o restante dos metadados sobre essa entrada.

Os diretórios especiais `.` e `..` não são listados ao ler um diretório.

Eis um exemplo mais substancial. O código a seguir copia

recursivamente uma árvore de diretórios de um local para outro no disco:

```
use std::fs;
use std::io;
use std::path::Path;

/// Copia o diretório `src` existente para o caminho de destino `dst`
fn copy_dir_to(src: &Path, dst: &Path) -> io::Result<()> {
    if !dst.is_dir() {
        fs::create_dir(dst)?;
    }

    for entry_result in src.read_dir()? {
        let entry = entry_result?;
        let file_type = entry.file_type()?;
        copy_to(&entry.path(), &file_type, &dst.join(entry.file_name()))?;
    }

    Ok(())
}
```

Uma função distinta, `copy_to`, copia entradas de diretório individuais:

```
/// Copia o que estiver em `src` para o caminho de destino `dst`
fn copy_to(src: &Path, src_type: &fs::FileType, dst: &Path)
    -> io::Result<()>
{
    if src_type.is_file() {
        fs::copy(src, dst)?;
    } else if src_type.is_dir() {
        copy_dir_to(src, dst)?;
    } else {
        return Err(io::Error::new(io::ErrorKind::Other,
                                   format!("don't know how to copy: {}",
                                           src.display())));
    }

    Ok(())
}
```

Recursos específicos da plataforma

Até agora, nossa função `copy_to` pode copiar arquivos e diretórios. Suponha que também queremos oferecer suporte a links simbólicos no Unix.

Não há uma maneira portátil de criar links simbólicos que funcionem tanto no Unix quanto no Windows, mas a biblioteca

padrão oferece uma função específica ao Unix `symlink`:

```
use std::os::unix::fs::symlink;
```

Com isso, nosso trabalho é fácil. Precisamos apenas adicionar um ramo à expressão `if` em `copy_to`:

```
...
} else if src_type.is_symlink() {
    let target = src.read_link()?;
    symlink(target, dst)?;
...

```

Isso funcionará desde que nosso programa seja compilado apenas para sistemas Unix, como Linux e macOS.

O módulo `std::os` contém vários recursos específicos de plataforma, como `symlink`. O corpo real de `std::os` na biblioteca padrão se parece com isto (quase como uma licença poética):

```
//! Funcionalidade específica ao sistema operacional
```

```
#[cfg(unix)]           pub mod unix;
#[cfg(windows)]        pub mod windows;
#[cfg(target_os = "ios")] pub mod ios;
#[cfg(target_os = "linux")] pub mod linux;
#[cfg(target_os = "macos")] pub mod macos;
...

```

O atributo `#[cfg]` indica compilação condicional: cada um desses módulos só existe em algumas plataformas. É por isso que nosso programa modificado, utilizando `std::os::unix`, vai compilar com sucesso apenas para Unix: em outras plataformas, `std::os::unix` não existe.

Se quisermos que nosso código seja compilado em todas as plataformas, com suporte para links simbólicos no Unix, devemos utilizar `#[cfg]` também em nosso programa. Nesse caso, é mais fácil importar `symlink` no Unix, ao definir nosso próprio stub `symlink` em outros sistemas:

```
#[cfg(unix)]
use std::os::unix::fs::symlink;

/// Implementação stub de `symlink` para plataformas que não o fornecem
#[cfg(not(unix))]
fn symlink<P: AsRef<Path>, Q: AsRef<Path>>(src: P, _dst: Q)
    -> std::io::Result<>
{
    Err(io::Error::new(io::ErrorKind::Other,
```

```

        format!("can't copy symbolic link: {}",
               src.as_ref().display()))
    }

```

Acontece que `symlink` é um caso especial. A maioria dos recursos específicos do Unix não são funções autônomas, mas sim extensões que adicionam novos métodos aos tipos na biblioteca padrão. (Discutimos traits de extensão em “Traits e tipos de outras pessoas”, na página 312.) Há um módulo `prelude` que pode ser utilizado para ativar todas essas extensões de uma só vez:

```
use std::os::unix::prelude::*;
```

Por exemplo, no Unix, isso adiciona um método `.mode()` a `std::fs::Permissions`, fornecendo acesso ao valor `u32` subjacente que representa permissões no Unix. Da mesma forma, estende `std::fs::Metadata` com acessores para os campos do valor `struct stat` subjacente – como `.uid()`, o ID de usuário do dono do arquivo.

Dito isso, o que está em `std::os` é bem básico. Muito mais funcionalidades específicas de plataforma estão disponíveis por meio de crates de terceiros, como `winreg` (crates.io/crates/winreg) para acessar o registro (registry) do Windows.

Redes

Um tutorial sobre redes está muito além do escopo deste livro. No entanto, se você já conhece um pouco programação de rede, esta seção o ajudará a começar a trabalhar com redes no Rust.

Para código de rede de baixo nível, comece com o módulo `std::net`, que fornece suporte a múltiplas plataformas para rede TCP e UDP. Use o crate `native_tls` para suporte SSL/TLS.

Esses módulos fornecem os blocos de construção simples de entrada e saída bloqueantes na rede. Você pode escrever um servidor simples em algumas linhas de código, utilizando `std::net` e gerando um thread para cada conexão. Por exemplo, eis um servidor de “eco”:

```

use std::net::TcpListener;
use std::io;
use std::thread::spawn;

```

```

/// Aceita conexões para sempre, gerando um thread para cada uma
fn echo_main(addr: &str) -> io::Result<()> {
    let listener = TcpListener::bind(addr)?;
    println!("listening on {}", addr);
    loop {
        // Espera que um cliente se conecte
        let (mut stream, addr) = listener.accept()?;
        println!("connection received from {}", addr);

        // Cria um thread para lidar com esse cliente
        let mut write_stream = stream.try_clone()?;
        spawn(move || {
            // Ecoa tudo o que recebemos do `fluxo` de volta para ele
            io::copy(&mut stream, &mut write_stream)
                .expect("error in client thread: ");
            println!("connection closed");
        });
    }
}

fn main() {
    echo_main("127.0.0.1:17007").expect("error: ");
}

```

Um servidor de eco simplesmente repete tudo o que você envia para ele. Esse tipo de código não é tão diferente daquilo que você escreveria no Java ou Python. (Discutiremos `std::thread::spawn()` no próximo capítulo.)

Contudo, para servidores de alto desempenho, você precisará utilizar entrada e saída assíncronas. O Capítulo 20 aborda o suporte do Rust para programação assíncrona e mostra o código completo para um cliente e servidor de rede.

Protocolos de nível superior são suportados por crates de terceiros. Por exemplo, o crate `request` oferece uma bela API para clientes HTTP. Eis um programa de linha de comando completo que busca qualquer documento com um `http:` ou `https:` URL e coloca-o no seu terminal. Esse código foi escrito utilizando `request = "0.11"`, com o recurso "blocking" ativado. `request` também fornece uma interface assíncrona.

```

use std::error::Error;
use std::io;

fn http_get_main(url: &str) -> Result<(), Box<dyn Error>> {
    // Envie a solicitação HTTP e obtenha uma resposta

```



```

    let mut response = request::blocking::get(url)?;
    if !response.status().is_success() {
        Err(format!("{}", response.status()))?;
    }

    // Leia o corpo da resposta e escreva-o em stdout
    let stdout = io::stdout();
    io::copy(&mut response, &mut stdout.lock())?;

    Ok(())
}

fn main() {
    let args: Vec<String> = std::env::args().collect();
    if args.len() != 2 {
        eprintln!("usage: http-get URL");
        return;
    }

    if let Err(err) = http_get_main(&args[1]) {
        eprintln!("error: {}", err);
    }
}

```

O framework `actix-web` para servidores HTTP oferece toques de alto nível, como os traits `Service` e `Transform`, que ajudam a compor um aplicativo a partir de partes conectáveis. O `crate websocket` implementa o protocolo WebSocket. E assim por diante. O Rust é uma linguagem jovem com um movimentado ecossistema de código-fonte aberto. O suporte para rede está se expandindo rapidamente.

CAPÍTULO 19

Concorrência

A longo prazo, não é aconselhável escrever grandes programas concorrentes em linguagens orientadas à máquina que permitem o uso irrestrito de locais de armazenamento e seus endereços. Não há como tornar esses programas confiáveis (mesmo com a ajuda de complicados mecanismos de hardware).

– Per Brinch Hansen (1977)

Padrões de comunicação são padrões de paralelismo.

– Whit Morriss

Se sua atitude em relação à concorrência mudou ao longo de sua carreira, você não está sozinho. É uma história comum.

A princípio, escrever código concorrente é fácil e divertido. As ferramentas – threads, bloqueios, filas e assim por diante – são fáceis de entender e utilizar. Existem muitas armadilhas, é verdade, mas felizmente você sabe quais são todas elas e toma cuidado para não cometer erros.

Em algum momento, você terá de depurar o código multithread de outra pessoa e será forçado a concluir que *algumas* pessoas realmente não deveriam usar essas ferramentas.

Então, em algum momento, você precisa depurar seu próprio código multithread.

A experiência inculca um ceticismo saudável, se não um cinismo total, em relação a todo código multithread. Isso é ajudado por artigos esporádicos explicando em detalhes assombrosos por que alguma linguagem multithread obviamente correta não funciona. (Tem a ver com “o modelo de memória”.) Mas você acaba encontrando uma abordagem sobre concorrência que acha que pode utilizar de forma realista sem cometer erros constantemente. Você pode adaptar praticamente tudo nessa linguagem e (se for

realmente bom) aprende a dizer “não” à complexidade adicional.

Naturalmente, existem muitas linguagens. Abordagens que os programadores de sistemas geralmente utilizam incluem:

- Um *thread de background* que tem um único trabalho e acorda periodicamente para fazê-lo.
- *Pools de trabalhadores (workers)* de propósito geral que se comunicam com os clientes por meio de *filas de tarefas*.
- *Pipelines* em que os dados fluem de um thread para o próximo, com cada thread fazendo um pouco do trabalho.
- *Paralelismo de dados*, em que se supõe (com ou sem razão) que todo computador fará principalmente apenas uma grande computação que, portanto, é dividida em n partes e executará n threads na esperança de colocar todos os n núcleos da máquina para funcionar de uma só vez.
- *Um mar de objetos sincronizados*, em que vários threads têm acesso aos mesmos dados e os problemas de concorrência são evitados utilizando esquemas de *bloqueio* ad hoc baseados em primitivas de baixo nível como mutexes. (O Java inclui suporte integrado para esse modelo, que era bastante popular nos anos 1990 e 2000.)
- *Operações inteiras atômicas* permitem que vários núcleos se comuniquem passando informações por meio de campos do tamanho de uma palavra de máquina. (Isso é ainda mais difícil de fazer corretamente do que todos os outros, a menos que os dados sendo trocados sejam literalmente apenas valores inteiros. Na prática, geralmente são ponteiros.)

Com o tempo, você poderá utilizar várias dessas abordagens e combiná-las com segurança. Você é um mestre da arte. E as coisas seriam ótimas, se ninguém mais tivesse permissão para modificar o sistema de forma nenhuma. Programas que utilizam bem threads estão cheios de regras tácitas.

O Rust oferece uma maneira melhor de utilizar a concorrência sem forçar todos os programas a adotar um único estilo (o que para programadores de sistemas não seria absolutamente uma solução),

mas suportando vários estilos com segurança. As regras tácitas são especificadas – no código – e aplicadas pelo compilador.

Você já ouviu falar que o Rust permite escrever programas concorrentes seguros e rápidos. Este é o capítulo em que mostramos como isso é feito. Abordaremos três maneiras de utilizar threads do Rust:

- Paralelismo fork-join
- Canais
- Estado mutável compartilhado

Ao longo do caminho, você utilizará tudo o que aprendeu até agora sobre a linguagem Rust. O cuidado que o Rust toma em relação a referências, mutabilidade e tempos de vida é valioso o suficiente em programas de thread único, mas é na programação concorrente que o verdadeiro significado dessas regras se torna aparente. Eles permitem expandir sua caixa de ferramentas, criar vários estilos de código multithread de forma rápida e correta – sem ceticismo, sem cinismo, sem medo.

Paralelismo fork-join

Os casos de uso mais simples para threads surgem quando temos várias tarefas completamente independentes que queremos fazer ao mesmo tempo.

Por exemplo, suponha que estamos fazendo processamento de linguagem natural em um grande corpus de documentos. Poderíamos escrever um loop:

```
fn process_files(filenames: Vec<String>) -> io::Result<()> {  
    for document in filenames {  
        let text = load(&document)?; // lê o arquivo fonte  
        let results = process(text); // calcula estatísticas  
        save(&document, results)?; // grava o arquivo de saída  
    }  
    Ok(())  
}
```

O programa seria executado como mostrado na Figura 19.1.

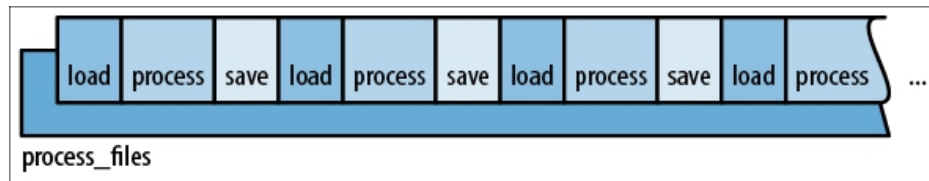


Figura 19.1: Execução de thread único de `process_files()`.

Como cada documento é processado de maneira separada, é relativamente fácil acelerar essa tarefa dividindo o corpus em partes e processando cada parte em um thread distinto, conforme mostrado na Figura 19.2.

Esse padrão é chamado de paralelismo fork-join (bifurcar-unir). Bifurcar (`fork`) é iniciar um novo thread e juntar (`join`) um thread é esperar ele terminar. Já vimos essa técnica: ela foi utilizada para acelerar o programa Mandelbrot no Capítulo 2.

O paralelismo fork-join é atraente por alguns motivos:

- É muito simples. Fork-join é fácil de implementar e o Rust torna mais fácil acertar.
- Evita gargalos. Não há bloqueio dos recursos compartilhados no modelo fork-join. A única vez que qualquer thread precisa esperar por outro é no final. Enquanto isso, cada thread pode executar livremente. Isso ajuda a manter baixa a sobrecarga de alternância de tarefas.
- A matemática do desempenho é simples. Na melhor das hipóteses, iniciando quatro threads, podemos concluir nosso trabalho em um quarto do tempo. A Figura 19.2 mostra um motivo pelo qual não devemos esperar esse aumento de velocidade ideal: talvez não sejamos capazes de distribuir o trabalho uniformemente por todos os threads. Outro motivo de cautela é que, às vezes, os programas fork-join devem passar algum tempo após a junção dos threads *combinando* os resultados computados pelos threads. Ou seja, isolar completamente as tarefas pode gerar algum trabalho extra. Ainda assim, além dessas duas coisas, qualquer programa vinculado à CPU com unidades de trabalho isoladas pode esperar uma melhoria significativa.

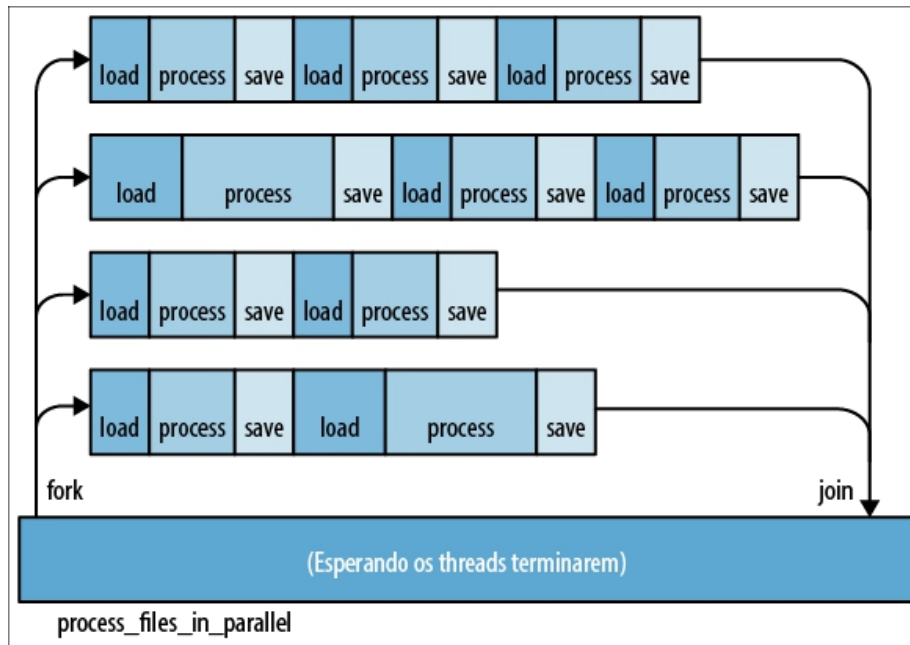


Figura 19.2: Processamento de arquivo multithread utilizando uma abordagem fork-join.

- É fácil raciocinar sobre a exatidão do programa. Um programa fork-join é *determinista* contanto que os threads estejam realmente isolados, como os threads de computação no programa Mandelbrot. O programa produz sempre o mesmo resultado, independentemente das variações na velocidade do thread. É um modelo de concorrência sem condições de corrida.

A principal desvantagem do fork-join é que ele requer unidades de trabalho isoladas. Mais adiante neste capítulo, consideraremos alguns problemas que não podem ser divididos tão claramente.

Por enquanto, vamos nos ater ao exemplo do processamento de linguagem natural. Mostraremos algumas maneiras de aplicar o padrão fork-join à função `process_files`.

spawn e join

A função `std::thread::spawn` inicia um novo thread:

```
use std::thread;

thread::spawn(|| {
    println!("hello from a child thread");
});
```

Ela recebe um argumento, uma closure ou função `FnOnce`. O Rust inicia um novo thread para executar o código dessa closure ou função. O novo thread é um thread real do sistema operacional com um stack próprio, assim como threads em C++, C# e Java.

Eis um exemplo mais substancial, utilizando `spawn` para implementar uma versão paralela da função `process_files` anterior:

```
use std::{thread, io};

fn process_files_in_parallel(filenames: Vec<String>) -> io::Result<()> {
    // Divide o trabalho em vários pedaços
    const NTHREADS: usize = 8;
    let worklists = split_vec_into_chunks(filenames, NTHREADS);

    // Fork: Gera um thread para lidar com cada pedaço
    let mut thread_handles = vec![];
    for worklist in worklists {
        thread_handles.push(
            thread::spawn(move || process_files(worklist))
        );
    }

    // Join: Aguarda até que todos os threads terminem
    for handle in thread_handles {
        handle.join().unwrap()?;
    }

    Ok(())
}
```

Vamos analisar essa função linha por linha.

```
fn process_files_in_parallel(filenames: Vec<String>) -> io::Result<()> {
```

Nossa nova função tem a mesma assinatura de tipo que a original `process_files`, tornando-a um substituto drop-in útil.

```
    // Divide o trabalho em vários pedaços
    const NTHREADS: usize = 8;
    let worklists = split_vec_into_chunks(filenames, NTHREADS);
```

Utilizamos uma função utilitária `split_vec_into_chunks`, não mostrada aqui, para dividir o trabalho. O resultado, `worklists`, é um vetor de vetores. Ele contém oito porções de tamanho uniforme do vetor original `filenames`.

```
    // Fork: Gera um thread para lidar com cada pedaço (chunk)
    let mut thread_handles = vec![];
    for worklist in worklists {
        thread_handles.push(
```

```

        thread::spawn(move || process_files(worklist))
    );
}

```

Geramos um thread para cada `worklist`. `spawn()` retorna um valor chamado `JoinHandle`, que utilizaremos mais tarde. Por enquanto, colocamos todos os `JoinHandles` em um vetor.

Observe como obtemos a lista dos nomes de arquivo no thread de trabalho:

- `worklist` é definido e preenchido pelo loop `for`, no thread pai.
- Assim que a closure `move` é criada, `worklist` é movido para a closure.
- `spawn` em seguida move a closure (incluindo o vetor `worklist`) para o novo thread filho.

Esses movimentos são baratos. Como os movimentos de `Vec<String>` que discutimos no Capítulo 4, `Strings` não são clonadas. Na verdade, nada é alocado ou liberado. Os únicos dados movidos são o próprio `Vec`: três palavras de máquina.

Quase todos os threads que você cria precisam de código e dados para iniciar. Closures do Rust, convenientemente, podem conter qualquer código que você desejar e quaisquer dados que você quiser.

Prosseguindo:

```

// Join: Aguarda até que todos os threads terminem
for handle in thread_handles {
    handle.join().unwrap()?;
}

```

Utilizamos o método `.join()` de `JoinHandles` que coletamos anteriormente para aguardar a conclusão de todos os oito threads. A junção (`join`) de threads geralmente é necessária para correta execução do programa, porque um programa Rust é encerrado assim que `main` retorna, mesmo se outros threads ainda estão em execução. Destrutores não são chamados; os threads extras são simplesmente eliminados. Se isso não é o que você deseja, certifique-se de juntar todos os threads de seu interesse antes de retornar de `main`.

Se conseguirmos passar por esse loop, isso significa que todos os oito threads filhos foram encerrados com sucesso. Nossa função,

portanto, termina retornando `Ok()`:

```
    Ok()  
}
```

Tratamento de erros entre threads

O código que utilizamos para esperar (`join`) os threads filhos em nosso exemplo é mais complicado do que parece, devido ao tratamento de erros. Vamos rever esta linha do código:

```
handle.join().unwrap()?;
```

O método `.join()` faz duas coisas legais para nós.

Primeiro, `handle.join()` retorna um `std::thread::Result` que é um erro se o thread filho gerou um pânico. Isso torna o processo de threading no Rust dramaticamente mais robusto do que em C++. No C++ um acesso de array fora dos limites é um comportamento indefinido, e não há como proteger o restante do sistema das consequências. No Rust, pânico é seguro e por thread. Os limites entre threads servem como uma parede corta-fogo contra pânico; o pânico não se espalha automaticamente de um thread para os threads que dependem dele. Em vez disso, pânico em um thread é relatado como um erro `Result` em outros threads. O programa como um todo pode ser facilmente recuperado.

Em nosso programa, porém, não tentamos nenhum tratamento de pânico sofisticado. Em vez disso, utilizamos imediatamente `.unwrap()` nesse `Result`, afirmando que é um resultado `Ok` e não um resultado `Err`. Se um thread filho gerasse um pânico, essa asserção falharia, portanto o thread pai também gerará um pânico. Estamos propagando explicitamente o pânico dos threads filhos ao thread pai.

Segundo, `handle.join()` passa o valor de retorno do thread filho de volta para o thread pai. A closure que passamos para `spawn` tem um tipo de retorno de `io::Result<()>`, porque é isso que `process_files` retorna. Esse valor de retorno não é descartado. Quando o thread filho é encerrado, o valor de retorno é salvo e `JoinHandle::join()` transfere esse valor de volta para o thread pai.

O tipo completo que é retornado por `handle.join()` nesse programa é `std::thread::Result<std::io::Result<()>>`. O `thread::Result` é parte da API `spawn/join`; `io::Result` é parte do nosso aplicativo.

No nosso caso, depois de desencapsular o `thread::Result`, utilizamos o operador `?` no `io::Result`, propagando explicitamente erros de E/S dos threads filhos ao thread pai.

Tudo isso pode parecer bastante intrincado. Mas considere que é apenas uma linha de código e compare isso com outras linguagens. O comportamento padrão no Java e C# é que as exceções em threads filhos são descarregadas no terminal e depois esquecidas. Em C++, o padrão é abortar o processo. No Rust, os erros são valores `Result` (dados) em vez de exceções (fluxo de controle). São entregues entre threads como qualquer outro valor. Sempre que você utiliza APIs de thread de baixo nível, acaba tendo de escrever um código de tratamento de erros cuidadoso, mas, como é necessário escrevê-lo, é muito bom ter por perto um `Result`.

Compartilhamento de dados imutáveis entre threads

Suponha que a análise que estamos fazendo exija um grande banco de dados de palavras e frases em inglês:

```
// antes
fn process_files(filenamees: Vec<String>)

// depois
fn process_files(filenamees: Vec<String>, glossary: &GigabyteMap)
```

Esse `glossary` será grande, então estamos passando-o por referência. Como podemos atualizar `process_files_in_parallel` para passar o glossário para os threads de trabalho?

A mudança óbvia não funciona:

```
fn process_files_in_parallel(filenamees: Vec<String>,
                             glossary: &GigabyteMap)
-> io::Result<()>
{
    ...
    for worklist in worklists {
        thread_handles.push(
```

```

        spawn(move || process_files(worklist, glossary)) // erro
    );
}
...
}

```

Simplesmente adicionamos um argumento `glossary` a nossa função e o passamos para `process_files`. O Rust reclama:

```

error: explicit lifetime required in the type of `glossary`
|
38 |         spawn(move || process_files(worklist, glossary)) // erro
|         ^^^^^^ lifetime `'static` required

```

O Rust está reclamando sobre o tempo de vida da closure que estamos passando para `spawn` e a mensagem “útil” que o compilador apresenta aqui não ajuda em nada.

`spawn` lança threads independentes. O Rust não tem como saber por quanto tempo o thread filho será executado, então ele supõe o pior: ele supõe que o thread filho pode continuar executando mesmo depois que o thread pai terminou e todos os valores no thread pai desapareceram. Obviamente, se o thread filho vai durar tanto tempo, a closure que está executando também precisa durar tanto tempo. Mas essa closure tem um tempo de vida limitado: depende da referência `glossary` e as referências não duram para sempre.

Observe que o Rust está certo em rejeitar esse código! Da forma como escrevemos essa função, é possível que um thread encontre um erro de E/S, fazendo `process_files_in_parallel` sair antes que os outros threads sejam encerrados. Threads filhos podem acabar tentando utilizar o glossário depois de o thread principal o liberar. Seria uma corrida – com comportamento indefinido quanto ao prêmio, caso o thread principal vencesse. O Rust não pode permitir isso.

Parece que `spawn` é muito aberto para suportar o compartilhamento de referências entre threads. Aliás, já vimos um caso como esse, em “Closures que roubam”, na página 382. Lá, nossa solução foi transferir a posse dos dados para o novo thread, utilizando uma closure `move`. Isso não funcionará aqui, pois há muitos threads que precisam utilizar os mesmos dados. Uma alternativa segura é usar `clone` em todo o glossário para cada thread, mas, como é grande,

queremos evitar isso. Felizmente, a biblioteca padrão oferece outra maneira: contagem de referência atômica.

Descrevemos Arc em “Rc e Arc: Posse compartilhada”, na página 125. É hora de colocá-lo em uso:

```
use std::sync::Arc;
fn process_files_in_parallel(filenames: Vec<String>,
                             glossary: Arc<GigabyteMap>)
    -> io::Result<()>
{
    ...
    for worklist in worklists {
        // Essa chamada para .clone() apenas clona o Arc e aumenta a
        // contagem de referências. Ela não clona o GigabyteMap
        let glossary_for_child = glossary.clone();
        thread_handles.push(
            spawn(move || process_files(worklist, &glossary_for_child))
        );
    }
    ...
}
```

Alteramos o tipo de `glossary`: para executar a análise em paralelo, o chamador deve passar um `Arc<GigabyteMap>`, um ponteiro inteligente para um `GigabyteMap` que foi movido para o heap, utilizando `Arc::new(giga_map)`.

Quando chamamos `glossary.clone()`, estamos criando uma cópia do ponteiro inteligente `Arc`, não todo o `GigabyteMap`. Isso equivale a incrementar uma contagem de referências.

Com essa alteração, o programa compila e roda, pois não depende mais dos tempos de vida das referências. Desde que qualquer thread possua um `Arc<GigabyteMap>`, ele manterá o mapa ativo, mesmo que o thread pai saia mais cedo. Não haverá corridas de dados (data races), porque os dados em um `Arc` são imutáveis.

Rayon

A função `spawn` da biblioteca padrão é uma primitiva importante, mas não foi projetada especificamente para paralelismo fork-join. APIs de fork-join melhores foram construídas sobre ela. Por exemplo, no Capítulo 2 utilizamos a biblioteca Crossbeam para dividir algum

trabalho em oito threads. Os threads com escopo do Crossbeam suportam o paralelismo fork-join naturalmente.

A biblioteca Rayon, de Niko Matsakis e Josh Stone, é outro exemplo. Ela fornece duas maneiras de executar tarefas simultaneamente:

```
use rayon::prelude::*;

// "faça 2 coisas em paralelo"
let (v1, v2) = rayon::join(fn1, fn2);

// "faça N coisas em paralelo"
giant_vector.par_iter().for_each(|value| {
    do_thing_with_value(value);
});
```

`rayon::join(fn1, fn2)` simplesmente chama ambas as funções e retorna ambos os resultados. O método `.par_iter()` cria um `ParallelIterator`, um valor com `map`, `filter` e outros métodos, bem como um `Iterator` do Rust. Nos dois casos, o Rayon utiliza seu próprio pool de threads de trabalho para distribuir o trabalho quando possível. Você simplesmente diz ao Rayon quais tarefas podem ser feitas em paralelo; Rayon gerencia threads e distribui o trabalho da melhor maneira possível.

Os diagramas na Figura 19.3 ilustram duas maneiras de pensar sobre a chamada `giant_vector.par_iter().for_each(...)`. (a) Rayon age como se gerasse um thread por elemento no vetor. (b) Nos bastidores, Rayon tem um thread de trabalho por núcleo de CPU, o que é mais eficiente. Esse pool de threads de trabalho é compartilhado por todos os threads do seu programa. Quando milhares de tarefas chegam ao mesmo tempo, Rayon divide o trabalho.

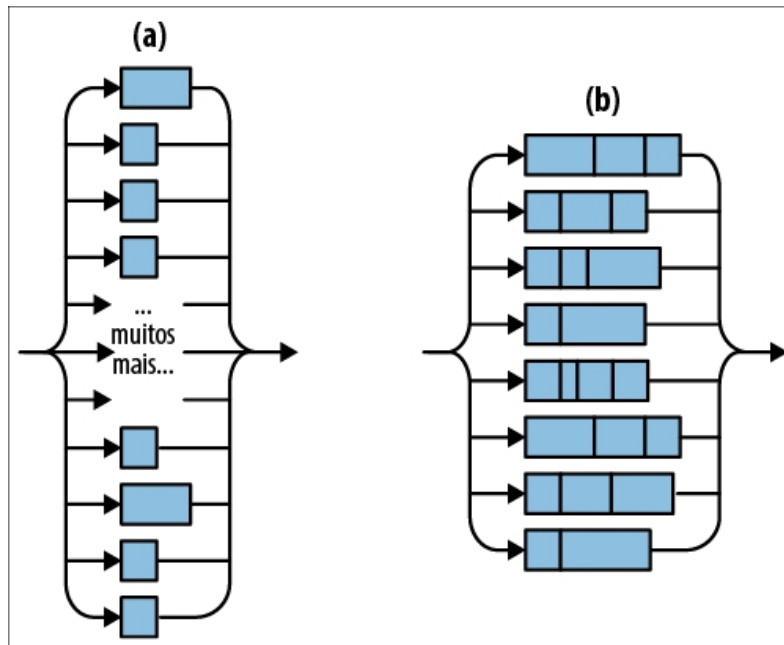


Figura 19.3: Rayon na teoria e na prática.

Eis uma versão de `process_files_in_parallel` utilizando Rayon e um `process_file` que recebe, em vez de `Vec<String>`, apenas um `&str`:

```
use rayon::prelude::*;
```

```
fn process_files_in_parallel(filenamees: Vec<String>, glossary: &GigabyteMap)
-> io::Result<()>
```

```
{
    filenamees.par_iter()
        .map(|filename| process_file(filename, glossary))
        .reduce_with(|r1, r2| {
            if r1.is_err() { r1 } else { r2 }
        })
        .unwrap_or(Ok(()))
}
```

Esse código é mais curto e menos complicado do que a versão utilizando `std::thread::spawn`. Vamos analisá-lo linha por linha:

- Primeiro, utilizamos `filenamees.par_iter()` para criar um iterador paralelo.
- Usamos `.map()` para chamar `process_file` em cada nome de arquivo. Isso produz um `ParallelIterator` sobre uma sequência de valores `io::Result<()>`.
- Utilizamos `.reduce_with()` para combinar os resultados. Aqui estamos mantendo o primeiro erro, se houver um, e descartando o

restante. Se quiséssemos acumular todos os erros, ou imprimi-los, poderíamos fazer isso aqui.

O método `.reduce_with()` também é útil quando você passa uma closure `.map()` que retorna um valor útil em caso de sucesso. Então você pode passar para `.reduce_with()` uma closure que sabe combinar dois resultados de sucesso.

- `reduce_with` retorna uma `Option` que é `None` somente se `filenames` estava vazio. Utilizamos o método `.unwrap_or()` de `Option` para produzir o resultado `Ok(())` nesse caso.

Nos bastidores, Rayon equilibra cargas de trabalho entre threads dinamicamente, utilizando uma técnica chamada roubo de trabalho (work stealing). Normalmente, ele fará um trabalho melhor mantendo todas as CPUs ocupadas do que podemos fazer dividindo manualmente o trabalho com antecedência, como em “spawn e join”, na página 568.

Como bônus, o Rayon oferece suporte ao compartilhamento de referências entre threads. Assegura-se que qualquer processamento paralelo que acontece nos bastidores seja concluído no momento em que `reduce_with` retorna. Isso explica por que conseguimos passar `glossary` para `process_file` mesmo que essa closure seja chamada em vários threads.

(Aliás, não é por acaso que utilizamos um método `map` e um método `reduce`. O modelo de programação MapReduce, popularizado pelo Google e Apache Hadoop, tem muito em comum com o fork-join. Pode ser visto como uma abordagem fork-join para consultar dados distribuídos.)

Revisitando o conjunto de Mandelbrot

No Capítulo 2, utilizamos concorrência fork-join para renderizar o conjunto Mandelbrot. Isso tornou a renderização quatro vezes mais rápida – impressionante, mas não tão impressionante quanto poderia ser, considerando que fizemos o programa gerar oito threads de trabalho e executá-lo em uma máquina de oito núcleos!

O problema é que não distribuímos a carga de trabalho

uniformemente. Calcular um pixel da imagem equivale a executar um loop (consulte “O que é realmente o conjunto de Mandelbrot”, na página 44). Acontece que as partes cinza-claro da imagem, onde o loop sai rapidamente, são muito mais rápidas de renderizar do que as partes pretas, onde o loop executa as 255 iterações completas. Portanto, embora tenhamos dividido a área em faixas horizontais de tamanho igual, estávamos criando cargas de trabalho desiguais, como mostra a Figura 19.4.

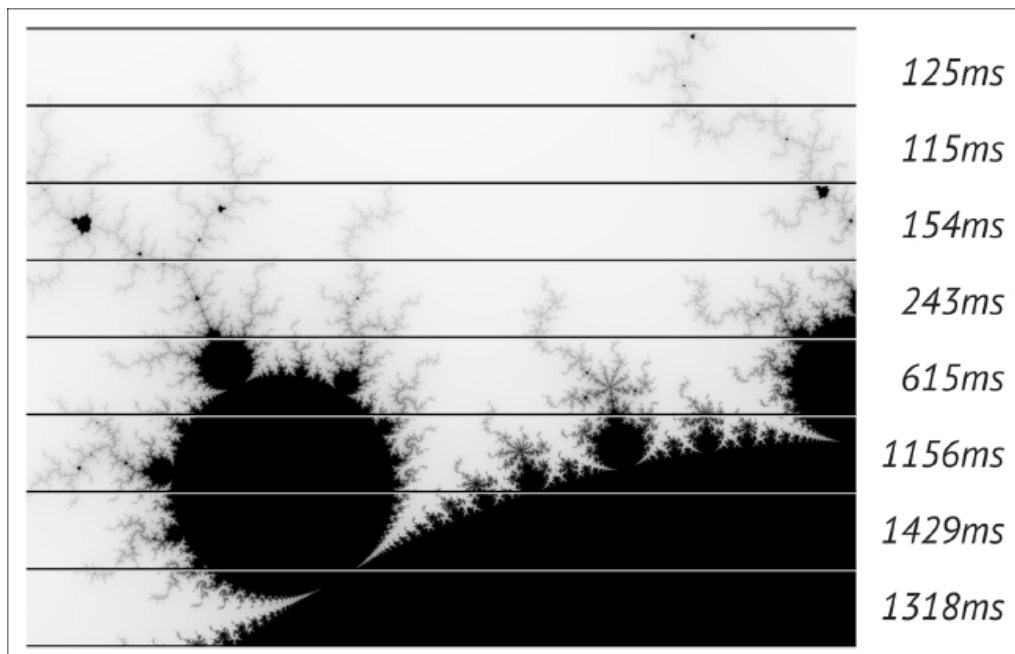


Figura 19.4: Distribuição de trabalho desigual no programa Mandelbrot.

Isso é fácil de corrigir utilizando o Rayon. Podemos simplesmente disparar uma tarefa paralela para cada linha de pixels na saída. Isso cria várias centenas de tarefas que o Rayon pode distribuir entre os threads. Graças ao roubo de trabalho, não importa se as tarefas variam em tamanho. O Rayon equilibrará o trabalho à medida que avança.

Eis o código. A primeira linha e a última linha são parte da função `main` que mostramos em “Um programa de Mandelbrot concorrente”, na página 56, mas alteramos o código de renderização, que é tudo entre:


```

let mut pixels = vec![0; bounds.0 * bounds.1];

// Escopo de fatiar 'pixels' em faixas horizontais
{
    let bands: Vec<(usize, &mut [u8])> = pixels
        .chunks_mut(bounds.0)
        .enumerate()
        .collect();

    bands.into_par_iter()
        .for_each(|(i, band)| {
            let top = i;
            let band_bounds = (bounds.0, 1);
            let band_upper_left = pixel_to_point(bounds, (0, top),
                                                    upper_left, lower_right);
            let band_lower_right = pixel_to_point(bounds, (bounds.0, top + 1),
                                                    upper_left, lower_right);
            render(band, band_bounds, band_upper_left, band_lower_right);
        });
}

```

```

write_image(&args[1], &pixels, bounds).expect("error writing PNG file");

```

Primeiro, criamos `bands`, a coleção de tarefas que passaremos para o Rayon. Cada tarefa é apenas uma tupla do tipo `(usize, &mut [u8])`: o número da linha, já que o cálculo exige isso e a fatia de `pixels` para preencher. Utilizamos o método `chunks_mut` para transformar o buffer de imagem em linhas, `enumerate` para anexar um número de linha a cada linha e `collect` para transformar todos os pares número-fatia em um vetor. (Precisamos de um vetor porque o Rayon cria iteradores paralelos apenas a partir de arrays e vetores.)

A seguir, transformamos `bands` em um iterador paralelo e utilizamos o método `.for_each()` para dizer a Rayon qual trabalho queremos fazer.

Como estamos utilizando o Rayon, devemos adicionar esta linha a *main.rs*:

```

use rayon::prelude::*;

```

e isto a *Cargo.toml*:

```

[dependencies]
rayon = "1"

```

Com essas alterações, o programa agora utiliza cerca de 7,75 núcleos em uma máquina de 8 núcleos. É 75% mais rápido do que antes, quando o trabalho foi dividido manualmente. E o código é

um pouco mais curto, refletindo os benefícios de deixar um crate fazer um trabalho (distribuição de trabalho) em vez de nós mesmos fazermos.

Canais

Um canal é um conduto unidirecional para enviar valores de um thread para outro. Em outras palavras, é uma fila thread-safe.

A Figura 19.5 ilustra como os canais são utilizados. Eles são algo como pipes do Unix: uma extremidade é para enviar dados e a outra para receber. As duas extremidades normalmente são possuídas por dois segmentos diferentes. Mas, embora os pipes no Unix sejam utilizados para enviar bytes, os canais são usados para enviar valores no Rust. `sender.send(item)` insere um único valor no canal; `receiver.recv()` remove um. A posse é transferida do thread de envio para o thread de recebimento. Se o canal estiver vazio, `receiver.recv()` bloqueia até um valor ser enviado.

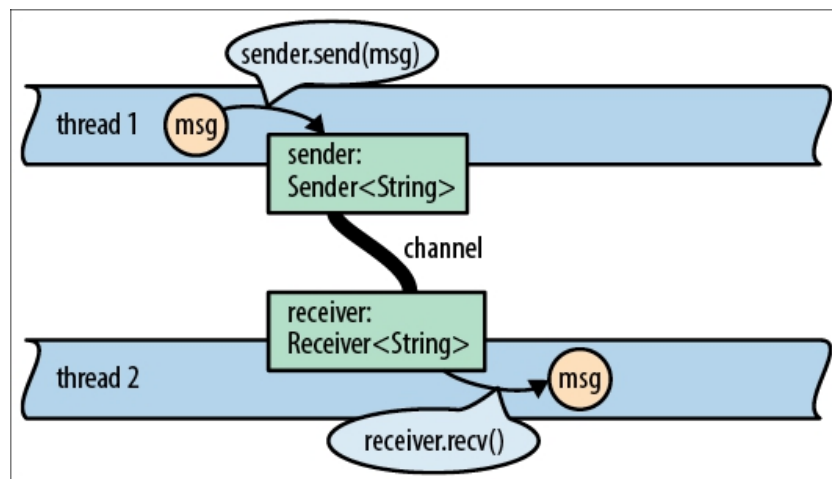


Figura 19.5: Um canal para Strings: a posse da string msg é transferida do thread 1 para o thread 2.

Com canais, threads podem se comunicar passando valores entre si. É uma maneira muito simples de os threads trabalharem juntos sem utilizar bloqueio (locking) ou memória compartilhada.

Isso não é uma técnica nova. Na linguagem Erlang, os processos e a passagem de mensagem são mantidos isolados há 30 anos. Os pipes

do Unix existem há quase 50 anos. Nossa tendência é pensar em pipes como fornecendo flexibilidade e capacidade de composição, não concorrência, mas, na verdade, eles fazem tudo o que foi dito acima. Um exemplo de pipeline do Unix é mostrado na Figura 19.6. Certamente é possível que os três programas funcionem ao mesmo tempo.

Canais do Rust são mais rápidos que pipes do Unix. Enviar um valor move-o em vez de copiá-lo e os movimentos são rápidos mesmo quando você está movendo estruturas de dados que contêm muitos megabytes de dados.

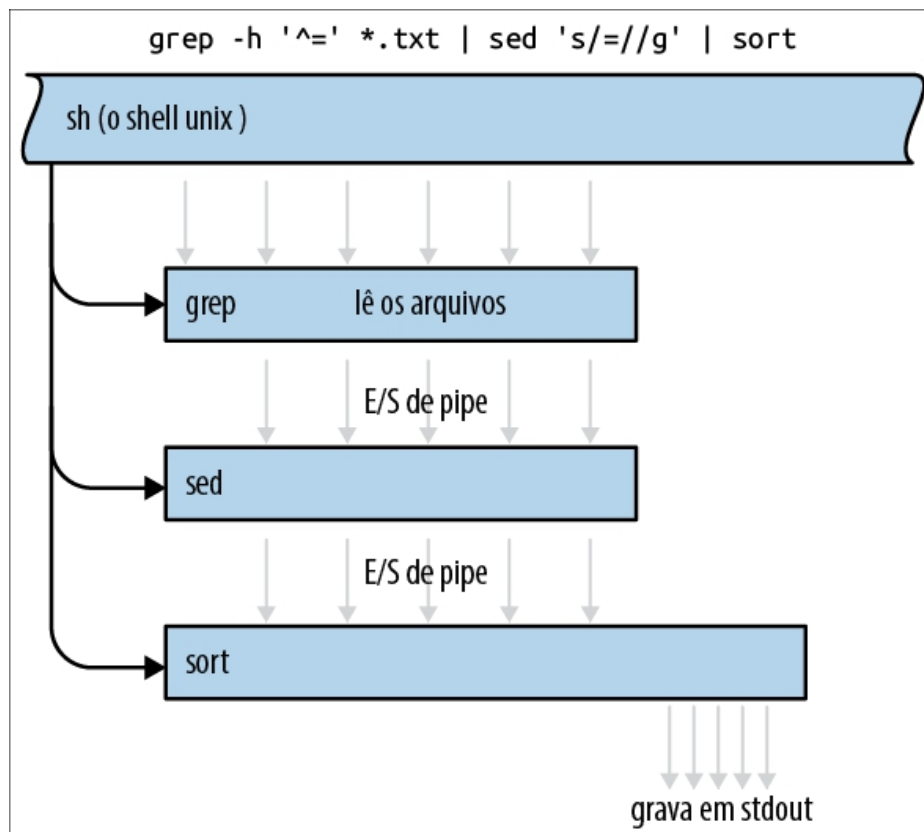


Figura 19.6: Execução de um pipeline do Unix.

Enviando valores

Nas próximas seções, utilizaremos canais para construir um programa concorrente que cria um índice invertido, um dos principais ingredientes de um mecanismo de pesquisa. Cada

mecanismo de pesquisa funciona em uma determinada coleção de documentos. O índice invertido é o banco de dados que informa quais palavras aparecem onde.

Mostraremos as partes do código relacionadas a threads e canais. O programa completo (<https://oreil.ly/yF3me>) é curto, cerca de mil linhas de código ao todo.

Nosso programa é estruturado como um pipeline, conforme mostrado na Figura 19.7. Pipelines são apenas uma das muitas maneiras de utilizar canais – discutiremos alguns outros usos mais adiante –, mas são uma maneira simples e direta de introduzir concorrência em um programa de thread única existente.

Utilizaremos um total de cinco threads, cada um realizando uma tarefa distinta. Cada thread produz continuamente saída durante o tempo de vida do programa. O primeiro thread, por exemplo, simplesmente lê os documentos de origem do disco para a memória, um por um. (Queremos um thread para fazer isso porque escreveremos o código mais simples possível aqui, utilizando `fs::read_to_string`, que é uma API que bloqueia. Não queremos que a CPU fique ociosa sempre que o disco estiver funcionando.) A saída dessa etapa é uma longa `String` por documento, assim esse thread é conectado ao próximo thread por um canal de `Strings`.

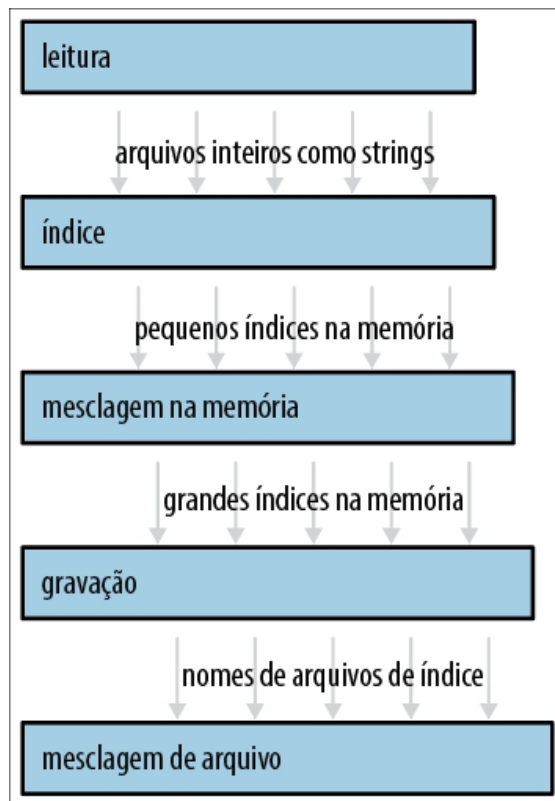


Figura 19.7: O pipeline do construtor de índice, no qual as setas representam valores enviados por meio de um canal entre um thread e outro (a E/S de disco não é mostrada).

Inicialmente, nosso programa vai gerar o thread que lê os arquivos. Suponha que `documents` é um `Vec<PathBuf>`, um vetor dos nomes de arquivo. O código para iniciar nosso thread de leitura de arquivo se parece com isto:

```

use std::{fs, thread};
use std::sync::mpsc;

let (sender, receiver) = mpsc::channel();

let handle = thread::spawn(move || {
    for filename in documents {
        let text = fs::read_to_string(filename)?;

        if sender.send(text).is_err() {
            break;
        }
    }
    Ok(())
});
```

Os canais são parte do módulo `std::sync::mpsc`. Explicaremos o que esse nome significa mais tarde; primeiro, vamos ver como esse código funciona. Começamos criando um canal:

```
let (sender, receiver) = mpsc::channel();
```

A função `channel` retorna um par de valores: um remetente e um destinatário. A estrutura de dados da fila subjacente é um detalhe de implementação que a biblioteca padrão não expõe.

Os canais são tipados. Vamos utilizar esse canal para enviar o texto de cada arquivo, assim temos um `sender` do tipo `Sender<String>` e um `receiver` do tipo `Receiver<String>`. Poderíamos ter solicitado explicitamente um canal de strings, escrevendo `mpsc::channel::<String>()`. Em vez disso, deixamos a inferência de tipo do Rust resolver isso.

```
let handle = thread::spawn(move || {
```

Como antes, utilizamos `std::thread::spawn` para iniciar um thread. A posse de `sender` (mas não de `receiver`) é transferida para o novo thread por meio dessa closure `move`.

As próximas linhas do código simplesmente leem arquivos do disco:

```
    for filename in documents {  
        let text = fs::read_to_string(filename)?;
```

Depois de ler um arquivo com sucesso, enviamos o texto para o canal:

```
        if sender.send(text).is_err() {  
            break;  
        }  
    }
```

`sender.send(text)` move o valor `text` para o canal. Com o tempo, ele será transferido novamente para quem receber o valor. Se `text` contém 10 linhas de texto ou 10 megabytes, essa operação copia três palavras de máquina (do tamanho de um struct `String`) e a chamada `receiver.recv()` correspondente também copiará três palavras de máquina.

Os métodos `send` e `recv` retornam `Results`, mas esses métodos falham apenas se a outra extremidade do canal for dropada. Uma chamada a `send` falha se o `Receiver` foi dropado porque, do contrário, o valor permaneceria no canal para sempre: sem um `Receiver`, não há como

um thread recebê-lo. Da mesma forma, uma chamada a `recv` falha se não houver valores esperando no canal e o `Sender` foi descartado porque, do contrário, `recv` esperaria para sempre: sem um `Sender`, não há como um thread enviar o próximo valor. Dropar a extremidade de um canal é a maneira normal de “desligar”, fechando a conexão quando você terminar.

Em nosso código, `sender.send(text)` só vai falhar se o thread do receptor foi encerrado antecipadamente. Isso é típico do código que utiliza canais. Se isso aconteceu deliberadamente ou devido a um erro, não há problema se nosso thread de leitura se desligar silenciosamente.

Quando isso acontece, ou o thread termina de ler todos os documentos, `Ok()` é retornado:

```
Ok()  
});
```

Observe que essa closure retorna um `Result`. Se o thread encontrar um erro de E/S, ele será encerrado imediatamente e o erro será armazenado na memória do thread `JoinHandle`.

Naturalmente, assim como qualquer outra linguagem de programação, o Rust admite muitas outras possibilidades quando chega o momento do tratamento de erros. Quando ocorre um erro, podemos apenas imprimi-lo utilizando `println!` e passar para o próximo arquivo. Podemos passar erros pelo mesmo canal que estamos utilizando para dados, tornando-o um canal de `Results` – ou criar um segundo canal apenas para erros. A abordagem que escolhemos aqui é leve e responsável: podemos utilizar o operador `?`, então não há um monte de código, ou mesmo um `try/catch` explícito como podemos ver no Java, e mesmo assim os erros não passarão silenciosamente.

Por conveniência, nosso programa empacota todo esse código em uma função que retorna tanto o `receiver` (que ainda não utilizamos) como o `JoinHandle` do novo thread:

```
fn start_file_reader_thread(documents: Vec<PathBuf>)  
-> (mpsc::Receiver<String>, thread::JoinHandle<io::Result<>>)  
{  
    let (sender, receiver) = mpsc::channel();
```

```

    let handle = thread::spawn(move || {
        ...
    });
    (receiver, handle)
}

```

Observe que essa função inicia o novo thread e retorna imediatamente. Escreveremos uma função como essa para cada etapa do nosso pipeline.

Recebendo valores

Agora temos um thread executando um loop que envia valores. Podemos gerar um segundo thread executando um loop que chama `receiver.recv()`:

```

while let Ok(text) = receiver.recv() {
    do_something_with(text);
}

```

Mas `Receivers` são iteráveis, então há uma maneira melhor de escrever isso:

```

for text in receiver {
    do_something_with(text);
}

```

Esses dois loops são equivalentes. De qualquer maneira como escrevemos isso, se o canal estiver vazio quando o controle atingir o topo do loop, o thread receptor será bloqueado até que algum outro thread envie um valor. O loop será encerrado normalmente quando o canal estiver vazio e o `Sender` tiver sido dropado. Em nosso programa, isso acontece naturalmente quando o thread leitor é encerrado. Esse thread está executando uma closure que possui a variável `sender`; quando a closure é encerrada, `sender` é dropada.

Agora podemos escrever o código para a segunda etapa do pipeline:

```

fn start_file_indexing_thread(texts: mpsc::Receiver<String>)
-> (mpsc::Receiver<InMemoryIndex>, thread::JoinHandle<()>)
{
    let (sender, receiver) = mpsc::channel();

    let handle = thread::spawn(move || {
        for (doc_id, text) in texts.into_iter().enumerate() {
            let index = InMemoryIndex::from_single_document(doc_id, text);

```



```

        if sender.send(index).is_err() {
            break;
        }
    }
});
(receiver, handle)
}

```

Essa função gera um thread que recebe valores `String` de um canal (`texts`) e envia valores `InMemoryIndex` para outro canal (`sender/receiver`). A tarefa desse thread é pegar cada um dos arquivos carregados na primeira etapa e transformar cada documento em um pequeno índice invertido na memória de um único arquivo.

O loop principal desse thread é simples e direto. Todo o trabalho de indexação de um documento é feito pela função `InMemoryIndex::from_single_document`. Não mostraremos o código-fonte aqui, mas ele divide a string de entrada nos limites do texto e, em seguida, produz um mapa das palavras a listas de posições.

Essa etapa não executa E/S, portanto não precisa lidar com `io::ErrorS`. Em vez de um `io::Result<()>`, retorna `()`.

Executando o pipeline

As três etapas restantes são semelhantes em termos de design. Cada uma consome um `Receiver` criado pela etapa anterior. Nosso objetivo para o restante do pipeline é mesclar todos os índices pequenos em um único arquivo de índice grande no disco. A maneira mais rápida que encontramos para fazer isso é em três etapas. Não mostraremos o código aqui, apenas as assinaturas de tipo dessas três funções. O código-fonte completo está on-line.

Primeiro, mesclamos os índices na memória até que fiquem pesados (etapa 3):

```

fn start_in_memory_merge_thread(file_indexes: mpsc::Receiver<InMemoryIndex>)
-> (mpsc::Receiver<InMemoryIndex>, thread::JoinHandle<()>)

```

Gravamos esses índices grandes no disco (etapa 4):

```

fn start_index_writer_thread(big_indexes: mpsc::Receiver<InMemoryIndex>,
                             output_dir: &Path)
-> (mpsc::Receiver<PathBuf>, thread::JoinHandle<io::Result<()>>)

```

Por fim, se houver vários arquivos grandes, vamos mesclá-los utilizando um algoritmo de mesclagem baseado em arquivo (etapa 5):

```
fn merge_index_files(files: mpsc::Receiver<PathBuf>, output_dir: &Path)
-> io::Result<()>
```

Essa última etapa não retorna um `Receiver`, porque é o fim da linha. Ela produz um único arquivo de saída no disco. Ela não retorna um `JoinHandle`, porque não nos preocupamos em gerar um thread nessa etapa. O trabalho é feito no thread do chamador.

Agora chegamos ao código que inicia os threads e verifica se há erros:

```
fn run_pipeline(documents: Vec<PathBuf>, output_dir: PathBuf)
-> io::Result<()>
{
    // Carrega todas as cinco etapas do pipeline
    let (texts, h1) = start_file_reader_thread(documents);
    let (pints, h2) = start_file_indexing_thread(texts);
    let (gallons, h3) = start_in_memory_merge_thread(pints);
    let (files, h4) = start_index_writer_thread(gallons, &output_dir);
    let result = merge_index_files(files, &output_dir);

    // Espera até que os threads sejam encerrados, parando em qualquer erro encontrado
    let r1 = h1.join().unwrap();
    h2.join().unwrap();
    h3.join().unwrap();
    let r4 = h4.join().unwrap();

    // Retorna o primeiro erro encontrado, se houver um.
    // (Acontece que h2 e h3 não podem falhar: esses threads
    // são puro processamento de dados na memória.)
    r1?;
    r4?;
    result
}
```

Como antes, utilizamos `.join().unwrap()` para propagar explicitamente o pânico gerado em threads filhos para o thread principal. A única outra coisa incomum aqui é que, em vez de utilizar `?` imediatamente, deixamos de lado os valores `io::Result` até que tenhamos juntado todos os quatro threads.

Esse pipeline é 40% mais rápido que o equivalente de thread único. Isso não é ruim para uma tarde de trabalho, mas insignificante perto

do aumento de 675% que obtivemos para o programa Mandelbrot. Claramente não saturamos a capacidade de E/S do sistema nem todos os núcleos da CPU. O que está acontecendo?

Pipelines são como linhas de montagem em uma fábrica: o desempenho é limitado pela produção da etapa mais lenta. Uma linha de montagem nova e dessintonizada pode ser tão lenta quanto a produção de unidades, mas as linhas de montagem recompensam o ajuste direcionado. No nosso caso, a medição mostra que a segunda etapa é o gargalo. Nosso thread de indexação utiliza `.to_lowercase()` e `.is_alphanumeric()`, então ele gasta muito tempo vasculhando tabelas Unicode. As outras etapas posteriores à indexação passam a maior parte do tempo em repouso em `Receiver::recv`, esperando a entrada.

Isso significa que devemos ser capazes de agilizar o processo. À medida que abordamos os gargalos, o grau de paralelismo aumentará. Agora que você sabe como utilizar canais e nosso programa é feito de fragmentos isolados de código, é fácil ver maneiras de resolver esse primeiro gargalo. Poderíamos otimizar manualmente o código para a segunda etapa, como qualquer outro código; dividir o trabalho em duas ou mais etapas; ou executar vários threads de indexação de arquivo de uma só vez.

Recursos e desempenho do canal

A `mpsc` parte de `std::sync::mpsc` significa multiprodutor, consumidor único, uma descrição concisa do tipo de comunicação que os canais do Rust fornecem.

Os canais em nosso programa de exemplo transportam valores entre um único emissor e um único receptor. Isso é um caso bastante comum. Mas os canais do Rust também oferecem suporte a vários remetentes, caso você precise, digamos, de um único thread que lide com solicitações de muitos threads de clientes, conforme mostrado na Figura 19.8.

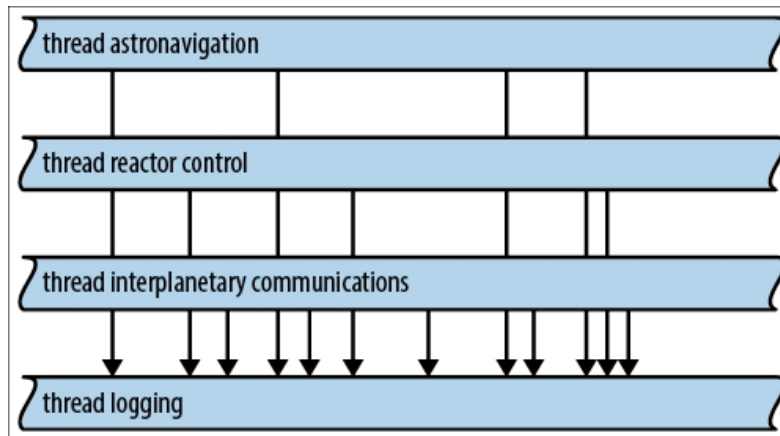


Figura 19.8: Um único canal recebendo solicitações de muitos remetentes.

`Sender<T>` implementa o trait `Clone`. Para obter um canal com vários remetentes, basta criar um canal normal e clonar o remetente quantas vezes quiser. Você pode mover cada valor `Sender` para um thread diferente.

Um `Receiver<T>` não pode ser clonado; assim, se for necessário que vários threads recebam valores do mesmo canal, você precisa de um `Mutex`. Mostraremos como fazer isso mais adiante neste capítulo.

Os canais do Rust são cuidadosamente otimizados. Quando um canal é criado pela primeira vez, o Rust utiliza uma implementação especial de fila "one-shot". Se você enviar apenas um objeto pelo canal, a sobrecarga será mínima. Se você enviar um segundo valor, o Rust vai alternar para uma implementação de fila diferente. Está realmente se adequando para o longo prazo, preparando o canal para transferir muitos valores enquanto minimiza a sobrecarga de alocação. E se você clonar o `Sender`, o Rust deve recorrer a outra implementação, uma que seja segura quando vários threads estão tentando enviar valores de uma só vez. Mas mesmo a mais lenta dessas três implementações é uma fila sem bloqueio (lock-free), portanto enviar ou receber um valor demanda, no máximo, algumas operações atômicas e uma alocação de heap, mais a própria movimentação. As chamadas de sistema são necessárias somente quando a fila está vazia e, portanto, o thread receptor precisa ser colocado no modo de espera. Nesse caso, é claro, o tráfego em seu

canal não é maximizado de qualquer maneira.

Apesar de todo esse trabalho de otimização, há um erro muito fácil que os aplicativos cometem em relação ao desempenho do canal: enviar valores mais rapidamente do que eles podem ser recebidos e processados. Isso faz com que uma demanda não atendida cada vez maior de valores se acumule no canal. Por exemplo, em nosso programa, descobrimos que o thread leitor de arquivo (etapa 1) poderia carregar arquivos muito mais rápido do que o thread de indexação de arquivos (etapa 2) poderia indexá-los. O resultado é que centenas de megabytes de dados brutos seriam lidos do disco e colocados na fila de uma só vez.

Esse tipo de mau comportamento custa memória e prejudica a localidade. Pior ainda, o thread de envio continua em execução, utilizando a CPU e outros recursos do sistema para enviar cada vez mais valores exatamente quando esses recursos são mais necessários na extremidade de recebimento.

Aqui, o Rust novamente pega uma página dos pipes do Unix. O Unix utiliza um truque elegante para fornecer alguma contrapressão para que os remetentes rápidos sejam forçados a desacelerar: cada pipe em um sistema Unix tem um tamanho fixo e, se um processo tentar escrever em um pipe momentaneamente cheio, o sistema simplesmente bloqueia esse processo até que haja espaço no pipe. O equivalente Rust é chamado canal síncrono:

```
use std::sync::mpsc;
```

```
let (sender, receiver) = mpsc::sync_channel(1000);
```

Um canal síncrono é exatamente como um canal regular, exceto que, ao criá-lo, você especifica quantos valores ele pode conter. Para um canal síncrono, `sender.send(value)` é potencialmente uma operação de bloqueio. Afinal, a ideia é que bloquear nem sempre é ruim. Em nosso programa de exemplo, alterar o `channel` em `start_file_reader_thread` para um `sync_channel` com espaço para 32 valores reduz o uso de memória em dois terços em nosso conjunto de dados de referência, sem diminuir a taxa de transferência.

Segurança de thread: Send e Sync

Até agora, temos agido como se todos os valores pudessem ser movidos e compartilhados livremente entre os threads. Isso é verdade, mas a história completa de segurança de thread do Rust depende de dois traits internos, `std::marker::Send` e `std::marker::Sync`.

- Tipos que implementam `Send` são seguros para passar por valor para outro thread. Eles podem ser movidos entre threads.
- Tipos que implementam `Sync` são seguros para passar por referência não-mut a outro thread. Eles podem ser compartilhados entre threads.

Por *seguro* aqui, queremos dizer a mesma coisa que sempre queremos dizer: livre de corridas de dados (data races) e outro comportamento indefinido.

Por exemplo, no `process_files_in_parallel` de exemplo, na página 567, utilizamos uma closure para passar um `Vec<String>` do thread pai para cada thread filho. Não especificamos isso na época, mas isso significa que o vetor e suas strings são alocados no thread pai, mas liberados no thread filho. O fato de que `Vec<String>` implementa `Send` é uma promessa de API de que está tudo bem: o alocador utilizado internamente por `Vec` e `String` é thread-safe.

(Se você fosse escrever explicitamente seus próprios tipos `Vec` e `String` com alocadores rápidos, mas não thread-safe, teria de implementá-los utilizando tipos que não são `Send`, como ponteiros inseguros. O Rust então inferiria que seus tipos `NonThreadSafeVec` e `NonThreadSafeString` não são `Send` e iria restringi-los ao uso de thread única. Mas isso é um caso raro.)

Como a Figura 19.9 ilustra, a maioria dos tipos são tanto `Send` como `Sync`. Você nem precisa utilizar `#[derive]` para obter essas traits em structs e enums em seu programa. O Rust faz isso por você. Um struct ou enum é `Send` se os campos são `Send` e `Sync` se os campos são `Sync`.

Alguns tipos são `Send`, mas não `Sync`. Isso geralmente é proposital, como no caso de `mpsc::Receiver`, em que este assegura que a

extremidade receptora de um canal `mpsc` é utilizada apenas por um thread por vez.

Os poucos tipos que não são nem `Send` nem `Sync` são principalmente aqueles que utilizam mutabilidade de uma maneira que não é thread-safe. Por exemplo, considere `std::rc::Rc<T>` o tipo dos ponteiros inteligentes de contagem de referências.

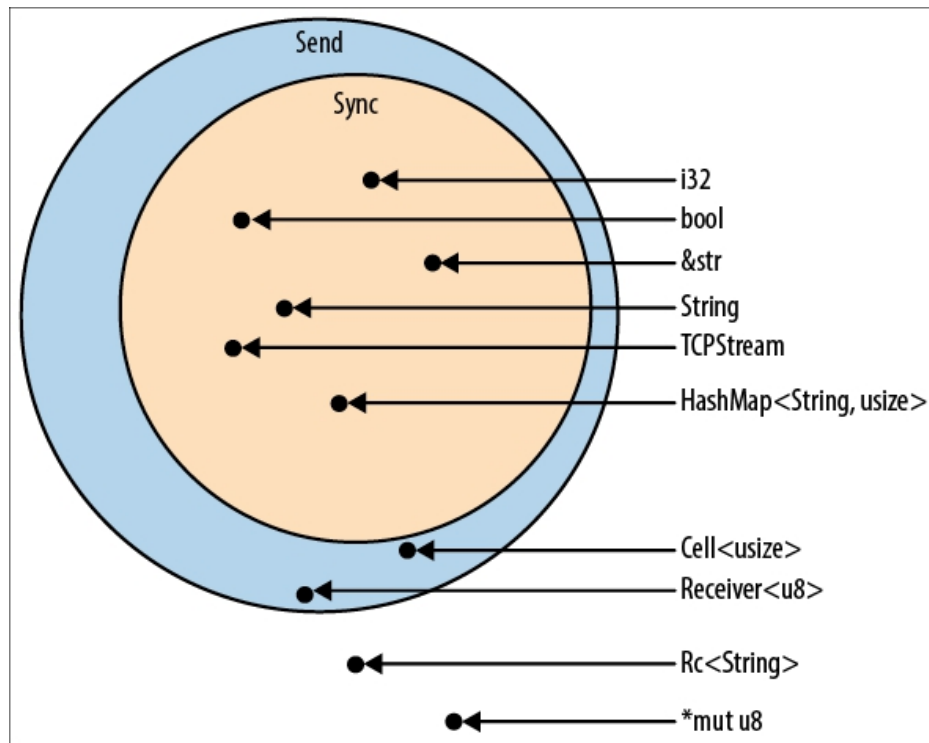


Figura 19.9: Tipos `Send` e `Sync`.

O que aconteceria se `Rc<String>` fosse `Sync`, permitindo que threads compartilhem um único `Rc` por meio de referências compartilhadas? Se ambos os threads tentarem clonar o `Rc` ao mesmo tempo, conforme mostrado na Figura 19.10, temos uma corrida de dados (data race) porque ambos os threads incrementam a contagem de referências compartilhadas. A contagem de referências pode se tornar imprecisa, levando mais tarde a um comportamento indefinido de uso após liberação (*use-after-free*) ou de liberação dupla (*double free*).

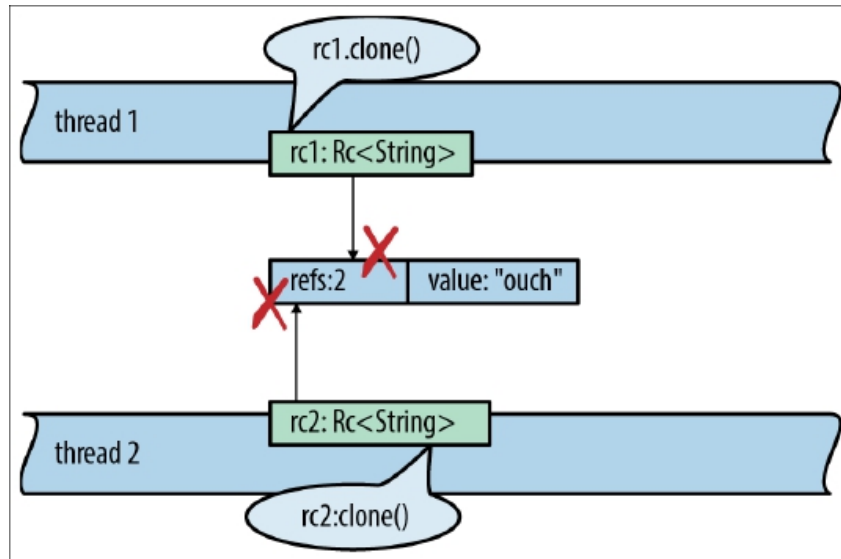


Figura 19.10: Por que `Rc<String>` não é `Sync` nem `Send`.

É claro que o Rust impede isso. Eis o código para configurar essa corrida de dados (data race):

```
use std::thread;
use std::rc::Rc;

fn main() {
    let rc1 = Rc::new("ouch".to_string());
    let rc2 = rc1.clone();
    thread::spawn(move || { // erro
        rc2.clone();
    });
    rc1.clone();
}
```

O Rust se recusa a compilá-lo, dando uma mensagem de erro detalhada:

```
error: `Rc<String>` cannot be sent between threads safely
|
10 |   thread::spawn(move || { // erro
|       ^^^^^ `Rc<String>` cannot be sent between threads safely
|
= help: the trait `std::marker::Send` is not implemented for `Rc<String>`
= note: required because it appears within the type `[closure@...]`
= note: required by `std::thread::spawn`
```

Agora você pode ver como `Send` e `Sync` ajudam o Rust a reforçar a segurança dos threads. Eles aparecem como limites na assinatura de tipo das funções que transferem dados entre os limites do thread.

Quando você utiliza `spawn` em um thread, a closure que você passar deve ser `Send`, o que significa que todos os valores que ele contém devem ser `Send`. Da mesma forma, se você quiser enviar valores por meio de um canal para outro thread, os valores devem ser `Send`.

Utilizando pipes em quase qualquer iterador para um canal

Nosso construtor de índice invertido é criado como um pipeline. O código é claro o suficiente, mas nos permite configurar canais manualmente e iniciar threads. Por outro lado, os pipelines do iterador que construímos no Capítulo 15 pareciam empacotar muito mais trabalho em apenas algumas linhas de código. Podemos construir algo assim para pipelines de threads?

Na verdade, seria bom se pudéssemos unificar pipelines de iteradores e pipelines de threads. Então, nosso construtor de índice pode ser escrito como um pipeline de iterador. Pode começar assim:

```
documents.into_iter()
  .map(read_whole_file)
  .errors_to(error_sender) // filtra os resultados de erro
  .off_thread()           // gera um thread para o trabalho acima
  .map(make_single_file_index)
  .off_thread()           // gera outro thread para a etapa 2
  ...
```

Traits permitem adicionar métodos aos tipos de biblioteca padrão, para que possamos realmente fazer isso. Começamos escrevendo um trait que declara o método que queremos:

```
use std::sync::mpsc;

pub trait OffThreadExt: Iterator {
  /// Transforma esse iterador em um iterador off-thread: as chamadas
  /// `next()` acontecem em um thread de trabalho separado, então o
  /// iterador e o corpo do seu loop são executados simultaneamente
  fn off_thread(self) -> mpsc::IntoIter<Self::Item>;
}
```

Em seguida, implementamos essa trait para tipos de iteradores. Ajuda o fato de que `mpsc::Receiver` já é iterável:

```
use std::thread;
```

```

impl<T> OffThreadExt for T
  where T: Iterator + Send + 'static,
        T::Item: Send + 'static
{
  fn off_thread(self) -> mpsc::IntoIter<Self::Item> {
    // Cria um canal para transferir itens do thread de trabalho.
    let (sender, receiver) = mpsc::sync_channel(1024);

    // Move esse iterador para um novo thread de trabalho e executa-o lá
    thread::spawn(move || {
      for item in self {
        if sender.send(item).is_err() {
          break;
        }
      }
    });

    // Retorna um iterador que extrai valores do canal
    receiver.into_iter()
  }
}

```

A cláusula `where` nesse código foi determinada por meio de um processo muito parecido com o descrito em Engenharia reversa de limites, na página 329. Inicialmente, tínhamos apenas isto:

```
impl<T> OffThreadExt for T
```

Ou seja, queríamos que a implementação funcionasse para todos os iteradores. O Rust não tinha nada disso. Como estamos utilizando `spawn` para mover um iterador do tipo `T` para um novo thread, devemos especificar `T: Iterator + Send + 'static`. Como estamos enviando os itens de volta por um canal, devemos especificar `T::Item: Send + 'static`. Com essas alterações, o Rust ficou satisfeito.

Esta é a característica do Rust em poucas palavras: estamos livres para adicionar uma poderosa ferramenta de concorrência a quase todos os iteradores da linguagem – mas não sem primeiro entender e documentar as restrições que tornam seu uso seguro.

Além de pipelines

Nesta seção, utilizamos pipelines como exemplos porque eles são uma maneira boa e óbvia de utilizar canais. Todo mundo os compreende. São concretos, práticos e determinísticos. Mas canais

são úteis para mais do que apenas pipelines. Eles também são uma maneira rápida e fácil de oferecer qualquer serviço assíncrono a outros threads no mesmo processo.

Por exemplo, suponha que você queira fazer log em seu próprio thread, como na Figura 19.8. Outros threads podem enviar mensagens de log para o thread de logging em um canal; como você pode clonar o `Sender` do canal, muitos threads clientes podem ter remetentes que enviam mensagens de log para o mesmo thread de logging.

A execução de um serviço como o log em seu próprio thread tem vantagens. O thread de registro em log pode alternar os arquivos de log sempre que necessário. Ele não precisa fazer nenhuma coordenação sofisticada com os outros threads. Esses threads não serão bloqueados. As mensagens se acumularão inofensivamente no canal por um momento até que o thread de registro em log volte a funcionar.

Canais também podem ser utilizados para casos em que um thread envia uma solicitação para outro thread e precisa obter algum tipo de resposta de volta. A solicitação do primeiro thread pode ser um struct ou uma tupla que inclui um `Sender`, uma espécie de envelope autoendereçado que o segundo thread utiliza para enviar a resposta. Isso não significa que a interação deve ser síncrona. O primeiro thread decide se deseja bloquear e aguardar a resposta ou utilizar o método `.try_rcv()` para pesquisar nele.

As ferramentas que apresentamos até agora – `fork-join` para computação altamente paralela, canais para componentes de conexão flexível – são suficientes para uma ampla variedade de aplicações. Mas ainda não terminamos.

Estado mutável compartilhado

Nos meses desde que você publicou o crate `fern_sim` no Capítulo 8, seu software de simulação de gimnospermas realmente decolou. Agora você está criando um jogo de estratégia multijogador em tempo real no qual oito jogadores competem para cultivar

gimnospermas autênticas em uma paisagem jurássica simulada. O servidor desse jogo é um aplicativo massivamente paralelo, com solicitações chegando em muitos threads. Como esses threads podem se coordenar para iniciar um jogo assim que oito jogadores estão disponíveis?

O problema a ser resolvido aqui é que muitos threads precisam acessar uma lista compartilhada de jogadores que estão esperando para entrar em um jogo. Esses dados são necessariamente mutáveis e compartilhados entre todos os threads. Se o Rust não tiver um estado mutável compartilhado, onde isso nos deixa?

Você pode resolver isso criando um novo thread cujo trabalho é gerenciar essa lista. Outros threads se comunicariam com ele por meio de canais. Naturalmente, isso custa um thread, que tem alguma sobrecarga do sistema operacional.

Outra opção é utilizar as ferramentas fornecidas pelo Rust para compartilhar dados mutáveis com segurança. Essas coisas existem. São primitivas de baixo nível que serão familiares a qualquer programador de sistema que tenha trabalhado com threads. Nesta seção, abordaremos mutexes, bloqueios de leitura/escrita, variáveis de condição e inteiros atômicos. Por fim, mostraremos como implementar variáveis globais mutáveis no Rust.

O que é um mutex?

Um mutex (ou lock) é utilizado para forçar vários threads a se revezarem ao acessar determinados dados. Apresentaremos os mutexes do Rust na próxima seção. Primeiro, faz sentido lembrar como que mutexes se parecem em outras linguagens. Um uso simples de um mutex em C++ pode se parecer com isto:

```
// C++ code, not Rust
void FernEmpireApp::JoinWaitingList(PlayerId player) {
    mutex.Acquire();

    waitingList.push_back(player);

    // Começa um jogo se tivermos jogadores suficientes esperando
    if (waitingList.size() >= GAME_SIZE) {
        vector<PlayerId> players;
```

```
        waitingList.swap(players);  
        StartGame(players);  
    }  
  
    mutex.Release();  
}
```

As chamadas `mutex.Acquire()` e `mutex.Release()` marcam o início e o fim de uma seção crítica nesse código. Para cada `mutex` em um programa, apenas um thread pode ser executado dentro de uma seção crítica por vez. Se um thread estiver em uma seção crítica, todos os outros threads que chamam `mutex.Acquire()` serão bloqueados até que o primeiro thread alcance `mutex.Release()`.

Dizemos que o mutex protege os dados: nesse caso, `mutex` protege `waitingList`. É responsabilidade do programador, porém, garantir que cada thread sempre adquira o mutex antes de acessar os dados e o libere depois.

Mutexes são úteis por vários motivos:

- Elas impedem corridas de dados (data races), situações em que threads competem para ler e escrever simultaneamente na mesma memória. As corridas de dados são um comportamento indefinido em C++ e Go. Linguagens gerenciadas como Java e C# prometem não travar, mas os resultados das corridas de dados ainda são (para resumir) sem sentido.
- Mesmo que não existissem corridas de dados, mesmo que todas as leituras e escritas ocorressem uma a uma na ordem do programa, sem um mutex, as ações de diferentes threads poderiam se intercalar de maneiras arbitrárias. Imagine tentar escrever um código que funcione mesmo que outros threads modifiquem os dados durante a execução. Imagine tentar depurá-los. Seria como se seu programa fosse assombrado.
- Mutexes suportam programação com *invariantes*, regras sobre os dados protegidos que são verdadeiros por construção quando você os configura e mantém por cada seção crítica.

Naturalmente, todos esses são realmente os mesmos motivos: condições de corrida descontroladas tornam a programação intratável. Mutexes trazem alguma ordem ao caos (embora não

tanto quanto canais ou fork-join).

No entanto, na maioria das linguagens, os mutexes são muito fáceis de confundir. No C++, como na maioria das linguagens, os dados e o bloqueio (lock) são objetos distintos. Idealmente, os comentários explicam que cada thread deve adquirir o mutex antes de acessar os dados:

```
class FernEmpireApp {  
    ...  
  
private:  
    // Lista de jogadores esperando para entrar em um jogo. Protegido por `mutex`  
    vector<PlayerId> waitingList;  
  
    // Bloqueio para adquirir antes de ler ou escrever `antidumping`  
    Mutex mutex;  
    ...  
};
```

Mas, mesmo com comentários tão interessantes, o compilador não pode impor acesso seguro aqui. Quando um fragmento de código se nega a adquirir o mutex, obtemos um comportamento indefinido. Na prática, isso significa bugs extremamente difíceis de reproduzir e corrigir.

Mesmo no Java, em que há alguma associação teórica entre objetos e mutexes, o relacionamento não é muito profundo. O compilador não faz nenhuma tentativa de aplicá-lo e, na prática, os dados protegidos por um bloqueio raramente são exatamente os campos do objeto associado. Em geral, inclui dados em vários objetos. Esquemas de bloqueio ainda são complicados. Os comentários ainda são a principal ferramenta para aplicá-los.

Mutex<T>

Agora vamos mostrar uma implementação da lista de espera no Rust. Em nosso servidor do jogo Fern Empire, cada jogador tem um ID único:

```
type PlayerId = u32;
```

A lista de espera é apenas uma coleção de jogadores:

```
const GAME_SIZE: usize = 8;
```

```
/// Uma lista de espera nunca aumenta para mais de GAME_SIZE jogadores
type WaitingList = Vec<PlayerId>;
```

A lista de espera é armazenada como um campo do `FernEmpireApp`, um singleton configurado em um `Arc` durante a inicialização do servidor. Cada thread tem um `Arc` apontando para ele. Contém toda a configuração compartilhada e outros fragmentos de que nosso programa precisa. A maior parte é somente leitura. Como a lista de espera é compartilhada e mutável, ela deve ser protegida por um `Mutex`:

```
use std::sync::Mutex;

/// Todos os threads têm acesso compartilhado a esse grande struct de contexto
struct FernEmpireApp {
    ...
    waiting_list: Mutex<WaitingList>,
    ...
}
```

Ao contrário do C++, no Rust os dados protegidos são armazenados dentro do `Mutex`. A configuração do `Mutex` se parece com isto:

```
use std::sync::Arc;

let app = Arc::new(FernEmpireApp {
    ...
    waiting_list: Mutex::new(vec![]),
    ...
});
```

Criar um novo `Mutex` é quase como criar um novo `Box` ou `Arc`, mas, enquanto `Box` e `Arc` significam alocação de heap, `Mutex` é apenas sobre bloqueio. Se você quer que o `Mutex` seja alocado no heap, tem de informar isso, como fizemos aqui utilizando `Arc::new` para todo o aplicativo e `Mutex::new` apenas para os dados protegidos. Esses tipos são comumente utilizados juntos: `Arc` é útil para compartilhar coisas entre threads e `Mutex` é útil para dados mutáveis compartilhados entre threads.

Agora podemos implementar o método `join_waiting_list` que utiliza o `mutex`:

```
impl FernEmpireApp {
    /// Adiciona um jogador à lista de espera para o próximo jogo
    /// Começa um novo jogo imediatamente se houver jogadores suficientes esperando
    fn join_waiting_list(&self, player: PlayerId) {
```

```

        // Bloqueia o mutex e obtém acesso aos dados internos
        // O escopo de `guard` é uma seção crítica
        let mut guard = self.waiting_list.lock().unwrap();

        // Agora faz a lógica do jogo
        guard.push(player);
        if guard.len() == GAME_SIZE {
            let players = guard.split_off(0);
            self.start_game(players);
        }
    }
}

```

A única maneira de obter os dados é chamar o método `.lock()`:

```
let mut guard = self.waiting_list.lock().unwrap();
```

`self.waiting_list.lock()` é bloqueado até que o mutex possa ser obtido. O valor `MutexGuard<WaitingList>` retornado por essa chamada de método é um encapsulador simples em torno de um `&mut WaitingList`. Graças às coerções `deref`, discutidas na página 362, podemos chamar métodos `WaitingList` diretamente no guarda:

```
guard.push(player);
```

O `guard` até nos empresta referências diretas aos dados subjacentes. O sistema de tempo de vida do Rust garante que essas referências não sobrevivam ao próprio `guard`. Não há como acessar os dados em um `Mutex` sem manter o bloqueio.

Quando `guard` é dropado, o bloqueio é liberado. Normalmente isso acontece no final do bloco, mas você também pode dropá-lo manualmente:

```

if guard.len() == GAME_SIZE {
    let players = guard.split_off(0);
    drop(guard); // não mantém a lista bloqueada ao iniciar um jogo
    self.start_game(players);
}

```

mut e Mutex

Pode parecer estranho – certamente nos pareceu estranho no começo – que nosso método `join_waiting_list` não recebe `self` por referência `mut`. Sua assinatura de tipo é:

```
fn join_waiting_list(&self, player: PlayerId)
```

A coleção subjacente, `Vec<PlayerId>`, requer uma referência `mut` quando

you call the `push` method. Its signature is:

```
pub fn push(&mut self, item: T)
```

E, porém, esse código compila e funciona bem. O que está acontecendo aqui?

In Rust, `&mut` means exclusive access. A simple `&` means shared access.

We're used to types passing access `&mut` from parent to child, from container to content. You just expect to call methods `&mut self` on `starships[id].engine` if you have a reference `&mut` to `starships` to begin with (or you own `starships`, in which case congratulations on being Elon Musk). That's the pattern, because, if you don't have exclusive access to the parent, Rust generally won't let you guarantee that you have exclusive access to the child.

But `Mutex` has a trick: locking. In reality, a mutex is just a way to do exactly this, provide exclusive (`mut`) access to internal data, even when many threads may have shared access (`non-mut`) to the `Mutex` itself.

The Rust type system tells you what `Mutex` does. It enforces dynamically exclusive access, something that's usually done statically, at compile time, by the Rust compiler.

(You should remember that `std::cell::RefCell` does the same thing, except without supporting multiple threads. `Mutex` and `RefCell` are both versions of internal mutability, which we cover on page 264.)

Por que mutexes nem sempre são uma boa ideia

Before we get into mutexes, let's present some approaches to concurrency that might seem strangely easy to use correctly if you're coming from C++. It's not coincidence: these approaches are designed to provide strong guarantees against the more confusing aspects of simultaneous programming. Programs that use exclusively fork-join parallelism are deterministic and can't deadlock.

Programas que utilizam canais quase sempre têm um bom comportamento. Aqueles que utilizam canais exclusivamente para pipelining, como nosso construtor de índice, são determinísticos: o tempo de entrega da mensagem pode variar, mas não afetará a saída. E assim por diante. As garantias sobre programas multithread são boas!

O design do `Mutex` do Rust quase certamente fará com que você utilize mutexes de forma mais sistemática e sensata do que nunca. Mas vale a pena parar e pensar sobre o que as garantias de segurança do Rust podem ou não ajudar.

O código de segurança do Rust não pode desencadear uma corrida de dados (data race), um tipo específico de bug em que vários threads leem e escrevem na mesma memória simultaneamente, produzindo resultados sem sentido. Isso é ótimo: corridas de dados são sempre bugs e não são raras em programas multithread reais.

Entretanto, threads que utilizam mutexes estão sujeitos a alguns outros problemas que o Rust não corrige para você:

- Os programas Rust válidos não podem ter corridas de dados, mas ainda podem ter outras *condições da corrida* – situações em que o comportamento de um programa depende do tempo entre os threads e pode, portanto, variar entre uma execução e outra. Algumas condições de corrida são benignas. Algumas se manifestam como falhas gerais e bugs incrivelmente difíceis de corrigir. O uso de mutexes de maneira não estruturada convida a condições de corrida. Cabe a você garantir que sejam benignas.
- O estado mutável compartilhado também afeta o design do programa. Onde os canais servem como um limite de abstração em seu código, facilitando a separação de componentes isolados para teste, os mutexes incentivam uma maneira de trabalhar do tipo “apenas adicione um método” que pode levar a uma bolha monolítica de código inter-relacionado.
- Por fim, os mutexes não são tão simples quanto parecem à primeira vista, como as próximas duas seções mostrarão.

Todos esses problemas são inerentes às ferramentas. Use uma

abordagem mais estruturada quando possível; utilize um `Mutex` quando é imperativo.

Deadlock

Um thread pode passar por um deadlock (impasse) ao tentar adquirir um bloqueio que já está mantendo:

```
let mut guard1 = self.waiting_list.lock().unwrap();  
let mut guard2 = self.waiting_list.lock().unwrap(); // deadlock
```

Suponha que a primeira chamada para `self.waiting_list.lock()` seja bem-sucedida, conseguindo o bloqueio. A segunda chamada vê que o bloqueio é mantido, então ela bloqueia, esperando que seja liberado. Ela vai esperar para sempre. O thread em espera é o mesmo que mantém o bloqueio.

Em outras palavras, o bloqueio em um `Mutex` não é um bloqueio recursivo.

Aqui o bug é óbvio. Em um programa real, as duas chamadas `lock()` podem estar em dois métodos diferentes, um dos quais chama o outro. O código para cada método, considerado separadamente, pareceria bem. Também existem outras maneiras de passar por um deadlock, envolvendo vários threads que adquirem vários mutexes de uma só vez. O sistema de empréstimo do Rust não pode protegê-lo contra um deadlock. A melhor proteção é manter as seções críticas pequenas: entre, faça seu trabalho e saia.

Também é possível passar por um deadlock com os canais. Por exemplo, dois threads podem bloquear, cada um esperando para receber uma mensagem do outro. Contudo, novamente, um bom projeto de programa pode lhe dar muita confiança de que isso não acontecerá na prática. Em um pipeline, como nosso construtor de índice invertido, o fluxo de dados é acíclico. O deadlock é tão improvável em um programa como em um pipeline de shell do Unix.

Mutexes envenenados

`Mutex::lock()` retorna um `Result` pelo mesmo motivo que `JoinHandle::join()` retorna: falhar elegantemente se outro thread gerou um pânico.

Quando escrevemos `handle.join().unwrap()`, estamos dizendo ao Rust que propague o pânico de um thread para outro. A linguagem `mutex.lock().unwrap()` é similar.

Se um thread gerar um pânico enquanto mantém um `Mutex`, o Rust marca o `Mutex` como envenenado. Qualquer tentativa subsequente de `lock` no `Mutex` envenenado obterá um resultado de erro. Nossa chamada a `.unwrap()` diz ao Rust que gere um pânico se isso acontecer, propagando o pânico do outro thread para este.

É ruim ter um mutex envenenado? Veneno parece mortal, mas esse cenário não é necessariamente fatal. Como dissemos no Capítulo 7, o pânico é seguro. Um thread que gerou um pânico deixa o restante do programa em um estado seguro.

A razão pela qual mutexes são envenenados quando geram um pânico, então, não é por medo de comportamento indefinido. Em vez disso, a preocupação é que você provavelmente programou com invariantes. Como seu programa gerou em pânico e saiu de uma seção crítica sem terminar o que estava fazendo, talvez tendo atualizado alguns campos dos dados protegidos, mas não outros, é possível que as invariantes agora não funcionem. O Rust envenena o mutex para evitar que outros threads entrem involuntariamente nessa situação não funcional e a tornem pior. Você ainda pode bloquear um mutex envenenado e acessar os dados internos, com exclusão mútua totalmente aplicada; consulte na documentação `PoisonError::into_inner()`. Mas você não fará isso por acaso.

Canais multiconsumidores usando mutexes

Mencionamos anteriormente que os canais do Rust são produtores múltiplos, consumidor único. Ou, para ser mais concreto, um canal tem apenas um `Receiver`. Não podemos ter um pool de threads onde muitos threads utilizam um canal `mpsc` único como uma lista de trabalho compartilhada.

No entanto, existe uma solução muito simples, utilizando apenas partes da biblioteca padrão. Podemos adicionar um `Mutex` em torno

do Receiver e compartilhá-lo de qualquer maneira. Eis um módulo que faz isso:

```
pub mod shared_channel {
    use std::sync::{Arc, Mutex};
    use std::sync::mpsc::{channel, Sender, Receiver};

    /// Um envoltório thread-safe em torno de um `Receiver`
    #[derive(Clone)]
    pub struct SharedReceiver<T>(Arc<Mutex<Receiver<T>>>);

    impl<T> Iterator for SharedReceiver<T> {
        type Item = T;

        /// Obtenha o próximo item do receptor encapsulado.
        fn next(&mut self) -> Option<T> {
            let guard = self.0.lock().unwrap();
            guard.recv().ok()
        }
    }

    /// Cria um novo canal cujo receptor pode ser compartilhado entre threads.
    /// Isso retorna um remetente e um destinatário, assim como o `channel()`
    /// de stdlib e às vezes funciona como um substituto drop-in.
    pub fn shared_channel<T>() -> (Sender<T>, SharedReceiver<T>) {
        let (sender, receiver) = channel();
        (sender, SharedReceiver(Arc::new(Mutex::new(receiver))))
    }
}
```

Estamos utilizando um `Arc<Mutex<Receiver<T>>>`. Os genéricos realmente se acumularam. Isso acontece com mais frequência no Rust do que no C++. Pode parecer confuso, mas muitas vezes, como nesse caso, apenas ler os nomes pode ajudar a explicar o que está acontecendo, conforme mostrado na Figura 19.11.

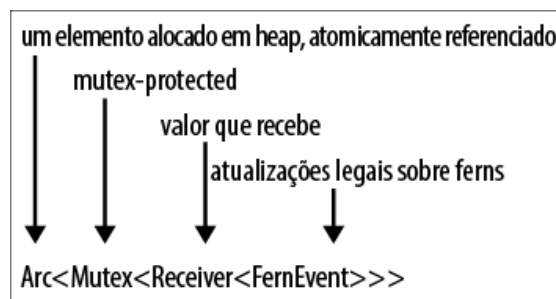


Figura 19.11: Como ler um tipo complexo.

Bloqueios de leitura/escrita (RwLock<T>)

Agora vamos passar dos mutexes para as outras ferramentas fornecidas em `std::sync`, o kit de ferramentas de sincronização de threads da biblioteca padrão do Rust. Passaremos rapidamente, pois uma discussão completa dessas ferramentas está além do escopo deste livro.

Programas de servidor geralmente têm informações de configuração que são carregadas uma vez e raramente são alteradas. A maioria dos threads apenas consulta a configuração, mas como a configuração pode mudar – pode ser possível solicitar que o servidor recarregue a configuração do disco, por exemplo – ele deve ser protegido por um bloqueio de qualquer maneira. Em casos como esse, um mutex pode funcionar, mas é um gargalo desnecessário. Threads não devem se revezar ao consultar a configuração se ela não mudar. Esse é um caso de bloqueio de leitura/escrita, ou `RwLock`.

Considerando que um mutex tem um único método `lock`, um bloqueio de leitura/escrita tem dois métodos de bloqueio, `read` e `write`. O método `RwLock::write` é como `Mutex::lock`. Espera acesso mut exclusivo aos dados protegidos. O método `RwLock::read` fornece acesso não-mut, com a vantagem de que é menos provável que tenha de esperar, porque muitos threads podem ler com segurança de uma só vez. Com um mutex, a qualquer momento, os dados protegidos têm apenas um leitor ou escritor (ou nenhum). Com um bloqueio de leitura/escrita, ele pode ter um escritor ou muitos leitores, quase como as referências do Rust em geral.

`FernEmpireApp` pode ter um struct para configuração, protegida por um `RwLock`:

```
use std::sync::RwLock;

struct FernEmpireApp {
    ...
    config: RwLock<AppConfig>,
    ...
}
```

Os métodos que leem a configuração utilizariam `RwLock::read()`:

```
/// Verdadeiro se o código de fungo experimental deve ser utilizado
```

```
fn mushrooms_enabled(&self) -> bool {
    let config_guard = self.config.read().unwrap();
    config_guard.mushrooms_enabled
}
```

O método para recarregar a configuração utilizaria `RwLock::write()`:

```
fn reload_config(&self) -> io::Result<()> {
    let new_config = AppConfig::load()?;
    let mut config_guard = self.config.write().unwrap();
    *config_guard = new_config;
    Ok(())
}
```

O Rust, é claro, é especialmente adequado para impor as regras de segurança em dados `RwLock`. O conceito de um único escritor ou vários leitores é central para o sistema de empréstimo do Rust. `self.config.read()` retorna um guard (guarda) que fornece acesso não-mut (compartilhado) ao `AppConfig`; `self.config.write()` retorna um tipo diferente de guarda que fornece acesso (exclusivo) mut.

Variáveis de condição (Condvar)

Frequentemente, um thread precisa esperar até que uma determinada condição se torne verdadeira:

- Durante o desligamento do servidor, o thread principal pode precisar esperar até que todos os outros threads terminem de sair.
- Quando um thread de trabalho não tem nada para fazer, ele precisa esperar até que haja alguns dados para processar.
- Um thread implementando um protocolo de consenso distribuído pode precisar esperar até que um quorum de pares tenha respondido.

Às vezes, há uma API de bloqueio conveniente para a condição exata que queremos esperar, como `JoinHandle::join` para o exemplo de desligamento do servidor. Em outros casos, não há API de bloqueio integrada. Os programas podem utilizar variáveis de condição para construir suas próprias. No Rust, o tipo `std::sync::Condvar` implementa variáveis de condição. Um `Condvar` tem métodos `.wait()` e `.notify_all()`; `.wait()` bloqueia até que algum outro thread chame `.notify_all()`.

Há um pouco mais do que isso, já que uma variável de condição é

sempre sobre uma condição específica de verdadeiro ou falso em relação a alguns dados protegidos por um `Mutex` determinado. Esse `Mutex` e a `Condvar`, portanto, estão relacionados. Uma explicação completa é mais do que temos espaço aqui, mas, para o benefício dos programadores que utilizaram variáveis de condição antes, mostraremos os dois conceitos-chave do código.

Quando a condição desejada se torna verdadeira, chamamos `Condvar::notify_all` (ou `notify_one`) para ativar qualquer thread em espera:

```
self.has_data_condvar.notify_all();
```

Para colocar em repouso e esperar que uma condição se torne verdadeira, utilizamos `Condvar::wait()`:

```
while !guard.has_data() {  
    guard = self.has_data_condvar.wait(guard).unwrap();  
}
```

Esse loop `while` é uma linguagem padrão para variáveis de condição. No entanto, a assinatura de `Condvar::wait` é incomum. Recebe um objeto `MutexGuard` por valor, o consome e retorna um novo `MutexGuard` se bem-sucedido. Isso capta a intuição de que o método `wait` libera o mutex e o `readquire` antes de retornar. Passar o `MutexGuard` por valor é uma forma de dizer, “concedo a você, método `.wait()`, minha autoridade exclusiva para liberar o mutex”.

Atômicos

O módulo `std::sync::atomic` contém tipos atômicos para programação concorrente sem bloqueio. Esses tipos são basicamente iguais aos atômicos do Standard C++, com alguns extras:

- `AtomicIsize` e `AtomicUsize` são tipos inteiros compartilhados correspondentes aos tipos `isize` e `usize` de thread único.
- `AtomicI8`, `AtomicI16`, `AtomicI32`, `AtomicI64` e suas variantes sem sinal como `AtomicU8` são tipos inteiros compartilhados que correspondem aos tipos de thread único `i8`, `i16` etc.
- Um `AtomicBool` é um valor `bool` compartilhado.
- Um `AtomicPtr<T>` é um valor compartilhado do tipo de ponteiro inseguro `*mut T`.

O uso adequado de dados atômicos está além do escopo deste livro. Basta dizer que múltiplos threads podem ler e escrever um valor atômico de uma só vez sem causar corridas de dados (data races).

Em vez dos operadores aritméticos e lógicos usuais, os tipos atômicos expõem métodos que executam operações atômicas, cargas individuais, armazenamentos, trocas e operações aritméticas que acontecem com segurança, como uma unidade, mesmo que outros threads também estejam executando operações atômicas que mexem na mesma posição na memória. Incrementar um `AtomicIsize` nomeado `atom` se parece com isto:

```
use std::sync::atomic::{AtomicIsize, Ordering};
```

```
let atom = AtomicIsize::new(0);  
atom.fetch_add(1, Ordering::SeqCst);
```

Esses métodos podem compilar para instruções de linguagem de máquina especializadas. Na arquitetura x86-64, essa chamada `.fetch_add()` compila para uma instrução `lock incq`, em que um `n += 1` ordinário pode compilar para uma instrução `incq` simples ou qualquer número de variações sobre esse tema. O compilador do Rust também precisa abrir mão de algumas otimizações em torno da operação atômica, pois – ao contrário de uma carga ou armazenamento normal – isso pode afetar legitimamente ou ser afetado por outros threads imediatamente.

O argumento `Ordering::SeqCst` é uma ordenação de memória. Ordenações de memória são algo como níveis de isolamento de transação em um banco de dados. Elas dizem ao sistema o quanto você se importa com noções filosóficas como causas anteriores a efeitos e tempo sem loops, em oposição ao desempenho. Ordenações de memória são cruciais para a exatidão do programa e são difíceis de entender e ponderar. Felizmente, a penalidade de desempenho ao escolher a consistência sequencial, a ordem de memória mais estrita, costuma ser bastante baixa – ao contrário da penalidade ao colocar um banco de dados SQL no modo `SERIALIZABLE`. Então, na dúvida, utilize `Ordering::SeqCst`. O Rust herda várias outras ordenações de memória da Standard C++ atômica, com várias garantias mais fracas sobre a natureza da existência e causalidade.

Não vamos discuti-las aqui.

Um uso simples de atômicos é para cancelamento. Suponha que temos um thread que está fazendo algum cálculo de longa duração, como renderizar um vídeo, e queremos cancelá-lo de forma assíncrona. O problema é comunicar ao thread que queremos que ele seja encerrado. Podemos fazer isso por meio de um `AtomicBool`:

```
use std::sync::Arc;
use std::sync::atomic::AtomicBool;

let cancel_flag = Arc::new(AtomicBool::new(false));
let worker_cancel_flag = cancel_flag.clone();
```

Esse código cria dois ponteiros inteligentes `Arc<AtomicBool>` que apontam para o mesmo `AtomicBool` alocado no heap, cujo valor inicial é `false`. O primeiro, nomeado `cancel_flag`, permanecerá no thread principal. O segundo, `worker_cancel_flag`, será movido para o thread de trabalho.

Eis o código para o thread de trabalho:

```
use std::thread;
use std::sync::atomic::Ordering;

let worker_handle = thread::spawn(move || {
    for pixel in animation.pixels_mut() {
        render(pixel); // traçado de raios - isso leva alguns microssegundos
        if worker_cancel_flag.load(Ordering::SeqCst) {
            return None;
        }
    }
    Some(animation)
});
```

Depois de renderizar cada pixel, o thread verifica o valor do sinalizador chamando o método `.load()`:

```
worker_cancel_flag.load(Ordering::SeqCst)
```

Se no thread principal decidirmos cancelar o thread de trabalho, armazenamos `true` em `AtomicBool` e esperamos o thread ser encerrado:

```
// Cancela a renderização
cancel_flag.store(true, Ordering::SeqCst);

// Descarta o resultado, que provavelmente é `None`
worker_handle.join().unwrap();
```

Naturalmente, existem outras maneiras de implementar isso. O

`AtomicBool` aqui poderia ser substituído por um `Mutex<bool>` ou um canal. A principal diferença é que atômicos têm sobrecarga mínima. Operações atômicas nunca utilizam chamadas de sistema. Uma carga ou armazenamento geralmente compila para uma única instrução da CPU.

Atômicos são uma forma de mutabilidade interna, como `Mutex` ou `RwLock`, então seus métodos recebem `self` por referência (não-mut) compartilhada. Isso as torna úteis como variáveis globais simples.

Variáveis globais

Suponha que estamos escrevendo código de rede. Queremos usar uma variável global, um contador que incrementamos toda vez que servimos um pacote:

```
/// Número de pacotes que o servidor tratou com sucesso
static PACKETS_SERVED: usize = 0;
```

Isso compila bem. Há apenas um problema. `PACKETS_SERVED` não é mutável, então nunca podemos alterá-lo.

O Rust faz tudo o que pode razoavelmente para desencorajar o estado mutável global. Constantes declaradas com `const` são, obviamente, imutáveis. Variáveis estáticas também são imutáveis por padrão, assim não há como obter uma referência `mut` a uma. Uma `static` pode ser declarada `mut`, mas acessá-la não é seguro. A insistência do Rust quanto à segurança de threads é uma das principais razões de todas essas regras.

O estado mutável global também tem consequências infelizes para engenharia de software: ele tende a tornar as várias partes de um programa mais estreitamente acopladas, mais difíceis de testar e mais difíceis de alterar posteriormente. Mesmo assim, em alguns casos simplesmente não há alternativa razoável, então é melhor descobrir uma maneira segura de declarar variáveis estáticas mutáveis.

A maneira mais simples de suportar o incremento de `PACKETS_SERVED`, mantendo-o thread-safe, é torná-lo um inteiro atômico:

```
use std::sync::atomic::AtomicUsize;
```

```
static PACKETS_SERVED: AtomicUsize = AtomicUsize::new(0);
```

Depois que essa variável estática é declarada, incrementar a contagem de pacotes é simples:

```
use std::sync::atomic::Ordering;
```

```
PACKETS_SERVED.fetch_add(1, Ordering::SeqCst);
```

Globais atômicos são limitadas a inteiros simples e booleanos. Contudo, criar uma variável global de qualquer outro tipo equivale a resolver dois problemas.

Primeiro, a variável deve ser thread-safe de alguma forma porque, do contrário, ela não pode ser global: por segurança, as variáveis estáticas devem ser tanto `Sync` como `!mut`. Felizmente, já vimos a solução para esse problema. O Rust tem tipos para compartilhar com segurança valores que mudam: `Mutex`, `RwLock` e os tipos atômicos. Esses tipos podem ser modificados mesmo quando declarados como `!mut`. É o que eles fazem. (Ver “mut e Mutex”, na página 597.)

Segundo, os inicializadores estáticos só podem chamar funções especificamente marcadas como `const`, que o compilador pode avaliar durante o tempo de compilação. Em outras palavras, sua saída é determinística; depende apenas de seus argumentos, não de qualquer outro estado ou E/S. Dessa forma, o compilador pode incorporar os resultados dessa computação como uma constante de tempo de compilação. Isso é semelhante a `constexpr` do C++.

Os construtores para os tipos `Atomic` (`AtomicUsize`, `AtomicBool` e assim por diante) são todos funções `const`, que permitiram criar um `static AtomicUsize` anteriormente. Alguns outros tipos, como `String`, `Ipv4Addr` e `Ipv6Addr`, têm construtores simples que também são `const`.

Você também pode definir suas próprias funções `const` simplesmente prefixando a assinatura da função com `const`. O Rust limita o que funções `const` podem fazer a um pequeno conjunto de operações, que são suficientes para serem úteis e, mesmo assim, não permitir resultados não determinísticos. Funções `const` não podem receber tipos como argumentos genéricos, apenas tempos de vida, e não é possível alocar memória ou operar em ponteiros brutos, mesmo em blocos `unsafe`. Podemos, porém, utilizar operações aritméticas

(incluindo aritmética de encapsulamento e saturação), operações lógicas que não causam curto-circuito e outras funções `const`. Por exemplo, podemos criar funções convenientes para tornar a definição de `static` e `const` mais fácil e reduzir a duplicação de código:

```
const fn mono_to_rgba(level: u8) -> Color {
    Color {
        red: level,
        green: level,
        blue: level,
        alpha: 0xFF
    }
}
const WHITE: Color = mono_to_rgba(255);
const BLACK: Color = mono_to_rgba(000);
```

Combinando essas técnicas, podemos ser tentados a escrever:

```
static HOSTNAME: Mutex<String> =
    Mutex::new(String::new()); // erro: chamadas em variáveis estáticas são
                                // limitadas a funções constantes, structs de
                                // tupla e variantes de tupla
```

Infelizmente, embora `AtomicUsize::new()` e `String::new()` sejam `const fn`, `Mutex::new()` não é. Para contornar essas limitações, precisamos utilizar o crate `lazy_static`.

Apresentamos o crate `lazy_static` em “Construindo valores regex preguiçosamente (lazily)”, na página 530. Definir uma variável com a macro `lazy_static!` permite utilizar qualquer expressão que você quiser para inicializá-la; ela é executada na primeira vez em que a variável é desreferenciada e o valor é salvo para todos os usos subsequentes.

Podemos declarar um `HashMap` global controlado por `Mutex` com `lazy_static` desta maneira:

```
use lazy_static::lazy_static;
use std::sync::Mutex;

lazy_static! {
    static ref HOSTNAME: Mutex<String> = Mutex::new(String::new());
}
```

A mesma técnica funciona para outras estruturas de dados complexas, como `HashMap` e `Deque`s. Também é bastante útil para variáveis estáticas que não são mutáveis, mas simplesmente

requerem inicialização não trivial.

Utilizar `lazy_static!` impõe um pequeno custo de desempenho em cada acesso aos dados estáticos. A implementação utiliza `std::sync::Once`, uma primitiva de sincronização de baixo nível projetada para inicialização única. Nos bastidores, sempre que uma variável estática preguiçosa é acessada, o programa executa uma instrução de carregamento atômico para verificar se a inicialização já ocorreu. (`Once` é de propósito bastante especial, então não vamos abordá-la em detalhes aqui. Geralmente, em vez disso, é mais conveniente utilizar `lazy_static!`. Contudo, é útil para inicializar bibliotecas não Rust; para um exemplo, ver “Uma interface segura para libgit2”, na página 764.)

Como é hackear código concorrente em Rust

Mostramos três técnicas para utilizar threads no Rust: paralelismo fork-join, canais e estado mutável compartilhado com bloqueios (locks). Nosso objetivo foi fornecer uma boa introdução às peças fornecidas pelo Rust, com foco na maneira como elas podem se encaixar em programas reais.

O Rust insiste na segurança, assim, a partir do momento que você decide escrever um programa multithread, o foco é a construção de uma comunicação segura e estruturada. Manter threads isolados é uma boa maneira de convencer o Rust de que o que você está fazendo é seguro. Acontece que o isolamento também é uma boa maneira de garantir que o que você está fazendo está correto e é sustentável. Mais uma vez, o Rust o orienta na direção de bons programas.

Mais importante, o Rust permite combinar técnicas e experimentar. Você pode iterar rapidamente: argumentar com o compilador faz com que você comece a trabalhar corretamente muito mais rápido do que depurar corridas de dados (data races).

CAPÍTULO 20

Programação assíncrona

Suponha que você esteja escrevendo um servidor de bate-papo. Para cada conexão de rede, há pacotes de entrada para analisar, pacotes de saída para montar, parâmetros de segurança para gerenciar, assinaturas de grupos de bate-papo para rastrear e assim por diante. Gerenciar tudo isso para muitas conexões simultaneamente exigirá alguma organização.

Idealmente, você pode apenas iniciar um thread separado para cada conexão de entrada:

```
use std::{net, thread};

let listener = net::TcpListener::bind(address)?;

for socket_result in listener.incoming() {
    let socket = socket_result?;
    let groups = chat_group_table.clone();
    thread::spawn(|| {
        log_error(serve(socket, groups));
    });
}
```

Para cada nova conexão, isso gera um novo thread que executa a função `serve`, que é capaz de se concentrar em gerenciar as necessidades de uma única conexão.

Isso funciona bem, até que tudo saia muito melhor do que o planejado e de repente você tenha dezenas de milhares de usuários. Não é incomum que o stack (pilha) de um thread cresça para 100 KiB ou mais, e provavelmente não é assim que você deseja gastar gigabites de memória do servidor. Threads são bons e necessários para distribuir o trabalho entre vários processadores, mas a demanda por memória é tanta que geralmente precisamos de formas complementares, utilizadas em conjunto com threads, para dividir o trabalho.

Você pode utilizar tarefas assíncronas do Rust para intercalar muitas atividades independentes em um único thread ou em um pool de threads de trabalho. Tarefas assíncronas são semelhantes a threads, mas são muito mais rápidas de criar, passam o controle entre si com mais eficiência e têm sobrecarga de memória em uma ordem de magnitude menor que a de um thread. É perfeitamente possível ter centenas de milhares de tarefas assíncronas executando simultaneamente em um único programa. Obviamente, seu aplicativo ainda pode ser limitado por outros fatores, como largura de banda de rede, velocidade de banco de dados, computação ou requisitos de memória inerentes ao trabalho, mas a sobrecarga de memória inerente ao uso de tarefas é muito menos significativa do que a de threads.

Geralmente, o código Rust assíncrono se parece muito com o código multithread comum, exceto que as operações que podem bloquear, como E/S ou aquisição de mutexes, precisam ser tratadas de maneira um pouco diferente. Tratar isso de forma especial fornece ao Rust mais informações sobre como seu código se comportará, o que torna possível o desempenho aprimorado. A versão assíncrona do código anterior se parece com isto:

```
use async_std::{net, task};

let listener = net::TcpListener::bind(address).await?;

let mut new_connections = listener.incoming();
while let Some(socket_result) = new_connections.next().await {
    let socket = socket_result?;
    let groups = chat_group_table.clone();
    task::spawn(async {
        log_error(serve(socket, groups).await);
    });
}
```

Isso utiliza os módulos de rede e tarefa do crate `async_std` e adiciona `.await` após as chamadas que podem ser bloqueadas. Mas a estrutura geral é a mesma da versão baseada em thread.

O objetivo deste capítulo não é apenas ajudá-lo a escrever código assíncrono, mas também mostrar como ele funciona em detalhes suficientes para que você possa prever como ele será executado em

seus aplicativos e ver onde pode ser mais valioso.

- Para mostrar a mecânica da programação assíncrona, apresentamos um conjunto mínimo de recursos de linguagem que abrange todos os conceitos básicos: futuros, funções assíncronas, expressões `await`, tarefas e executores `block_on` e `spawn_local`.
- Em seguida, apresentamos os blocos assíncronos e o executor `spawn`. São essenciais para realizar um trabalho real, mas, conceitualmente, são apenas variantes dos recursos que acabamos de mencionar. No processo, apontamos alguns problemas que você provavelmente encontrará que são exclusivos da programação assíncrona e explicamos como lidar com eles.
- Para mostrar todas essas partes funcionando em conjunto, analisamos o código completo de um servidor e cliente de bate-papo, do qual o fragmento de código anterior faz parte.
- Para ilustrar como funcionam as primitivas de futuros e executores, apresentamos implementações simples, mas funcionais, de `spawn_blocking` e `block_on`.
- Por fim, explicamos o tipo `Pin`, que aparece de tempos em tempos em interfaces assíncronas para garantir que a função assíncrona e os futuros de bloco sejam utilizados com segurança.

De síncrono a assíncrono

Considere o que acontece quando você chama a seguinte função (não assíncrona, completamente tradicional):

```
use std::io::prelude::*;
use std::net;

fn cheapo_request(host: &str, port: u16, path: &str)
    -> std::io::Result<String>
{
    let mut socket = net::TcpStream::connect((host, port))?;

    let request = format!("GET {} HTTP/1.1\r\nHost: {}\r\n\r\n", path, host);
    socket.write_all(request.as_bytes());
    socket.shutdown(net::Shutdown::Write)?;

    let mut response = String::new();
    socket.read_to_string(&mut response)?;
```

```

    Ok(response)
}

```

Isso abre uma conexão TCP com um servidor da web, envia uma solicitação HTTP básica em um protocolo desatualizado¹. A Figura 20.1 mostra a execução dessa função ao longo do tempo.

Esse diagrama mostra como o call stack (pilha de chamada) de função se comporta à medida que o tempo passa da esquerda para a direita. Cada chamada de função é um box, colocado sobre o chamador. Obviamente, a função `cheapo_request` roda durante toda a execução. Ela chama funções da biblioteca padrão do Rust como `TcpStream::connect` e `TcpStream` de `write_all` e `read_to_string`. Essas, por sua vez, chamam outras funções, mas, com o tempo, o programa faz chamadas de sistema, solicitações ao sistema operacional para realmente fazer algo, como abrir uma conexão TCP ou ler ou escrever alguns dados.

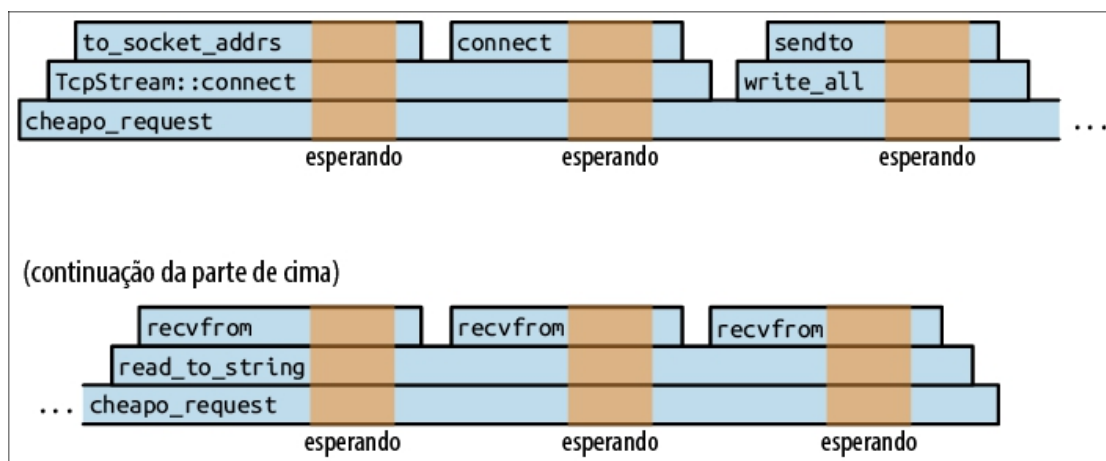


Figura 20.1: Progresso de uma solicitação HTTP síncrona (áreas cinza mais escuras estão esperando o sistema operacional).

Os fundos cinza mais escuros marcam os momentos em que o programa espera que o sistema operacional conclua a chamada do sistema. Não desenhamos esses tempos em termos de escala. Se desenhássemos, todo o diagrama seria cinza mais escuro: na prática, essa função passa quase todo o tempo esperando o sistema operacional. A execução do código anterior seriam fragmentos estreitos entre as chamadas do sistema.

Enquanto essa função espera pelo retorno das chamadas do sistema, seu único thread é bloqueado: ela não pode fazer mais nada até que a chamada do sistema termine. Não é incomum que o stack de um thread tenha um tamanho de dezenas ou centenas de kilobytes; assim, se isso fosse um fragmento de algum sistema maior, com muitos threads trabalhando em tarefas semelhantes, bloquear os recursos desses threads para não fazer nada além de esperar pode se tornar muito caro.

Para contornar isso, um thread precisa ser capaz de assumir outro trabalho enquanto espera a conclusão das chamadas do sistema. Mas não é óbvio como alcançar isso. Por exemplo, a assinatura da função que estamos utilizando para ler a resposta do soquete é:

```
fn read_to_string(&mut self, buf: &mut String) -> std::io::Result<usize>;
```

É escrito no tipo: essa função não retorna até que o trabalho seja concluído ou algo dê errado. Essa função é síncrona: o chamador retoma quando a operação é concluída. Se quisermos utilizar nosso thread para outras coisas enquanto o sistema operacional faz o trabalho, precisaremos de uma nova biblioteca de E/S que forneça uma versão assíncrona dessa função.

Futuros

A abordagem do Rust para suportar operações assíncronas é introduzir um trait, `std::future::Future`:

```
trait Future {  
    type Output;  
    // Por enquanto, leia `Pin<&mut Self>` como `&mut Self`  
    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;  
}  
  
enum Poll<T> {  
    Ready(T),  
    Pending,  
}
```

Um `Future` representa uma operação que você pode testar se foi concluída. O método `poll` de um futuro nunca espera que a operação termine: sempre retorna imediatamente. Se a operação estiver concluída, `poll` retorna `Poll::Ready(output)`, em que `output` é o resultado

final. Do contrário, retorna `Pending`. Se e quando vale a pena verificar o futuro novamente, ele promete nos informar invocando um waker, uma função callback fornecida no `Context`. Chamamos isso de “modelo piñata” da programação assíncrona: a única coisa que você pode fazer com um futuro é listá-lo em um `poll` até que um valor se diferencie.

Todos os sistemas operacionais modernos incluem variantes das chamadas de sistema que podemos utilizar para implementar esse tipo de interface de verificação (polling). No Unix e Windows, por exemplo, se você colocar um soquete de rede no modo sem bloqueio, as leituras e escritas retornarão um erro se bloquearem; você tem de tentar novamente mais tarde.

Portanto, uma versão assíncrona de `read_to_string` teria uma assinatura mais ou menos assim:

```
fn read_to_string(&mut self, buf: &mut String)
-> impl Future<Output = Result<usize>>;
```

É a mesma assinatura que mostramos anteriormente, exceto pelo tipo de retorno: a versão assíncrona retorna um futuro de um `Result<usize>`. Você precisará verificar esse futuro até obter um `Ready(result)` a partir dele. Cada vez que é verificada, a leitura prossegue até onde conseguir. O `result` final fornece o valor de sucesso ou um valor de erro, assim como uma operação de E/S comum. Este é o padrão geral: a versão assíncrona de qualquer função recebe os mesmos argumentos da versão síncrona, mas o tipo de retorno tem um `Future` em torno dela.

Na verdade, chamar essa versão `read_to_string` não lê nada; a única responsabilidade é construir e retornar um futuro que fará o trabalho real quando verificado. Esse futuro deve conter todas as informações necessárias para realizar a solicitação feita pela chamada. Por exemplo, o futuro retornado por `read_to_string` deve lembrar o fluxo de entrada em que foi chamado, e a `String` à qual deve anexar os dados recebidos. De fato, como o futuro contém as referências `self` e `buf`, a assinatura adequada para `read_to_string` deve ser:

```
fn read_to_string<'a>(&'a mut self, buf: &'a mut String)
-> impl Future<Output = Result<usize>> + 'a;
```

Isso adiciona tempos de vida para indicar que o futuro retornado só pode existir enquanto os valores que `self` e `buf` pegam emprestados também existirem.

O crate `async-std` fornece versões assíncronas de todos os recursos de E/S de `std`, incluindo um trait `Read` assíncrono com um método `read_to_string`. `async-std` segue de perto o design de `std`, reutilizando os tipos de `std` em suas próprias interfaces sempre que possível, portanto erros, resultados, endereços de rede e a maioria dos outros dados associados são compatíveis entre os dois mundos. Familiaridade com `std` ajuda a utilizar `async-std` e vice-versa.

Uma das regras do trait `Future` é que, uma vez que um futuro retornou `Poll::Ready`, ele pode presumir que nunca mais será verificado. Alguns futuros apenas retornam `Poll::Pending` para sempre se forem verificados excessivamente; outros podem gerar um pânico ou travar. (Eles não devem, porém, violar a memória ou a segurança do thread ou causar um comportamento indefinido.) O método adaptador `fuse` no trait `Future` transforma qualquer futuro em um que simplesmente retorna `Poll::Pending` continuamente. Mas todas as formas usuais de consumir futuros respeitam essa regra, então `fuse` geralmente não é necessário.

Se a verificação (polling) parece ineficiente, não se preocupe. A arquitetura assíncrona do Rust é cuidadosamente projetada para que desde que as funções básicas de E/S como `read_to_string` sejam implementadas corretamente, você só vai verificar um futuro quando valer a pena. Sempre que `poll` é chamado, algo em algum lugar deve retornar `Ready`, ou pelo menos progredir em direção a esse objetivo. Explicaremos como isso funciona em “Primitivas de futuro e executores: Quando vale a pena verificar um futuro novamente?”, na página 659.

Mas utilizar futuros parece um desafio: quando você faz uma verificação, o que deve fazer ao receber `Poll::Pending`? Por enquanto, terá de verificar algum outro trabalho que esse thread pode fazer, sem esquecer de voltar a esse futuro mais tarde e verificá-lo novamente. Todo o programa ficará cheio de tubulações que

monitoram quem está pendente e o que deve ser feito quando estiver pronto. A simplicidade da nossa função `cheapo_request` é arruinada.

Boas notícias! Não é.

Funções assíncronas e expressões `await`

Eis uma versão de `cheapo_request` escrita como uma função assíncrona:

```
use async_std::io::prelude::*;
use async_std::net;

async fn cheapo_request(host: &str, port: u16, path: &str)
    -> std::io::Result<String>
{
    let mut socket = net::TcpStream::connect((host, port)).await?;

    let request = format!("GET {} HTTP/1.1\r\nHost: {}\r\n\r\n", path, host);
    socket.write_all(request.as_bytes()).await?;
    socket.shutdown(net::Shutdown::Write)?;

    let mut response = String::new();
    socket.read_to_string(&mut response).await?;

    Ok(response)
}
```

Isso é token por token igual à nossa versão original, exceto:

- A função começa com `async fn` em vez de `fn`.
- Ela utiliza as versões assíncronas do `crate async_std` de `TcpStream::connect`, `write_all` e `read_to_string`. Todas retornam futuros de seus resultados. (Os exemplos nesta seção utilizam a versão 1.7 de `async_std`.)
- Após cada chamada que retorna um futuro, o código informa `.await`. Embora isso pareça uma referência a um campo struct chamado `await`, na verdade, é uma sintaxe especial incorporada à linguagem para aguardar até que um futuro esteja pronto. Uma expressão `await` é avaliada como o valor final do futuro. É assim que a função obtém os resultados de `connect`, `write_all` e `read_to_string`.

Ao contrário de uma função comum, quando você chama uma função assíncrona, ela retorna imediatamente, antes que o corpo comece a execução. Obviamente, o valor de retorno final da

chamada ainda não foi calculado; o que você obtém é um futuro do valor final. Assim, se você executar este código:

```
let response = cheapo_request(host, port, path);
```

então `response` será um futuro de um `std::io::Result<String>` e o corpo de `cheapo_request` ainda não iniciou a execução. Você não precisa ajustar o tipo de retorno de uma função assíncrona; o Rust trata automaticamente `async fn f(...) -> T` como uma função que retorna um futuro de um `T`, não um `T` diretamente.

O futuro retornado por uma função assíncrona agrupa todas as informações que o corpo da função precisará para executar: os argumentos da função, espaço para as variáveis locais e assim por diante. (É como se você tivesse capturado o stack frame da chamada como um valor Rust comum.) Assim, `response` deve conter os valores passados para `host`, `port` e `path`, uma vez que o corpo de `cheapo_request` vai precisar deles para executar.

O tipo específico do futuro é gerado automaticamente pelo compilador, com base no corpo e argumentos da função. Esse tipo não tem nome; tudo o que você sabe sobre ele é que implementa `Future<Output=R>`, em que `R` é o tipo de retorno da função assíncrona. Nesse sentido, futuros das funções assíncronas são como closures: closures também possuem tipos anônimos, gerados pelo compilador, que implementam os traits `FnOnce`, `Fn` e `FnMut`.

Quando você verifica pela primeira vez o futuro retornado por `cheapo_request`, a execução começa na parte superior do corpo da função e continua até o primeiro `await` do futuro retornado por `TcpStream::connect`. A expressão `await` verifica o futuro `connect` e, se não estiver pronto, retorna `Poll::Pending` para o próprio chamador: a verificação do futuro de `cheapo_request` não pode passar desse primeiro `await` até uma verificação do futuro de `TcpStream::connect` retornar `Poll::Ready`. Portanto, um equivalente aproximado da expressão `TcpStream::connect(...).await` pode ser:

```
{
    // Nota: isso é pseudocódigo, não Rust inválido
    let connect_future = TcpStream::connect(...);
    'retry_point:
    match connect_future.poll(cx) {
```

```

Poll::Ready(value) => value,
Poll::Pending => {
    // Organizar a próxima `verificação` de `cheapo_request`'s
    // future para retomar a execução em 'retry_point'
    ...
    return Poll::Pending;
}
}
}

```

Uma expressão `await` toma posse do futuro e então o verifica. Se estiver pronto, o valor final do futuro é o valor da expressão `await` e a execução continua. Caso contrário, retorna `Poll::Pending` ao próprio chamador.

Mas, crucialmente, a próxima verificação do futuro de `cheapo_request` não começa na parte superior da função novamente: em vez disso, ela retoma a execução no meio da função no ponto em que está prestes a verificar `connect_future`. Só avançamos para o restante da função assíncrona depois de o futuro estar pronto.

À medida que o futuro de `cheapo_request` continua a ser verificado, ele percorrerá o corpo da função de um `await` ao próximo, movendo-se apenas quando o subfuturo que está esperando estiver pronto. Assim, quantas vezes o futuro de `cheapo_request` deve ser verificado depende do comportamento dos subfuturos e do próprio fluxo de controle da função. O futuro de `cheapo_request` rastreia o ponto em que o próximo `poll` deve retomar e todo o estado local – variáveis, argumentos, temporários – de que a retomada precisará.

A capacidade de suspender a execução no meio da função e retomar mais tarde é exclusiva das funções assíncronas. Quando uma função comum retorna, o stack frame desaparece para sempre. Como as expressões `await` dependem da capacidade de retomar, você só pode usá-las dentro de funções assíncronas.

No momento em que este livro foi escrito, o Rust ainda não permitia que traits tivessem métodos assíncronos. Somente funções livres e funções inerentes a um tipo específico podem ser assíncronas. Levantar essa restrição exigirá uma série de alterações na linguagem. Enquanto isso, se você precisar definir traits que incluam funções assíncronas, considere utilizar o crate `async-trait`, que fornece

uma solução alternativa baseada em macro.

Chamando funções assíncronas a partir do código síncrono: `block_on`

De certa forma, as funções assíncronas apenas passam a bola. É verdade que é fácil obter o valor do futuro em uma função assíncrona: basta usar `await`. Mas a própria função assíncrona retorna um futuro, então agora é trabalho do chamador fazer de alguma forma a verificação. Em última análise, na verdade alguém tem de esperar por um valor.

Podemos chamar `cheapo_request` de uma função síncrona comum (como `main`) utilizando a função `task::block_on` de `async_std`, que recebe um futuro e o verifica até que produza um valor:

```
fn main() -> std::io::Result<()> {  
    use async_std::task;  
  
    let response = task::block_on(cheapo_request("example.com", 80, "/"));?  
    println!("{}", response);  
    Ok(())  
}
```

Como `block_on` é uma função síncrona que produz o valor final de uma função assíncrona, você pode considerá-la um adaptador entre o mundo assíncrono e o mundo síncrono. Mas seu caráter de bloqueio também significa que você nunca deve utilizar `block_on` dentro de uma função assíncrona: bloquearia todo o thread até o valor estar pronto. Em vez disso, utilize `await`.

A Figura 20.2 mostra uma possível execução de `main`.

A linha do tempo na parte superior, “Visualização simplificada”, mostra uma visualização abstrata das chamadas assíncronas do programa: `cheapo_request` primeiro chama `TcpStream::connect` para obter um soquete e, em seguida, chama `write_all` e `read_to_string` nesse soquete. Então retorna. Isso é muito semelhante à linha do tempo para a versão síncrona de `cheapo_request` anteriormente neste capítulo.

Mas cada uma dessas chamadas assíncronas é um processo de várias etapas: um futuro é criado e, em seguida, verificado até que

esteja pronto, talvez criando e verificando outros subfuturos no processo. A linha do tempo na parte inferior, “Implementação”, mostra as chamadas síncronas reais que implementam esse comportamento assíncrono. Essa é uma boa oportunidade para analisar exatamente o que está acontecendo na execução assíncrona comum:

- Primeiro, `main` chama `cheapo_request`, que retorna um futuro `A` do seu resultado final. Então `main` passa esse futuro para `async_std::block_on`, que o verifica.
- Verificar o futuro `A` permite que o corpo de `cheapo_request` inicie a execução. Isso chama `TcpStream::connect` para obter um futuro `B` de um soquete e, em seguida, aguarda isso. Mais precisamente, como `TcpStream::connect` pode encontrar um erro, `B` é um futuro de um `Result<TcpStream, std::io::Error>`.

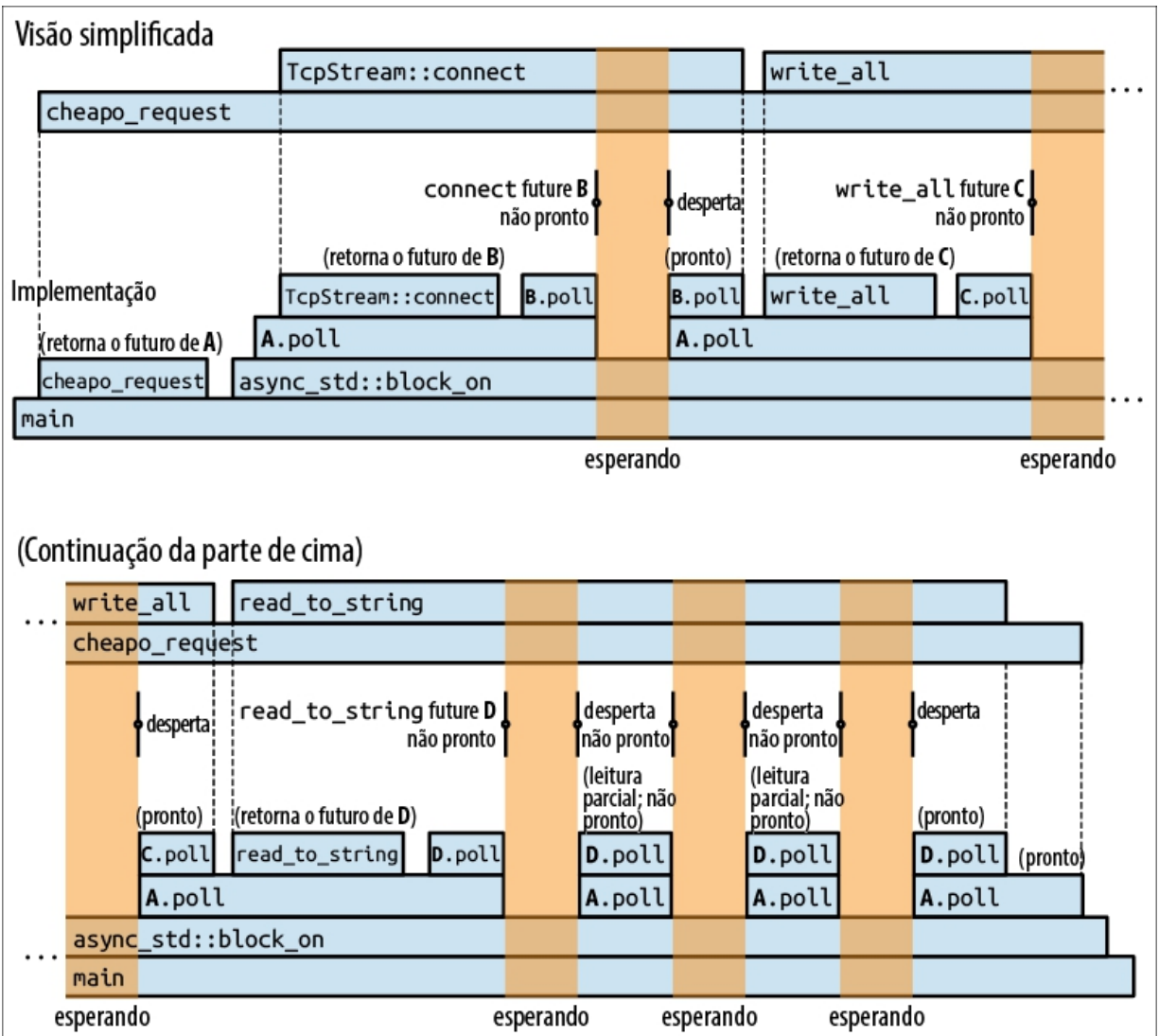


Figura 20.2: Bloqueio em uma função assíncrona.

- Futuro B é verificado por `await`. Como a conexão de rede ainda não foi estabelecida, `B.poll` retorna `Poll::Pending`, mas organiza para ativar a tarefa de chamada assim que o soquete estiver pronto.
- Como o futuro B não estava pronto, `A.poll` retorna `Poll::Pending` ao próprio chamador, `block_on`.
- Como `block_on` não tem nada melhor a fazer, entra em estado de repouso. Toda o thread é bloqueado agora.
- Quando a conexão de B está pronta para uso, ela ativa a tarefa que a verificou. Isso ativa `block_on` e tenta verificar o futuro A novamente.

- A verificação A faz com que `cheapo_request` retome na primeira `await`, em que verifica B novamente.
- Dessa vez, B está pronto: a criação do soquete está completa, então retorna `Poll::Ready(Ok(socket))` para `A.poll`.
- A chamada assíncrona para `TcpStream::connect` agora está completa. O valor da expressão `TcpStream::connect(...).await` é, portanto, `Ok(socket)`.
- A execução do corpo de `cheapo_request` continua normalmente, construindo a string de solicitação utilizando a macro `format!` e passando-a para `socket.write_all`.
- Como a `socket.write_all` é uma função assíncrona, ela retorna um futuro `c` do seu resultado, que `cheapo_request` devidamente aguarda.

O restante da história é semelhante. Na execução mostrada na Figura 20.2, o futuro de `socket.read_to_string` é verificado quatro vezes antes de estar pronto; cada uma dessas ativações lê alguns dados do soquete, mas `read_to_string` é especificado para ler até o final da entrada e isso requer várias operações.

Não parece muito difícil apenas escrever um loop que chama `poll` repetidamente. Mas o que torna `async_std::task::block_on` valioso é que sabe como entrar em repouso até que o futuro realmente valha a pena ser verificado novamente, em vez de desperdiçar o tempo do processador e a duração da bateria, gerando bilhões de chamadas `poll` infrutíferas. Os futuros retornados por funções básicas de E/S como `connect` e `read_to_string` retêm o waker (callback) fornecido pelo `Context` passado para `poll` e invoca-o quando `block_on` deve acordar e tentar verificar novamente. Mostraremos exatamente como isso funciona implementando uma versão simples de `block_on` em “Primitivas de futuros e executores: Quando vale a pena verificar um futuro novamente?”, na página 659.

Assim como a versão síncrona original que apresentamos anteriormente, essa versão assíncrona de `cheapo_request` passa quase todo o tempo esperando a conclusão das operações. Se o eixo do tempo fosse desenhado em escala, o diagrama seria quase inteiramente cinza-escuro, com pequenas partes de computação ocorrendo quando o programa é ativado.

Isso é uma grande quantidade de detalhes. Felizmente, em geral você pode pensar apenas em termos da linha de tempo simplificada na parte superior: algumas chamadas de função são síncronas, outras são assíncronas e precisam de uma `await`, mas todas são apenas chamadas de função. O sucesso do suporte assíncrono do Rust depende de ajudar os programadores a trabalhar com a visão simplificada na prática, sem se distrair com o vaivém da implementação.

Gerando tarefas assíncronas

A função `async_std::task::block_on` é bloqueada até que o valor de um futuro esteja pronto. Mas bloquear um thread completamente em um único futuro não é melhor do que uma chamada síncrona: o objetivo deste capítulo é fazer com que o thread realize outro trabalho enquanto espera.

Para isso, você pode utilizar `async_std::task::spawn_local`. Essa função recebe um futuro e o adiciona a um pool que `block_on` tentará verificar sempre que o futuro em que está bloqueando não estiver pronto. Assim, se você passar um monte de futuros para `spawn_local` e depois aplicar `block_on` a um futuro do resultado final, `block_on` vai verificar cada futuro gerado sempre que for capaz de avançar, executando todo o pool simultaneamente até que o resultado esteja pronto.

No momento em que este livro era escrito, `spawn_local` estava disponível em `async-std` somente se você habilitasse esse recurso `unstable` do crate. Para fazer isso, você precisará modificar `async-std` em *Cargo.toml* com uma linha como esta:

```
async-std = { version = "1", features = ["unstable"] }
```

A função `spawn_local` é um análogo assíncrono da função `std::thread::spawn` da biblioteca padrão para iniciar threads:

- `std::thread::spawn(c)` recebe uma closure `c` e inicia um thread executando-o, retornando uma `std::thread::JoinHandle` cujo `join` espera o thread terminar e retorna o que quer que `c` retornou.
- `async_std::task::spawn_local(f)` recebe o futuro `f` e o adiciona ao pool para ser verificado quando o thread atual chama `block_on`. `spawn_local`

retorna seu próprio tipo `async_std::task::JoinHandle`, ele mesmo um futuro que você pode esperar para recuperar o valor final de `f`.

Por exemplo, suponha que queremos criar todo um conjunto de solicitações HTTP simultaneamente. Eis uma primeira tentativa:

```
pub async fn many_requests(requests: Vec<(String, u16, String)>)
    -> Vec<std::io::Result<String>>
{
    use async_std::task;

    let mut handles = vec![];
    for (host, port, path) in requests {
        handles.push(task::spawn_local(cheapo_request(&host, port, &path)));
    }

    let mut results = vec![];
    for handle in handles {
        results.push(handle.await);
    }

    results
}
```

Essa função chama `cheapo_request` em cada elemento de `requests`, passando o futuro de cada chamada para `spawn_local`. Ela coleta o `JoinHandle` resultante em um vetor e então aguarda cada um deles. Sem problemas aguardar os manipuladores de junção em qualquer ordem: como as solicitações já foram geradas, seus futuros serão verificados conforme necessário sempre que esse thread chamar `block_on` e não tem nada melhor para fazer. Todas as solicitações serão executadas simultaneamente. Assim que estiverem completas, `many_requests` retorna os resultados para seu chamador.

O código anterior está quase correto, mas o verificador de empréstimos do Rust está preocupado com o tempo de vida do futuro de `cheapo_request`:

error: `host` does not live long enough

```
handles.push(task::spawn_local(cheapo_request(&host, port, &path)));
                                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
                                |
                                | borrowed value does not
                                | live long enough
                                |
                                argument requires that `host` is borrowed for `static`
}
```

- `host` dropped here while still borrowed

Há um erro semelhante para `path` também.

Naturalmente, se passarmos referências para uma função assíncrona, o futuro que ela retorna deverá conter essas referências, portanto o futuro não pode sobreviver com segurança aos valores que eles pedem emprestados. Essa é a mesma restrição que se aplica a qualquer valor que contém referências.

O problema é que `spawn_local` não pode ter certeza de que você vai esperar a tarefa terminar antes de `host` e `path` serem dropados. Na verdade, `spawn_local` só aceita futuros cujos tempos de vida são `'static`, porque você pode simplesmente ignorar o `JoinHandle` que ele retorna e permitir que a tarefa continue pelo restante da execução do programa. Isso não é exclusivo de tarefas assíncronas: você receberá um erro semelhante se tentar utilizar `std::thread::spawn` para iniciar um thread cujo closure captura referências a variáveis locais.

Uma maneira de corrigir isso é criar outra função assíncrona que utiliza versões próprias dos argumentos:

```
async fn cheapo_owning_request(host: String, port: u16, path: String)
    -> std::io::Result<String> {
    cheapo_request(&host, port, &path).await
}
```

Essa função recebe `Strings` em vez de referências `&str`, então seu futuro possui as strings `host` e `path`, e seu tempo de vida é `'static`. O verificador de empréstimo pode ver que ela espera imediatamente o futuro de `cheapo_request` e, portanto, se esse futuro será verificado, as variáveis `host` e `path` que ele toma emprestadas ainda devem estar disponíveis. Tudo está bem.

Utilizando `cheapo_owning_request`, você pode gerar todas as suas solicitações da seguinte forma:

```
for (host, port, path) in requests {
    handles.push(task::spawn_local(cheapo_owning_request(host, port, path)));
}
```

Você pode chamar `many_requests` da função `main` síncrona, com `block_on`:

```
let requests = vec![
    ("example.com".to_string(), 80, "/".to_string()),
    ("www.red-bean.com".to_string(), 80, "/".to_string()),
```

```

    ("en.wikipedia.org".to_string(), 80, "/".to_string()),
];

let results = async_std::task::block_on(many_requests(requests));
for result in results {
    match result {
        Ok(response) => println!("{}", response),
        Err(err) => eprintln!("error: {}", err),
    }
}
}

```

Esse código executa todas as três solicitações simultaneamente de dentro da chamada para `block_on`. Cada uma progride à medida que surge a oportunidade enquanto as outras estão bloqueadas, todas no thread de chamada. A Figura 20.3 mostra uma possível execução das três chamadas para `cheapo_request`.

(Incentivamos você mesmo a tentar executar esse código, com chamadas `eprintln!` adicionadas na parte superior de `cheapo_request` e depois de cada expressão `await` para que possa ver como as chamadas se intercalam de forma diferente entre uma execução e outra.)

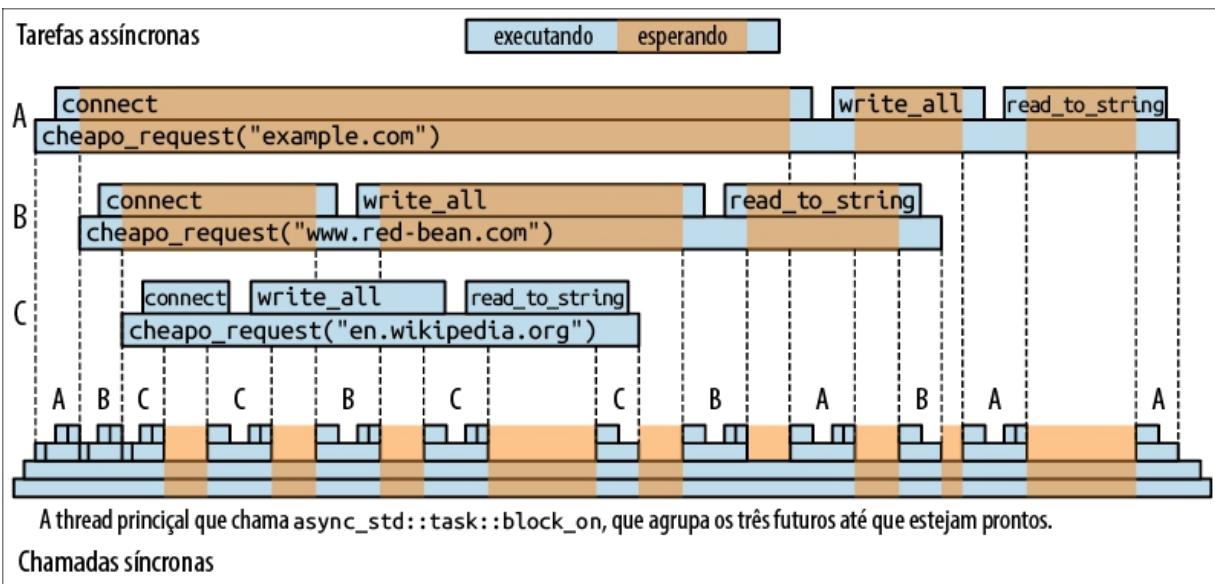


Figura 20.3: Executando três tarefas assíncronas em um único thread.

A chamada para `many_requests` (não mostrada, para simplificar) gerou três tarefas assíncronas, que rotulamos A, B e C. `block_on` começa

verificando `A`, que começa a se conectar a `example.com`. Assim que isso retorna `Poll::Pending`, `block_on` volta a atenção para a próxima tarefa gerada, verificando o futuro `B` e com o tempo `C`, que começam a se conectar aos seus respectivos servidores.

Quando todos os futuros verificáveis retornarem `Poll::Pending`, `block_on` entra em repouso até um dos futuros `TcpStream::connect` indicar que vale a pena verificar a tarefa novamente.

Nessa execução, o servidor `en.wikipedia.org` responde mais rapidamente do que os outros, de modo que a tarefa termina primeiro. Quando uma tarefa gerada é concluída, ela salva o valor em `JoinHandle` e marca-a como pronto, para que `many_requests` prossiga quando a espera. Com o tempo, as outras chamadas para `cheapo_request` serão bem-sucedidas ou retornarão um erro, e a própria `many_requests` pode retornar. Por fim, `main` recebe o vetor dos resultados de `block_on`.

Toda essa execução ocorre em um único thread, as três chamadas para `cheapo_request` sendo intercaladas umas com as outras por meio de verificações sucessivas de seus futuros. Uma chamada assíncrona oferece a aparência de uma única chamada de função executada até a conclusão, mas essa chamada assíncrona é realizada por uma série de chamadas síncronas para o método `poll` do futuro. Cada chamada `poll` individual retorna rapidamente, liberando o thread para que outra chamada assíncrona seja executada.

Por fim, alcançamos o objetivo que definimos no início do capítulo: permitir que um thread assuma outro trabalho enquanto espera que a E/S seja concluída para que os recursos do thread não permaneçam ociosos. Melhor ainda, esse objetivo foi alcançado com um código que se parece muito com o código Rust comum: algumas das funções são marcadas `async`, algumas das chamadas de função são seguidas por `.await` e utilizamos funções de `async_std` em vez de `std`, mas, caso contrário, é um código Rust comum.

Uma diferença importante a ter em mente entre tarefas assíncronas e threads é que a alternância entre uma tarefa assíncrona e outra só ocorre em expressões `await`, quando o futuro sendo esperado retorna `Poll::Pending`. Isso significa que, se você colocar uma computação de

longa duração em `cheapo_request`, nenhuma das outras tarefas para as quais você passou `spawn_local` terá a chance de executar até terminar. Com threads, esse problema não surge: o sistema operacional pode suspender qualquer thread a qualquer momento e definir temporizadores para garantir que nenhum thread monopolize o processador. O código assíncrono depende da cooperação voluntária dos futuros que compartilham o thread. Se você precisar que cálculos de execução longa coexistam com código assíncrono, “Cálculos de longa duração: `yield_now` e `spawn_blocking`”, na página 634, mais adiante neste capítulo descreve algumas opções.

Blocos assíncronos

Além das funções assíncronas, o Rust também suporta blocos assíncronos. Enquanto uma instrução de bloco comum retorna o valor da última expressão, um bloco assíncrono retorna um futuro do valor da última expressão. Você pode utilizar expressões `await` dentro de um bloco assíncrono.

Um bloco assíncrono se parece com uma instrução de bloco comum, precedido pela palavra-chave `async`:

```
let serve_one = async {
    use async_std::net;

    // Escuta conexões e aceita uma
    let listener = net::TcpListener::bind("localhost:8087").await?;
    let (mut socket, _addr) = listener.accept().await?;

    // Fala com o cliente no `socket`
    ...
};
```

Isso inicializa `serve_one` com um futuro que, quando verificado, escuta e trata uma única conexão TCP. O corpo do bloco só inicia a execução depois que `serve_one` é verificado, assim como uma chamada de função assíncrona só começa a execução depois que seu futuro é verificado.

Se você aplicar o operador `?` a um erro em um bloco assíncrono, ele apenas retorna do bloco, não da função circundante. Por exemplo, se a chamada `bind` anterior retornar um erro, o operador `?` vai

retorná-lo como o valor final de `serve_one`. De forma similar, expressões `return` retornam do bloco assíncrono, não da função delimitadora.

Se um bloco assíncrono referenciar variáveis definidas no código circundante, seu futuro captura os valores, assim como uma closure capturaria. E assim como closures `move` (ver “Closures que roubam”, na página 382), você pode iniciar o bloco com `async move` para tomar posse dos valores capturados, em vez de apenas manter referências a eles.

Blocos assíncronos fornecem uma maneira concisa de separar uma seção de código que você quer executar de forma assíncrona. Por exemplo, na seção anterior, `spawn_local` exigiu um futuro `'static`, assim definimos a função empacotadora `cheapo_owning_request` para fornecer um futuro que tomou posse dos argumentos. Você pode obter o mesmo efeito sem a distração de uma função empacotadora simplesmente chamando `cheapo_request` de um bloco assíncrono:

```
pub async fn many_requests(requests: Vec<(String, u16, String)>)
    -> Vec<std::io::Result<String>>
{
    use async_std::task;

    let mut handles = vec![];
    for (host, port, path) in requests {
        handles.push(task::spawn_local(async move {
            cheapo_request(&host, port, &path).await
        }));
    }
    ...
}
```

Como isso é um bloco `async move`, seu futuro toma posse dos valores `String host` e `path`, da mesma maneira como uma closure `move` tomaria. Em seguida, passa referências para `cheapo_request`. O verificador de empréstimo pode ver que a expressão `await` do bloco toma posse do futuro de `cheapo_request`, assim as referências a `host` e `path` não podem sobreviver às variáveis capturadas que tomam emprestadas. O bloco assíncrono realiza a mesma coisa que `cheapo_owning_request`, mas com menos código.

Um problema que você pode encontrar é que não há sintaxe para

especificar o tipo de retorno de um bloco assíncrono, análogo a `-> T` seguindo os argumentos de uma função assíncrona. Isso pode causar problemas ao utilizar o operador `?`:

```
let input = async_std::io::stdin();
let future = async {
    let mut line = String::new();

    // Isso retorna `std::io::Result<usize>`
    input.read_line(&mut line).await?;

    println!("Read line: {}", line);

    Ok(())
};
```

Isso falha com o seguinte erro:

```
error: type annotations needed
  |
48 |     let future = async {
  |         ----- consider giving `future` a type
...
60 |         Ok(())
  |         ^^ cannot infer type for type parameter `E` declared
  |            on the enum `Result`
```

O Rust não pode dizer qual deve ser o tipo de retorno do bloco assíncrono. O método `read_line` retorna `Result<(), std::io::Error>`, mas, como o operador `?` utiliza o trait `From` para converter o tipo de erro em questão para o que a situação exigir, o tipo de retorno do bloco assíncrono pode ser `Result<(), E>` para qualquer tipo `E` que implementa `From<std::io::Error>`.

Versões futuras do Rust provavelmente adicionarão sintaxe para indicar um tipo `async` de retorno do bloco. Por enquanto, você pode contornar o problema especificando o tipo do bloco `Ok` final:

```
let future = async {
    ...
    Ok::<(), std::io::Error>(( ))
};
```

Como `Result` é um tipo genérico que espera os tipos de sucesso e erro como os parâmetros, podemos especificar esses parâmetros de tipo ao utilizar `Ok` ou `Err` como mostrado aqui.

Construindo funções assíncronas a partir de blocos assíncronos

Os blocos assíncronos fornecem outra maneira de obter o mesmo efeito de uma função assíncrona, com um pouco mais de flexibilidade. Por exemplo, poderíamos escrever nosso `cheapo_request` de exemplo como uma função síncrona comum que retorna o futuro de um bloco assíncrono:

```
use std::io;
use std::future::Future;

fn cheapo_request<'a>(host: &'a str, port: u16, path: &'a str)
-> impl Future<Output = io::Result<String>> + 'a
{
    async move {
        ... function body ...
    }
}
```

Ao chamar essa versão da função, ela retorna imediatamente o futuro do valor do bloco assíncrono. Isso captura os argumentos da função e se comporta exatamente como o futuro que a função assíncrona teria retornado. Como não estamos utilizando a sintaxe `async fn`, precisamos escrever o `impl Future` no tipo de retorno, mas, no que diz respeito aos chamadores, essas duas definições são implementações intercambiáveis da mesma assinatura de função.

Essa segunda abordagem pode ser útil quando você deseja fazer algum cálculo imediatamente quando a função é chamada, antes de criar o futuro de seu resultado. Por exemplo, outra forma de conciliar `cheapo_request` com `spawn_local` seria transformá-lo em uma função síncrona retornando um futuro `'static` que captura cópias totalmente possuídas de seus argumentos:

```
fn cheapo_request(host: &str, port: u16, path: &str)
-> impl Future<Output = io::Result<String>> + 'static
{
    let host = host.to_string();
    let path = path.to_string();

    async move {
        ... use &*host, port, and path ...
    }
}
```

```
}
```

Essa versão permite a captura de blocos assíncronos `host` e `path` como valores `String` possuídos, não referências `&str`. Como o futuro possui todos os dados de que precisa para ser executado, ele é válido para o tempo de vida `'static`. (Especificamos `+ 'static` na assinatura mostrada anteriormente, mas `'static` é o padrão para tipos de retorno `-> impl`; portanto, omiti-lo não teria efeito.)

Como essa versão de `cheapo_request` retorna futuros que são `'static`, podemos passá-los diretamente para `spawn_local`:

```
let join_handle = async_std::task::spawn_local(
    cheapo_request("areweasyncyet.rs", 80, "/")
);
```

```
... other work ...
```

```
let response = join_handle.await?;
```

Gerando tarefas assíncronas em um pool de threads

Os exemplos que mostramos até agora passam a maior parte do tempo esperando E/S, mas algumas cargas de trabalho são mais uma mistura de trabalho de processador e bloqueio. Quando você tem computação suficiente para fazer e um único processador não consegue acompanhar, pode utilizar `async_std::task::spawn` para gerar um futuro em um pool de threads de trabalho dedicados a verificar futuros que estão prontos para progredir.

`async_std::task::spawn` é utilizado como `async_std::task::spawn_local`:

```
use async_std::task;

let mut handles = vec![];
for (host, port, path) in requests {
    handles.push(task::spawn(async move {
        cheapo_request(&host, port, &path).await
    }));
}
...
```

Como `spawn_local`, `spawn` retorna um valor `JoinHandle` que você pode esperar para obter o valor final do futuro. Mas, ao contrário de

`spawn_local`, o futuro não precisa esperar que você chame `block_on` antes de ser verificado. Assim que um dos threads do pool de threads estiver livre, ele tentará verificá-lo.

Na prática, `spawn` é mais utilizado do que `spawn_local`, simplesmente porque as pessoas gostam de saber que suas cargas de trabalho, independentemente da combinação de computação e bloqueio, são balanceadas nos recursos da máquina.

Uma coisa a ter em mente ao utilizar `spawn` é que o pool de threads tenta permanecer ocupado, assim o futuro é verificado por qualquer thread que estiver disponível. Uma chamada assíncrona pode iniciar a execução em um thread, bloquear em uma expressão `await` e ser retomada em um thread diferente. Portanto, embora seja uma simplificação razoável visualizar uma chamada de função assíncrona como uma execução de código única e conectada (na verdade, o objetivo das funções assíncronas e expressões `await` é encorajá-lo a pensar dessa maneira), a chamada pode, na verdade, ser realizada por muitos threads diferentes.

Se você estiver utilizando armazenamento local de threads, pode ser surpreendente ver os dados que coloca aí antes de uma expressão `await` ser substituída por algo totalmente diferente posteriormente, porque a tarefa agora está sendo verificada por um thread diferente do pool. Se isso for um problema, você deverá utilizar o armazenamento local de tarefas; ver na documentação do crate `async-std` da macro `task_local!` os detalhes.

Mas seu futuro implementa `Send`?

Há uma restrição que `spawn` impõe que `spawn_local` não impõe. Como o futuro está sendo enviado para outro thread para ser executado, o futuro deve implementar o trait marcador `Send`. Discutimos `Send` em “Segurança de thread: `Send` e `Sync`”, na página 588. Um futuro é `Send` somente se todos os valores que ele contém forem `Send`: todos os argumentos de função, variáveis locais e até mesmo valores temporários anônimos devem ser seguros para serem movidos para outro thread.

Como antes, esse requisito não é exclusivo de tarefas assíncronas: você receberá um erro semelhante se tentar utilizar `std::thread::spawn` para iniciar um thread cuja closure captura valores não-`Send`. A diferença é que, enquanto a closure passada para `std::thread::spawn` permanece no thread que foi criado para executá-la, um futuro gerado em um pool de threads pode mover-se entre um thread e outro sempre que ele espera.

Essa restrição pode ser problemática. Por exemplo, o código a seguir parece bastante inocente:

```
use async_std::task;
use std::rc::Rc;

async fn reluctant() -> String {
    let string = Rc::new("ref-counted string".to_string());

    some_asynchronous_thing().await;
    format!("Your splendid string: {}", string)
}

task::spawn(reluctant());
```

O futuro de uma função assíncrona precisa conter informações suficientes para que a função continue de uma expressão `await`. Nesse caso, futuro de `reluctant` deve utilizar `string` depois de `await`, então o futuro vai, pelo menos às vezes, conter um valor `Rc<String>`. Como ponteiros `Rc` não podem ser compartilhados com segurança entre threads, o próprio futuro não pode ser `Send`. E como `spawn` só aceita futuros que são `Send`, Rust reclama:

```
error: future cannot be sent between threads safely
|
17 |     task::spawn(reluctant());
|     ^^^^^^^^^^^^^ future returned by `reluctant` is not `Send`
|
|
127 | T: Future + Send + 'static,
|     ---- required by this bound in `async_std::task::spawn`
|
= help: within `impl Future`, the trait `Send` is not implemented
       for `Rc<String>`
note: future is not `Send` as this value is used across an await
|
10 |     let string = Rc::new("ref-counted string".to_string());
```



```

|          ----- has type `Rc<String>` which is not `Send`
11 |
12 |     some_asynchronous_thing().await;
|     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
|     await occurs here, with `string` maybe used later
...
15 | }
|   - `string` is later dropped here

```

Essa mensagem de erro é longa, mas contém muitos detalhes úteis:

- Isso explica por que o futuro precisa ser `Send`: `task::spawn` requer isso.
- Explica qual valor não é `Send`: a variável local `string`, cujo tipo é `Rc<String>`.
- Isso explica por que `string` afeta o futuro: está no escopo depois do `await` indicado.

Há duas maneiras de corrigir esse problema. Uma delas é restringir o escopo do valor não-`Send` para que não abranja nenhuma expressão `await` e, portanto, não precisa ser salvo no futuro da função:

```

async fn reluctant() -> String {
    let return_value = {
        let string = Rc::new("ref-counted string".to_string());
        format!("Your splendid string: {}", string)
        // A `Rc<String>` sai do escopo aqui...
    };

    // ... e assim não está disponível quando suspendemos aqui
    some_asynchronous_thing().await;

    return_value
}

```

Outra solução é simplesmente utilizar `std::sync::Arc` em vez de `Rc`. `Arc` utiliza atualizações atômicas para gerenciar as contagens de referências, o que o torna um pouco mais lento, mas ponteiros `Arc` são `Send`.

Embora com o tempo você aprenderá a reconhecer e evitar tipos não-`Send`, eles podem ser inicialmente um pouco surpreendentes. (Pelo menos, seus autores frequentemente se surpreendiam.) Por exemplo, o código Rust mais antigo às vezes utiliza tipos de

resultados genéricos como este:

```
// Não recomendado!  
type GenericError = Box<dyn std::error::Error>;  
type GenericResult<T> = Result<T, GenericError>;
```

Esse tipo `GenericError` utiliza um objeto `trait` em `box` para conter um valor de qualquer tipo que implemente `std::error::Error`. Mas não impõe restrições adicionais: se alguém tivesse um tipo não-`Send` que implementasse `Error`, ele poderia converter um valor em `box` desse tipo em um `GenericError`. Devido a essa possibilidade, `GenericError` não é `Send` e o código a seguir não funcionará:

```
fn some_fallible_thing() -> GenericResult<i32> {  
    ...  
}  
  
// O futuro desta função não é `Send` ...  
async fn unfortunate() {  
    // ... porque o valor dessa chamada ...  
    match some_fallible_thing() {  
        Err(error) => {  
            report_error(error);  
        }  
        Ok(output) => {  
            // ... está ativo nessa espera ...  
            use_output(output).await;  
        }  
    }  
}  
  
// ... e assim esse `spawn` é um erro  
async_std::task::spawn(unfortunate());
```

Como no exemplo anterior, a mensagem de erro do compilador explica o que está acontecendo, apontando para o tipo `Result` como o culpado. Como o Rust considera o resultado de `some_fallible_thing` estando presente durante toda a instrução `match`, incluindo a expressão `await`, ele determina que o futuro de `unfortunate` não é `Send`. Isso é um erro com o qual o Rust se preocupa excessivamente: embora seja verdade que `GenericError` não é seguro para ser enviado para outro thread, a `await` só ocorre quando o resultado é `Ok`, então o valor de erro nunca existe quando esperamos o futuro de `use_output`.

A solução ideal é utilizar tipos de erros genéricos mais rígidos, como

os que sugerimos em “Trabalhando com vários tipos de erro”, na página 197:

```
type GenericError = Box<dyn std::error::Error + Send + Sync + 'static>;  
type GenericResult<T> = Result<T, GenericError>;
```

Esse objeto `trait` requer explicitamente que o tipo de erro subjacente implemente `Send` e está tudo bem.

Se o futuro não é `Send` e você não pode torná-lo convenientemente nisso, então ainda pode utilizar `spawn_local` para executá-lo no thread atual. Claro que precisará certificar-se de que o thread chame `block_on` em algum momento, para lhe dar uma chance de ser executado, e você não se beneficiará de distribuir o trabalho por vários processadores.

Cálculos de longa duração: `yield_now` e `spawn_blocking`

Para que um futuro compartilhe perfeitamente o thread com outras tarefas, o método `poll` sempre deve retornar o mais rápido possível. Mas, se você estiver realizando uma computação longa, pode levar muito tempo para alcançar a próxima `await`, fazendo com que outras tarefas assíncronas esperem mais tempo do que você gostaria para que tenham a vez no thread.

Uma maneira de evitar isso é simplesmente fazer `await` de alguma coisa ocasionalmente. A função `async_std::task::yield_now` retorna um futuro simples projetado para isso:

```
while computation_not_done() {  
    ... do one medium-sized step of computation ...  
    async_std::task::yield_now().await;  
}
```

A primeira vez que o futuro `yield_now` é verificado, ele retorna `Poll::Pending`, mas diz que vale a pena verificar novamente em breve. O efeito é que a chamada assíncrona abandona o thread e outras tarefas têm a chance de serem executadas, mas a chamada entrará na fila em breve. A segunda vez que o futuro `yield_now` é verificado, ele retorna `Poll::Ready(())` e a função assíncrona pode retomar a execução.

Mas essa abordagem nem sempre é viável. Se você estiver utilizando um crate externo para fazer cálculos de longa duração ou evocar o C ou C++, pode não ser conveniente alterar esse código para ser mais compatível com assíncrono. Ou pode ser difícil garantir que cada caminho ao longo dos cálculos certamente alcançará a `await` de tempos em tempos.

Para casos como esse, você pode utilizar `async_std::task::spawn_blocking`. Essa função recebe uma closure, inicia a execução em seu próprio thread e retorna um futuro do valor de retorno. O código assíncrono pode aguardar esse futuro, liberando o thread para outras tarefas até o cálculo ser concluído. Inserindo o trabalho duro em um thread separado, você pode deixar o sistema operacional cuidar de fazer com que ele compartilhe o processador de maneira adequada.

Por exemplo, suponha que precisamos verificar as senhas fornecidas pelos usuários em relação às versões com hash que armazenamos em nosso banco de dados de autenticação. Por segurança, a verificação de uma senha precisa ser computacionalmente intensiva para que, mesmo que os invasores obtenham uma cópia de nosso banco de dados, eles não possam simplesmente tentar trilhões de senhas possíveis para ver se alguma corresponde. O crate `argonautica` fornece uma função de hash projetada especificamente para armazenar senhas: um hash `argonautica` apropriadamente gerado leva uma fração significativa de um segundo para verificar. Podemos utilizar `argonautica` (versão 0.2) em nosso aplicativo assíncrono assim:

```
async fn verify_password(password: &str, hash: &str, key: &str)
    -> Result<bool, argonautica::Error>
{
    // Faz cópias dos argumentos, para que a closure possa ser 'estática'
    let password = password.to_string();
    let hash = hash.to_string();
    let key = key.to_string();

    async_std::task::spawn_blocking(move || {
        argonautica::Verifier::default()
            .with_hash(hash)
            .with_password(password)
            .with_secret_key(key)
            .verify()
    }).await
}
```

```
}
```

Isso retorna `Ok(true)` se `password` corresponder com `hash`, dado `key`, uma chave para o banco de dados como um todo. Ao fazer a verificação na closure passada para `spawn_blocking`, enviamos o cálculo caro para um thread próprio, garantindo que isso não afete nossa capacidade de resposta às solicitações de outros usuários.

Comparando projetos assíncronos

De várias maneiras, a abordagem do Rust à programação assíncrona lembra aquela adotada por outras linguagens. Por exemplo, JavaScript, C# e o Rust têm funções assíncronas com expressões `await`. E todas essas linguagens têm valores que representam cálculos incompletos: O Rust chama-os “futuros”; já JavaScript chama-os “promessas” e o C# chama-os “tarefas”, mas todos representam um valor pelo qual você pode ter de esperar.

O uso de verificações (polling) pelo Rust, porém, é incomum. Em JavaScript e C#, uma função assíncrona começa a ser executada assim que é chamada, e há um loop de evento global integrado à biblioteca do sistema que retoma as chamadas de função assíncrona suspensas quando os valores que estavam aguardando se tornam disponíveis. No Rust, porém, uma chamada assíncrona não faz nada até que você a passe para uma função como `block_on`, `spawn` ou `spawn_local` que vai verificá-la e conduzir o trabalho até a conclusão. Essas funções, chamadas executores, desempenham o papel que outras linguagens abrangem com um loop de evento global.

Como o Rust faz com que você, o programador, escolha um executor para verificar os futuros, o Rust não precisa de um loop de evento global integrado ao sistema. O crate `async-std` oferece as funções executoras que utilizamos neste capítulo até agora, mas o crate `tokio`, que utilizaremos mais adiante neste capítulo, define um conjunto próprio de funções executoras semelhantes. E, no final deste capítulo, implementaremos nosso próprio executor. Você pode utilizar todas as três no mesmo programa.

Um cliente HTTP assíncrono real

Seríamos negligentes se não mostrássemos um exemplo do uso de um crate de cliente HTTP assíncrono adequado, já que é muito fácil, e há vários crates bons a escolher, incluindo `reqwest` e `surf`.

Eis uma reescrita de `many_requests`, ainda mais simples do que a baseada em `cheapo_request`, que utiliza `surf` para executar uma série de solicitações simultaneamente. Você precisará dessas dependências no arquivo *Cargo.toml*:

```
[dependencies]
async-std = "1.7"
surf = "1.0"
```

Então, podemos definir `many_requests` do seguinte modo:

```
pub async fn many_requests(urls: &[String])
    -> Vec<Result<String, surf::Exception>>
{
    let client = surf::Client::new();

    let mut handles = vec![];
    for url in urls {
        let request = client.get(&url).recv_string();
        handles.push(async_std::task::spawn(request));
    }

    let mut results = vec![];
    for handle in handles {
        results.push(handle.await);
    }

    results
}

fn main() {
    let requests = &["http://example.com".to_string(),
                     "https://www.red-bean.com".to_string(),
                     "https://en.wikipedia.org/wiki/Main_Page".to_string()];

    let results = async_std::task::block_on(many_requests(requests));
    for result in results {
        match result {
            Ok(response) => println!("*** {}\\n", response),
            Err(err) => eprintln!(",error: {}\\n", err),
        }
    }
}
```

Utilizar um único `surf::Client` para produzir todas as nossas solicitações permite reutilizar conexões HTTP se várias delas forem direcionadas ao mesmo servidor. E nenhum bloco assíncrono é necessário: como `recv_string` é um método assíncrono que retorna um futuro `Send + 'static'`, podemos passar o futuro diretamente para `spawn`.

Um cliente e servidor assíncronos

É hora de pegar as ideias-chave que discutimos até agora e reuni-las em um programa de trabalho. Em grande parte, os aplicativos assíncronos lembram aplicativos multithread comuns, mas há novas oportunidades de código compacto e expressivo que você pode procurar.

O exemplo desta seção é um servidor e cliente de bate-papo. Confira o código completo (<https://oreil.ly/QFSUS>). Sistemas de bate-papo reais são complicados, com preocupações que vão desde segurança e reconexão até privacidade e moderação, mas reduzimos o nosso a um conjunto austero de recursos para focar alguns pontos de interesse.

Em particular, queremos lidar bem com a contrapressão. Com isso queremos dizer que, se um cliente tiver uma conexão de rede lenta ou que cai totalmente, isso nunca afetará a capacidade de outros clientes de trocar mensagens em seu próprio ritmo. E como um cliente lento não deve fazer com que o servidor gaste memória ilimitada, mantendo o acúmulo crescente de mensagens, nosso servidor deve descartar mensagens de clientes que não conseguem acompanhar e notificá-los de que seu fluxo está incompleto. (Um servidor de bate-papo real registraria as mensagens no disco e permitiria que os clientes recuperassem aquelas que perderam, mas deixamos isso de fora.)

Iniciamos o projeto com o comando `cargo new --lib async-chat` e inserimos o texto a seguir em *async-chat/Cargo.toml*:

```
[package]
name = "async-chat"
version = "0.1.0"
authors = ["You <you@example.com>"]
```

```
edition = "2021"
```

```
[dependencies]
```

```
async-std = { version = "1.7", features = ["unstable"] }
```

```
tokio = { version = "1.0", features = ["sync"] }
```

```
serde = { version = "1.0", features = ["derive", "rc"] }
```

```
serde_json = "1.0"
```

Dependemos de quatro crates:

- O crate `async-std` é a coleção de utilitários e primitivas de E/S assíncronos que utilizamos ao longo do capítulo.
- O crate `tokio` é outra coleção de primitivas assíncronas como `async-std`, um dos mais antigos e maduros. É amplamente utilizado e mantém seu design e implementação de alto padrão, mas requer um pouco mais de cuidado para utilizar do que `async-std`.

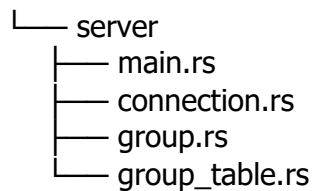
`tokio` é um crate grande, mas precisamos apenas de um componente dele, assim o campo `features = ["sync"]` na linha de dependência de *Cargo.toml* de `tokio` se resume às partes de que precisamos, tornando isso uma dependência leve.

Quando o ecossistema de bibliotecas assíncronas era menos maduro, as pessoas evitavam utilizar tanto `tokio` como `async-std` no mesmo programa, mas os dois projetos têm cooperado para garantir que isso funcione, desde que as regras documentadas de cada crate sejam seguidas.

- Os crates `serde` e `serde_json` que vimos antes, no Capítulo 18. Eles fornecem ferramentas convenientes e eficientes para gerar e analisar JSON, que nosso protocolo de bate-papo utiliza para representar dados na rede. Queremos utilizar alguns recursos opcionais de `serde`, portanto nós os selecionamos ao fornecer a dependência.

Toda a estrutura do aplicativo de bate-papo, cliente e servidor, se parece com isto:

```
async-chat
├── Cargo.toml
├── src
│   ├── lib.rs
│   ├── utils.rs
│   └── bin
│       └── client.rs
```

Esse layout de pacote utiliza um recurso do Cargo que abordamos em “Diretório `src/bin`”, na página 224: além do crate principal da biblioteca, `src/lib.rs`, com seu submódulo `src/utils.rs`, também inclui dois executáveis:

- `src/bin/client.rs` é um executável de arquivo único para o cliente de bate-papo.
- `src/bin/server` é o executável do servidor, distribuído em quatro arquivos: `main.rs` contém a função `main` e há três submódulos, `connection.rs`, `group.rs` e `group_table.rs`.

Apresentaremos o conteúdo de cada arquivo de origem ao longo do capítulo, mas depois que todos estiverem no lugar, se você digitar `cargo build` nessa árvore, que compila o crate da biblioteca e, em seguida, constrói os dois executáveis. Cargo inclui automaticamente o crate da biblioteca como uma dependência, tornando-o um local conveniente para colocar as definições compartilhadas pelo cliente e servidor. De forma similar, `cargo check` verifica toda a árvore de origem. Para executar qualquer um dos executáveis, você pode utilizar comandos como estes:

```
$ cargo run --release --bin server -- localhost:8088
$ cargo run --release --bin client -- localhost:8088
```

A opção `--bin` indica qual executável executar e quaisquer argumentos após a opção `--` são passados para o próprio executável. Nosso cliente e servidor só querem saber o endereço do servidor e a porta TCP.

Tipos de erro e resultado

O módulo `crate utils` da biblioteca define os tipos de resultado e erro que utilizaremos em todo o aplicativo. A partir de `src/utils.rs`:

```
use std::error::Error;

pub type ChatError = Box<dyn Error + Send + Sync + 'static>;
pub type ChatResult<T> = Result<T, ChatError>;
```

Esses são os tipos de erro de propósito geral que sugerimos em “Trabalhando com vários tipos de erro”, na página 197. Cada um dos crates `async_std`, `serde_json` e `tokio` define tipos de erro próprios, mas o operador `?` pode convertê-los automaticamente em um `ChatError`, utilizando a implementação da biblioteca padrão do trait `From` que pode converter qualquer tipo de erro adequado em `Box<dyn Error + Send + Sync + 'static>`. O requisito de ser `Send` e `Sync` garante que, se uma tarefa gerada em outro thread falhar, ela poderá relatar o erro com segurança para o thread principal.

Em um aplicativo real, considere utilizar o crate `anyhow`, que fornece tipos `Error` e `Result` semelhantes a esses. O crate `anyhow` é fácil de utilizar e oferece alguns recursos interessantes além do que nosso `ChatError` e `ChatResult` podem oferecer.

O protocolo

O crate da biblioteca captura todo o nosso protocolo de bate-papo nesses dois tipos, definidos em *lib.rs*:

```
use serde::{Deserialize, Serialize};
use std::sync::Arc;

pub mod utils;

#[derive(Debug, Deserialize, Serialize, PartialEq)]
pub enum FromClient {
    Join { group_name: Arc<String> },
    Post {
        group_name: Arc<String>,
        message: Arc<String>,
    },
}

#[derive(Debug, Deserialize, Serialize, PartialEq)]
pub enum FromServer {
    Message {
        group_name: Arc<String>,
        message: Arc<String>,
    },
    Error(String),
}

#[test]
```

```

fn test_fromclient_json() {
    use std::sync::Arc;

    let from_client = FromClient::Post {
        group_name: Arc::new("Dogs".to_string()),
        message: Arc::new("Samoyeds rock!".to_string()),
    };

    let json = serde_json::to_string(&from_client).unwrap();
    assert_eq!(json,
        r#"{"Post":{"group_name":"Dogs","message":"Samoyeds rock!"}}"#);

    assert_eq!(serde_json::from_str::(<FromClient>(&json).unwrap(),
        from_client);
}

```

O enum `FromClient` representa os pacotes que um cliente pode enviar para o servidor: ele pode pedir para entrar em um grupo e enviar mensagens para qualquer grupo no qual tenha entrado. `FromServer` representa o que o servidor pode enviar de volta: mensagens postadas em algum grupo e mensagens de erro. Utilizar uma contagem de referência `Arc<String>` em vez de uma `String` simples ajuda o servidor a evitar criar cópias das strings à medida que gerencia grupos e distribui mensagens.

Os atributos `#[derive]` dizem ao crate `serde` que gere implementações dos traits `Serialize` e `Deserialize` para `FromClient` e `FromServer`. Isso permite chamar `serde_json::to_string` para convertê-los em valores JSON, enviá-los pela rede e, finalmente, chamar `serde_json::from_str` para convertê-los de volta em suas formas do Rust.

O teste da unidade `test_fromclient_json` ilustra como isso é utilizado. Considerando a implementação `Serialize` derivada de `serde`, podemos chamar `serde_json::to_string` para transformar o valor `FromClient` específico neste JSON:

```

{"Post":{"group_name":"Dogs","message":"Samoyeds rock!"}}

```

Então, a implementação `Deserialize` derivada analisa isso de volta em um valor `FromClient` equivalente. Observe que os ponteiros `Arc` em `FromClient` não têm efeito na forma serializada: as strings contadas por referência aparecem diretamente como valores de membro do objeto JSON.

Recebendo a entrada do usuário: Fluxos assíncronos

A primeira responsabilidade do nosso cliente de bate-papo é ler os comandos do usuário e enviar os pacotes correspondentes ao servidor. O gerenciamento de uma interface de usuário adequada está além do escopo deste capítulo, portanto faremos a coisa mais simples possível que funcione: ler as linhas diretamente da entrada padrão. O seguinte código é inserido em *src/bin/client.rs*:

```
use async_std::prelude::*;
use async_chat::utils::{self, ChatResult};
use async_std::io;
use async_std::net;

async fn send_commands(mut to_server: net::TcpStream) -> ChatResult<()> {
    println!("Commands:\n\
        join GROUP\n\
        post GROUP MESSAGE...\n\
        Type Control-D (on Unix) or Control-Z (on Windows) \
        to close the connection.");

    let mut command_lines = io::BufReader::new(io::stdin()).lines();
    while let Some(command_result) = command_lines.next().await {
        let command = command_result?;
        // Consulta o repositório do GitHub para obter a definição de `parse_command`
        let request = match parse_command(&command) {
            Some(request) => request,
            None => continue,
        };

        utils::send_as_json(&mut to_server, &request).await?;
        to_server.flush().await?;
    }

    Ok(())
}
```

Isso chama `async_std::io::stdin` para obter um identificador assíncrono na entrada padrão do cliente, envolve-o em um `async_std::io::BufReader` para armazená-lo em buffer e, em seguida, chamar `lines` para processar a entrada do usuário linha por linha. Ele tenta analisar cada linha como um comando correspondente a algum valor `FromClient` e, se for bem-sucedido, envia esse valor para o servidor. Se

o usuário inserir um comando não reconhecido, `parse_command` imprime uma mensagem de erro e retorna `None`, assim `send_commands` pode contornar o loop novamente. Se o usuário digitar uma indicação de fim de arquivo, o fluxo `lines` retorna `None` e `send_commands` retorna. Isso é muito parecido com o código que você escreveria em um programa síncrono comum, exceto pelo fato de que ele utiliza versões `async_std` dos recursos da biblioteca.

O método assíncrono `lines` de `BufReader` é interessante. Ele não pode retornar um iterador, a maneira como a biblioteca padrão faz: o método `Iterator::next` é uma função síncrona comum, portanto chamar `command_lines.next()` bloquearia o thread até que a próxima linha esteja pronta. Em vez disso, `lines` retorna um fluxo dos valores `Result<String>`. Um fluxo é o análogo assíncrono de um iterador: ele produz uma sequência de valores sob demanda, de maneira amigável a assíncrono. Eis a definição do trait `Stream`, do módulo `async_std::stream`:

```
trait Stream {
    type Item;

    // Por enquanto, leia `Pin<&mut Self>` como `&mut Self`
    fn poll_next(self: Pin<&mut Self>, cx: &mut Context<'_>)
        -> Poll<Option<Self::Item>>;
}
```

Você pode analisar isso como um híbrido dos traits `Iterator` e `Future`. Como um iterador, um `Stream` tem um tipo `Item` associado e usa `Option` para indicar quando a sequência terminou. Mas, como um futuro, um fluxo deve ser verificado: para obter o próximo item (ou saber que o fluxo terminou), você deve chamar `poll_next` até que retorne `Poll::Ready`. A implementação de um fluxo `poll_next` sempre deve retornar rapidamente, sem bloqueio. E se um fluxo retornar `Poll::Pending`, ele deverá notificar o chamador quando vale a pena verificar novamente por meio do `Context`.

O método `poll_next` é difícil de utilizar diretamente, mas geralmente não é necessário fazer isso. Assim como os iteradores, fluxos têm uma ampla coleção de métodos utilitários, como `filter` e `map`. Entre esses há um método `next`, que retorna um futuro da próxima `Option<Self::Item>` do fluxo. Em vez de verificar o fluxo explicitamente,

you can call `next` and wait for the future it returns.

Putting these parts together, `send_commands` consumes the flow of lines of input by making a loop over the values generated by a flow using `next` with `while let`:

```
while let Some(item) = stream.next().await {  
    ... use item ...  
}
```

(Future versions of Rust will probably introduce a variant of the asynchronous loop `for` for consuming flows, just as a common loop `for` consumes values of `Iterator`.)

Check a flow after it has terminated – or, in other words, after it has returned `Poll::Ready(None)` to indicate the end of the flow – is like calling `next` on an iterator after it has returned `None`, or checking a future after it has returned `Poll::Ready`: the trait `Stream` does not specify what the flow should do, and some flows can behave badly. Like futures and iterators, flows have a `fuse` method to guarantee that these calls behave in a predictable way, when necessary; see the documentation for details.

When working with flows, it is important to remember to use the prelude `async_std`:

```
use async_std::prelude::*;
```

This happens because the utility methods for the trait `Stream`, such as `next`, `map`, `filter` and so on, are not defined in the trait `Stream` itself. Instead, they are standard methods of a different trait, `StreamExt`, which is implemented automatically for all `Stream`s:

```
pub trait StreamExt: Stream {  
    ... define utility methods as default methods ...  
}  
  
impl<T: Stream> StreamExt for T { }
```

This is an example of the trait extension pattern that we described in “Traits and types of other people”, on page 312. The module `async_std::prelude` places the `StreamExt` methods in scope, so that using the prelude guarantees that the methods remain visible in your code.

Enviando pacotes

Para transmitir pacotes em um soquete de rede, nosso cliente e servidor utilizam a função `send_as_json` do módulo `utils` do crate de biblioteca:

```
use async_std::prelude::*;
use serde::Serialize;
use std::marker::Unpin;

pub async fn send_as_json<S, P>(outbound: &mut S, packet: &P) -> ChatResult<()>
where
    S: async_std::io::Write + Unpin,
    P: Serialize,
{
    let mut json = serde_json::to_string(&packet)?;
    json.push('\n');
    outbound.write_all(json.as_bytes()).await?;
    Ok(())
}
```

Essa função constrói a representação JSON de `packet` como uma `String`, adiciona uma nova linha ao final e grava tudo em `outbound`.

Da cláusula `where`, podemos ver que `send_as_json` é bastante flexível. O tipo de pacote a ser enviado, `P`, pode ser qualquer coisa que implemente `serde::Serialize`. O fluxo de saída `s` pode ser qualquer coisa que implemente `async_std::io::Write`, a versão assíncrona do trait `std::io::Write` para fluxos de saída. Isso é suficiente para enviar valores `FromClient` e `FromServer` em um `TcpStream` assíncrono. Manter a definição de `send_as_json` genérico garante que ele não depende dos detalhes do fluxo ou dos tipos de pacotes de maneiras surpreendentes: `send_as_json` só pode utilizar métodos desses traits.

A restrição `Unpin` em `s` precisa utilizar o método `write_all`. Discutiremos fixação e desafixação de ponteiros mais adiante neste capítulo, mas, por enquanto, basta adicionar restrições `Unpin` para variáveis de tipo onde necessário; o compilador do Rust apontará esses casos se você esquecer.

Em vez de serializar o pacote diretamente para o fluxo `outbound`, `send_as_json` serializa para uma `String` temporária e então grava isso em `outbound`. O crate `serde_json` fornece funções para serializar valores

diretamente para fluxos de saída, mas essas funções só suportam fluxos síncronos. Gravar em fluxos assíncronos exigiria mudanças fundamentais tanto em `serde_json` como no núcleo independente de formato do crate `serde`, uma vez que os traits em torno dos quais eles são projetados têm métodos síncronos.

Tal como acontece com os fluxos, na verdade muitos dos métodos dos traits de I/O de `async_std` são definidos nos traits de extensão, por isso é importante lembrar `use async_std::prelude::*` sempre que você estiver utilizando-os.

Recebendo pacotes: mais fluxos assíncronos

Para receber pacotes, nosso servidor e cliente utilizarão essa função do módulo `utils` para receber valores `FromClient` e `FromServer` de um soquete TCP armazenado em buffer assíncrono, um `async_std::io::BufReader<TcpStream>`:

```
use serde::de::DeserializeOwned;

pub fn receive_as_json<S, P>(inbound: S) -> impl Stream<Item = ChatResult<P>>
    where S: async_std::io::BufRead + Unpin,
          P: DeserializeOwned,
{
    inbound.lines()
        .map(|line_result| -> ChatResult<P> {
            let line = line_result?;
            let parsed = serde_json::from_str::<P>(&line)?;
            Ok(parsed)
        })
}
```

Como `send_as_json`, essa função é genérica no fluxo de entrada e nos tipos de pacote:

- O tipo de fluxo `s` deve implementar `async_std::io::BufRead`, o análogo assíncrono de `std::io::BufRead`, representando um fluxo de bytes de entrada em buffer.
- O tipo de pacote `P` deve implementar `DeserializeOwned`, uma variante mais estrita de `Deserialize` do trait `serde`. Para eficiência, `Deserialize` pode produzir valores `&str` e `&[u8]` que pegam emprestado o conteúdo

diretamente do buffer do qual foram desserializados, para evitar a cópia de dados. Em nosso caso, entretanto, isso não é bom: precisamos retornar os valores desserializados para nosso chamador, então eles devem existir por mais tempo que os buffers dos quais os analisamos. Um tipo que implementa `DeserializeOwned` é sempre independente do buffer do qual foi desserializado.

Chamar `inbound.lines()` fornece um `Stream` dos valores `std::io::Result<String>`. Em seguida, utilizamos o adaptador `map` do fluxo para aplicar uma closure a cada item, tratando erros e analisando cada linha como a forma JSON de um valor do tipo `P`. Isso fornece um fluxo de valores `ChatResult<P>`, que retornamos diretamente. O tipo de retorno da função é:

```
impl Stream<Item = ChatResult<P>>
```

Isso indica que retornamos algum tipo que produz uma sequência de valores `ChatResult<P>` de forma assíncrona, mas nosso chamador não pode dizer exatamente qual tipo isso é. Como a closure que passamos para `map` tem de qualquer maneira um tipo anônimo, esse é o tipo mais específico que `receive_as_json` possivelmente poderia retornar.

Observe que `receive_as_json` não é, em si, uma função assíncrona. É uma função comum que retorna um valor assíncrono, um fluxo. Compreender a mecânica do suporte assíncrono do Rust mais profundamente do que “apenas adicionar `async` e `.await` em todos os lugares” abre o potencial para definições claras, flexíveis e eficientes como essa, que tiram o máximo proveito da linguagem.

Para ver como `receive_as_json` é utilizado, eis a função `handle_replies` do nosso cliente de bate-papo de *src/bin/client.rs*, que recebe um fluxo de valores `FromServer` da rede e os imprime para o usuário visualizar:

```
use async_chat::FromServer;

async fn handle_replies(from_server: net::TcpStream) -> ChatResult<()> {
    let buffered = io::BufReader::new(from_server);
    let mut reply_stream = utils::receive_as_json(buffered);
    while let Some(reply) = reply_stream.next().await {
        match reply? {
            FromServer::Message { group_name, message } => {
                println!("message posted to {}: {}", group_name, message);
            }
        }
    }
}
```

```

    }
    FromServer::Error(message) => {
        println!("error from server: {}", message);
    }
}
}
Ok(())
}

```

Essa função usa um soquete que recebe dados do servidor, empacota um `BufReader` em volta (preste atenção, na versão do `async_std`) e, em seguida, passa isso para `receive_as_json` a fim de obter um fluxo dos valores `FromServer` de entrada. Então ele utiliza um loop `while let` para lidar com as respostas recebidas, verificando resultados de erro e imprimindo cada resposta do servidor para o usuário visualizar.

A função main do cliente

Como apresentamos tanto `send_commands` como `handle_replies`, podemos mostrar a função principal do cliente de bate-papo, de *src/bin/client.rs*:

```

use async_std::task;

fn main() -> ChatResult<()> {
    let address = std::env::args().nth(1)
        .expect("Usage: client ADDRESS:PORT");

    task::block_on(async {
        let socket = net::TcpStream::connect(address).await?;
        socket.set_nodelay(true)?;

        let to_server = send_commands(socket.clone());
        let from_server = handle_replies(socket);

        from_server.race(to_server).await?;

        Ok(())
    })
}

```

Tendo obtido o endereço do servidor a partir da linha de comando, `main` tem uma série de funções assíncronas que quer chamar, então ele encapsula o restante da função em um bloco assíncrono e passa

o futuro do bloco para `async_std::task::block_on` a fim de ser executado.

Uma vez estabelecida a conexão, queremos que as funções `send_commands` e `handle_replies` sejam executadas em conjunto, para que possamos ver as mensagens de outras pessoas chegarem enquanto digitamos. Se inserimos o indicador de fim de arquivo ou se a conexão com o servidor cair, o programa deve sair.

Dado o que fizemos em outras partes do capítulo, você pode esperar um código como este:

```
let to_server = task::spawn(send_commands(socket.clone()));
let from_server = task::spawn(handle_replies(socket));

to_server.await?;
from_server.await?;
```

Mas, como aguardamos os dois identificadores de junção, isso fornece um programa que é encerrado assim que ambas as tarefas são concluídas. Queremos sair assim que qualquer um terminar. O método `race` em futuros realiza isso. A chamada `from_server.race(to_server)` retorna um novo futuro que verifica tanto `from_server` e `to_server` e retorna `Poll::Ready(v)` assim que um deles estiver pronto. Ambos os futuros devem ter o mesmo tipo de saída: o valor final é aquele do futuro finalizado primeiro. O futuro incompleto é dropado.

O método `race`, com muitos outros recursos úteis, é definido no `trait async_std::prelude::FutureExt`, que `async_std::prelude` torna visível para nós.

Nesse ponto, a única parte do código do cliente que não mostramos é a função `parse_command`. Isso é código de tratamento de texto bastante simples e direto, assim não mostraremos sua definição aqui. Consulte o código completo no repositório Git para obter detalhes.

A função main do servidor

Eis todo o conteúdo do arquivo principal para o servidor, *src/bin/server/main.rs*:

```
use async_std::prelude::*;
use async_chat::utils::ChatResult;
use std::sync::Arc;

mod connection;
```

```

mod group;
mod group_table;

use connection::serve;

fn main() -> ChatResult<()> {
    let address = std::env::args().nth(1).expect("Usage: server ADDRESS");

    let chat_group_table = Arc::new(group_table::GroupTable::new());

    async_std::task::block_on(async {
        // Esse código foi mostrado na introdução do capítulo
        use async_std::{net, task};

        let listener = net::TcpListener::bind(address).await?;

        let mut new_connections = listener.incoming();
        while let Some(socket_result) = new_connections.next().await {
            let socket = socket_result?;
            let groups = chat_group_table.clone();
            task::spawn(async {
                log_error(serve(socket, groups).await);
            });
        }

        Ok(())
    })
}

fn log_error(result: ChatResult<()>) {
    if let Err(error) = result {
        eprintln!("Error: {}", error);
    }
}

```

A função `main` do servidor se parece com a do cliente: faz um pouco de configuração e depois chama `block_on` para executar um bloco assíncrono que realiza o trabalho real. Para lidar com as conexões recebidas dos clientes, ela cria um soquete `TcpListener`, cujo método `incoming` retorna um fluxo de valores `std::io::Result<TcpStream>`.

Para cada conexão de entrada, geramos uma tarefa assíncrona executando a função `connection::serve`. Cada tarefa também recebe uma referência a um valor `GroupTable` que representa a lista atual de grupos de bate-papo do nosso servidor, compartilhada por todas as conexões por meio de um ponteiro de contagem de referências `Arc`.

Se `connection::serve` retorna um erro, registramos uma mensagem na

saída de erro padrão e deixamos a tarefa encerrar. Outras conexões continuam funcionando normalmente.

Lidando com conexões de bate-papo: Mutexes assíncronos

Eis o burro de carga do servidor: a função `serve` do módulo `connection` em `src/bin/server/connection.rs`:

```
use async_chat::{FromClient, FromServer};
use async_chat::utils::{self, ChatResult};
use async_std::prelude::*;
use async_std::io::BufReader;
use async_std::net::TcpStream;
use async_std::sync::Arc;

use crate::group_table::GroupTable;

pub async fn serve(socket: TcpStream, groups: Arc<GroupTable>)
    -> ChatResult<()>
{
    let outbound = Arc::new(Outbound::new(socket.clone()));

    let buffered = BufReader::new(socket);
    let mut from_client = utils::receive_as_json(buffered);
    while let Some(request_result) = from_client.next().await {
        let request = request_result?;

        let result = match request {
            FromClient::Join { group_name } => {
                let group = groups.get_or_create(group_name);
                group.join(outbound.clone());
                Ok(())
            }
            FromClient::Post { group_name, message } => {
                match groups.get(&group_name) {
                    Some(group) => {
                        group.post(message);
                        Ok(())
                    }
                    None => {
                        Err(format!("Group '{}' does not exist", group_name))
                    }
                }
            }
        }
    }
};
```

```

        if let Err(message) = result {
            let report = FromServer::Error(message);
            outbound.send(report).await?;
        }
    }
    Ok(())
}

```

Isso é quase uma imagem espelhada da função `handle_replies` do cliente: a maior parte do código é um loop tratando um fluxo de entrada dos valores `FromClient`, construídos a partir de um fluxo TCP em buffer com `receive_as_json`. Se ocorrer um erro, geramos um pacote `FromServer::Error` para transmitir as más notícias de volta ao cliente.

Além das mensagens de erro, os clientes também querem receber mensagens dos grupos de bate-papo aos quais ingressaram, portanto a conexão com o cliente precisa ser compartilhada com cada grupo. Poderíamos simplesmente criar para todos um clone do `TcpStream`, mas, se duas dessas fontes tentarem gravar um pacote no soquete ao mesmo tempo, a saída poderá ser intercalada e o cliente acabará recebendo JSON distorcido. Precisamos providenciar acesso simultâneo seguro à conexão.

Isso é gerenciado com o tipo `Outbound`, definido em *src/bin/server/connection.rs* do seguinte modo:

```

use async_std::sync::Mutex;

pub struct Outbound(Mutex<TcpStream>);

impl Outbound {
    pub fn new(to_client: TcpStream) -> Outbound {
        Outbound(Mutex::new(to_client))
    }

    pub async fn send(&self, packet: FromServer) -> ChatResult<()> {
        let mut guard = self.0.lock().await;
        utils::send_as_json(&mut *guard, &packet).await?;
        guard.flush().await?;
        Ok(())
    }
}

```

Quando criado, um valor `Outbound` toma posse de um `TcpStream` e o encapsula em um `Mutex` para garantir que apenas uma tarefa possa

usá-lo por vez. A função `serve` envolve cada `Outbound` em um ponteiro de contagem de referências `Arc` para que todos os grupos dos quais o cliente participa possam apontar para a mesma instância `Outbound` compartilhada.

Uma chamada para `Outbound::send` primeiro bloqueia o mutex, retornando um valor de guarda que desreferencia para `TcpStream` no lado de dentro. Utilizamos `send_as_json` para transmitir `packet` e finalmente chamamos `guard.flush()` para garantir que não seja transmitido pela metade em algum buffer em algum lugar. (Para nosso conhecimento, na verdade `TcpStream` não armazena dados em buffer, mas o trait `Write` permite que suas implementações façam isso, então não devemos correr nenhum risco.)

A expressão `&mut *guard` permite contornar o fato de que o Rust não aplica coerções `deref` para atender aos limites de trait. Em vez disso, desreferenciamos explicitamente o guarda do mutex e então emprestamos uma referência mutável ao `TcpStream` que ele protege, produzindo o `&mut TcpStream` que `send_as_json` requer.

Observe que `Outbound` utiliza o tipo `async_std::sync::Mutex`, não o `Mutex` da biblioteca padrão. Há três razões disso.

Primeiro, o `Mutex` da biblioteca padrão pode se comportar mal se uma tarefa for suspensa enquanto mantém uma guarda do mutex. Se o thread que estava executando essa tarefa selecionar outra tarefa que tenta bloquear o mesmo `Mutex`, surge o problema: do ponto de vista do `Mutex`, o thread que já o possui está tentando bloqueá-lo novamente. O `Mutex` padrão não foi projetado para lidar com esse caso, então ele gera em pânico ou trava. (Ele nunca concederá o bloqueio de forma inadequada.) Há um trabalho em andamento para fazer o Rust detectar esse problema em tempo de compilação e emitir um aviso sempre que uma guarda `std::sync::Mutex` está ativa em uma expressão `await`. Como `Outbound::send` precisa manter o bloqueio enquanto aguarda os futuros de `send_as_json` e `guard.flush`, ele deve utilizar `Mutex` de `async_std`.

Segundo, o método `lock` assíncrono de `Mutex` retorna um futuro de uma guarda, portanto uma tarefa esperando para bloquear um

mutex gera um thread para outras tarefas utilizarem até que o mutex esteja pronto. (Se o mutex já estiver disponível, o futuro `lock` estará pronto imediatamente, e a tarefa não será suspensa.) O método `lock` do `Mutex` padrão, por outro lado, fixa todo o thread ao esperar para adquirir o bloqueio. Como o código anterior contém o mutex enquanto transmite um pacote pela rede, isso pode demorar um pouco.

Por fim, o `Mutex` padrão só deve ser desbloqueado pelo mesmo thread que o bloqueou. Para impor isso, o tipo de guarda do mutex padrão não implementa `Send`: não pode ser transmitido para outros threads. Isso significa que um futuro contendo essa guarda não implementa por si só `Send` e não pode ser passado para `spawn` a fim de ser executado em um pool de threads; só pode ser executado com `block_on` ou `spawn_local`. A guarda de um `async_std::Mutex` implementa `Send`, portanto não há problema em usá-la em tarefas geradas.

A tabela de grupo: Mutexes síncronos

Mas a moral da história não é tão simples quanto “sempre utilize `async_std::sync::Mutex` no código assíncrono”. Frequentemente, não há necessidade de esperar nada ao manter um mutex, e o bloqueio não é mantido por muito tempo. Nesses casos, o `Mutex` da biblioteca padrão pode ser muito mais eficiente. O tipo `GroupTable` do nosso servidor de bate-papo ilustra esse caso. Eis o conteúdo completo de *src/bin/server/group_table.rs*:

```
use crate::group::Group;
use std::collections::HashMap;
use std::sync::{Arc, Mutex};

pub struct GroupTable(Mutex<HashMap<Arc<String>, Arc<Group>>>);

impl GroupTable {
    pub fn new() -> GroupTable {
        GroupTable(Mutex::new(HashMap::new()))
    }

    pub fn get(&self, name: &String) -> Option<Arc<Group>> {
        self.0.lock()
            .unwrap()
            .get(name)
    }
}
```



```

        .cloned()
    }

    pub fn get_or_create(&self, name: Arc<String>) -> Arc<Group> {
        self.0.lock()
            .unwrap()
            .entry(name.clone())
            .or_insert_with(|| Arc::new(Group::new(name)))
            .clone()
    }
}

```

Um `GroupTable` é simplesmente uma tabela hash protegida por mutex, mapeando nos nomes dos grupos de bate-papo os grupos reais, ambos gerenciados utilizando ponteiros contados por referência. Os métodos `get` e `get_or_create` bloqueiam o mutex, executam algumas operações de tabela hash, talvez algumas alocações e retornam.

Em `GroupTable`, utilizamos um `std::sync::Mutex` simples. Não há nenhum código assíncrono nesse módulo, então não há `await` a evitar. De fato, se quiséssemos utilizar `async_std::sync::Mutex` aqui, precisaríamos tornar `get` e `get_or_create` funções assíncronas, que introduzem a sobrecarga da criação de futuros, suspensões e retomadas que geram pouco benefício: o mutex é bloqueado apenas para algumas operações de hash e talvez algumas alocações.

Se nosso servidor de bate-papo tivesse milhões de usuários e o mutex `GroupTable` gerasse um gargalo, torná-lo assíncrono não resolveria o problema. Provavelmente seria melhor utilizar algum tipo de coleção especializada para acesso simultâneo em vez de `HashMap`. Por exemplo, o `crate dashmap` fornece esse tipo.

Grupos de bate-papo: canais de transmissão do tokio

No nosso servidor, o tipo `group::Group` representa um grupo de bate-papo. Esse tipo só precisa suportar os dois métodos que `connection::serve` chama: `join`, para adicionar um novo membro, e `post`, para postar uma mensagem. Cada mensagem postada precisa ser distribuída a todos os membros.

É aqui que abordamos o desafio mencionado anteriormente da

contrapressão. Existem várias necessidades tensas umas com as outras:

- Se um membro não conseguir acompanhar as mensagens postadas no grupo – se ele tiver uma conexão de rede lenta, por exemplo –, outros membros do grupo não devem ser afetados.
- Mesmo que um membro fique para trás, deve haver meios para que eles retornem à conversa e continuem a participar de alguma forma.
- A memória gasta para armazenar o buffer de mensagens não deve crescer sem limites.

Como esses desafios são comuns ao implementar padrões de comunicação muitos-para-muitos, o crate `tokio` fornece um tipo de canal de transmissão que implementa um conjunto razoável de compensações. Um canal de transmissão `tokio` é uma fila de valores (no nosso caso, mensagens de bate-papo) que permite que qualquer número de threads ou tarefas diferentes enviem e recebam valores. É chamado de canal de “transmissão” porque cada consumidor recebe sua própria cópia de cada valor enviado. (O tipo de valor deve implementar `Clone`.)

Normalmente, um canal de transmissão retém uma mensagem na fila até que todos os consumidores tenham obtido sua cópia. Mas, se o comprimento da fila exceder a capacidade máxima do canal, especificada quando ele é criado, as mensagens mais antigas serão descartadas. Qualquer consumidor que não conseguir acompanhar receberá um erro na próxima vez que tentar obter a próxima mensagem e o canal os atualizará de acordo com a mensagem mais antiga ainda disponível.

Por exemplo, a Figura 20.4 mostra um canal de transmissão com capacidade máxima de 16 valores.

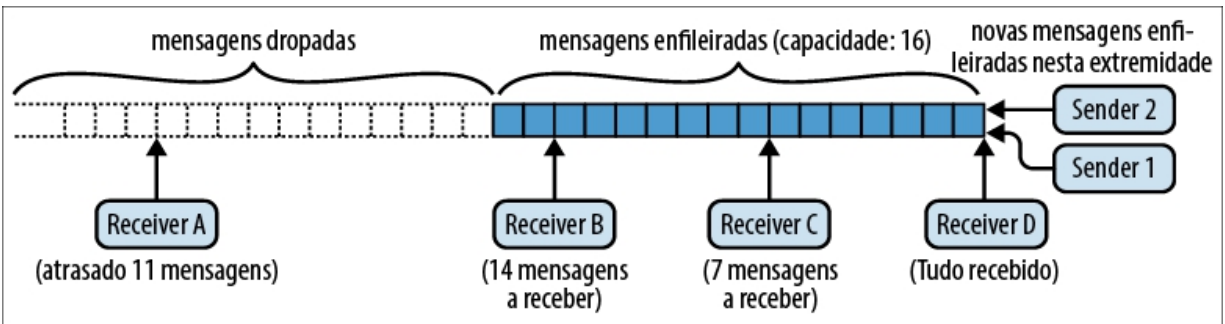


Figura 20.4: Um canal de transmissão tokio.

Há dois remetentes enfileirando mensagens e quatro receptores desenfileirando-as – ou, mais precisamente, copiando mensagens da fila. O receptor B tem 14 mensagens ainda para receber, o receptor C tem 7 e o receptor D está totalmente atualizado. O receptor A ficou para trás e 11 mensagens foram descartadas antes que ele pudesse vê-las. Sua próxima tentativa de receber uma mensagem falhará, retornando um erro indicando a situação e ele será enviado ao final atual da fila.

Nosso servidor de bate-papo representa cada grupo de bate-papo como um canal de transmissão que contém valores `Arc<String>`: postar uma mensagem no grupo a transmite a todos os membros atuais. Eis a definição do tipo `group::Group`, especificado em `src/bin/server/group.rs`:

```
use async_std::task;
use crate::connection::Outbound;
use std::sync::Arc;
use tokio::sync::broadcast;

pub struct Group {
    name: Arc<String>,
    sender: broadcast::Sender<Arc<String>>
}

impl Group {
    pub fn new(name: Arc<String>) -> Group {
        let (sender, _receiver) = broadcast::channel(1000);
        Group { name, sender }
    }

    pub fn join(&self, outbound: Arc<Outbound>) {
        let receiver = self.sender.subscribe();
    }
}
```

```

        task::spawn(handle_subscriber(self.name.clone(),
                                      receiver,
                                      outbound));
    }

    pub fn post(&self, message: Arc<String>) {
        // Isso só retorna um erro quando não há assinantes. O
        // lado de saída da conexão pode ser encerrado, descartando sua
        // assinatura, um pouco antes do lado entrante, que pode acabar
        // tentando enviar uma mensagem para um grupo vazio
        let _ignored = self.sender.send(message);
    }
}

```

Um struct `Group` contém o nome do grupo de bate-papo, com um `broadcast::Sender` representando a extremidade de envio do canal de transmissão do grupo. A função `Group::new` chama `broadcast::channel` para criar um canal de transmissão com capacidade máxima de 1.000 mensagens. A função `channel` retorna um remetente e um destinatário, mas não precisamos do destinatário nesse momento, pois o grupo ainda não tem nenhum membro.

Para adicionar um novo membro ao grupo, o método `Group::join` chama `subscribe` do remetente para criar um novo destinatário para o canal. Em seguida, ele gera uma nova tarefa assíncrona para monitorar esse destinatário esperando por mensagens e para escrevê-las de volta no cliente, na função `handle_subscribe`.

Com esses detalhes em mãos, o método `Group::post` é simples e direto: ele simplesmente envia a mensagem ao canal de transmissão. Como os valores transportados pelo canal são valores `Arc<String>`, dar a cada destinatário uma cópia própria de uma mensagem apenas aumenta a contagem de referências da mensagem, sem nenhuma cópia ou alocação no heap. Depois que todos os assinantes transmitiram a mensagem, a contagem de referências cai a zero, e a mensagem é liberada.

Eis a definição de `handle_subscriber`:

```

use async_chat::FromServer;
use tokio::sync::broadcast::error::RecvError;

async fn handle_subscriber(group_name: Arc<String>,
                           mut receiver: broadcast::Receiver<Arc<String>>,

```

```

        outbound: Arc<Outbound>)
    {
        loop {
            let packet = match receiver.recv().await {
                Ok(message) => FromServer::Message {
                    group_name: group_name.clone(),
                    message: message.clone(),
                },
                Err(RecvError::Lagged(n)) => FromServer::Error(
                    format!("Dropped {} messages from {}.", n, group_name)
                ),
                Err(RecvError::Closed) => break,
            };
            if outbound.send(packet).await.is_err() {
                break;
            }
        }
    }
}

```

Embora os detalhes sejam diferentes, a forma dessa função é familiar: é um loop que recebe mensagens do canal de transmissão e envia-as de volta ao cliente por meio do valor `Outbound` compartilhado. Se o loop não conseguir acompanhar o canal de transmissão, ele receberá um erro `Lagged`, que é devidamente reportado ao cliente.

Se o envio de um pacote de volta ao cliente falhar completamente, talvez porque a conexão foi fechada, `handle_subscriber` sai do loop e retorna, fazendo com que a tarefa assíncrona seja encerrada. Isso dropa o `Receiver` do canal de transmissão, cancelando a inscrição no canal. Dessa forma, quando uma conexão é dropada, cada uma de suas associações de grupo é limpa na próxima vez que o grupo tentar enviar uma mensagem.

Nossos grupos de bate-papo nunca fecham, pois nunca removemos um grupo da tabela de grupos, mas apenas por questão de completude, `handle_subscriber` está pronto para lidar com um erro `Closed` ao sair da tarefa.

Observe que estamos criando uma nova tarefa assíncrona para cada associação de grupo de cada cliente. Isso é viável porque tarefas assíncronas utilizam muito menos memória do que threads e porque

alternar entre uma tarefa assíncrona e outra dentro de um processo é bastante eficiente.

Esse, então, é o código completo para o servidor de bate-papo. É um pouco espartano e há muitos recursos mais valiosos nos crates `async_std`, `tokio` e `futures` do que podemos abranger neste livro, mas idealmente esse exemplo estendido consegue ilustrar como alguns dos recursos do ecossistema assíncrono trabalham juntos: tarefas assíncronas, fluxos, traits de E/S assíncronos, canais e mutexes de ambos os tipos.

Primitivas de futuro e executores: Quando vale a pena verificar um futuro novamente?

O servidor de bate-papo mostra como podemos escrever código utilizando primitivas assíncronas como `TcpListener` e `broadcast` no canal e utiliza executores como `block_on` e `spawn` para conduzir a execução. Agora podemos analisar como essas coisas são implementadas. A questão chave é, quando um futuro retorna `Poll::Pending`, como ele se coordena com o executor para pesquisá-lo novamente no momento certo?

Pense no que acontece quando executamos um código como este, da função `main` de bate-papo do cliente:

```
task::block_on(async {  
    let socket = net::TcpStream::connect(address).await?;  
    ...  
})
```

A primeira vez que `block_on` verifica o futuro do bloco assíncrono, a conexão de rede quase certamente não está pronta imediatamente, então `block_on` entra em repouso. Mas quando ele deve acordar? De alguma forma, quando a conexão de rede estiver pronta, `TcpStream` precisa dizer a `block_on` que deve tentar verificar novamente o futuro do bloco assíncrono porque agora ele sabe disso, `await` será concluído e a execução do bloco assíncrono poderá avançar.

Quando um executor como `block_on` verifica um futuro, ele deve ser

passado em um retorno de chamada denominado waker. Se o futuro ainda não está pronto, as regras do trait `Future` informam que deve retornar `Poll::Pending` por enquanto, e organizar para que o waker seja invocado mais tarde, se e quando o futuro vale a pena ser verificado novamente.

Portanto, uma implementação manuscrita de `Future` muitas vezes se parece com algo assim:

```
use std::task::Waker;

struct MyPrimitiveFuture {
    ...
    waker: Option<Waker>,
}

impl Future for MyPrimitiveFuture {
    type Output = ...;

    fn poll(mut self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<...> {
        ...

        if ... future is ready ... {
            return Poll::Ready(final_value);
        }

        // Salva o waker para mais tarde
        self.waker = Some(cx.waker().clone());
        Poll::Pending
    }
}
```

Em outras palavras, se o valor do futuro está pronto, retorne-o. Caso contrário, armazene um clone do waker `Context` em algum lugar e retorne `Poll::Pending`.

Quando vale a pena verificar o futuro novamente, o futuro deve notificar o último executor que o pesquisou chamando o método `wake` do waker:

```
// Se houver um waker, invoca-o e limpa `self.waker`
if let Some(waker) = self.waker.take() {
    waker.wake();
}
```

Idealmente, o executor e o futuro se revezam verificando e chamando o waker: o executor verifica o futuro e entra em repouso, então o futuro invoca o waker, assim o executor é ativado e verifica

o futuro novamente.

Futuros de funções e blocos assíncronos não lidam com os próprios wakers. Eles simplesmente repassam o contexto que lhes é dado aos subfuturos que aguardam, delegando a eles a obrigação de salvar e invocar wakers. Em nosso cliente de bate-papo, a primeira verificação do futuro do bloco assíncrono apenas passa o contexto enquanto aguarda o futuro de `TcpStream::connect`. As verificações subsequentes também passam o contexto para qualquer futuro que o bloco aguarda a seguir.

O tratamento do futuro de `TcpStream::connect` sendo verificado conforme mostrado no exemplo anterior: ele entrega o waker para um thread auxiliar que aguarda a conexão estar pronta e, em seguida, invoca o waker.

`Waker` implementa `Clone` e `Send`, portanto um futuro sempre pode criar sua própria cópia do waker e enviá-la para outros threads conforme necessário. O método `Waker::wake` consome o waker. Há também um método `wake_by_ref` que não consome, mas alguns executores podem implementar a versão de consumo de forma um pouco mais eficiente. (A diferença é no máximo um `clone`.)

É inofensivo para um executor verificar um futuro várias vezes, apenas ineficiente. Os futuros, porém, devem ter o cuidado de invocar um waker apenas quando a verificação fizer progresso real: um ciclo de ativações e verificações espúrias pode impedir que um executor entre em repouso, desperdiçando energia e deixando o processador menos responsivo a outras tarefas.

Agora que mostramos como os executores e as primitivas de futuro se comunicam, nós mesmos implementaremos uma primitiva de futuro e, em seguida, passaremos por uma implementação do executor `block_on`.

Invocando wakers: `spawn_blocking`

No início do capítulo, descrevemos a função `spawn_blocking`, que inicia uma determinada closure em execução em outro thread e retorna um futuro de seu valor de retorno. Agora temos todas as peças de

que precisamos para implementar `spawn_blocking` nós mesmos. Para simplificar, nossa versão cria um novo thread para cada closure, em vez de utilizar um pool de threads, como a versão do `async_std` implementa.

Embora `spawn_blocking` retorne um futuro, não vamos escrevê-lo como um `async fn`. Em vez disso, será uma função síncrona comum que retorna um struct, `SpawnBlocking`, no qual implementaremos `Future` nós mesmos.

A assinatura do nosso `spawn_blocking` é como a seguir:

```
pub fn spawn_blocking<T, F>(closure: F) -> SpawnBlocking<T>
where F: FnOnce() -> T,
      F: Send + 'static,
      T: Send + 'static,
```

Como precisamos enviar a closure para outro thread e trazer o valor de retorno de volta, tanto a closure `F` como seu valor de retorno `T` devem implementar `Send`. E como não fazemos ideia de quanto tempo o thread será executado, ambas devem ser `'static` também. Esses são os mesmos limites que o próprio `std::thread::spawn` impõe.

`SpawnBlocking<T>` é um futuro do valor de retorno da closure. Eis sua definição:

```
use std::sync::{Arc, Mutex};
use std::task::Waker;

pub struct SpawnBlocking<T>(Arc<Mutex<Shared<T>>>);

struct Shared<T> {
    value: Option<T>,
    waker: Option<Waker>,
}
```

O struct `Shared` deve servir como um ponto de encontro entre o futuro e o thread que executa a closure, portanto pertence a um `Arc` e é protegida com um `Mutex`. (Um mutex síncrono é bom aqui.) Verificar o futuro verifica se `value` está presente e salva o `waker` em `waker` se não estiver presente. O thread que executa a closure salva o valor de retorno em `value` e então invoca `waker`, se presente.

Eis a definição completa de `spawn_blocking`:

```
pub fn spawn_blocking<T, F>(closure: F) -> SpawnBlocking<T>
where F: FnOnce() -> T,
```

```

    F: Send + 'static,
    T: Send + 'static,
{
    let inner = Arc::new(Mutex::new(Shared {
        value: None,
        waker: None,
    }));

    std::thread::spawn({
        let inner = inner.clone();
        move || {
            let value = closure();

            let maybe_waker = {
                let mut guard = inner.lock().unwrap();
                guard.value = Some(value);
                guard.waker.take()
            };

            if let Some(waker) = maybe_waker {
                waker.wake();
            }
        }
    });

    SpawnBlocking(inner)
}

```

Depois de criar o valor `Shared`, isso gera um thread para executar a closure, armazenar o resultado no campo `value` de `Shared` e invocar o waker, se houver um.

Podemos implementar `Future` para `SpawnBlocking` do seguinte modo:

```

use std::future::Future;
use std::pin::Pin;
use std::task::{Context, Poll};

impl<T: Send> Future for SpawnBlocking<T> {
    type Output = T;

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<T> {
        let mut guard = self.0.lock().unwrap();
        if let Some(value) = guard.value.take() {
            return Poll::Ready(value);
        }

        guard.waker = Some(cx.waker().clone());
        Poll::Pending
    }
}

```

```
}
```

Verificar um `SpawnBlocking` verifica se o valor da closure está pronto, tomando posse e retornando-o se estiver pronto. Caso contrário, o futuro ainda estará pendente, assim salva um clone do waker do contexto no campo `waker` do futuro.

Depois que `Future` retornou `Poll::Ready`, você não deve pesquisá-lo novamente. Todas as formas usuais de consumir futuros, como `await` e `block_on`, respeitam essa regra. Se um futuro `SpawnBlocking` for verificado várias vezes, nada especialmente terrível acontece, mas também não faz nenhum esforço para lidar com esse caso. Isso é típico para futuros escritos à mão.

Implementando `block_on`

Além de podermos implementar primitivas de futuro, também temos todas as peças de que precisamos para construir um executor simples. Nesta seção, escreveremos nossa própria versão de `block_on`. Será um pouco mais simples do que a versão de `async_std`; por exemplo, ela não suporta `spawn_local`, variáveis locais de tarefa ou invocações aninhadas (chamar `block_on` de código assíncrono). Mas é suficiente executar nosso cliente e servidor de bate-papo.

Eis o código:

```
use waker_fn::waker_fn;    // Cargo.toml: waker-fn = "1.1"
use futures_lite::pin;     // Cargo.toml: futures-lite = "1.11"
use crossbeam::sync::Parker; // Cargo.toml: crossbeam = "0.8"
use std::future::Future;
use std::task::{Context, Poll};

fn block_on<F: Future>(future: F) -> F::Output {
    let parker = Parker::new();
    let unparker = parker.unparker().clone();
    let waker = waker_fn(move || unparker.unpark());
    let mut context = Context::from_waker(&waker);

    pin!(future);

    loop {
        match future.as_mut().poll(&mut context) {
            Poll::Ready(value) => return value,
            Poll::Pending => parker.park(),
        }
    }
}
```

```
}  
}
```

Isso é muito curto, mas há muita coisa acontecendo, então vamos analisar uma parte de cada vez.

```
let parker = Parker::new();  
let unparker = parker.unparker().clone();
```

O tipo `Parker` do `crates::crossbeam` é uma primitiva de bloqueio simples: chamar `parker.park()` bloqueia o thread até que alguém chame `.unpark()` no `Unparker` correspondente, que você obtém de antemão chamando `parker.unparker()`. Se você usar `unpark` em um thread que ainda não está bloqueado por `park`, a próxima chamada para `park` retornará imediatamente, sem bloquear. Nosso `block_on` utilizará o `Parker` para esperar sempre que o futuro não estiver pronto, e o waker que passamos para os futuros vai remover o bloqueio.

```
let waker = waker_fn(move || unparker.unpark());
```

A função `waker_fn`, a partir do crate de mesmo nome, cria um `Waker` de uma determinada closure. Aqui criamos um `Waker` que, quando invocado, chama a closure `move || unparker.unpark()`. Você também pode criar wakers implementando o trait `std::task::Wake`, mas `waker_fn` é um pouco mais conveniente aqui.

```
pin!(future);
```

Dada uma variável contendo um futuro do tipo `F`, a macro `pin!` toma posse do futuro e declara uma nova variável de mesmo nome cujo tipo é `Pin<&mut F>` e que pega emprestado o futuro. Isso nos dá o `Pin<&mut Self>` exigido pelo método `poll`. Por motivos que explicaremos na próxima seção, futuros de funções e blocos assíncronos devem ser referenciados por meio de um `Pin` antes que possam ser verificados.

```
loop {  
    match future.as_mut().poll(&mut context) {  
        Poll::Ready(value) => return value,  
        Poll::Pending => parker.park(),  
    }  
}
```

Por fim, o loop de verificação é bastante simples. Passando um contexto que transporta nosso waker, verificamos o futuro até que ele retorne `Poll::Ready`. Se retornar `Poll::Pending`, bloqueamos (`park`) o

thread, que bloqueia até `waker` ser invocado. Então tentamos novamente.

A chamada `as_mut` permite verificar `future` sem renunciar à posse; explicaremos melhor isso na próxima seção.

Fixando

Embora funções e blocos assíncronos sejam essenciais para escrever código assíncrono claro, lidar com os futuros requer um pouco de cuidado. O tipo `Pin` ajuda o Rust a garantir que sejam utilizados com segurança.

Nesta seção, mostraremos por que os futuros de chamadas e blocos de função assíncronos não podem ser manipulados tão livremente quanto os valores Rust comuns. Então vamos mostrar como `Pin` serve como um “selo de aprovação” em ponteiros com os quais podemos contar para gerenciar esses futuros com segurança. Por fim, mostraremos algumas maneiras de trabalhar com valores `Pin`.

As duas fases da vida de um futuro

Considere esta função assíncrona simples:

```
use async_std::io::prelude::*;
use async_std::{io, net};

async fn fetch_string(address: &str) -> io::Result<String> {
    ❶
    let mut socket = net::TcpStream::connect(address).await❷?;
    let mut buf = String::new();
    socket.read_to_string(&mut buf).await❸?;
    Ok(buf)
}
```

Ela abre uma conexão TCP com o endereço fornecido e retorna, como um `String`, o que quer que o servidor queira enviar. Os pontos rotulados ❶, ❷ e ❸ são os pontos de retomada, os pontos no código da função assíncrona nos quais a execução pode ser suspensa.

Suponha que você o chame, sem esperar, assim:

```
let response = fetch_string("localhost:6502");
```

Agora `response` é um futuro pronto para começar a execução no início

de `fetch_string`, com o argumento dado. Na memória, o futuro se parece com a Figura 20.5.

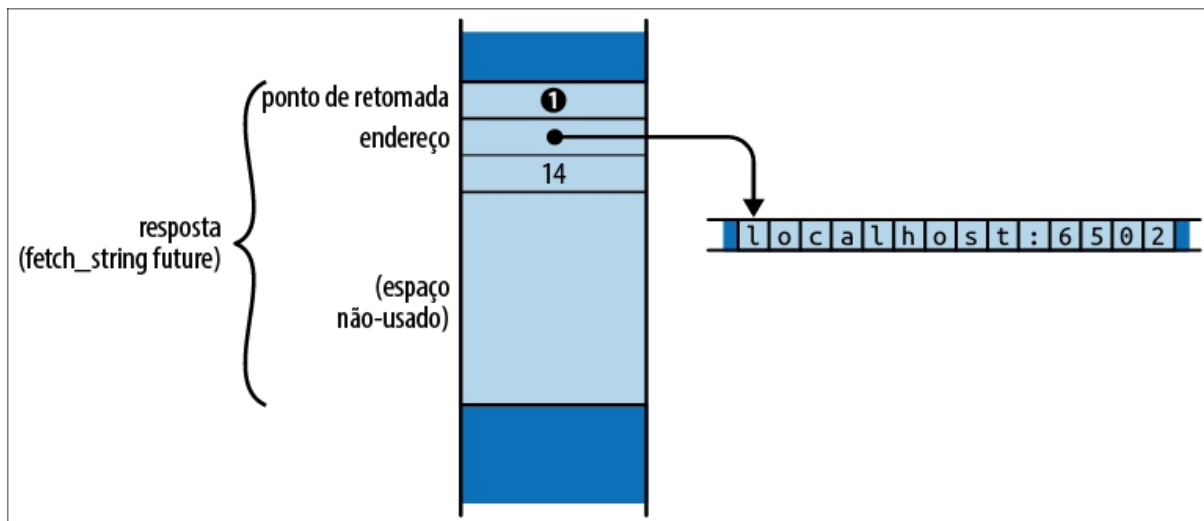


Figura 20.5: O futuro construído para uma chamada para `fetch_string`.

Como acabamos de criar esse futuro, ele diz que a execução deve começar no ponto de retomada ❶, na parte superior do corpo da função. Nesse estado, os únicos valores de que um futuro precisa para prosseguir são os argumentos de função.

Agora suponha que você pesquise `response` algumas vezes e alcance este ponto no corpo da função:

```
socket.read_to_string(&mut buf).await❷?;
```

Suponha ainda que o resultado de `read_to_string` não está pronto, então a verificação retorna `Poll::Pending`. Nesse ponto, o futuro se parece com a Figura 20.6.

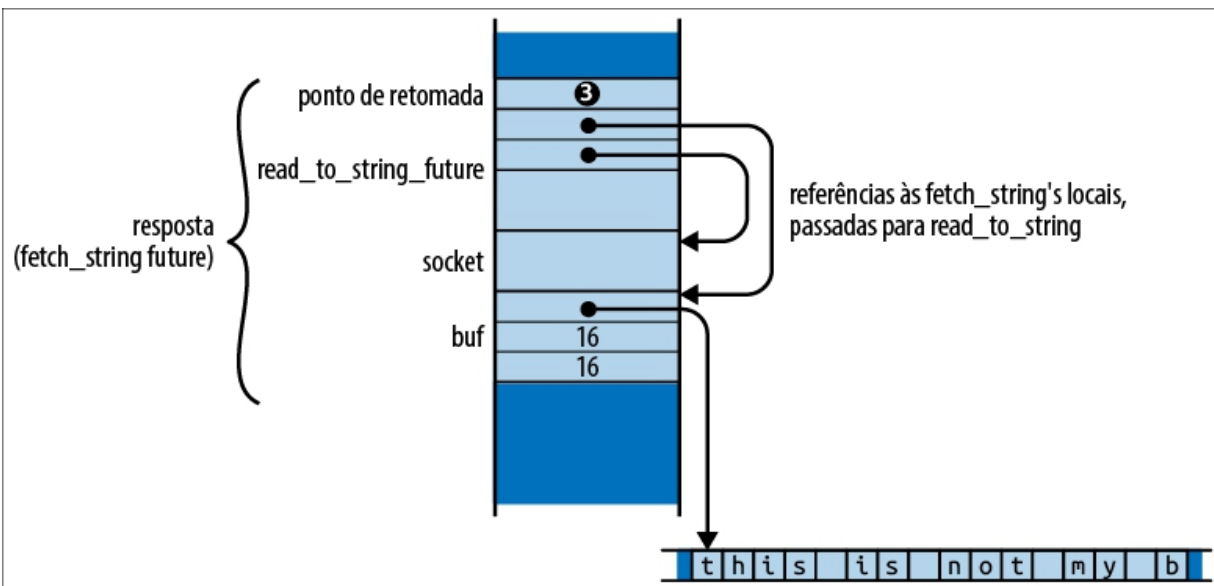


Figura 20.6: O mesmo futuro, no meio da espera read_to_string.

Um futuro sempre deve conter todas as informações necessárias para retomar a execução na próxima vez que for verificado. Nesse caso é:

- Ponto de retomada ③, informando que a execução deve ser retomada na verificação `await` do futuro `read_to_string`.
- As variáveis que estão ativas nesse ponto de retomada: `socket` e `buf`. O valor de `address` não está mais presente no futuro, pois a função não precisa mais dele.
- O subfuturo `read_to_string`, em que a expressão `await` está no meio da verificação.

Note que a chamada para `read_to_string` pegou emprestadas referências a `socket` e `buf`. Em uma função síncrona, todas as variáveis locais residem no `stack`, mas, em uma função assíncrona, as variáveis locais que estão vivas em um `stack await` devem estar localizadas no futuro, para que estejam disponíveis quando forem verificadas novamente. Pegar emprestada uma referência a tal variável pega emprestada uma parte do futuro.

Mas o Rust exige que os valores não sejam movidos enquanto são emprestados. Suponha que você fosse mover esse futuro para um novo local:

```
let new_variable = response;
```

O Rust não tem como encontrar todas as referências ativas e ajustá-las de acordo. Em vez de apontar para `socket` e `buf` em seus novos locais, as referências continuam apontando para seus locais antigos no agora não inicializado `response`. Elas se tornaram ponteiros que apontam para áreas não mais alocadas, conforme mostrado na Figura 20.7.

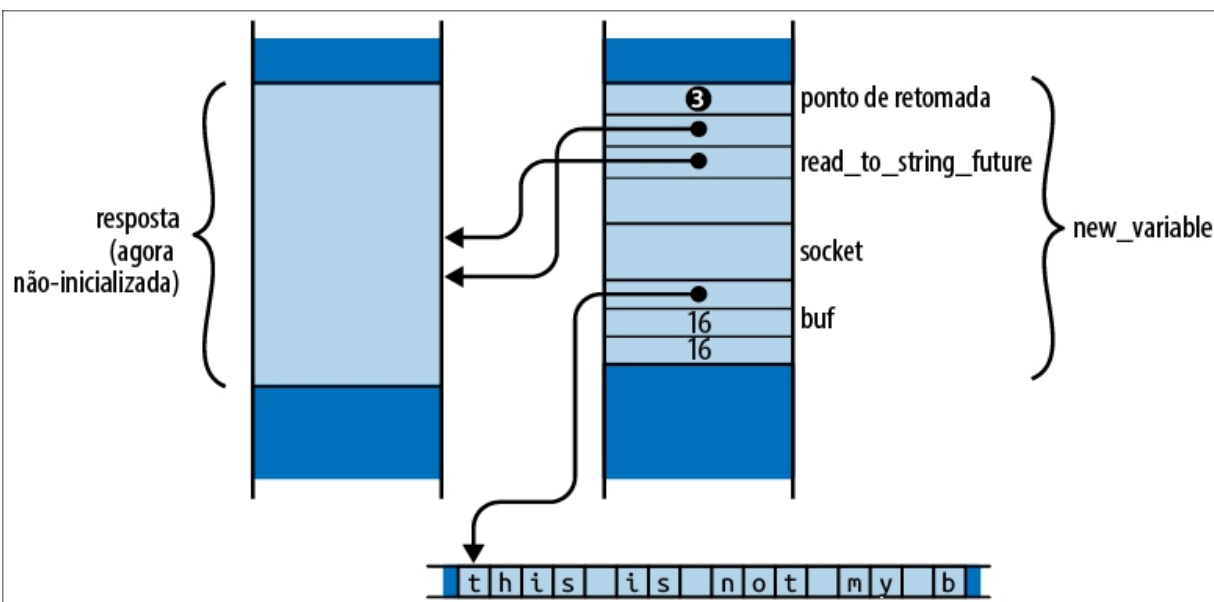


Figura 20.7: futuro de `fetch_string`, movido enquanto emprestado (o Rust impede isso).

Evitar que os valores emprestados sejam movidos geralmente é responsabilidade do verificador de empréstimos. O verificador de empréstimos trata as variáveis como as raízes das árvores de posse, mas, ao contrário das variáveis armazenadas no stack (pilha), as variáveis armazenadas em futuros são movidas se o próprio futuro se mover. Isso significa que os empréstimos de `socket` e `buf` afetam não apenas o que `fetch_string` pode fazer com suas próprias variáveis, mas o que o chamador pode fazer de maneira segura com `response`, o futuro que os contém. Os futuros das funções assíncronas são um ponto cego para o verificador de empréstimos, o que o Rust deve abranger de alguma forma se quiser manter suas promessas de segurança na memória.

A solução do Rust para esse problema baseia-se na percepção de

que os futuros sempre são seguros para serem movidos quando são criados pela primeira vez e só se tornam inseguros para serem movidos quando são verificados. Um futuro que acabou de ser criado chamando uma função assíncrona simplesmente contém um ponto de retomada e os valores do argumento. Eles estão apenas no escopo do corpo da função assíncrona, que ainda não iniciou a execução. Apenas verificar um futuro já pode pegar emprestado seu conteúdo.

A partir disso, podemos ver que todo futuro tem dois estágios de tempo de vida:

- O primeiro estágio começa quando o futuro é criado. Como o corpo da função ainda não começou a execução, nenhuma parte dele pode ser emprestada ainda. Nesse ponto, é tão seguro mover quanto qualquer outro valor Rust.
- O segundo estágio começa na primeira vez que o futuro é verificado. Depois que o corpo da função começou a execução, ele pode pegar referências emprestadas para variáveis armazenadas no futuro e então esperar, deixando essa parte do futuro emprestada. Começando após a primeira verificação, devemos supor que não é seguro mover o futuro.

A flexibilidade do primeiro estágio de tempo de vida é o que nos permite passar futuros para `block_on` e `spawn` e chamar métodos adaptadores como `race` e `fuse`, todos os quais aceitam futuros por valor. Na verdade, até mesmo a chamada de função assíncrona que criou inicialmente o futuro precisou retorná-lo ao chamador; isso também foi um movimento.

Para entrar no segundo estágio de tempo de vida, o futuro deve ser verificado. O método `poll` exige que o futuro seja passado como um valor `Pin<&mut Self>`. `Pin` é um empacotador para tipos de ponteiro (como `&mut Self`) que restringe como os ponteiros podem ser utilizados, garantindo que seus referentes (como `Self`) não possam ser movidos novamente. Assim, você deve produzir um ponteiro empacotado em `Pin` para o futuro antes que possa pesquisá-lo.

Essa, então, é a estratégia do Rust para manter os futuros seguros:

só é seguro mover um futuro depois que foi verificado; você só pode verificar um futuro depois que tenha construído um ponteiro empacotador `Pin` para ele; e, uma vez feito isso, o futuro não pode ser movido.

“Um valor que você não pode mover” parece impossível: movimentos estão por toda parte no Rust. Explicaremos exatamente como `Pin` protege futuros na próxima seção.

Embora essa seção tenha discutido funções assíncronas, tudo aqui também se aplica a blocos assíncronos. Um futuro recém-criado de um bloco assíncrono simplesmente captura as variáveis que utilizará do código circundante, como uma closure. Somente verificar o futuro pode criar referências a seu conteúdo, tornando-o inseguro movê-lo.

Lembre-se de que essa fragilidade de movimento é limitada a futuros de funções e blocos assíncronos, com as implementações especiais `Future` geradas pelo compilador. Se você implementar `Future` manualmente para seus próprios tipos, como fizemos para nosso tipo `SpawnBlocking` em “Invocando wakers: `spawn_blocking`”, na página 661, é perfeitamente seguro mover esses futuros antes e depois de serem verificados. Em qualquer implementação `poll` escrita à mão, o verificador de empréstimo garante que quaisquer referências que você tenha emprestado para partes de `self` desaparecem no momento em que `poll` retorna. Devemos lidar com seus futuros com cuidado porque funções e blocos assíncronos podem suspender a execução no meio de uma chamada de função, com empréstimos em andamento.

Ponteiros fixados

O tipo `Pin` é um envoltório dos ponteiros para futuros que restringe como os ponteiros podem ser utilizados para garantir que os futuros não possam ser movidos depois de serem verificados. Essas restrições podem ser suspensas para futuros que não se importam em serem movidos, mas são essenciais para verificar com segurança futuros de funções e blocos assíncronos.

Por ponteiro, queremos dizer qualquer tipo que implemente `Deref` e

possivelmente `DerefMut`. Um `Pin` em torno de um ponteiro é chamado ponteiro fixado. `Pin<&mut T>` e `Pin<Box<T>>` são típicos.

A definição de `Pin` na biblioteca padrão é simples:

```
pub struct Pin<P> {  
    pointer: P,  
}
```

Observe que o campo `pointer` não é `pub`. Isso significa que a única maneira de construir ou utilizar um `Pin` é por meio de métodos cuidadosamente escolhidos que o tipo fornece.

Dado o futuro de uma função ou bloco assíncrono, existem apenas algumas maneiras de obter um ponteiro fixado para ele:

- A macro `pin!`, do crate `futures-lite`, mascara uma variável do tipo `T` com um novo tipo `Pin<&mut T>`. A nova variável aponta para o valor do original, que foi movido para um local temporário anônimo no stack (pilha). Quando a variável sai do escopo, o valor é dropado. Usamos `pin!` na nossa implementação `block_on` para fixar o futuro que queríamos verificar.
- O construtor `Box::pin` da biblioteca padrão toma posse de um valor de qualquer tipo `T`, move-o para o heap e retorna um `Pin<Box<T>>`.
- `Pin<Box<T>>` implementa `From<Box<T>>`, assim `Pin::from(boxed)` toma posse de `boxed` e devolve a você um `box` fixado apontando para o mesmo `T` no heap.

Todas as maneiras de obter um ponteiro fixado para esses futuros implicam desistir da posse do futuro e não há como recuperá-la. O próprio ponteiro fixado pode ser movido da maneira que você quiser, é claro, mas mover um ponteiro não move seu referente. Portanto, a posse de um ponteiro fixado para um futuro serve como prova de que você desistiu permanentemente da capacidade de mover esse futuro. Isso é tudo o que precisamos saber para que ele possa ser verificado com segurança.

Depois de fixar um futuro, se quiser pesquisá-lo, todos os tipos `Pin<pointer to T>` têm um método `as_mut` que desreferencia o ponteiro e retorna o `Pin<&mut T>` que `poll` requer.

O método `as_mut` também pode ajudá-lo a verificar um futuro sem

abrir mão da posse. Nossa implementação `block_on` usou-o nesta função:

```
pin!(future);  
loop {  
    match future.as_mut().poll(&mut context) {  
        Poll::Ready(value) => return value,  
        Poll::Pending => parker.park(),  
    }  
}
```

Aqui a macro `pin!` foi redeclarada `future` como um `Pin<&mut F>`, então poderíamos apenas passar isso para `poll`. Mas as referências mutáveis não são `Copy`, assim `Pin<&mut F>` também não pode ser `Copy`, o que significa que chamar `future.poll()` diretamente tomaria posse de `future`, deixando a próxima iteração do loop com uma variável não inicializada. Para evitar isso, chamamos `future.as_mut()` para novamente pegar emprestado um novo `Pin<&mut F>` para cada iteração do loop.

Não há como obter uma referência `&mut` a um futuro fixado: se fosse possível, você poderia utilizar `std::mem::replace` OU `std::mem::swap` para removê-la e colocar um futuro diferente em seu lugar.

A razão pela qual não precisamos nos preocupar em fixar futuros no código assíncrono comum é que todas as maneiras mais comuns de obter o valor de um futuro – aguardando ou passando para um executor – tomam posse do futuro e gerenciam a fixação internamente. Por exemplo, nossa implementação `block_on` toma posse do futuro e utiliza a macro `pin!` para produzir o `Pin<&mut F>` necessário para verificar. Uma expressão `await` também toma posse do futuro e utiliza uma abordagem semelhante à macro `pin!` internamente.

Trait Unpin

Entretanto, nem todos os futuros exigem esse tipo de tratamento cuidadoso. Para qualquer implementação escrita à mão de `Future` para um tipo comum, como nosso tipo `SpawnBlocking` mencionado anteriormente, as restrições na construção e uso de ponteiros fixados são desnecessárias.

Esses tipos duráveis implementam o trait do marcador `Unpin`:

```
trait Unpin { }
```

Quase todos os tipos no Rust implementam automaticamente `Unpin`, utilizando suporte especial no compilador. Os futuros de função e bloco assíncronos são as exceções a essa regra.

Para tipos `Unpin`, `Pin` não impõe qualquer tipo de restrição. Você pode criar um ponteiro fixado a partir de um ponteiro comum com `Pin::new` e recuperar o ponteiro com `Pin::into_inner`. O `Pin` por si só passa as próprias implementações `Deref` e `DerefMut` para o ponteiro.

Por exemplo, `String` implementa `Unpin`, assim podemos escrever:

```
let mut string = "Pinned?".to_string();
let mut pinned: Pin<&mut String> = Pin::new(&mut string);

pinned.push_str(" Not");
Pin::into_inner(pinned).push_str(" so much.");

let new_home = string;
assert_eq!(new_home, "Pinned? Not so much.");
```

Mesmo depois de criar um `Pin<&mut String>`, temos acesso mutável total à `string` e podemos movê-la para uma nova variável assim que o `Pin` foi consumido por `into_inner` e a referência mutável desaparecer. Assim, para os tipos que são `Unpin` – que são quase todos eles – `Pin` é um envoltório enfadonho em torno de ponteiros para esse tipo.

Isso significa que, quando você implementa `Future` para seus próprios tipos `Unpin`, sua implementação `poll` pode tratar `self` como se fosse `&mut Self`, não `Pin<&mut Self>`. Fixar torna-se algo que você pode ignorar.

Pode ser surpreendente saber que `Pin<&mut F>` e `Pin<Box<F>>` implementam `Unpin`, mesmo se `F` não implementar. Isso não soa bem – como um `Pin` pode ser `Unpin`? – mas, se você pensar cuidadosamente sobre o que cada termo significa, faz sentido. Mesmo se for não seguro mover `F` depois de ter sido verificado, um ponteiro para ele sempre é seguro de ser movido, verificado ou não. Apenas o ponteiro se move; seu referente frágil permanece parado.

É útil entender isso quando você quer passar o futuro de uma função ou bloco assíncrono para uma função que só aceita futuros `Unpin`. (Essas funções são raras em `async_std`, mas menos raras em outros lugares do ecossistema assíncrono.) `Pin<Box<F>>` é `Unpin`

mesmo se `F` não é, assim aplicar `Box::pin` a um futuro de uma função ou bloco assíncrono fornece um futuro que você pode utilizar em qualquer lugar, ao custo de uma alocação de heap.

Existem vários métodos inseguros para trabalhar com `Pin` que permitem que você faça o que quiser com o ponteiro e seu alvo, mesmo para tipos de alvo que não são `Unpin`. Mas, como explicado no Capítulo 22, o Rust não pode verificar se esses métodos estão sendo utilizados corretamente; você se torna responsável por garantir a segurança do código que os utiliza.

Quando o código assíncrono é útil?

O código assíncrono é mais complicado de escrever do que o código multithread. Você tem de utilizar as primitivas de sincronização e E/S corretos, dividir cálculos de execução longa manualmente ou desviá-los para outros threads e gerenciar outros detalhes, como fixação, que não surgem no código com thread. Então, quais vantagens específicas o código assíncrono oferece?

Duas afirmações que ouvirá com frequência não resistem a uma inspeção cuidadosa:

- “O código assíncrono é ótimo para E/S.” Isso não está correto. Se seu aplicativo estiver gastando tempo esperando E/S, torná-lo assíncrono não fará com que a E/S seja executada mais rapidamente. Não há nada sobre interfaces de E/S assíncronas geralmente utilizadas hoje que as torne mais eficientes do que suas contrapartes síncronas. O sistema operacional tem o mesmo trabalho a fazer de qualquer maneira. (Na verdade, uma operação de E/S assíncrona que não está pronta deve ser tentada novamente mais tarde, portanto são necessárias duas chamadas de sistema para que seja concluída em vez de uma.)
- “O código assíncrono é mais fácil de escrever do que o código multithread.” Em linguagens como JavaScript e Python, isso pode ser verdade. Nessas linguagens, os programadores utilizam `async/await` como uma forma bem-comportada de concorrência: há um único thread de execução e as interrupções ocorrem

apenas em expressões `await`, assim muitas vezes não há necessidade de um mutex para manter os dados consistentes: apenas não espere enquanto estiver utilizando-o! É muito mais fácil entender seu código quando as alternâncias de tarefas só ocorrem com sua permissão explícita.

Mas esse argumento não se aplica ao Rust, no qual os threads não são tão problemáticos. Depois de seu programa ser compilado, ele está livre de corridas de dados (data races). O comportamento não determinístico limita-se a recursos de sincronização como mutexes, canais, atômicos e assim por diante, que foram projetados para lidar com isso. Portanto, o código assíncrono não tem nenhuma vantagem exclusiva em ajudá-lo a ver quando outros threads podem afetá-lo; isso está claro em *todo* código Rust seguro.

E, naturalmente, o suporte assíncrono do Rust realmente brilha quando utilizado em combinação com threads. Seria uma pena desistir disso.

Portanto, quais são as vantagens reais do código assíncrono?

- *Tarefas assíncronas podem utilizar menos memória.* No Linux, o uso de memória por um thread começa em 20 KiB, contando o espaço do usuário e do kernel.² Os futuros podem ser muito menores: os futuros do nosso servidor de bate-papo têm algumas centenas de bytes de tamanho e estão ficando menores à medida que o compilador do Rust melhora.
- *Tarefas assíncronas são mais rápidas de criar.* No Linux, a criação de um thread leva cerca de 15 μ s. Gerar uma tarefa assíncrona leva cerca de 300 ns, cerca de um quinto do tempo.
- *Alternâncias de contexto são mais rápidas entre tarefas assíncronas do que entre threads do sistema operacional, 0,2 μ s versus 1,7 μ s no Linux.³* Contudo, esses são os números de melhor caso para cada um: se a alternância é devido à prontidão de E/S, ambos os custos aumentam para 1,7 μ s. Se a alternância é entre threads ou tarefas em diferentes núcleos do processador também faz uma grande diferença: a comunicação entre os

núcleos é muito lenta.

Isso nos dá uma dica de quais tipos de problemas o código assíncrono pode resolver. Por exemplo, um servidor assíncrono pode utilizar menos memória por tarefa e, portanto, ser capaz de lidar com mais conexões simultâneas. (Provavelmente é aqui que o código assíncrono obtém sua reputação de ser “bom para E/S”.) Ou, se seu projeto é naturalmente organizado como muitas tarefas independentes se comunicando entre si, então os baixos custos por tarefa, tempos de criação curtos e alternâncias rápidas de contexto são vantagens importantes. É por isso que os servidores de bate-papo são o exemplo clássico da programação assíncrona, mas jogos com vários jogadores e roteadores de rede provavelmente também seriam bons usos.

Em outras situações, o uso de assíncrono é menos claro. Se seu programa tiver um pool de threads fazendo cálculos pesados ou permanecer ocioso esperando a conclusão da E/S, as vantagens listadas antes provavelmente não terão uma grande influência sobre o desempenho. Você terá de otimizar os cálculos, encontrar uma conexão de rede mais rápida ou fazer outra coisa que realmente afete o fator limitante.

Na prática, todos os relatos de implementação de servidores de alto volume que encontramos enfatizavam a importância da medição, ajuste e uma campanha incansável para identificar e remover fontes de contenção entre as tarefas. Uma arquitetura assíncrona não permitirá que você ignore qualquer parte desse trabalho. Na verdade, embora existam muitas ferramentas prontas para avaliar o comportamento de programas multithread, as tarefas assíncronas do Rust são invisíveis a essas ferramentas e, portanto, requerem ferramentas próprias. (Como um ancião sábio disse uma vez, “agora você tem *dois* problemas.”)

Mesmo que você não utilize código assíncrono agora, é bom saber que a opção existe se você tiver a sorte de estar muito mais ocupado do que agora.

¹ Se você realmente precisa de um cliente HTTP, considere utilizar qualquer um dos muitos

crates excelentes como surf ou reqwest que farão o trabalho de forma adequada e assíncrona.

- 2 Isso inclui a memória do kernel e conta as páginas físicas alocadas para o thread, não as páginas virtuais ainda a serem alocadas. Os números são semelhantes no macOS e no Windows.
- 3 Alternâncias de contexto no Linux também costumavam estar no intervalo de 0,2 μ s, até que o kernel foi forçado a utilizar técnicas mais lentas devido a falhas de segurança do processador.

CAPÍTULO 21

Macros

Um centão (do latim "centone", manta de retalhos) é um poema composto inteiramente de versos citados por outro poeta.

– Matt Madden

O Rust oferece suporte a macros, uma maneira de estender a linguagem de maneiras que vão além do que você pode fazer apenas com funções. Por exemplo, vimos a macro `assert_eq!`, que é útil para testes:

```
assert_eq!(gcd(6, 10), 2);
```

Isso poderia ter sido escrito como uma função genérica, mas a macro `assert_eq!` faz várias coisas que funções não podem fazer. Uma delas é que, quando uma asserção falha, `assert_eq!` gera uma mensagem de erro contendo o nome do arquivo e o número da linha da asserção. Funções não têm como obter essas informações. Macros conseguem, porque a maneira como funcionam é completamente diferente.

Macros são uma espécie de abreviação. Durante a compilação, antes que os tipos sejam verificados e muito antes de qualquer código de máquina ser gerado, cada chamada de macro é expandida – ou seja, é substituída por algum código Rust. A chamada de macro anterior se expande para algo mais ou menos assim:

```
match (&gcd(6, 10), &2) {
  (left_val, right_val) => {
    if !(*left_val == *right_val) {
      panic!("assertion failed: `(left == right)`,\n\
        (left: `{:?}`, right: `{:?}`)", left_val, right_val);
    }
  }
}
```

`panic!` também é uma macro, que se expande para ainda mais código Rust (não mostrado aqui). Esse código utiliza duas outras macros,

`file!()` e `line!()`. Depois que cada chamada de macro no crate estiver totalmente expandida, o Rust passa para a próxima fase de compilação.

Em tempo de execução, uma falha de asserção se pareceria com isto (e indicaria um bug na função `gcd()`, uma vez que 2 é a resposta correta):

```
thread 'main' panicked at 'assertion failed: `(left == right)` , (left: `17` ,  
right: `2`)', gcd.rs:7
```

Se você vem do C++, talvez tenha tido algumas experiências ruins com macros. As macros do Rust adotam uma abordagem diferente, semelhante à `syntax-rules` do Scheme. Em comparação com as macros do C++, as macros do Rust estão mais bem integradas com o restante da linguagem e, portanto, menos propensas a erros. Chamadas de macro são sempre marcadas com um ponto de exclamação, para que se destaquem quando você estiver lendo o código e não possam ser chamadas acidentalmente quando você pretendia chamar uma função. Macros do Rust nunca inserem colchetes ou parênteses não correspondentes. E as macros do Rust vêm com correspondência de padrões, tornando mais fácil escrever macros que são fáceis de manter e atraentes de utilizar.

Neste capítulo, mostraremos como escrever macros utilizando vários exemplos simples. Mas, como muito do Rust, as macros recompensam a compreensão profunda, portanto examinaremos o projeto de uma macro mais complicada que permite incorporar literais JSON diretamente em nossos programas. Mas há mais sobre macros do que podemos abordar neste livro, assim terminaremos com algumas dicas para um estudo mais aprofundado, tanto de técnicas avançadas para as ferramentas que mostramos aqui como para um recurso ainda mais poderoso chamado macros procedurais.

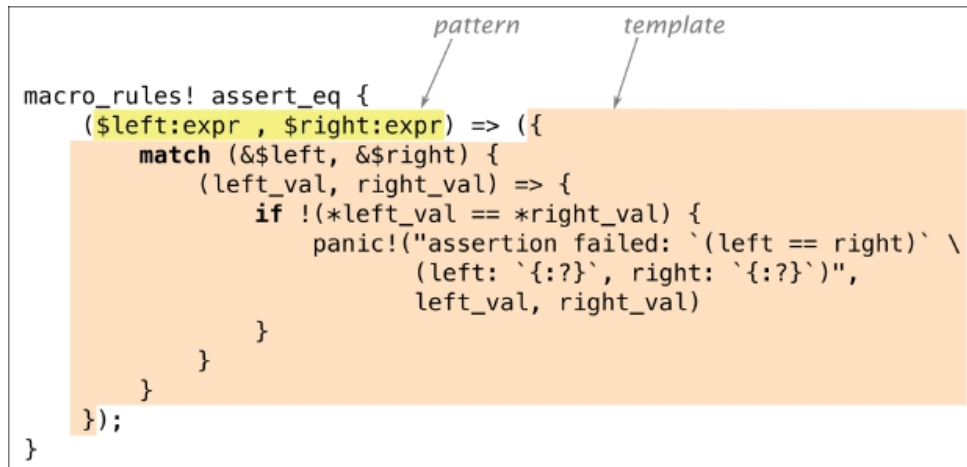
Noções básicas de macro

A Figura 21.1 mostra parte do código-fonte para a macro `assert_eq!`.

`macro_rules!` é a principal maneira de definir macros no Rust. Note que não há `!` depois de `assert_eq` na definição dessa macro: o `!` só é

incluído ao chamar uma macro, não ao defini-la.

Nem todas as macros são definidas dessa maneira: algumas, como `file!`, `line!` e `macro_rules!`, são integradas no compilador, e discutiremos outra abordagem, chamada macros procedurais, no final deste capítulo. Mas, na maioria das vezes, vamos nos concentrar em `macro_rules!`, que é (até agora) a maneira mais fácil de escrever suas próprias macros.



O diagrama mostra o código da macro `assert_eq!` dentro de `macro_rules!`. O código é o seguinte:

```
macro_rules! assert_eq {  
    ($left:expr, $right:expr) => {  
        match (&$left, &$right) {  
            (left_val, right_val) => {  
                if !(*left_val == *right_val) {  
                    panic!("assertion failed: `(left == right)` \n  
                        (left: `{:?}`, right: `{:?}`)",  
                        left_val, right_val)  
                }  
            }  
        }  
    }  
};
```

Dois rótulos com setas apontam para partes do código: "pattern" aponta para o padrão `($left:expr, $right:expr)` e "template" aponta para o corpo da macro `{ ... }`.

Figura 21.1: A macro `assert_eq!`.

Uma macro definida com `macro_rules!` funciona inteiramente por correspondência de padrões. O corpo de uma macro é apenas uma série de regras:

```
( pattern1 ) => ( template1 );
```

```
( pattern2 ) => ( template2 );
```

...

A versão de `assert_eq!` na Figura 21.1 tem apenas um pattern (padrão) e um template.

Aliás, você pode utilizar colchetes ou chaves em vez de parênteses ao redor do padrão ou template; não faz diferença para o Rust. Da mesma forma, quando você chama uma macro, todos são equivalentes:

```
assert_eq!(gcd(6, 10), 2);
```

```
assert_eq![gcd(6, 10), 2];
```

```
assert_eq!{gcd(6, 10), 2}
```

A única diferença é que o ponto e vírgula geralmente é opcional

após as chaves. Por convenção, utilizamos parênteses ao chamar `assert_eq!`, colchetes para `vec!` e chaves para `macro_rules!`.

Agora que mostramos um exemplo simples de expansão de uma macro e a definição que a gerou, podemos entrar nos detalhes necessários para colocar isso em funcionamento:

- Explicaremos exatamente como o Rust encontra e expande as definições de macro em seu programa.
- Apontaremos algumas sutilezas inerentes ao processo de geração de código a partir de templates de macro.
- Por fim, mostraremos como os padrões lidam com estruturas repetitivas.

Noções básicas de expansão de macro

O Rust expande macros muito cedo durante a compilação. O compilador lê o código-fonte do começo ao fim, definindo e expandindo macros à medida que avança. Você não pode chamar uma macro antes de ser definida, porque o Rust expande cada chamada de macro antes mesmo de analisar o restante do programa. (Por outro lado, funções e outros itens não precisam estar em nenhuma ordem específica. Não há problema em chamar uma função que só será definida mais tarde no crate.)

Quando Rust expande uma chamada de macro `assert_eq!`, o que acontece é muito parecido com a avaliação de uma expressão `match`. O Rust primeiro corresponde os argumentos com o padrão, conforme mostrado na Figura 21.2.

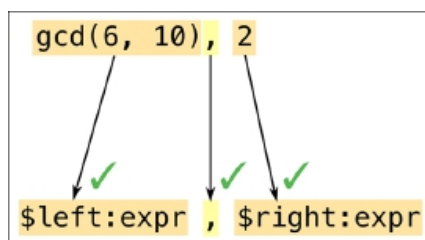


Figura 21.2: Expandindo uma macro, parte 1: correspondência de padrão dos argumentos.

Padrões de macro são uma minilinguagem dentro do Rust. Eles são

essencialmente expressões regulares para correspondência de código. Mas onde as expressões regulares operam em caracteres, os padrões operam em tokens – os números, nomes, sinais de pontuação e assim por diante, que são os blocos de construção dos programas Rust. Isso significa que você pode utilizar comentários e espaços em branco livremente em padrões de macro para torná-los o mais legíveis possível. Comentários e espaços em branco não são tokens, portanto não afetam a correspondência.

Outra diferença importante entre expressões regulares e padrões de macro é que parênteses, colchetes e chaves sempre ocorrem em pares correspondentes no Rust. Isso é verificado antes que as macros sejam expandidas, não apenas em padrões de macro, mas em toda a linguagem.

Neste exemplo, nosso padrão contém o fragmento `$left:expr`, que diz ao Rust que combine uma expressão (nesse caso, `gcd(6, 10)`) e atribua a ela o nome `$left`. O Rust então corresponde a vírgula no padrão com a vírgula seguindo os argumentos de `gcd`. Assim como as expressões regulares, os padrões têm apenas alguns caracteres especiais que desencadeiam um comportamento de correspondência interessante; todo o restante, como essa vírgula, deve corresponder literalmente ou a correspondência falhará. Por fim, o Rust corresponde a expressão `2` e atribui a ela o nome `$right`.

Ambos os fragmentos de código nesse padrão são do tipo `expr`: eles esperam expressões. Veremos outros tipos de fragmentos de código em “Tipos de fragmento”, na página 689.

Como esse padrão correspondeu a todos os argumentos, o Rust expande o template correspondente (Figura 21.3).

```

{
  match (&$left, &$right) {
    (left_val, right_val) => {
      if !(*left_val == *right_val) {
        panic!("assertion failed: `(left == right)` \
          (left: `{:?}`, right: `{:?}`)",
          left_val, right_val)
      }
    }
  }
}

```

substitui por gcd(6, 10)
substitui por 2

Figura 21.3: Expandindo uma macro, parte 2: preenchendo o template.

O Rust substitui `$left` e `$right` pelos fragmentos de código encontrados durante a correspondência.

É um erro comum incluir o tipo de fragmento no template de saída: escrevendo `$left:expr` em vez de apenas `$left`. O Rust não detecta imediatamente esse tipo de erro. Ele vê `$left` como uma substituição, e então trata `:expr` da mesma maneira como tudo no template: tokens a serem incluídos na saída da macro. Portanto, os erros não ocorrerão até que você chame a macro; em seguida, ele gerará uma saída falsa que não será compilada. Se você receber mensagens de erro como `cannot find type `expr` in this scope` e `help: maybe you meant to use a path separator here` ao utilizar uma nova macro, verifique o erro nisso. (“Depurando macros”, na página 687, oferece conselhos mais gerais para situações como essa.)

Templates de macro não são muito diferentes de qualquer uma das dezenas de linguagens de template comumente utilizadas na programação web. A única diferença – e é significativa – é que a saída é código Rust.

Consequências não intencionais

Conectar fragmentos de código a templates é sutilmente diferente do código regular que funciona com valores. Essas diferenças nem sempre são óbvias no início. A macro que vimos, `assert_eq!`, contém alguns trechos de código ligeiramente estranhos por motivos que dizem muito sobre a programação de macros. Vejamos dois trechos

especificamente curiosos.

Primeiro, por que essa macro cria as variáveis `left_val` e `right_val`? Existe algum motivo para não simplificarmos o template para se parecer com isso?

```
if !($left == $right) {
    panic!("assertion failed: `(left == right)` \
        (left: `{:?}`, right: `{:?}`)", $left, $right)
}
```

Para responder a essa pergunta, tente expandir mentalmente a chamada de macro `assert_eq!(letters.pop(), Some('z'))`. Qual seria a saída? Naturalmente, o Rust conectaria as expressões correspondidas ao template em vários lugares. Parece uma má ideia avaliar todas as expressões novamente ao criar a mensagem de erro e não apenas porque levaria o dobro do tempo: como `letters.pop()` remove um valor de um vetor, ele produzirá um valor diferente na segunda vez que a chamarmos! É por isso que a macro real calcula `$left` e `$right` apenas uma vez e armazena os valores.

Passando para a segunda pergunta: por que essa macro empresta referências aos valores de `$left` e `$right`? Por que não apenas armazenar os valores em variáveis, dessa maneira?

```
macro_rules! bad_assert_eq {
    ($left:expr, $right:expr) => ({
        match ($left, $right) {
            (left_val, right_val) => {
                if !(left_val == right_val) {
                    panic!("assertion failed" /* ... */);
                }
            }
        }
    });
}
```

Para o caso particular que estamos considerando, em que os argumentos de macro são números inteiros, isso funcionaria bem. Mas se o chamador passou, digamos, uma variável `String` como `$left` ou `$right`, esse código moveria o valor para fora da variável!

```
fn main() {
    let s = "a rose".to_string();
    bad_assert_eq!(s, "a rose");
    println!("confirmed: {} is a rose", s); // erro: uso do valor movido "s"
```



```
}
```

Como não queremos que asserções movam valores, a macro pega referências emprestadas.

(Você deve ter se perguntado por que a macro utiliza `match` em vez de `let` para definir as variáveis. Nós também nos perguntamos. Acontece que não há nenhuma razão específica para isso. `let` teria sido equivalente.)

Resumindo, as macros podem fazer coisas surpreendentes. Se coisas estranhas acontecerem em torno de uma macro que você escreveu, uma boa aposta é que a culpa é da macro.

Um bug que você não verá é este bug clássico de macro C++:

```
// macro C++ com bugs para adicionar 1 a um número
#define ADD_ONE(n) n + 1
```

Por razões familiares para a maioria dos programadores C++ e que não vale a pena explicar completamente aqui, código comum como `ADD_ONE(1) * 10` OU `ADD_ONE(1 << 4)` produz resultados muito surpreendentes com essa macro. Para corrigi-lo, você adicionaria mais parênteses à definição da macro. Isso não é necessário no Rust, porque as macros do Rust estão mais bem integradas à linguagem. O Rust sabe quando está lidando com expressões, então efetivamente adiciona parênteses sempre que cola uma expressão em outra.

Repetição

A macro `vec!` padrão tem duas formas:

```
// Repete um valor N vezes
let buffer = vec![0_u8; 1000];
```

```
// Uma lista de valores, separados por vírgulas
let numbers = vec!["udon", "ramen", "soba"];
```

Pode ser implementada desta maneira:

```
macro_rules! vec {
  ($elem:expr ; $n:expr) => {
    ::std::vec::from_elem($elem, $n)
  };
  ( $( $x:expr ),* ) => {
    <[_]>::into_vec(Box::new([ $( $x ),* ]))
  }
}
```

```

};
( $( $x:expr ),+ ,) => {
    vec![ $( $x ),* ]
};
}

```

Há três regras aqui. Explicaremos como várias regras funcionam e, em seguida, examinaremos uma regra por vez.

Quando o Rust expande uma chamada de macro como `vec![1, 2, 3]`, ele começa tentando corresponder os argumentos 1, 2, 3 ao padrão da primeira regra, nesse caso `$elem:expr ; $n:expr`. A correspondência disso falha: 1 é uma expressão, mas o padrão requer um ponto e vírgula depois disso, e não há um. Então, o Rust passa para a segunda regra etc. Se nenhuma regra corresponder, é um erro.

A primeira regra lida com usos como `vec![0u8; 1000]`. Acontece que existe uma função padrão (mas não documentada), `std::vec::from_elem`, que faz exatamente o que é necessário aqui, assim essa regra é simples e direta.

A segunda regra trata `vec!["udon", "ramen", "soba"]`. O padrão, `$( $x:expr ),*`, utiliza um recurso que não vimos antes: repetição. Corresponde a 0 ou mais expressões, separadas por vírgulas. Mais geralmente, a sintaxe `$( PATTERN ),*` é utilizada para corresponder a qualquer lista separada por vírgulas, em que cada item da lista corresponde a `PATTERN`.

O `*` (asterisco) aqui tem o mesmo significado que em expressões regulares (“0 ou mais”) embora regexps reconhecidamente não tenham um repetidor `*`. Você também pode utilizar `+` para exigir pelo menos uma correspondência, ou `?` para zero ou uma correspondência. A Tabela 21.1 fornece o conjunto completo dos padrões de repetição.

Tabela 21.1: Padrões de repetição

Padrão	Significado
<code>\$(...)*</code>	Corresponder com 0 ou mais vezes sem separador
<code>\$(...),*</code>	Corresponder 0 ou mais vezes, separados por vírgulas
<code>\$(...);*</code>	Corresponder 0 ou mais vezes, separados por ponto e vírgula

Padrão	Significado
<code>\$(...)+</code>	Corresponder com 1 ou mais vezes sem separador
<code>\$(...),+</code>	Corresponder 1 ou mais vezes, separados por vírgulas
<code>\$(...);+</code>	Corresponder 1 ou mais vezes, separados por ponto e vírgula
<code>\$(...)?</code>	Corresponder com 0 ou 1 vez sem separador
<code>\$(...),?</code>	Corresponder 0 ou 1 vez, separados por vírgulas
<code>\$(...);?</code>	Corresponder 0 ou 1 vez, separados por ponto e vírgula

O fragmento de código `$x` não é apenas uma única expressão, mas uma lista de expressões. O template dessa regra também utiliza sintaxe de repetição:

```
<[_]>::into_vec(Box::new([ $( $x ),* ]))
```

Mais uma vez, existem métodos padrão que fazem exatamente o que precisamos. Esse código cria um array em `box` e, em seguida, utiliza o método `[T]::into_vec` para converter o array em `box` em um vetor.

O primeiro fragmento, `<[_]>`, é uma maneira incomum de escrever o tipo “fatia de algo” e, ao mesmo tempo, esperar que o Rust infira o tipo de elemento. Tipos cujos nomes são identificadores simples podem ser utilizados em expressões sem problemas, mas tipos como `fn()`, `&str` ou `[_]` devem ser colocados entre colchetes.

A repetição vem no final do template, em que temos `$( $x ),*`. Essa `$(...),*` é a mesma sintaxe que vimos no padrão. Ela itera pela lista de expressões que correspondemos para `$x` e insere todas elas no template, separadas por vírgulas.

Nesse caso, a saída repetida se parece com a entrada. Mas isso não tem de ser o caso. Poderíamos ter escrito a regra desta maneira:

```
( $( $x:expr ),* ) => {
    {
        let mut v = Vec::new();
        $( v.push($x); )*
        v
    }
};
```

Aqui, a parte do template que diz `$( v.push($x); )*` insere uma chamada a `v.push()` para cada expressão em `$x`. Um braço de macro pode se

expandir para uma sequência de expressões, mas aqui precisamos apenas de uma única expressão, assim empacotamos o conjunto do vetor em um bloco.

Ao contrário do restante do Rust, padrões que utilizam `$( ... ),*` não suportam automaticamente uma vírgula final opcional. No entanto, há um truque padrão para oferecer suporte a vírgulas finais adicionando uma regra extra. É isso que a terceira regra da nossa macro `vec!` faz:

```
( $( $x:expr ),+ , ) => { // se houver vírgula à direita,  
    vec! [ $( $x ),* ]    // tente novamente sem ela  
};
```

Utilizamos `$( ... ),+ ,` para corresponder a uma lista com uma vírgula extra. Então, no template, chamamos `vec!` recursivamente, deixando a vírgula extra de fora. Dessa vez, a segunda regra corresponderá.

Macros internas

O compilador do Rust fornece várias macros que são úteis quando você está definindo suas próprias macros. Nenhuma delas poderia ser implementada utilizando `macro_rules!` sozinha. Elas são codificadas em `rustc`:

`file!()`, `line!()`, `column!()`

`file!()` se expande para um literal de string: o nome do arquivo atual. `line!()` e `column!()` se expandem para literais `u32` que fornecem a linha e a coluna atuais (contando a partir de 1).

Se uma macro chama outra, que chama outra, tudo em arquivos diferentes e a última macro chama `file!()`, `line!()` ou `column!()`, ela se expandirá para indicar o local da primeira chamada de macro.

`stringify!(...tokens...)`

Expandem em um literal de string contendo os tokens fornecidos. A macro `assert!` utiliza isso para gerar uma mensagem de erro que inclui o código da assertiva.

Chamadas de macro no argumento não são expandidas: `stringify!(line!())` se expande para a string `"line!()"`.

O Rust constrói a string a partir dos tokens, então não há quebras

de linha ou comentários na string.

```
concat!(str0, str1, ...)
```

Expande em um único literal de string criado pela concatenação de seus argumentos.

O Rust também define essas macros para consultar o ambiente de construção:

```
cfg!(...)
```

Expande a uma constante booleana, `true` se a configuração de compilação atual corresponder à condição entre parênteses. Por exemplo, `cfg!(debug_assertions)` é verdadeiro se você estiver compilando com as asserções de depuração habilitadas.

Essa macro suporta exatamente a mesma sintaxe que o atributo `#` `[cfg(...)]` descrito em “Atributos”, na página 226, mas, em vez de compilação condicional, você obtém uma resposta verdadeira ou falsa.

```
env!("VAR_NAME")
```

Expande em uma string: o valor da variável de ambiente especificada em tempo de compilação. Se a variável não existir, é um erro de compilação.

Isso seria bastante inútil, exceto que o Cargo define muitas variáveis de ambiente interessantes ao compilar um crate. Por exemplo, para obter a string da versão atual do seu crate, você pode escrever:

```
let version = env!("CARGO_PKG_VERSION");
```

Uma lista completa dessas variáveis de ambiente está incluída na documentação do Cargo (<https://oreil.ly/CQyuz>).

```
option_env!("VAR_NAME")
```

Isso é o mesmo que `env!` exceto que retorna um `Option<&'static str>` que será `None` se a variável especificada não estiver definida.

Outras três macros integradas permitem incorporar código ou dados de outro arquivo:

```
include!("file.rs")
```

Expande para o conteúdo do arquivo especificado, que deve ser um código Rust válido – uma expressão ou uma sequência de itens.

```
include_str!("file.txt")
```

Expande para um `&'static str` contendo o texto do arquivo especificado. Você pode utilizar isso desta maneira:

```
const COMPOSITOR_SHADER: &str =  
    include_str!("../resources/compositor.glsl");
```

Se o arquivo não existir ou não for UTF-8 válido, você receberá um erro de compilação.

```
include_bytes!("file.dat")
```

Isso é a mesma coisa, exceto que o arquivo é tratado como dados binários, não como texto UTF-8. O resultado é um `&'static [u8]`.

Como todas as macros, essas são processadas em tempo de compilação. Se o arquivo não existir ou não puder ser lido, a compilação falhará. Não podem falhar em tempo de execução. Em todos os casos, se o nome do arquivo for um caminho relativo, ele será resolvido em relação ao diretório que contém o arquivo atual.

O Rust também fornece várias macros convenientes que não abordamos anteriormente:

```
todo!(), unimplemented!()
```

Elas são equivalentes a `panic!()`, mas transmitem uma intenção diferente. `unimplemented!()` é inserida em cláusulas `if`, braços de `match` e outros casos que ainda não foram tratados. Sempre gera um pânico. `todo!()` é praticamente o mesmo, mas transmite a ideia de que esse código simplesmente ainda não foi escrito; alguns IDEs detectam essa macro e a sinalizam.

```
matches!(value, pattern)
```

Compara um valor com um padrão e retorna `true` se corresponder ou, do contrário, `false`. É equivalente a escrever:

```
match value {  
    pattern => true,  
    _ => false  
}
```

Se você está procurando um exercício básico sobre como escrever macros, essa é uma boa macro para replicar – especialmente porque a implementação real, que você pode ver na documentação da biblioteca padrão, é bem simples.

Depurando macros

A depuração de uma macro excêntrica pode ser um desafio. O maior problema é a falta de visibilidade do processo de expansão de macros. O Rust geralmente expande todas as macros, encontra algum tipo de erro e, em seguida, imprime uma mensagem de erro que não mostra o código totalmente expandido que contém o erro!

Eis três ferramentas para ajudar a solucionar problemas de macros. (Todos esses recursos são instáveis, mas como foram realmente projetados para uso durante o desenvolvimento, não no código que você verificaria, isso não é um grande problema na prática.)

Primeiro e mais simples, você pode solicitar a `rustc` que mostre a aparência do seu código depois de expandir todas as macros. Utilize `cargo build --verbose` para ver como o Cargo está invocando `rustc`. Copie a linha de comando `rustc` e adicione `-Z unstable-options --pretty expanded` como opções. O código totalmente expandido é colocado no seu terminal. Infelizmente, isso só funciona se o código estiver livre de erros de sintaxe.

Segundo, o Rust fornece uma macro `log_syntax!()` que simplesmente imprime os argumentos no terminal em tempo de compilação. Você pode utilizar isso para depuração no estilo de `println!`. Essa macro requer o flag de recurso `#![feature(log_syntax)]`.

Terceiro, você pode solicitar ao compilador do Rust que registre todas as chamadas de macro no terminal. Insira `trace_macros!(true);` em algum lugar do seu código. A partir desse ponto, sempre que o Rust expande uma macro, ele imprimirá o nome e os argumentos da macro. Por exemplo, considere este programa:

```
#![feature(trace_macros)]

fn main() {
    trace_macros!(true);
    let numbers = vec![1, 2, 3];
    trace_macros!(false);
    println!("total: {}", numbers.iter().sum::<u64>());
}
```

Ele produz esta saída:

```
$ rustup override set nightly
```

```

...
$ rustc trace_example.rs
note: trace_macro
--> trace_example.rs:5:19
|
5 |   let numbers = vec![1, 2, 3];
|                   ^^^^^^^^^^^^^
|
= note: expanding `vec! { 1, 2, 3 }`
= note: to `<[_]> :: into_vec ( box [ 1, 2, 3 ] )`

```

O compilador mostra o código de cada chamada de macro, antes e depois da expansão. A linha `trace_macros!(false);` desativa o rastreamento novamente, então a chamada para `println!()` não é rastreada.

Construindo a macro `json!`

Já discutimos os principais recursos de `macro_rules!`. Nesta seção, desenvolveremos de forma incremental uma macro para criar dados JSON. Utilizaremos esse exemplo para mostrar como é desenvolver uma macro, apresentaremos as poucas peças restantes de `macro_rules!` e ofereceremos alguns conselhos sobre como garantir que suas macros se comportem conforme desejado.

No Capítulo 10, introduzimos esse `enum` para representar dados JSON:

```

#[derive(Clone, PartialEq, Debug)]
enum Json {
    Null,
    Boolean(bool),
    Number(f64),
    String(String),
    Array(Vec<Json>),
    Object(Box<HashMap<String, Json>>)
}

```

A sintaxe para especificar valores `Json` é, infelizmente, bastante prolixa:

```

let students = Json::Array(vec![
    Json::Object(Box::new(vec![
        ("name".to_string(), Json::String("Jim Blandy".to_string())),
        ("class_of".to_string(), Json::Number(1926.0)),
        ("major".to_string(), Json::String("Tibetan throat singing".to_string()))
    ]))
])

```



```

].into_iter().collect()))),
Json::Object(Box::new(vec![
    ("name".to_string(), Json::String("Jason Orendorff".to_string())),
    ("class_of".to_string(), Json::Number(1702.0)),
    ("major".to_string(), Json::String("Knots".to_string()))
]).into_iter().collect()))
]);

```

Queremos ser capazes de escrever isso utilizando uma sintaxe mais parecida com JSON:

```

let students = json!([
    {
        "name": "Jim Blandy",
        "class_of": 1926,
        "major": "Tibetan throat singing"
    },
    {
        "name": "Jason Orendorff",
        "class_of": 1702,
        "major": "Knots"
    }
]);

```

O que queremos é uma macro `json!` que utiliza um valor JSON como argumento e se expande para uma expressão do Rust como a do exemplo anterior.

Tipos de fragmento

A primeira tarefa ao escrever qualquer macro complexa é descobrir como corresponder, ou analisar, a entrada desejada.

Já podemos ver que a macro terá várias regras, porque existem vários tipos diferentes de coisas nos dados JSON: objetos, arrays, números e assim por diante. Na verdade, podemos supor que teremos uma regra para cada tipo JSON:

```

macro_rules! json {
    (null) => { Json::Null };
    ([ ... ]) => { Json::Array(...) };
    ({ ... }) => { Json::Object(...) };
    (???) => { Json::Boolean(...) };
    (???) => { Json::Number(...) };
    (???) => { Json::String(...) };
}

```

Isso não está totalmente correto, pois os padrões de macro não oferecem nenhuma maneira de separar os últimos três casos, mas veremos como lidar com isso mais tarde. Os primeiros três casos, pelo menos, claramente iniciam com tokens diferentes, então vamos começar com eles.

A primeira regra já funciona:

```
macro_rules! json {
  (null) => {
    Json::Null
  }
}

#[test]
fn json_null() {
  assert_eq!(json!(null), Json::Null); // passa!
}
```

Para adicionar suporte para arrays JSON, podemos tentar corresponder os elementos como `exprS`:

```
macro_rules! json {
  (null) => {
    Json::Null
  };
  ([ $( $element:expr ),* ]) => {
    Json::Array(vec![ $( $element ),* ])
  };
}
```

Infelizmente, isso não corresponde a todos os arrays JSON. Eis um teste que ilustra o problema:

```
#[test]
fn json_array_with_json_element() {
  let macro_generated_value = json!(
    [
      // jSON válido que não corresponde a `element:expr`
      {
        "pitch": 440.0
      }
    ]
  );
  let hand_coded_value =
    Json::Array(vec![
      Json::Object(Box::new(vec![
        ("pitch".to_string(), Json::Number(440.0))
      ]))
    ])
}
```

```

        ].into_iter().collect()))
    });
    assert_eq!(macro_generated_value, hand_coded_value);
}

```

O padrão `$( $element:expr ),*` significa “uma lista separada por vírgulas de expressões Rust”. Mas muitos valores JSON, particularmente objetos, não são expressões Rust válidas. Eles não corresponderão.

Como nem todo fragmento de código que você deseja corresponder é uma expressão, o Rust oferece suporte a vários outros tipos de fragmentos, listados na Tabela 21.2.

Tabela 21.2: Tipos de fragmentos suportados por `macro_rules!`

Tipo de fragmento	Correspondências (com exemplos)	Pode ser seguido por...
<code>expr</code>	Uma expressão: <code>2 + 2, "udon", x.len()</code>	<code>=> , ;</code>
<code>stmt</code>	Uma expressão ou declaração, sem nenhum ponto e vírgula à direita (difícil de utilizar; em vez disso, tente <code>expr</code> ou <code>block</code>)	<code>=> , ;</code>
<code>ty</code>	Um tipo: <code>String, Vec<u8>, (&str, bool), dyn Read + Send</code>	<code>=> , ; = { [: > as where</code>
<code>path</code>	Um caminho (discutido na página 217): <code>ferns, ::std::sync::mpsc</code>	<code>=> , ; = { [: > as where</code>
<code>pat</code>	Um padrão (discutido na página 281): <code>_, Some(ref x)</code>	<code>=> , = if in</code>
<code>item</code>	Um item (discutido na página 168): <code>struct Point { x: f64, y: f64 }, mod ferns;</code>	Qualquer coisa
<code>block</code>	Um bloco (discutido na página 167): <code>{ s += "ok\n"; true }</code>	Qualquer coisa
<code>meta</code>	O corpo de um atributo (discutido na página 226): <code>inline, derive(Copy, Clone), doc="3D models."</code>	Qualquer coisa
<code>literal</code>	Um valor literal: <code>1024, "Hello, world!", 1_000_000f64</code>	Qualquer coisa
<code>lifetime</code>	Um tempo de vida: <code>'a, 'item, 'static</code>	Qualquer coisa
<code>vis</code>	Um especificador de visibilidade: <code>pub, pub(crate), pub(in module::submodule)</code>	Qualquer coisa
<code>ident</code>	Um identificador: <code>std, Json, longish_variable_name</code>	Qualquer coisa
<code>tt</code>	Uma árvore de tokens (ver texto):	Qualquer coisa

Tipo de fragmento	Correspondências (com exemplos)	Pode ser seguido por...
	<code>;; >=, {}, [0 1 (+ 0 1)]</code>	

A maioria das opções nessa tabela impõe estritamente a sintaxe do Rust. O tipo `expr` corresponde apenas a expressões Rust (não a valores JSON), `ty` corresponde apenas a tipos Rust e assim por diante. Não são extensíveis: não há como definir novos operadores aritméticos ou novas palavras-chave que `expr` reconheceria. Não poderemos fazer com que nenhum desses dados JSON arbitrários correspondam.

Os últimos dois, `ident` e `tt`, oferecem suporte a argumentos de correspondência de macro que não se parecem com código Rust. `ident` corresponde a qualquer identificador. `Tt` corresponde a uma única árvore de tokens: seja um par de colchetes devidamente correspondidos, `(...)`, `[...]` ou `{...}` e tudo entre eles, incluindo árvores de token aninhadas ou um único token que não seja um colchete, como `1926` ou `"Knots"`.

Árvores de tokens são exatamente o que precisamos para nossa macro `json!`. Cada valor JSON é uma única árvore de tokens: números, strings, valores booleanos e `null` são todos tokens únicos; objetos e arrays são colocados entre colchetes. Assim, podemos escrever os padrões desta maneira:

```
macro_rules! json {
  (null) => {
    Json::Null
  };
  ([ $( $element:tt ),* ]) => {
    Json::Array(...)
  };
  ({ $( $key:tt : $value:tt ),* }) => {
    Json::Object(...)
  };
  ($other:tt) => {
    ... // A FAZER: retornar Number, String ou Boolean
  };
}
```

Essa versão da macro `json!` pode corresponder a todos os dados

JSON. Agora só precisamos produzir o código Rust correto.

Para garantir que o Rust possa ganhar novos recursos sintáticos no futuro sem quebrar nenhuma macro que você escreve hoje, o Rust restringe os tokens que aparecem nos padrões logo após um fragmento. A coluna “Pode ser seguido por...” da Tabela 21.2 mostra quais tokens são permitidos. Por exemplo, o padrão `$x:expr ~ $y:expr` é um erro, porque `~` não é permitido depois de um `expr`. O padrão `$vars:pat => $handler:expr` está bem, porque `$vars:pat` é seguido pela seta `=>`, um dos tokens permitidos para um `pat` e `$handler:expr` é seguido por nada, o que é sempre permitido.

Recursão em macros

Já vimos um caso trivial de uma macro chamando a si mesma: nossa implementação de `vec!` utiliza recursão para suportar vírgulas à direita. Aqui podemos mostrar um exemplo mais significativo: `json!` precisa chamar a si mesma recursivamente.

Podemos tentar oferecer suporte a arrays JSON sem utilizar recursão, assim:

```
([ $( $element:tt ),* ]) => {  
    Json::Array(vec![ $( $element ),* ])  
};
```

Mas isso não funcionaria. Estaríamos colando dados JSON (as árvores de token `$element`) diretamente em uma expressão Rust. São duas linguagens diferentes.

Precisamos converter cada elemento do array do formato JSON em formato do Rust. Felizmente, existe uma macro que faz isso: a que estamos escrevendo!

```
([ $( $element:tt ),* ]) => {  
    Json::Array(vec![ $( json!($element) ),* ])  
};
```

Os objetos podem ser suportados da mesma forma:

```
({ $( $key:tt : $value:tt ),* }) => {  
    Json::Object(Box::new(vec![  
        $( ($key.to_string(), json!($value)) ),*  
    ].into_iter().collect()))  
};
```

O compilador impõe um limite de recursão nas macros: 64 chamadas, por padrão. Isso é mais do que suficiente para usos normais de `json!`, mas macros recursivas complexas às vezes atingem o limite. Você pode ajustá-lo adicionando esse atributo na parte superior do crate onde a macro é utilizada:

```
#![recursion_limit = "256"]
```

Nossa macro `json!` está quase completa. Tudo o que resta é oferecer suporte a valores booleanos, numéricos e de string.

Usando traits com macros

Escrever macros complexas sempre é um quebra-cabeça. É importante lembrar que as próprias macros não são a única ferramenta de resolução de quebra-cabeças à sua disposição.

Aqui, precisamos suportar `json!(true)`, `json!(1.0)` e `json!("yes")`, convertendo o valor, seja qual for ele, no tipo de valor `Json`. Mas as macros não são boas para distinguir tipos. Podemos imaginar escrever:

```
macro_rules! json {
  (true) => {
    Json::Boolean(true)
  };
  (false) => {
    Json::Boolean(false)
  };
  ...
}
```

Essa abordagem falha imediatamente. Existem apenas dois valores booleanos, mas ainda mais números do que isso e ainda mais strings.

Felizmente, existe uma maneira padrão de converter valores de vários tipos em um tipo específico: o trait `From`, abrangido na página 371. Nós simplesmente precisamos implementar esse trait para alguns tipos:

```
impl From<bool> for Json {
  fn from(b: bool) -> Json {
    Json::Boolean(b)
  }
}
```

```

impl From<i32> for Json {
    fn from(i: i32) -> Json {
        Json::Number(i as f64)
    }
}

impl From<String> for Json {
    fn from(s: String) -> Json {
        Json::String(s)
    }
}

impl<'a> From<&'a str> for Json {
    fn from(s: &'a str) -> Json {
        Json::String(s.to_string())
    }
}

...

```

Na verdade, todos os 12 tipos numéricos devem ter implementações muito semelhantes, então pode fazer sentido escrever uma macro, apenas para evitar o copiar e colar:

```

macro_rules! impl_from_num_for_json {
    ( $( $t:ident )* ) => {
        $(
            impl From<$t> for Json {
                fn from(n: $t) -> Json {
                    Json::Number(n as f64)
                }
            }
        )*
    };
}

impl_from_num_for_json!(u8 i8 u16 i16 u32 i32 u64 i64 u128 i128
                        usize isize f32 f64);

```

Agora podemos utilizar `Json::from(value)` para converter um `value` de qualquer tipo suportado em `Json`. Na nossa macro, se parecerá com isto:

```

( $other:tt ) => {
    Json::from($other) // Trata booleano/número/string
};

```

Adicionar essa regra à nossa macro `json!` faz com que passe em todos os testes que escrevemos até agora. Juntando todas as peças, atualmente se parece com isto:

```
macro_rules! json {
    (null) => {
        Json::Null
    };
    ([ $( $element:tt ),* ]) => {
        Json::Array(vec![ $( json!($element) ),* ])
    };
    ({ $( $key:tt : $value:tt ),* }) => {
        Json::Object(Box::new(vec![
            $( ($key.to_string(), json!($value)) ),*
        ].into_iter().collect()))
    };
    ( $other:tt ) => {
        Json::from($other) // Trata booleano/número/string
    };
}
```

Acontece que a macro inesperadamente suporta o uso de variáveis e até mesmo expressões Rust arbitrárias dentro de dados JSON, um recurso extra útil:

```
let width = 4.0;
let desc =
    json!({
        "width": width,
        "height": (width * 9.0 / 4.0)
    });
```

Como `(width * 9.0 / 4.0)` é colocado entre parênteses, é uma única árvore de tokens, assim a macro corresponde de maneira bem-sucedida a `$value:tt` ao analisar o objeto.

Escopo e higiene

Um aspecto surpreendentemente complicado de escrever macros é que elas envolvem colar código de diferentes escopos. Portanto, as próximas páginas abordam as duas maneiras como o Rust lida com a criação de escopo: uma maneira para variáveis e argumentos locais e outra maneira para todo o resto.

Para mostrar por que isso é importante, vamos reescrever nossa regra para analisar objetos JSON (a terceira regra na macro `json!` mostrada anteriormente) para eliminar o vetor temporário. Podemos escrever assim:

```
{ $( $key:tt : $value:tt ),* } => {
```



```

    {
        let mut fields = Box::new(HashMap::new());
        $( fields.insert($key.to_string(), json!($value)); )*
        Json::Object(fields)
    }
};

```

Agora estamos preenchendo o `HashMap` sem utilizar `collect()`, mas chamando repetidamente o método `.insert()`. Isso significa que precisamos armazenar o mapa em uma variável temporária, que chamamos `fields`.

Mas então o que acontece se o código que chama `json!` começar a utilizar uma variável própria, também chamada `fields`?

```

let fields = "Fields, W.C.";
let role = json!({
    "name": "Larson E. Whipsnade",
    "actor": fields
});

```

A expansão da macro colaria dois fragmentos de código, ambos utilizando o nome `fields` para coisas diferentes!

```

let fields = "Fields, W.C.";
let role = {
    let mut fields = Box::new(HashMap::new());
    fields.insert("name".to_string(), Json::from("Larson E. Whipsnade"));
    fields.insert("actor".to_string(), Json::from(fields));
    Json::Object(fields)
};

```

Isso pode parecer uma armadilha inevitável sempre que as macros utilizam variáveis temporárias, e você já pode estar pensando nas possíveis correções. Talvez devêssemos renomear a variável que a macro `json!` define para algo que os chamadores provavelmente não passarão: em vez de `fields`, poderíamos chamá-la `__json$fields`.

A surpresa aqui é que a macro funciona como está. O Rust renomeia a variável para você! Esse recurso, implementado pela primeira vez em macros Scheme, chama-se *higiene* e, portanto, diz-se que o Rust tem macros *higiênicas*.

A maneira mais fácil de entender a higiene de macros é imaginar que, sempre que uma macro é expandida, as partes da expansão que vêm da própria macro são pintadas com uma cor diferente.

Variáveis de cores diferentes, então, são tratadas como se tivessem nomes diferentes:

```
let fields = "Fields, W.C.";
let role = {
    let mut fields = Box::new(HashMap::new());
    fields.insert("name".to_string(), Json::from("Larson E. Whipsnade"));
    fields.insert("actor".to_string(), Json::from(fields));
    Json::Object(fields)
};
```

Observe que os fragmentos de código que foram passados pelo chamador da macro e colados na saída, como "name" e "actor", mantêm a cor original (preto). Somente os tokens originados do template de macro são pintados.

Agora há uma variável chamada `fields` (declarada no chamador) e uma variável separada chamada `fields` (introduzida pela macro). Como os nomes têm cores diferentes, as duas variáveis não são confundidas.

Se uma macro realmente precisar referenciar uma variável no escopo do chamador, o chamador deverá passar o nome da variável para a macro.

(A metáfora da pintura não pretende ser uma descrição exata de como a higiene funciona. O mecanismo real é ainda um pouco mais inteligente do que isso, reconhecendo dois identificadores como iguais, independentemente da "pintura", se eles referenciarão uma variável comum que esteja no escopo da macro e de seu chamador. Mas casos como esse são raros no Rust. Se entender o exemplo anterior, você saberá o suficiente para utilizar macros higiênicas.)

Você deve ter notado que muitos outros identificadores foram pintados com uma ou mais cores à medida que as macros foram expandidas: `Box`, `HashMap` e `Json`, por exemplo. Apesar da pintura, o Rust não teve problemas para reconhecer esses nomes de tipo. Isso ocorre porque a higiene no Rust é limitada a variáveis e argumentos locais. Quando se trata de constantes, tipos, métodos, módulos, estática e nomes de macro, o Rust não vê cores.

Isso significa que, se nossa macro `json!` for utilizada em um módulo em que `Box`, `HashMap` ou `Json` não estão no escopo, a macro não

funcionará. Mostraremos como evitar esse problema na próxima seção.

Primeiro, vamos considerar um caso em que a higiene rigorosa do Rust atrapalha, e precisamos contornar isso. Suponha que temos muitas funções que contêm esta linha de código:

```
let req = ServerRequest::new(server_socket.session());
```

Copiar e colar essa linha é trabalhoso. Podemos utilizar uma macro em vez disso?

```
macro_rules! setup_req {
    () => {
        let req = ServerRequest::new(server_socket.session());
    }
}

fn handle_http_request(server_socket: &ServerSocket) {
    setup_req!(); // declara `req`, utiliza `server_socket`
    ... // código que utiliza `req`
}
```

Quando este livro foi escrito, isso não funcionava. Seria necessário que o nome `server_socket` na macro referenciasse ao `server_socket` local declarado na função, e vice-versa para a variável `req`. Mas a higiene evita que nomes em macros “colidam” com nomes em outros escopos – mesmo em casos como esse, em que é isso que você deseja.

A solução é passar para a macro todos os identificadores que você planeja utilizar dentro e fora do código da macro:

```
macro_rules! setup_req {
    ($req:ident, $server_socket:ident) => {
        let $req = ServerRequest::new($server_socket.session());
    }
}

fn handle_http_request(server_socket: &ServerSocket) {
    setup_req!(req, server_socket);
    ... // código que utiliza `req`
}
```

Como `req` e `server_socket` agora são fornecidos pela função, elas têm a “cor” certa para esse escopo.

A higiene torna o uso dessa macro um pouco mais complicado, mas

isso é um recurso, não um bug: é mais fácil raciocinar sobre macros higiênicas sabendo que elas não podem bagunçar variáveis locais de uma maneira dissimulada. Se você procurar um identificador como `server_socket` em uma função, encontrará todos os lugares onde ela é utilizada, incluindo chamadas de macro.

Importando e exportando macros

Como as macros são expandidas no início da compilação, antes que o Rust conheça a estrutura completa do módulo do seu projeto, o compilador tem funcionalidades especiais para exportá-las e importá-las.

As macros visíveis em um módulo são automaticamente visíveis nos módulos filhos. Para exportar macros de um módulo até o módulo pai, utilize o atributo `#[macro_use]`. Por exemplo, suponha que nosso *lib.rs* se pareça com isto:

```
#[macro_use] mod macros;  
mod client;  
mod server;
```

Todas as macros definidas no módulo `macros` são importadas em *lib.rs* e, portanto, visíveis por todo o restante do crate, inclusive em `client` e `server`.

Macros marcadas com `#[macro_export]` são automaticamente `pub` e podem ser referenciadas por caminho, como outros itens.

Por exemplo, o crate `lazy_static` fornece uma macro chamada `lazy_static`, que é marcada com `#[macro_export]`. Para utilizar essa macro em seu próprio crate, você escreveria:

```
use lazy_static::lazy_static;  
lazy_static!{ }
```

Depois que uma macro é importada, ela pode ser utilizada como qualquer outro item:

```
use lazy_static::lazy_static;  
  
mod m {  
    crate::lazy_static!{ }  
}
```

Obviamente, fazer qualquer uma dessas coisas significa que sua

macro pode ser chamada em outros módulos. Uma macro exportada, portanto, não deve depender de nada que esteja no escopo – não há como dizer o que estará no escopo em que será utilizada. Mesmo os recursos do prelúdio padrão podem ser escondidos.

Em vez disso, a macro deve utilizar caminhos absolutos para quaisquer nomes que utilizar. `macro_rules!` fornece o fragmento especial `$crate` para ajudar nisso. Não é o mesmo que `crate`, que é uma palavra-chave que pode ser utilizada em caminhos em qualquer lugar, não apenas em macros. `$crate` age como um caminho absoluto para o módulo raiz do `crate` em que a macro foi definida. Em vez de dizer `Json`, podemos escrever `$crate::Json`, que funciona mesmo se `Json` não tiver sido importada. `HashMap` pode ser alterado para `::std::collections::HashMap` OU `$crate::macros::HashMap`. Nesse último caso, teremos de reexportar `HashMap`, porque `$crate` não pode ser utilizado para acessar recursos privados de um `crate`. Na verdade, apenas se expande para algo como `::jsonlib`, um caminho comum. As regras de visibilidade não são afetadas.

Depois de mover a macro para um módulo próprio `macros` e modificá-lo para utilizar `$crate`, ela se parece com isto. Esta é a versão final:

```
// macros.rs
pub use std::collections::HashMap;
pub use std::boxed::Box;
pub use std::string::ToString;

#[macro_export]
macro_rules! json {
    (null) => {
        $crate::Json::Null
    };
    ([ $( $element:tt ),* ]) => {
        $crate::Json::Array(vec![ $( json!($element) ),* ])
    };
    ({ $( $key:tt : $value:tt ),* }) => {
        {
            let mut fields = $crate::macros::Box::new(
                $crate::macros::HashMap::new());
            $(
                fields.insert($crate::macros::ToString::to_string($key),
                    json!($value));
            )
        }
    }
}
```

```

        )*
        $crate::Json::Object(fields)
    }
};
($other:tt) => {
    $crate::Json::from($other)
};
}

```

Como o método `.to_string()` é parte do trait `ToString` padrão, utilizamos `$crate` para referenciar isso também, utilizando a sintaxe que introduzimos em “Chamadas de método totalmente qualificadas”, na página 318: `$crate::macros::ToString::to_string($key)`. No nosso caso, isso não é estritamente necessário para fazer a macro funcionar, porque `ToString` está no prelúdio padrão. Mas, se você estiver chamando métodos de um trait que pode não estar no escopo no ponto em que a macro é chamada, uma chamada de método totalmente qualificada é a melhor maneira de fazer isso.

Evitando erros de sintaxe durante a correspondência

A macro a seguir parece razoável, mas causa alguns problemas ao Rust:

```

macro_rules! complain {
    ($msg:expr) => {
        println!("Complaint filed: {}", $msg)
    };
    (user : $userid:tt , $msg:expr) => {
        println!("Complaint from user {}: {}", $userid, $msg)
    };
}

```

Suponha que a chamemos desta maneira:

```
complain!(user: "jimb", "the AI lab's chatbots keep picking on me");
```

Aos olhos humanos, isso obviamente corresponde ao segundo padrão. Mas o Rust tenta inicialmente a primeira regra, tentando corresponder toda a entrada a `$msg:expr`. É aqui que as coisas começam a dar errado para nós. `user: "jimb"` não é uma expressão, é claro, então temos um erro de sintaxe. O Rust se recusa a varrer um erro de sintaxe para debaixo do tapete – as macros já são difíceis o

suficiente para depurar. Em vez disso, é relatado imediatamente e a compilação é interrompida.

Se qualquer outro token em um padrão não corresponder, o Rust passa para a próxima regra. Somente os erros de sintaxe são fatais, e ocorrem apenas ao tentar corresponder fragmentos.

O problema aqui não é tão difícil de entender: estamos tentando corresponder um fragmento, `$msg:expr`, na regra errada. Não vai corresponder porque nem devíamos estar aqui. O chamador queria a outra regra. Existem duas maneiras fáceis de evitar isso.

Primeiro, evite regras confusas. Poderíamos, por exemplo, alterar a macro para que cada padrão comece com um identificador diferente:

```
macro_rules! complain {  
    (msg : $msg:expr) => {  
        println!("Complaint filed: {}", $msg);  
    };  
    (user : $userid:tt , msg : $msg:expr) => {  
        println!("Complaint from user {}: {}", $userid, $msg);  
    };  
}
```

Quando os argumentos da macro começam com `msg`, obteremos a regra 1. Quando eles começam com `user`, obteremos a regra 2. De qualquer forma, sabemos que temos a regra correta antes de tentarmos corresponder um fragmento.

A outra maneira de evitar erros espúrios de sintaxe é colocar regras mais específicas primeiro. Colocar a regra `user`: primeiro corrige o problema com `complain!`, porque a regra que causa o erro de sintaxe nunca é alcançada.

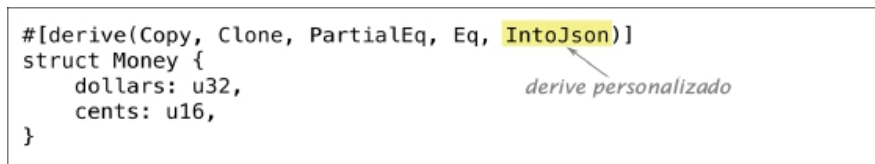
Para além de `macro_rules!`

Padrões de macro podem analisar entradas ainda mais complicadas do que JSON, mas descobrimos que a complexidade sai rapidamente de controle.

The Little Book of Rust Macros (<https://oreil.ly/nZ2HP>), de Daniel Keep et al., é um excelente manual sobre programação `macro_rules!` avançada. O livro é claro e inteligente e descreve todos os aspectos

da expansão de macros em mais detalhes do que apresentamos aqui. Ele também discute várias técnicas muito inteligentes para colocar padrões *macro_rules!* em serviço como uma espécie de linguagem de programação esotérica, para analisar entradas complexas. Estamos menos entusiasmados sobre isso. Use com cuidado.

O Rust 1.15 introduziu um mecanismo distinto chamado macros procedurais. Macros procedurais suportam a extensão do atributo `#[derive]` para lidar com derivações personalizadas, conforme mostrado na Figura 21.4, bem como criar atributos personalizados e novas macros que são invocadas da mesma maneira como as macros *macro_rules!* discutidas anteriormente.



```
#[derive(Copy, Clone, PartialEq, Eq, IntoJson)]
struct Money {
    dollars: u32,
    cents: u16,
}
```

derive personalizado

O diagrama mostra um código Rust dentro de uma caixa retangular. O código define um atributo `#[derive(Copy, Clone, PartialEq, Eq, IntoJson)]` sobre uma estrutura `Money` com campos `dollars: u32` e `cents: u16`. O texto `derive personalizado` com uma seta aponta para o atributo `IntoJson` na lista de atributos.

Figura 21.4: Invocando uma macro procedural `IntoJson` hipotética por meio de um atributo `#[derive]`.

Não há trait `IntoJson`, mas isso não importa: uma macro procedural pode utilizar esse gancho para inserir o código que quiser (nesse caso, provavelmente `impl From<Money> for Json { ... }`).

O que torna uma macro procedural “procedural” é que ela é implementada como uma função Rust, não como um conjunto de regras declarativas. Essa função interage com o compilador por meio de uma fina camada de abstração e pode ser arbitrariamente complexa. Por exemplo, a biblioteca de banco de dados *diesel* utiliza macros procedurais para se conectar a um banco de dados e gerar código com base no esquema desse banco de dados em tempo de compilação.

Como as macros procedurais interagem com as partes internas do compilador, escrever macros eficazes requer uma compreensão de como o compilador opera, o que está fora do escopo deste livro. No entanto, é amplamente abordado na documentação on-line (<https://oreil.ly/0xB2x>).

Talvez, depois de ler tudo isso, você tenha decidido que odeia macros. O que então? Uma alternativa é gerar código Rust utilizando um script de build. A documentação de Cargo (<https://oreil.ly/42irF>) mostra como fazer isso passo a passo. Envolve escrever um programa que gera o código Rust que você quer, adicionar uma linha a *Cargo.toml* para executar esse programa como parte do processo de build e utilizar `include!` para obter o código gerado em seu crate.

CAPÍTULO 22

Código inseguro

*Ninguém me suponha fraca ou débil, nem sossegada; outro é meu caráter:
dura para os inimigos, benévola para os amigos.
Porque de tais pessoas, a vida é gloriosíssima.*
– Eurípides, Medeia

A alegria secreta da programação de sistemas é que, sob cada linguagem segura e abstração cuidadosamente projetada, existe um turbilhão de códigos de máquina e manipulações de bits totalmente inseguros. Você também pode escrever isso no Rust.

A linguagem que apresentamos até este ponto no livro garante que seus programas estejam livres de erros de memória e corridas de dados de forma totalmente automática, por meio de tipos, tempos de vida, verificações de limites etc. Mas esse tipo de raciocínio automatizado tem limites; existem muitas técnicas valiosas que o Rust não consegue reconhecer como seguras.

Código inseguro permite que você diga ao Rust, “estou optando por utilizar recursos cuja segurança você não pode garantir”. Ao marcar um bloco ou função como insegura, você adquire a capacidade de chamar funções `unsafe` na biblioteca padrão, desreferenciar ponteiros inseguros e chamar funções escritas em outras linguagens como C e C++, entre outros poderes. As outras verificações de segurança do Rust ainda se aplicam: verificações de tipo, verificações de tempo de vida e verificações de limites em índices ocorrem normalmente. Código inseguro permite apenas um pequeno conjunto de recursos adicionais.

Essa capacidade de ultrapassar os limites do Rust seguro é o que torna possível implementar muitos dos recursos mais fundamentais do Rust no próprio Rust, assim como C e C++ são utilizados para

implementar suas próprias bibliotecas padrão. Código inseguro é o que permite que o tipo `Vec` gerencie seu buffer com eficiência; o módulo `std::io` conversar com o sistema operacional; e os módulos `std::thread` e `std::sync` fornecerem primitivas de concorrência.

Este capítulo aborda os fundamentos do trabalho com recursos não seguros:

- Os blocos `unsafe` do Rust estabelecem o limite entre o código Rust comum e seguro e o código que utiliza recursos inseguros.
- Você pode marcar funções como `unsafe`, alertando os chamadores sobre a presença de contratos extras que eles devem seguir para evitar comportamento indefinido.
- Ponteiros brutos e seus métodos permitem acesso irrestrito à memória e permitem criar estruturas de dados que o sistema de tipos do Rust proibiria. Embora as referências do Rust sejam seguras, mas restritas, os ponteiros brutos, como qualquer programador C ou C++ sabe, são uma ferramenta poderosa e precisa.
- Compreender a definição de comportamento indefinido vai ajudá-lo a entender por que pode haver consequências muito mais sérias do que apenas obter resultados incorretos.
- Traits inseguros, análogos a funções `unsafe`, impõem um contrato que cada implementação (em vez de cada chamador) deve seguir.

Inseguro de quê?

No início deste livro, mostramos um programa C que falha de maneira surpreendente porque não segue uma das regras prescritas pelo padrão C. Você pode fazer o mesmo no Rust:

```
$ cat crash.rs
fn main() {
    let mut a: usize = 0;
    let ptr = &mut a as *mut usize;
    unsafe {
        *ptr.offset(3) = 0x7fff72f484c;
    }
}
```

```
$ cargo build
```

```
Compiling unsafe-samples v0.1.0
Finished debug [unoptimized + debuginfo] target(s) in 0.44s
$ ../../target/debug/crash
crash: Error: .netrc file is readable by others.
crash: Remove password or make file unreadable by others.
Segmentation fault (core dumped)
$
```

Esse programa pega emprestada uma referência mutável à variável local `a`, converte-a em um ponteiro bruto do tipo `*mut usize` e então utiliza o método `offset` para produzir um ponteiro cujo endereço é três palavras mais adiante na memória. Isso acontece onde o endereço de retorno de `main` é armazenado. O programa sobrescreve o endereço de retorno com uma constante, de modo que retornar de `main` apresenta um comportamento surpreendente. O que torna essa falha possível é o uso incorreto de recursos inseguros do programa – nesse caso, a capacidade de desreferenciar ponteiros brutos.

Um recurso inseguro é aquele que impõe um contrato: regras que o Rust não pode aplicar automaticamente, mas que você deve seguir para evitar comportamento indefinido.

Um contrato vai além das verificações usuais de tipo e verificações de tempo de vida, impondo outras regras específicas a esse recurso inseguro. Normalmente, o próprio Rust não sabe nada sobre o contrato; é explicado apenas na documentação do recurso. Por exemplo, o tipo de ponteiro bruto tem um contrato que proíbe desreferenciar um ponteiro que avançou para além do final de seu referente original. A expressão `*ptr.offset(3) = ...` neste exemplo quebra esse contrato. Mas, como mostra a transcrição, o Rust compila o programa sem reclamar: as verificações de segurança não detectam essa violação. Ao utilizar recursos inseguros, você, como programador, assume a responsabilidade de verificar se seu código está de acordo com os contratos.

Muitos recursos têm regras que você deve seguir para usá-los corretamente, mas essas regras não são contratos no sentido que queremos dizer aqui, a menos que as possíveis consequências incluam comportamento indefinido. Comportamento indefinido é comportamento que o Rust supõe firmemente que seu código nunca

pode exibir. Por exemplo, o Rust supõe que você não substituirá o endereço de retorno de uma chamada de função por outra coisa. O código que passa nas verificações de segurança usuais do Rust e está em conformidade com os contratos dos recursos inseguros que ele utiliza não pode fazer isso. Como o programa viola o contrato de ponteiro bruto, seu comportamento é indefinido e sai dos trilhos.

Se seu código exibir comportamento indefinido, você violou sua metade do acordo com o Rust, e o Rust se recusa a prever as consequências. Extrair mensagens de erro irrelevantes das profundezas das bibliotecas do sistema e travar é uma consequência possível; entregar o controle do seu computador para um invasor é outra. Os efeitos podem variar de uma versão do Rust para outra, sem aviso prévio. Às vezes, porém, o comportamento indefinido não tem consequências visíveis. Por exemplo, se a função `main` nunca retorna (talvez chame `std::process::exit` para encerrar o programa antecipadamente), então o endereço de retorno corrompido provavelmente não será importante.

Você só pode utilizar recursos inseguros dentro de um bloco `unsafe` ou uma função `unsafe`; explicaremos ambos nas seções a seguir. Isso torna mais difícil utilizar recursos inseguros sem saber: forçando-o a escrever um bloco ou função `unsafe`, o Rust garante que você reconheceu que seu código pode ter regras adicionais a serem seguidas.

Blocos inseguros

Um bloco `unsafe` se parece com um bloco Rust comum precedido pela palavra-chave `unsafe`, com a diferença de que você pode utilizar recursos inseguros no bloco:

```
unsafe {  
    String::from_utf8_unchecked(ascii)  
}
```

Sem a palavra-chave `unsafe` na frente do bloco, o Rust se oporia ao uso de `from_utf8_unchecked`, que é uma função `unsafe` (insegura). Com o bloco `unsafe` em torno dela, você pode utilizar esse código em qualquer lugar.

Como um bloco Rust comum, o valor de um bloco `unsafe` é o da expressão final, ou `()` se não houver um. A chamada para `String::from_utf8_unchecked` mostrada anteriormente fornece o valor do bloco.

Um bloco `unsafe` libera cinco opções adicionais para você:

- Você pode chamar funções `unsafe`. Cada função `unsafe` deve especificar seu próprio contrato, dependendo do propósito.
- Você pode desreferenciar ponteiros brutos. O código seguro pode passar ponteiros brutos, compará-los e criá-los por conversão de referências (ou mesmo de números inteiros), mas apenas código inseguro pode realmente usá-los para acessar a memória. Abordaremos ponteiros brutos em detalhes e explicaremos como usá-los com segurança em “Ponteiros brutos”, na página 719.
- Você pode acessar os campos de `unions`, que o compilador não pode ter certeza se contêm padrões de bits válidos para seus respectivos tipos.
- Você pode acessar variáveis `static` mutáveis. Como explicado em “Variáveis globais”, na página 606, o Rust não pode ter certeza quando threads estão utilizando variáveis `static` mutáveis, portanto o contrato exige que você garanta que todo o acesso esteja sincronizado corretamente.
- Você pode acessar funções e variáveis declaradas por meio da interface de função externa (Foreign Function Interface ou FFI) do Rust. Essas são consideradas `unsafe` mesmo quando imutáveis, pois são visíveis para códigos escritos em outras linguagens que podem não respeitar as regras de segurança do Rust.

Na verdade, restringir recursos inseguros a blocos `unsafe` não o impede de fazer o que você quer. É perfeitamente possível inserir um bloco `unsafe` em seu código e seguir em frente. O benefício da regra reside principalmente em chamar a atenção humana para o código cuja segurança o Rust não pode garantir:

- Você não utilizará acidentalmente recursos inseguros e descobrirá que foi responsável por contratos que nem sabia que existiam.
- Um bloco `unsafe` atrai mais atenção dos revisores. Alguns projetos

até tem automação para garantir isso, sinalizando mudanças no código que afetam blocos `unsafe` que requerem atenção especial.

- Ao pensar em escrever um bloco `unsafe`, você pode reservar um momento para se perguntar se sua tarefa realmente requer tais medidas. Se é para desempenho, você tem medições para mostrar que isso é um gargalo? Talvez haja uma boa maneira de alcançar a mesma coisa no Rust seguro.

Exemplo: Um tipo de string ASCII eficiente

Eis a definição de `Ascii`, um tipo de string que garante que o conteúdo sempre seja ASCII válido. Esse tipo utiliza um recurso inseguro para fornecer conversão com custo zero em `String`:

```
mod my_ascii {
    /// Uma string codificada em ASCII
    #[derive(Debug, Eq, PartialEq)]
    pub struct Ascii(
        // Isso deve conter apenas texto ASCII bem formado:
        // bytes de `0` a `0x7f`.
        Vec<u8>
    );

    impl Ascii {
        /// Cria um `Ascii` a partir do texto ASCII em `bytes`.
        /// Retorna um erro `NotAsciiError` se `bytes` contém
        /// algum caractere não-ASCII.
        pub fn from_bytes(bytes: Vec<u8>) -> Result<Ascii, NotAsciiError> {
            if bytes.iter().any(|&byte| !byte.is_ascii()) {
                return Err(NotAsciiError(bytes));
            }
            Ok(Ascii(bytes))
        }
    }
}

// Quando a conversão falha, retornamos o vetor que não conseguimos converter.
// Isso deve implementar `std::error::Error`; omitido por brevidade.
#[derive(Debug, Eq, PartialEq)]
pub struct NotAsciiError(pub Vec<u8>);

// Conversão segura e eficiente, implementada utilizando código inseguro
impl From<Ascii> for String {
    fn from(ascii: Ascii) -> String {
        // Se esse módulo não tem bugs, isso é seguro, porque
```

```

        // texto ASCII bem formado também é UTF-8 bem formado
        unsafe { String::from_utf8_unchecked(ascii.0) }
    }
}
...
}

```

O segredo desse módulo é a definição do tipo `Ascii`. O tipo em si é marcado `pub`, para torná-lo visível fora do módulo `my_ascii`. Mas o elemento `Vec<u8>` do tipo não é público, então apenas o módulo `my_ascii` pode construir um valor `Ascii` ou referenciar seu elemento. Isso dá ao código do módulo controle total sobre o que pode ou não aparecer aí. Contanto que os construtores e métodos públicos assegurem que os valores `Ascii` recém-criados estejam bem formados e permaneçam assim ao longo dos tempos de vida, então o restante do programa não poderá violar essa regra. E, de fato, o construtor público `Ascii::from_bytes` verifica cuidadosamente o vetor fornecido antes de concordar em construir um `Ascii` a partir dele. Por questão de brevidade, não mostramos nenhum método, mas você pode imaginar um conjunto de métodos de manipulação de texto que garante que valores `Ascii` sempre contenham texto ASCII apropriado, assim como métodos de `String` garantem que conteúdo permanece UTF-8 bem formado.

Esse arranjo nos permite implementar `From<Ascii>` para `String` de maneira muito eficiente. A função insegura `String::from_utf8_unchecked` recebe um vetor de bytes e constrói um `String` a partir dele sem verificar se o conteúdo é texto UTF-8 bem formado; o contrato da função mantém o chamador responsável por isso. Felizmente, as regras impostas pelo tipo `Ascii` são exatamente o que precisamos para atender o contrato de `from_utf8_unchecked`. Como explicamos em “UTF-8”, na página 489, qualquer bloco de texto ASCII também é UTF-8 bem formado, portanto o `Vec<u8>` subjacente de um `Ascii` está imediatamente pronto para servir como um buffer de `String`.

Com essas definições em vigor, você pode escrever:

```

use my_ascii::Ascii;

let bytes: Vec<u8> = b"ASCII and ye shall receive".to_vec();

// Essa chamada não envolve alocação ou cópias de texto, apenas uma varredura

```



```
let ascii: Ascii = Ascii::from_bytes(bytes)
    .unwrap(); // Sabemos que esses bytes escolhidos estão ok

// Essa chamada tem custo zero: sem alocação, cópias ou varreduras
let string = String::from(ascii);

assert_eq!(string, "ASCII and ye shall receive");
```

Nenhum bloco `unsafe` é necessário para utilizar `Ascii`. Implementamos uma interface segura utilizando operações inseguras e organizamos para cumprir os contratos dependendo apenas do próprio código do módulo, não do comportamento dos usuários.

Um `Ascii` nada mais é do que um envelope em torno de um `Vec<u8>`, escondido dentro de um módulo que impõe regras extras sobre o conteúdo. Um tipo como esse se chama tipo novo (newtype), um padrão comum no Rust. O próprio tipo `String` do Rust é definido exatamente da mesma maneira, exceto que o conteúdo é restrito a UTF-8, não ASCII. Na verdade, eis a definição de `String` da biblioteca padrão:

```
pub struct String {
    vec: Vec<u8>,
}
```

No nível de máquina, com os tipos do Rust fora de cena, um tipo novo e seu elemento têm representações idênticas na memória, portanto a construção de um novo tipo não requer nenhuma instrução de máquina. Em `Ascii::from_bytes`, a expressão `Ascii(bytes)` simplesmente considera que a representação de `Vec<u8>` agora contém um valor `Ascii`. De forma similar, `String::from_utf8_unchecked` provavelmente não requer instruções de máquina quando embutido (inlined): o `Vec<u8>` agora é considerado um `String`.

Funções inseguras

Uma definição de função `unsafe` se parece com uma definição de função comum precedida pela palavra-chave `unsafe`. O corpo de uma função `unsafe` é automaticamente considerado um bloco `unsafe`.

Você só pode chamar funções `unsafe` dentro de blocos `unsafe`. Isso significa que marcar uma função `unsafe` avisa os chamadores que a função tem um contrato que eles devem atender para evitar

comportamento indefinido.

Por exemplo, eis um novo construtor para o tipo `Ascii` que introduzimos antes que constrói um `Ascii` de um vetor de bytes sem verificar se o conteúdo é ASCII válido:

```
// Isso deve ser colocado dentro do módulo `my_ascii`
impl Ascii {
    /// Constrói um valor `Ascii` de `bytes`, sem verificar
    /// se `bytes` realmente contém ASCII bem formado.
    ///
    /// Esse construtor é infalível, e retorna um `Ascii` diretamente, em vez de
    /// um `Resultado<Ascii, NotAsciiError>` como o construtor `from_bytes` retorna
    ///
    /// # Segurança
    ///
    /// O chamador deve garantir que `bytes` contenha apenas caracteres ASCII:
    /// bytes não maiores que 0x7f. Do contrário, o efeito é indefinido
    pub unsafe fn from_bytes_unchecked(bytes: Vec<u8>) -> Ascii {
        Ascii(bytes)
    }
}
```

Presumivelmente, de alguma forma a chamada de código `Ascii::from_bytes_unchecked` já sabe que o vetor em mãos contém apenas caracteres ASCII, então a verificação que `Ascii::from_bytes` insiste em realizar seria perda de tempo, e o chamador teria de escrever um código para lidar com os resultados `Err` que ele sabe que nunca ocorrerão. `Ascii::from_bytes_unchecked` permite que esse chamador evite as verificações e o tratamento de erros.

Mas antes enfatizamos a importância dos construtores e métodos públicos assegurando que os valores `Ascii` estejam bem formados. O código `from_bytes_unchecked` não cumpre essa responsabilidade?

Não exatamente: `from_bytes_unchecked` cumpre suas obrigações transmitindo-as ao chamador por meio de seu contrato. A presença desse contrato é o que torna correto marcar essa função `unsafe`: apesar do fato de que a função em si não realiza operações inseguras, os chamadores devem seguir regras que o Rust não pode impor automaticamente para evitar comportamento indefinido.

Você pode provocar comportamento indefinido quebrando o contrato de `Ascii::from_bytes_unchecked`? Sim. Você pode construir um `String`

contendo UTF-8 malformado da seguinte forma:

```
// Imagine que esse vetor seja o resultado de algum processo complicado
// que esperávamos que produzisse ASCII. Algo está errado!
let bytes = vec![0xf7, 0xbf, 0xbf, 0xbf];

let ascii = unsafe {
    // O contrato dessa função insegura foi violado
    // quando `bytes` contém bytes não ASCII.
    Ascii::from_bytes_unchecked(bytes)
};

let bogus: String = ascii.into();

// `bogus` agora contém UTF-8 malformado. A análise de seu primeiro caractere produz
// um `char` que não é um ponto de código Unicode válido. Isso é comportamento
// indefinido, assim a linguagem não informa como essa asserção deve se comportar
assert_eq!(bogus.chars().next().unwrap() as u32, 0x1fffff);
```

Em certas versões do Rust, em algumas plataformas, observou-se que essa asserção falhou com a seguinte mensagem de erro divertida:

```
thread 'main' panicked at 'assertion failed: `(left == right)`
  left: `2097151`,
 right: `2097151`, src/main.rs:42:5
```

Esses dois números parecem iguais para nós, mas isso não é culpa do Rust; é culpa do bloco `unsafe` anterior. Quando dizemos que comportamento indefinido leva a resultados imprevisíveis, é a isso que nos referimos.

Isso ilustra dois fatos cruciais sobre bugs e código inseguro:

- *Bugs que ocorrem antes que o bloco `unsafe` possa quebrar contratos.* Se um bloco `unsafe` causa comportamento indefinido, pode depender não apenas do código no próprio bloco, mas também do código que fornece os valores nos quais ele opera. Tudo de que seu código `unsafe` depende para atender os contratos é fundamental para a segurança. A conversão de `Ascii` em `String` baseada em `String::from_utf8_unchecked` está bem definida apenas se o restante do módulo mantiver adequadamente invariantes de `Ascii`.
- As consequências de quebrar um contrato podem aparecer depois que você sai do bloco `unsafe`. O comportamento indefinido cortejado pelo descumprimento do contrato de um recurso

inseguro muitas vezes não ocorre dentro do próprio bloco `unsafe`. Construir um `String` falso como mostrado antes só pode causar problemas muito mais tarde na execução do programa.

Essencialmente, o verificador de tipos, o verificador de empréstimos e outras verificações estáticas do Rust inspecionam seu programa e tentam gerar evidência de que ele não pode exibir comportamento indefinido. Quando o Rust compila seu programa com sucesso, isso significa que ele conseguiu provar que seu código é sólido. Um bloco `unsafe` é uma lacuna nesse processo em que você diz ao Rust: “Esse código é bom, confie em mim”. Se sua afirmação é verdadeira pode depender de qualquer parte do programa que influencia o que acontece no bloco `unsafe` e as consequências de estar errado podem aparecer em qualquer lugar influenciado pelo bloco `unsafe`. Escrever a palavra-chave `unsafe` equivale a um lembrete de que você não está obtendo todos os benefícios das verificações de segurança da linguagem.

Dada a escolha, você deve naturalmente preferir criar interfaces seguras, sem contratos. É muito mais fácil trabalhar com elas, pois os usuários podem contar com as verificações de segurança do Rust para garantir que o código esteja livre de comportamento indefinido. Mesmo que sua implementação utilize recursos inseguros, é melhor utilizar os tipos, tempos de vida e sistema de módulos do Rust para cumprir os contratos usando apenas o que você pode garantir, em vez de passar responsabilidades para seus chamadores.

Infelizmente, não é incomum encontrar funções inseguras em ambientes cuja documentação não se preocupa em explicar os contratos. Espera-se que você mesmo deduza as regras, com base em sua experiência e conhecimento de como o código se comporta. Se você já se perguntou de modo constrangedor se o que está fazendo com uma API C ou C++ está OK, então você sabe como é.

Bloco inseguro ou função insegura?

Você pode se perguntar se deve utilizar um bloco `unsafe` ou apenas marcar toda a função como insegura. A abordagem que

recomendamos é primeiro tomar uma decisão sobre a função:

- Se for possível utilizar indevidamente a função de uma forma que compila bem, mas ainda causa um comportamento indefinido, você deverá marcá-la como insegura. As regras para utilizar a função corretamente são o contrato; a existência de um contrato é o que torna a função insegura.
- Do contrário, a função é segura: nenhuma chamada com os tipos corretos pode causar um comportamento indefinido. Ela não deve ser marcada `unsafe`.

Se a função utiliza recursos inseguros no corpo é irrelevante; o que importa é a presença de um contrato. Antes, mostramos uma função insegura que não utiliza recursos inseguros, e uma função segura que utiliza recursos inseguros.

Não marque uma função segura `unsafe` só porque você utiliza recursos inseguros no corpo. Isso torna a função mais difícil de utilizar e confunde os leitores que (corretamente) esperam encontrar um contrato explicado em algum lugar. Em vez disso, utilize um bloco `unsafe`, mesmo que seja o corpo inteiro da função.

Comportamento indefinido

Na introdução, dissemos que o termo comportamento indefinido significa “comportamento que o Rust supõe firmemente que seu código nunca poderia exibir”. Essa é uma frase estranha, especialmente porque sabemos, por nossa experiência com outras linguagens, que esses comportamentos ocorrem acidentalmente com alguma frequência. Por que esse conceito é útil para definir as obrigações do código inseguro?

Um compilador é um tradutor de uma linguagem de programação em outra. O compilador do Rust pega um programa Rust e o traduz em um programa equivalente em linguagem de máquina. Mas o que significa dizer que dois programas em linguagens tão diferentes são equivalentes?

Felizmente, essa questão é mais fácil para programadores do que para linguistas. Costumamos dizer que dois programas são

equivalentes se eles sempre terão o mesmo comportamento visível quando executados: eles fazem as mesmas chamadas de sistema, interagem com bibliotecas estrangeiras de maneiras equivalentes e assim por diante. É um pouco como um teste de Turing de programas: se você não consegue dizer se está interagindo com o original ou com a tradução, então eles são equivalentes.

Agora considere o seguinte código:

```
let i = 10;
very_trustworthy(&i);
println!("{}", i * 100);
```

Mesmo não sabendo nada sobre a definição de `very_trustworthy`, podemos ver que recebe apenas uma referência compartilhada a `i`, assim a chamada não pode mudar o valor `i`. Como o valor passado para `println!` sempre será 1000, o Rust pode traduzir esse código em linguagem de máquina como se tivéssemos escrito:

```
very_trustworthy(&10);
println!("{}", 1000);
```

Essa versão transformada tem o mesmo comportamento visível da original, e provavelmente é um pouco mais rápida. Mas faz sentido considerar o desempenho dessa versão apenas se concordarmos que ela tem o mesmo significado da original. E se `very_trustworthy` fosse definido da seguinte forma?

```
fn very_trustworthy(shared: &i32) {
    unsafe {
        // Transforma a referência compartilhada em um ponteiro mutável.
        // Isso é um comportamento indefinido.
        let mutable = shared as *const i32 as *mut i32;
        *mutable = 20;
    }
}
```

Esse código quebra as regras para referências compartilhadas: ele muda o valor de `i` para 20, embora deva ser imutável porque `i` é emprestado para compartilhamento. Como resultado, a transformação que fizemos no chamador agora tem um efeito bem visível: se o Rust transformar o código, o programa imprimirá 1000; se não alterar o código e utilizar o novo valor de `i`, ele imprimirá 2000. Quebrar as regras para referências compartilhadas em `very_trustworthy`

significa que as referências compartilhadas não se comportarão como esperado nos chamadores.

Esse tipo de problema surge em quase todo tipo de transformação que o Rust pode tentar. Até mesmo colocar inline uma função no local de chamada supõe, entre outras coisas, que, quando a chamada termina, o fluxo de controle retornará ao local de chamada. Mas abrimos o capítulo com um exemplo de código malcomportado que viola até mesmo essa suposição.

É basicamente impossível para o Rust (ou qualquer outra linguagem) avaliar se uma transformação em um programa preserva o significado, a menos que possa confiar nos recursos fundamentais da linguagem para se comportar conforme projetado. E se apresentam ou não esse comportamento pode depender não apenas do código em mãos, mas de outras partes potencialmente distantes do programa. Para fazer qualquer coisa no seu código, o Rust deve supor que o restante de seu programa é bem-comportado.

Aqui, então, estão as regras do Rust para programas bem-comportados:

- O programa não deve ler a memória não inicializada.
- O programa não deve criar valores primitivos inválidos:
 - Referências, boxes ou ponteiros `fn` que são `null`.
 - valores `bool` que não são `0` ou `1`.
 - valores `enum` com valores discriminantes inválidos.
 - valores `char` que não são válidos, pontos de código Unicode não substitutos.
 - valores `str` que não são UTF-8 bem formados.
 - Ponteiros (fat) gordos com comprimentos inválidos de `vtables/fatia`.
 - Qualquer valor do tipo “never” (nunca), escrito `!`, para funções que não retornam.
- As regras para referências explicadas no Capítulo 5 devem ser seguidas. Nenhuma referência pode sobreviver ao referente; acesso compartilhado é acesso somente leitura; e acesso mutável é acesso exclusivo.

- O programa não deve desreferenciar ponteiros nulos, alinhados incorretamente ou pendentes.
- O programa não deve utilizar um ponteiro para acessar a memória fora da alocação à qual o ponteiro está associado. Explicaremos essa regra em detalhes em “Desreferenciando ponteiros brutos com segurança”, na página 722.
- O programa deve estar livre de corridas de dados (data races). Uma corrida de dados ocorre quando dois threads acessam a mesma posição na memória sem sincronização, e pelo menos um dos acessos é uma gravação.
- O programa não deve terminar em uma chamada feita de outra linguagem, via interface de função externa, como explicado em “Desempilhamento”, na página 189.
- O programa deve cumprir os contratos das funções da biblioteca padrão.

Como ainda não temos um modelo completo da semântica do Rust para código `unsafe`, essa lista provavelmente vai evoluir ao longo do tempo, mas esses provavelmente permanecerão proibidos.

Qualquer violação dessas regras constitui um comportamento indefinido e torna os esforços do Rust para otimizar seu programa e traduzi-lo em linguagem de máquina não confiável. Se você quebrar a última regra e passar UTF-8 malformado para `String::from_utf8_unchecked`, talvez 2097151 não seja tão igual a 2097151 afinal de contas.

O código Rust que não utiliza recursos inseguros tem a garantia de seguir todas as regras anteriores, uma vez compilado (supondo que o compilador não tenha bugs; estamos chegando lá, mas a curva nunca cruzará a assíntota). Somente quando você utiliza recursos não seguros, essas regras se tornam sua responsabilidade.

Em C e C++, o fato de seu programa compilar sem erros ou avisos significa muito menos; como mencionamos na introdução deste livro, mesmo os melhores programas C e C++ escritos por projetos respeitados que mantêm o código em padrões elevados exibem um comportamento indefinido na prática.

Traits inseguros

Um trait inseguro é um trait que tem um contrato que o Rust não pode verificar ou impor que os implementadores satisfaçam para evitar comportamento indefinido. Para implementar um trait inseguro, você deve marcar a implementação como insegura. Cabe a você entender o contrato do trait e certificar-se de que seu tipo o satisfaça.

Uma função que limita as variáveis de tipo com um trait inseguro é tipicamente aquela que utiliza traits inseguros e só atende os contratos dependendo do contrato do trait inseguro. Uma implementação incorreta do trait pode fazer com que tal função exiba um comportamento indefinido.

`std::marker::Send` e `std::marker::Sync` são os exemplos clássicos de traits inseguros. Esses traits não definem nenhum método, portanto são triviais de implementar para qualquer tipo que você desejar. Mas eles têm contratos: `Send` exige que os implementadores estejam seguros para mover para outro thread e `Sync` exige que sejam seguros para compartilhar entre threads por meio de referências compartilhadas. Implementar `Send` para um tipo inadequado, por exemplo, faria com que `std::sync::Mutex` não mais estivesse protegido de corridas de dados (data races).

Como um exemplo simples, a biblioteca padrão do Rust costumava incluir um trait inseguro, `core::nonzero::Zeroable`, para tipos que podem ser inicializados com segurança definindo todos os seus bytes como zero. Claramente, sem problemas zerar um `usize`, mas zerar um `&T` fornece uma referência nula, que causará uma falha se desreferenciada. Para os tipos que fossem `Zeroable`, algumas otimizações eram possíveis: você poderia inicializar uma array deles rapidamente com `std::ptr::write_bytes` (o equivalente do Rust a `memset`) ou utilizar chamadas do sistema operacional que alocam páginas zeradas. (`Zeroable` era instável e foi movido para uso interno apenas no crate `num` no Rust 1.26, mas é um bom exemplo simples e do mundo real.)

`Zeroable` era um trait marcador típico, sem métodos ou tipos

associados:

```
pub unsafe trait Zeroable {}
```

As implementações para os tipos apropriados eram igualmente simples e diretas:

```
unsafe impl Zeroable for u8 {}
unsafe impl Zeroable for i32 {}
unsafe impl Zeroable for usize {}
// e assim por diante para todos os tipos de inteiros
```

Com essas definições, poderíamos escrever uma função que aloca rapidamente um vetor de um determinado comprimento contendo um tipo `Zeroable`:

```
use core::nonzero::Zeroable;

fn zeroed_vector<T>(len: usize) -> Vec<T>
    where T: Zeroable
{
    let mut vec = Vec::with_capacity(len);
    unsafe {
        std::ptr::write_bytes(vec.as_mut_ptr(), 0, len);
        vec.set_len(len);
    }
    vec
}
```

Essa função começa criando um `Vec` vazio com a capacidade necessária e depois chama `write_bytes` para preencher o buffer não ocupado com zeros. (A função `write_byte` trata `len` como um número de elementos τ , não um número de bytes, então essa chamada preenche todo o buffer.) O método `set_len` de um vetor altera o comprimento sem fazer nada no buffer; isso é inseguro, porque você deve garantir que o espaço de buffer recém-incluído realmente contenha valores inicializados corretamente do tipo τ . Mas é exatamente isso que o `T: Zeroable` vinculado estabelece: um bloco de zero bytes representa um valor τ válido. Nosso uso de `set_len` foi seguro.

Aqui, nós o colocamos em uso:

```
let v: Vec<usize> = zeroed_vector(100_000);
assert!(v.iter().all(|&u| u == 0));
```

Claramente, `Zeroable` deve ser um `trait` inseguro, pois uma implementação que não respeita o contrato pode levar a um

comportamento indefinido:

```
struct HoldsRef<'a>(&'a mut i32);

unsafe impl<'a> Zeroable for HoldsRef<'a> { }

let mut v: Vec<HoldsRef> = zeroed_vector(1);
*v[0].0 = 1; // falha: desreferencia ponteiro nulo
```

O Rust não faz ideia de qual é o significado de `Zeroable`, portanto não pode dizer quando está sendo implementado para um tipo inapropriado. Como acontece com qualquer outro recurso inseguro, cabe a você entender e cumprir o contrato de um `trait` inseguro.

Observe que o código inseguro não deve depender de `traits` comuns e seguros serem implementados corretamente. Por exemplo, suponha que haja uma implementação do `trait std::hash::Hasher` que simplesmente retornou um valor de hash aleatório, sem relação aos valores com hash. O `trait` exige que usar hash nos mesmos bits duas vezes produza o mesmo valor de hash, mas essa implementação não atende a esse requisito; simplesmente está incorreta. Mas, como `Hasher` não é um `trait` inseguro, o código inseguro não deve exibir comportamento indefinido ao utilizar esse hasher. O tipo `std::collections::HashMap` é cuidadosamente escrito para respeitar os contratos dos recursos inseguros que ele utiliza, independentemente de como o hasher se comporta. Certamente, a tabela não funcionará corretamente: as pesquisas falharão e as entradas aparecerão e desaparecerão aleatoriamente. Mas a tabela não exibirá comportamento indefinido.

Ponteiros brutos

Um ponteiro bruto no Rust é um ponteiro irrestrito. Você pode utilizar ponteiros brutos para formar todos os tipos de estruturas que os tipos de ponteiro verificados do Rust não podem, como listas duplamente vinculadas ou grafos arbitrários dos objetos. Mas, como os ponteiros brutos são muito flexíveis, o Rust não pode dizer se você os está utilizando com segurança ou não, então você pode desreferenciá-los apenas em um bloco `unsafe`.

Os ponteiros brutos são essencialmente equivalentes aos ponteiros C

ou C++, portanto também são úteis para interagir com o código escrito nessas linguagens.

Existem dois tipos de ponteiros brutos:

- Um `*mut T` é um ponteiro bruto para um `T` que permite modificar o referente.
- Um `*const T` é um ponteiro bruto para um `T` que permite apenas a leitura de seu referente.

(Não há um tipo `*T` simples; você deve sempre especificar `const` ou `mut`.)

Você pode criar um ponteiro bruto por conversão de uma referência, e desreferenciá-lo com o operador `*`:

```
let mut x = 10;
let ptr_x = &mut x as *mut i32;

let y = Box::new(20);
let ptr_y = &*y as *const i32;

unsafe {
    *ptr_x += *ptr_y;
}
assert_eq!(x, 30);
```

Ao contrário de boxes e referências, ponteiros brutos podem ser nulos, como `NULL` em C++ ou `nullptr` em C:

```
fn option_to_raw<T>(opt: Option<&T>) -> *const T {
    match opt {
        None => std::ptr::null(),
        Some(r) => r as *const T
    }
}

assert!(!option_to_raw(Some(&("pea", "pod"))).is_null());
assert_eq!(option_to_raw::<i32>(None), std::ptr::null());
```

Esse exemplo não tem blocos `unsafe`: criar ponteiros brutos, passá-los e compará-los é seguro. Somente desreferenciar um ponteiro bruto não é seguro.

Um ponteiro bruto para um tipo não dimensionado é um ponteiro gordo (fat), assim como a referência correspondente ou tipo `Box` seriam. Um ponteiro `*const [u8]` inclui um comprimento com o endereço e um objeto `trait` como um ponteiro `*mut dyn std::io::Write`

carrega uma `vtable`.

Embora o Rust desreferencie implicitamente tipos de ponteiro seguros em várias situações, as desreferências de ponteiros brutos devem ser explícitas:

- O operador `.` não desreferenciará implicitamente um ponteiro bruto; você deve escrever `(*raw).field` OU `(*raw).method(...)`.
- Ponteiros brutos não implementam `Deref`, assim coerções `deref` não se aplicam a eles.
- Operadores como `==` e `<` comparam ponteiros brutos como endereços: dois ponteiros brutos são iguais se apontarem para a mesma posição na memória. Da mesma forma, usar `hash` em um ponteiro bruto propaga o endereço para o qual ele aponta, não o valor de seu referente.
- Traits de formatação como `std::fmt::Display` seguem as referências automaticamente, mas não manipulam ponteiros brutos. As exceções são `std::fmt::Debug` e `std::fmt::Pointer`, que mostram ponteiros brutos como endereços hexadecimais, sem desreferenciá-los.

Ao contrário do operador `+` em C e C++, o `+` do Rust não lida com ponteiros brutos, mas você pode executar aritmética de ponteiro por meio dos métodos `offset` e `wrapping_offset`, ou os métodos `add`, `sub`, `wrapping_add` e `wrapping_sub` mais convenientes. Inversamente, o método `offset_from` fornece a distância entre dois ponteiros em bytes, embora sejamos responsáveis por garantir que o início e o fim estejam na mesma região na memória (o mesmo `Vec`, por exemplo):

```
let trucks = vec!["garbage truck", "dump truck", "moonstruck"];
let first: *const &str = &trucks[0];
let last: *const &str = &trucks[2];
assert_eq!(unsafe { last.offset_from(first) }, 2);
assert_eq!(unsafe { first.offset_from(last) }, -2);
```

Nenhuma conversão explícita é necessária para `first` e `last`; apenas especificar o tipo é suficiente. O Rust implicitamente converte referências em ponteiros brutos (mas não o contrário, é claro).

O operador `as` permite quase todas as conversões plausíveis de referências a ponteiros brutos ou entre dois tipos de ponteiro bruto. Contudo, pode ser necessário dividir uma conversão complexa em

uma série de etapas mais simples. Por exemplo:

```
&vec![42_u8] as *const String; // erro: conversão inválida  
&vec![42_u8] as *const Vec<u8> as *const String; // permitido
```

Observe que `as` não converterá ponteiros brutos em referências. Essas conversões seriam inseguras, e `as` deve permanecer uma operação segura. Em vez disso, você deve desreferenciar o ponteiro bruto (em um bloco `unsafe`) e, em seguida, pegar emprestado o valor resultante.

Tenha muito cuidado ao fazer isso: uma referência produzida dessa maneira tem um tempo de vida irrestrito: não há limite de quanto tempo ela pode viver, já que o ponteiro bruto não dá ao Rust nada para basear essa decisão. Em “Uma interface segura para libgit2”, na página 764, mais adiante neste capítulo, mostramos vários exemplos de como restringir adequadamente os tempos de vida.

Muitos tipos têm métodos `as_ptr` e `as_mut_ptr` que retornam um ponteiro bruto para o conteúdo. Por exemplo, fatias de array e strings retornam ponteiros para seus primeiros elementos, e alguns iteradores retornam um ponteiro para o próximo elemento que eles produzirão. Tipos de ponteiro proprietários como `Box`, `Rc` e `Arc` têm funções `into_raw` e `from_raw` que convertem de e para ponteiros brutos. Alguns dos contratos desses métodos impõem requisitos surpreendentes, portanto verifique sua documentação antes de usá-los.

Você também pode construir ponteiros brutos por conversão de inteiros, embora os únicos inteiros nos quais você pode confiar para isso geralmente sejam aqueles obtidos de um ponteiro em primeiro lugar. “Exemplo: `RefWithFlag`”, na página 723, utiliza ponteiros brutos dessa maneira.

Ao contrário de referências, ponteiros brutos não são nem `Send` nem `Sync`. Como resultado, qualquer tipo que inclua ponteiros brutos não implementa esses traits por padrão. Não há nada inerentemente inseguro sobre enviar ou compartilhar ponteiros brutos entre threads; afinal, aonde quer que eles vão, você ainda precisa de um bloco `unsafe` para desreferenciá-los. Mas, considerando as funções que os ponteiros brutos normalmente desempenham, os designers

de linguagem consideraram esse comportamento o padrão mais útil. Já discutimos como implementar `Send` e `Sync` em “Traits inseguros”, na página 717.

Desreferenciando ponteiros brutos com segurança

Eis algumas diretrizes de bom senso para utilizar ponteiros brutos com segurança:

- Desreferenciar ponteiros nulos ou ponteiros perdidos é um comportamento indefinido, assim como é referenciar a memória não inicializada ou valores que saíram do escopo.
- Desreferenciar ponteiros que não estão alinhados corretamente de acordo com o tipo de referência é um comportamento indefinido.
- Você pode pegar emprestados valores de um ponteiro bruto desreferenciado apenas se isso obedecer às regras de segurança de referência explicadas no Capítulo 5: nenhuma referência pode sobreviver a seu referente, acesso compartilhado é acesso somente leitura e acesso mutável é acesso exclusivo. (Essa regra é facilmente violada por acidente, pois os ponteiros brutos são frequentemente utilizados para criar estruturas de dados com compartilhamento ou posse não padrão.)
- Você só pode utilizar o referente de um ponteiro bruto se for um valor bem formado de seu tipo. Por exemplo, você deve garantir que desreferenciar um `*const char` produza um ponto de código Unicode adequado e não substituto.
- Você só pode utilizar os métodos `offset` e `wrapping_offset` em ponteiros brutos a fim de apontar para bytes dentro da variável ou bloco de memória alocado em heap ao qual o ponteiro original se refere, ou para o primeiro byte além dessa região.

Se você fizer aritmética de ponteiro convertendo o ponteiro em um inteiro, fazendo aritmética no inteiro e depois convertê-lo de volta em um ponteiro, o resultado deve ser um ponteiro que as regras para o método `offset` teria permitido que você produzisse.

- Se você atribuir ao referente de um ponteiro bruto, você não deverá violar as invariantes de qualquer tipo do qual o referente é parte. Por exemplo, se houver um `*mut u8` apontando para um byte de um `String`, você só pode armazenar valores nesse `u8` que permitem que o `String` contenha UTF-8 bem formado.

Deixando de lado a regra de empréstimo, essas são essencialmente as mesmas regras que você deve seguir ao utilizar ponteiros em C ou C++.

A razão para não violar as invariantes dos tipos deve ser clara. Muitos dos tipos padrão do Rust utilizam código inseguro na implementação, mas ainda fornecem interfaces seguras na suposição de que as verificações de segurança, o sistema de módulos e as regras de visibilidade do Rust serão respeitados. O uso de ponteiros brutos para contornar essas medidas de proteção pode levar a um comportamento indefinido.

O contrato completo e exato para ponteiros brutos não é facilmente declarado e pode mudar à medida que a linguagem evolui. Mas os princípios descritos aqui devem mantê-lo em território seguro.

Exemplo: RefWithFlag

Eis um exemplo de como pegar um clássico¹ hack no nível de bits possibilitado por ponteiros brutos e encapsulado como um tipo completamente seguro do Rust. Esse módulo define um tipo, `RefWithFlag<'a, T>`, que contém um `&'a T` e um `bool`, como a tupla `(&'a T, bool)`, e ainda assim consegue ocupar apenas uma palavra de máquina em vez de duas. Esse tipo de técnica é utilizado regularmente em coletores de lixo e máquinas virtuais, em que certos tipos – digamos, o tipo que representa um objeto – são tão numerosos que adicionar até mesmo uma única palavra a cada valor aumentaria drasticamente o uso da memória:

```
mod ref_with_flag {  
    use std::marker::PhantomData;  
    use std::mem::align_of;  
  
    /// Um &'T e um bool, agrupados em uma única palavra.  
    /// O tipo 'T deve exigir alinhamento de pelo menos dois bytes.
```



```

///
/// Se você é o tipo de programador que nunca encontrou um ponteiro cujo
/// 2º-bit você não queria roubar, bem, agora você pode fazer isso com segurança!
/// ("Mas não é tão excitante assim...")
pub struct RefWithFlag<'a, T> {
    ptr_and_bit: usize,
    behaves_like: PhantomData<'a T> // não ocupa espaço
}

impl<'a, T: 'a> RefWithFlag<'a, T> {
    pub fn new(ptr: &'a T, flag: bool) -> RefWithFlag<T> {
        assert!(align_of::() % 2 == 0);
        RefWithFlag {
            ptr_and_bit: ptr as *const T as usize | flag as usize,
            behaves_like: PhantomData
        }
    }

    pub fn get_ref(&self) -> &'a T {
        unsafe {
            let ptr = (self.ptr_and_bit & !1) as *const T;
            &*ptr
        }
    }

    pub fn get_flag(&self) -> bool {
        self.ptr_and_bit & 1 != 0
    }
}

```

Esse código tira proveito do fato de que muitos tipos devem ser colocados em endereços pares na memória: uma vez que o bit menos significativo de um endereço par é sempre zero, podemos armazenar algo mais aí e então reconstruir de forma confiável o endereço original apenas mascarando o bit na parte inferior. Nem todos os tipos se qualificam; por exemplo, os tipos `u8` e `(bool, [i8; 2])` podem ser inseridos em qualquer endereço. Mas podemos verificar o alinhamento do tipo na construção e recusar tipos que não funcionarão.

Você pode utilizar `RefWithFlag` desta maneira:

```

use ref_with_flag::RefWithFlag;

let vec = vec![10, 20, 30];
let flagged = RefWithFlag::new(&vec, true);

```

```
assert_eq!(flagged.get_ref()[1], 20);
assert_eq!(flagged.get_flag(), true);
```

O construtor `RefWithFlag::new` recebe uma referência e um valor `bool`, verifica se o tipo de referência é adequado e, em seguida, converte a referência em um ponteiro bruto e então em um `usize`. O tipo `usize` é definido para ser grande o suficiente a fim de conter um ponteiro em qualquer processador para o qual estamos compilando, portanto converter um ponteiro bruto em um `usize` e de volta é bem definido. Assim que temos um `usize`, sabemos que deve ser par, então podemos utilizar o operador OR bit a bit `|` ou para combiná-lo com o `bool`, que convertemos em um número inteiro 0 ou 1.

O método `get_flag` extrai o componente `bool` de um `RefWithFlag`. É simples: basta mascarar o bit na parte inferior e verificar se é diferente de zero.

O método `get_ref` extrai a referência de um `RefWithFlag`. Primeiro, ele mascara o bit na parte inferior de `usize` e o converte em um ponteiro bruto. O operador `as` não converterá ponteiros brutos em referências, mas podemos desreferenciar o ponteiro bruto (em um bloco `unsafe`, naturalmente) e emprestá-lo. Pegar emprestado o referente de um ponteiro bruto fornece uma referência com um tempo de vida ilimitado: O Rust concederá à referência qualquer tempo de vida que faça o código em torno dela passar, se houver um. Mas normalmente há algum tempo de vida específico que é mais preciso e, portanto, captura mais erros. Nesse caso, como o tipo de retorno de `get_ref` é `&'a T`, o Rust vê que o tempo de vida da referência é o mesmo que o parâmetro do tempo de vida de `RefWithFlag'a`, que é exatamente o que queremos: esse é o tempo de vida da referência com a qual começamos.

Na memória, um `RefWithFlag` se parece com um `usize`: como `PhantomData` é um tipo de tamanho zero, o campo `behaves_like` não ocupa espaço na estrutura. Mas o `PhantomData` é necessário para que o Rust saiba tratar os tempos de vida no código que utiliza `RefWithFlag`. Imagine com que o tipo se pareceria sem o campo `behaves_like`:

```
// Isso não vai compilar
pub struct RefWithFlag<'a, T: 'a> {
    ptr_and_bit: usize
```

```
}
```

No Capítulo 5, ressaltamos que qualquer estrutura que contém referências não deve sobreviver aos valores emprestados, para que as referências não se tornem ponteiros perdidos. A estrutura deve obedecer às restrições que se aplicam aos seus campos. Isso certamente se aplica a `RefWithFlag`: no código de exemplo que acabamos de ver, `flagged` não deve sobreviver a `vec`, já que `flagged.get_ref()` retorna uma referência a ele. Mas nosso tipo `RefWithFlag` reduzido não contém nenhuma referência e nunca utiliza o parâmetro de tempo de vida `'a`. É apenas um `usize`. Como o Rust pode saber que quaisquer restrições se aplicam ao tempo de vida de `flagged`? Incluir um campo `PhantomData<&'a T>` diz ao Rust que trate `RefWithFlag<'a, T>` como se contivesse um `&'a T`, sem realmente afetar a representação do struct.

Embora o Rust realmente não saiba o que está acontecendo (é isso que torna `RefWithFlag` inseguro), ele fará o possível para ajudá-lo nisso. Se você omitir o campo `behaves_like`, o Rust vai reclamar que os parâmetros `'a` e `T` não são utilizados e sugere o uso de um `PhantomData`.

`RefWithFlag` utiliza as mesmas táticas do tipo `Ascii` que apresentamos anteriormente para evitar comportamento indefinido no bloco `unsafe`. O próprio tipo é `pub`, mas seus campos não, o que significa que apenas o código dentro do módulo `ref_with_flag` pode criar ou analisar dentro de um valor `RefWithFlag`. Você não precisa inspecionar muito código para ter certeza de que o campo `ptr_and_bit` está bem construído.

Ponteiros que podem ser nulos

Um ponteiro bruto nulo no Rust é um endereço zero, assim como em C e C++. Para qualquer tipo `T`, a função `std::ptr::null<T>` retorna um ponteiro nulo `*const T`, e `std::ptr::null_mut<T>` retorna um ponteiro nulo `*mut T`.

Existem algumas maneiras de verificar se um ponteiro bruto é nulo. A mais simples é o método `is_null`, mas o método `as_ref` pode ser mais

conveniente: ele recebe um ponteiro `*const T` e retorna um `Option<&'a T>`, transformando um ponteiro nulo em um `None`. Da mesma forma, o método `as_mut` converte ponteiros `*mut T` em valores `Option<&'a mut T>`.

Tamanhos e alinhamentos de tipos

Um valor de qualquer tipo `Sized` ocupa um número constante de bytes na memória e deve ser colocado em um endereço que seja múltiplo de algum valor de alinhamento, determinado pela arquitetura da máquina. Por exemplo, uma tupla `(i32, i32)` ocupa oito bytes e a maioria dos processadores prefere que ela seja colocada em um endereço múltiplo de quatro.

A chamada `std::mem::size_of::<T>()` retorna o tamanho de um valor do tipo `T`, em bytes, e `std::mem::align_of::<T>()` retorna o alinhamento requerido. Por exemplo:

```
assert_eq!(std::mem::size_of::<i64>(), 8);
assert_eq!(std::mem::align_of::<(i32, i32)>(), 4);
```

O alinhamento de qualquer tipo é sempre uma potência de dois.

O tamanho de um tipo é sempre arredondado para um múltiplo de seu alinhamento, mesmo que tecnicamente possa caber em menos espaço. Por exemplo, embora uma tupla como `(f32, u8)` requeira apenas cinco bytes, `size_of::<(f32, u8)>()` é 8, porque `align_of::<(f32, u8)>()` é 4. Isso garante que, se você tiver um array, o tamanho do tipo de elemento sempre refletirá o espaçamento entre um elemento e o seguinte.

Para tipos não dimensionados, o tamanho e o alinhamento dependem do valor disponível. Dada uma referência a um valor não dimensionado, as funções `std::mem::size_of_val` e `std::mem::align_of_val` retornam o tamanho e o alinhamento do valor. Essas funções podem operar em referências tanto a tipos `Sized` quanto a tipos não dimensionados (`unsized`):

```
// Ponteiros gordos para fatias armazenam o comprimento de seu referente
let slice: &[i32] = &[1, 3, 9, 27, 81];
assert_eq!(std::mem::size_of_val(slice), 20);

let text: &str = "alligator";
```

```

assert_eq!(std::mem::size_of_val(text), 9);

use std::fmt::Display;
let unremarkable: &dyn Display = &193_u8;
let remarkable: &dyn Display = &0.0072973525664;

// Estes retornam o tamanho/alinhamento do valor para o qual o objeto trait
// aponta, não os do objeto trait em si. Essa informação vem da vtable
// a qual o objeto trait se refere
assert_eq!(std::mem::size_of_val(unremarkable), 1);
assert_eq!(std::mem::align_of_val(remarkable), 8);

```

Aritmética de ponteiro

O Rust apresenta os elementos de um array, fatia ou vetor como um único bloco contíguo de memória, como mostrado na Figura 22.1. Os elementos são regularmente espaçados, de modo que, se cada elemento ocupa `size` bytes, então o i -ésimo elemento começa com o $i * \text{size}$ -ésimo byte.

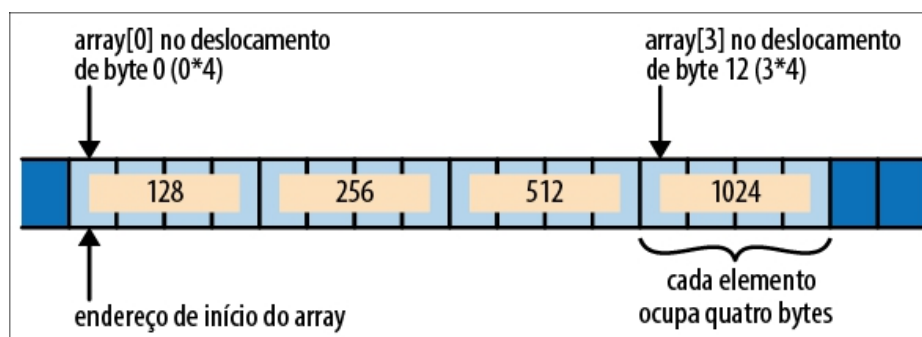


Figura 22.1: Um array na memória.

Uma boa consequência disso é que, se você tiver dois ponteiros brutos para elementos de um array, comparar os ponteiros fornecerá os mesmos resultados que comparar os índices dos elementos: se $i < j$, então um ponteiro bruto para o i -ésimo elemento é menor que um ponteiro bruto para o j -ésimo elemento. Isso torna os ponteiros brutos úteis como limites em percursos de array. Na verdade, o iterador simples da biblioteca padrão sobre uma fatia foi originalmente definido assim:

```

struct Iter<'a, T> {
    ptr: *const T,
    end: *const T,
    ...
}

```

```
}
```

O campo `ptr` aponta para o próximo elemento que a iteração deve produzir, e o campo `end` serve como o limite: quando `ptr == end`, a iteração está concluída.

Outra boa consequência do layout de array: se `element_ptr` é um ponteiro bruto `*const T` ou `*mut T` para o *i*-ésimo elemento de algum array, então `element_ptr.offset(o)` é um ponteiro bruto para o *(i + o)*-ésimo elemento. Sua definição é equivalente a isto:

```
fn offset<T>(ptr: *const T, count: isize) -> *const T
  where T: Sized
{
    let bytes_per_element = std::mem::size_of::<T>() as isize;
    let byte_offset = count * bytes_per_element;
    (ptr as isize).checked_add(byte_offset).unwrap() as *const T
}
```

A função `std::mem::size_of::<T>` retorna o tamanho do tipo `T` em bytes. Como `isize` é, por definição, grande o suficiente para conter um endereço, você pode converter o ponteiro base em um `isize`, fazer a aritmética nesse valor e, em seguida, converter o resultado de volta em um ponteiro.

Não há problemas em produzir um ponteiro para o primeiro byte após o final de um array. Você não pode desreferenciar esse ponteiro, mas pode ser útil para representar o limite de um loop ou para verificações de limites.

No entanto, é um comportamento indefinido utilizar `offset` para produzir um ponteiro além desse ponto ou antes do início do array, mesmo que você nunca o desreferencie. Por questão de otimização, o Rust quer supor que `ptr.offset(i) > ptr` quando *i* é positivo e que `ptr.offset(i) < ptr` quando *i* é negativo. Essa suposição parece segura, mas não se sustenta se a aritmética em `offset` ultrapassa um valor `isize`. Se *i* é obrigado a permanecer dentro do mesmo array que `ptr`, nenhum estouro pode ocorrer: afinal de contas, o próprio array não ultrapassa os limites do espaço de endereço. (Para tornar os ponteiros para o primeiro byte após o final seguros, o Rust nunca coloca valores na extremidade superior do espaço de endereço.)

Se você precisar deslocar ponteiros para além dos limites do array

ao qual eles estão associados, utilize o método `wrapping_offset`. Isso é equivalente a `offset`, mas o Rust não faz suposições sobre a ordem relativa de `ptr.wrapping_offset(i)` e `ptr` em si. Naturalmente, você ainda não pode desreferenciar esses ponteiros, a menos que eles estejam dentro do array.

Movendo dados para dentro e para fora da memória

Se você estiver implementando um tipo que gerencia sua própria memória, precisará rastrear quais partes da memória contêm valores ativos e quais não são inicializadas, assim como o Rust faz com variáveis locais. Considere este código:

```
let pot = "pasta".to_string();  
let plate = pot;
```

Após a execução desse código, a situação se parecerá com a Figura 22.2.

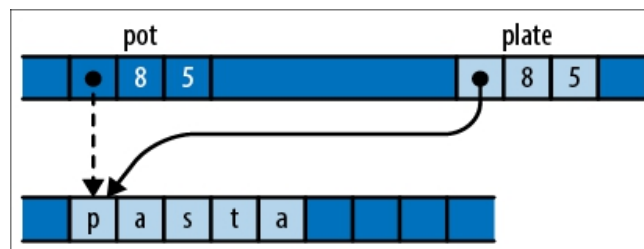


Figura 22.2: Movendo uma string de uma variável local para outra.

Após a atribuição, `pot` não é inicializado e `plate` é o proprietário da string.

No nível de máquina, não está especificado o que um movimento faz com a fonte, mas na prática geralmente não faz nada. A atribuição provavelmente ainda permite que `pot` contenha um ponteiro, capacidade e comprimento para a string. Naturalmente, seria desastroso tratar isso como um valor ativo, e o Rust garante que você não faça isso.

As mesmas considerações se aplicam a estruturas de dados que gerenciam sua própria memória. Suponha que você execute este código:

```
let mut noodles = vec!["udon".to_string()];
let soba = "soba".to_string();
let last;
```

Na memória, o estado se parece com a Figura 22.3.

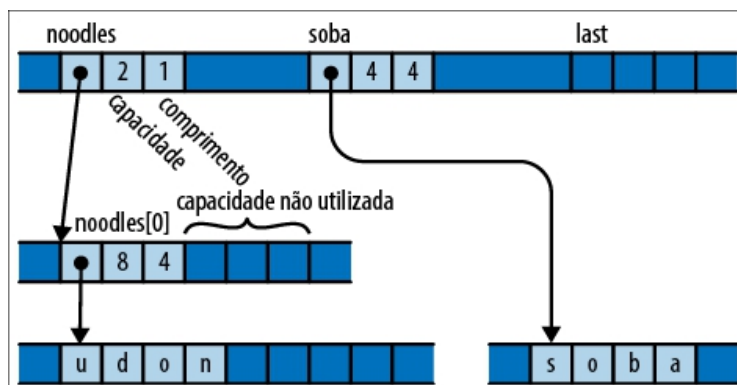


Figura 22.3: Um vetor com capacidade sobressalente não inicializada.

O vetor tem a capacidade sobressalente de conter mais um elemento, mas seu conteúdo é lixo, provavelmente o que quer que a memória continha anteriormente. Suponha que você execute este código:

```
noodles.push(soba);
```

Inserir a string no vetor transforma essa memória não inicializada em um novo elemento, conforme ilustrado na Figura 22.4.

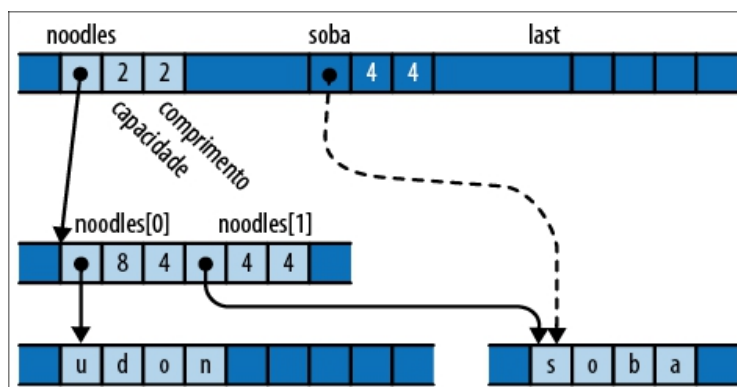


Figura 22.4: Depois de inserir o valor de `soba` no vetor.

O vetor inicializou o espaço vazio para possuir a string e incrementou o comprimento para marcá-lo como um novo elemento ativo. O vetor agora é o proprietário da string; você pode referenciar

o segundo elemento e dropar o vetor liberaria ambas as strings. E `soba` agora não está inicializada.

Por fim, considere o que acontece quando extraímos um valor do vetor:

```
last = noodles.pop().unwrap();
```

Na memória, as coisas agora se parecem com a Figura 22.5.

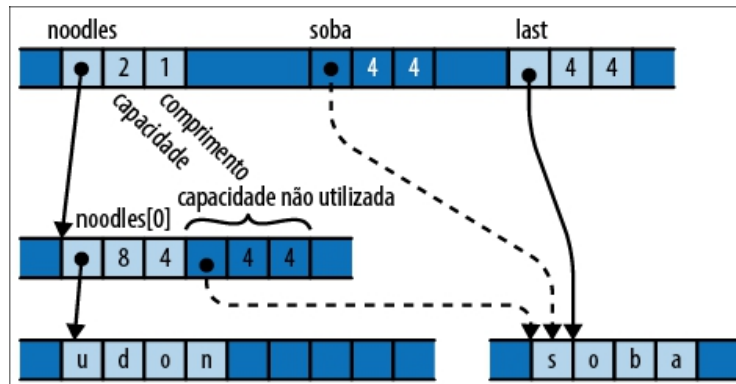


Figura 22.5: Depois de inserir um elemento do vetor em `last`.

A variável `last` tomou posse da string. O vetor diminuiu o comprimento para indicar que o espaço que costumava conter a string agora não está mais inicializado.

Assim como com `pot` e `pasta` anteriormente, todos os três de `soba`, `last` e o espaço livre do vetor provavelmente contêm padrões de bits idênticos. Mas apenas `last` é considerado proprietário do valor. Tratar qualquer um dos outros dois locais como ainda ativos seria um erro.

A verdadeira definição de um valor inicializado é aquele que é tratado como ativo. Escrever nos bytes de um valor geralmente é uma parte necessária da inicialização, mas apenas porque isso prepara o valor para ser tratado como ativo. Um movimento e uma cópia têm o mesmo efeito na memória; a diferença entre os dois é que, após um movimento, a origem não é mais tratada como ativa, enquanto, após uma cópia, tanto a origem quanto o destino estão ativos.

O Rust rastreia quais variáveis locais estão ativas em tempo de compilação e evita que você utilize variáveis cujos valores foram movidos para outro lugar. Tipos como `Vec`, `HashMap`, `Box` e assim por

diante rastreiam seus buffers dinamicamente. Se implementar um tipo que gerencia sua própria memória, você precisará fazer a mesma coisa.

O Rust fornece duas operações essenciais para implementar esses tipos:

`std::ptr::read(src)`

Move um valor para fora do local para o qual `src` aponta, transferindo a posse para o chamador. O argumento `src` deve ser um ponteiro bruto `*const T`, em que `T` é um tipo dimensionado. Depois de chamar essa função, o conteúdo de `*src` não é afetado, mas, a menos que `T` seja `Copy`, você deve garantir que seu programa o trate como memória não inicializada.

Essa é a operação por trás de `Vec::pop`. Inserir um valor chama `read` para mover o valor para fora do buffer e, em seguida, decrementa o comprimento para marcar esse espaço como capacidade não inicializada.

`std::ptr::write(dest, value)`

Move `value` no local para o qual `dest` aponta, que deve ser memória não inicializada antes da chamada. O referente agora possui o valor. Aqui, `dest` deve ser um ponteiro bruto `*mut T` e `value` um valor `T`, em que `T` é um tipo dimensionado.

Essa é a operação por trás de `Vec::push`. Inserir um valor chama `write` para mover o valor para o próximo espaço disponível e, em seguida, incrementa o comprimento para marcar esse espaço como um elemento válido.

Ambas são funções livres, não métodos nos tipos de ponteiro bruto. Observe que você não pode fazer essas coisas com nenhum dos tipos de ponteiro seguro do Rust. Todas exigem que os referentes sejam inicializados o tempo todo, portanto transformar memória não inicializada em um valor, ou vice-versa, está fora do alcance delas. Ponteiros brutos são apropriados.

A biblioteca padrão também fornece funções para mover arrays dos valores de um bloco de memória para outro:

`std::ptr::copy(src, dst, count)`

Move `count` valores do array na memória, começando em `src` para a memória em `dst`, como se você tivesse escrito um loop das chamadas `read` e `write` para movê-las uma de cada vez. A memória de destino deve estar desinicializada antes da chamada e depois a memória de origem é deixada desinicializada. Os argumentos `src` e `dest` devem ser ponteiros brutos `*const T` e `*mut T`, e `count` deve ser um `usize`.

`ptr::copy_to(dst, count)`

Uma versão mais conveniente de `copy`, que move `count` valores do array na memória, começando em `ptr` a `dst`, em vez de utilizar o ponto inicial como um argumento.

`std::ptr::copy_nonoverlapping(src, dst, count)`

Como a chamada correspondente para `copy`, exceto que o contrato ainda exige que os blocos de memória de origem e destino não se sobreponham. Isso pode ser um pouco mais rápido do que chamar `copy`.

`ptr::copy_to_nonoverlapping(dst, count)`

Uma versão mais conveniente de `copy_nonoverlapping`, como `copy_to`.

Existem outras duas famílias de funções `read` e `write`, também no módulo `std::ptr`:

`read_unaligned`, `write_unaligned`

Essas funções são como `read` e `write`, exceto que o ponteiro não precisa estar alinhado como normalmente exigido para o tipo de referência. Essas funções podem ser mais lentas do que as funções `read` e `write` simples.

`read_volatile`, `write_volatile`

Essas funções são equivalentes a leituras e gravações voláteis em C ou C++.

Exemplo: `GapBufferGenericName`

Eis um exemplo que coloca em uso as funções de ponteiro bruto que acabamos de descrever.

Suponha que você esteja escrevendo um editor de texto e esteja

procurando um tipo para representar o texto. Poderia escolher `String` e utilizar os métodos `insert` e `remove` para inserir e excluir caracteres à medida que o usuário digita. Mas, se eles estiverem editando texto no início de um arquivo grande, esses métodos podem ser caros: inserir um novo caractere envolve deslocar todo o restante da string para a direita na memória e a exclusão desloca tudo de volta para a esquerda. Você quer que essas operações comuns sejam mais baratas.

O editor de texto Emacs utiliza uma estrutura de dados simples chamada gap buffer que pode inserir e excluir caracteres em tempo constante. Considerando que um `String` mantém toda a sua capacidade sobressalente no final do texto, o que torna `push` e `pop` baratos, um gap buffer mantém sua capacidade sobressalente no meio do texto, no ponto em que a edição está ocorrendo. Essa capacidade sobressalente chama-se lacuna. Inserir ou excluir elementos em uma lacuna é fácil: você simplesmente reduz ou aumenta a lacuna conforme necessário. Pode mover a lacuna para qualquer local que desejar, deslocando o texto de um lado da lacuna para o outro. Quando a lacuna está vazia, você migra para um buffer maior.

Embora a inserção e exclusão em um gap buffer sejam rápidas, alterar a posição em que elas ocorrem envolve mover a lacuna para a nova posição. Deslocar os elementos requer tempo proporcional à distância que está sendo movida. Felizmente, a atividade típica de edição envolve fazer várias alterações em uma vizinhança do buffer antes de sair e mexer no texto em outro lugar.

Nesta seção, implementaremos um gap buffer no Rust. Para evitar distração pelo UTF-8, criaremos valores `char` para nosso armazenamento em buffer diretamente, mas os princípios da operação seriam os mesmos se armazenássemos o texto de alguma outra forma.

Primeiro, mostraremos um gap buffer em ação. Esse código cria um `GapBuffer`, insere algum texto nele e, em seguida, move o ponto de inserção para que permaneça um pouco antes da última palavra:

```
let mut buf = GapBuffer::new();
```

```
buf.insert_iter("Lord of the Rings".chars());
buf.set_position(12);
```

Depois de executar esse código, o buffer se parece com a Figura 22.6.

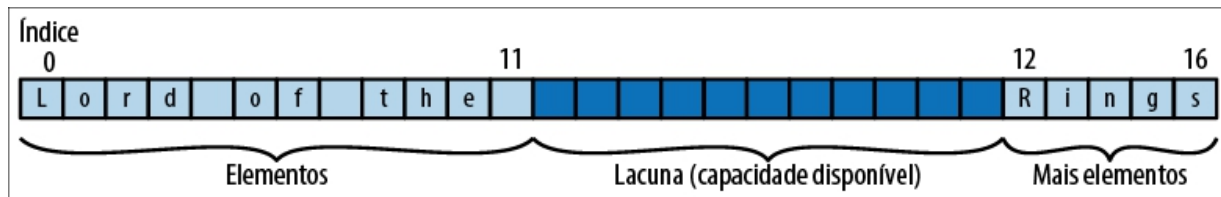


Figura 22.6: Um gap buffer contendo algum texto.

A inserção é uma questão de preencher a lacuna com o novo texto. Este código adiciona uma palavra e destrói o filme:

```
buf.insert_iter("Onion ".chars());
```

Isso resulta no estado mostrado na Figura 22.7.

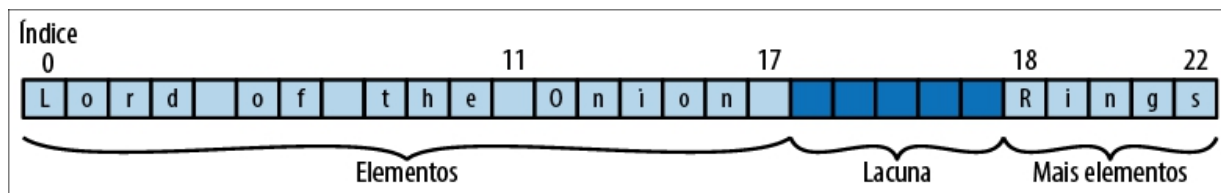


Figura 22.7: Um gap buffer contendo mais algum texto.

Eis nosso tipo `GapBuffer`:

```
use std;
use std::ops::Range;
pub struct GapBuffer<T> {
    // Armazenamento de elementos. Isso tem a capacidade de que precisamos,
    // mas seu comprimento permanece sempre zero. O GapBuffer insere os
    // elementos e o intervalo nessa capacidade "não utilizada" do `Vec`.
    storage: Vec<T>,

    // Intervalo de elementos não inicializados no meio do `armazenamento`.
    // Os elementos antes e depois desse intervalo são sempre inicializados.
    gap: Range<usize>
}
```

`GapBuffer` utiliza o campo `storage` de uma maneira estranha². Na verdade, ele nunca armazena nenhum elemento no vetor – ou não completamente. Ele simplesmente chama `Vec::with_capacity(n)` para obter um bloco de memória grande o suficiente para conter `n`

valores, obtém ponteiros brutos para essa memória por meio dos métodos `as_ptr` e `as_mut_ptr` do vetor e, em seguida, utiliza o buffer diretamente para seus próprios propósitos. O comprimento do vetor sempre permanece zero. Quando o `Vec` é dropado, o `Vec` não tenta liberar os elementos, porque não sabe que tem algum, mas libera o bloco de memória. Isso é o que `GapBuffer` quer; ele tem sua própria implementação de `Drop` que sabe onde estão os elementos ativos e os dropa corretamente.

Os métodos mais simples de `GapBuffer` são o que você esperaria:

```
impl<T> GapBuffer<T> {
    pub fn new() -> GapBuffer<T> {
        GapBuffer { storage: Vec::new(), gap: 0..0 }
    }

    /// Retorna o número de elementos que esse GapBuffer pode conter sem
    /// realocação.
    pub fn capacity(&self) -> usize {
        self.storage.capacity()
    }

    /// Retorna o número de elementos que esse GapBuffer contém atualmente
    pub fn len(&self) -> usize {
        self.capacity() - self.gap.len()
    }

    /// Retorna a posição de inserção atual
    pub fn position(&self) -> usize {
        self.gap.start
    }

    ...
}
```

Ele limpa muitas das funções a seguir para que haja um método utilitário que retorne um ponteiro bruto para o elemento do buffer em um determinado índice. Isso sendo Rust, acabamos precisando de um método para ponteiros `mut` e um para `const`. Ao contrário dos métodos anteriores, esses não são públicos. Continuando esse bloco `impl`:

```
/// Retorna um ponteiro para o `index`-ésimo elemento do armazenamento
/// subjacente, independentemente da lacuna
///
/// Segurança: `index` deve ser um índice válido dentro de `self.storage`
unsafe fn space(&self, index: usize) -> *const T {
```

```

    self.storage.as_ptr().offset(index as isize)
}

/// Retorna um ponteiro mutável para o `index`-ésimo elemento
/// do armazenamento subjacente, independentemente da lacuna.
///
/// Segurança: `index` deve ser um índice válido dentro de `self.storage`
unsafe fn space_mut(&mut self, index: usize) -> *mut T {
    self.storage.as_mut_ptr().offset(index as isize)
}

```

Para encontrar o elemento em um determinado índice, você deve considerar se o índice está antes ou depois da lacuna e ajustar adequadamente:

```

/// Retorna o deslocamento no buffer do `index`-ésimo elemento, levando
/// a lacuna em consideração. Isso não verifica se o índice está dentro
/// do intervalo, mas nunca retorna um índice na lacuna
fn index_to_raw(&self, index: usize) -> usize {
    if index < self.gap.start {
        index
    } else {
        index + self.gap.len()
    }
}

/// Retorna uma referência ao `index`-ésimo elemento,
/// ou `None` se `index` estiver fora dos limites
pub fn get(&self, index: usize) -> Option<&T> {
    let raw = self.index_to_raw(index);
    if raw < self.capacity() {
        unsafe {
            // Acabamos de verificar `raw` contra self.capacity(),
            // e index_to_raw ignora a lacuna, então isso é seguro
            Some(&*self.space(raw))
        }
    } else {
        None
    }
}

```

Quando começamos a fazer inserções e exclusões em uma parte diferente do buffer, precisamos mover a lacuna para o novo local. Mover a lacuna para a direita implica deslocar os elementos para a esquerda e vice-versa, assim como uma bolha em um nível de bolha de ar se move em uma direção quando o líquido flui na outra:

```

/// Define a posição de inserção atual como `pos`.
/// Se `pos` estiver fora dos limites, gera um pânico.

```

```

pub fn set_position(&mut self, pos: usize) {
    if pos > self.len() {
        panic!("index {} out of range for GapBuffer", pos);
    }

    unsafe {
        let gap = self.gap.clone();
        if pos > gap.start {
            // `pos` entra após a lacuna. Move a lacuna para a direita
            // deslocando os elementos após a lacuna para antes dele
            let distance = pos - gap.start;
            std::ptr::copy(self.space(gap.end),
                           self.space_mut(gap.start),
                           distance);
        } else if pos < gap.start {
            // `pos` entra antes da lacuna. Move a lacuna para a esquerda
            // deslocando elementos antes da lacuna para depois dela
            let distance = gap.start - pos;
            std::ptr::copy(self.space(pos),
                           self.space_mut(gap.end - distance),
                           distance);
        }
        self.gap = pos .. pos + gap.len();
    }
}

```

Essa função utiliza o método `std::ptr::copy` para deslocar os elementos; `copy` requer que o destino seja não inicializado e deixa a origem não inicializada. Os intervalos de origem e destino podem se sobrepor, mas `copy` lida com esse caso corretamente. Como a lacuna é memória não inicializada antes da chamada e a função ajusta a posição da lacuna para abranger o espaço vago pela cópia, o contrato das funções `copy` é atendido.

A inserção e remoção de elementos são relativamente simples. A inserção ocupa um espaço da lacuna para o novo elemento, enquanto a remoção move um valor para fora e aumenta a lacuna para abranger o espaço que costumava ocupar:

```

/// Insere `elt` na posição de inserção atual,
/// e deixa a posição de inserção depois dele
pub fn insert(&mut self, elt: T) {
    if self.gap.len() == 0 {
        self.enlarge_gap();
    }
}

```



```

    unsafe {
        let index = self.gap.start;
        std::ptr::write(self.space_mut(index), elt);
    }
    self.gap.start += 1;
}

/// Insere os elementos gerados por `iter` na posição de inserção
/// atual e deixa a posição de inserção depois deles
pub fn insert_iter<I>(&mut self, iterable: I)
    where I: IntoIterator<Item=T>
{
    for item in iterable {
        self.insert(item)
    }
}

/// Remove o elemento logo após a posição de inserção e retorna-o, ou
/// retorna `None` se a posição de inserção está no final do GapBuffer
pub fn remove(&mut self) -> Option<T> {
    if self.gap.end == self.capacity() {
        return None;
    }

    let element = unsafe {
        std::ptr::read(self.space(self.gap.end))
    };
    self.gap.end += 1;
    Some(element)
}

```

Semelhante à maneira como `Vec` utiliza `std::ptr::write` para inserir e `std::ptr::read` para pop, `GapBuffer` utiliza `write` para insert e `read` para remove. E assim como `Vec` deve ajustar o comprimento para manter o limite entre os elementos inicializados e a capacidade sobressalente, `GapBuffer` ajusta a lacuna.

Quando a lacuna for preenchida, o método `insert` deve aumentar o buffer para adquirir mais espaço livre. O método `enlarge_gap` (o último no bloco `impl`) lida com isto:

```

/// Duplica a capacidade do `self.storage`.
fn enlarge_gap(&mut self) {
    let mut new_capacity = self.capacity() * 2;
    if new_capacity == 0 {
        // O vetor existente está vazio.
        // Escolhe uma capacidade inicial razoável.
    }
}

```

```

    new_capacity = 4;
}

// Não fazemos ideia do que redimensionar um Vec faz com sua capacidade
// "não utilizada". Assim, basta criar um novo vetor e mover os elementos
let mut new = Vec::with_capacity(new_capacity);
let after_gap = self.capacity() - self.gap.end;
let new_gap = self.gap.start .. new.capacity() - after_gap;

unsafe {
    // Move os elementos que entram antes da lacuna
    std::ptr::copy_nonoverlapping(self.space(0),
                                   new.as_mut_ptr(),
                                   self.gap.start);

    // Move os elementos que entram após a lacuna
    let new_gap_end = new.as_mut_ptr().offset(new_gap.end as isize);
    std::ptr::copy_nonoverlapping(self.space(self.gap.end),
                                   new_gap_end,
                                   after_gap);
}

// Isso libera o Vec antigo, mas não dropa nenhum
// elemento, porque o comprimento do Vec é zero
self.storage = new;
self.gap = new_gap;
}

```

Embora `set_position` deva utilizar `copy` para mover elementos para frente e para trás na lacuna, `enlarge_gap` pode utilizar `copy_nonoverlapping`, pois está movendo elementos para um buffer totalmente novo.

Mover o novo vetor para `self.storage` dropa o vetor antigo. Como o comprimento é zero, o vetor antigo acredita que não tem elementos para dropar e simplesmente libera o buffer. De forma impecável, `copy_nonoverlapping` deixa sua fonte não inicializada, então o vetor antigo está correto nessa crença: todos os elementos agora são possuídos pelo novo vetor.

Por fim, precisamos ter certeza de que dropar um `GapBuffer` dropa todos os seus elementos:

```

impl<T> Drop for GapBuffer<T> {
    fn drop(&mut self) {
        unsafe {
            for i in 0 .. self.gap.start {
                std::ptr::drop_in_place(self.space_mut(i));
            }
        }
    }
}

```

```

    }
    for i in self.gap.end .. self.capacity() {
        std::ptr::drop_in_place(self.space_mut(i));
    }
}
}
}

```

Os elementos estão antes e depois da lacuna, então iteramos por cada região e utilizamos a função `std::ptr::drop_in_place` para dropar cada um. A função `drop_in_place` é um utilitário que se comporta como `drop(std::ptr::read(ptr))`, mas não se preocupa em mover o valor para o chamador (e, portanto, funciona em tipos não dimensionados). E, assim como em `enlarge_gap`, no momento em que o vetor `self.storage` é dropado, seu buffer realmente não está inicializado.

Como os outros tipos que mostramos neste capítulo, `GapBuffer` garante que suas próprias invariantes sejam suficientes para assegurar que o contrato de cada recurso não seguro que ele utiliza seja seguido, portanto nenhum de seus métodos públicos precisa ser marcado como não seguro. `GapBuffer` implementa uma interface segura para um recurso que não pode ser escrito de forma eficiente em código seguro.

Segurança contra pânico em código inseguro

No Rust, pânicos geralmente não podem causar um comportamento indefinido; a macro `panic!` não é um recurso inseguro. Mas, quando você decide trabalhar com código inseguro, a segurança contra pânico torna-se parte do seu trabalho.

Considere o método `GapBuffer::remove` da seção anterior:

```

pub fn remove(&mut self) -> Option<T> {
    if self.gap.end == self.capacity() {
        return None;
    }

    let element = unsafe {
        std::ptr::read(self.space(self.gap.end))
    };
    self.gap.end += 1;
}

```

```
    Some(element)
}
```

A chamada para `read` move o elemento imediatamente após a lacuna para fora do buffer, deixando para trás o espaço não inicializado. Nesse ponto, o `GapBuffer` está em um estado inconsistente: quebramos a invariante de que todos os elementos fora da lacuna devem ser inicializados. Felizmente, a próxima instrução aumenta a lacuna para abranger esse espaço, então, quando retornamos, a invariante é mantida novamente.

Mas considere o que aconteceria se, após a chamada para `read`, mas antes do ajuste em `self.gap.end`, esse código tentasse utilizar um recurso que pode gerar um pânico – digamos, indexar uma fatia. Sair do método abruptamente em qualquer lugar entre essas duas ações deixaria o `GapBuffer` com um elemento não inicializado fora da lacuna. A próxima chamada para `remove` poderia tentar `read` de novo; mesmo simplesmente dropar o `GapBuffer` tentaria dropá-lo. Ambos são comportamentos indefinidos, porque acessam a memória não inicializada.

É praticamente inevitável que os métodos de um tipo relaxem momentaneamente as invariantes do tipo enquanto eles fazem o trabalho e, em seguida, coloquem tudo de volta em ordem antes de retornarem. Um método intermediário de pânico pode interromper esse processo de limpeza, deixando o tipo em um estado inconsistente.

Se o tipo utiliza apenas código seguro, essa inconsistência pode fazer com que o tipo se comporte mal, mas não pode introduzir um comportamento indefinido. Mas o código que utiliza recursos inseguros geralmente conta com suas invariantes para respeitar os contratos desses recursos. Invariantes quebradas levam a contratos quebrados, que levam a um comportamento indefinido.

Ao trabalhar com recursos inseguros, você deve tomar cuidado especial para identificar essas regiões sensíveis do código em que as invariantes permanecem temporariamente relaxadas e garantir que não façam nada que possa gerar um pânico.

Reinterpretando a memória com uniões

O Rust fornece muitas abstrações úteis, mas, em última análise, o software que você escreve está simplesmente manipulando bytes. Uniões são um dos recursos mais poderosos do Rust para manipular esses bytes e escolher como eles são interpretados. Por exemplo, qualquer coleção de 32 bits – 4 bytes – pode ser interpretada como um número inteiro ou como um número de ponto flutuante. Qualquer uma das interpretações é válida, embora a interpretação dos dados destinados a uma como a outra provavelmente resulte em um absurdo.

Uma união representando uma coleção de bytes que pode ser interpretada como um número inteiro ou um número de ponto flutuante seria escrita da seguinte forma:

```
union FloatOrInt {  
    f: f32,  
    i: i32,  
}
```

Isso é uma união com dois campos, `f` e `i`. Eles podem ser atribuídos exatamente como os campos de um struct, mas, ao construir uma união, ao contrário de um struct, você deve escolher exatamente um. Da mesma forma que os campos de um struct se referem a diferentes posições na memória, os campos de uma união referenciam diferentes interpretações da mesma sequência de bits. Atribuir a um campo diferente significa simplesmente sobrescrever alguns ou todos esses bits, de acordo com um tipo apropriado. Aqui, `one` referencia um único intervalo de memória de 32 bits, que primeiro armazena 1 codificado como um inteiro simples, então 1.0 como um número de ponto flutuante IEEE 754. Assim que `f` é escrito, o valor previamente gravado no `FloatOrInt` é substituído:

```
let mut one = FloatOrInt { i: 1 };  
assert_eq!(unsafe { one.i }, 0x00_00_00_01);  
one.f = 1.0;  
assert_eq!(unsafe { one.i }, 0x3F_80_00_00);
```

Pela mesma razão, o tamanho de uma união é determinado por seu maior campo. Por exemplo, essa união tem 64 bits de tamanho, embora `SmallOrLarge::s` seja apenas um `bool`:

```
union SmallOrLarge {  
  s: bool,  
  l: u64  
}
```

Embora a construção de uma união ou a atribuição a seus campos seja totalmente segura, a leitura de qualquer campo de uma união é sempre insegura:

```
let u = SmallOrLarge { l: 1337 };  
println!("{}", unsafe {u.l}); // imprime 1337
```

Isso ocorre porque, ao contrário de enums, uniões não têm uma tag. O compilador não adiciona bits extras para diferenciar as variantes. Não há como saber em tempo de execução se um `SmallOrLarge` deve ser interpretado como um `u64` ou um `bool`, a menos que o programa tenha algum contexto extra.

Também não há garantia interna de que o padrão de bits de um determinado campo seja válido. Por exemplo, escrever em um campo `l` do valor `SmallOrLarge` substituirá seu campo `s`, criando um padrão de bits que definitivamente não significa nada útil e provavelmente não é `bool` válido. Portanto, embora a escrita em campos de união seja segura, toda leitura requer `unsafe`. Ler de `u.s` é permitido somente quando os bits do campo `s` formam um `bool` válido; caso contrário, isso é um comportamento indefinido.

Com essas restrições em mente, as uniões podem ser uma maneira útil de reinterpretar temporariamente alguns dados, especialmente ao fazer cálculos na representação de valores em vez dos próprios valores. Por exemplo, o tipo `FloatOrInt` mencionado anteriormente pode ser facilmente utilizado para imprimir os bits individuais de um número de ponto flutuante, embora `f32` não implemente o formatador `Binary`:

```
let float = FloatOrInt { f: 31337.0 };  
// imprime 100011011110100110100100000000  
println!("{}", unsafe { float.i });
```

Embora esses exemplos simples quase certamente funcionem como esperado em qualquer versão do compilador, não há garantia de que qualquer campo comece em um local específico, a menos que um atributo seja adicionado à definição da `union` informando ao

compilador como organizar os dados na memória. Adicionar o atributo `#[repr(C)]` garante que todos os campos comecem no deslocamento 0, em vez de qualquer lugar que o compilador quiser. Com essa garantia em vigor, o comportamento de substituição pode ser utilizado para extrair bits individuais, como o bit de sinal de um inteiro:

```
#[repr(C)]
union SignExtractor {
    value: i64,
    bytes: [u8; 8]
}

fn sign(int: i64) -> bool {
    let se = SignExtractor { value: int };
    println!( "{:b} ({}:?)", unsafe { se.value }, unsafe { se.bytes } );
    unsafe { se.bytes[7] >= 0b10000000 }
}

assert_eq!(sign(-1), true);
assert_eq!(sign(1), false);
assert_eq!(sign(i64::MAX), false);
assert_eq!(sign(i64::MIN), true);
```

Aqui, o bit de sinal é o bit mais significativo do byte mais significativo. Como os processadores x86 são little-endian, a ordem desses bytes é invertida; o byte mais significativo não é `bytes[0]`, mas `bytes[7]`. Normalmente, isso não é algo com o qual o código Rust tenha de lidar, mas como esse código está trabalhando diretamente com a representação na memória do `i64`, esses detalhes de baixo nível tornam-se importantes.

Como uniões não sabem como dropar o conteúdo, todos os seus campos devem ser `Copy`. Entretanto, se você simplesmente precisa armazenar um `String` em uma união, há uma solução alternativa; consulte na documentação da biblioteca padrão `std::mem::ManuallyDrop`.

Correspondendo uniões

A correspondência em uma união Rust é como a correspondência em um `struct`, exceto que cada padrão precisa especificar exatamente um campo:

```
unsafe {
```

```

match u {
    SmallOrLarge { s: true } => { println!("boolean true"); }
    SmallOrLarge { l: 2 } => { println!("integer 2"); }
    _ => { println!("something else"); }
}
}

```

Um braço `match` que corresponde a uma variante de união sem especificar um valor sempre será bem-sucedido. O código a seguir causará um comportamento indefinido se o último campo escrito de `u` era `u.i`:

```

// Comportamento indefinido!
unsafe {
    match u {
        FloatOrInt { f } => { println!("float {}", f) },
        // aviso: padrão inacessível
        FloatOrInt { i } => { println!("int {}", i) }
    }
}

```

Pegando emprestadas uniões

Pegar emprestado um campo de uma união pega emprestada toda a união. Isso significa que, seguindo as regras normais de empréstimo, tomar emprestado um campo como mutável impede qualquer empréstimo adicional nele ou em outros campos e tomar emprestado um campo como imutável significa que não pode haver empréstimos mutáveis em nenhum campo.

Como veremos no próximo capítulo, o Rust ajuda a construir interfaces seguras não apenas para seu próprio código inseguro, mas também para códigos escritos em outras linguagens. Inseguro é, como o nome indica, carregado, mas, se utilizado com cuidado, pode capacitá-lo a criar código de alto desempenho que retém as garantias que os programadores Rust desfrutam.

[1](#) Bem, é um clássico de onde viemos.

[2](#) Existem maneiras melhores de lidar com isso utilizando o tipo `RawVec` do `crate alloc` do `crate` interno do compilador, mas esse `crate` continua instável.

CAPÍTULO 23

Funções externas

Ciberespaço. Complexidade impensável. Linhas de luz variavam no não espaço da mente, aglomerados e constelações de dados. Como luzes da cidade, se afastando...

—William Gibson, Neuromancer

Tragicamente, nem todo programa no mundo é escrito no Rust. Existem muitas bibliotecas e interfaces cruciais implementadas em outras linguagens que gostaríamos de utilizar em nossos programas Rust. A interface de função externa (FFI) do Rust permite que o código Rust chame funções escritas em C e, em alguns casos, C++. Como a maioria dos sistemas operacionais oferece interfaces C, a interface de função externa do Rust permite acesso imediato a todos os tipos de recursos de baixo nível.

Neste capítulo, escreveremos um programa que vincula a `libgit2`, uma biblioteca do C para trabalhar com o sistema de controle de versão Git. Primeiro, mostraremos como é utilizar funções C diretamente do Rust, usando os recursos inseguros demonstrados no capítulo anterior. Em seguida, mostraremos como construir uma interface segura para `libgit2`, pegando inspiração no crate `git2-rs` de código aberto, que faz exatamente isso.

Vamos supor que você esteja familiarizado com C e a mecânica de compilar e vincular programas C. Trabalhar em C++ é semelhante. Também presumiremos que você esteja familiarizado com o sistema de controle de versão Git.

Existem crates Rust para comunicação com muitas outras linguagens, incluindo Python, JavaScript, Lua e Java. Não temos espaço para abordá-las aqui, mas, em última análise, todas essas interfaces são construídas utilizando a interface de função externa do C, portanto este capítulo deve lhe dar uma vantagem inicial

independentemente da linguagem com a qual você precisa trabalhar.

Encontrando representações de dados comuns

O denominador comum do Rust e C é linguagem de máquina; portanto, para prever como que os valores Rust se parecem no código C, ou vice-versa, você precisa considerar as representações no nível de máquina. Ao longo do livro, fizemos questão de mostrar como os valores são realmente representados na memória, então você provavelmente notou que os mundos de dados do C e Rust têm muito em comum: um `usize` do Rust e um `size_t` do C são idênticos, por exemplo, e structs são fundamentalmente a mesma ideia nas duas linguagens. Para estabelecer uma correspondência entre os tipos Rust e C, começaremos com tipos primitivos e depois passaremos para tipos mais complicados.

Dado seu uso primário como uma linguagem de programação de sistemas, o C sempre foi surpreendentemente flexível sobre as representações de seus tipos: um `int` normalmente tem 32 bits de comprimento, mas pode ser mais longo ou tão curto quanto 16 bits; um `char` do C pode ser com sinal ou sem sinal; e assim por diante. Para lidar com essa variabilidade, o módulo `std::os::raw` do Rust define um conjunto de tipos Rust que garantem ter a mesma representação que certos tipos C (Tabela 23.1). Eles abrangem os tipos inteiros primitivos e de tipos de caractere.

Tabela 23.1: Tipos `std::os::raw` no Rust

Tipo no C	<code>std::os::raw</code> typecorrespondente
short	<code>c_short</code>
int	<code>c_int</code>
long	<code>c_long</code>
long long	<code>c_longlong</code>
unsigned short	<code>c_ushort</code>
unsigned, unsigned int	<code>c_uint</code>
unsigned long	<code>c_ulong</code>

Tipo no C	std::os::raw typecorrespondente
unsigned long long	c_ulonglong
char	c_char
signed char	c_schar
unsigned char	c_uchar
float	c_float
double	c_double
void *, const void *	*mut c_void, *const c_void

Algumas observações sobre a Tabela 23.1:

- Exceto por `c_void`, todos os tipos Rust aqui são aliases para algum tipo Rust primitivo: `c_char`, por exemplo, é `i8` ou `u8`.
- Um `bool` Rust é equivalente a um `bool` C ou C++.
- O tipo `char` de 32 bits do Rust não é o análogo de `wchar_t`, cuja largura e codificação variam entre uma implementação e outra. O tipo `char32_t` do C é mais próximo, mas não se garante que sua codificação seja Unicode.
- Tipos `usize` e `isize` primitivos do Rust têm as mesmas representações que `size_t` e `ptrdiff_t` do C.
- Os ponteiros C e C++ e as referências C++ correspondem aos tipos de ponteiro bruto do Rust, `*mut T` e `*const T`.
- Tecnicamente, o padrão C permite que as implementações usem representações para as quais o Rust não tem um tipo correspondente: inteiros de 36 bits, representações de sinal e magnitude para valores com sinal e assim por diante. Na prática, em todas as plataformas para as quais o Rust foi portado, todo tipo inteiro C comum tem uma correspondência no Rust.

Para definir tipos de struct Rust compatíveis com structs C, você pode utilizar o atributo `#[repr(C)]`. Inserir `#[repr(C)]` acima de uma definição de struct solicita que o Rust disponha os campos do struct na memória da mesma forma como um compilador C disporia o tipo de struct C análogo. Por exemplo, o arquivo de cabeçalho *git2/errors.h* de `libgit2` define o seguinte struct C para fornecer detalhes sobre um erro relatado anteriormente:

```
typedef struct {
```

```

    char *message;
    int klass;
} git_error;

```

Você pode definir um tipo Rust com uma representação idêntica como a seguir:

```

use std::os::raw::{c_char, c_int};

#[repr(C)]
pub struct git_error {
    pub message: *const c_char,
    pub klass: c_int
}

```

O atributo `#[repr(C)]` só afeta o layout do próprio struct, não as representações dos campos individuais; portanto, para corresponder ao struct C, cada campo também deve utilizar o tipo semelhante a C: `*const c_char` para `char *`, `c_int` para `int` e assim por diante.

Nesse caso específico, o atributo `#[repr(C)]` provavelmente não altera o layout de `git_error`. Na verdade, não há muitas maneiras interessantes de dispor um ponteiro e um inteiro. Mas, embora o C e C++ garantam que os membros de uma estrutura apareçam na memória na ordem em que são declarados, cada um em um endereço distinto, o Rust reordena os campos para minimizar o tamanho geral do struct e os tipos de tamanho zero não ocupam espaço. O atributo `#[repr(C)]` diz ao Rust que siga as regras do C para o tipo fornecido.

Você também pode utilizar `#[repr(C)]` para controlar a representação de enums no estilo C:

```

#[repr(C)]
#[allow(non_camel_case_types)]
enum git_error_code {
    GIT_OK      = 0,
    GIT_ERROR   = -1,
    GIT_ENOTFOUND = -3,
    GIT_EEXISTS  = -4,
    ...
}

```

Normalmente, o Rust joga todos os tipos de jogos ao escolher como representar enums. Por exemplo, mencionamos o truque que o Rust utiliza para armazenar `Option<T>` em uma única palavra (se `T` estiver

dimensionado). Sem `#[repr(C)]`, o Rust utilizaria um único byte para representar o enum `git_error_code`; com `#[repr(C)]`, o Rust utiliza um valor do tamanho de um `int` do C, assim como o C usaria.

Você também pode solicitar ao Rust que dê a um enum a mesma representação de algum tipo inteiro. Iniciar a definição anterior com `#[repr(i16)]` forneceria um tipo de 16 bits com a mesma representação que o enum C++ a seguir:

```
#include <stdint.h>

enum git_error_code: int16_t {
    GIT_OK      = 0,
    GIT_ERROR   = -1,
    GIT_ENOTFOUND = -3,
    GIT_EEXISTS  = -4,
    ...
};
```

Como mencionado anteriormente, `#[repr(C)]` aplica-se também a uniões. Campos das uniões `#[repr(C)]` sempre começam no primeiro bit da memória da união – índice 0.

Suponha que você tenha um struct C que utiliza uma união para armazenar alguns dados e um valor de tag para indicar qual campo da união deve ser utilizado, semelhante a um enum Rust.

```
enum tag {
    FLOAT = 0,
    INT   = 1,
};

union number {
    float f;
    short i;
};

struct tagged_number {
    tag t;
    number n;
};
```

O código Rust pode interoperar com essa estrutura aplicando `#[repr(C)]` aos tipos enum, struct e union e utilizando uma instrução `match` que seleciona um campo de união dentro de uma estrutura maior com base no tag:

```
#[repr(C)]
```

```

enum Tag {
    Float = 0,
    Int = 1
}

#[repr(C)]
union FloatOrInt {
    f: f32,
    i: i32,
}

#[repr(C)]
struct Value {
    tag: Tag,
    union: FloatOrInt
}

fn is_zero(v: Value) -> bool {
    use self::Tag::*;
    unsafe {
        match v {
            Value { tag: Int, union: FloatOrInt { i: 0 } } => true,
            Value { tag: Float, union: FloatOrInt { f: num } } => (num == 0.0),
            _ => false
        }
    }
}

```

Mesmo estruturas complexas podem ser facilmente utilizadas dentro dos limites do FFI usando essa técnica.

Passar strings entre o Rust e o C é um pouco mais difícil. O C representa uma string como um ponteiro para um array de caracteres, terminado com um caractere nulo. O Rust, por outro lado, armazena o comprimento de uma string explicitamente, seja como um campo de um `String` ou como a segunda palavra de uma referência gorda `&str`. Strings Rust não são terminadas em nulo; na verdade, elas podem incluir caracteres nulos no conteúdo, como qualquer outro caractere.

Isso significa que você não pode pegar emprestada uma string Rust como uma string C: se você passar um ponteiro de código C para uma string Rust, ele poderá confundir um caractere nulo incorporado com o final da string ou sair do final procurando por uma terminação nula que não está lá. Por outro lado, você pode conseguir pegar emprestada uma string C como um `&str` Rust, desde que o conteúdo seja UTF-8 bem formado.

Essa situação efetivamente força Rust a tratar strings C como tipos totalmente distintos de `String` e `&str`. No módulo `std::ffi`, os tipos `CString` e `CStr` representam arrays proprietários e emprestados terminados com um byte nulo. Comparado com `String` e `str`, os métodos em `CString` e `CStr` são bastante limitados, restritos à construção e conversão em outros tipos. Mostraremos esses tipos em ação na próxima seção.

Declarando funções e variáveis externas

Um bloco `extern` declara funções ou variáveis definidas em alguma outra biblioteca com a qual o executável Rust final será vinculado (lincado). Por exemplo, na maioria das plataformas, cada programa Rust é vinculado à biblioteca C padrão, portanto podemos informar ao Rust sobre a função `strlen` da biblioteca C assim:

```
use std::os::raw::c_char;

extern {
    fn strlen(s: *const c_char) -> usize;
}
```

Isso fornece ao Rust o nome e o tipo da função, deixando a definição para ser vinculada posteriormente.

O Rust supõe que funções declaradas dentro de blocos `extern` usam convenções do C para passar argumentos e aceitar valores de retorno. Elas são definidos como funções `unsafe`. Essas são as escolhas certas para `strlen`: é de fato uma função C e sua especificação no C requer que você passe um ponteiro válido para uma string devidamente terminada, que é um contrato que o Rust não pode impor. (Quase qualquer função que recebe um ponteiro bruto deve ser `unsafe`: Rust seguro pode construir ponteiros brutos de números inteiros arbitrários e desreferenciar tal ponteiro seria um comportamento indefinido.)

Com esse bloco `extern`, podemos chamar `strlen` como qualquer outra função do Rust, embora seu tipo o revele como um turista:

```
use std::ffi::CString;

let rust_str = "I'll be back";
let null_terminated = CString::new(rust_str).unwrap();
unsafe {
    assert_eq!(strlen(null_terminated.as_ptr()), 12);
}
```

```
}
```

A função `CString::new` cria uma string C terminada em nulo. Ela primeiro verifica no argumento caracteres nulos embutidos, já que eles não podem ser representados em uma string C, e retorna um erro se encontrar algum (daí a necessidade de `unwrap` no resultado). Caso contrário, adiciona um byte nulo ao final e retorna um `CString` possuindo os caracteres resultantes.

O custo de `CString::new` depende de que tipo você passa. Aceita qualquer coisa que implementa `Into<Vec<u8>>`. Passar uma `&str` implica uma alocação e uma cópia, uma vez que a conversão em `Vec<u8>` constrói uma cópia alocada em heap da string para o vetor possuir. Mas passar um `String` por valor simplesmente consome a string e assume o buffer; portanto, a menos que anexar o caractere nulo force o redimensionamento do buffer, a conversão não requer absolutamente nenhuma cópia de texto ou alocação.

`CString` desreferência a `CStr`, cujo método `as_ptr` retorna um `*const c_char` apontando para o início da string. Esse é o tipo que `strlen` espera. No exemplo, `strlen` percorre a string, encontra o caractere nulo que `CString::new` inseriu aí e retorna o comprimento, como uma contagem de bytes.

Você também pode declarar variáveis globais em blocos `extern`. Os sistemas POSIX têm uma variável global chamada `environ` que contém os valores das variáveis de ambiente do processo. No C é declarada:

```
extern char **environ;
```

No Rust, você diria:

```
use std::ffi::CStr;
use std::os::raw::c_char;
extern {
    static environ: *mut *mut c_char;
}
```

Para imprimir o primeiro elemento do ambiente, você pode escrever:

```
unsafe {
    if !environ.is_null() && !(*environ).is_null() {
        let var = CStr::from_ptr(*environ);
        println!("first environment variable: {}",
            var.to_string_lossy())
    }
}
```



```
}
```

Depois de certificar-se de que `environ` tem um primeiro elemento, o código chama `CStr::from_ptr` para construir um `CStr` que o empresta. O método `to_string_lossy` retorna um `Cow<str>`: se a string C contém UTF-8 bem formado, `Cow` pega emprestado o conteúdo como um `&str`, não incluindo o byte nulo de término. Por outro lado, `to_string_lossy` cria uma cópia do texto no heap, substitui as sequências UTF-8 malformadas pelo caractere de substituição Unicode oficial, e cria um `Cow` proprietário a partir disso. De qualquer forma, o resultado implementa `Display`, para que você possa imprimi-lo com o parâmetro de formato `{}`.

Usando funções de bibliotecas

Para utilizar as funções fornecidas por uma biblioteca específica, você pode colocar um atributo `#[link]` na parte superior do bloco `extern` que nomeia a biblioteca com a qual o Rust deve vincular o executável. Por exemplo, eis um programa que chama métodos de inicialização e desligamento de `libgit2`, mas não faz mais nada:

```
use std::os::raw::c_int;

#[link(name = "git2")]
extern {
    pub fn git_libgit2_init() -> c_int;
    pub fn git_libgit2_shutdown() -> c_int;
}

fn main() {
    unsafe {
        git_libgit2_init();
        git_libgit2_shutdown();
    }
}
```

O bloco `extern` declara as funções externas como antes. O atributo `#[link(name = "git2")]` deixa uma nota no crate informando que, quando o Rust cria o executável final ou a biblioteca compartilhada, ele deve estar vinculado à biblioteca `git2`. O Rust utiliza o linker do sistema para construir executáveis; no Unix, isso passa o argumento `-lgit2` na linha de comando do linker; no Windows passa `git2.LIB`.

Os atributos `#[link]` também funcionam nos crates de biblioteca. Ao criar um programa que depende de outros crates, o Cargo reúne as notas de link de todo o grafo de dependência e as inclui no link final. Nesse exemplo, se você quiser acompanhar em sua própria máquina, precisará construir `libgit2` para você mesmo. Utilizamos `libgit2` (<https://oreil.ly/T1dPr>), versão 0.25.1. Para compilar `libgit2`, você precisará instalar a ferramenta de compilação CMake e a linguagem Python; utilizamos CMake (<https://cmake.org>) versão 3.8.0 e Python (<https://www.python.org>) versão 2.7.13.

As instruções completas para criar `libgit2` estão disponíveis no site, mas são tão simples que mostraremos aqui o essencial. No Linux, suponha que você já descompactou o código-fonte da biblioteca no diretório `/home/jimb/libgit2-0.25.1`:

```
$ cd /home/jimb/libgit2-0.25.1
$ mkdir build
$ cd build
$ cmake ..
$ cmake --build .
```

No Linux, isso produz uma biblioteca compartilhada `/home/jimb/libgit2-0.25.1/build/libgit2.so.0.25.1` com o aninhamento usual de links simbólicos apontando para ele, incluindo um nomeado `libgit2.so`. No macOS, os resultados são semelhantes, mas a biblioteca é nomeada `libgit2.dylib`.

No Windows, as coisas também são simples. Suponha que você descompactou o código-fonte no diretório `C:\Users\JimB\libgit2-0.25.1`. Em um prompt de comando do Visual Studio:

```
> cd C:\Users\JimB\libgit2-0.25.1
> mkdir build
> cd build
> cmake -A x64 ..
> cmake --build .
```

Esses são os mesmos comandos utilizados no Linux, exceto que você deve solicitar uma compilação de 64 bits ao executar o CMake na primeira vez para corresponder ao seu compilador do Rust. (Se você instalou o conjunto de ferramentas Rust de 32 bits, deverá omitir a flag `-A x64` para o primeiro comando `cmake`.) Isso produz uma biblioteca de importação `git2.LIB` e uma biblioteca de links dinâmicos

git2.dll, ambas no diretório *C:\Users\JimB\libgit2-0.25.1\build\Debug*. (As instruções restantes são mostradas para Unix, exceto onde no Windows é substancialmente diferente.)

Crie o programa Rust em um diretório separado:

```
$ cd /home/jimb
$ cargo new --bin git-toy
Created binary (application) `git-toy` package
```

Pegue o código mostrado anteriormente e coloque-o em *src/main.rs*. Naturalmente, se tentar construir isso, o Rust não fará ideia de onde encontrar o *libgit2* que você construiu:

```
$ cd git-toy
$ cargo run
Compiling git-toy v0.1.0 (/home/jimb/git-toy)
error: linking with `cc` failed: exit status: 1
|
= note: /usr/bin/ld: error: cannot find -lgit2
src/main.rs:11: error: undefined reference to 'git_libgit2_init'
src/main.rs:12: error: undefined reference to 'git_libgit2_shutdown'
collect2: error: ld returned 1 exit status

error: could not compile `git-toy` due to previous error
```

Você pode dizer ao Rust onde procurar por bibliotecas escrevendo um script de compilação, código Rust que o Cargo compila e executa em tempo de compilação. Os scripts de compilação podem fazer todo tipo de coisas: gerar código dinamicamente, compilar código C para ser incluído no crate e assim por diante. Nesse caso, tudo o que você precisa é adicionar um caminho de pesquisa de biblioteca ao comando de link do executável. Quando o Cargo executa o script de compilação, ele analisa a saída do script para obter informações desse tipo, de modo que o script de compilação simplesmente precisa imprimir a mágica certa em sua saída padrão.

Para criar seu script de compilação, adicione um arquivo chamado *build.rs* no mesmo diretório do arquivo *Cargo.toml*, com o seguinte conteúdo:

```
fn main() {
    println!(r"cargo:rustc-link-search=native=/home/jimb/libgit2-0.25.1/build");
}
```

Esse é o caminho correto para o Linux; no Windows, você mudaria o caminho depois do texto `native=` para *C:\Users\JimB\libgit2-0.25.1\build\Debug*.

(Estamos cortando alguns cantos para manter esse exemplo simples; em um aplicativo real, você deve evitar o uso de paths absolutos em seu script de compilação. Citamos a documentação que mostra como fazer isso no final desta seção.)

Agora você quase pode executar o programa. No macOS, pode funcionar imediatamente; em um sistema Linux, você provavelmente verá algo como o seguinte:

```
$ cargo run
  Compiling git-toy v0.1.0 (/tmp/rustbook-transcript-tests/git-toy)
  Finished dev [unoptimized + debuginfo] target(s)
  Running `target/debug/git-toy`
target/debug/git-toy: error while loading shared libraries:
libgit2.so.25: cannot open shared object file: No such file or directory
```

Isso significa que, embora o Cargo tenha conseguido vincular o executável à biblioteca, ele não sabe onde encontrar a biblioteca compartilhada em tempo de execução. O Windows relata essa falha exibindo uma caixa de diálogo. No Linux, você deve definir a variável de ambiente `LD_LIBRARY_PATH`:

```
$ export LD_LIBRARY_PATH=/home/jimb/libgit2-0.25.1/build:$LD_LIBRARY_PATH
$ cargo run
  Finished dev [unoptimized + debuginfo] target(s) in 0.0 secs
  Running `target/debug/git-toy`
```

No macOS, em vez disso, pode ser necessário definir `DYLD_LIBRARY_PATH`.

No Windows, você deve definir a variável de ambiente `PATH`:

```
> set PATH=C:\Users\JimB\libgit2-0.25.1\build\Debug;%PATH%
> cargo run
  Finished dev [unoptimized + debuginfo] target(s) in 0.0 secs
  Running `target/debug/git-toy`
>
```

Naturalmente, em um aplicativo que será usado em outros lugares, você deseja evitar a necessidade de definir variáveis de ambiente apenas para localizar o código de sua biblioteca. Uma alternativa é vincular estaticamente a biblioteca C ao crate. Isso copia os arquivos de objeto da biblioteca para o arquivo `.rlib`, com os arquivos de objeto e metadados para o código Rust do crate. Toda a coleção então participa do link final.

É uma convenção do Cargo que um crate que fornece acesso a uma biblioteca C deve ser nomeado `LIB-sys`, em que `LIB` é o nome da biblioteca C. Um crate `-sys` não deve conter nada além da biblioteca vinculada estaticamente e os módulos Rust contendo blocos `extern` e definições de tipo. As interfaces de nível superior pertencem então aos crates que dependem do crate `-sys`. Isso permite que vários crates upstream dependam do mesmo crate `-sys`, supondo que haja uma única versão do crate `-sys` que atende às necessidades de todos. Para obter todos os detalhes sobre o suporte do Cargo para criar scripts e vincular com bibliotecas do sistema, consulte a documentação on-line do Cargo (<https://oreil.ly/Rxa1D>). Ela mostra como evitar caminhos absolutos em scripts de construção, controlar flags de compilação, utilizar ferramentas como `pkg-config` e assim por diante. O crate `git2-rs` também fornece bons exemplos para emular; o script de construção (`build`) lida com algumas situações complexas.

Uma interface bruta para libgit2

Descobrir como utilizar `libgit2` corretamente exige duas perguntas:

- O que é preciso para utilizar funções `libgit2` no Rust?
- Como podemos construir uma interface Rust segura em torno delas?

Responderemos a essas perguntas uma de cada vez. Nesta seção, escreveremos um programa que é essencialmente um único bloco `unsafe` gigante preenchido com código Rust não idiomático, refletindo o choque de sistemas de tipos e convenções inerentes à mistura de linguagens. Chamaremos isso interface bruta. O código ficará confuso, mas deixará claro todas as etapas que devem ocorrer para que o código em Rust use a `libgit2`.

Então, na próxima seção, construiremos uma interface segura para `libgit2` que coloca os tipos do Rust em uso aplicando as regras que `libgit2` impõe aos usuários. Felizmente, `libgit2` é uma biblioteca C excepcionalmente bem projetada, portanto as perguntas que os requisitos de segurança do Rust nos forçam a fazer têm respostas

muito boas, e podemos construir uma interface Rust idiomática sem funções `unsafe`.

O programa que escreveremos é muito simples: ele pega um caminho como um argumento de linha de comando, abre o repositório Git e imprime o head commit. Mas isso é suficiente para ilustrar as principais estratégias para construir interfaces Rust seguras e idiomáticas.

Para a interface bruta, o programa acabará precisando de uma coleção um pouco maior de funções e tipos de `libgit2` do que utilizamos antes, assim faz sentido mover o bloco `extern` para um módulo próprio. Vamos criar um arquivo chamado *raw.rs* no *git-toy/src* cujo conteúdo é como a seguir:

```
#![allow(non_camel_case_types)]

use std::os::raw::{c_int, c_char, c_uchar};

#[link(name = "git2")]
extern {
    pub fn git_libgit2_init() -> c_int;
    pub fn git_libgit2_shutdown() -> c_int;
    pub fn giterr_last() -> *const git_error;

    pub fn git_repository_open(out: *mut *mut git_repository,
                               path: *const c_char) -> c_int;
    pub fn git_repository_free(repo: *mut git_repository);
    pub fn git_reference_name_to_id(out: *mut git_oid,
                                     repo: *mut git_repository,
                                     reference: *const c_char) -> c_int;

    pub fn git_commit_lookup(out: *mut *mut git_commit,
                              repo: *mut git_repository,
                              id: *const git_oid) -> c_int;

    pub fn git_commit_author(commit: *const git_commit) -> *const git_signature;
    pub fn git_commit_message(commit: *const git_commit) -> *const c_char;
    pub fn git_commit_free(commit: *mut git_commit);
}

#[repr(C)] pub struct git_repository { _private: [u8; 0] }
#[repr(C)] pub struct git_commit { _private: [u8; 0] }

#[repr(C)]
pub struct git_error {
    pub message: *const c_char,
```

```

    pub klass: c_int
}

pub const GIT_OID_RAWSZ: usize = 20;

#[repr(C)]
pub struct git_oid {
    pub id: [c_uchar; GIT_OID_RAWSZ]
}

pub type git_time_t = i64;

#[repr(C)]
pub struct git_time {
    pub time: git_time_t,
    pub offset: c_int
}

#[repr(C)]
pub struct git_signature {
    pub name: *const c_char,
    pub email: *const c_char,
    pub when: git_time
}

```

Cada item aqui é modelado a partir das declarações de arquivos de cabeçalho (header) da própria libgit2. Por exemplo, *libgit2-0.25.1/include/git2/repository.h* inclui esta declaração:

```
extern int git_repository_open(git_repository **out, const char *path);
```

Essa função tenta abrir o repositório Git em `path`. Se tudo der certo, ele cria um objeto `git_repository` e armazena um ponteiro para ele no local apontado por `out`. A declaração Rust equivalente é a seguinte:

```
pub fn git_repository_open(out: *mut *mut git_repository,
    path: *const c_char) -> c_int;
```

Os arquivos de cabeçalho públicos libgit2 definem o tipo `git_repository` como um typedef para um tipo de struct incompleto:

```
typedef struct git_repository git_repository;
```

Como os detalhes desse tipo são privados para a biblioteca, os cabeçalhos públicos nunca definem `struct git_repository`, garantindo que os usuários da biblioteca nunca possam criar uma instância desse tipo. Um análogo possível a um tipo de struct incompleto no Rust é:

```
#[repr(C)] pub struct git_repository { _private: [u8; 0] }
```

Isso é um tipo de struct contendo um array sem elementos. Como o campo `_private` não é `pub`, valores desse tipo não podem ser

construídos fora desse módulo, o que é perfeito como o reflexo de um tipo C que só a `libgit2` deve construir, e que é manipulado somente por meio de ponteiros brutos.

Escrever grande blocos `extern` à mão pode ser uma tarefa árdua. Se você estiver criando uma interface Rust para uma biblioteca C complexa, tente utilizar o crate `bindgen`, que tem funções que podem ser utilizadas em seu script de build para analisar arquivos de cabeçalho C e gerar as declarações Rust correspondentes automaticamente. Não temos espaço para mostrar `bindgen` em ação aqui, mas a página de `bindgen` em [crates.io](https://crates.io/crates/bindgen) (crates.io/crates/bindgen) inclui links para a documentação.

Em seguida, vamos reescrever `main.rs` completamente. Primeiro, precisamos declarar o módulo `raw`:

```
mod raw;
```

De acordo com as convenções de `libgit2`, as funções que podem falhar retornam um código inteiro que é positivo ou zero em caso de sucesso e negativo em caso de falha. Se ocorrer um erro, a função `giterr_last` vai retornar um ponteiro para uma estrutura `git_error` fornecendo mais detalhes sobre o que está errado. `libgit2` possui essa estrutura, assim não precisamos liberá-la, mas ela pode ser sobrescrita na próxima chamada de biblioteca que fizermos. Uma interface Rust adequada utilizaria `Result`, mas na versão bruta, queremos utilizar as funções `libgit2` exatamente como estão, então teremos de rodar nossa própria função para lidar com erros:

```
use std::ffi::CStr;
use std::os::raw::c_int;

fn check(activity: &'static str, status: c_int) -> c_int {
    if status < 0 {
        unsafe {
            let error = &*raw::giterr_last();
            println!("error while {}: {} ({})",
                    activity,
                    CStr::from_ptr(error.message).to_string_lossy(),
                    error.class);
            std::process::exit(1);
        }
    }
}
```



```
    status
}
```

Utilizaremos essa função para verificar os resultados das chamadas `libgit2` como a seguir:

```
check("initializing library", raw::git_libgit2_init());
```

Isso utiliza os mesmos métodos `CStr` usados anteriormente: `from_ptr` para construir o `CStr` de uma string C e `to_string_lossy` para transformar isso em algo que o Rust possa imprimir.

Em seguida, precisamos de uma função para imprimir um commit:

```
unsafe fn show_commit(commit: *const raw::git_commit) {
    let author = raw::git_commit_author(commit);

    let name = CStr::from_ptr((*author).name).to_string_lossy();
    let email = CStr::from_ptr((*author).email).to_string_lossy();
    println!("{}", <{}>\n", name, email);

    let message = raw::git_commit_message(commit);
    println!("{}", CStr::from_ptr(message).to_string_lossy());
}
```

Dado um ponteiro para um `git_commit`, `show_commit` chama `git_commit_author` e `git_commit_message` para recuperar as informações de que necessita. Essas duas funções seguem uma convenção de que a documentação `libgit2` explica o seguinte:

Se uma função retorna um objeto como um valor de retorno, essa função é um getter e o tempo de vida do objeto está vinculado ao objeto pai.

Em termos do Rust, `author` e `message` são emprestadas de `commit`: `show_commit` não precisa liberá-las, mas não deve segurá-las depois que `commit` for liberado. Como essa API utiliza ponteiros brutos, o Rust não verificará os tempos de vida para nós: se criarmos acidentalmente ponteiros perdidos, provavelmente só descobriremos isso depois de o programa falhar.

O código anterior supõe que esses campos contêm texto UTF-8, o que nem sempre é correto. O Git também permite outras codificações. Interpretar essas strings corretamente provavelmente envolveria o uso do `crate encoding`. Por questão de brevidade, vamos minimizar essas questões aqui.

A função `main` do nosso programa é como a seguir:

```
use std::ffi::CString;
use std::mem;
use std::ptr;
use std::os::raw::c_char;

fn main() {
    let path = std::env::args().skip(1).next()
        .expect("usage: git-toy PATH");
    let path = CString::new(path)
        .expect("path contains null characters");

    unsafe {
        check("initializing library", raw::git_libgit2_init());

        let mut repo = ptr::null_mut();
        check("opening repository",
            raw::git_repository_open(&mut repo, path.as_ptr()));

        let c_name = b"HEAD\0".as_ptr() as *const c_char;
        let oid = {
            let mut oid = mem::MaybeUninit::uninit();
            check("looking up HEAD",
                raw::git_reference_name_to_id(oid.as_mut_ptr(), repo, c_name));
            oid.assume_init()
        };

        let mut commit = ptr::null_mut();
        check("looking up commit",
            raw::git_commit_lookup(&mut commit, repo, &oid));

        show_commit(commit);

        raw::git_commit_free(commit);

        raw::git_repository_free(repo);

        check("shutting down library", raw::git_libgit2_shutdown());
    }
}
```

Isso começa com o código para lidar com o argumento de `path` e inicializar a biblioteca, os quais vimos antes. O primeiro código novo é:

```
let mut repo = ptr::null_mut();
check("opening repository",
    raw::git_repository_open(&mut repo, path.as_ptr()));
```

A chamada para `git_repository_open` tenta abrir o repositório Git no caminho fornecido. Se bem-sucedido, aloca um novo objeto

`git_repository` para ele e define `repo` para apontar para isso. O Rust implicitamente traduz referências em ponteiros brutos, então passar `&mut repo` aqui fornece o `*mut *mut git_repository` que a chamada espera.

Isso mostra outra convenção `libgit2` em uso (da documentação `libgit2`):

Os objetos que são retornados por meio do primeiro argumento como um ponteiro para ponteiro são possuídos pelo chamador e ele é responsável por liberá-los.

Em termos do Rust, funções como `git_repository_open` passam a posse do novo valor para o chamador.

Em seguida, considere o código que procura o objeto hash do commit principal atual do repositório:

```
let oid = {  
    let mut oid = mem::MaybeUninit::uninit();  
    check("looking up HEAD",  
        raw::git_reference_name_to_id(oid.as_mut_ptr(), repo, c_name));  
    oid.assume_init()  
};
```

O tipo `git_oid` armazena um identificador de objeto – um código hash de 160 bits que o Git utiliza internamente (e em toda a sua agradável interface de usuário) para identificar commits, versões individuais de arquivos etc. Essa chamada para `git_reference_name_to_id` procura o identificador de objeto do commit "HEAD" atual.

Em C é perfeitamente normal inicializar uma variável passando um ponteiro para ela para alguma função que preenche o valor; é assim que `git_reference_name_to_id` espera tratar o primeiro argumento. Mas o Rust não deixará pegar emprestada uma referência a uma variável não inicializada. Poderíamos inicializar `oid` com zeros, mas isso é um desperdício: qualquer valor armazenado aí será simplesmente sobrescrito.

É possível solicitar ao Rust que nos dê memória não inicializada, mas, como a leitura da memória não inicializada a qualquer momento é um comportamento indefinido instantâneo, o Rust fornece uma abstração, `MaybeUninit`, para facilitar seu uso. `MaybeUninit<T>` diz ao compilador que reserve memória suficiente para o tipo `T` e só a manipule depois de você informar que é seguro fazer

isso. Embora essa memória seja propriedade do `MaybeUninit`, o compilador também evitará certas otimizações que podem causar um comportamento indefinido mesmo sem qualquer acesso explícito à memória não inicializada em seu código.

`MaybeUninit` fornece um método, `as_mut_ptr()`, que produz um `*mut T` apontando para a memória potencialmente não inicializada que ela empacota. Passando esse ponteiro para uma função externa que inicializa a memória e, em seguida, chamando o método inseguro `assume_init` no `MaybeUninit` para produzir uma `T` totalmente inicializada, você pode evitar o comportamento indefinido sem a sobrecarga adicional resultante da inicialização e eliminação imediata de um valor. `assume_init` é inseguro porque o chamar em um `MaybeUninit` sem ter certeza de que a memória foi realmente inicializada, causará imediatamente um comportamento indefinido.

Nesse caso, é seguro porque `git_reference_name_to_id` inicializa a memória pertencente ao `MaybeUninit`. Poderíamos também utilizar `MaybeUninit` para as variáveis `repo` e `commit`, mas, como são apenas palavras únicas, vamos em frente e as inicializamos para nulo:

```
let mut commit = ptr::null_mut();
check("looking up commit",
    raw::git_commit_lookup(&mut commit, repo, &oid));
```

Isso seleciona o identificador de objeto do `commit` e procura o `commit` real, armazenando um ponteiro `git_commit` em `commit` no caso de sucesso.

O restante da função `main` deve ser autoexplicativo. Ela chama a função `show_commit` definida anteriormente, libera os objetos de `commit` e do repositório e fecha a biblioteca.

Agora podemos experimentar o programa em qualquer repositório Git disponível:

```
$ cargo run /home/jimb/rbattle
Finished dev [unoptimized + debuginfo] target(s) in 0.0 secs
Running `target/debug/git-toy /home/jimb/rbattle`
Jim Blandy <jimb@red-bean.com>
Animate goop a bit.
```

Uma interface segura para `libgit2`

A interface bruta para `libgit2` é um exemplo perfeito de um recurso inseguro: certamente pode ser utilizada corretamente (como fazemos aqui, até onde sabemos), mas o Rust não pode impor as regras que você deve seguir. Projetar uma API segura para uma biblioteca como essa é uma questão de identificar todas essas regras e, em seguida, encontrar maneiras de transformar qualquer violação delas em um tipo ou erro de verificação de empréstimos.

Eis, então, as regras de `libgit2` para os recursos que o programa utiliza:

- Você deve chamar `git_libgit2_init` antes de utilizar qualquer outra função da biblioteca. Você não deve utilizar nenhuma função de biblioteca após chamar `git_libgit2_shutdown`.
- Todos os valores passados para funções `libgit2` devem ser totalmente inicializados, exceto para parâmetros de saída.
- Quando uma chamada falha, os parâmetros de saída passados para conter os resultados da chamada são mantidos não inicializados, e você não deve utilizar os valores.
- Um objeto `git_commit` referencia o objeto `git_repository` do qual é derivado, então o primeiro não deve sobreviver ao último. (Isso não está escrito na documentação `libgit2`; nós inferimos isso da presença de certas funções na interface e depois verificamos lendo o código-fonte.)
- Da mesma forma, uma `git_signature` é sempre emprestada de um dado `git_commit`, e o primeiro não deve sobreviver ao último. (A documentação abrange esse caso.)
- A mensagem associada a um commit, o nome e endereço de e-mail do autor são todos emprestados do commit e não devem ser utilizados depois que o commit é liberado.
- Depois que objeto `libgit2` foi liberado, ele nunca deve ser utilizado novamente.

Acontece que você pode construir uma interface Rust para `libgit2` que impõe todas essas regras, seja por meio do sistema de tipos do Rust ou gerenciando detalhes internamente.

Antes de começarmos, vamos reestruturar um pouco o projeto.

Queremos ter um módulo `git` que exporte a interface segura, da qual a interface bruta do programa anterior é um submódulo privado.

Toda a árvore de origem se parecerá com isto:

```
git-toy/
├── Cargo.toml
├── build.rs
├── src/
│   ├── main.rs
│   └── git/
│       ├── mod.rs
│       └── raw.rs
```

Seguindo as regras que explicamos em “Módulos em arquivos separados”, na página 215, a origem do módulo `git` aparece em `git/mod.rs` e a origem do submódulo `git::raw` reside em `git/raw.rs`.

Mais uma vez, vamos reescrever `main.rs` completamente. Devemos começar com uma declaração do módulo `git`:

```
mod git;
```

Então, precisaremos criar o subdiretório `git` e inserir `raw.rs` nele:

```
$ cd /home/jimb/git-toy
$ mkdir src/git
$ mv src/raw.rs src/git/raw.rs
```

O módulo `git` precisa declarar seu submódulo `raw`. O arquivo `src/git/mod.rs` deve informar:

```
mod raw;
```

Como não é `pub`, esse submódulo não é visível para o programa principal.

Daqui a pouco precisaremos utilizar algumas funções do crate `libc`, então devemos adicionar uma dependência em `Cargo.toml`. O arquivo completo agora informa:

```
[package]
name = "git-toy"
version = "0.1.0"
authors = ["You <you@example.com>"]
edition = "2021"
[dependencies]
libc = "0.2"
```

Agora que reestruturamos nossos módulos, vamos considerar o tratamento de erros. Mesmo a função de inicialização de `libgit2` pode

retornar um código de erro, portanto precisaremos resolver isso antes de começar. Uma interface Rust idiomática precisa de seu próprio tipo `Error` que captura o código de falha `libgit2`, bem como a mensagem de erro e classe de `giterr_last`. Um tipo de erro adequado deve implementar os traits `Error`, `Debug` e `Display` usuais. Então, ele precisa de seu próprio tipo `Result` que utiliza o tipo `Error`. Eis as definições necessárias em *src/git/mod.rs*:

```
use std::error;
use std::fmt;
use std::result;

#[derive(Debug)]
pub struct Error {
    code: i32,
    message: String,
    class: i32
}

impl fmt::Display for Error {
    fn fmt(&self, f: &mut fmt::Formatter) -> result::Result<(), fmt::Error> {
        // Exibir um `Error` simplesmente exibe a mensagem de libgit2
        self.message.fmt(f)
    }
}

impl error::Error for Error { }

pub type Result<T> = result::Result<T, Error>;
```

Para verificar o resultado das chamadas de biblioteca bruta, o módulo precisa de uma função que transforma um código de retorno `libgit2` em um `Result`:

```
use std::os::raw::c_int;
use std::ffi::CStr;

fn check(code: c_int) -> Result<c_int> {
    if code >= 0 {
        return Ok(code);
    }

    unsafe {
        let error = raw::giterr_last();

        // libgit2 garante que (*error).message sempre termine em
        // não nulo e nulo, assim essa chamada é segura
        let message = CStr::from_ptr((*error).message)
            .to_string_lossy()
```

```

        .into_owned();
    Err(Error {
        code: code as i32,
        message,
        class: (*error).klass as i32
    })
}
}

```

A principal diferença entre isso e a função `check` da versão bruta é que constrói um valor `Error` em vez de imprimir uma mensagem de erro e sair imediatamente.

Agora estamos prontos para lidar com a inicialização da `libgit2`. A interface segura fornecerá um tipo `Repository` que representa um repositório Git aberto, com métodos para resolver referências, procurar commits etc. Continuando em *git/mod.rs*, eis a definição de `Repository`:

```

/// Um repositório Git.
pub struct Repository {
    // Isso sempre deve ser um ponteiro para uma estrutura `git_repository` ativa.
    // Nenhum outro `Repository` pode apontar para ele
    raw: *mut raw::git_repository
}

```

O campo `raw` de um `Repository` não é público. Como apenas o código neste módulo pode acessar o ponteiro `raw::git_repository`, a codificação correta desse módulo deve garantir que o ponteiro seja sempre utilizado corretamente.

A única maneira de criar um `Repository` é abrir com sucesso um novo repositório Git, o que garantirá que cada `Repository` aponte para um objeto `git_repository` distinto:

```

use std::path::Path;
use std::ptr;

impl Repository {
    pub fn open<P: AsRef<Path>>>(path: P) -> Result<Repository> {
        ensure_initialized();
        let path = path_to_cstring(path.as_ref())?;
        let mut repo = ptr::null_mut();
        unsafe {
            check(raw::git_repository_open(&mut repo, path.as_ptr()))?;
        }
    }
}

```



```

        Ok(Repository { raw: repo })
    }
}

```

Como a única maneira de fazer qualquer coisa com a interface segura é começar com um valor `Repository` e `Repository::open` inicia com uma chamada para `ensure_initialized`, podemos ter certeza de que `ensure_initialized` será chamada antes de quaisquer funções da `libgit2`. Sua definição é como a seguir:

```

fn ensure_initialized() {
    static ONCE: std::sync::Once = std::sync::Once::new();
    ONCE.call_once(|| {
        unsafe {
            check(raw::git_libgit2_init())
                .expect("initializing libgit2 failed");
            assert_eq!(libc::atexit(shutdown), 0);
        }
    });
}

extern fn shutdown() {
    unsafe {
        if let Err(e) = check(raw::git_libgit2_shutdown()) {
            eprintln!("shutting down libgit2 failed: {}", e);
            std::process::abort();
        }
    }
}

```

O tipo `std::sync::Once` ajuda a executar o código de inicialização de maneira thread-safe. Somente o primeiro thread a chamar `ONCE.call_once` executa a closure específica. Quaisquer chamadas subsequentes, por esse thread ou qualquer outro, são bloqueadas até que a primeira seja concluída e, em seguida, retornam imediatamente, sem executar a closure novamente. Depois que a closure concluiu, chamar `ONCE.call_once` é barato, exigindo nada mais do que uma leitura atômica de uma flag armazenada em `ONCE`.

No código anterior, a closure de inicialização chama `git_libgit2_init` e verifica o resultado. Ele aposta um pouco e utiliza `expect` para garantir que a inicialização seja bem-sucedida, em vez de tentar propagar os erros de volta ao chamador.

Para garantir que o programa chama `git_libgit2_shutdown`, a closure de

inicialização utiliza a função `atexit` da biblioteca C, que leva um ponteiro para uma função a ser invocada antes que o processo seja encerrado. Closures Rust não podem servir como ponteiros de função C: uma closure é um valor de algum tipo anônimo que transporta os valores de quaisquer variáveis que ela capture ou referencie; um ponteiro de função C é simplesmente um ponteiro. Contudo, os tipos `fn` do Rust funcionam bem, desde que você os declare `extern` para que o Rust saiba utilizar as convenções de chamada do C. A função local `shutdown` é adequada e garante que `libgit2` seja desligada adequadamente.

Em “Desempilhamento”, na página 189, mencionamos que é um comportamento indefinido que gera um pânico cruzar os limites da linguagem. A chamada de `atexit` para `shutdown` é esse limite, por isso é essencial que `shutdown` não entre em pânico. Essa é a razão pela qual `shutdown` simplesmente não pode utilizar `.expect` para lidar com erros relatados de `raw::git_libgit2_shutdown`. Em vez disso, ele deve relatar o erro e encerrar o processo. O POSIX proíbe chamar `exit` dentro de um manipulador `atexit`, então `shutdown` chama `std::process::abort` para encerrar o programa abruptamente.

Pode ser possível chamar `git_libgit2_shutdown` mais cedo – digamos, quando o último valor `Repository` é dropado. Mas, independentemente de como organizamos as coisas, chamar `git_libgit2_shutdown` deve ser responsabilidade da API segura. No momento em que é chamada, o uso de quaisquer objetos `libgit2` existentes se torna inseguro, assim a API segura não deve expor essa função diretamente.

Um ponteiro bruto de `Repository` sempre deve apontar para um objeto `git_repository` ativo. Isso implica que a única maneira de fechar um repositório é dropar o valor `Repository` que o possui:

```
impl Drop for Repository {
    fn drop(&mut self) {
        unsafe {
            raw::git_repository_free(self.raw);
        }
    }
}
```

Ao chamar `git_repository_free` somente quando o único ponteiro para o

`raw::git_repository` está prestes a ser encerrado, o tipo `Repository` também garante que o ponteiro nunca será utilizado após ser liberado.

O método `Repository::open` utiliza uma função privada chamada `path_to_cstring`, que tem duas definições – uma para sistemas do tipo Unix e outra para Windows:

```
use std::ffi::CString;

#[cfg(unix)]
fn path_to_cstring(path: &Path) -> Result<CString> {
    // O método `as_bytes` só existe em sistemas do tipo Unix
    use std::os::unix::ffi::OsStrExt;
    Ok(CString::new(path.as_os_str().as_bytes())?)
}

#[cfg(windows)]
fn path_to_cstring(path: &Path) -> Result<CString> {
    // Tenta converter em UTF-8. Se isso falhar, libgit2 não pode lidar
    // com o caminho de nenhuma maneira
    match path.to_str() {
        Some(s) => Ok(CString::new(s)?),
        None => {
            let message = format!("Couldn't convert path '{}' to UTF-8",
                                   path.display());
            Err(message.into())
        }
    }
}
```

A interface `libgit2` torna esse código um pouco complicado. Em todas as plataformas, `libgit2` aceita caminhos como strings C terminadas em nulo. No Windows, `libgit2` supõe que essas strings C contêm UTF-8 bem formado, e as converte internamente nos caminhos de 16 bits que o Windows requer. Isso geralmente funciona, mas não é o ideal. O Windows permite nomes de arquivo que não são Unicode bem formados e, portanto, não podem ser representados em UTF-8. Se você tiver tal arquivo, é impossível passar seu nome para `libgit2`.

No Rust, a representação adequada de um caminho de sistema de arquivos é um `std::path::Path`, cuidadosamente projetado para lidar com qualquer caminho que possa aparecer no Windows ou POSIX. Isso significa que existem valores `Path` no Windows que não podem ser passados para `libgit2`, porque eles não são UTF-8 bem formados. Assim, embora o comportamento de `path_to_cstring` esteja abaixo do

ideal, na verdade é o melhor que podemos fazer dada a interface de libgit2.

As duas definições `path_to_cstring` que acabamos de mostrar dependem de conversões no nosso tipo `Error`: o operador `?` tenta essas conversões, e a versão do Windows chama explicitamente `.into()`. Essas conversões não são dignas de nota:

```
impl From<String> for Error {
    fn from(message: String) -> Error {
        Error { code: -1, message, class: 0 }
    }
}
// NulError é o que `CString::new` retorna se uma string tem zero bytes embutidos
impl From<std::ffi::NulError> for Error {
    fn from(e: std::ffi::NulError) -> Error {
        Error { code: -1, message: e.to_string(), class: 0 }
    }
}
```

Em seguida, vamos descobrir como resolver uma referência do Git a um identificador de objeto. Como um identificador de objeto é apenas um valor hash de 20 bytes, é perfeitamente possível expô-lo na API segura:

```
/// O identificador de algum tipo de objeto armazenado no banco de dados
/// de objetos Git: um commit, árvore, blob, tag etc. Isso é um amplo hash
/// do conteúdo do objeto
pub struct Oid {
    pub raw: raw::git_oid
}
```

Vamos adicionar um método a `Repository` para realizar a consulta:

```
use std::mem;
use std::os::raw::c_char;

impl Repository {
    pub fn reference_name_to_id(&self, name: &str) -> Result<Oid> {
        let name = CString::new(name)?;
        unsafe {
            let oid = {
                let mut oid = mem::MaybeUninit::uninit();
                check(raw::git_reference_name_to_id(
                    oid.as_mut_ptr(), self.raw,
                    name.as_ptr() as *const c_char));
                oid.assume_init()
            };
        }
    }
}
```

```

        Ok(Oid { raw: oid })
    }
}

```

Embora `oid` seja mantido não inicializado quando a pesquisa falha, essa função garante que o chamador nunca possa ver o valor não inicializado simplesmente seguindo a linguagem `Result` do Rust: ou o chamador recebe um `Ok` que transporta um valor `Oid` adequadamente inicializado, ou recebe um `Err`.

Em seguida, o módulo precisa de uma maneira de recuperar os commits do repositório. Vamos definir um tipo `Commit` da seguinte forma:

```

use std::marker::PhantomData;

pub struct Commit<'repo> {
    // Isso sempre deve ser um ponteiro para uma estrutura `git_commit` utilizável
    raw: *mut raw::git_commit,
    _marker: PhantomData<&'repo Repository>
}

```

Como mencionado anteriormente, um objeto `git_commit` nunca deve sobreviver ao objeto `git_repository` do qual foi recuperado. Os tempos de vida do Rust permitem que o código capture essa regra com precisão.

O exemplo `RefWithFlag` anterior neste capítulo usou um campo `PhantomData` para dizer ao Rust que trate um tipo como se contivesse uma referência com um determinado tempo de vida, embora o tipo aparentemente não contivesse tal referência. O tipo `Commit` precisa fazer algo semelhante. Nesse caso, o tipo `_marker` do campo é `PhantomData<&'repo Repository>`, indicando que o Rust deve tratar `Commit<'repo>` como se contivesse uma referência com tempo de vida `'repo` a algum `Repository`.

O método para consultar um commit é o seguinte:

```

impl Repository {
    pub fn find_commit(&self, oid: &Oid) -> Result<Commit> {
        let mut commit = ptr::null_mut();
        unsafe {
            check(raw::git_commit_lookup(&mut commit, self.raw, &oid.raw));
        }
        Ok(Commit { raw: commit, _marker: PhantomData })
    }
}

```

```
}
}
```

Como isso relaciona o tempo de vida de `Commit` ao tempo de vida de `Repository`? A assinatura de `find_commit` omite os tempos de vida das referências envolvidas de acordo com as regras descritas em “Omitindo parâmetros de tempo de vida”, na página 151. Se fôssemos especificar os tempos de vida, a assinatura completa seria:

```
fn find_commit<'repo, 'id>(&'repo self, oid: &'id Oid)
-> Result<Commit<'repo>>
```

É exatamente isso que queremos: O Rust trata o `Commit` retornado como se pegasse algo emprestado de `self`, que é o `Repository`.

Quando um `Commit` é dropado, ele deve liberar `raw::git_commit`:

```
impl<'repo> Drop for Commit<'repo> {
    fn drop(&mut self) {
        unsafe {
            raw::git_commit_free(self.raw);
        }
    }
}
```

A partir de um `Commit`, você pode pegar emprestado `Signature` (um nome e endereço de e-mail) e o texto da mensagem de confirmação:

```
impl<'repo> Commit<'repo> {
    pub fn author(&self) -> Signature {
        unsafe {
            Signature {
                raw: raw::git_commit_author(self.raw),
                _marker: PhantomData
            }
        }
    }
    pub fn message(&self) -> Option<&str> {
        unsafe {
            let message = raw::git_commit_message(self.raw);
            char_ptr_to_str(self, message)
        }
    }
}
```

Eis o tipo `Signature`:

```
pub struct Signature<'text> {
    raw: *const raw::git_signature,
```

```

    _marker: PhantomData<&'text str>
}

```

Um objeto `git_signature` sempre pega emprestado o texto de outro lugar; em particular, assinaturas retornadas por `git_commit_author` pegam emprestado o texto de `git_commit`. Assim, nosso tipo `Signature` seguro inclui um `PhantomData<&'text str>` para dizer ao Rust que se comporte como se contivesse um `&str` com um tempo de vida de `'text`. Assim como antes, `Commit::author` conecta adequadamente esse tempo de vida `'text` do `Signature` que ele retorna ao do `Commit` sem que seja necessário escrever nada. O método `Commit::message` faz o mesmo com o `Option<&str>` que contém a mensagem de confirmação.

Um `Signature` inclui métodos para recuperar o nome e o endereço de e-mail do autor:

```

impl<'text> Signature<'text> {
    /// Retorna o nome do autor como `&str`,
    /// ou `None` se não é UTF-8 bem formado.
    pub fn name(&self) -> Option<&str> {
        unsafe {
            char_ptr_to_str(self, (*self.raw).name)
        }
    }

    /// Retorna o e-mail do autor como `&str`,
    /// ou `None` se não é UTF-8 bem formado.
    pub fn email(&self) -> Option<&str> {
        unsafe {
            char_ptr_to_str(self, (*self.raw).email)
        }
    }
}

```

Os métodos anteriores dependem de uma função utilitária privada

`char_ptr_to_str`:

```

/// Tenta pegar emprestada uma `&str` de `ptr`, dado que `ptr` pode ser nulo
/// ou uma referência a UTF-8 malformado. Fornece ao resultado um tempo de vida
/// como se fosse emprestado de `_owner`.
///
/// Segurança: se `ptr` é não nulo, deve apontar para uma string C terminada em
/// nulo que é segura para acessar durante pelo menos o tempo de vida de `_owner`
unsafe fn char_ptr_to_str<T>(_owner: &T, ptr: *const c_char) -> Option<&str> {
    if ptr.is_null() {
        return None;
    } else {

```

```

        CStr::from_ptr(ptr).to_str().ok()
    }
}

```

O valor do parâmetro `_owner` nunca é utilizado, mas seu tempo de vida é. Tornar explícitos os tempos de vida na assinatura dessa função nos dá:

```

fn char_ptr_to_str<'o, T: 'o>(_owner: &'o T, ptr: *const c_char)
-> Option<&'o str>

```

A função `CStr::from_ptr` retorna um `&CStr` cujo tempo de vida é completamente ilimitado, uma vez que foi emprestado de um ponteiro bruto desreferenciado. Tempos de vida ilimitados são quase sempre imprecisos, por isso é bom restringi-los o mais rápido possível. Incluir o parâmetro `_owner` faz com que o Rust atribua o tempo de vida ao tipo do valor de retorno, assim os chamadores podem receber uma referência limitada com mais precisão.

Não está claro a partir da documentação de `libgit2` se os ponteiros de `email` e `author` de um `git_signature` podem ser nulos, apesar de a documentação para `libgit2` ser muito boa. Seus autores vasculharam o código-fonte por algum tempo sem serem capazes de se convencer de uma forma ou outra e finalmente decidiram que é melhor `char_ptr_to_str` estar preparada para ponteiros nulos apenas por precaução. No Rust, essa pergunta é respondida imediatamente pelo tipo: se é `&str`, você pode contar com o fato de que a string estará aí; se é `Option<&str>`, é opcional.

Por fim, fornecemos interfaces seguras para todas as funcionalidades de que precisamos. A nova função `main` em `src/main.rs` é bastante reduzida e se parece com um código Rust real:

```

fn main() {
    let path = std::env::args_os().skip(1).next()
        .expect("usage: git-toy PATH");

    let repo = git::Repository::open(&path)
        .expect("opening repository");

    let commit_oid = repo.reference_name_to_id("HEAD")
        .expect("looking up 'HEAD' reference");

    let commit = repo.find_commit(&commit_oid)
        .expect("looking up commit");
}

```



```
let author = commit.author();
println!("{}", <{}>\n",
    author.name().unwrap_or("(none)",
    author.email().unwrap_or("(none)"));

println!("{}", commit.message().unwrap_or("(none)"));
}
```

Neste capítulo, passamos de interfaces simplistas que não fornecem muitas garantias de segurança a uma API segura envolvendo uma API inerentemente insegura, fazendo com que qualquer violação do contrato dessa última seja um erro de tipo do Rust. O resultado é uma interface que o Rust pode garantir que você utilize corretamente. Na maioria das vezes, as regras que fizemos o Rust impor são os tipos de regras que os programadores C e C++ acabam impondo a si mesmos de qualquer maneira. O que faz o Rust parecer muito mais rigoroso do que C e C++ não é que as regras sejam tão estranhas, mas que essa imposição é mecânica e abrangente.

Conclusão

O Rust não é uma linguagem simples. O objetivo dele é abranger dois mundos muito diferentes. Ele é uma linguagem de programação moderna, segura por design, com conveniências como closures e iteradores, mas visa colocá-lo no controle das capacidades brutas da máquina em que é executado, com sobrecarga mínima em tempo de execução.

Os contornos da linguagem são determinados por esses objetivos. O Rust consegue preencher a maior parte da lacuna com código seguro. Seu verificador de empréstimos (borrow checker) e abstrações de custo zero o colocam o mais próximo possível do metal (nível da máquina), sem arriscar um comportamento indefinido. Quando isso não é suficiente ou quando você deseja aproveitar o código C existente, o código inseguro e a interface de função externa estão disponíveis. Mas, novamente, a linguagem não oferece apenas esses recursos inseguros e deseja boa sorte. O objetivo é sempre utilizar recursos não seguros para criar APIs seguras. Foi o que fizemos com libgit2. É também o que a equipe do

Rust fez com `Box`, `Vec`, as outras coleções, canais e muito mais: a biblioteca padrão está cheia de abstrações seguras, implementadas com algum código inseguro nos bastidores.

Uma linguagem com as ambições do Rust talvez não fosse destinada a ser a mais simples das ferramentas. Mas o Rust é seguro, rápido, concorrente – e eficaz. Use-o para construir sistemas grandes, rápidos, seguros e robustos que aproveitam todo o poder do hardware em que são executados. Use-o para melhorar o software.

Sobre os autores

Jim Blandy trabalha com programação desde 1981 e escreve software livre desde 1990. Ele foi o mantenedor do GNU Emacs e GNU Guile, e mantenedor do GDB, o GNU Debugger. É um dos primeiros projetistas do sistema de controle de versão do Subversion. Jim agora trabalha nas imagens gráficas do Firefox e na renderização para o Mozilla.

Jason Orendorff trabalha em projetos Rust internos no GitHub. Ele trabalhou anteriormente no mecanismo JavaScript SpiderMonkey no Mozilla. Seus interesses são gramática, culinária, viagens no tempo e ajudar as pessoas a aprender temas complicados.

Leonora Tindall é entusiasta do sistema de tipos e engenheira de software que utiliza o Rust, o Elixir e outras linguagens avançadas para criar softwares de sistemas robustos e resilientes em áreas de alto impacto, como assistência médica e de propriedade e controle de dados. Ela trabalha em uma variedade de projetos de código-fonte aberto, desde algoritmos genéticos que evoluem programas em linguagens estranhas até bibliotecas básicas e do ecossistema de crates do Rust, e gosta da experiência de contribuir para projetos diversificados de apoio comunitário. Em seu tempo livre, Leonora constrói dispositivos eletrônicos para síntese de áudio e é uma radioamadora ávida. Sua paixão por hardware também se estende à sua prática de engenharia de software. Ela criou software de aplicativos para rádios LoRa em Rust e Python e utiliza software e hardware do tipo “faça você mesmo” para criar música eletrônica experimental em um sintetizador Eurorack.

Colofão

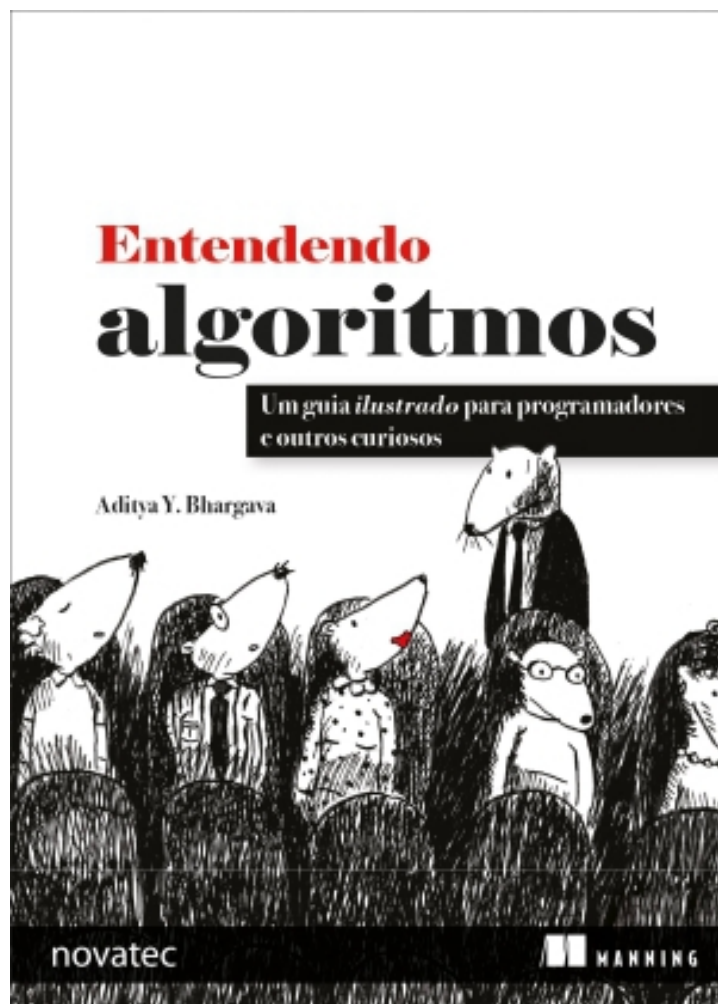
O animal na capa do *Programação em Rust* é um caranguejo de Montagu (*Xantho hydrophilus*), encontrado no nordeste do Oceano Atlântico e no Mar Mediterrâneo. Ele vive sob rochas e pedregulhos durante a maré baixa. Se for exposto ao levantar uma rocha, suas pinças se manterão agressivamente no ar e ele vai abri-las para que pareça maior.

Esse caranguejo tem aparência robusta e musculosa e uma carapaça ampla com cerca de 70 mm de largura. A borda da carapaça é sulcada, e a cor é amarelada ou marrom-avermelhada. Ele tem 10 patas: o tamanho do par dianteiro (os quelípodes) é igual com garras ou pinças de ponta preta; então, há três pares de patas robustas e relativamente curtas para se movimentar; e o último par de patas é para nadar. Eles andam e nadam de lado.

Esse caranguejo é onívoro. Alimentam-se principalmente de algas, caracóis e caranguejos de outras espécies. São ativos principalmente à noite. As fêmeas ovíparas são encontradas de março a julho, e as larvas estão presentes no plâncton durante a maior parte do verão.

Muitos dos animais nas capas da O'Reilly são espécies ameaçadas; todos eles são importantes para o mundo.

A ilustração da capa é de Karen Montgomery, baseada em uma imagem de *Wood's Natural History*.



Entendendo Algoritmos

Bhargava, Aditya Y.

9788575226629

264 páginas

[Compre agora e leia](#)

Um guia ilustrado para programadores e outros curiosos. Um algoritmo nada mais é do que um procedimento passo a

passo para a resolução de um problema. Os algoritmos que você mais utilizará como um programador já foram descobertos, testados e provados. Se você quer entendê-los, mas se recusa a estudar páginas e mais páginas de provas, este é o livro certo. Este guia cativante e completamente ilustrado torna simples aprender como utilizar os principais algoritmos nos seus programas. O livro Entendendo Algoritmos apresenta uma abordagem agradável para esse tópico essencial da ciência da computação. Nele, você aprenderá como aplicar algoritmos comuns nos problemas de programação enfrentados diariamente. Você começará com tarefas básicas como a ordenação e a pesquisa. Com a prática, você enfrentará problemas mais complexos, como a compressão de dados e a inteligência artificial. Cada exemplo é apresentado em detalhes e inclui diagramas e códigos completos em Python. Ao final deste livro, você terá dominado algoritmos amplamente aplicáveis e saberá quando e onde utilizá-los. O que este livro inclui

- A abordagem de algoritmos de pesquisa, ordenação e algoritmos gráficos
- Mais de 400 imagens com descrições detalhadas
- Comparações de desempenho entre algoritmos
- Exemplos de código em Python

Este livro de fácil leitura e repleto de imagens é destinado a programadores autodidatas, engenheiros ou pessoas que gostariam de recordar o assunto.

[Compre agora e leia](#)



Introdução à Linguagem SQL

Nield, Thomas

9788575227466

144 páginas

[Compre agora e leia](#)

Atualmente as empresas estão coletando dados a taxas exponenciais e mesmo assim poucas pessoas sabem como acessá-los de maneira relevante. Se você trabalha em uma empresa ou é profissional de TI, este curto guia prático lhe ensinará como obter e transformar dados com o SQL de maneira significativa. Você dominará rapidamente os aspectos básicos do SQL e aprenderá como criar seus próprios bancos de dados. O autor Thomas Nield fornece exercícios no decorrer de todo o livro para ajudá-lo a praticar em casa suas recém descobertas aptidões no uso do SQL, sem precisar empregar um ambiente de servidor de banco de dados. Além de aprender a usar instruções-chave do SQL para encontrar e manipular seus dados, você descobrirá como projetar e gerenciar eficientemente bancos de dados que atendam às suas necessidades. Também veremos como:

- Explorar bancos de dados relacionais, usando modelos leves e centralizados
- Usar o SQLite e o SQLiteStudio para criar bancos de dados leves em minutos
- Consultar e transformar dados de maneira significativa usando SELECT, WHERE, GROUP BY e ORDER BY
- Associar tabelas para obter uma visualização mais completa dos dados da empresa
- Construir nossas próprias tabelas e bancos de dados centralizados usando princípios de design normalizado
- Gerenciar dados aprendendo como inserir, excluir e atualizar registros

[Compre agora e leia](#)



Estruturas de dados e algoritmos com JavaScript

Groner, Loiane

9788575227282

408 páginas

[Compre agora e leia](#)

Uma estrutura de dados é uma maneira particular de organizar dados em um computador com o intuito de usar os recursos de modo eficaz. As estruturas de dados e os algoritmos são a base de todas as soluções para qualquer problema de programação. Com este livro, você aprenderá a escrever códigos complexos e eficazes usando os recursos mais recentes da ES 2017. O livro Estruturas de dados e algoritmos com JavaScript começa abordando o básico sobre JavaScript e apresenta a ECMAScript 2017, antes de passar gradualmente para as estruturas de dados mais importantes, como arrays, filas, pilhas e listas ligadas. Você adquirirá um conhecimento profundo sobre como as tabelas hash e as estruturas de dados para conjuntos funcionam, assim como de que modo as árvores e os mapas hash podem ser usados para buscar arquivos em um disco rígido ou para representar um banco de dados. Este livro serve como um caminho para você mergulhar mais fundo no JavaScript. Você também terá uma melhor compreensão de como e por que os grafos – uma das estruturas de dados mais complexas que há – são amplamente usados em sistemas de navegação por GPS e em redes sociais. Próximo ao final do livro, você descobrirá como todas as teorias apresentadas podem ser aplicadas para solucionar problemas do mundo real, trabalhando com as próprias redes de computador e com pesquisas no Facebook. Você aprenderá a:

- declarar, inicializar, adicionar e remover itens de arrays, pilhas e filas;
- criar e usar listas ligadas, duplamente ligadas e ligadas circulares;
- armazenar elementos únicos em tabelas hash, dicionários e conjuntos;
-

explorar o uso de árvores binárias e árvores binárias de busca; • ordenar estruturas de dados usando algoritmos como bubble sort, selection sort, insertion sort, merge sort e quick sort; • pesquisar elementos em estruturas de dados usando ordenação sequencial e busca binária

[Compre agora e leia](#)



Fundamentos de HTML5 e CSS3

Silva, Maurício Samy

9788575227084

304 páginas

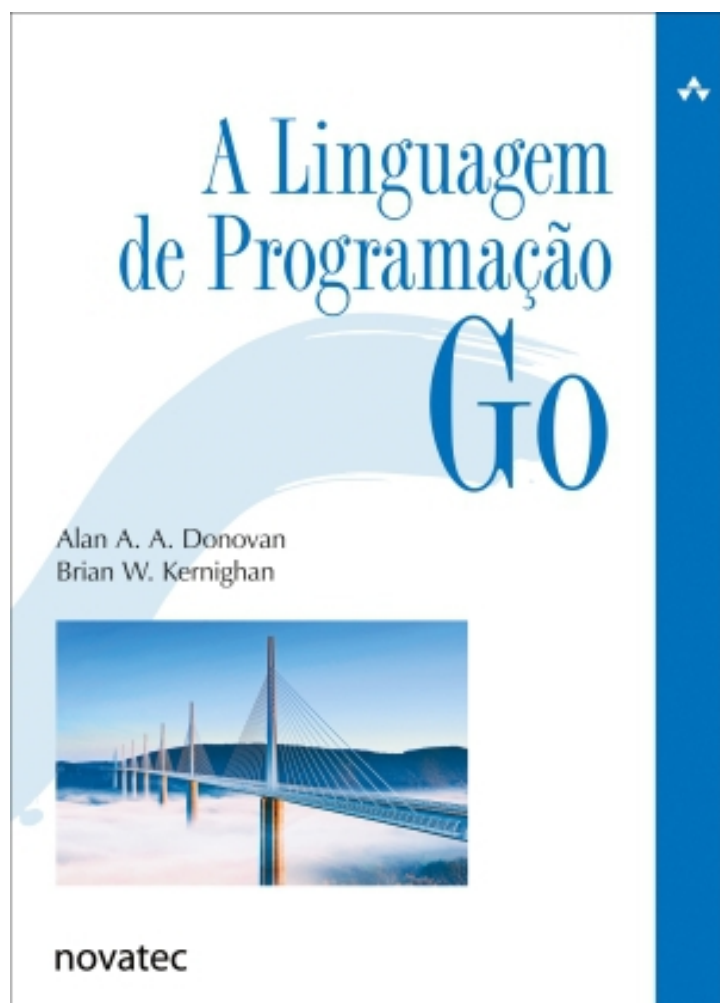
[Compre agora e leia](#)

Fundamentos de HTML5 e CSS3 tem o objetivo de fornecer aos iniciantes e estudantes da área de desenvolvimento web

conceitos básicos e fundamentos da marcação HTML e estilização CSS, para a criação de sites, interfaces gráficas e aplicações para a web. Maujor aborda as funcionalidades da HTML5 e das CSS3 de forma clara, em linguagem didática, mostrando vários exemplos práticos em funcionamento no site do livro. Mesmo sem conhecimento prévio, com este livro o leitor será capaz de:

- Criar um código totalmente semântico empregando os elementos da linguagem HTML5.
- Usar os atributos da linguagem HTML5 para criar elementos gráficos ricos no desenvolvimento de aplicações web.
- Inserir mídia sem dependência de plugins de terceiros ou extensões proprietárias.
- Desenvolver formulários altamente interativos com validação no lado do cliente utilizando atributos criados especialmente para essas finalidades.
- Conhecer os mecanismos de aplicação de estilos, sua sintaxe, suas propriedades básicas, esquemas de posicionamento, valores e unidades CSS3.
- Usar as propriedades avançadas das CSS3 para aplicação de fundos, bordas, sombras, cores e opacidade.
- Desenvolver layouts simples com uso das CSS3.

[Compre agora e leia](#)



A Linguagem de Programação Go

Donovan, Alan A. A.

9788575226551

480 páginas

[Compre agora e leia](#)

A linguagem de programação Go é a fonte mais confiável para qualquer programador que queira conhecer Go. O livro

mostra como escrever código claro e idiomático em Go para resolver problemas do mundo real. Esta obra não pressupõe conhecimentos prévios de Go nem experiência com qualquer linguagem específica, portanto você a achará acessível, independentemente de se sentir mais à vontade com JavaScript, Ruby, Python, Java ou C++.

[Compre agora e leia](#)